

Evolution Simulator

DOCUMENTATION OF NEA

Ethan Cheng 5045
KEGS 16325
H446 2026



Screen Recording
2026-06-08 22:07:52.v

See attached video : “EvoSim Demo”

5 min demo of final program ^

(Note: the venv setup file featured in the video was generated by AI and used to help create a virtual environment to run the project using VS Code editor. AI was NOT used in the devising of the solution except where explicitly mentioned to help with research and idea bounceback, and all program code was written by myself)

TABLE OF CONTENTS

DOCUMENTATION OF NEA	1
Analysis	4
Problem Identification	4
Stakeholders.....	8
Research.....	10
Essential Features	23
Limitations.....	23
Requirements.....	24
Hardware.....	24
Software	25
Success Criteria	25
Design overview.....	27
phase 1~ world.....	28
Overview	28
Design.....	28
Development & Testing	35
Post-Testing.....	43
Review	46
phase 2~ animas	47
Design.....	47
Development & Testing	62
Post-Testing.....	109
Review	112
phase 3~ ui	113
Design.....	113
Development & Testing	117
Post-Testing.....	223
Review	238
phase 4~ data.....	239
Design.....	239
Development.....	240
Testing.....	241
Review	248

EVALUATION	249
Achievement of Success Criteria.....	250
«Success Criteria»	251
Usability Features.....	281
Limitations and Issues	283
Works Cited.....	285
APPENDIX - CODE LISTING	286
angle.py.....	287
anima.py.....	288
biome.py	310
cell.py	311
extratraits.py.....	314
flora.py	316
fruit.py.....	317
main.py	318
nickname.py	323
realm_settings.py.....	325
realm.py	327
simulation.py.....	335
simulationdata.py	336
spatialhash.py	341
toDISPLAY.py.....	343
ui_design.py	345
ui_element.py	353
ui_factfile.py	356
ui_sim_plane.py.....	359
ui_terminal.py.....	362
ui.py.....	363
world_facts.py	366
APPENDIX - NOTES	367

ANALYSIS

Problem Identification

Evolution by natural selection is a long process requiring many generations to observe the effects. This presents an inherent hurdle when trying to explain or visualise the concept, and our explanations often cannot provide a full understanding of just what evolution is and how it works. Simple rules such as variation, competition and inheritance can be easily and suitably modelled by a computer simulation, and presented intuitively to learners in a way a textbook does not.

Thus my project is to create an evolutionary simulator, modelling the process of natural selection in a clear and accessible way. The simulation will allow the user to visualise how populations evolve through the generations, to observe the effects of mutation and survival pressure, and to collect quantitative data on how the population changes. The user will be able to configure the starting position, so they can learn for themselves how their changes to an environment or to the initial genes of a population can change- or not change- how the population develops and ends up.

Justification of Problem

By abstracting real biological processes into a programmable model, users can experiment with scenarios that would be impossible or impractical to observe in reality. Thousands of generations can be compressed into minutes, mutation rates changed instantly, or environments created that do not exist in nature.

From a technical perspective, this problem is suitable for a Non-Examined Assessment (NEA) project because it requires a range of computational techniques and features at sufficient complexity. With the potentially many independent organisms and other objects dispersed throughout the simulation, each acting by its own methods, it is clear that an Object-Oriented approach will be paramount. My project will demonstrate:

- **Object-oriented programming:** OOP is defined as the programming paradigm that organises software design around objects representing real-world entities. The four fundamental pillars of OOP are encapsulation, inheritance, polymorphism and abstraction, and each of these will tie neatly into my design. The key strengths of OOP are the modularity, reusability and ease of maintenance of code that comes with the broken-down structured approach; there is ease of testing and experimenting at each class level, with little interdependence in the program. I can achieve this paradigm using Python's built-in OOP support, and pygame.

Organisms as objects	Organisms in my simulation can be modelled as objects. This is ideal because: - Organisms are real-world entities - Organisms each have their own traits which are unique to individuals, and perform tasks independently. I can use encapsulated attributes and methods to model these; the encapsulation keeps each organism independently working at the same time - Organisms will not be as complex as their real-world counterparts; I can abstract out the key traits and behaviours using tightly defined classes - Organisms will have traits defined at birth, as some combination of their parents', with
----------------------	--

	some degrees of mutation. The non-deterministic algorithm for this can be carried out in an object constructor. The modelling of evolution is a simple case of observing how these traits evolve over time - When organisms die, it is easy to delete the object; and when they are born, it is easy to create a new instance. The number of organisms is undetermined and will vary throughout the program; OOP's flexibility is crucial for this - A population's starting traits and number can be inputted pre-simulation, and passed as parameters to the program; traits become attributes and a loop applies creation n times - There may be more than one species of organism in the simulation; since they will bear resemblance to each other, they can each inherit from a parent class of 'Organism' or some such - Different species may perform similar tasks differently; this of course is supported by polymorphism, where the same identifiers refer to variant methods - By modelling objects on an x-y plane, interaction between objects becomes intuitive and possible
Environmental elements as objects	Environmental elements, such as food and water sources, can be modelled as objects. OOP is suitable here because: - Environmental elements are real-world entities - They are independent of each other; we can use encapsulation to bundle instances with their methods - They will not be as complex as their real-world counterparts; abstraction simplifies and leaves the necessary - They are multiple, and can be instantiated anywhere in the simulation; I can pass x and y values through to an object's attributes to achieve this - I may give the user control of where the elements are and how many; ditto to the previous point

- **Abstraction:** This is defined by the process of removing unnecessary details and creating simple models to represent a more complex reality; the features essential to the requirement are the ones preserved. Abstraction saves time and effort while reducing fluff which might

distract from the point of a program. Evolution of an ecosystem is incredibly complicated and complex, so abstraction will be a key process permeating all areas of design.

Abstraction of organism traits and behaviours	I will identify a small number of traits and behaviours required to build a working model with competition between organisms being a driving factor of evolution; so, speed of an organism is a pertinent trait, but hair curliness is not. Interactions between organisms will be simple, where more complex social behaviours may evolve in real life. Again, the simple principles of evolution do not require great complexity to demonstrate, and in fact complexity can often cloud the effects. Abstraction removes this unnecessary complexity.
Abstraction of the environment	In real life, the environment is complex and ever-changing; intricacies and completeness is not required to give an informative simulation; here, I am not simulating the planet, but an isolated case designed to highlight evolutionary processes. So, a simple terrain modelled as a 2D plane, with environmental elements as objects occupying space in the terrain, will be sufficient
Abstraction of genetic inheritance	The genetic processes behind inheritance are complex and still not fully understood today. However there are simple patterns we can trace to give us not the processes behind the outputs, but the outputs themselves, from given outputs over a large sample. Abstraction means I can use mathematical modelling to reach simply such outputs
Abstraction of physics	The world is modelled on a 2D plane; precise physics are not only unrequired but impossible to model

- **Decomposition:** This is defined as the process of dividing and subdividing a complex problem into smaller subproblems which are each easier to solve. Decomposition simplifies the development process and allows for focus on individual components and their interactions.

Object-Oriented approach	The Object-Oriented approach provides inherent decomposition through the breaking down of the problem through using smaller classes which can be handled individually
Use of contained algorithms	I will program algorithms which are individually testable, dividing the whole program problem into these smaller sets

	with specific functionality that can be conquered by themselves before being recombined
--	---

- **Algorithmic methods:** These methods relate to the design and implementation of algorithms, defined as sets of step-by-step instructions which provide a solution to a specific problem. I will create flowcharts to show the logic of an algorithm before creating the code for it; having the simple framework to design an algorithm around should be beneficial. My program will make heavy use of algorithms, as they are required to simulate many of the features.

Organism behaviour; organisms will decide what they should currently be prioritising; they will roam, eat, drink, sleep, mate, and more.	Algorithms will be used to determine the organisms' state and carry out the tasks pertaining to them. Algorithms are required to simulate the movement of the organisms as they traverse the plane
Reproduction and mutation; when organisms reproduce they will create new organisms with traits inherited as some combination of their parents', modified also by random mutations. This is the core of what the project is attempting to simulate.	This will require a non-deterministic algorithm which will be one of the core focuses of the project
Simulation management; the user will be able to manipulate the simulation, moving to certain points in time, pausing the simulation, etc	Algorithms will be implemented to coordinate the simulation during these user inputs
Landform generation; terrain pools (ie water and ice) will be present in the simulation plane	Algorithms will be used to non-deterministically generate these pools in the plane

- **Data structures,** to store populations, genomes, and generational statistics.

Objects in the simulation are able to interact independently and individually	Animas, floras, genomes etc are objects with their own attributes and methods
The simulation and realm themselves are operable individually and enclosed with all specific info	Realms and simulation are objects with attributes and methods
The program can store and display records from a database including population statistics such as average lifespan and genome	A database will be realised as an object with each record being an attribute which is updated via helper methods
There will be an integrated terminal which displays all 'print' lines to the user through the pygame window interface	This terminal will also be an object with its content being an attribute which is updated with each print line as I will rewrite the sys function of 'print'
Organisms are able to detect elements within their range of vision without requiring iteration through all organisms in the realm	A spatial hash will be implemented which is a data structure which drastically reduces the per-frame operations of each anima object as they will only need to iterate

	through the animas in their own spatial bucket (in a 600x500 realm with 20x20 buckets, space is restricted by 750x)
There are information sets which will be required for various functions and behaviours	Lists, dictionaries and sets will be used to store data pertinent to object methods, e.g. a dictionary mapping flora names to their attributes. These may be constants which I fine tune during development but are not changed at runtime; they act as facts about the world (e.g. a tree produces apples)

- **Mathematical modelling**, including probability (for mutation), optimisation (emergent adaptation), and statistics (to measure population change).
- **Visualisation**, to make the abstract process of evolution observable in real time.
- **Numerical methods**, which are used to get values which represent estimates of values pertaining to certain physical-equivalent phenomena

It should be noted that the project will NOT provide a full, comprehensive or even accurate model of evolution, as it is in real life. Such an undertaking would be far too complex and obviously unfeasible. Instead, my project is merely be a tool by which the basic concepts can be explored, and presented in a friendly manner. So, the scenarios created will not resemble real life ecosystems or biology, but are rather an abstraction of the underlying principles.

Stakeholders

I believe anyone can benefit from a simulation like this, but my focus group of stakeholders will be as follows:

Firstly, the project needs to be suitable for secondary school-aged students taking Biology at KS3, KS4 or KS5 (or the equivalent in their country). These students will be mature enough to understand the concepts and use the program to its fullest, as a strong supplement to their education, either in lessons or at home; the program will be built for a computer, so at school will require for students to be in a computing suite, or watching their teacher present from a monitor. I may later include mobile integration, but since this leans away from the younger demographic, and does not resemble most mobile games, there is no pressure to do so. The simplicity of the design would allow such integration though. Being a slightly more serious project, the computer-based program is the natural home, and will allow me to, for instance, perform data analysis by connecting it directly to Excel. In order to ensure a high level of suitability for this stakeholder group, I will conduct a series of interviews on a few such students, and incorporate their feedback into my design, and while building my solution I will maintain correspondence with them to keep a continuous feedback loop. Students at this level will be aware of mutations, inheritance and variation, and these will be focal to what the project tries to explore and explain.

The strength of the program as an educational tool can only fully be measured with respect to the views of the education-givers, that is, teachers and tutors. This second stakeholder group will be ably represented by my biology and computer science teachers. I will first gather an outline of bullet points from them, then when the project is more complete, they can use the project in some of their classes, and measure how successful it is in conveying evolution compared to classes using only a textbook. I think teachers will appreciate how the simulation can give a practical illustration to theoretical knowledge, and I will consult my biology teacher specifically on what subtopics are within the

evolution topic on our UK A-level specification. The goal for the end product is to allow teachers to have a fun and intuitive program that they can use in their lessons, either by displaying it upon a projector or interactive whiteboard, or bringing students into a computing suite so they can use it themselves. The program is appropriate for this educational purpose because it is informative and provides insight alongside textbook material, as well as being entertaining enough for students not to get bored quickly, which I am sure teachers will appreciate.

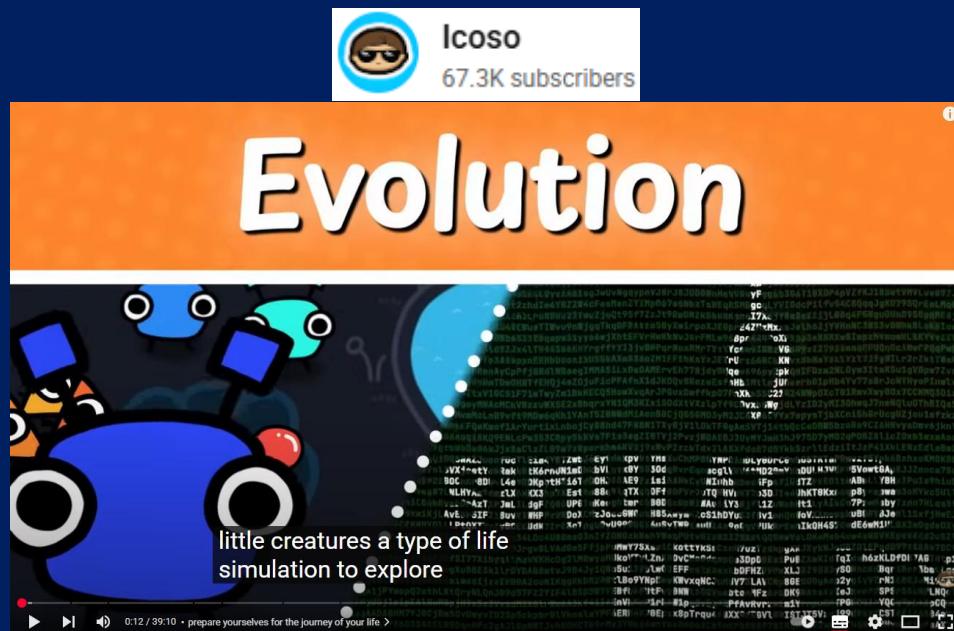
Perhaps ambitiously, I will also target simply all individuals who might find entertainment or interest in the subject matter. Such individuals will mostly likely be people from age 11 and up (though more mature younger children are included also). In light of this broad tertiary group, I will aim to keep the program easily understandable and accessible for those outside a classroom setting, not requiring understanding of language or maths which is too technical so as needing a teacher to explain. The focus is to have a project which, while strong as an educational tool, is also enjoyable for those with no greater knowledge or understanding of biology. These individuals will be able to download the program on their computer and run it, and the simplicity and ease of use will make it appropriate to their needs.

So the primary stakeholders of this project will be biology students and other individuals curious as to how evolution may function, as well as teachers using it as an educational tool.

Research

As part of my research I watched several videos by people who had made similar projects. This allowed me to see how such a project could be started and built up, the scope I should aim for, as well as the various directions I could take the program.

The first such video I watched was one 'I Made an Evolution Simulator (with silly little guys)' by Icoso, on YouTube.



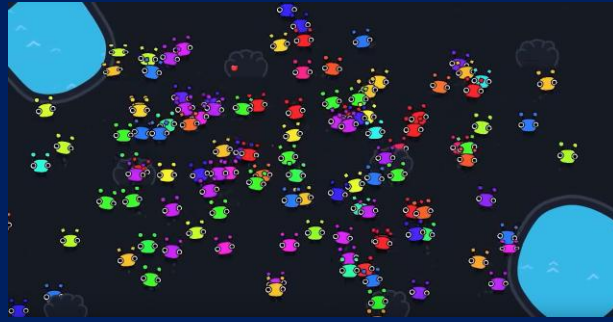
The video was a fast-paced account of how an evolution simulator was made, focusing on the features rather than the programming involved, which was secondary to the video. While the video did not give me much idea of how to make the actual code, I liked the visuals, which were both simplistic and eye-catching, as well as the way the video clearly showed how the various aspects of evolution can be simulated.

```
for each (gooblet)
{
    speedTrait += gooblet.speed;
    sightTrait += gooblet.sight;
    childTrait += gooblet.child;
}

avgSpeedTrait = speedTrait / numberOfGooblets;
avgSightTrait = sightTrait / numberOfGooblets;
avgChildTrait = childTrait / numberOfGooblets;
```

Looking at a brief snippet of code in the video, we can see that Icoso used an object-oriented approach, confirming that this is a good paradigm to use. He also stated that the program was written in Unity, a game engine in C# with strong OOP support. Personally I have no experience with Unity so will stick with Python, but I can appreciate how the game engine would be a strong choice for designing a simulation which visually and in many ways mechanically resembles a game.

I will later try to direct my research towards specifically Python projects, where possible.



I am inclined to take visual inspiration from his simulation; bright colours with high contrast and saturation, clearly defined against the background, will give my project visual clarity and pleasantness. This graphical style is most associated with younger children, however, and not my target stakeholder group, so I will make certain to bring this up as a point of feedback.



Watching the video, I began to consider the philosophical aspects of the exercise. In the subject of metaphysics of god, there is a common idea, that, we humans, simply from the empirical observation of ourselves and the intricate mechanics of this world, can infer a divine creator. Many philosophers of old would even go so far as to say a perfect being was implied, this, God. However, were life to resemble a computer simulation - could the creatures inhabiting it infer that I, the creator, be a perfect being? What can we infer from our experience if we know only the program, and not the mind which shaped it?

Being an evolution simulator, Icoso of course implemented mutations. He outlined quite a detailed process resembling the scrambling of genetic proteins in the real world. While I do not know the exact details of his algorithm, Icoso outlined its effects. I will aim to use a similar algorithm, whereby semi-random variation can be obtained.



Throughout the video, Icoso showed the graphs of characteristics which evolved over time in his simulations, which he obtained by connecting his program directly to an Excel spreadsheet. This

is a feature I will definitely look to include as well as not only would I personally find it interesting, but it would also be invaluable on the project's educational side (so students can spot trends etc) as well as allow for deeper mathematical analysis whereby one could come up with theories for why something has turned out the way it has, and by using the data from many many simulations we can build a more complete model and picture of the evolution my project describes. Possible data analysis could include:

- Graphing the values of the trait variables over time/generation (as Icoso did). A graph of the mean, as well as information about the variance, potential outliers, skewness etc could all be points of interest. Especially if the data is multimodal (which would signify a branching of the species a la sympatry, not allopatric speciation which would have to be simulated through a geographical barrier) Including traits graphed with and against each other
- Graphing the population(s) over time (also as Icoso did). These trends could show typical predator/prey population cycles, trend towards extinction or stability, the effects of overabundance etc
- Creating a geographical map of population frequency (my own idea); I would use a gradient of colours, most likely cold to warm, to represent on average how populated each 'segment' of the environment space was, throughout the simulation; this would show trends and patterns in how groupings of environmental elements (or their lack) would impact the surrounding population
- Various other variables to be graphed, for instance biodiversity, which will tell us whether species converge or diverge over time, and how quickly

I would run the simulation many times, possibly hundreds or thousands, to get a deep look at the trends and see how much variation and chance come into play.



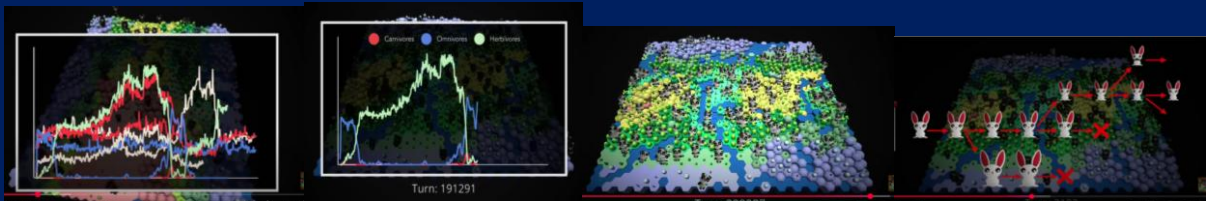
I noted the gradual process by which the program was built up. Icoso tended to add one or two new traits at a time, or developed a single new feature. This perfectly resembles an iterative approach where features are added slowly as the program develops, each iteration working as intended within itself, but as an improvement on the last. I will aim to produce the same effect on my work and ensure I do not attempt to add too much at once, or I risk scope creep and buggy code when the pieces do not fit together. This should also make development faster and more focused.

I will aim to improve on Icoso's simulation by better creating the conditions for different species to evolve and selection to occur on a more varied scale; in Icoso's simulation, traits always drifted towards a stable mean and there was a lack of drive to adapt. This is largely due to Icoso's stagnant environment, where in real life a constantly changing environment leads to constant change and adaptation. Therefore I will simulate change in my environment which will create the selective pressure necessary for constant change, as well as attempting to derive better conditions for speciation, which plays an important role in the science.

The second video i watched was xxx



I then stumbled upon 'EightLittleBears', a channel dedicated to an evolution simulator the creator had been making (the channel has been dormant for the last 11 months, but still has 12 videos which I found very helpful and interesting and went through one by one).



From the first video, I liked the creator's mention of his interest in potentially using his project as a foray into real-life ecosystems and simulation, which resonated with the aim of mine. He produced graphs in his videos showing the data of his species which was split into the discrete types of 'Omnivore', 'Herbivore', 'Carnivore'; I may borrow this- having watched videos where the projects featured organisms with predominantly continuous traits, I noted this creator's use of the discrete. [NOTE: it was later shown that these discrete types were actually just approximations of traits which were in fact continuous, eg herbivores had a value of $x < k$, where x was the diet preference value and k was some boundary constant; however the breaking up into discrete types is still something I could use] The creator's simulation was eye-catching, although quite cluttered and it was difficult to observe and understand what was happening, due to the bears all looking the same and moving over and around each other with no indication of what they were doing. In my project, I may use similar visuals, but try to retain more simplicity, intuitiveness and ease of comprehension in what was happening at both a macro and micro level. Indicators of state will help to achieve this; while I understand graphics are not a focus of the NEA specification, they will certainly be crucial to my goal here.

Between all the videos I have seen so far, all of them have featured environments much more complex than the one I was planning on. This has made me feel that if I want interesting insights and a more analogous simulation of evolution, I also might need to up the complexity of my environment, through introducing variant terrains, elevation, or some other features. This simulation, in particular, featured a complex terrain system modelled after the Whittaker biome system, with untraversable water cells, and biomes which each had different characteristics which would drive adaptation and evolution. Environments at this level of complexity, being more analogous to real life, might generate more realistic data. The environment was also hexagonally grid-based; where in Icoso's simulation and in the one I was planning the plane was continuous and traversed in minute degrees, this creator had opted for individual cells to be limiting, compact units with attached information of the resources available upon it, and traversed in discrete turns, where organisms would require three turns to cross from one cell to another adjacent, and other amounts

of turns were required for each action an organism could take. This is a vastly different approach with some advantages and drawbacks, which I shall consider as I decide whether to adopt it.

The chief advantages I can think of are simplicity, ease of movement logic, and compact information where cell data can be easily stored in a single data structure with rigidity; also ease of determining what happens at each cell, and the 'decisions' of organisms (in the video the creator used the simple logic of a creature consuming all the necessary and suitable resources at its cell before moving on). Additionally, complex behaviours can be more simply modelled since all interactions occur only between things on the same cell, and are often hidden, while in Icoso's paradigm everything can be interact with everything in its radius, leading to some scaling issues.

The chief disadvantages are the sense of restriction on the organisms, reduction in fluidity, and requirement of using a hexagonal grid, as a square-based one would be too simple. There is also the effect of continuous movement being easier to track; it simply feels more natural and purposeful, like the wanderings of an organism as we see in the real world, as opposed to the methodical movements on a grid. I would also have to somewhat abandon the idea of organism speed, at least as a continuous variable trait, and this would be one of the most intuitive to implement, in my eyes. The creator did implement speed in the video; however it was very difficult to visualise due to the discrete nature, whereas velocity of an object as it traverses a plane continuously as far more ergonomic to the user in this role.

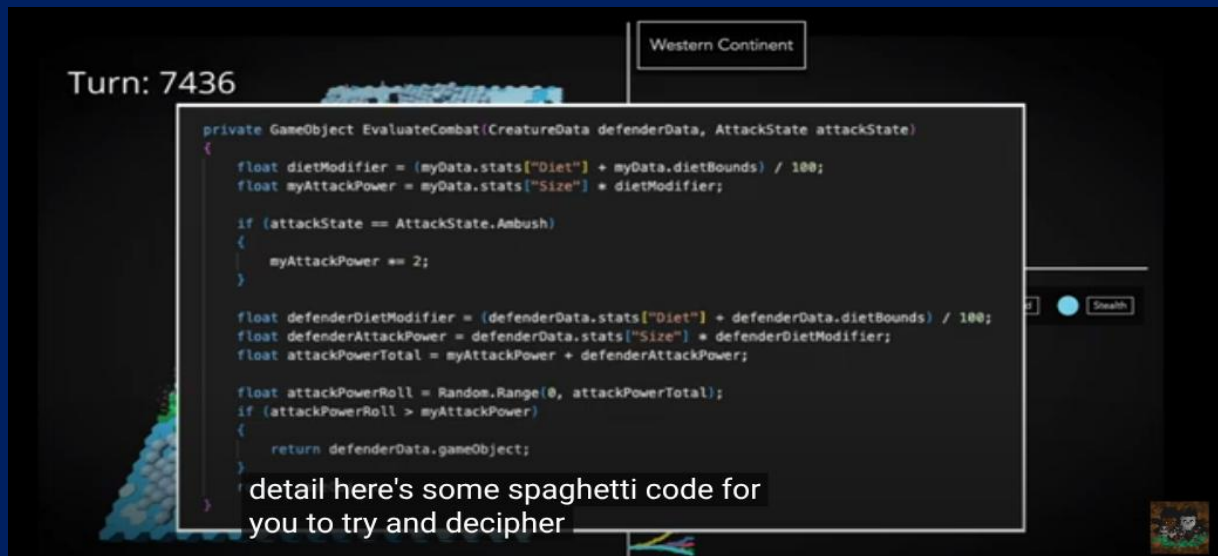
```
private bool CheckForMutation(GameObject newCreature, CreatureData newData)
{
    int roll = Random.Range(1, 101);

    if (roll <= mutationChance)
    {
        int amount = Random.Range(-maxMutationAmount, maxMutationAmount + 1);
        MutateStats(newCreature, amount, newData);
        return true;
    }
    return false;
}

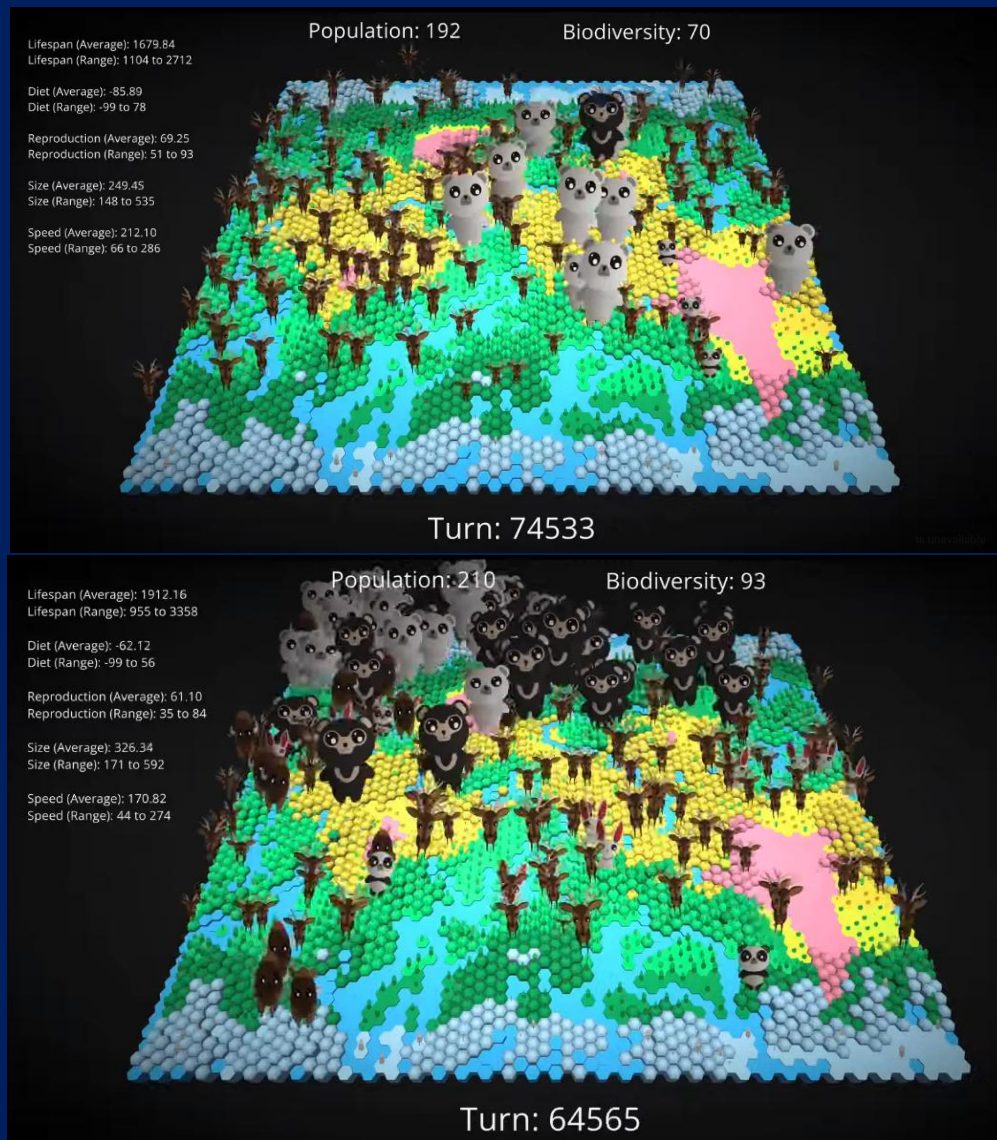
private void MutateStats(GameObject newCreature, int mutationAmountPerCent, CreatureData newData)
{
    int index = Random.Range(newData.firstMutableIndex, creatureData.stats.Count);
    KeyValuePair<string, float> pair = creatureData.stats.ElementAt(index);

    newData.UpdateStat(pair.Key, creatureData.stats[pair.Key], mutationAmountPerCent);
    newData.LogMutation(pair.Key, mutationAmountPerCent, newData.stats[pair.Key]);
}
```

This creator's implementation of mutation was somewhat simple; each birthed organism had a one-third chance to mutate a trait value upwards or downwards by up to 30%. Here I think having some more dynamic implementation would be more interesting, such as the one provided by Icoso. However, for simplicity's sake, I will probably adopt EightLittleBear's version. It fulfils near the same functionality and is easier to both manage and follow in the simulation.

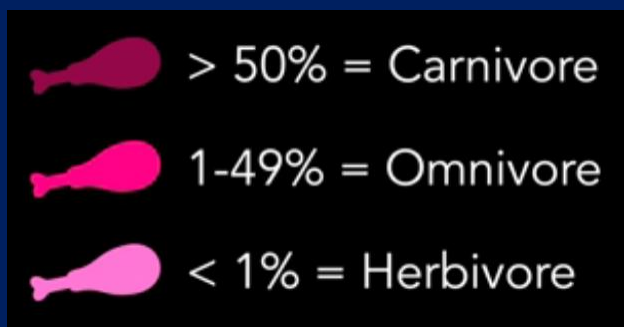


This code provides a concrete example for the point I made earlier about the grid model's greater suitability for complex interactions. Trying to implement similar logic using Icoso's paradigm would be far more difficult, especially as the size of the population grows. However, I have to say once again that Icoso's model would allow for a greater visualisation of the event. In fact, in the case of an ambush, it is not difficult to imagine behaviour which would allow for it.



Name	Diet	Perception	Reproduction	Size	Speed	Stealth
Farm Pig	Omnivore	10	40	210	10	10
Giant Panda	Herbivore	20	280	170	30	30
Sumatran Striped Rabbit	Herbivore	30	20	20	110	180
Cottontail Rabbit	Herbivore	40	30	10	120	190
Arctic Hare	Herbivore	50	10	30	200	200
Red River Hog	Omnivore	60	60	190	20	20
European Bear	Omnivore	70	50	150	80	140
Warthog	Omnivore	80	110	130	150	120
Pygmy Hog	Omnivore	90	70	40	60	170
Giant Eland	Herbivore	100	170	1200	210	40
Red Deer	Herbivore	110	230	240	140	130
Springbok	Herbivore	120	140	70	270	150
Gazelle	Herbivore	130	130	60	260	160
Tiger	Carnivore	140	210	250	160	250
Lion	Carnivore	150	200	220	250	210
Jaguar	Carnivore	160	190	180	220	240
Cheetah	Carnivore	170	80	90	280	220
Puma	Carnivore	180	100	120	240	230
Leopard	Carnivore	190	120	100	180	280
Snow Leopard	Carnivore	200	150	80	190	270
Lynx	Carnivore	210	90	50	230	260
Sloth Bear	Omnivore	220	250	160	40	90
Sun Bear	Omnivore	230	240	110	100	110
Spectacled Bear	Omnivore	240	180	140	90	100
Asiatic Black Bear	Omnivore	250	160	200	70	80
American Black Bear	Omnivore	260	220	230	170	70
Polar Bear	Carnivore	270	270	270	50	50
Brown Bear	Omnivore	280	260	260	130	60

Taken from his second video, we can see from these screenshots that EightLittleBears improved on the visualisation aspect by designing unique avatars representing different 'species'. As I understand, these species are not actually distinct in the strict biological sense; all organisms in his simulation can mate with each other. However they are helpful visual markers where organisms of the same species have similar stats; for each creature born an algorithm runs through a csv file containing records of each template species, and finds the one with stats most resembling the birthed creature. So the species is simply an approximation of the stats, represented visually with an avatar, and has no additional bearing. This approach would be interesting for me to adopt though complicated; on the one hand, Icoso's method of having two totally distinct species [which vary in stats throughout their population but maintain within their roles] is simple and effectively shows competition, selective pressure, and a predator-prey relationship. On the other hand, EightLittleBears' approach allows for more dynamic and interesting results; the most key difference is that animals' diets can mutate, so a herbivore can gradually become more omnivorous if the environment supports it, and likewise a carnivore can have children leaning more into plants for their food. This avoids the problem seen in many simulations including Icoso's where overpredation leads to the extinction of the prey creatures, followed shortly by the predator's own doom. In many cases it is difficult (though possible) to create a long-term stable ecosystem using the Icoso paradigm, avoiding any extinctions, while still allowing enough flexibility for the different traits to have time to evolve. EightLittleBears avoids this problem by having all animals be homologous to some extent; predators can arise naturally from prey; prey from predators. It creates balance and fluidity and creates added complexity without causing visual confusion- we can easily tell how a creature will behave from its avatar approximation. We can recall that complexity is good as long as well manageable within the scope, well resembling what the project is actually simulating, and causing little detriment to the user understanding of the simulation, because it allows for results which can tell us more about the world of the model; increased complexity means increased relevance to the complex real world, more possibilities in the results also gives us more information when any specific case arises.

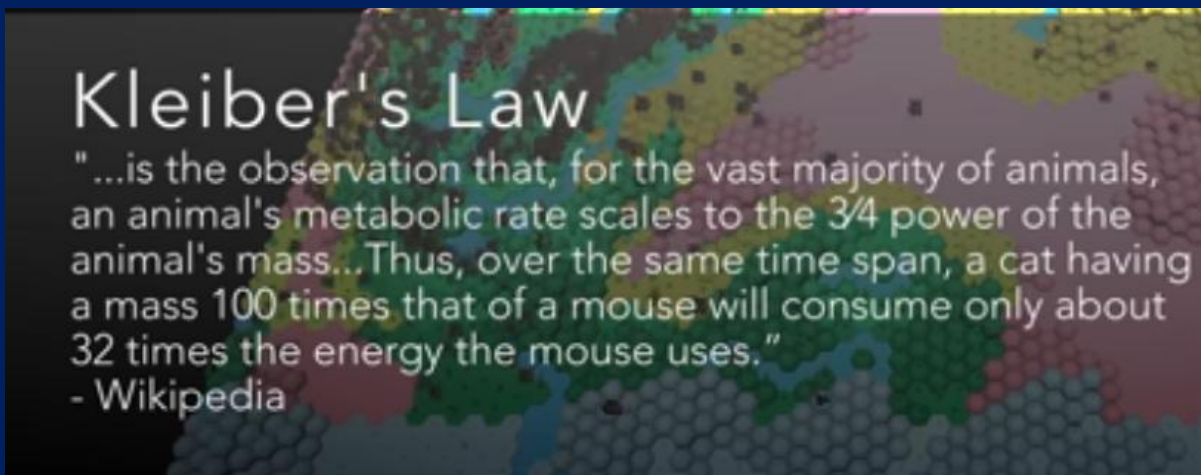


Here is the specific metric EightLittleBears used to determine diet type. Difference in diet is probably the greatest differentiator between any two creatures in this simulation, so EightLittleBears perceptively created three distinct pools of species, one for each diet class, so that a herbivorous species will never represent a carnivore, and likewise for any other incongruent combinations. Within, for example, the panda species, there will be some pandas faster than others; there is no set upper or lower limit for speed; but there will never be a carnivorous panda; the moment a creature is born with diet preference $\geq 1\%$ it cannot be a panda (though diet preference can still vary within the panda species from 0-1%). This is a salient decision and should I take a similar approach I will certainly adopt the like choice.

Tile Stats			Creature Values		
Plant Type	Available		Rating	Roll	Obtained
Algae	10		0.1	0.2	0
Fruit	20		0.2	0.3	0
Grass	30		0.5	0.2	15
Leaves	60		0.3	0.1	20
Meat	10		0	0.1	0
Nectar	30		0	0.2	0
Pollen	15		0	0.7	0
Sap	5		0		
Seeds	5		0.1		
Wood	25		0		

Dividing plants into many smaller subcategories adds complexity, but with not too much benefit to what I am trying to accomplish. It is hard to track this many preferences and the resulting evolution, while interesting, is not worth the time and complexity investment, and even within the simulation may dilute the pressure pool from what I want to focus on, which is the other, more visualisable traits.

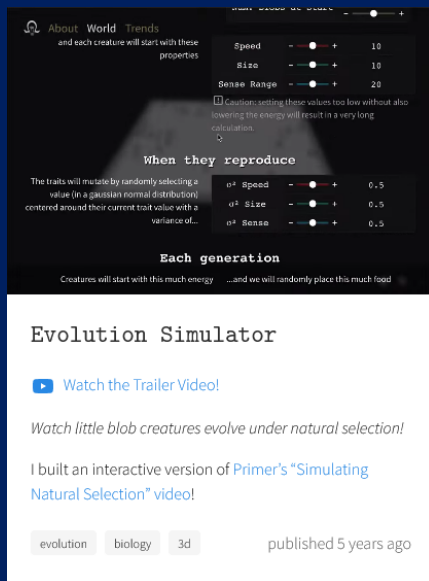
Alternatively, if I use Icoso's paradigm, I can create these algae, fruit, grass etc as their own sprites on the map to be eaten; this would create a highly visualisable level of complexity, but would also require a biome system to make it make more sense, otherwise the best things to prefer would assumedly simply be what is in most abundance, which is uniform across the map. Even with a biome system, I am not sure what insights are to be gained from this, but I may yet give it a try.



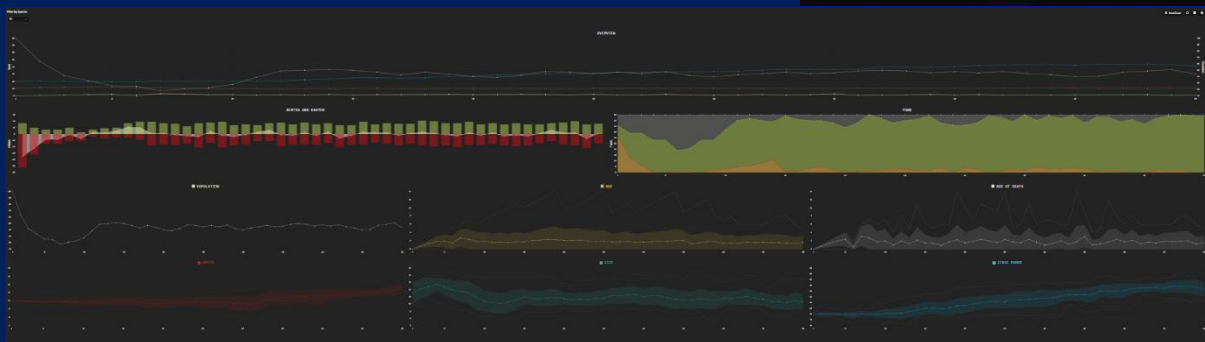
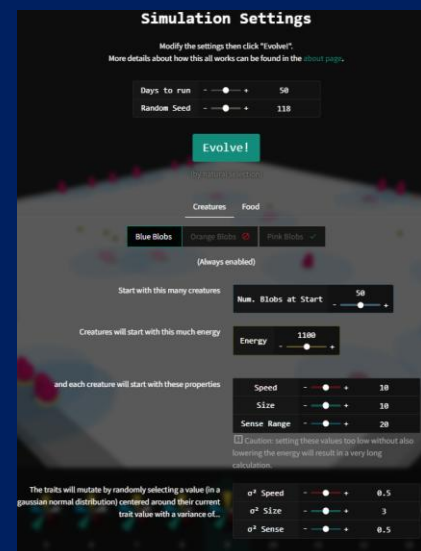
EightLittleBears uses Kleiber's Law in his implementation of energy. Rules such as these are good to be aware of when making mathematical models because following them will mean more lifelike results in generation, which is definitely a plus in any project aiming to give us insights on the simulation topic. Of course trying to integrate every mathematical rule I come across would create needless complexity, but I think adopting simple rules like Kleiber's Law is a good heuristic. I do not feel bound to such models however; I will always adopt the model that best suits my program.

In my design I may incorporate general mathematical rules which will aid my modelling.

As part of my research I found an evolution simulator made by MinuteLabs, inspired by the Primer series which I was also inspired by. I found the interface to be excellent, with intuitive icons toggling

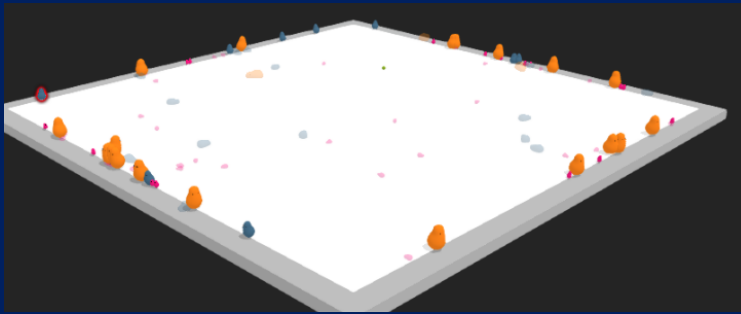
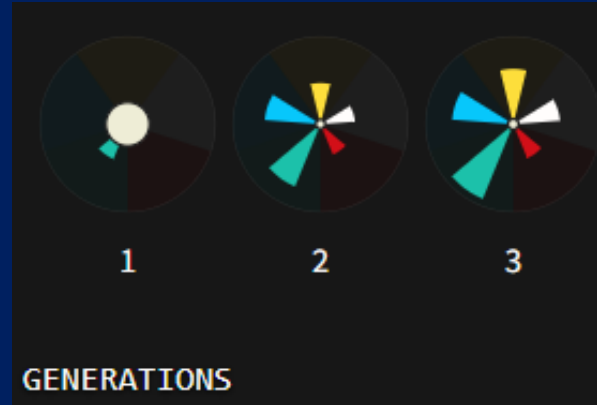


different actions and setting screens, and a clean mostly monochrome design which feels perfect for mature students; I feel inspired to implement a similar UI. The simulation takes place on a board which allows for camera manipulation and rotation using the mouse, which feels nice under the fingers. The organisms are 'blobs' which are simple and aesthetically pleasing. Toggling a trait display setting shows the various traits of each blob as corresponding loci and shading, which is nice since it allows you to choose whether you want a more in-depth look while watching the simulation. There are speed settings so you can control how fast everything happens.



This is the project's trends tab which showed how population and species traits changed over time. I found the tab very well put-together, though the colours were quite dim and visually unappealing. Analysis-wise, this tab is crucial. I will create something similar, though I may perform and create more complex data analysis and graphs, as I mentioned with my research on EightLittleBears. The formatting is strong and it is easy to read and understand the graphs.

The way the simulation is traversed is likely going to be very different from how mine will be. MinuteLabs generates all generations of the simulation very quickly as soon as the start button is pressed; the simulation then plays through each generation chronologically, but with the choice for the user to jump to any generation they wish to view, going forwards or backwards as they please. This has the advantage of flexibility and usability; the user gets total control. However, for my project I will likely run the simulation in real time as the user is viewing it. This is for ease of programming, as I can better coordinate the graphical and mathematical. I thus lose the mentioned advantage, but this is a necessary loss for the safety of scope.



There were some elements which I thought could have been improved upon. For instance, all the internal mechanics and mathematics were under the hood, and adjusting the trait variables felt slightly arbitrary not having any further information beyond the names of the traits. For an example, simply including in

parentheses a formula describing the relationship between size and energy would have made it much easier to adjust things how I wanted; as it was it took me several rounds of trial and error to find reasonable combinations of traits for an interesting simulation, though this is not too severe a limitation in itself; it simply shows a lack of information display, which for an educational project I would require greater focus on. The environment was very simple, with food randomly spawning around the map, and blobs eating either food or other smaller blobs. On the one hand, the simplicity means we can zoom in on how each iteration fits together; there are not too many different things to try or focus on, which might be exhausting, but there is still room for experimentation in the settings. However due to how one-dimensional everything is, there is not a complete sense that I am watching evolution take place in true form; in most cases the blobs barely changed trait values, even with manually increased variation, or they tended to drift towards the global best values, which depended on an individual energy level which somewhat jarringly could not change or evolve. Here, the only advantage of increased size is the ability to eat smaller blobs, which in my opinion is too big an abstraction; in a lot of cases where there are two species with a vast size gap, the exact size values are under no pressure, because a slightly less small blob has strictly no advantage over a small blob. I think the main reason for uninteresting convergences is the lack of environment diversity. The board is one-note and unchanging, and the only elements to be played with are the settings of the other blobs; the simulation does allow up to 3 species which allows for dynamic relationships; however there are still at least a few issues I identified with the system which prevented me from getting full use from the simulation which I think has much potential.

Some major limitations of MinuteSims' project:

- Lack of clarity in blob decision making, including how they prioritised chasing blobs or finding stationary food (there was a mode where you could view their current status, but this was limited to one blob at the time which you had to be impractically zoomed into and following, though this was cute aesthetically; still the internal logic is hidden)

- Global omnivorousness meaning that there was little predator/prey feedback, so predation was not effective as a selection pressure; predator/prey relations not similar to those in the real world
- The uncustomisable environment
- Incompleteness of simulation mechanics information

I can improve on these limitations by adding customisation and elements of interest to the environment itself, as well as making diet a trait which can vary between individuals or species. I will ensure more complete information of simulation mechanics is presented to the user in a dedicated tutorial section as well as positioning such information in sensible areas of the interface. I will use animation to provide visualisation for internal logic of objects.

The creators used Java, which uses static types as opposed to Python's dynamic approach. While Java is known for its strict adherence to OOP principles and ease of scalability in its projects, I am much more well-versed in Python and so will proceed with the language I am most comfortable with.

In the final stage of my research, I worked with project stakeholders to form a better idea of an ideal solution.

I first interviewed A-level and GCSE Biology students; the interviews were as follows:

Subject #1 : Sabeshan Shayan // A-level

Q: Do you find anything lacking in the way evolution is presented in the school curriculum?

A: They don't really address the scale of evolution, like, like, like, like, y'know you like this is millions of years innit. Y'know homo erectus they should talk more about homo erectus, homo erebus, homo habilis, but there were lots of homo unaccounted for like homo erectus, ebilus, neanderthals...

Q: Do you think evolution benefits from showing rather than telling as a form of education?

A: How the f*** do you show evolution? How do you show it? Because evolution takes place over millions of years. So you can't really show it.

Q: What if we could use computers to simulate evolution, such that they could see the processes for themselves, rather than simply being told abstract definitions and concepts?

A: It would be good for them, yeah, it would make learning about it easier.

Q: Rank the following for priority in such a simulation: aesthetics/complexity/intuitiveness/customisability

A: 1. Intuitiveness 2. Complexity 3. Customisability 4. Aesthetics

Q: What traits do you think it would make sense to include? And how many for sufficient complexity?

A: Size, as in you know like muscular build, I think appearance, takes into account like eye colour or facial structure. Speed.

Q: What behaviours do you think it would make sense to include?

A: I think mating is quite important; I think widespread disasters like asteroid or disease, and also low scale ones, like a variable that represents wide scale ones, and one that represents low scale ones. And you need to represent selection pressures that cause them to adapt. And also you should probably trace their geographical position, like what area they're moving.

Q: What do you think should be included in the data analysis section?

A: That's interesting. Things like average x like variables like average speed, you know the traits, graph that data, how population changes, and how they adapt, what else... umm I don't even know what else you can graph umm ahh uh uh maybe uh on where you have these specific changes or where these changes happen in the environment you can have new disaster or new predator or new disease.

Q: Well, thanks for your responses! On a final note- what else would you like to see included in the program.

A: Maybe a slider so you can go to different points specifically in time, of the animal. Maybe for your graphs you can have lines of best fit or general trends. You could have automatic data analysis of which factors affected the population more. Like whether disease or predators affected the population more. You could maybe have a small biography on each of the different species. That's pretty much it.

Subject #2: Babu Srivasthan // A-level

Q: Do you find anything lacking in the way evolution is presented in the school curriculum?

A: Yeah, I think they don't talk enough about how animals kinda change slowly over a really long time. Like, it's not like they turn into something new overnight, but they don't really say that. Also, they should explain more about how monkeys became humans because that's, like, the coolest thing I've witnessed, but they don't even teach it. It's all a bit, you know, missing the good bits.

Q: Do you think evolution benefits from showing rather than telling as a form of education?

A: Yeah, totally. If they showed us evolution, it'd be like watching a giant animal dance party or a fish trying to do the moonwalk on land. Maybe throw in some dinosaurs doing the floss or monkeys wearing sunglasses, just to make it extra crazy. That way, biology wouldn't be so boring and dull—it'd be a wild animal circus or something! Plus, I'd remember that forever, like, "Wow, fish with legs doing the moonwalk—that's epic!" Showing stuff would be way more fun and a lot less snooze-inducing.

Q: What would you say are the key fundamentals of evolution?

A: It's kind of boring.

Q: What are the most interesting features of evolution?

A: Monkeys.

Analysis of the interview data:

A data analysis section complete with tracking for variables like speed and trait values seems like a well-liked idea. A time slider would also be a good usability feature. I need to make sure the visuals are strong and simple; this is the core strength of the product. Possibly world events could be used to shape or drive evolution by providing different and spontaneous conditions for selection. Subject #1 gave the idea of a species biography which would be an informative and cute

idea. Intuitiveness of use should be a priority as the stakeholders may not have the time, ability or patience to investigate how a complex system can be used.

The responses given by the students will be key in shaping the design of this project, and I will ensure to get continual feedback moving forwards with each iteration.

Essential Features

The essential features of the proposed computational solution are as follows:

1. Simulation of organisms and environment interacting and changing over time *to operate under mathematical and logical models*
2. User interface which will allow users to set up and interact with simulation
3. Data analysis section of the UI which displays graphs of trends and points of interest

These features are fundamental to my solution to the problem identified. The project is a tool for interest and education, on the subject of evolution and the evolutionary behaviours. Suitable realism is achieved through mathematical/logical models pertaining to the natural source; of course the method of learning best promoted by a computational approach is that of visualisation, hence [1].

To maximise usability, the solution requires an interface [2] which allows users to input a starting situation which the simulation will develop from. The more variance and customisation options here the more use we can get from the simulation. Once the simulation begins, certain functions such as pause/play, speed control, camera control, and information presentation, etc, will streamline the user experience and should not be too difficult to implement.

Even though graphical communication is a priority, a dedicated data analysis section [3] will provide detailed information to the user beyond what they may simply observe from the simulation. The trends observed can help users to formulate evolutionary hypotheses, which propels the educational purpose of the solution. Viewing the graphs allows users to get both general overviews and a more in-depth picture of the story, depending on the time they put in to connect the dots. Users can expect trends of population, age and traits, to be filtered by species to give a database-style approach.

Limitations

There are some certain limitations to the solution to be observed:

1. Limitation in scope
2. Limitation in accuracy
3. Limitation in computation and storage

This being a small-scale project by one person, there is an immediate limitation in scope [1]. The program must be basic, without many additional features; while this does not contradict with the highly abstracted nature of an ideal educational tool for evolution (as an all-detailed complex simulation would be little better at demonstrating the principles than this one abstracted), it does mean I may be forced not to include some features I otherwise would, or build the structure for a greater program that could conceive of more and greater cases to simulate.

Of course, the real-world mechanics of biology and evolution will not be accurately represented [2]; the program should not be used as a substitute for formal or alternate education, but as a supplement by which certain principles may be illustrated. Users should not rely on the program to provide an accurate depiction of evolution in any fine detail; there is a limitation in accuracy due to the scope [1*] as the real world is too fine and with too many factors non-extraneous to evolutionary development to create the accurate ideal (this is no severe limitation due to the lack of benefit in too great an adherence to an accurate model as it becomes inverse to simplicity which is paramount to the teaching tool).

The components of hardware and software impose a limitation on how much memory or processing power can be dedicated to the program [3]. Overloading the number of objects in the simulation or running it for too long may exhaust the computational or storage capabilities at the user end. This is of course proportional to the individual computer's specification. To address this limitation, I will impose a limit on the number of objects that can be present within the simulation at once, though this will sacrifice consistency and accuracy [2]. I will likewise limit the maximum duration of a simulation run.

Requirements

The computational solution is required to achieve the following:

- Mathematically simulates how organisms can evolve under selection pressure
- Provides a clear and intuitive visualisation of the mathematics
- Is compatible with a school environment
- Has highly interactive and customisable setup *so users can test many different scenarios*
- Enhances user understanding of evolution
- Presents well-organised and informative data regarding the simulation

These requirements all relate directly to the problem of students and non-students lacking a simulator for evolution which would allow them to better understand its workings as well as have fun exploring.

There are hardware and software requirements on the user end:

Hardware

Requirement	Justification
Xx RAM	Memory must be sufficient to run software
Xx Secondary storage	Memory must be sufficient to store program and data
MHz Processor	Processing power should be sufficient to run software at an acceptable speed
Monitor	To provide the basic user interface
Keyboard	This is a basic input device which will be needed to interact with the program
Mouse or Trackpad	Ditto as above
Sound output device	There may be sound elements such as organism death sfx
iPhone iOS xx+ (hardware)	Encompasses the necessary features; mobile users can disregard the other non-storage requirements

*Blue highlights indicate the possible alternatives for a mobile integration

**Yellow highlights indicate hardware which is optional and extraneous to the basic features

Software

Requirement	Justification
Suitable OS	Operating system must be compatible with the software and capable of properly running it
iPhone iOS xx+ (software)	Encompasses the necessary features; mobile users can disregard the other non-storage requirements

*Blue highlights indicate the possible alternatives for a mobile integration

**Yellow highlights indicate software which is optional and extraneous to the basic features

Success Criteria

The computational solution will be successful if it fulfils the following criteria, composed of the requirements as outlined in the previous section:

- Mathematically simulates how organisms can evolve under selection pressure
 - A population of organisms has traits which change each generation
 - Organisms inherit traits as some combination of their parents' traits and random mutation
 - Different combinations of traits in an individual affect its tendency to survive and to reproduce
 - Traits correlating with tendency to survive and to reproduce are likely to be better represented in each passing generation, as opposed to less represented
- Provides a clear and intuitive visualisation of the mathematics
 - There is a stakeholder consensus that the visual logic is sound and graphics appropriate
 - Organisms with differing traits have corresponding* differences in their visual representation (*these may not be 1-to-1, or actual in some cases)
- Is compatible with a school environment
 - The software can be run on KEGS school monitors
 - There is a stakeholder consensus that the program remains within the bounds of an appropriate educational school project
- Has highly interactive and customisable setup *so users can test many different scenarios*
 - The starting population, initial traits of organisms, and initial environment layout are determined by user inputs
 - Users can modify organism appearance to create their own 'species' for ease of visual following and differentiation
 - There are at least three distinct trait combinations which populations can converge to, depending on the starting position
 - There is at least one situation in which all organisms become extinct
 - The stable range of population differs depending on the environment setup
 - The environment can be altered in such a way that 'speciation occurs', where two populations converge differently in a single simulation
- Enhances user understanding of evolution
 - The program is shown to be conducive to greater understanding of evolutionary mechanics
 - The program includes written information explaining evolutionary concepts
- Presents well-organised and informative data regarding the simulation
 - The program features a section where data and trends can be viewed

- Each trait and population variable is featured in this section; the average value is graphed alongside individual values over time, with key values such as the maximum and minimum, and variance, made clear
- Data on a simulation can be downloaded onto the user's computer, where it may be stored as either a screen capture, flat file, or text document detailing key values

Bullet points detail indicators which show if a criterion has been hit.

A project which has clearly achieved all criteria is a successful one. I aim to create a program where each of these features can be identified.

DESIGN OVERVIEW

It is important for me to be able to distinguish between what is within the project's scope, and what is ultimately unfeasible under the constraints I have. So in my design section, I will try to detail as many potential features and ideas as I can, while being cognisant that many will not end up in the final product. I will weigh the importance of each of these features with the feedback from the stakeholders to ensure I include what is truly best and most essential.

I am programming the project in Visual Studio Code with Python, using the pygame extension.

Here is what I will work on during each phase of development:
Phase 1 – Create the environment, coding for randomisation of water and plant spawns, and developing the logic for plant behaviour

Phase 2 – Create the animals; their logic and behaviour, with rudimentary sprites depicting them in simulation

Phase 3 – Create the user interface, including a menu, selection screen for simulation set up, and more informative sprites for animals and plants

Phase 4 – Create the data tab whereby users can examine the trends and patterns generated in simulation; the data should be downloadable as an Excel file to their computer

PHASE 1~ WORLD

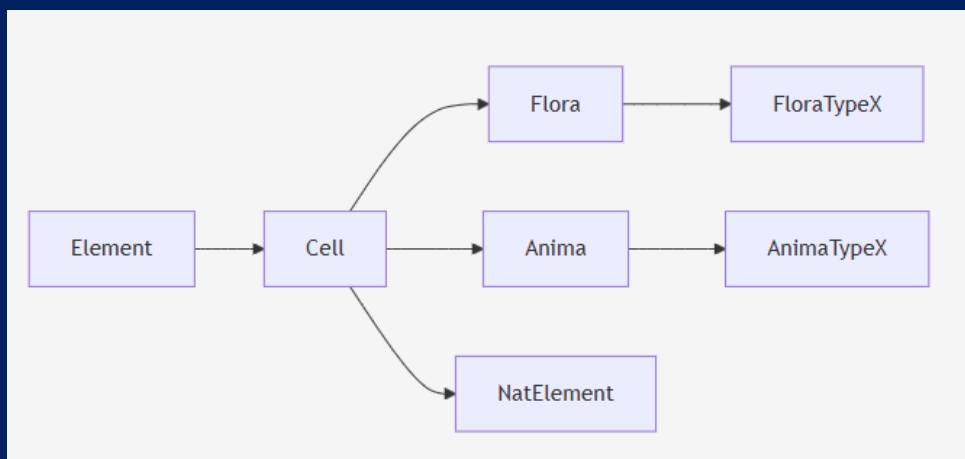
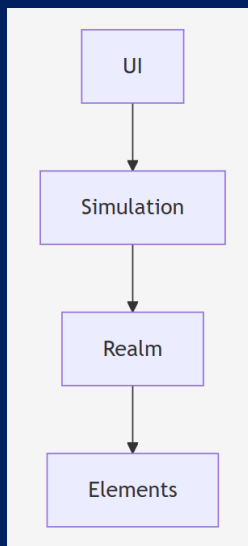
Overview

The phase will be complete when the following can be demonstrated:

- Working simulation framework in which worlds 'Realms' can be added and destroyed
- Plants can be spawned into a realm, and its data added to the realm's frame
- Water can be spawned into a realm, and its data added to the realm's frame
- Time can be incremented in the simulation
- Plants are affected by time, and can bear different species of fruit
- Plants and water can be removed from the realm
- The simulation runs without noticeable bugs or difficulties

Design

The class hierarchy of the project will be UI allowing for instances of Simulation which uses an instance of Realm, with Realm containing all elements $\text{Element} > (\text{Cell} > (\text{Flora} > \text{FloraTypeX}) = (\text{Anima} > \text{AnimaTypeX})) = \text{NatElement}$



(template used to make relationship diagrams representing class hierarchy and family)

UI and Simulation are modularly separated, and so I start with the most basic class which is necessary for the project's core loop: the Simulation class. This class will handle all the meta-simulation aspects including time manipulation and simulation creation/deletion.

Class	Simulation
Attributes	realm time
Methods	constructor tick pause play end
Parent	None

Now, the Realm class, which will contain the data for all elements being simulated.

Class	Realm
Attributes	ElementList: animaList, floraList, natList
Methods	constructor end
Parent	None

When an element is spawned, it is added to the relevant sublist in ElementList. *If it is an anima it is given a record in the data analysis section and population data will update to reflect the addition. The anima's data is written to its record while it remains alive, and it will also influence population data.*

When the anima dies it is removed from ElementList and deleted. *Its record in the data analysis section becomes fixed, and population data/graphs from its period of life will include its presence, but it will no longer affect future updates.*

In this phase of development, from above only the **bold yellow text** will be implemented.

All elements will inherit from a basic element template 'Cell'. The Cell class has no parent and consists of some few simple attributes and methods:

Class	Cell
Attributes	x y
Methods	constructor reproduce end
Parent	None

'x' and 'y' denote the cell's position on the world plane. The constructor initialises the object passing in the initial x and y values. 'reproduce' duplicates the cell. 'end' deletes the cell and removes all references to it (for anima cells I may the data pertaining to the cell may be stored still in the data analysis section).

Plants will be stationary elements of nature existing on the world plane. They will inherit from the Cell class. The name of the plant class is 'Flora'.

Plants will consist of two elements: the flower element and the fruit element. The flower is the primary part of the plant and is not consumed by the producers. The general flower population is designed as an environmental constant, in the sense that flowers die and spawn at preset ranges of frequency and random position; no effect on the flower or its death/spawn attributes are produced by organisms or the environment over time; this is because we are observing the evolution of animals only, so plants here can be seen as a varying factor, but without their own individual adaptation or focus, beyond a random scattering and coming in and out of existence- if I gave plants reproductive and death cycles simulating natural selection as in the animas, I would be focusing more on consumer/producer ecosystem population cycles rather than evolution, and these two are both conflicting and different in scope, so I shall not do so. I do not wish for there to be great variance in the numbers and distribution of plants over time; it should be stable, so that the conditions for evolution are more stable, and a more focused study of evolution can be produced, much in the way that a scientific test is better guaranteed with control variables kept fixed. A final note is that evolution tends to occur over millions of years, where while conditions fluctuate, there is a general stream of continuity where dramatic changes are more rare; such is the ideal form for the environment given to facilitate evolution of the animas, so fickleness in common factors will not do, most especially if we see any extinctions. I move on to say that the fruit is what is consumed by the animas; they are spawned in by parent flowers, and once consumed or overripen they will go out of existence, and if consumed the anima will increment satiation. I may explore animas allowing flowers to proliferate by spreading the seeds of consumed fruit, but this will be a non-essential later update.

As of the early stages, I have in mind two plants to be added: one which spawns only in water, and the other which spawns away from water. I will call these the lily and the tree, respectively. Their attributes can be customised in the settings, but by default the tree will spawn infrequently and produce large fruit at a slow rate while the lily will be more common and produce smaller fruit at a faster rate. I find this to be a good default model because I hypothesise that slower creatures will be able to survive near water by feeding purely from the lilies which are abundant and produce fruit quickly, such that the slow creatures, which have lesser consumption needs, may meet them both in a single location which they need not often stray from. However faster creatures may move quickly between foraging for food at trees, and finding water, which will be geographically separated. The greater fruits of trees are compensation for this effort. So I hope a sort of diversification might occur, with slow water-dwellers, and fast tree-consumers, which would be analogous to species diversifying to fill their niche in the real world via natural selection.

I would need to develop algorithms pertaining to the generation of plants, and their behaviour whilst alive; each plant being an object in the OOP framework, they would run their behaviours in 'parallel', updating based on an update() method each tick of the simulation. The class structure would be as follows:

Cell > Plant > Tree = Lily

The use of OOP here enables for the easy addition of new plants; I would simply add new classes which inherit from the established Plant superclass, which in turn inherits from the established Cell superclass which is parent too to animas. Polymorphism allows me to have similar methods function differently between species, and encapsulation means each object is self-contained and can be easily operated on in accordance with its methods to simulate individual behaviours.

Class	Cell
Attributes	x y
Methods	constructor

	reproduce end
Parent	None

Class	Flora
Attributes	x y max fruit fruit count fruit rate
Methods	constructor reproduce bear end
Parent	Cell

Class	Tree
Attributes	x y max fruit fruit count fruit rate
Methods	constructor reproduce end
Parent	Flora

Class	Lily
Attributes	x y max fruit fruit count fruit rate
Methods	constructor reproduce end
Parent	Flora

There is the question of flora colliding with each other on spawn. The chance of this happening is quite high, and I have several choices of how to handle this case:

- 1) Ignore and allow flora to overlap
- 2) When spawning flora, check if there is a collision and adjust position if so
- 3) When spawning flora, check if there is a collision and re-generate position if so (use of while loop)
- 4) When spawning flora, check if there is a collision and merge flora if so (would delete the second flora and adjust attribute values of the first)

I will initially proceed with 1) and if, in development and testing, this seems problematic, the other options do not appear difficult to implement.

Plants can be deleted from the simulation by removing the associated flora from the element list. (This may become slightly more complicated when data analysis is introduced in phase 4)

Flora will for this phase be stationary and fixed in the simulation plane, once generated.

The implementation of water requires that I flesh out how the location system works.

The world should not be grid-based as this limits me to discrete values and adds a layer of unnecessary abstraction to the simulation itself.

Instead, each element has the 'x' and 'y' attributes which denote their place in the abstract world plane. The simulation logic will run by reading these x and y attributes, and the GUI logic is then fixed upon the simulation objects so that it depends on the simulation logic, but not the other way around.

The world plane will be divided into cells only for spatial hashing, so detection logics will rely on a grid-based system, but this is only for optimisation of searching and will not affect the output of the processes themselves compared to if instead the whole list of elements was searched.

Water will then be generated as sets of polygons which are stored within a list in a given realm alongside the lists for floras and animas. Then, the simulation can check whether an element is in water by querying whether its x/y centre or various collision x/y points are within any polygon of any set within the list. The GUI can use the list to generate the sets of polygons. There is then no need for a discrete grid.

Water will be randomly generated but certain factors of generation will be controllable by the user (phase 3).

To reduce water polygons can be deleted or some of their points moved towards each other. To delete a whole body of water the set can be deleted.

Water will for this phase be stationary and fixed in the simulation plane, once generated.

When time is incremented in the simulation, flora will have a chance to bear fruit according to their 'rate'. I may consider allowing a single flora to spawn multiple different species of fruit later, but for simplicity, each flora will currently spawn only once species of fruit, which is then their species. Different species of fruit will have different effects on an anima's status once consumed, and different species of flora will have different methods of generation. An anima will have varying levels of preference toward each fruit, determined genetically. This creates varying environments within the simulation and possibly allows different anima species to emerge without all animas competing with all others.

Some default fruit objects will be stored with the Fruit class in 'fruit.py'. Fruit objects may also be randomly generated with a name from a set list and attributes from a set range.

Customisation of fruits pre-simulation will be available to users (phase 3).

The system of fruits and flora was found to be clear and intuitive by the student stakeholders.

The world will be generated by first generating terrain according to the arguments passed into the realm constructor, then likewise generating the flora.

In phase 1, there will be three types of terrain: grass, water and ice. By default land is the 'grass' terrain, upon which trees and most plants can grow. The 'water' terrain is characterised by alleviating anima thirst and facilitating the growth of lilies. In the 'ice' terrain anima movement will be slowed and no plants (for now) will grow.

Terrain types are allocated to the world space in the following way:

For each terrain, 'pools' of that terrain, realised using 'noisy' polar curves, will be generated until a 'limit' attribute for that terrain, pre-passed into the realm constructor as an argument, is exceeded. Where pools collide, one terrain will override the other according to a fixed hierarchy (water > ice > grass).

Each polar curve is generated by first selecting a centre from a random x and y position, then, for each gradation angle from 0 to the gradation realm constant (say, 1000), and varying the radius by incrementing it by some trigonometric function of the angle offset by a random constant. The pool is then formed from these points which are each saved, one at each gradation angle.

These pools are stored as polygons of points within the terrain's corresponding sublist nested within 'realm.environment'. These polygons are filled in with True values by a function determining whether a coordinate is inside the polygon. Animas moving through the world can query the terrain of their current position by checking the boolean value at the corresponding position within the nested sublists.

Flora are generated at random positions (using random x and y in the plane) within compatible terrain until the corresponding flora limit attribute is exceeded (each plant species receives its own flora limit, passed through together in a list to the realm constructor). Flora logically exist only at their centre, but visually will be represented by larger circles which may overlap (the same will be true for animas later).

Moving into development, I need to identify and justify the **test data** to be used in the following section:

- I will examine the terminal output when the element object directory of each element in the list belonging to a testing realm is printed as a list; this will allow me to check if the object creation has been correct with the constructor establishing the right attributes
- I will examine the terminal output when the list of elements in a realm's flora list is printed; this will allow me to check that the right kind of object has been correctly created and added to the list with the test functions
- I will examine the terminal output when the realm's main element list is printed; this will allow me to check that the structural hierarchy of the realm object and its primary attribute is correct
- I will examine the terminal output when various element attributes are printed for all members of the flora list; this will allow me to verify that the values given to these attributes are valid and correctly validated

I will now identify and justify any **further data** to be used in the post development phase:

- I will test to ensure that calling the realm tick function increments realm time by printing the time attribute; this ensures correct functionality

- I will examine the output of printing the realm environment; this ensures that several functions have all succeeded in producing the necessary output, and allows me to examine the created realm list so the exact nature of its elements can be known and compared with the values recognised within anima logic in the next development phase
- I will examine the output of printing the various realm status checking methods for attributes post-generation; this similarly ensures that the same few functions have succeeded in producing the necessary output, with more explicit values that can be compared to the inputs (of wetness, iciness, etc)
- I will examine the terminal output when print lines are embedded into the pool generation function giving each iteration count number and describing key events; this ensures that pool generation performs a correct number of iterations, checking the iteration limit and allowing further verification if the difficult pool generation becomes stuck or produces an invalid output, in tandem with the other tests

Development & Testing

It is important when starting the coding that I do not add too much overhead too quickly; I should first build only what is necessary for the demonstration of the project focus: evolution.

The simplest loop I can create to accommodate this involves:

1. Variation
2. Reproduction
3. Selection
4. Death

I will not develop UI, the data analysis system, and more complex behaviours yet.

Under the OOP framework, I should make a simulation class which will contain all the animals and plants in runtime.

I will call this class Realm.

The following code is implemented on 03.01.2026:

```
class Realm:
# this is where it all happens!
    def __init__(self, myRealm):
        self.realm = myRealm
        self.time = 0

    def tick(self):
# each tick represents one cycle in the simulation; it is the atomic unit of time in the active realm
        self.realm.update()
        self.time += 1
```

The time variable gives the value of the number of ticks which have elapsed since the simulation start; later, additional functionality can be added to allow manipulation of time, so that the simulation can be reset to previous ticks.

This was my original draft, but after discussion with a peer, we realised that if I had only one class managing both the individual cells, and also the meta-simulation of time control and multiple realms, the code could get messy. So I decided to add another class to serve as the backbone of the project, within which the individual realms could be run.

```
class Simulation:
    def __init__(self):
        self.realms = []
        self.time = 0

    def birth_realm(self, newRealm):
        self.realms.append(newRealm)

    def tick(self):
#increment time for all realms
        self.time += 1

        for realm in self.realms:
            realm.tick()

    def tock(self, realmName):
#increment time for a specific world
```

```

for realm in self.realms:
    if realm.name == realmName:
        realm.tick()

```

I made two distinct methods `tick()` and `tock()` for specific and universal updating. In the case of `tock()`, the simulation itself does not increment in time, because the universal time is meant to be a general indicator for how much time has elapsed across worlds, and so individual realm manipulation should not impact this. This also maintains consistency for how `tick()` operates across all objects.

```

class Realm:
    def __init__(self, name, primordialList):
        self.time = 0
        self.name = name
        self.elementList = primordialList

    def add_element(self, newElement):
        self.elementList.append(newElement)

    def tick(self):
        #increment time for all elements
        for element in self.elementList:
            element.tick()

```

In a new file named 'cell.py', I made the Cell class:

```

2  class Cell:
3      def __init__(self, spawnx, spawny):
4          self.x = x
5          self.y = y
6
7      def reproduce(self):
8          pass
9
10     def end(self):
11         del self
12

```

Currently its attributes are simply its x and y positions on the world plane. These start at the values of the arguments passed through to the `spawnx` and `spawny` parameters.

The `reproduce()` method currently does not do anything.

I wrote some code for the Flora class but unfortunately it was deleted by VSCode.

```

class Flora(Cell):
    def __init__(self, spawnx, spawny, capacity, rate, fruitNo = 0):
        super().__init__(spawnx, spawny)

```

```
self.capacity = capacity
self.rate = rate
self.fruitNo = fruitNo
```

Above is the new code (23/01/26)

27/01:

For a while I have had a 'could not find node.js' error making me unable to run the program on my computer. I ensured the interpreter for my file was the correct python interpreter. I ran python in my command terminal and realised python was not properly added to my path, even though I have other projects able to run it. This was because they had a venv set up whereas this project was attempting to run the default node.js. So I decided to install python globally on my computer. I downloaded python from the Microsoft store. However I read that this was not a good long term solution. I deleted store python. From python.org I downloaded the Python 64-bit installer and installed python globally on my computer, adding it to my path. This enabled me to run Python files manually as Python, but by default VS Code still attempted to run using node. I considered deleting Code Runner, but instead I amended my launch.json, setting type to python and console to an integrated terminal. I now have python downloaded correctly and accessible by VS Code.

With VS Code debugging tools I am now able to fix all the syntax errors by running main.py with all class files imported.

"if realm.name = realmName:" was corrected to "if realm.name == realmName:"

Line 37 of simulation.py

I added a SPACE + H keybind shortcut to run the current file as a Python file.

Unfortunately chords on VS Code do not work in the way I expected. I removed the chord.

I fixed various other syntax and runtime errors.

I tested using the following code:

```
def main():
    mySim = Simulation()
    testElementList = []
    testRealm = Realm("forest", testElementList) #birth new test realm
    mySim.add_realm(testRealm)

    return 0
```

There were no runtime errors.

I used the following test code to test the structure of the program:

Set up-

```
mySim = Simulation()

testElementList = []
for i in range(10):
```

```

        testElementList.append(Flora(random.randint(-128, 128),
random.randint(-128, 128), 10, 0.1))

    testRealm = Realm("forest", testElementList) #birth new test realm

    mySim.add_realm(testRealm)

```

Test code-

```

for element in testRealm.elementList:
    print(dir(element))
print(mySim.realms[0].elementList)
print(testRealm.elementList)
for element in testRealm.elementList:
    print(element.capacity, element.fruitNo, element.x, element.y)

```

Terminal output-

```

['class', 'delattr', 'dict', 'dir', 'doc', 'eq', 'firstlineno', 'format', 'ge', 'getattr', 'getstate', 'gt', 'hash',
'init', 'init_subclass', 'le', 'lt', 'module', 'ne', 'new', 'reduce', 'reduce_ex', 'repr', 'setattr', 'sizeof',
'static_attributes', 'str', 'subclasshook', 'weakref', 'bear', 'capacity', 'end', 'fruitNo', 'rate', 'reproduce',
'x', 'y']

```

```

['class', 'delattr', 'dict', 'dir', 'doc', 'eq', 'firstlineno', 'format', 'ge', 'getattr', 'getstate', 'gt', 'hash',
'init', 'init_subclass', 'le', 'lt', 'module', 'ne', 'new', 'reduce', 'reduce_ex', 'repr', 'setattr', 'sizeof',
'static_attributes', 'str', 'subclasshook', 'weakref', 'bear', 'capacity', 'end', 'fruitNo', 'rate', 'reproduce',
'x', 'y']

```

```

['class', 'delattr', 'dict', 'dir', 'doc', 'eq', 'firstlineno', 'format', 'ge', 'getattr', 'getstate', 'gt', 'hash',
'init', 'init_subclass', 'le', 'lt', 'module', 'ne', 'new', 'reduce', 'reduce_ex', 'repr', 'setattr', 'sizeof',
'static_attributes', 'str', 'subclasshook', 'weakref', 'bear', 'capacity', 'end', 'fruitNo', 'rate', 'reproduce',
'x', 'y']

```

```

['class', 'delattr', 'dict', 'dir', 'doc', 'eq', 'firstlineno', 'format', 'ge', 'getattr', 'getstate', 'gt', 'hash',
'init', 'init_subclass', 'le', 'lt', 'module', 'ne', 'new', 'reduce', 'reduce_ex', 'repr', 'setattr', 'sizeof',
'static_attributes', 'str', 'subclasshook', 'weakref', 'bear', 'capacity', 'end', 'fruitNo', 'rate', 'reproduce',
'x', 'y']

```

```

['class', 'delattr', 'dict', 'dir', 'doc', 'eq', 'firstlineno', 'format', 'ge', 'getattr', 'getstate', 'gt', 'hash',
'init', 'init_subclass', 'le', 'lt', 'module', 'ne', 'new', 'reduce', 'reduce_ex', 'repr', 'setattr', 'sizeof',
'static_attributes', 'str', 'subclasshook', 'weakref', 'bear', 'capacity', 'end', 'fruitNo', 'rate', 'reproduce',
'x', 'y']

```

```

['class', 'delattr', 'dict', 'dir', 'doc', 'eq', 'firstlineno', 'format', 'ge', 'getattr', 'getstate', 'gt', 'hash',
'init', 'init_subclass', 'le', 'lt', 'module', 'ne', 'new', 'reduce', 'reduce_ex', 'repr', 'setattr', 'sizeof',
'static_attributes', 'str', 'subclasshook', 'weakref', 'bear', 'capacity', 'end', 'fruitNo', 'rate', 'reproduce',
'x', 'y']

```



```
['class', 'delattr', 'dict', 'dir', 'doc', 'eq', 'firstlineno', 'format', 'ge', 'getattr', 'getstate', 'gt', 'hash',
'init', 'init_subclass', 'le', 'lt', 'module', 'ne', 'new', 'reduce', 'reduce_ex', 'repr', 'setattr', 'sizeof',
'static_attributes', 'str', 'subclasshook', 'weakref', 'bear', 'capacity', 'end', 'fruitNo', 'rate', 'reproduce',
'x', 'y']
```

```
['class', 'delattr', 'dict', 'dir', 'doc', 'eq', 'firstlineno', 'format', 'ge', 'getattr', 'getstate', 'gt', 'hash',
'init', 'init_subclass', 'le', 'lt', 'module', 'ne', 'new', 'reduce', 'reduce_ex', 'repr', 'setattr', 'sizeof',
'static_attributes', 'str', 'subclasshook', 'weakref', 'bear', 'capacity', 'end', 'fruitNo', 'rate', 'reproduce',
'x', 'y']
```

```
['class', 'delattr', 'dict', 'dir', 'doc', 'eq', 'firstlineno', 'format', 'ge', 'getattr', 'getstate', 'gt', 'hash',
'init', 'init_subclass', 'le', 'lt', 'module', 'ne', 'new', 'reduce', 'reduce_ex', 'repr', 'setattr', 'sizeof',
'static_attributes', 'str', 'subclasshook', 'weakref', 'bear', 'capacity', 'end', 'fruitNo', 'rate', 'reproduce',
'x', 'y']
```

```
['class', 'delattr', 'dict', 'dir', 'doc', 'eq', 'firstlineno', 'format', 'ge', 'getattr', 'getstate', 'gt', 'hash',
'init', 'init_subclass', 'le', 'lt', 'module', 'ne', 'new', 'reduce', 'reduce_ex', 'repr', 'setattr', 'sizeof',
'static_attributes', 'str', 'subclasshook', 'weakref', 'bear', 'capacity', 'end', 'fruitNo', 'rate', 'reproduce',
'x', 'y']
```

```
[<flora.Flora object at 0x000001BFD0F612B0>, <flora.Flora object at 0x000001BFD0E574D0>,
<flora.Flora object at 0x000001BFD0E57390>, <flora.Flora object at 0x000001BFD0E37950>,
<flora.Flora object at 0x000001BFD0E37CE0>, <flora.Flora object at 0x000001BFD0F70950>,
<flora.Flora object at 0x000001BFD0E5B240>, <flora.Flora object at 0x000001BFD0E5B350>,
<flora.Flora object at 0x000001BFD0467C50>, <flora.Flora object at 0x000001BFD0467D50>]
[<flora.Flora object at 0x000001BFD0F612B0>, <flora.Flora object at 0x000001BFD0E574D0>,
<flora.Flora object at 0x000001BFD0E57390>, <flora.Flora object at 0x000001BFD0E37950>,
<flora.Flora object at 0x000001BFD0E37CE0>, <flora.Flora object at 0x000001BFD0F70950>,
<flora.Flora object at 0x000001BFD0E5B240>, <flora.Flora object at 0x000001BFD0E5B350>,
<flora.Flora object at 0x000001BFD0467C50>, <flora.Flora object at 0x000001BFD0467D50>]
```

```
10 0 80 106 10 0 -94 115 10 0 78 -85 10 0 -100 -99 10 0 -97 -65 10 0 -5 -7 10 0 -46 120 10 0 -32 116 10
0 48 -95 10 0 126 -17
```

Line 1: directory function creates the same output for each object as they have the same attributes and methods.

Line 2/3: Since testRealm and mySim.realms[0] refer to the same object, these lines are identical. We can see the output is a list containing the objects with their hexadecimal memory locations. We can see that these locations are not evenly spaced and appear slightly random.

Line 4: The output is expected with no identified errors. We can see that each value of a flora's capacity is 10 as is passed through the initialiser; the value of 'fruitNo' is 0 as this is the default value and only 4 arguments are passed through, and the x and y positions are randomised in the range (-128, 128) exclusive of the upper value.

Repeatedly running the program does not introduce or see any errors.

29/01:

I debated whether animas should be able to consume all the fruit of a given flora, or simply take one/a few, which would allow multiple animas to feed from the same flora and also make feeding more balanced rather than prone to lucky swings.

Here is the current contents of 'fruit.py':

```
class Fruit():
    def __init__(self, name = "Apple", statusChange = [-25, 0, 0, 0]):
        self.name = name
        self.statusChange = statusChange

apple = Fruit()
lily = Fruit("Lily", [-10, 0, 0, 0])
lavendar = Fruit("Lavendar", [0, 0, 5, -25])
melon = Fruit("Melon", [-30, -20, 0, 0])
```

17/02:

Today I am focusing on the generation of the environment.

Many public and private helper methods are added to 'Realm', including 'generate_world()' and 'generate_pool()'. The constructor is updated. Necessary constants are added. 'angle.py' is updated to include the pi constant and trigonometric functions.

_0 for legacy, all others replaced with _

Terrain developed; pools generated with polar functions:

```
while self.check_water() < self.wetness and iteration_count <
MAX_WATER_GENERATION_ITERATION_COUNT:
    cx = random.randint(0, self.width - 1) #floats?
    cy = random.randint(0, self.height - 1)
    pool_radius = random.randint(WATER_MIN_RADIUS, WATER_MAX_RADIUS)
    self.generate_pool(cx, cy, pool_radius, 1)

    iteration_count += 1
    print(iteration_count)
    print("iteration^")
```

```
def generate_pool(self, cx, cy, radius, type):
    #using parametric polar curves to generate curved pools of varying shape

    angle_start = random.randint(0, POOL_ANGLE_GRADATION)
    r = radius
    pool_points = []
    for theta in range (POOL_ANGLE_GRADATION):
        normalising_factor = 2 * PI / POOL_ANGLE_GRADATION
        n_theta = theta * normalising_factor
        t = (angle_start + theta) * normalising_factor
        r = max(r + random.randint(-3, 3) * sin(t) + 3 * cos(t), 1)
        # -> collect points using rsintheta and rcostheta
        pool_points.append((cx + r*sin(n_theta), cy + r*cos(n_theta)))
    print(theta)
```

```

        for point in pool_points:
            print(point)
            if 0 <= point[0] < self.width and 0 <= point[1] < self.height:
                self.environment[int(point[0])][int(point[1])] =
terrain_index[type] #temporary code, need fill in           possible solution:
keep choosing adjacent tile closest to next (polar?) function result until back
to original tile, then closed object can be easily filled in using edge of pool
as boundaries for fill in, for each row start filling in when reach one and
stop when reach next, but need to cover edge cases of pools that go off map,
cant then use polarity, so how?
            # -> draw inside polygon of points, checking for edge of world
            #fill in with terrain_index[type]

```

Currently I have not yet implemented pools being filled in as this would take a slightly more complicated logic, taking into account pools which go out of bounds of the world.

So I must think of a way to “fill in” the discrete cells within a pool with edge formed by the polar equation (rounding edge boundaries to the nearest integer, as the polar equations give outputs in the rational set- logically real, but rational due to computer rounding to give floating point binary representation).

If all pools had edges within bounds then I could simply open and close pools using the edges as markings using the following procedure: After generating each pool edge shape, I should organise the points into the row that the point resides in. Then, moving down row by row and from left to right, once an edge point is encountered, then begin to fill in the points toward the right, until another point is encountered. So the state of filling in points is toggled and dependent on the parity of the point encountered in each row; pool area is enclosed by even-parity points to the left and odd-parity points to the right (taking the first point as having an index 0). If points are adjacent the toggle occurs only once for the group of adjacent points so they are treated by the algorithm as a single point.

One issue with this method is that I need to take into account pools which spill over realm bounds (pools colliding is irrelevant as each pool is taken in isolation and only fills in positively; pools overlapping is then benign).

Another limitation of this is it could get the shape of the pool wrong; it takes the naive approach of assuming that the area between two points on an edge in a row is filled in, according to the parity of the area. This does not correctly handle the cases where the pool edge is only at a certain y level for a smaller stretch of the pool.

Potentially a better method would be to continuously fill in the line segments between the centre and each point, through the angle theta so that each point is accounted for. Since the polar generation of points means that at any given angle there is only one point in its direction, this will not mis-handle the shape of the pool. The main consideration is how wide to make each segment; too broad and the edges become messy and lose detail; too narrow and the pool may have empty areas left unfilled-in, as well as the edges being potentially jagged. I could choose to handle the tuning differently based on where in the pool was being filled, but this seems like unnecessary complication for the task.

So I thought of an improvement on the original method: this could be to first fill in the lines between each continuous point around the edge, and then perform the procedure as normal BUT ALSO checking vertically; a point is only inside the pool if it meets the criteria for both the horizontal and vertical checks. In practice, this would mean creating two separate true/false lists with values corresponding with each point, and then filling in the points on realm.environment according to the

condition of BOTH lists returning true at the given point (alternatively, add points to each list if fitting the directional criteria, and then the condition would be whether a point is within both lists). Perhaps more efficiently, in fact, I could use one list where the point condition is either an integer value of 2 (1 in each direction), or points being included twice in a list (using the list.count method)

Post-Testing

****This is the post-phase testing, but there has been abundant testing throughout development which can be seen in the previous section. The purpose of this section is to formalise testing which was completed after the phase's development was finalised in order to ensure the complete package functions as intended; a more holistic approach will be taken****

Using the dummy testing object 'testRealm':

```
testRealm = Realm("forest", 100, 100, 0.2, 0.05, [{"tree", 200}, {"lily", 10}], test_element_list) #birth new test realm

mySim.add_realm(testRealm)
```

Test type	Test statements	Test data	Expected Result	Actual Result
	<pre>print(testRealm.time) testRealm.tick() print(testRealm.time) mySim.tick() print(testRealm.time)</pre>	--	0 /n 1 /n 2	0 /n 1 /n 2
	<pre>print(testRealm.environment)</pre>	--	Array with 100 sublists of 100 string items, mostly "grass" which randomly distributed flora strings	Array with 100 sublists of 100 string items, mostly "grass" which randomly distributed flora strings
	<pre>print(testRealm.check_water()) print(testRealm.count_anima()) print(testRealm.count_flora()) print(testRealm.check_grass())</pre>	--	0.0 20 200 0.98	0.0 20 200 0.98
	<pre>while self.check_water() < self.wetness and iteration_count < MAX_WATER_GENERATION_ITERATION_COUNT: cx = random.randint(0, self.width - 1) #floats? cy = random.randint(0, self.height - 1) pool_radius = random.randint(WATER_MIN_RADIUS, WATER_MAX_RADIUS) self.generate_pool(cx, cy, pool_radius, 1) iteration_count += 1 print(iteration_count)</pre>	--	Sequence of integers from 0 up to some integer below 99 (the max iteration count), each followed by 'iteration^' and sandwiching the theta/point print lines	etc 57 iteration^ --- 58 iteration^

	<code>print("iteration^")</code>				
	<pre> for theta in range (POOL_ANGLE_GRADATION): normalising_factor = 2 * PI / POOL_ANGLE_GRADATION n_theta = theta * normalising_factor t = (angle_start + theta) * normalising_factor r = max(r + random.randint(-3, 3) * sin(t) + 3 * cos(t), 1) # -> collect points using rsintheta and rcostheta pool_points.append((cx + r*sin(n_theta), cy + r*cos(n_theta))) print(theta) for point in pool_points: print(point) if 0 <= point[0] < self.width and 0 <= point[1] < self.height: self.environment[int(point[0])][int(point[1])] = terrain_index[type] </pre>		numbers from 0 to 99, followed by 100 (x,y) pairs where each is close to the last	0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 92 93 94 95 96 97 98 99 (92.0, 104.0687103342445) (93.38570578722914, 101.02516277551723) (94.36707585906291, 97.73733480033465) (94.91076537321372, 94.25876644793375) (95.22347247461367, 91.5545963885647) (95.33804355036301, 89.27344168132628) (94.92620966052746, 86.39076194536784) (93.88338689353289, 83.00240088066388) (92.61815128145449, 80.12441300676385) (92.535826794979, 79.84432792550201) (92.58778525229248, 79.80901699437494)	

				(92.63742398974868, 79.77051324277579) (71.47511101051487, 122.61760934469919) (75.41039151493891, 120.90056807407299) (79.2773061895008, 118.15642528909494) (82.31691941657414, 116.71310891600797) (85.15533594820927, 114.88098551053297) (87.81465286896585, 112.13043396963772) (90.0979447880969, 109.2323018611016) 54 iteration^
--	--	--	--	--

Review

This development phase concluded with the fundamental realm and flora files set up and ready for the introduction of animas in the next phase.

The program is now at a state at which multiple realms containing an environment with natural elements such as grass, water, ice and fruit-bearing plants can be added to a simulation object's realm list.

Time can be incremented in the simulation, with the effect of causing plants to have a chance to bear fruit.

There is a list of different plant species each with different traits and spawning fruit with different effects.

The simulation logic external to animals is now complete.

I have tested the features of this development phase by creating realms and adding flora objects to their element lists; I generated environments including each natural element as randomly generating according to their category. I examined the output terminal for each of various testing print lines and found the results to be in line with what I expected for the phase.

The totality of this phase goes towards meeting my success criteria as I have created a reasonably detailed abstraction of a real-world environment in which animas will be able to interact with the natural elements.

PHASE 2~ ANIMAS

Design

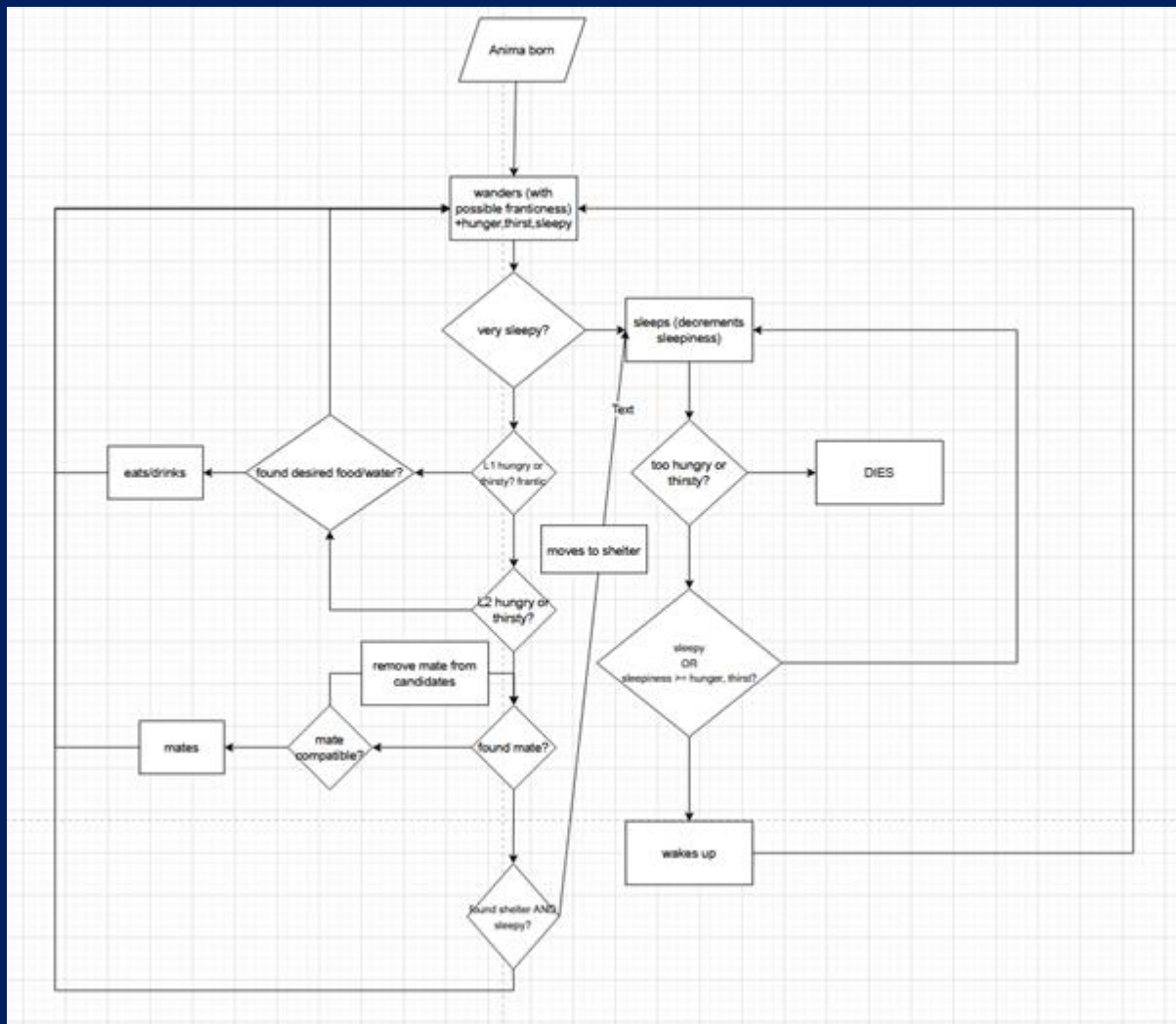
I first looked at the animas: here I define animas as my abstraction of an animal; so 'animal' refers to the real-life biological entity, and 'anima' the computer abstraction.

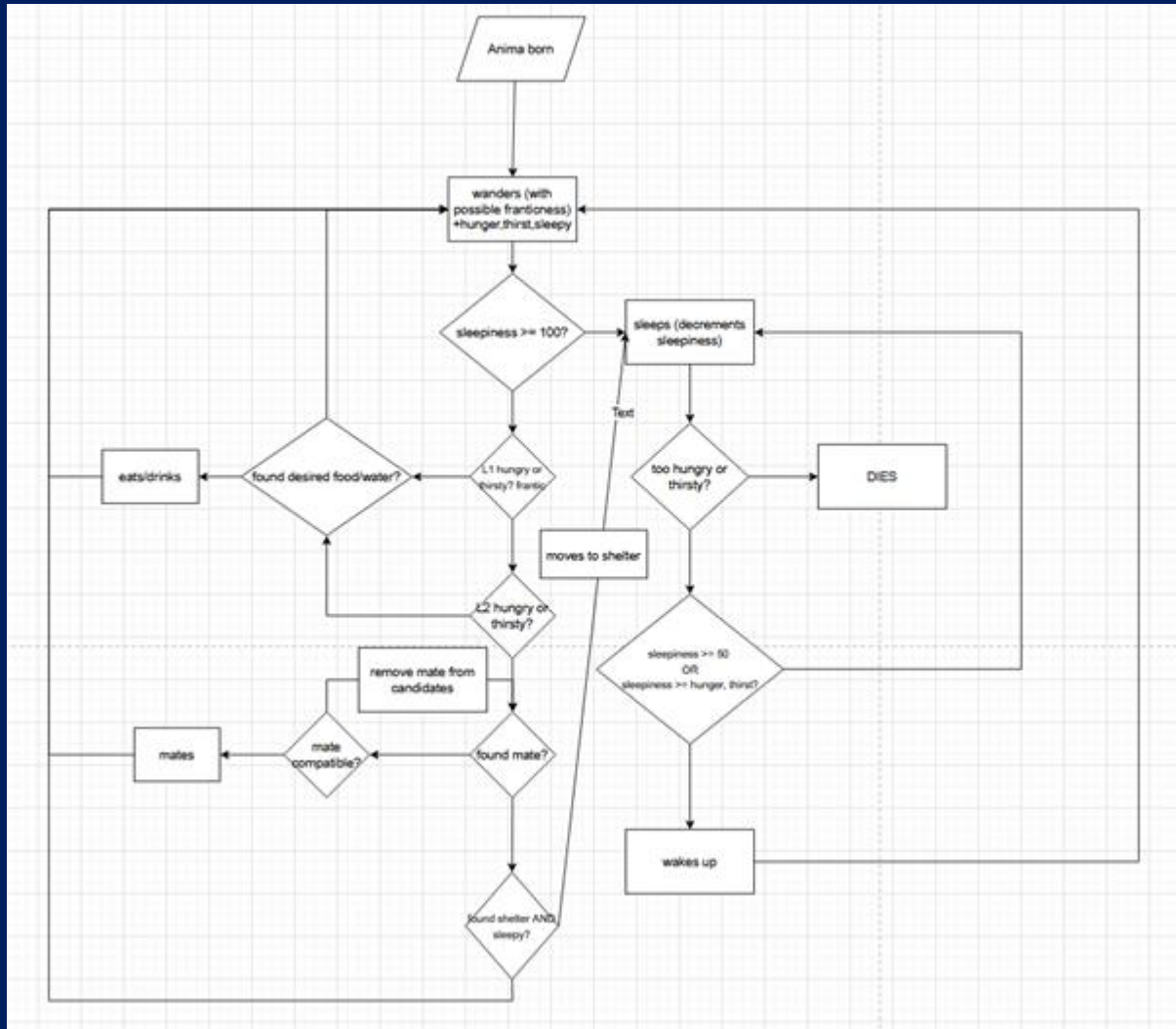
Anima will be a class inheriting from Cell:

Class	Cell
Attributes	x y
Methods	constructor reproduce end
Parent	None

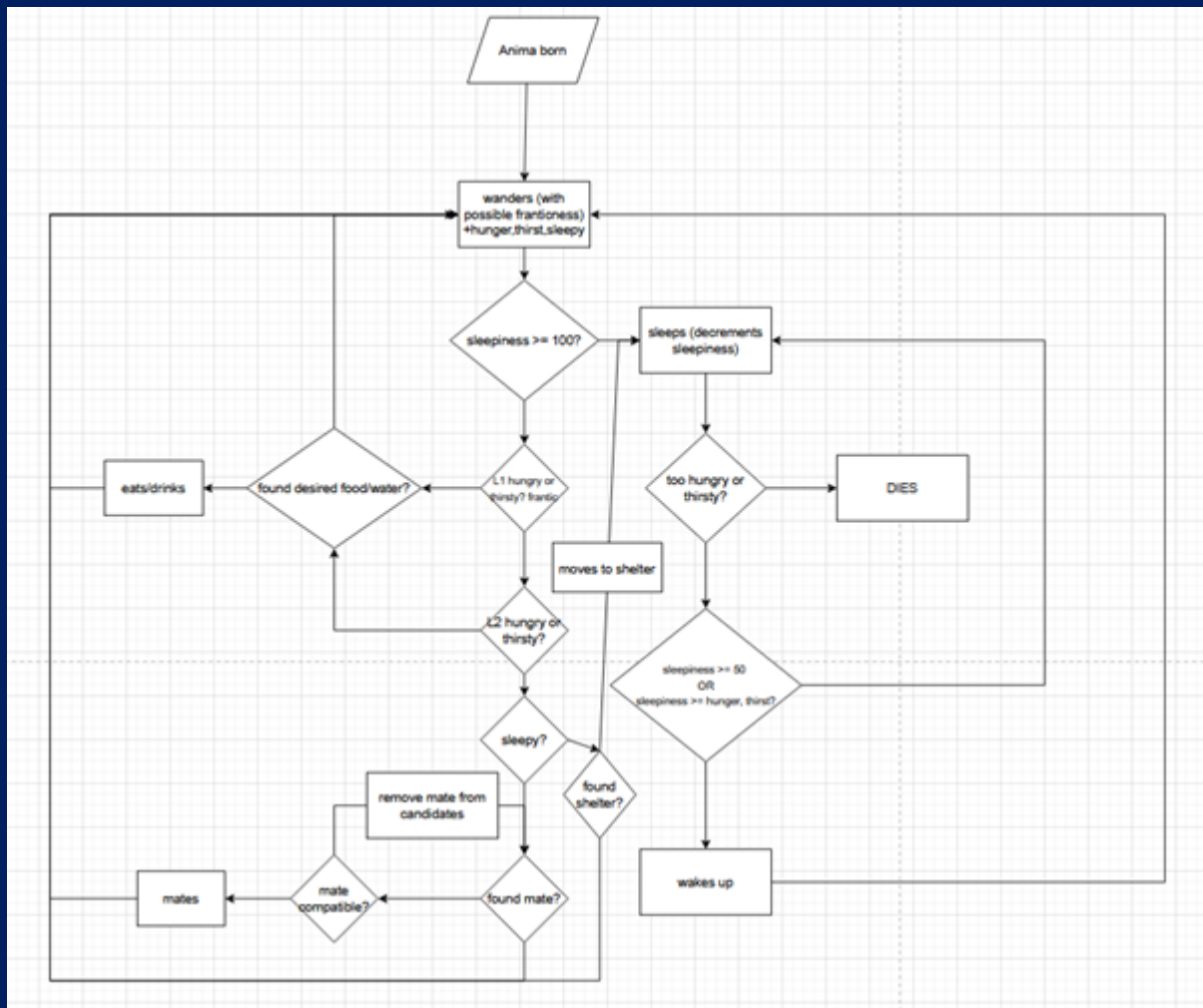
I first experimented with different object logics using *draw.io* to create representational flow diagrams. My first flowchart of an animal logic, dubbed A, includes sleeping within the available actions, which I was tentative about, as the added complexity does not necessarily lead to interesting development, insofar as sleeping would simply entail the animal being idle for a period of time.

Here are two versions of this draft, with the second including some arbitrary sample numbers for the conditionals (I used the input/output symbol instead of the start symbol here):





As we can see, animas wander around, accumulating hunger, thirst and sleepiness. If they are too fatigued, they will be forced to sleep for a while; otherwise if they are too hungry or thirsty they will be only searching for food and/or water [there are two phases to this, a frantic phase and non-frantic; in the frantic phase animas move faster, covering a greater area of the map with more dedicated pathfinding, and becoming sleepier faster]. If they are neither hungry nor thirsty they will look for a mate or shelter. In this draft I realised that animas mating even while sleepy did not make much sense, and decided to make a revised version where mating only occurs if all three of hunger, thirst and sleepiness are low:



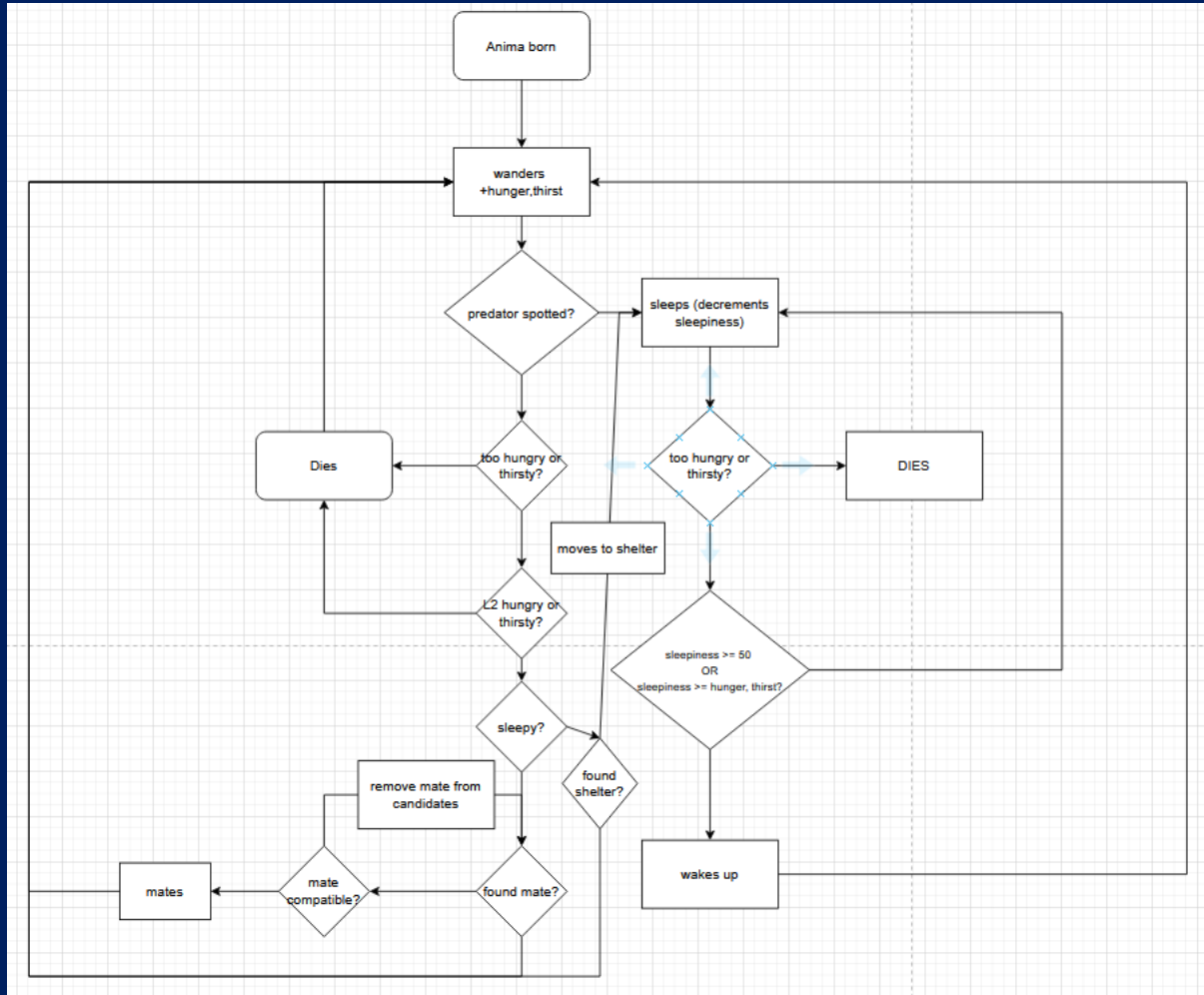
Now instead of having a clash between mating and finding a shelter, there is a clear hierarchy of priorities established (there are benefits and costs to this, namely simplicity & understandability vs interesting complexity).

It can be noted that animas can only die in their sleep. If animas are at a limit of hunger or thirst, but too sleepy to look for food or water, then they die. I have not yet added predation so this is the only method of death.

Anima logic B was the next developed model. I felt that the sleep mechanic did not add sufficient value to the simulation, and I also wanted to allow prey to flee from predators. I thought about giving creatures a flashlight beam-style vision, where they would rotate constantly at a genetically determined speed. Of course then they would only see things temporarily and have to keep waiting for their vision to rotate to an object they would progress towards, so I thought that once a creature found something it wanted to move to/away from, it would train its vision on this object until the task was complete. I would have to code the centre of eyesight tend direction toward the centre of the desired objection, by rotation each frame. However this would make creatures 'tunnel visioned' and left very open to predators when moving towards or away from anything else. I could instead implement that the direction of eyesight is always the direction of movement, and when fleeing from predators I could make prey enter a panic mode where they would simply run in the opposite direction of the predator for a while, then glance backwards to see if they were still chasing, before continuing to wander. While panicking, animals would not eat. While I could implement algorithms which would allow creatures to calculate risk/reward in eating or drinking while panicking to simulate greater

intelligence, this will not fall under the scope of the basic anima logic for now. In addition, instead of using a cycle-based flowchart where the entire flowchart loops each cycle which generally denotes one frame, I will use a state-based flowchart where the program will simply check what state the creature is in each frame, checking the conditions for changing to a different state. Such is the approach I used for anima logic B.

Anima logic B:



I considered also ageing as a mechanic; ageing is of course a staple of biology, having great effect on most organisms on the macro scale. There were benefits to implementation, and I had concerns as well. Here I applied the AI tool ChatGPT to aid with my decision process.

On 24/11/2025 I asked the query: 'how would implementing an ageing system affect the goals of 1) design simplicity 2) insightfulness and being interesting 3) representation and accuracy with regards to evolutionary mechanics'

Great question — ageing seems like a small mechanic, but it has **big conceptual effects** on both the *design* and the *evolutionary logic* of a simulation. Here's a structured analysis of how adding ageing affects your three goals.

1. Design Simplicity

✓ Pros (when ageing *helps* simplicity)

- Provides a clean, universal exit condition
Instead of micromanaging starvation, predation, or arbitrary culling, organisms naturally disappear on their own.
- Lets you structure life cycles
Splitting behaviour into: juvenile → reproductive → elderly phases can make behaviour design more modular.
- Gives predictable turnover
Prevents runaway population growth without complicated balancing mechanisms.

✗ Cons (when ageing makes design more complex)

- Requires a per-tick age update and a death check — trivial code, but adds a new dimension.
- Raises design questions that might snowball:
 - Should traits affect lifespan?
 - Should reproduction shorten lifespan?
 - Should ageing affect performance (speed, fertility, vision)?
- If not careful, ageing can interact with other systems (mutations, resource scarcity, predation) in unexpected ways, complicating balancing.

Summary for simplicity

Ageing adds *slight* implementation complexity but can *reduce* complexity of overall population management.

It is a net win if you keep ageing effects minimal and unified, e.g.:

“Organisms die at age = max_age (plus random noise).”

2. Insightfulness & Interesting Dynamics

Ageing tends to *dramatically increase* evolutionary richness.

✓ How ageing increases insightfulness

- Trade-off evolution becomes visible
Lifespan interacts with reproduction strategy:
 - long-lived, slow reproducers
 - short-lived, fast reproducers
(classic r/K selection pressures)
- Selection for early reproduction



The reply was:

This mirrors real-world evolutionary pressures where genes expressed before reproduction are strongly selected, while late-life traits drift neutrally.

- **Allows evolution of senescence**

Age-related decline can appear without explicitly programming it, simply due to diminishing selection pressure with age.

- **Generational waves**

Populations rise and fall rhythmically as cohorts move through their lifespans.

✓ **Makes simulation more interesting to watch**

- Distinct life phases make organisms feel more “alive.”
- Age-related performance shifts create emergent behaviours:
 - slower elders falling behind
 - young aggressive individuals competing
- Population structure (age pyramids) becomes a metric.

Insightfulness summary:

Ageing almost always increases the educational and emergent value of the simulation.

3. Representational Accuracy (Evolutionary Mechanics)

Here ageing is extremely relevant.

In real evolutionary theory:

✓ **Why ageing is evolutionarily accurate**

- Senescence is near-universal in multicellular organisms.
- Evolution primarily acts on **reproductive fitness**, not post-reproductive survival.
- Classical evolutionary models (Medawar, Williams, Hamilton) show:
 - deleterious late-life mutations accumulate (weak selection)
 - antagonistic pleiotropy favours traits beneficial early, harmful later
- Age-dependent mortality is fundamental in:
 - life-history theory
 - demographic selection
 - mutation accumulation
 - reproductive timing
 - investment strategies

✓ **Including ageing enables:**

- Evolution of life-history traits
(e.g. organisms evolve whether to reproduce earlier or later)
- More realistic population dynamics
- Multiple coexisting strategies balancing survival vs reproduction

✗ Potential risks

- If implemented naively (e.g. all organisms die at exactly age 100), it may produce unrealistic "clockwork" dynamics.
- Too much age complexity can overshadow other evolutionary processes.

Accuracy summary:

Ageing is one of the most evolution-relevant mechanisms you can add, second only to mutation and reproduction.

Overall Conclusion

Should you include ageing in your sim?

Probably yes, unless your design goal is extreme minimalism.

Impact by goal:

Goal	Effect of adding ageing
1. Simplicity	Slightly more code; often reduces overall balancing complexity.
2. Insightfulness	Strongly increases the richness and watchability of evolution.
3. Accuracy	Huge gain; ageing is essential to realistic life-history evolution.

★ Recommended minimal ageing system for your project

To maximise clarity while avoiding design creep:

Base system:

- Organism has `age`.
- Organism dies at `max_age ± random(variance)`.

Optional light extensions:

- Reproduction cost reduces remaining lifespan.
- Traits can affect lifespan (e.g. higher metabolism → shorter life).

Avoid at first:

- Per-organ or per-trait senescence curves
- Multi-factor mortality tables
- Behavioural differences per age stage

Start simple and only add complexity if you see **interesting** evolutionary pressure that warrants it.

I then queried: 'my other concern is how ageing would in fact impact the simulation, ie does what it changes actually promote evolutionary diversity and accuracy, or does it drift more towards short term life cycles with attention taken away from the evolution; and furthermore, is a natural timer for creatures' lifespan arbitrary with my main goal of focusing on evolution. why do creatures need to be limited in this regard, and will it impact genetic trends 1) at all? 2) in an interesting way? 3) to create convergence rather than diversity'

First: What happens if you *do not include ageing*?

A world without ageing is already possible in biology: certain bacteria, some jellyfish, some colonial organisms.

Evolution in a no-ageing model tends to do this:

- Traits under selection pressure become extremely long-term optimised, because individuals accumulate fitness indefinitely.
- Lineage stagnation sets in: highly fit organisms can "dominate forever" because they never die unless killed.
- Mutation becomes riskier because long-lived dominant organisms crowd out novelty.
- Evolution slows dramatically once hyper-competitive lineages appear.

In simulations, this often produces:

- Near-immortal "apex builds"
- Decreased turnover
- Reduced exploration of the trait space

In other words:

No ageing does not promote diversity; it suppresses it.

No ageing does not promote accuracy; it creates artificial immortal champions.

This is why ageing evolved in nature: **loss of older organisms increases population turnover**, which enables evolution to explore the trait space.

So removing ageing entirely usually harms evolutionary exploration.

Now: How does ageing actually impact evolutionary dynamics?

Let's answer your three research questions systematically.

✓ 1) Does ageing impact genetic trends *at all*?

Yes — massively.

Ageing enforces:

- Population turnover
- Selective pressure on early-life traits
- Neutral drift on late-life traits

This prevents dominant lineages from outlasting competitors indefinitely.

If you don't include ageing:

- Individuals accumulate mutations forever.
- You get hyper-optimised organisms that never c ↓
- Evolution becomes mutation-limited instead of selection-limited.

The reply:

Ageing restores the *temporal flow* of evolution.

So yes: it impacts genetics strongly and in predictable ways.

✓ 2) Does it change trends *in an interesting way*?

Yes, if implemented thoughtfully.

Ageing introduces:

Interesting evolutionary pressures

- faster versus slower reproduction strategies
- risky "live fast, die young" lineages versus slow stable ones
- evolution of optimal lifespan
- evolution of early-life performance vs late-life decline
- selection for survival vs fertility trade-offs

These trade-offs are the heart of *life history evolution*, one of the most studied parts of evolutionary theory.

Ageing is one of the main generators of interesting evolutionary patterns.

But: only if you let lifespan evolve

A fixed "all organisms die at age 100" produces shallow dynamics.

Letting lifespan be *heritable and selectable* creates very rich patterns.

✓ 3) Does ageing reduce diversity or cause convergence?

Naive fixed ageing can cause convergence:

If every organism must die at age 60, evolution tends to optimise for:

- maximum reproduction before 60
- maximum resource exploitation early
- traits that maximise short-term gains

This can indeed turn the world into a "short lifespan, reproduce fast, burn out and die" space — a kind of r-selection convergence.

BUT

If lifespan is:

- heritable
- mutable
- has trade-offs
- interacts with ecology



... then ageing actually increases diversity, because different ecological niches promote different life-history strategies.

You can get:

- long-lived slow reproducers in stable environments
- short-lived fast reproducers in chaotic or resource-poor environments
- mixed polymorphisms that oscillate
- coexistence of multiple strategies depending on density, predation, competition

Ageing increases diversity when coupled with ecological complexity.

Is ageing arbitrary in an evolution-focused sim?

No — and here's the key insight:

Evolution requires death.

Without generational turnover, evolution stops.

Ageing is not "arbitrary"; it is a mechanism that regulates the *rate* of generational flow.

You need *some* mechanism that prevents individuals from living forever, because immortality produces:

- dominance monopolies
- stagnation
- reduced evolutionary movement
- extreme selection pressure favouring "safe" traits rather than exploration

You *must* prune old individuals somehow.

Ageing is the most biologically grounded and evolutionarily meaningful way to do that.

But what if you don't want arbitrary timers?

Then you can implement selection-driven mortality instead of ageing-driven mortality:

Alternatives:

- death only from starvation
- death from predation
- death from environmental damage
- death from physical decay (traits degrade)
- reproduction causes physical exhaustion (cost-of-reproduction model)
- long-lived individuals accumulate mutations or metabolic inefficiencies

These mechanisms create *emergent ageing* without assigning a hard-coded lifespan.

To maximise:

- simplicity
- evolutionary depth
- diversity

Use this minimal, non-arbitrary ageing system:

1. Age counter — trivial to implement

```
python
self.age += 1
```

[Copy code](#)

2. Lifespan as a trait — evolvable

```
python
self.lifespan = genome['lifespan']
```

[Copy code](#)

3. Mortality probability increases with age but is stochastic

Not a fixed "die at 100", but something like:

```
ini
death_prob = (age / lifespan)^k
```

[Copy code](#)

where k controls curve shape.

Why this works:

- organisms with longer or shorter lifespan evolve naturally
- no absolute timers — lifespan emerges from genetics
- more dynamic turnover → more evolutionary exploration
- ecological conditions generate multiple viable strategies

This system:

- is simple
- is biologically meaningful
- avoids forced convergence
- produces diversity rather than suppressing it
- keeps focus on evolution, not micromanaged life cycles

If you'd like

I can outline:

- a full mathematical model for age-dependent mortality
- examples of how different lifespan strategies evolve
- an OOP architecture for ageing integrated with your organism class

- a justification in evolutionary theory (Medawar/Williams/Hamilton)
- simulations of what happens with vs. without ageing

Just tell me which direction you want next.

The responses do strongly advocate for a 'thoughtful' implementation for ageing, although I was still unsure due to the fact that it seems to imply a more rich and dynamic framework for the rest of the simulation than what I currently intend; with a limited number of traits to add, I am not sure that ageing should be one of them, and I also do not foresee the issues ChatGPT outlined should I not add ageing to be real problems for me; if there ends up being dominant individuals this is similar to dominance of a trait as a whole and would be embraced by me; this represents fitness well; if mutation is riskier this simply implies a decrease in fitness; if evolution becomes slower then we have reached sufficient fitness for my outlined aim.

(idea: kick force vs bite force, kick behind bite in front, lets prey with strong bite force attempt to defend selves by turning round and confronting; kick uses energy to push chaser backwards but does little damage, bite does more damage, however how do i make it so turning round and waiting to kick

is not equivalent to flee and kick when in range? Making waiting superior as less energy is wasted. Perhaps I make predators waste more energy when running, so kicking to stall while running is superior. Repeatedly kicking while stationary simply uses more energy than running and kicking when pred gets close. Could also put cooldown on kick)

**** mate and predator/prey in same pool may get messy, might have to change to an asexual model****

(Note: see Development for how I tackled this)

The simulation will handle predation primarily using the traits of strength, speed and diet preference.

If an anima identifies another anima, it will calculate a reaction based on its needs and status.

When an anima decides to give chase to another, the former becomes predator and the latter prey. The decision is made on the basis of the diet preference and state of the predator, and the strength and speed of both creatures.

The prey may not immediately notice the predator which would give the latter a headstart.

If the prey escapes the predator's field of vision, the predator will give up the chase (I could make the predators keep chasing in the general direction for a short while, but this risks complicating decision procedures as this would factor into each tick for every anima, as well as a naive approach of a flat timer meaning the predator could neglect important needs while in this momentary chase)

The predator could also decide to give up due to the urgency of other needs.

I have two avenues to handle the collision between predator and prey:
1) They fight to the death; this could happen instantaneously or over a short period; the victor is decided based on strength and speed, with a probabilistic formula.

2) They exchange damage and if both survive the prey once again flees. Depending on the predator's state, they might decide to give chase or give up. If they give up, they will default to an idle state for a few seconds; this is so I do not have to write code to forcefully remind the anima not to chase at each tick of the simulation, or create what can be abstracted to a graph linking each anima to each other. Damage given is again decided by a probabilistic formula factoring strength and speed, and the attribute of damage is stored as the first element of the anima's status; damage can then also be considered as $(100 - \text{vitality})$

For now I will opt for the simpler model of 1), and if I have time during Phase 5 I will update this to 2)

The animas will require a larger number of attributes and methods, including genetic attributes pertaining to their vision, strength, speed and diet. The animas will have attributes pertaining to their instant condition; these will include their current velocity: a vector consisting of dx and dy which will have a maximum magnitude equal to their genetic speed: their current behavioural mode, their current target (set to None if not applicable or if none such exists*), their status: referring to the values determining how they prioritise actions satiating each of their behavioural desires and determining how they can interact with the world, inclusive of their death: their goals, referring to which desires are being currently acted on and influencing behaviour, and their age, referring to how many ticks have passed in the simulation since their creation.

They will inherit from the Cell class, polymorphically overriding the reproduce() and end() methods.

*a different value may be used in this case, maybe -1 in parameters

Class	Cell
Attributes	x y
Methods	constructor reproduce end
Parent	None

Class	Anima
Attributes	x y dx dy mode target goals [] status []: damage, hunger, thirst, sleepiness age speed vision strength diet preference
Methods	constructor reproduce end setGoals chooseMode checkSurroundings eat drink sleep
Parent	Cell

Vision will be handled via raycasting from anima centrepnts. Spatial hashing optimises decisions on anima position lookup.

Mating

The genetic combination algorithm is one of the most important elements in the project, as mutation is one of the fundamental pillars of natural selection theory.

The algorithm will iterate through the length of an anima genome (which will be universal to all animas) and append values to a new genome list such that it will coalesce in the structure of the parents' genomes. Since an anima's genome is composed of elements of various data types, the combination algorithm, on each iteration, should take one of a number of cases, each pertaining to one of the data types which a genome element (we call this a chromosome) can take.

Most chromosomes will be of the float type; to combine parent chromosomes the mean of their values is taken and offset by some random factor scaled to the class-wide mutation factor (which will be 0.05 by default).

Chromosomes of the int type will undergo a similar procedure; however the final result will be rounded to the nearest integer

The tuple type will be reserved for the 'diet' attribute. The combination of parent diets will be more complex, and dependent on the value of a class-wide HYBRID_DIETS_ON boolean variable.

Control of mutation class-wide variables will be given to the user pre-simulation.

Test Design

I was able to test each algorithm in isolation and together in order to ensure the anima code works at each level of the simulation.

The nature of the program is such that anima testing will become much easier when the simulation UI is implemented, so testing will be more limited in this development phase.

Development & Testing

The class majorly to be developed in this phase of development is the 'Anima' class.

```

18 class Anima(Cell):
19     def __init__(self, speed, vision, spawnx, spawny):
20         self.x = spawnx
21         self.y = spawny
22         self.dx = 0
23         self.dy = 0
24         self.speed = speed
25         self.vision = vision
26         self.status = [0,0,0,0]
27         self.target = ""
28         self.goals = []
29         self.mode = 1 # 0 sleep 1 roam 2 approach
30

```

These are the initial attributes I have given to the class. I have not yet incorporated the strength, age and diet preference attributes which were present in the design.

```

def setGoals(self):
    #this method updates the goals of the associated anima, creating a new
    empty list and appending at least one int element goal; the greatest need is
    appended first, then if the second greatest need is close enough in urgency
    then it is appended too, and so on
    #per tick
    self.goals = []
    needs = self.status
    prevGreatestNeed = 0
    for i in range(len(needs)):
        greatestNeed = np.argmax(needs)
        if needs[greatestNeed] > prevGreatestNeed - 3 * math.sqrt(100-
prevGreatestNeed):
            needs.append(greatestNeed)
            prevGreatestNeed = needs[greatestNeed]
            needs[greatestNeed] = 0
    return

```

```

44
45 def chooseMode(self):
46     #this method updates the mode of the associated anima based on anima logic, setting the int variable mode to the correct value (0:sleep/1:roam/2:approach)
47     #per tick
48     if self.mode != 0:
49
50         if self.mode == 1:
51             target = self.checkSurroundings()
52             if target != "":
53                 self.mode == 2
54
55         if self.mode == 2:
56
57             if self.sleepiness >= 100 or (self.mode == 0 and self.sleepiness >= self.hunger, self.thirst):
58                 self.mode = 0
59             else:
60                 self.mode = -1
61
62 def checkSurroundings(self):
63     for goal in goals:
64         #if visible, then set. priority based on list order which is variable priority
65         return True
66     return False
67
68 def eat(self, source):
69     #removes fruit from flora and decreases hunger
70
71 def drink(self, source):
72
73
74 def sleep(self, duration):
75     time.sleep(duration)
76
77 def reproduce(self):
78     pass

```

29/01:

```

def tick(self):
    self.chooseMode()
    if self.mode == 0:
        self.status[2] -= 1

    if self.mode == 1:
        self.move
        self.checkSurroundings()

    if self.mode == 2:
        self.dx = self.speed * (self.target.x - self.x) /
math.hypot(self.target.x - self.x, self.target.y - self.y)
        self.dy = self.speed * (self.target.y - self.y) /
math.hypot(self.target.x - self.x, self.target.y - self.y)
        self.move
        self.checkSurroundings()

    if self.status[3] > 0:
        self.status[3] -= 0.5
        #heal damage every tick

```

For the movement logic, I have intentionally made anima check surroundings only after moving, so every anima moves based on the position of their target from one tick ago. The reason for this is twofold:

- 1) All anima have equal 'knowledge'; it will not be the case that when all animas are ticked, some move towards their target's position last tick, and some this tick
- 2) There is a kind of built in 'reaction time'.

I then realised that this was incorrect and my code did not make sense. I made a revised version of movement requiring the tick to be broken down into two mini-ticks. The first tick (decision) will fix the actions and velocity of each anima. The second tick (action) will move and change the status of each anima accordingly.

Animas must first decide what they will do during action. If they are interacting with a stationary element in range or one they does not require their movement to be in range from, their velocity may be set to zero. We use spatial hashing to find the objects of interest near animas and if within vision range, the animas may decide to interact based on their status.

...

For the adjustment of velocity in the decision tick when roaming I initially wrote:

```
self.dx += self.speed * random.uniform(-0.1, 0.1)
self.dy = (self.speed) ^ 2 - (self.dx) ^ 2
```

I knew however that there were two issues to be resolved here:

- 1) When an anima awoke from sleeping it would instantly have very low horizontal speed and high vertical speed, and this imbalance was not naturally correcting.
- 2) self.dy is always positive

There are a few ways to resolve problem 1 though none I can think of currently that would not impact the logic beyond this restricted scope.

Problem 2 is more severe; I liked the code I wrote here as it was simple and elegant while giving the symmetrical (with respect to x and y) velocity changes I needed; the only issue was how to differentiate between a positive and negative direction for y, and how to make the transition between the two smooth and consistent.

10/02:

To resolve these two problems I can shift my paradigm from Cartesian to polar. Cartesian coordinates have a fixed orientation where movement will be premised around the fixed horizontal and vertical vectors; this is unhelpful because anima movement is symmetrical with full-360 degree movement required to be flexible and easy to model. Polar coordinates are premised around a central origin, allowing for flexible movement from this origin. Since I require movements to be tailored around the centre of my animas, this approach is more apt. All directions are given equal respect because the parameter used is the angle of direction from some initial line; this line made at this angle can be broken into horizontal and vertical components for movement, but adjustment of velocity comes from incrementing or decrementing the angle, giving smoother movement and keeping direction continuous throughout 360 degrees; there is no discernible difference given between any two directions over a long period.

When approaching stationary objects an anima may decelerate to a halt.

During the action tick all animas will have their position incremented by their velocity vectors.

Besides position the main attribute of interest during selection is status, a list containing four integer elements in the range 0 to 100, representing the urgency in priority between four basic needs. These are hunger, thirst, sleepiness, and injury.

Each action tick, after moving and/or taking any actions, an anima's hunger, thirst and sleepiness will be incremented (except sleepiness when asleep), and their injury will be decremented. Eating fruits from flora will have various effects on status, usually including hunger reduction.

Animas will be born slightly hungry and thirsty.

15/02

I developed more on the movement and positioning. I realised before running the code that in python's math library the functions treat angles as expressed in radians. So I adjusted parameter values to reflect this. I made the file 'angle.py' with one function 'lerple' outputting linear interpolation between two angles given the two boundary angles and maximum turn. This can be used for calculating angle adjustment for moving elements each tick (namely anima).

Here is the file 'angle.py':

```
1 import math
2
3 def lerple(current, true, max_turn):
4     change = (true - current - math.pi) % (2 * math.pi) - math.pi
5     change = max(-max_turn, min(max_turn, change))
6     return min(max_turn, max(-max_turn, change))
```

I imported the function into the anima file and I changed the logic of anima decision ticks so that all modes run a shared code block with fixes the velocity change:

```
self.angle = lerple(self.angle, true_angle, self.turnspeed)
self. (variable) dy: Any math.cos(self.angle)
self.dy = self.speed * math.sin(self.angle)
```

The genetic traits 'turnspeed' and 'toughness' were added. Another attribute 'roam_angle' was added'.

Roaming and fleeing were adjusted according to the new structure. Each mode now functionally adjusts 'true_angle'.

The old code is deleted:

```
"""self.dx = self.speed * (self.target.x - self.x) / math.hypot(self.target.x
- self.x, self.target.y - self.y)
        self.dy = self.speed * (self.target.y - self.y) /
math.hypot(self.target.x - self.x, self.target.y - self.y)"""
```

*dinosaur

The updated code is as follows:

```

if self.mode == 1:
    if random.uniform(1) < 0.05:
        self.roam_angle = random.uniform(0, 2 * math.pi)
        true_angle = self.roam_angle
    ...

if self.mode == 2:
    dsx = self.target.x - self.x
    dsy = self.target.y - self.y
    true_angle = math.atan2(dsx, dsy)

if self.mode == 3: #flee
    dsx = self.target.x - self.x
    dsy = self.target.y - self.y
    true_angle = (math.atan2(dsx, dsy) + math.pi) % (2 * math.pi)

self.angle = lerple(self.angle, true_angle, self.turnspeed)
self.dx = self.speed * math.cos(self.angle)
self.dy = self.speed * math.sin(self.angle)

```

16/02:

I fixed some minor errors, including the wrong variable 'needs' being used instead of 'self.goals' in 'setGoals()'

The main task for this day was to create a class for the spatial hash function. In a new file 'spatialhash.py', I wrote the following code:

```

class SpatialHash:
    def __init__(self, cell_size, world_width, world_height):
        self.width = world_width // cell_size + 1
        self.height = world_height // cell_size + 1
        self.unit = cell_size
        self.buckets = [[] for _0 in range(self.width * self.height)]

    def __get_hash(self, x, y):
        return x//self.unit + y//self.unit * self.width

    def get_neighbours(self, x, y):
        my_bucket = self.__get_hash(x, y)
        neighbour_list = []
        for i in (-1, 0, 1):
            for j in (-1, 0, 1):
                bucket_to_check = my_bucket + i + j*self.width
                if bucket_to_check >= 0 and bucket_to_check <
len(self.buckets):
                    neighbour_list.extend(self.buckets[bucket_to_check])

```

```
#check water as well

def add_element(self, element):
    bucket = self.__get_hash(self, element.x, element.y)
    self.buckets[bucket].append(element)
    element.bucket = bucket

def update_element(self, element):
    current_b = self.__get_hash(self, element.x, element.y)
    prev_b = element.bucket
    if current_b != prev_b:
        if prev_b >= 0:
            self.buckets[prev_b].remove(element)
        self.buckets[current_b].append(element)
        element.bucket = current_b

def tick(self, element_list):
    for element in element_list:
        self.update_element(self, element)
```

17/02:

The condition for checking a bucket did not account for buckets wrapping around horizontally at the ends and beginnings of rows; this would not affect program logic as there is a separate distance check anyways; however for cleanliness this was amended to:

```
if bucket_to_check >= 0 and bucket_to_check < len(self.buckets) and not (i==-1
and my_bucket % self.width == 0) and not (i==1 and (my_bucket+1) % self.width
== 0):
```

If I ever wanted to make the world wrap around at edges, I would need to adjust the logic here.

I will now enter the final stage of development for phase 1, namely the world generation, including water, terrain and plants (see: phase 1 - Development)

(Fixed number of buckets generated for spatial hash)

18/02 - 22/02:

I reworked and developed the anima general tick methods and the many helper methods, some of which are logically private but which I have given public identifiers as they are useful for result testing (nonetheless python does not logically distinguish the two in syntax). I finally was able to simulate behaviour for the animas, though there were many bugs which were required to be fixed. I isolated methods and attributes testing each individually and adjusting until behaviour was expected. There is a bug which I fixed by moving set_action() from within a method which was run every tick (here set_action was unconditional) to another method also run every tick but where set_action was fixed under some condition. This made the effect achieved in full correctness, where previously it was not

ever accessed at all; this remains the only bug fix where I do not know how the previous state, while I know how it was incorrect, produced the result it did, which was counter to my intuition, producing the negative result rather than an abundance of the positive.

The anima class constructor is now as follows:

```
def __init__(self, id, naming_mode, genome, realm, spawnx, spawny):

    super().__init__(realm, spawnx, spawny)

    self.id = id

    self.angle = random.uniform(0, 2 * PI)
    self.roam_angle = random.uniform(0, 2 * PI)
    self.dx = 0
    self.dy = 0

    self.age = 0
    self.status = [25,25,0,0,-999] #hunger, thirst, sleepiness, damage,
    mating drive (genetic) cannot be >80
    self.target = None
    self.action = [self.do_nothing,self.do_nothing]
    self.rejectedList = []
    self.eatenList = []
    self.offspring = []
    self.parents = []
    self.goals = []
    self.mode = 1 # 0 sleep 1 roam 2 approach 3 flee

    self.locked = False
    self.fleeing = False
    self.hunting = False
    self.post_hunt = False

    self.conditions = []
    self.responses = []

    self.asleep = False
    self.sleep_debt = 0
    self.dream = self.do_nothing #***

    self.genome = genome
    self.diet = genome[0]
    self.speed = genome[1][0]

    self.terrain_factor_sum = genome[1][1][0] * GRASS_BIAS +
    genome[1][1][1] * WATER_BIAS + genome[1][1][2]
    self.grass_factor = genome[1][1][0]/self.terrain_factor_sum *
    TERRAIN_FACTOR_WEIGHTED_SUM
```



```

        self.amphibian_factor = genome[1][1][1]/self.terrain_factor_sum *
TERRAIN_FACTOR_WEIGHTED_SUM
        self.ice_factor = genome[1][1][2]/self.terrain_factor_sum *
TERRAIN_FACTOR_WEIGHTED_SUM

        self.turnspeed = genome[2]
        self.vision = genome[3]
        self.strength = genome[4]
        self.toughness = genome[5]
        self.birth_size = self.size()

        self.mating_drive = genome[6]
        self.boost_factor = genome[7]
        self.injury_threshold = genome[8]
        self.maturity_age = genome[9]
        self.senescence_age = (SENESCENCE_AGE + self.maturity_age) *
sqrt(self.size())
        self.refractory = genome[10]
        self.extra_traits = genome[11]
        self.reach = 2.5

```

Attributes extraneous to the constructor include:

- self.cells_to_check

Mating has been implemented. The offspring of two parents will have traits which are some combination of its parents'; depending on the combination mode this can be the average offset by some random mutation factor, or take one of its parents offset by the same, for each trait. For discrete traits a random parent is picked, but there is a chance both parents' traits are inherited simultaneously where possible, or neither. Mutation introduces a trait value which can be new from both parents.

```

def mix_genomes(self, mate):
    new_genome = []
    for chromosome in range(len(self.genome)):
        if isinstance(self.genome[chromosome], list):
            new_genome.append([])
            for genotype in range(len(self.genome[chromosome])):
#incompatible with list diet and list speeds

new_genome[chromosome].append(mix_genes(type(self.genome[chromosome][gene]), se
lf.genome[chromosome][gene], mate.genome[chromosome][gene]))
        else:
            new_genome.append(mix_genes(type(self.genome[chromosome]),
self.genome[chromosome], mate.genome[chromosome]))
    return new_genome

```

```
def mix_genes(gene_type, A, B):
    if gene_type == tuple:
        new_gene = ()
        if random.uniform(0,1) < 0.1 and HYBRID_DIETS_ON == True:
            #omnivore
            for item in A:
                if item not in new_gene and random.uniform(0,1) < 0.95:
                    new_gene = new_gene + (item)
            for item in B:
                if item not in new_gene and random.uniform(0,1) < 0.95:
                    new_gene = new_gene + (item)
        elif random.uniform(0,1) < 0.5:
            for item in A:
                if item not in new_gene and random.uniform(0,1) < 0.95:
                    new_gene = new_gene + (item)
        else:
            for item in B:
                if item not in new_gene and random.uniform(0,1) < 0.95:
                    new_gene = new_gene + (item)
        if new_gene == None or random.uniform(0,1) < 0.05:
            new_gene = new_gene + (random.sample(food_list))
        return new_gene

    elif gene_type == int or gene_type == float:
        new_gene = (A+B)/2 * (1 + random.uniform(-2,2) * MUTATION_FACTOR)
        new_gene = max(new_gene, 0.1)
        new_gene = min(new_gene, 100)
        #should i set limits for each gene specifically? would require overhaul of this and use of a dictionary
        return new_gene
```

Diet was made a tuple so that distinct logic from the speed chromosome (which was also a list) can be used. The genome-mixing is hard-coded in this way.

An anima's actions are now implemented via an action dictionary with values being lists of size two, holding the thinking element and the acting element (both methods of the anima; "self.do_nothing" is the placeholder here and takes a dummy argument set to None by default)

I will now proceed to attempt to get a working simulation; there will be many bugs requiring addressing.

I worked through a few runtime errors; these were often the result of incorrect syntax in a module such as using curly brackets instead of square and vice versa, or trying to use floats as list indices. These were fixed by correcting the syntax (ie using the right bracket or using int() to truncate floats and convert to an integer type). Runtime errors also included passing self as the first argument in a method call, and trying to access a list element with index out of range due to incorrectly positioned code, or trying to use 'Realm' methods on a None type due to animas being initially declared without a realm; the last issue was resolved by changing the anima constructor such that no methods were dependent on the realm attribute; animas created by birth are assigned a realm by argument; I was initially hesitant to do this as I would rather animas be able to interact in a vacuum, but a test setting realm would be similar to this anyways, and if animas have a position then they might as well have a realm corresponding to this; animas to be passed in from a primordial list have their naming mode passed in manually by argument. Finally there were also logical errors which were made apparent due to my many lines of testing output lines; these included animas never mating because their mating drive was set to 0 on the first tick due to the goal setting method using and mutating a reference to anima.status instead of a temporary clone, and thus mutating the elements within the anima method itself (in this case, setting mating drive to 0 or -999). Here I had to amend the code to use list slicing, ie *needs = anima.status => needs = anima.status[:]* (<= 23/02)

Some alterations made from phase 1:
-> Floras added to the spatial hash (though they are not updated once added, being stationary)

-> Bugs fixed (runtime, syntax, logical)

23/02:

```
Oudouxidde [['apple'], [8, 0.5, 0.3], 28, 10, 9, 9, 26] was born!
['name', 'genotype']
name
Vouuwe [['apple'], [3, 0.5, 0.3], 30, 8, 13, 8, 24] was born!
```

```

[1/]
A creature died.
21
[21]
A creature died.
1
[1]
A creature died.
22
[22]
A creature died.
20
[20]
A creature died.
Eithax [[None], [5.010795027108198, 0.4847402427709375, 0.2831888941674094], 27.274148405601835, 5.1838245001344125, 11.791626033640574, 8.45659645679183, 24.984824176460343] drank...
Eeplui [['apple'], [5, 0.5, 0.3], 21, 7, 13, 9, 28] drank...
Nhoonihh [[None], [4.826378170741825, 0.45741803446705653, 0.2737437618463725], 19.23306933925955, 5.608550001655568, 8.617021927164975, 9.347196726337879, 21.016970057563643] drank...
Shhh! Siv [[None], [3.872434194949136, 0.5150223970775416, 0.276077756918187], 18.893050131130337, 8.65690630800128, 8.299305708008047, 9.753109694031453, 26.23436572671666] is sleeping...
Hyruzix [[None], [4.702995383631002, 0.44513157493233346, 0.28174584267604763], 18.274913551691917, 5.874049795046467, 10.448103746026991, 8.469249635574068, 24.67164149505913] drank...
16
[16]
A creature died.
Groupythumee [[None], [3.102362084661137, 0.5468570650177769, 0.30754870658148026], 23.4503802433792327, 7.700460046498162, 8.291221976529082, 7.739211734959902, 23.655008001694906] drank...
Oiyh [[None], [4.15689535075318, 0.47024670142222863, 0.2564458022644579], 19.712592367149497, 7.114369408924276, 10.14896387921058, 8.47263824066672, 28.440914023743304] drank...
Eezoosucta [[None], [5.386715887554622, 0.4977919415059547, 0.32129315312685985], 19.65813108115906, 8.131748343203927, 8.41664130368123, 8.949809185367167, 23.967937862538072] drank...
Uklytittui [['apple'], [3, 0.5, 0.3], 20, 6, 6, 6, 20] drank...
Owaoth [['apple'], [3, 0.5, 0.3], 30, 6, 12, 13, 23] drank...
Isoubu [[None], [4.840779335487875, 0.5156115429762002, 0.2791362843138228], 21.914382765992542, 7.282176068971076, 9.313714977493717, 9.725844995587927, 26.71500595666845] drank...
20
[20]
A creature died.
Eizaj [['apple'], [3, 0.5, 0.3], 19, 7, 9, 6, 27] drank...
Iradto [[None], [5.593745665940105, 0.4694621671657051, 0.3136967130635793], 18.46581670159533, 9.225255061075119, 7.69297028892752, 8.337778406034623, 26.368758681893716] drank...
Usevapro [[None], [6.0450942039020036, 0.4777022267043210, 0.2762452218101013], 22.724925561343273, 7.892121144603828, 9.547272042832342, 9.735381766009182, 24.41814007058101] drank...
Abunnizeij [[None], [4.989047868767151, 0.4806186359006053, 0.28930962028044676], 22.180030983864793, 6.25501218177629, 12.97539382144254, 10.754390691970126, 23.17647079257886] drank...
Mawwee [['apple'], [4, 0.5, 0.3], 16, 7, 11, 5, 27] drank...
Ofathed [[None], [3.7880110547987326, 0.49359346060520073, 0.2827519609245733], 18.174111806642728, 6.347085962259425, 10.16958853066326, 5.438632821732932, 28.31137903949326] drank...
Eibibill [[None], [5.8943539293673775, 0.53677674789584418, 0.30537884920772546], 19.003483254468144, 8.963780881175536, 9.593641457437128, 8.162158607114893, 22.220869285778614] drank...
Uxateev [['apple'], [6, 0.5, 0.3], 21, 5, 15, 10, 20] drank...
Eithax [[None], [5.010795027108198, 0.4847402427709375, 0.2831888941674094], 27.274148405601835, 5.1838245001344125, 11.791626033640574, 8.45659645679183, 24.984824176460343] drank...
Ginurei [[None], [4.535221591075431, 0.48843787764759067, 0.31358747059062664], 20.645812732639868, 6.143501259384825, 12.97666449344742, 11.715365627432933, 21.914510273642133] drank...
24
Nhoonihh [[None], [4.826378170741825, 0.45741803446705653, 0.2737437618463725], 19.23306933925955, 5.608550001655568, 8.617021927164975, 9.347196726337879, 21.016970057563643] drank...
Hyruzix [[None], [4.702995383631002, 0.44513157493233346, 0.28174584267604763], 18.274913551691917, 5.874049795046467, 10.448103746026991, 8.469249635574068, 24.67164149505913] drank...
24
[24]

```

```

Eizaj [['apple'], [3, 0.5, 0.3], 19, 7, 9, 6, 27] drank...
Oonolou [['apple'], [3, 0.5, 0.3], 22, 5, 9, 9, 21] ate apple
Oonolou [['apple'], [3, 0.5, 0.3], 22, 5, 9, 9, 21] drank...
Eizaj [['apple'], [3, 0.5, 0.3], 19, 7, 9, 6, 27] drank...

```

```

2
Shhh! Eizaj [['apple'], [3, 0.5, 0.3], 19, 7, 9, 6, 27] is sleeping...
Shhh! Eizaj [['apple'], [3, 0.5, 0.3], 19, 7, 9, 6, 27] is sleeping...
Eizaj [['apple'], [3, 0.5, 0.3], 19, 7, 9, 6, 27] drank...
Eizaj [['apple'], [3, 0.5, 0.3], 19, 7, 9, 6, 27] drank...
Oonolou [['apple'], [3, 0.5, 0.3], 22, 5, 9, 9, 21] drank...
Oonolou [['apple'], [3, 0.5, 0.3], 22, 5, 9, 9, 21] drank...

```

```

4
[4]
A creature died.
Oonolou [['apple'], [3, 0.5, 0.3], 22, 5, 9, 9, 21] drank...

```

```

1
Oonolou [['apple'], [3, 0.5, 0.3], 22, 5, 9, 9, 21] drank...
Oonolou [['apple'], [3, 0.5, 0.3], 22, 5, 9, 9, 21] drank...
Oonolou [['apple'], [3, 0.5, 0.3], 22, 5, 9, 9, 21] drank...
Oonolou [['apple'], [3, 0.5, 0.3], 22, 5, 9, 9, 21] drank...

```

```

1
Oonolou [['apple'], [3, 0.5, 0.3], 22, 5, 9, 9, 21] drank...
Oonolou [['apple'], [3, 0.5, 0.3], 22, 5, 9, 9, 21] drank...
Oonolou [['apple'], [3, 0.5, 0.3], 22, 5, 9, 9, 21] drank...

```

```

1
Oonolou [['apple'], [3, 0.5, 0.3], 22, 5, 9, 9, 21] drank...
5

```

```

[5]
A creature died.

```

```

0
0
0

```

```

(.venv) PS C:\Users\ethan\OneDrive\Documents\projectsim>

```

```

def main():
    mySim = Simulation()

    test_element_list = []
    #for i in range(20):
    #    test_element_list.append(Flora(i, random.randint(-960, 960) ,
    random.randint(-540, 540), 10, 0.1)) code expired

    for i in range(20):
        test_element_list.append(Anima(i, "name/genome", ["apple"],
[random.randint(2, 8),0.5,0.3], random.randint(15, 30), random.randint(5,10),
random.randint(5,15), random.randint(5,15), random.randint(20,30)], None,
random.randint(0, 99) , random.randint(0, 99)))

    print("Generating realm...")
    testRealm = Realm(nickname.generate_nickname("realm"), 100, 100, 0.2, 0.05,
[["tree", 200], ["bush", 100], ["lily", 20]],test_element_list, "name/genome")
#birth new test realm

    mySim.add_realm(testRealm)

    """for element in testRealm.elements:
        print(dir(element))
    print(mySim.realms[0].elements)
    print(testRealm.elements)
    for element in testRealm.elements[0]:
        print(element.capacity, element.fruitNo, element.x, element.y,
element.fruit, element.id)"""

    #print(testRealm.environment)
    #print(len(testRealm.spatialhash.buckets))
    #print(testRealm.spatialhash.buckets)

    print(testRealm.check_water())
    print(testRealm.check_ice())

    print(testRealm.count_flora())
    print(testRealm.check_grass())
    print(testRealm.environment)
    for batch in range(20):
        for tick in range(25):
            #print(testRealm.time)
            testRealm.tick()
            #print([[anim == anima.target.target]] for anima in
testRealm.elements[1] if anima.target != None])
            #print([[anima.distance_to(anima.target), anima.goals,
anima.target, (anima.vision+anima.speed+anima.target.speed>=

```

```

anima.distance_to(anima.target))] for anima in testRealm.elements[1] if
anima.target != None])
    #print([[anima.round_status(),round(anima.x),      round(anima.y),
round(anima.dx), round(anima.dy), anima.mode, anima.target, anima.goals] for
anima in testRealm.elements[1]]) #anima.status, anima.genome,anima.x, anima.y,
anima.dx, anima.dy, anima.mode, anima.target, anima.goals
    #print([flora.fruitNo for flora in testRealm.elements[0]])
    print(testRealm.count_anima())

return 0

```

After more bug fixes... It finally works! Using test-purpose print lines such as “A creature is born!”, I can verify that creatures now:

- Eat
- Drink
- Sleep
- Mate
- Die

Given enough simulation ticks, the population of creatures will at first expand, then decrease until extinction (with current testRealm input; I will test other inputs to try and attain other population patterns, such as stability within certain bounds, or heavy fluctuation)

I made “nickname.py” to handle the creation and generation of nicknames, based on the main class parameter of “type”, which types as string “anima”, “realm”, etc

Now entities may be assigned a nickname built according to the value of “creature_naming_mode”, which can take “nameless”, “random”, “preset”, etc

If the value is “random”, a nickname is generated using the random nickname generation algorithm I wrote, after identifying certain orthographic/phonological patterns; the nicknames are designed to be readable and unique, with phonological variation.

The nickname is generated based on ‘batches’- each batch is a combination of a consonant phoneme and vowel phoneme; consonant phonemes have a chance to be a consonant double and if the first generated consonant is a ‘leader’ such as ‘s’, ‘g’ or ‘k’ there is a chance that a ‘follower’ such as ‘r’ or ‘l’ is appended, and the follower list is expanded to include, for instance, ‘n’, provided the phoneme is not of the first batch (because ‘apnar’ is orthographically sensible, while ‘pnar’ by itself is not). There are some other considerations as well within the phoneme. Each batch will begin with the same phoneme type as each other batch in order to construct sensible words; this is because generally vowel and consonant phonemes work well in a word when alternating; this is simply the base of our phonetics (‘cat’ reads better than ‘cta’). I greatly reduce the number of weird adjacent letter combinations by alternating vowel and consonant phonemes. The final batch is 50% likely to be one phoneme only, and 50% likely to have both; this simulates a uniform chance of length of each word in phonemes rather than batches, for more fluidity. Titles and endings are morphemic units corresponding to the entity in question (eg. Realms may have the ending ‘topia’). Endings are given as a random element within a list nested within a greater list which is the value for the key (‘type’) in the ending dictionary; the nested list corresponds to whether the last letter of the last batch was a vowel or consonant; there are thus two nested lists and ‘topia’ in the vowel-correspondent list will match to

'atopia' in the consonant equivalent. The final word is the combination of a title (at some probability), all vowel-consonant batches, and an ending (at some probability).

24/02:

I have finished with nickname.py for now. It can generate names (prev hzzedm, gssn, ew, ew, no, moozzu, xee, now first three Meipyda, Wuibiwee and Cunaot) for anima which I think are rather good. First realm name was Einewweirattov's World.

I was unsatisfied with the first batch of names as two letter names are too short, and I was unsure why some names had only consonants. However I adjusted the random range of letter batches from (1,4) to (2,5) and it produced a satisfactory effect. 5 batch names were too long though so I pushed it down to (2,4).

Adjusted the nickname generation algorithm and tested simulation outcomes based on different parameters.

Anima.end() was not working for some reason, so I moved animas to the realm's graveyard list upon death so they were no longer updated (I was planning on doing this anyway).

Animas tend to last a few hundred cycles, with one or two surviving far beyond others; there is a long tail in the distribution of death times.

Notes:

Creature diet: "a creature ate another!"

Family cannibalism boolean toggle

More boolean toggles

27/02:

Problem & Solution: newborn animas had a diet value of a list with a single Nonetype element. This was because diets were lists, whereas the code I had used to account for diet selected only for tuples; the correction here was simply to make all instances of diets tuples. However it does raise an underlying concern of relying on data types to map to anima attributes. It should be a manageable dependency, but without use of a dictionary or generalised plan I could make similar mistakes or be forced to use datatypes I would not want to. The alternative would be to have a group of functions, each handling one or a group of attributes, and I may shift to this if my current model proves untenable later; however this will be unlikely.

01/03:

Problem & Solution: the fixed mating drive is not well integrated into the rest of the system. It leads to two perverse consequences:

1. Animas' principal desire upon being born is to mate (this makes most of the selection pointless, as animas are able to pass on their genes without having to first survive the challenges of the environment)
2. Animas sleep only when their sleepiness is at a certain threshold, and after sleep their sleepiness will most likely still be high enough to prevent mating drive from being selected using the setGoals() function. Even if it were not, hunger and thirst would most likely be, and by the time food and water are found, sleepiness almost certainly will prevent mating.

Consequences 1 and 2 combine to mean that instead of surviving in order to reproduce, animas reproduce, and then survive to no consequence.

There are a few solutions, which address either 1, 2 or both:

- Animas are born with certain levels of hunger, thirst and/or sleepiness (will require setting in constructor according to fixed or constant values, or alternatively set to values passed in as arguments which would allow for more in-program flexibility)
- Animas sleep until their sleepiness reaches 0
- Hunger and thirst increment by less or not at all when the anima is sleeping (not at all would perhaps be too strong for large animas which do not have to fear being eaten)
- Mating drive behaves like other needs, increasing from zero and decreasing after mating (this could mean tiredness does not need to be incremented so much)
- Animas only gain mating drive once they have reached a certain age (modelling childhood), or they gain it slowly until the genetic limit is reached

I will adjust and add features to solve the issue as the project develops (tackled on 06/03)

Currently animas are too vulnerable to predators when sleeping. In this most primitive implementation of predators, animas are treated the same as fruits, eaten instantaneously once in range as long as they are weak enough compared to the predator. Instead, I will implement a damage and health system which will simulate combat; then, sleeping creates advantage for the attack as they get a free attack, but the prey still has the chance to flee or retaliate. Additionally, I will give animas (very limited) vision when asleep, based on a new genotype of sleep sensitivity so that they can still anticipate attacks. The tradeoff for sleep sensitivity will be the rate of sleepiness decrease through sleep required, as the anima is in a 'lighter' sleep. I am still considering how to implement the various tradeoffs for positive traits such as speed and vision. Rate of sleepiness decrease is a simple tradeoff, as well as rate of sleepiness increase while awake.

02/03:

Problem & Solution: the error of animas having targets incredibly far away returned on home version, but this resolved itself somehow when I used school version.

Problem & Solution: animas having None as diet. This confused me as I specifically added safeguards so that diet was always at least (x). I investigated using a test block:

```
if self.diet == None:
    print("No diet")
    print(genotype)
```

I knew all animas I added via the primordial list had tuple diets and tuple begot tuple when genes were mixed. However it was in runtime and coinciding with a new birth that the error was flagged (the error in specific was during food checking where None types could not have their index accessed).

And on researching tuple definition I discovered that single element tuples require a comma after the element, ie ("A") + ("B") gives the string "AB". Therefore to fix the issue I needed to add a comma after all cases of single element tuples, of which there were a fair few. As an example:


```

if random.uniform(0,1) < 0.1 and HYBRID_DIETS_ON == True:
    for item in A:
        if item not in new_gene and random.uniform(0,1) < 0.95:
            new_gene = new_gene + (item,)
    for item in B:
        if item not in new_gene and random.uniform(0,1) < 0.95:
            new_gene = new_gene + (item,)

```

This resolved the issue completely.

Problem & Solution: I noticed when reading my `mix_genes()` code in order to investigate the previous problem that the chance for animas mutating to inherit neither parents' diet items and so to develop a random one (which could also coincide with their parents') was in the best case a 1 in 400. This seemed too low, so I changed the code to only run a 1 in 20 once instead of twice, and combining the parental branches from two 50% branches into one branch which selected a parent at random. Here is the result:

```

elif random.uniform(0,1) < 0.95:
    for item in random.choice([A, B]):
        if item not in new_gene and random.uniform(0,1) < 0.95:
            new_gene = new_gene + (item,)
if new_gene == () or random.uniform(0,1) < DIET_MUTATION_PLUS_FACTOR:
    new_gene = new_gene + (random.choice(food_list),)

```

Mutations are now more likely.

Problem & Solution: Anima birth message was displaying parent name rather than newborn; this was easily noticeable when animas were apparently being born after already eating or drinking. Fixed by simply changing object in f string from `self` to `newborn`.

0/03

Problem & Solution: I have finally achieved **the first case of predation!** there was an error thrown when an anima attempted to eat meat, because animas have no `fruitNo` attribute. So I created new attributes and methods to handle animas being eaten and streamlined animas and floras in this case. I decided that animas would leave a carcass on death and this could be eaten by meat-eating animas. So the graveyard's role was now to hold carcasses which do not require iteration for decision and action and I made a new list 'ghostyard' which fulfilled the graveyard's old role; animas enter it when their corpse loses all chunks. Animas are removed from the spatial hash now only once moved to the ghostyard. Meat eaters behave differently from plant eaters in that a carcass can be eaten from multiple times by each predator (this is so that animas who expend energy to hunt and kill can be rewarded maximally). I may later tweak this so scavengers can only take one chunk each, on the same principle, and using a kill list for the anima (similar to the `eatenList` but in reverse), so that the condition for eating is logically (in `killList` or not in `eatenList`). Of course other methods required adjustment as well, for example `remove_elements` being renamed to 'bury_elements'. I chose not to have a `remove_elements` at all for the realm class, as animas never truly are removed. The method converting to a ghost is called 'ascend_element', and there is a corresponding plural method.

I debate on whether the scavenger/hunter split does not show significant difference from a general fast and small/slow and large trend.

06/03

Feature- Hunting: If an anima's food target is alive and in range, it will be attacked instead of eaten, by some function which takes the difference in predator strength and prey toughness as a parameter. Currently anima's treat meat equally alive or dead, though it might make more sense for animas to target corpses which will not run away and will be safer to target. Fleeing animas now also fight back as their tick action. The next steps will be to implement corpse decay and prey fight or flight responses.

Added corpse decay: Anima food chunks decay according to the realm's constant rate of decay (can be thought of as temperature and salinity). Like before, when a corpse loses all its chunks the anima ascends to the ghostyard.

Added fight or flight: Animas can choose to either fight back against an attacker or to flee further away on each tick, gaining a speed boost. The advantage of fighting is that animas become slower the more injured they are, and once past a genetic injury threshold, they stop attacking and disengage in combat (so predators switch to roaming, and prey flee).

```
Gruinidyn ["meat"], ["5.0", "0.5", "0.3"], "22.0", "10.0", "7.0", "10.0", "30.0", "0.54", "72.0", "84.0"] struck Oufufeis ["apple"], ["2.0", "0.5", "0.3"], "20.0", "9.0", "8.0", "12.0", "22.0", "0.96", "78.0", "75.0"]
Webbaz ["meat"], ["6.0", "0.5", "0.3"], "16.0", "8.0", "9.0", "6.0", "20.0", "0.1", "40.0", "106.0"] struck Glugokookh ["berry"], ["3.0", "0.5", "0.3"], "25.0", "7.0", "12.0", "14.0", "22.0", "0.38", "55.0", "64.0"]
Nubaf ["meat"], ["7.0", "0.5", "0.3"], "22.0", "9.0", "8.0", "15.0", "27.0", "0.7", "60.0", "83.0"] struck Petwa ["apple"], ["6.0", "0.5", "0.3"], "29.0", "10.0", "8.0", "15.0", "25.0", "0.90", "97.0", "60.0"]
Uthuk ["apple"], ["3.0", "0.5", "0.3"], "27.0", "10.0", "6.0", "10.0", "28.0", "0.95", "40.0", "119.0"] ate apple
Zitha ["berry"], ["3.0", "0.5", "0.3"], "22.0", "10.0", "6.0", "10.0", "20.0", "0.07", "35.0", "70.0"] ate berry
Webbaz ["meat"], ["6.0", "0.5", "0.3"], "16.0", "8.0", "9.0", "6.0", "20.0", "0.1", "40.0", "106.0"] struck Glugokookh ["berry"], ["3.0", "0.5", "0.3"], "25.0", "7.0", "12.0", "14.0", "22.0", "0.38", "55.0", "64.0"]
Nubaf ["meat"], ["7.0", "0.5", "0.3"], "22.0", "9.0", "8.0", "15.0", "27.0", "0.7", "60.0", "83.0"] struck Petwa ["apple"], ["6.0", "0.5", "0.3"], "29.0", "10.0", "8.0", "15.0", "25.0", "0.90", "97.0", "60.0"]
Aodyz ["berry"], ["5.0", "0.5", "0.3"], "21.0", "7.0", "6.0", "9.0", "23.0", "0.46", "56.0", "104.0"] ate berry
Gruinidyn ["meat"], ["5.0", "0.5", "0.3"], "22.0", "10.0", "7.0", "10.0", "30.0", "0.54", "72.0", "84.0"] struck Oufufeis ["apple"], ["2.0", "0.5", "0.3"], "20.0", "9.0", "8.0", "12.0", "22.0", "0.96", "78.0", "75.0"]
Webbaz ["meat"], ["6.0", "0.5", "0.3"], "16.0", "8.0", "9.0", "6.0", "20.0", "0.1", "40.0", "106.0"] struck Glugokookh ["berry"], ["3.0", "0.5", "0.3"], "25.0", "7.0", "12.0", "14.0", "22.0", "0.38", "55.0", "64.0"]
Nubaf ["meat"], ["7.0", "0.5", "0.3"], "22.0", "9.0", "8.0", "15.0", "27.0", "0.7", "60.0", "83.0"] struck Petwa ["apple"], ["6.0", "0.5", "0.3"], "29.0", "10.0", "8.0", "15.0", "25.0", "0.90", "97.0", "60.0"]
Ythuk ["apple", "berry"], ["6.1", "0.54", "0.29"], "23.5", "9.1", "11.9", "10.7", "19.9", "0.11", "100.0", "75.5"] ate apple
Iweefweecu ["berry"], ["7.2", "0.52", "0.33"], "25.1", "9.5", "15.4", "11.3", "29.7", "0.57", "68.0", "41.7"] was born!
Ajezouco ["berry"], ["7.0", "0.5", "0.3"], "28.0", "6.0", "9.0", "12.0", "30.0", "0.05", "81.0", "80.0"] ate berry
Pad ["apple"], ["5.0", "0.5", "0.3"], "26.0", "10.0", "7.0", "5.0", "22.0", "0.31", "57.0", "107.0"] ate apple
Zitha ["berry"], ["3.0", "0.5", "0.3"], "22.0", "10.0", "6.0", "10.0", "20.0", "0.07", "35.0", "70.0"] ate berry
Gruinidyn ["meat"], ["5.0", "0.5", "0.3"], "22.0", "10.0", "7.0", "10.0", "30.0", "0.54", "72.0", "84.0"] struck Oufufeis ["apple"], ["2.0", "0.5", "0.3"], "20.0", "9.0", "8.0", "12.0", "22.0", "0.96", "78.0", "75.0"]
Vicli ["meat"], ["4.0", "0.5", "0.3"], "21.0", "9.0", "14.0", "8.0", "21.0", "0.91", "88.0", "85.0"] struck Ihuiz ["apple"], ["7.0", "0.5", "0.3"], "30.0", "10.0", "8.0", "7.0", "24.0", "0.57", "42.0", "53.0"]
Webbaz ["meat"], ["6.0", "0.5", "0.3"], "16.0", "8.0", "9.0", "6.0", "20.0", "0.1", "40.0", "106.0"] struck Glugokookh ["berry"], ["3.0", "0.5", "0.3"], "25.0", "7.0", "12.0", "14.0", "22.0", "0.38", "55.0", "64.0"]
Nubaf ["meat"], ["7.0", "0.5", "0.3"], "22.0", "9.0", "8.0", "15.0", "27.0", "0.7", "60.0", "83.0"] struck Petwa ["apple"], ["6.0", "0.5", "0.3"], "29.0", "10.0", "8.0", "15.0", "25.0", "0.90", "97.0", "60.0"]
Aodyz ["berry"], ["5.0", "0.5", "0.3"], "21.0", "7.0", "6.0", "9.0", "23.0", "0.46", "56.0", "104.0"] ate berry
Gruinidyn ["meat"], ["5.0", "0.5", "0.3"], "22.0", "10.0", "7.0", "10.0", "30.0", "0.54", "72.0", "84.0"] struck Oufufeis ["apple"], ["2.0", "0.5", "0.3"], "20.0", "9.0", "8.0", "12.0", "22.0", "0.96", "78.0", "75.0"]
Vicli ["meat"], ["4.0", "0.5", "0.3"], "21.0", "9.0", "14.0", "8.0", "21.0", "0.91", "88.0", "85.0"] struck Ihuiz ["apple"], ["7.0", "0.5", "0.3"], "30.0", "10.0", "8.0", "7.0", "24.0", "0.57", "42.0", "53.0"]
Webbaz ["meat"], ["6.0", "0.5", "0.3"], "16.0", "8.0", "9.0", "6.0", "20.0", "0.1", "40.0", "106.0"] struck Glugokookh ["berry"], ["3.0", "0.5", "0.3"], "25.0", "7.0", "12.0", "14.0", "22.0", "0.38", "55.0", "64.0"]
Nubaf ["meat"], ["7.0", "0.5", "0.3"], "22.0", "9.0", "8.0", "15.0", "27.0", "0.7", "60.0", "83.0"] struck Petwa ["apple"], ["6.0", "0.5", "0.3"], "29.0", "10.0", "8.0", "15.0", "25.0", "0.90", "97.0", "60.0"]
Ythuk ["apple", "berry"], ["6.1", "0.54", "0.29"], "23.5", "9.1", "11.9", "10.7", "19.9", "0.11", "100.0", "75.5"] ate apple
Ihuiz ["apple"], ["7.0", "0.5", "0.3"], "30.0", "10.0", "8.0", "7.0", "24.0", "0.57", "42.0", "53.0"] died. :(
```

Many bugs fixed. One example – I added print lines for each defence action and noticed they never executed in the terminal. I then realised that there was faulty logic preventing the defence decision function from ever executing. I traced the error backwards line by line until I reached the source: the statement setting 'fleeing' to false was placed after the function toggling 'fleeing' conditionally, instead of before, which meant 'fleeing' was always false. Simply swapping the positions of the two statements meant that 'fleeing' was conditionally true after the block, and the defence statements could execute.

Another example of a bug was that creatures could feel threatened by already dead predators, entering flee mode and triggering fight-or-flight actions where they would boost away from or attack the corpse. I amended the condition in the `check_for_danger()` function to include an anima's status of living. This is one of many examples where new features and code mean I have to revisit old code which the new features are relevant to; these maintenance requirements reward attention to detail and full knowledge and familiarity of the code I have written, which I believe I possess.

One more example of a bug, another logical error, was that the damage dealt by creatures had neither the distribution nor was within the bounds of what I expected when I wrote the damage algorithm.

The original algorithm was [VER-LOST**]

I adjusted it to:

```

damage = max(1, DAMAGE_FACTOR * (1+random.uniform(-
2*RANDOM_DAMAGE_FACTOR,2*RANDOM_DAMAGE_FACTOR)) * (self.strength -
anima.toughness))
if anima.is_injured():
    damage = damage * 1.1 + 2

```

This new algorithm was more in line with how I wanted the creatures to be balanced.

Added ageing and growth. As part of addressing the problem of reproduction outlined earlier, I added an ageing system where animas only developed sexual desire (which becomes a fixed value, as before) upon reaching a genetically determined maturity age. There is of course a tradeoff, which for now is that while immature, animas increase in strength and toughness each tick, as if growing up. These increases do not contribute to the negative strength/toughness tradeoff of increased food demand. Once mature, animas no longer gain the essentially ‘free’ strength and toughness growth, so a high maturity age means a more fit adult, but a lower maturity age means the creature does not have to survive for as long before it can start reproducing.

Food rework. Food reworked so that larger creatures require more food, and food values also adjusted heavily. I wanted to fit this to Kleiber’s Law for more simulation realism but that would require more of an overhaul of how creatures work, so may come down the line. In fact, since animas only take one fruit from each flora they reach, this may have been a misstep, and I may re-evaluate with testing (larger animas should also take more from each plant which would balance out with requiring more; the adjustment might not have made sense on this account).

Added ‘todisplay.py’. Text displayed to the terminal will now be cleaned and formatted through this. Nickname generating function also makes use of it to ensure correct rou

08/03

Today’s to-do:

- Resolve sleep vs mating drive conflict
- Simulate evolution by natural selection in a more controlled environment (meaning creatures must not dwindle and go extinct until at least a few generations)
- Simulate predation naturally arising from a population of herbivores
- Store population traits and trends in a complex datatype which is updated each tick

```

def is_threatened_by(self, threat):
    return isinstance(threat, Anima) and (threat.toughness > self.strength
or threat.strength > self.toughness) and "meat" in threat.diet and not
self.is_family(threat)

```

```

def is_threat_to(self, other):
    return isinstance(other, Anima) and other.is_threatened_by(self)

```

```

def __check_for_mates(self):
    nearest_mate = None
    nearest_dist = self.vision + 1

    for element in self.realm.spatialhash.get_neighbours(self.x, self.y):

```

```

        if not element in self.rejectedList and isinstance(element, Anima)
and not element == self:          #prevent children and parents?
            dist = self.distance_to(element)
            if min(dist, nearest_dist, self.vision) == dist:    #vision for
emergency clamping
                if 4 in element.goals and not self.is_threat_to(element):
#predator cannot mate with its prey
                    nearest_dist = dist
                    nearest_mate = element
                else:
                    self.rejectedList.append(element)
            return nearest_mate

def __check_for_danger(self):
    nearest_danger = None
    nearest_dist = self.vision + 1

    for threat in self.realm.spatialhash.get_neighbours(self.x, self.y):
        if isinstance(threat, Anima):
            dist = self.distance_to(threat)
            if min(dist, nearest_dist, self.vision) == dist:    #vision for
emergency clamping
                if self.is_threatened_by(threat):
                    nearest_dist = dist
                    nearest_danger = threat
                    self.fleeing = True
    return nearest_danger

```

##Predators should also only attack prey if they judge a good chance of winning (ie either strength greater than prey toughness or toughness greater than prey strength or prey is injured). A predator that is injured, or is dead, is not a threat, and will not hunt (but in the case of injury it will hunt if hunger is greater than injury).

So for a predator the condition for hunting should assume the form:

```

((self.is_threat_to(element)      or      element.is_dead())      and      not
self.is_family(element))

```

For the threat method, see the following condition:

And for a prey creature the condition for another anima being a predator threat is logically:

```

(isinstance(threat, Anima) and not threat.is_dead() and not threat.asleep and
(not threat.is_injured() or threat.status[0] > threat.status[3]) and
((threat.strength > self.toughness) or self.is_injured()) and "meat" in
threat.diet and not self.is_family(threat))

```

Problem & Solution: Mutation is currently modelled as an offset given by a random uniform variable. This is less realistic to real biology than modelling it as noise produced by a normal distribution, which would also make the simulation more interesting by creating extreme animas (such extreme animas could potentially speed up evolution or promote speciation- though currently this kind of speciation is not very likely due to there not being a cutoff in genome difference for mating, besides the predator/prey cutoff). I imported numpy and used the `numpy.random.normal()` function to implement this.

NORMAL DISTRIBUTION MUTATION

ROUND INTEGER CHROMOSOMES

For greater accuracy, I replaced 'genotype' in the program with 'genome', and 'chromosome' with 'genotype'. This does not affect code logic.

Now that the simulation works to a degree with the more limited test realm I have been using, I wrote a new test realm with greater variety in the flora species, and anima diets to match. The new realm is 200x200 instead of 100x100, so I also greatly increased the numbers of each element. I also added a new flora 'wildgrass' which is abundant by provides very little food and exhausts quickly.

Here is part of the terminal output:

```

Onipaaxy lily 6.0 15.0 ate lily
Vumouzaod lavender 9.0 6.0 ate lavender
Xohoocouppou lavender 7.0 5.0 ate lavender
Vebabbi melon 9.0 13.0 struck Ififtuw meat 15.0 8.0 and dealt 3.811534326367288 damage
Mrs Ugruwaog wildgrass 8.0 15.0 ate wildgrass
Mewoo lily 8.0 15.0 ate lily
Blue Uigy wildgrass 12.0 9.0 ate wildgrass
Eexoodo apple 11.0 14.0 ate apple
Mr Oubreezi berry 10.0 12.0 ate berry
Oxowaolum melon 15.0 12.0 ate melon
Iduhhag apple 12.0 7.0 ate apple
Tujjounai lily 12.0 9.0 ate lily
Ylibb lily 9.0 8.0 ate lily
Ooxuh wildgrass 15.0 7.0 is boosting to get away
Cippujj apple 13.0 11.0 ate apple
Ijathi wildgrass 9.0 12.0 ate wildgrass
Ififtuw meat 15.0 8.0 struck Darao lily 12.0 6.0 and dealt 57.96038421039765 damage
Ewaomyb wildgrass 15.0 15.0 ate wildgrass
Douxai lily 8.0 6.0 ate lily
Agexui lily 7.0 6.0 ate lily
Elei lavender 14.0 10.0 struck Laf meat 12.0 13.0 and dealt 3.4573885343046173 damage
Biceemm lily 13.0 7.0 struck Rawoucoog meat 15.0 7.0 and dealt 22.552236220294937 damage
Haokagsuhee lily 7.0 14.0 ate lily
Crouffuz lily 9.0 9.0 ate lily
Afuizzu apple 6.0 13.0 ate apple
Eedexathi wildgrass 10.0 14.0 ate wildgrass
Ounuilee berry 9.0 11.0 ate berry
Tuku Enui apple 5.0 8.0 is boosting to get away
Saok melon 15.0 12.0 struck Uiwovvyz meat 14.0 6.0 and dealt 39.832592976828444 damage
Aojothe berry 14.0 8.0 ate berry
Lil' Lug berry 15.0 9.0 ate berry
Obouxexuubb lily 11.0 13.0 ate lily
Blue Bux melon 7.0 9.0 ate melon
Fysseeneeli lily 9.0 12.0 ate lily
Mrs Pwocac apple 12.0 7.0 ate apple
Ourooloonnox apple 14.0 6.0 struck Dupe meat 15.0 6.0 and dealt 35.7081481229107 damage
Adod lavender 8.0 10.0 ate lavender
Blue Miloveecu apple 6.0 13.0 ate apple
Opleevuzaod lavender 13.0 15.0 ate lavender
Etheenoo lavender 11.0 5.0 ate lavender
Aoly lily 14.0 13.0 ate lily
Khen berry 14.0 15.0 ate berry
Muhhaccik lily 6.0 12.0 ate lily
Mr Counoumou meat 13.0 10.0 struck Geftepp meat 6.0 8.0 and dealt 33.534628350276286 damage
Goohanokk berry 12.0 9.0 ate berry
Ojao apple 10.0 10.0 ate apple
Ozeruzzo apple 5.0 9.0 ate apple

```

Problem & Solution: I added a third tick for living animas- tickResponse. There was awkwardness in the timing of anima being woken up, as this needed to happen after all action ticks resolved (otherwise animas would awaken and then sleep, messing with the sleep debt mechanic; this would be messy) - but if it were to happen on the following tick this would require residue flags each tick. So I added responses, which will be a new category of anima methods. During the action tick, response bound methods will be appended to the anima's response list attribute. During the response tick, all responses in the response list will be executed. This allows for all dependencies to resolve cleanly between each stage: decision > action > response. Currently the only response is 'awaken()', and more response methods will be added as necessary.

Problem & Solution: Since I wrote a new realm, the None type diet error has returned. I first traced the error through the todisplay, nickname, and anima files. Then, recognising it as an error that was resolved in the past, I was able to make the same fix of adding the missing trailing comma to the new single element tuple. I also added a raised error to mix_genomes() so that I would have a custom error message should this or a similar bug happen again.

Added senescence: Animas degrade once they reach a certain age. This is true to real life, and keeps populations fresh and selection moving, so old and successful animas do not stifle or dominate competition. To make death an inevitability, senescent animas suffer damage each tick and cannot heal their wounds (simulating age-sickness in animals and their weakened immune and repair systems).

```
if self.age >= self.senescence_age:
    self.update(3, 0.3)

def update(self, need, quantity, sign=1):
    if need == 3 and self.is_senescent():
        return

    self.status[need] += quantity * sign
    if self.status[need] < 0:
        self.status[need] = 0
```

Tomorrow's to-do:

- Resolve sleep vs mating drive conflict
- Simulate evolution by natural selection in a more controlled environment (meaning creatures must not dwindle and go extinct until at least a few generations)
- Simulate predation naturally arising from a population of herbivores
- Store population traits and trends in a complex datatype which is updated each tick

09/03:

Problem & Solution: I noticed in the terminal that animas were being startled awake without a corresponding message showing they had been attacked, when logically the two should always be coupled. After reviewing my code I concluded that this must have been because while a fight action is required for the awoken response to be added to an anima's responses, there was *no process whereby the method was removed*. This means that once added the response would be perpetually awakened each time it attempted sleep. I went back to the terminal and traced each case of the error and found my theory validated; it was the same two animas triggering the message over and over. To fix the issue, responses will now be cleared at the end of each response tick. This was an example of a new feature requiring testing; since I added it quite late yesterday I hadn't tested it much yet.

```
def tickRespond(self):
    for response in self.responses:
        response()
    self.responses = []
```

Refactoring: Changed anima.py to make use of angle.py instead of importing math.py. I considered only importing into angle.py the necessary functions of math.py; however this would mean I would have to change the function names to avoid naming clash, so I didn't do so.

```
from angle import *
from nickname import *
from todisplay import *
import random
from math import sqrt
```

Problem & Solution: While refactoring I realised that my parameters for atan2 were given as (x,y) instead of what it should be: (y,x). This error would have been far more easily detected with simulation UI in place, but as is, the effect of the change may be difficult to measure. If I understand correctly, then previously animas would have been moving at an angle offset equal to twice the distance of the angle from $\pi/4$ (as $y/x = 1/(x/y)$ and $|\text{atan}(1/x)| = \pi/2 - |\text{atan}(x)|$).

Problem & Solution: For a while there have been unaccounted for irregular terminal messages of a seemingly random number followed by the same number in square brackets. I assumed this was a per-tick message and checked all the tick functions but could not find the source. I searched through every file until I found in spatialhash.py the messages which I had used to check if an anima's stored bucket matched up with its entry in the spatial hash, whenever an anima was removed from the list. I double checked the terminal and sure enough these messages always followed anima body decay, which would trigger the removal. I commented out the messages.

```
#print(element.bucket)
#print([b for b in range(self.width * self.height) if element in self.buckets[b]])
```

Advancement: I made some general adjustments and from now on will run simulations with higher tick numbers while I code, since the simulations run quite slowly now and I can't instantly receive full terminal feedback on pressing 'run'. Anima mating drive increases over time now. Changed energy expenses per tick and for boosting and changed per-tick update amounts due to animas dying too easily from thirst compared to other needs.

Problem & Solution: I received a key error from the autopsy function, and realised that an anima must have died from its mating drive exceeding 100. So now there is a class-wide constant hard limit for anima mating drive. At first I added this limit to the increasing function, but for greater safety I moved it to the general update function alongside the recovery limit on senescent animas.

Test result: I ran a 2500-tick simulation with 250 initial animas and at the end there were 60. This is a good sign that the population will not go extinct in measurable timeframes. The next major step will be to run a 10000-tick simulation with the simulation data analysis in place. I have started adding realm death cause totaller.

11/03:

Added simulation data class- realm data class will store statistics and analysis for given realm. Added death causes list which gives how many animas died of each cause.

Test result: Ran 10000-tick simulation with the strange result of no deaths from thirst. More deaths from injury than from hunger. No deaths from either sleepiness or mating drive, which is logically sound.

Added multiple births- There is a chance for animas to spawn twins, triplets, etc. These can be identical (very rare), or non-identical.

```
while random.random() < TWIN_ODDS:
    if random.random() < TWINS_IDENTICAL_ODDS:
        identical_newborn = Anima(self.realm.anima_id(), self.realm.creature_naming_mode, new_genome, self.realm, (self.x+1)%self.realm.width, self.y)
        self.realm.add_element(identical_newborn)
        print(f"{newborn} and {identical_newborn} are identical twins!")
    else:
        new_genome = self.mix_genomes(mate)
        twin_newborn = Anima(self.realm.anima_id(), self.realm.creature_naming_mode, new_genome, self.realm, (self.x+1)%self.realm.width, self.y+1)
        self.realm.add_element(twin_newborn)
        print(f"{newborn} and {twin_newborn} are twins!")
```

10000 tick sim result:


```

34
9975
[['apple', 0], ['berry', 0], ['melon', 0], ['lily', 0], ['lavendar', 0], ['meat', 0], ['wildgrass', 0]]
['apple', 'berry', 'melon', 'lily', 'lavendar', 'meat', 'wildgrass']
None
Ubruhhaot [berry], 13.6 13.7 ate berry
Veetheedoxee [apple], 7.6 11.8 ate apple
Itrozigood [berry], 11.9 10.7 ate berry
Tubbikkibui [apple], 20.1 20.9 ate apple
Uthysbejj [apple], 6.3 10.7 ate apple
Ubruhhaot [berry], 13.6 13.7 ate berry
Tubbikkibui [apple], 20.1 20.9 ate apple
Eeheigajiff [wildgrass], 5.3 12.0 ate wildgrass
Eeheigajiff [wildgrass], 5.3 12.0 ate wildgrass
Vassouda [apple], 8.0 14.1 ate apple
Dapi [berry], 10.3 18.5 ate berry
Waoz [wildgrass], 12.9 6.0 ate wildgrass
Dapi [berry], 10.3 18.5 ate berry
Tuku Oochetho [apple], 14.0 12.9 ate apple
Waoz [wildgrass], 12.9 6.0 ate wildgrass
Egamofith [wildgrass], 9.6 12.6 ate wildgrass
Eekyvvueci [melon], 13.4 15.5 ate melon
Nall [berry], 7.3 7.9 ate berry
Waoz [wildgrass], 12.9 6.0 ate wildgrass
Egamofith [wildgrass], 9.6 12.6 ate wildgrass
Higgo [berry], 15.9 13.8 ate berry
Eekyvvueci [melon], 13.4 15.5 ate melon
Dr Detaz [wildgrass], 5.4 11.1 ate wildgrass
Waoz [wildgrass], 12.9 6.0 ate wildgrass
Eihu [berry], 17.0 19.8 ate berry
Zobodetw [melon], 11.6 13.5 ate melon
Uija [apple], 11.1 11.8 ate apple
Dr Detaz [wildgrass], 5.4 11.1 ate wildgrass
Egamofith [wildgrass], 9.6 12.6 ate wildgrass
Higgo [berry], 15.9 13.8 ate berry
Raaxeithouxee [apple], 14.4 9.6 ate apple
Iry [berry], 7.6 14.7 ate berry
Kodoo [berry], 8.9 10.6 ate berry
Ubazooddi [apple], 5.1 8.9 ate apple
Eihu [berry], 17.0 19.8 ate berry
Heiti [berry], 8.6 10.4 ate berry
Zodeww [berry], 5.5 8.8 ate berry
Abyseelao [berry], 10.8 10.9 ate berry
Ilowaotw [apple], 5.3 12.8 ate apple
Kodoo [berry], 8.9 10.6 ate berry
Oufa [berry], 8.3 9.1 ate berry
Howuiwit [berry], 10.3 11.8 ate berry
Heiti [berry], 8.6 10.4 ate berry
Dapi [berry], 10.3 18.5 ate berry
Ethawoubn [melon], 16.7 14.7 ate melon
Vassouda [apple], 8.0 14.1 ate apple

```


Dapi [berry], 10.3 18.5 ate berry
 Ethaowoubn [melon], 16.7 14.7 ate melon
 Eidubeexi [apple], 5.5 7.8 ate apple
 Tuku Oochetho [apple], 14.0 12.9 ate apple
 Tubbikkibui [apple], 20.1 20.9 ate apple
 Ubruhhaot [berry], 13.6 13.7 ate berry
 Itrozigood [berry], 11.9 10.7 ate berry
 Higgo [berry], 15.9 13.8 ate berry
 Tubbikkibui [apple], 20.1 20.9 ate apple
 Waoz [wildgrass], 12.9 6.0 ate wildgrass
 Ubruhhaot [berry], 13.6 13.7 ate berry
 Veetheedoxee [apple], 7.6 11.8 ate apple
 Itrozigood [berry], 11.9 10.7 ate berry
 Higgo [berry], 15.9 13.8 ate berry
 Raaxeithouxee [apple], 14.4 9.6 ate apple
 Oninyd [melon], 10.5 20.0 ate melon
 Uthysbejj [apple], 6.3 10.7 ate apple
 Nall [berry], 7.3 7.9 ate berry
 Waoz [wildgrass], 12.9 6.0 ate wildgrass
 Eihu [berry], 17.0 19.8 ate berry
 Neiz [apple], 7.5 7.3 ate apple
 Waoz [wildgrass], 12.9 6.0 ate wildgrass
 Eeheigajiff [wildgrass], 5.3 12.0 ate wildgrass
 Dapi [berry], 10.3 18.5 ate berry
 Eihu [berry], 17.0 19.8 ate berry
 Iry [berry], 7.6 14.7 ate berry
 Uija [apple], 11.1 11.8 ate apple
 Dr Detaz [wildgrass], 5.4 11.1 ate wildgrass
 Waoz [wildgrass], 12.9 6.0 ate wildgrass
 Ofwurewwa [apple], 6.8 5.1 ate apple
 Egamofith [wildgrass], 9.6 12.6 ate wildgrass
 Tuku Oochetho [apple], 14.0 12.9 ate apple
 Iry [berry], 7.6 14.7 ate berry
 Mrs Ooled [lily], 10.4 6.9 ate lily
 Dr Detaz [wildgrass], 5.4 11.1 ate wildgrass
 Eeheigajiff [wildgrass], 5.3 12.0 ate wildgrass
 Heiti [berry], 8.6 10.4 ate berry
 Egamofith [wildgrass], 9.6 12.6 ate wildgrass
 Dapi [berry], 10.3 18.5 ate berry
 Zodeww [berry], 5.5 8.8 ate berry
 Vassouda [apple], 8.0 14.1 ate apple
 Heiti [berry], 8.6 10.4 ate berry
 Egamofith [wildgrass], 9.6 12.6 ate wildgrass
 Kodoo [berry], 8.9 10.6 ate berry
 Howuiwit [berry], 10.3 11.8 ate berry
 34
 10000
 [['apple', 0], ['berry', 0], ['melon', 0], ['lily', 0], ['lavendar', 0], ['meat', 0], ['wildgrass', 0]]
 ['apple', 'berry', 'melon', 'lily', 'lavendar', 'meat', 'wildgrass']
 None
 [164|2|0|124|0],

```

[[25|[[berry], |[2.0|0.5|0.3], |22.3|10.3|10.1|10.3|25.0|0.49|63.3|81.1|24.9], ], |[50|[[berry],
|[2.0|0.51|0.31], |23.0|10.6|10.2|10.6|25.7|0.49|64.8|80.6|25.5], ], |[75|[[berry],
|[2.1|0.54|0.33], |24.2|11.2|10.8|11.3|27.0|0.51|66.8|82.0|26.8], ], |[100|[[berry],
|[2.2|0.55|0.33], |24.6|11.2|11.0|11.6|27.5|0.51|67.6|83.1|27.4], ], |[125|[[berry],
|[2.2|0.56|0.33], |24.9|11.4|11.2|11.9|28.1|0.54|68.7|84.1|28.2], ], |[150|[[berry],
|[2.2|0.56|0.34], |25.1|11.4|11.3|12.0|28.3|0.55|68.7|84.2|28.4], ], |[175|[[berry],
|[2.2|0.56|0.34], |25.1|11.3|11.3|12.2|28.5|0.55|68.8|85.6|29.2], ], |[200|[[berry],
|[2.2|0.56|0.33], |24.7|11.0|11.1|12.2|28.3|0.54|67.6|85.4|29.2], ], |[225|[[berry],
|[2.2|0.56|0.34], |24.8|10.8|11.0|12.3|28.5|0.55|67.9|86.4|28.7], ], |[250|[[berry],
|[2.2|0.56|0.34], |24.7|10.8|11.1|12.4|28.7|0.53|69.0|86.2|28.6], ], |[275|[[berry],
|[2.2|0.56|0.33], |25.0|10.7|11.0|12.6|28.8|0.52|69.6|85.9|28.4], ], |[300|[[berry],
|[2.2|0.56|0.34], |24.9|10.6|10.9|12.6|28.6|0.52|69.8|86.4|28.9], ], |[325|[[berry],
|[2.2|0.55|0.34], |24.9|10.6|10.7|12.5|28.4|0.5|68.9|85.7|28.0], ], |[350|[[berry],
|[2.2|0.55|0.34], |25.0|10.8|10.8|12.5|28.6|0.52|69.3|85.9|28.1], ], |[375|[[melon],
|[2.3|0.56|0.35], |25.0|10.9|10.8|12.6|28.8|0.54|69.6|86.7|28.0], ], |[400|[[melon],
|[2.3|0.56|0.34], |25.4|11.1|10.7|12.5|29.1|0.55|71.0|86.5|26.7], ], |[425|[[melon],
|[2.3|0.57|0.34], |25.6|11.2|10.7|12.5|29.6|0.59|71.4|87.0|26.1], ], |[450|[[melon],
|[2.3|0.57|0.34], |25.5|11.2|10.6|12.4|29.6|0.61|71.2|87.0|25.8], ], |[475|[[melon],
|[2.3|0.57|0.35], |25.6|11.4|10.7|12.4|29.8|0.63|71.2|87.4|25.5], ], |[500|[[melon],
|[2.3|0.57|0.35], |25.6|11.4|10.7|12.4|29.8|0.63|71.2|87.4|25.5], ], |[525|[[melon],
|[2.3|0.57|0.35], |25.5|11.4|10.8|12.4|29.9|0.63|71.2|87.1|25.7], ], |[550|[[melon],
|[2.3|0.57|0.35], |25.4|11.4|10.7|12.4|29.9|0.63|71.6|87.2|25.7], ], |[575|[[melon],
|[2.3|0.57|0.35], |25.4|11.4|10.7|12.4|29.7|0.63|71.5|87.7|26.0], ], |[600|[[melon],
|[2.3|0.57|0.35], |25.3|11.4|10.7|12.4|29.5|0.62|71.1|87.6|26.1], ], |[625|[[melon],
|[2.3|0.57|0.35], |25.3|11.4|10.7|12.4|29.5|0.62|71.1|87.6|26.1], ], |[650|[[melon],
|[2.3|0.57|0.35], |25.3|11.4|10.7|12.4|29.5|0.62|71.1|87.6|26.1], ], |[675|[[melon],
|[2.3|0.57|0.35], |25.3|11.4|10.7|12.4|29.5|0.62|71.1|87.6|26.1], ], |[700|[[melon],
|[2.3|0.57|0.35], |25.3|11.4|10.7|12.4|29.5|0.62|71.1|87.6|26.1], ], |[725|[[melon],
|[2.3|0.57|0.35], |25.3|11.4|10.7|12.4|29.5|0.63|71.0|87.3|26.1], ], |[750|[[melon],
|[2.3|0.57|0.35], |25.3|11.4|10.5|12.4|29.5|0.63|71.5|87.8|25.8], ], |[775|[[melon],
|[2.3|0.57|0.35], |25.3|11.4|10.5|12.4|29.5|0.63|71.5|87.8|25.8], ], |[800|[[melon],
|[2.3|0.57|0.35], |25.3|11.4|10.4|12.4|29.4|0.63|71.5|87.7|25.6], ], |[825|[[melon],
|[2.3|0.56|0.35], |25.1|11.3|10.4|12.4|28.9|0.61|71.0|87.4|25.9], ], |[850|[[melon],
|[2.3|0.56|0.35], |25.2|11.3|10.4|12.4|29.0|0.6|71.4|87.0|26.3],
], |[875|[[melon], |[2.3|0.56|0.34], |25.0|11.3|10.4|12.3|28.9|0.6|71.5|86.9|25.9], ],
|[900|[[melon], |[2.3|0.56|0.34], |25.0|11.3|10.4|12.3|28.9|0.6|71.5|86.9|25.9], ],
|[925|[[melon], |[2.3|0.56|0.34], |25.0|11.3|10.4|12.3|28.9|0.59|71.6|87.2|26.2], ],
|[950|[[melon], |[2.3|0.56|0.34], |25.0|11.3|10.4|12.3|28.9|0.59|71.6|87.2|26.2], ],
|[975|[[melon], |[2.3|0.56|0.35], |25.1|11.3|10.4|12.4|28.9|0.59|71.3|87.4|26.5], ],
|[1000|[[melon], |[2.3|0.56|0.35], |25.2|11.3|10.4|12.4|29.0|0.6|71.2|87.2|26.6], ],
|[1025|[[melon], |[2.3|0.56|0.35], |25.2|11.3|10.4|12.4|29.0|0.6|71.2|87.2|26.6], ],
|[1050|[[melon], |[2.3|0.56|0.35], |25.4|11.5|10.4|12.5|29.1|0.6|71.3|87.0|26.7], ],
|[1075|[[melon], |[2.3|0.56|0.35], |25.4|11.5|10.4|12.5|29.1|0.6|71.3|87.0|26.7], ],
|[1100|[[melon], |[2.3|0.56|0.35], |25.4|11.5|10.4|12.5|29.1|0.6|71.3|87.0|26.7], ],
|[1125|[[melon], |[2.3|0.56|0.35], |25.4|11.5|10.4|12.5|29.1|0.6|71.3|87.0|26.7], ],
|[1150|[[melon], |[2.3|0.56|0.35], |25.4|11.5|10.5|12.5|29.1|0.6|71.2|87.1|26.9], ],
|[1175|[[melon], |[2.4|0.57|0.35], |25.6|11.6|10.5|12.5|29.3|0.61|71.6|87.2|27.4], ],
...
...
...
|[9000|[[berry], |[2.4|0.57|0.36], |27.3|12.2|10.2|11.9|29.9|0.65|73.5|87.0|25.9], ],

```

[9025 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.2 11.9 29.9 0.65 73.5 87.0 25.9],	,
[9050 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.2 11.9 29.9 0.65 73.5 87.0 25.9],	,
[9075 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.2 11.9 29.9 0.65 73.5 87.0 25.9],	,
[9100 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9125 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9150 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9175 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9200 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9225 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9250 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9275 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9300 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9325 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9350 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9375 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9400 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9425 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9450 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9475 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9500 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9525 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9550 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9575 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9600 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9625 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9650 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9675 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9700 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9725 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9750 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9775 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9800 [[berry],	[2.4 0.57 0.36],	27.3 12.2 10.1 11.9 30.2 0.67 74.3 87.1 26.1],	,
[9825 [[berry],	[2.4 0.58 0.37],	27.3 12.2 10.1 12.0 30.2 0.66 74.3 87.0 25.5],	,
[9850 [[berry],	[2.4 0.58 0.37],	27.3 12.2 10.1 12.0 30.2 0.66 74.3 87.0 25.5],	,
[9875 [[berry],	[2.4 0.58 0.37],	27.3 12.2 10.1 12.0 30.2 0.66 74.3 87.0 25.5],	,
[9900 [[berry],	[2.4 0.58 0.37],	27.3 12.2 10.1 12.0 30.2 0.66 74.3 87.0 25.5],	,
[9925 [[berry],	[2.4 0.58 0.37],	27.3 12.2 10.1 12.0 30.2 0.66 74.3 87.0 25.5],	,
[9950 [[berry],	[2.4 0.58 0.37],	27.3 12.2 10.1 12.0 30.2 0.66 74.3 87.0 25.5],	,
[9975 [[berry],	[2.4 0.58 0.37],	27.3 12.2 10.1 12.0 30.2 0.66 74.3 87.0 25.5],	,
[10000 [[berry],	[2.4 0.58 0.37],	27.3 12.2 10.1 12.0 30.2 0.66 74.3 87.0 25.5],	,],
[25 [[apple 28],	[berry 41],	[melon 30],	[lily 33],
[lavendar 37],	[meat 36],	[wildgrass 31],	,],
[50 [[apple 30],	[berry 42],	[melon 30],	[lily 36],
[lavendar 36],	[meat 35],	[wildgrass 33],	,],
[75 [[apple 32],	[berry 45],	[melon 30],	[lily 38],
[lavendar 40],	[meat 38],	[wildgrass 40],	,],
[100 [[apple 30],	[berry 45],	[melon 30],	[lily 33],
[lavendar 39],	[meat 37],	[wildgrass 39],	,],
[125 [[apple 28],	[berry 45],	[melon 32],	[lily 33],
[lavendar 39],	[meat 36],	[wildgrass 39],	,],
[150 [[apple 26],	[berry 45],	[melon 33],	[lily 29],
[lavendar 36],	[meat 35],	[wildgrass 37],	,],
[175 [[apple 25],	[berry 41],	[melon 30],	[lily 28],
[lavendar 27],	[meat 33],	[wildgrass 34],	,],
[200 [[apple 24],	[berry 35],	[melon 29],	[lily 28],
[lavendar 18],	[meat 29],	[wildgrass 32],	,],
[225 [[apple 23],	[berry 30],	[melon 26],	[lily 26],
[lavendar 15],	[meat 22],	[wildgrass 27],	,],
[250 [[apple 21],	[berry 30],		


```

|berry|13], |melon|5], |lily|1], |lavendar|0], |meat|0], |wildgrass|4], ], ],
|9500|[[apple|12], |berry|13], |melon|5], |lily|1], |lavendar|0], |meat|0], |wildgrass|4], ],
], |9525|[[apple|12], |berry|13], |melon|5], |lily|1], |lavendar|0], |meat|0], |wildgrass|4],
], ], |9550|[[apple|12], |berry|13], |melon|5], |lily|1], |lavendar|0], |meat|0],
|wildgrass|4], ], ], |9575|[[apple|12], |berry|13], |melon|5], |lily|1], |lavendar|0],
|meat|0], |wildgrass|4], ], ], |9600|[[apple|12], |berry|13], |melon|5], |lily|1],
|lavendar|0], |meat|0], |wildgrass|4], ], ], |9625|[[apple|12], |berry|13], |melon|5],
|lily|1], |lavendar|0], |meat|0], |wildgrass|4], ], ], |9650|[[apple|12], |berry|13],
|melon|5], |lily|1], |lavendar|0], |meat|0], |wildgrass|4], ], ], |9675|[[apple|12],
|berry|13], |melon|5], |lily|1], |lavendar|0], |meat|0], |wildgrass|4], ], ],
|9700|[[apple|12], |berry|13], |melon|5], |lily|1], |lavendar|0], |meat|0], |wildgrass|4], ],
], |9725|[[apple|12], |berry|13], |melon|5], |lily|1], |lavendar|0], |meat|0], |wildgrass|4],
], ], |9750|[[apple|12], |berry|13], |melon|5], |lily|1], |lavendar|0], |meat|0],
|wildgrass|4], ], ], |9775|[[apple|12], |berry|13], |melon|5], |lily|1], |lavendar|0],
|meat|0],
|wildgrass|4], ], ], |9800|[[apple|12], |berry|13], |melon|5], |lily|1], |lavendar|0],
|meat|0], |wildgrass|4], ], ], |9825|[[apple|12], |berry|13], |melon|4], |lily|1],
|lavendar|0], |meat|0], |wildgrass|4], ], ], |9850|[[apple|12], |berry|13], |melon|4],
|lily|1], |lavendar|0], |meat|0], |wildgrass|4], ], ], |9875|[[apple|12], |berry|13],
|melon|4], |lily|1], |lavendar|0], |meat|0], |wildgrass|4], ], ], |9900|[[apple|12],
|berry|13], |melon|4], |lily|1], |lavendar|0], |meat|0], |wildgrass|4], ], ],
|9925|[[apple|12], |berry|13], |melon|4], |lily|1], |lavendar|0], |meat|0], |wildgrass|4], ],
], |9950|[[apple|12], |berry|13], |melon|4], |lily|1], |lavendar|0], |meat|0], |wildgrass|4],
], ], |9975|[[apple|12], |berry|13], |melon|4], |lily|1], |lavendar|0], |meat|0],
|wildgrass|4], ], ], |10000|[[apple|12], |berry|13], |melon|4], |lily|1], |lavendar|0],
|meat|0], |wildgrass|4], ], ], ],

```

16/03:

```

60
10000
[['apple', 0], ['berry', 0], ['melon', 0], ['lily', 0], ['lavendar', 0], ['meat', 0], ['wildgrass', 0]]
['apple', 'berry', 'melon', 'lily', 'lavendar', 'meat', 'wildgrass']
[206|1|0|104|0],
//////////
//////////
//////////
[[25|[[apple], |2.0|0.5|0.3], |22.4|10.1|9.7|10.0|24.9|0.53|66.7|77.6|25.4], ], |50|[[apple],
|2.1|0.51|0.31], |23.1|10.4|10.0|10.4|25.6|0.54|67.7|77.9|25.6], ], |75|[[apple],
|2.2|0.55|0.33], |24.9|11.0|10.7|11.2|27.6|0.59|71.6|79.8|27.5], ], |100|[[apple],
|2.3|0.58|0.34], |25.7|11.3|11.0|11.7|28.5|0.61|73.7|81.5|28.6], ], |125|[[apple],
|2.3|0.58|0.34], |26.0|11.5|11.2|11.9|28.9|0.62|74.3|81.9|29.2], ], |150|[[apple],
|2.3|0.59|0.35], |26.1|11.4|11.4|12.0|29.1|0.62|75.2|82.4|29.3], ], |175|[[apple],
|2.4|0.59|0.35], |26.3|11.4|11.4|12.1|29.5|0.62|75.2|82.8|29.9], ], |200|[[berry],
|2.4|0.6|0.36], |26.8|11.4|11.6|12.3|30.1|0.63|75.8|83.2|30.8], ], |225|[[berry],
|2.4|0.61|0.37], |27.3|11.4|11.7|12.5|30.8|0.62|75.6|84.2|31.0], ], |250|[[berry],
|2.5|0.62|0.37], |27.7|11.4|11.9|12.6|31.3|0.62|76.0|84.7|31.3], ], |275|[[berry],
|2.5|0.64|0.38], |28.4|11.5|12.2|12.8|32.0|0.63|76.9|85.3|31.6], ], |300|[[berry],
|2.5|0.64|0.39], |28.6|11.5|12.3|12.7|32.2|0.64|76.7|84.7|31.5], ], |325|[[berry],
|2.5|0.65|0.39], |28.8|11.6|12.4|12.8|32.4|0.64|77.5|85.5|32.0], ], |350|[[berry],
|2.6|0.65|0.39], |28.8|11.6|12.4|12.8|32.4|0.64|77.8|85.4|31.5], ], |375|[[berry],
|2.6|0.64|0.38], |28.7|11.5|12.2|12.7|32.3|0.64|77.5|85.6|31.2], ], |400|[[berry],
|2.5|0.63|0.38], |28.6|11.4|12.1|12.7|32.0|0.64|77.5|85.5|31.3], ], |425|[[berry],

```

[2.5 0.63 0.37],	28.6 11.4 12.1 12.7 32.0 0.64 77.8 86.2 31.5],	,	[450 [[berry],
[2.5 0.63 0.37],	28.6 11.4 12.0 12.7 32.0 0.64 77.6 86.0 31.3],	,	[475 [[berry],
[2.5 0.63 0.38],	28.5 11.4 12.1 12.7 32.1 0.64 77.8 86.4 31.5],	,	[500 [[berry],
[2.5 0.63 0.38],	28.7 11.4 12.1 12.8 32.2 0.64 77.6 86.1 31.8],	,	[525 [[berry],
[2.5 0.63 0.37],	28.6 11.4 12.1 12.7 32.1 0.64 77.3 86.1 31.6],	,	[550 [[berry],
[2.5 0.63 0.37],	28.5 11.4 12.0 12.7 32.0 0.63 77.4 86.0 31.8],	,	[575 [[berry],
[2.5 0.63 0.37],	28.5 11.4 12.0 12.6 31.8 0.62 77.1 85.7 31.6],	,	[600 [[berry],
[2.5 0.63 0.37],	28.6 11.5 12.0 12.7 31.9 0.62 77.2 86.0 32.0],	,	[625 [[berry],
[2.5 0.63 0.37],	28.4 11.5 12.0 12.6 31.7 0.62 76.6 86.0 31.6],	,	[650 [[berry],
[2.5 0.63 0.38],	28.6 11.5 12.0 12.6 31.8 0.62 77.5 86.3 31.6],	,	[675 [[berry],
[2.5 0.63 0.38],	28.6 11.6 12.0 12.6 32.0 0.62 78.1 86.5 31.6],	,	[700 [[berry],
[2.5 0.63 0.38],	28.6 11.6 12.0 12.7 32.0 0.62 78.2 86.6 31.6],	,	[725 [[berry],
[2.6 0.64 0.38],	28.8 11.6 12.0 12.8 32.2 0.63 78.8 87.0 31.8],	,	[750 [[berry],
[2.6 0.64 0.38],	28.7 11.6 12.0 12.7 32.2 0.63 78.8 86.8 31.9],	,	[775 [[berry],
[2.6 0.64 0.38],	28.7 11.6 12.0 12.7 32.2 0.63 78.8 86.8 31.9],	,	[800 [[berry],
[2.6 0.64 0.38],	28.7 11.6 12.0 12.7 32.2 0.63 78.8 86.8 31.9],	,	[825 [[berry],
[2.6 0.64 0.38],	28.7 11.6 12.0 12.7 32.2 0.63 78.8 86.8 31.9],	,	[850 [[berry],
[2.6 0.64 0.38],	28.7 11.6 12.0 12.7 32.2 0.63 78.8 86.8 31.9],	,	[875 [[berry],
[2.6 0.64 0.38],	28.8 11.6 12.0 12.7 32.3 0.63 78.9 87.0 31.9],	,	[900 [[berry],
[2.6 0.64 0.38],	28.8 11.6 12.0 12.7 32.3 0.63 78.9 87.0 31.9],	,	[925 [[berry],
[2.6 0.64 0.38],	28.8 11.6 12.0 12.7 32.3 0.63 78.9 87.0 31.9],	,	[950 [[berry],
[2.6 0.64 0.38],	28.9 11.6 12.0 12.7 32.3 0.62 79.0 87.3 32.1],	,	[975 [[berry],
[2.6 0.64 0.38],	29.0 11.7 12.1 12.7 32.4 0.62 78.8 87.2 32.1],	,	[1000 [[berry],
[2.6 0.64 0.38],	29.1 11.8 12.1 12.8 32.5 0.62 78.9 87.4 32.0],	,	[1025 [[berry],
[2.6 0.64 0.38],	29.1 11.8 12.1 12.8 32.5 0.62 78.9 87.4 32.0],	,	[1050 [[berry],
[2.6 0.64 0.38],	29.1 11.8 12.1 12.8 32.5 0.62 78.9 87.4 32.0],	,	[1075 [[berry],
[2.6 0.64 0.38],	28.9 11.8 12.0 12.6 32.3 0.61 78.7 87.3 32.1],	,	[1100 [[berry],
[2.6 0.64 0.38],	29.0 11.8 12.1 12.6 32.4 0.62 79.0 87.1 32.3],	,	
...			
...			
...			
[9000 [[berry],	[2.6 0.61 0.38],	28.0 12.1 11.5 12.3 31.9 0.56 79.1 87.5 32.1],	,
[9025 [[berry],	[2.6 0.61 0.38],	28.0 12.1 11.5 12.3 31.9 0.56 79.1 87.5 32.1],	,
[9050 [[berry],	[2.6 0.61 0.38],	28.0 12.1 11.5 12.3 31.9 0.56 79.1 87.5 32.1],	,
[9075 [[berry],	[2.6 0.61 0.38],	28.0 12.1 11.5 12.3 31.9 0.56 79.1 87.5 32.1],	,
[9100 [[berry],	[2.6 0.61 0.38],	28.0 12.1 11.5 12.3 31.9 0.56 79.1 87.5 32.1],	,
[9125 [[berry],	[2.6 0.61 0.38],	28.0 12.1 11.5 12.3 31.9 0.56 79.1 87.5 32.1],	,
[9150 [[berry],	[2.6 0.61 0.38],	28.0 12.1 11.5 12.3 31.9 0.56 79.1 87.5 32.1],	,
[9175 [[berry],	[2.6 0.61 0.38],	28.0 12.1 11.5 12.3 31.9 0.56 79.1 87.5 32.1],	,
[9200 [[berry],	[2.6 0.61 0.38],	28.0 12.1 11.5 12.3 31.9 0.56 79.1 87.5 32.1],	,
[9225 [[berry],	[2.6 0.61 0.38],	27.7 12.0 11.5 12.3 31.6 0.55 79.5 87.3 31.3],	,
[9250 [[berry],	[2.6 0.61 0.38],	27.7 12.0 11.5 12.3 31.6 0.55 79.5 87.3 31.3],	,
[9275 [[berry],	[2.6 0.61 0.38],	27.7 12.0 11.5 12.3 31.6 0.55 79.5 87.3 31.3],	,
[9300 [[berry],	[2.6 0.61 0.38],	27.7 12.0 11.5 12.3 31.6 0.55 79.5 87.3 31.3],	,
[9325 [[berry],	[2.6 0.61 0.38],	27.7 12.0 11.5 12.3 31.6 0.55 79.5 87.3 31.3],	,
[9350 [[berry],	[2.6 0.61 0.38],	27.7 12.0 11.5 12.3 31.6 0.55 79.5 87.3 31.3],	,
[9375 [[berry],	[2.6 0.61 0.38],	27.7 12.0 11.5 12.3 31.6 0.55 79.5 87.3 31.3],	,
[9400 [[berry],	[2.6 0.61 0.38],	27.7 12.1 11.6 12.3 31.6 0.56 79.8 87.2 31.8],	,
[9425 [[berry],	[2.6 0.61 0.38],	27.9 12.1 11.5 12.3 31.7 0.56 79.9 87.7 31.6],	,
[9450 [[berry],	[2.6 0.61 0.38],	27.9 12.1 11.5 12.3 31.7 0.56 79.9 87.7 31.6],	,
[9475 [[berry],	[2.6 0.61 0.38],	27.9 12.1 11.5 12.3 31.7 0.56 79.9 87.7 31.6],	,


```

|[9500|[[berry],    |[2.6|0.61|0.38],    |27.9|12.1|11.5|12.3|31.7|0.56|79.9|87.7|31.6],    ],
|[9525|[[berry],    |[2.6|0.61|0.38],    |27.9|12.1|11.5|12.3|31.7|0.56|79.9|87.7|31.6],    ],
|[9550|[[berry],    |[2.6|0.6|0.38],    |27.7|12.1|11.4|12.3|31.3|0.55|79.7|87.5|31.2],    ],
|[9575|[[berry],    |[2.6|0.6|0.38],    |27.7|12.1|11.4|12.3|31.3|0.55|79.7|87.5|31.2],    ],
|[9600|[[berry],    |[2.6|0.6|0.38],    |27.7|12.1|11.4|12.3|31.3|0.55|79.7|87.5|31.2],    ],
|[9625|[[berry],    |[2.6|0.6|0.38],    |27.7|12.1|11.4|12.3|31.3|0.55|79.7|87.5|31.2],    ],
|[9650|[[berry],    |[2.6|0.6|0.38],    |27.7|12.1|11.4|12.3|31.3|0.55|79.7|87.5|31.2],    ],
|[9675|[[berry],    |[2.6|0.6|0.38],    |27.7|12.1|11.4|12.3|31.3|0.55|79.7|87.5|31.2],    ],
|[9700|[[berry],    |[2.6|0.6|0.38],    |27.7|12.1|11.4|12.3|31.3|0.55|79.7|87.5|31.2],    ],
|[9725|[[berry],    |[2.6|0.6|0.38],    |27.7|12.1|11.4|12.3|31.3|0.55|79.7|87.5|31.2],    ],
|[9750|[[berry],    |[2.6|0.6|0.38],    |27.7|12.1|11.4|12.3|31.3|0.55|79.7|87.5|31.2],    ],
|[9775|[[berry],    |[2.6|0.6|0.38],    |27.7|12.1|11.4|12.3|31.3|0.55|79.7|87.5|31.2],    ],
|[9800|[[berry],    |[2.6|0.6|0.38],    |27.7|12.1|11.4|12.3|31.3|0.55|79.7|87.5|31.2],    ],
|[9825|[[berry],    |[2.6|0.6|0.38],    |27.7|12.1|11.4|12.3|31.3|0.55|79.7|87.5|31.2],    ],
|[9850|[[berry],    |[2.6|0.6|0.38],    |27.7|12.1|11.4|12.3|31.3|0.55|79.7|87.5|31.2],    ],
|[9875|[[berry],    |[2.6|0.6|0.38],    |27.7|12.1|11.4|12.3|31.3|0.55|79.7|87.5|31.2],    ],
|[9900|[[berry],    |[2.6|0.6|0.38],    |27.7|12.1|11.4|12.3|31.3|0.55|79.7|87.5|31.2],    ],
|[9925|[[berry],    |[2.6|0.6|0.38],    |27.7|12.1|11.4|12.3|31.3|0.55|79.7|87.5|31.2],    ],
|[9950|[[berry],    |[2.6|0.6|0.38],    |27.7|12.1|11.4|12.3|31.3|0.55|79.7|87.5|31.2],    ],
|[9975|[[berry],    |[2.6|0.6|0.38],    |27.7|12.1|11.4|12.3|31.3|0.55|79.7|87.5|31.2],    ],
|[10000|[[berry], |[2.6|0.6|0.38], |27.7|12.1|11.4|12.3|31.3|0.55|79.7|87.5|31.2], ], ],
#####
#####
#####
[[25|[[apple|43],    |[berry|36],    |[melon|34],    |[lily|28],    |[lavendar|32],    |[meat|28],
|[wildgrass|38], ], ], |[50|[[apple|44], |[berry|39], |[melon|35], |[lily|30], |[lavendar|36],
|[meat|28], |[wildgrass|42], ], ], |[75|[[apple|53], |[berry|45], |[melon|41], |[lily|32],
|[lavendar|41], |[meat|33], |[wildgrass|49], ], ], |[100|[[apple|54], |[berry|44], |[melon|39],
|[lily|33], |[lavendar|39], |[meat|31], |[wildgrass|47], ], ], |[125|[[apple|50], |[berry|44],
|[melon|38], |[lily|34], |[lavendar|36], |[meat|28], |[wildgrass|47], ], ], |[150|[[apple|49],
|[berry|45], |[melon|40], |[lily|34], |[lavendar|31], |[meat|25], |[wildgrass|47], ], ],
|[175|[[apple|49], |[berry|48], |[melon|40], |[lily|36], |[lavendar|25], |[meat|22],
|[wildgrass|46], ], ], |[200|[[apple|48], |[berry|50], |[melon|40], |[lily|38], |[lavendar|20],
|[meat|20], |[wildgrass|42], ], ], |[225|[[apple|44], |[berry|51], |[melon|37], |[lily|38],
|[lavendar|14], |[meat|13], |[wildgrass|41], ], ], |[250|[[apple|43], |[berry|51], |[melon|35],
|[lily|36], |[lavendar|8], |[meat|13], |[wildgrass|38], ], ], |[275|[[apple|40], |[berry|50],
|[melon|36], |[lily|35], |[lavendar|5], |[meat|8], |[wildgrass|36], ], ], |[300|[[apple|39],
|[berry|49], |[melon|35], |[lily|35], |[lavendar|5], |[meat|6], |[wildgrass|34], ], ],
|[325|[[apple|39], |[berry|51], |[melon|35], |[lily|34], |[lavendar|3], |[meat|6], |[wildgrass|33],
], ], |[350|[[apple|38], |[berry|51], |[melon|35], |[lily|34], |[lavendar|1], |[meat|5],
|[wildgrass|32], ], ], |[375|[[apple|35], |[berry|49], |[melon|33], |[lily|33], |[lavendar|1],
|[meat|5], |[wildgrass|31], ], ], |[400|[[apple|34], |[berry|47], |[melon|33], |[lily|34],
|[lavendar|1], |[meat|5], |[wildgrass|31], ], ], |[425|[[apple|33], |[berry|45], |[melon|33],
|[lily|34], |[lavendar|1], |[meat|4], |[wildgrass|29], ], ], |[450|[[apple|33], |[berry|44],
|[melon|33], |[lily|34], |[lavendar|1], |[meat|4], |[wildgrass|27], ], ], |[475|[[apple|30],
|[berry|44], |[melon|33], |[lily|34], |[lavendar|0], |[meat|4], |[wildgrass|26], ], ],
|[500|[[apple|30], |[berry|44], |[melon|32], |[lily|33], |[lavendar|0], |[meat|2], |[wildgrass|25],
], ], |[525|[[apple|30], |[berry|44], |[melon|30], |[lily|33], |[lavendar|0], |[meat|2],
|[wildgrass|24], ], ], |[550|[[apple|30], |[berry|44], |[melon|30], |[lily|33], |[lavendar|0],
|[meat|2], |[wildgrass|22], ], ], |[575|[[apple|30], |[berry|42], |[melon|29], |[lily|33],
|[lavendar|0], |[meat|2], |[wildgrass|21], ], ], |[600|[[apple|29], |[berry|42], |[melon|29],

```


[illegible]

[illegible]

[illegible]


```

[[lily|3], [[lavendar|0], [[meat|0], [[wildgrass|8], ], ], [[9575|[[apple|17], [[berry|28],
[[melon|4], [[lily|3], [[lavendar|0], [[meat|0], [[wildgrass|8], ], ], [[9600|[[apple|17],
[[berry|28], [[melon|4], [[lily|3], [[lavendar|0], [[meat|0], [[wildgrass|8], ], ],
[[9625|[[apple|17], [[berry|28], [[melon|4], [[lily|3], [[lavendar|0], [[meat|0], [[wildgrass|8], ],
], ], [[9650|[[apple|17], [[berry|28], [[melon|4], [[lily|3], [[lavendar|0], [[meat|0], [[wildgrass|8],
], ], ], [[9675|[[apple|17], [[berry|28], [[melon|4], [[lily|3], [[lavendar|0], [[meat|0],
[[wildgrass|8], ], ], ], [[9700|[[apple|17], [[berry|28], [[melon|4], [[lily|3], [[lavendar|0],
[[meat|0], [[wildgrass|8], ], ], ], [[9725|[[apple|17], [[berry|28], [[melon|4], [[lily|3],
[[lavendar|0], [[meat|0], [[wildgrass|8], ], ], ], [[9750|[[apple|17], [[berry|28], [[melon|4],
[[lily|3], [[lavendar|0], [[meat|0], [[wildgrass|8], ], ], ], [[9775|[[apple|17], [[berry|28],
[[melon|4], [[lily|3], [[lavendar|0], [[meat|0], [[wildgrass|8], ], ], ], [[9800|[[apple|17],
[[berry|28], [[melon|4], [[lily|3], [[lavendar|0], [[meat|0], [[wildgrass|8], ], ], ],
[[9825|[[apple|17], [[berry|28], [[melon|4], [[lily|3], [[lavendar|0], [[meat|0], [[wildgrass|8], ],
], ], [[9850|[[apple|17], [[berry|28], [[melon|4], [[lily|3], [[lavendar|0], [[meat|0], [[wildgrass|8],
], ], ], [[9875|[[apple|17], [[berry|28], [[melon|4], [[lily|3], [[lavendar|0], [[meat|0],
[[wildgrass|8], ], ], ], [[9900|[[apple|17], [[berry|28], [[melon|4], [[lily|3], [[lavendar|0],
[[meat|0], [[wildgrass|8], ], ], ], [[9925|[[apple|17], [[berry|28], [[melon|4], [[lily|3],
[[lavendar|0], [[meat|0], [[wildgrass|8], ], ], ], [[9950|[[apple|17], [[berry|28], [[melon|4],
[[lily|3], [[lavendar|0], [[meat|0], [[wildgrass|8], ], ], ], [[9975|[[apple|17], [[berry|28],
[[melon|4], [[lily|3], [[lavendar|0], [[meat|0], [[wildgrass|8], ], ], ], [[10000|[[apple|17],
[[berry|28], [[melon|4], [[lily|3], [[lavendar|0], [[meat|0], [[wildgrass|8], ], ], ],
////////////////////
////////////////////
////////////////////
[[25|12.3], [[50|16.0], [[75|62.5], [[100|88.2], [[125|103.4], [[150|121.9], [[175|147.1],
[[200|161.9], [[225|197.9], [[250|201.4], [[275|233.3], [[300|221.7], [[325|253.3],
[[350|281.5], [[375|274.9], [[400|257.8], [[425|267.2], [[450|302.0], [[475|325.6],
[[500|371.9], [[525|417.5], [[550|426.8], [[575|450.2], [[600|473.2], [[625|489.6],
[[650|514.9], [[675|603.7], [[700|609.1], [[725|637.0], [[750|678.1], [[775|680.7],
[[800|680.7], [[825|680.7], [[850|680.7], [[875|699.8], [[900|699.8], [[925|699.8],
[[950|761.7], [[975|792.7], [[1000|877.1], [[1025|877.1], [[1050|877.1], [[1075|908.0],
[[1100|968.9], [[1125|957.0], [[1150|957.0], [[1175|957.0], [[1200|957.0], [[1225|957.0],
[[1250|987.1], [[1275|987.1], [[1300|987.1], [[1325|987.1], [[1350|1.07k], [[1375|1.1k],
[[1400|1.12k], [[1425|1.14k], [[1450|1.14k], [[1475|1.14k], [[1500|1.09k], [[1525|1.09k],
[[1550|1.09k], [[1575|1.15k], [[1600|1.15k], [[1625|1.01k], [[1650|1.01k], [[1675|977.2],
[[1700|977.2], [[1725|977.2], [[1750|1.01k], [[1775|977.4], [[1800|977.4], [[1825|977.4],
[[1850|977.4], [[1875|907.8], [[1900|907.8], [[1925|969.4], [[1950|1.06k], [[1975|1.11k],
[[2000|1.2k], [[2025|1.2k], [[2050|1.29k], [[2075|1.43k], [[2100|1.63k], [[2125|1.82k],
[[2150|1.94k], [[2175|1.94k], [[2200|1.96k], [[2225|1.96k], [[2250|1.96k], [[2275|1.96k],
[[2300|1.99k], [[2325|2.02k], [[2350|2.03k], [[2375|2.03k], [[2400|1.95k], [[2425|2.04k],
[[2450|2.04k], [[2475|2.04k], [[2500|2.04k], [[2525|2.08k], [[2550|2.08k], [[2575|2.09k],
[[2600|2.17k], [[2625|2.17k], [[2650|2.17k], [[2675|2.17k], [[2700|2.17k], [[2725|2.17k],
[[2750|2.17k], [[2775|2.17k], [[2800|2.17k], [[2825|2.17k], [[2850|2.17k], [[2875|2.17k],
[[2900|2.17k], [[2925|2.17k], [[2950|2.17k], [[2975|2.28k], [[3000|2.28k], [[3025|2.28k],
[[3050|2.28k], [[3075|2.28k], [[3100|2.28k], [[3125|2.28k], [[3150|2.28k], [[3175|2.28k],
[[3200|2.28k], [[3225|2.28k], [[3250|2.28k], [[3275|2.28k], [[3300|2.37k], [[3325|2.4k],
[[3350|2.4k], [[3375|2.51k], [[3400|2.61k], [[3425|2.79k], [[3450|2.79k], [[3475|2.79k],
[[3500|2.79k], [[3525|2.79k], [[3550|2.79k], [[3575|2.79k], [[3600|2.83k], [[3625|2.83k],
[[3650|3.0k], [[3675|3.0k], [[3700|3.12k], [[3725|3.27k], [[3750|3.27k], [[3775|3.27k],
[[3800|3.27k], [[3825|3.27k], [[3850|3.27k], [[3875|3.27k], [[3900|3.27k], [[3925|3.27k],
[[3950|3.27k], [[3975|3.27k], [[4000|3.27k], [[4025|3.27k], [[4050|3.27k], [[4075|3.27k],

```

```

|[4100|3.27k], |[4125|3.27k], |[4150|3.27k], |[4175|3.27k], |[4200|3.35k], |[4225|3.42k],
|[4250|3.42k], |[4275|3.42k], |[4300|3.42k], |[4325|3.42k], |[4350|3.5k], |[4375|3.5k],
|[4400|3.55k], |[4425|3.55k], |[4450|3.55k], |[4475|3.55k], |[4500|3.55k], |[4525|3.68k],
|[4550|3.68k], |[4575|3.68k], |[4600|3.75k], |[4625|3.79k], |[4650|3.79k], |[4675|3.87k],
|[4700|3.93k], |[4725|3.93k], |[4750|4.1k], |[4775|4.1k], |[4800|4.1k], |[4825|4.1k],
|[4850|4.1k], |[4875|4.1k], |[4900|4.1k], |[4925|4.1k], |[4950|4.1k], |[4975|4.26k],
|[5000|4.26k], |[5025|4.26k], |[5050|4.26k], |[5075|4.4k], |[5100|4.4k], |[5125|4.4k],
|[5150|4.4k], |[5175|4.4k], |[5200|4.4k], |[5225|4.4k], |[5250|4.4k], |[5275|4.4k], |[5300|4.4k],
|[5325|4.4k], |[5350|4.4k], |[5375|4.4k], |[5400|4.4k], |[5425|4.4k], |[5450|4.4k], |[5475|4.4k],
|[5500|4.4k], |[5525|4.4k], |[5550|4.4k], |[5575|4.4k], |[5600|4.4k], |[5625|4.59k],
|[5650|4.59k], |[5675|4.79k], |[5700|4.79k], |[5725|4.79k], |[5750|4.79k], |[5775|4.79k],
|[5800|4.79k], |[5825|4.79k], |[5850|4.79k], |[5875|4.79k], |[5900|5.1k], |[5925|5.24k],
|[5950|5.24k], |[5975|5.24k], |[6000|5.24k], |[6025|5.24k], |[6050|5.24k], |[6075|5.24k],
|[6100|5.24k], |[6125|5.24k], |[6150|5.24k], |[6175|5.24k], |[6200|5.24k], |[6225|5.24k],
|[6250|5.24k], |[6275|5.24k], |[6300|5.24k], |[6325|5.24k], |[6350|5.24k], |[6375|5.24k],
|[6400|5.24k], |[6425|5.24k], |[6450|5.24k], |[6475|5.24k], |[6500|5.24k], |[6525|5.24k],
|[6550|5.24k], |[6575|5.24k], |[6600|5.24k], |[6625|5.24k], |[6650|5.24k], |[6675|5.24k],
|[6700|5.24k], |[6725|5.24k], |[6750|5.24k], |[6775|5.24k], |[6800|5.24k], |[6825|5.24k],
|[6850|5.24k], |[6875|5.24k], |[6900|5.24k], |[6925|5.49k], |[6950|5.49k], |[6975|5.49k],
|[7000|5.49k], |[7025|5.49k], |[7050|5.49k], |[7075|5.49k], |[7100|5.49k], |[7125|5.49k],
|[7150|5.49k], |[7175|5.49k], |[7200|5.63k], |[7225|5.72k], |[7250|5.72k], |[7275|5.72k],
|[7300|5.86k], |[7325|5.98k], |[7350|5.98k], |[7375|5.98k], |[7400|5.98k], |[7425|5.98k],
|[7450|5.98k], |[7475|5.98k], |[7500|5.98k], |[7525|5.98k], |[7550|5.98k], |[7575|5.98k],
|[7600|5.98k], |[7625|5.98k], |[7650|5.98k], |[7675|5.98k], |[7700|5.98k], |[7725|5.98k],
|[7750|5.98k], |[7775|5.98k], |[7800|5.98k], |[7825|5.98k], |[7850|5.98k], |[7875|6.16k],
|[7900|6.32k], |[7925|6.44k], |[7950|6.44k], |[7975|6.44k], |[8000|6.44k], |[8025|6.44k],
|[8050|6.44k], |[8075|6.44k], |[8100|6.44k], |[8125|6.44k], |[8150|6.44k], |[8175|6.44k],
|[8200|6.44k], |[8225|6.44k], |[8250|6.44k], |[8275|6.44k], |[8300|6.71k], |[8325|6.96k],
|[8350|6.96k], |[8375|6.96k], |[8400|6.96k], |[8425|6.96k], |[8450|6.96k], |[8475|6.96k],
|[8500|6.96k], |[8525|6.96k], |[8550|6.96k], |[8575|6.96k], |[8600|6.96k], |[8625|6.96k],
|[8650|6.96k], |[8675|6.96k], |[8700|6.96k], |[8725|6.96k], |[8750|6.96k], |[8775|6.96k],
|[8800|6.96k], |[8825|6.96k], |[8850|6.96k], |[8875|6.96k], |[8900|6.96k], |[8925|6.96k],
|[8950|6.96k], |[8975|6.96k], |[9000|7.14k], |[9025|7.28k], |[9050|7.28k], |[9075|7.28k],
|[9100|7.28k], |[9125|7.28k], |[9150|7.28k], |[9175|7.28k], |[9200|7.28k], |[9225|7.44k],
|[9250|7.6k], |[9275|7.6k], |[9300|7.6k], |[9325|7.6k], |[9350|7.6k], |[9375|7.6k],
|[9400|7.95k], |[9425|8.08k], |[9450|8.2k], |[9475|8.2k], |[9500|8.2k], |[9525|8.2k],
|[9550|8.43k], |[9575|8.43k], |[9600|8.43k], |[9625|8.43k], |[9650|8.43k], |[9675|8.43k],
|[9700|8.43k], |[9725|8.43k], |[9750|8.43k], |[9775|8.43k], |[9800|8.43k], |[9825|8.43k],
|[9850|8.43k], |[9875|8.43k], |[9900|8.43k], |[9925|8.43k], |[9950|8.43k], |[9975|8.43k],
|[10000|8.43k], ],
////////////////////////////////////
////////////////////////////////////
////////////////////////////////////
[[25|[6.2|10.5|19.8], ], |[50|[8.5|13.0|22.2], ], |[75|[58.5|66.0|67.8], ], |[100|[89.2|93.0|98.5],
], |[125|[101.8|115.0|117.8], ], |[150|[100.2|136.0|142.0], ], |[175|[121.0|166.0|172.0], ],
|[200|[138.0|179.0|185.0], ], |[225|[162.0|217.5|221.2], ], |[250|[164.0|191.0|238.5], ],
|[275|[193.2|259.5|266.8], ], |[300|[161.5|219.0|281.8], ], |[325|[191.0|288.0|304.0], ],
|[350|[234.5|304.0|337.0], ], |[375|[220.0|302.0|339.0], ], |[400|[164.8|270.5|344.2], ],
|[425|[136.0|203.0|405.5], ], |[450|[165.5|392.0|408.5], ], |[475|[164.0|392.0|460.0], ],
|[500|[268.0|400.0|473.5], ], |[525|[362.5|461.0|475.2], ], |[550|[350.0|470.0|485.0], ],
|[575|[362.5|467.5|521.8], ], |[600|[342.8|499.5|571.2], ], |[625|[321.0|563.0|586.0], ],

```

[650 [357.0 585.0 628.0],],	[675 [602.2 637.5 651.8],],	[700 [602.2 646.0 655.8],],
[725 [644.0 657.0 710.0],],	[750 [652.0 673.0 715.0],],	[775 [653.8 678.5 716.8],],
[800 [653.8 678.5 716.8],],	[825 [653.8 678.5 716.8],],	[850 [653.8 678.5 716.8],],
[875 [653.8 697.0 724.8],],	[900 [653.8 697.0 724.8],],	[925 [653.8 697.0 724.8],],
[950 [681.2 718.5 880.8],],	[975 [703.5 727.5 937.0],],	[1000 [730.2 937.0 977.8],],
[1025 [730.2 937.0 977.8],],	[1050 [730.2 937.0 977.8],],	[1075 [862.0 954.0 983.0],],
[1100 [937.0 970.0 988.0],],	[1125 [937.0 970.0 988.0],],	[1150 [937.0 970.0 988.0],],
[1175 [937.0 970.0 988.0],],	[1200 [937.0 970.0 988.0],],	[1225 [937.0 970.0 988.0],],
[1250 [949.8 979.5 1.02k],],	[1275 [949.8 979.5 1.02k],],	[1300 [949.8 979.5 1.02k],],
[1325 [949.8 979.5 1.02k],],	[1350 [973.0 990.0 1.26k],],	[1375 [973.0 1.04k 1.34k],],
[1400 [976.0 1.1k 1.34k],],	[1425 [980.5 1.17k 1.34k],],	[1450 [980.5 1.17k 1.34k],],
[1475 [980.5 1.17k 1.34k],],	[1500 [908.8 1.27k 1.34k],],	[1525 [908.8 1.27k 1.34k],],
[1550 [908.8 1.27k 1.34k],],	[1575 [1.01k 1.3k 1.35k],],	[1600 [1.01k 1.3k 1.35k],],
[1625 [382.0 1.3k 1.36k],],	[1650 [374.0 1.31k 1.37k],],	[1675 [362.0 1.3k 1.42k],],
[1700 [362.0 1.3k 1.42k],],	[1725 [362.0 1.3k 1.42k],],	[1750 [374.0 1.31k 1.37k],],
[1775 [362.0 1.3k 1.42k],],	[1800 [362.0 1.3k 1.42k],],	[1825 [362.0 1.3k 1.42k],],
[1850 [362.0 1.3k 1.42k],],	[1875 [362.0 989.0 1.4k],],	[1900 [362.0 989.0 1.4k],],
[1925 [362.0 989.0 1.58k],],	[1950 [374.0 1.3k 1.62k],],	[1975 [374.0 1.34k 1.88k],],
[2000 [386.0 1.45k 1.89k],],	[2025 [386.0 1.45k 1.89k],],	[2050 [390.0 1.62k 1.92k],],
[2075 [606.0 1.8k 1.97k],],	[2100 [1.34k 1.9k 2.03k],],	[2125 [1.84k 1.94k 2.04k],],
[2150 [1.87k 1.94k 2.04k],],	[2175 [1.87k 1.94k 2.04k],],	[2200 [1.87k 1.98k 2.06k],],
[2225 [1.87k 1.98k 2.06k],],	[2250 [1.87k 1.98k 2.06k],],	[2275 [1.87k 1.98k 2.06k],],
[2300 [1.87k 2.02k 2.09k],],	[2325 [1.88k 2.03k 2.12k],],	[2350 [1.89k 2.04k 2.14k],],
[2375 [1.89k 2.04k 2.14k],],	[2400 [1.86k 2.03k 2.12k],],	[2425 [1.89k 2.1k 2.34k],],
[2450 [1.89k 2.1k 2.34k],],	[2475 [1.89k 2.1k 2.34k],],	[2500 [1.89k 2.1k 2.34k],],
[2525 [1.9k 2.12k 2.41k],],	[2550 [1.89k 2.15k 2.41k],],	[2575 [1.9k 2.18k 2.41k],],
[2600 [2.06k 2.26k 2.43k],],	[2625 [2.06k 2.26k 2.43k],],	[2650 [2.06k 2.26k 2.43k],],
[2675 [2.06k 2.26k 2.43k],],	[2700 [2.06k 2.26k 2.43k],],	[2725 [2.06k 2.26k 2.43k],],
[2750 [2.06k 2.26k 2.43k],],	[2775 [2.06k 2.26k 2.43k],],	[2800 [2.06k 2.26k 2.43k],],
[2825 [2.06k 2.26k 2.43k],],	[2850 [2.06k 2.26k 2.43k],],	[2875 [2.06k 2.26k 2.43k],],
[2900 [2.06k 2.26k 2.43k],],	[2925 [2.06k 2.26k 2.43k],],	[2950 [2.06k 2.26k 2.43k],],
[2975 [2.17k 2.36k 2.48k],],	[3000 [2.17k 2.36k 2.48k],],	[3025 [2.17k 2.36k 2.48k],],
[3050 [2.17k 2.36k 2.48k],],	[3075 [2.17k 2.36k 2.48k],],	[3100 [2.17k 2.36k 2.48k],],
[3125 [2.17k 2.36k 2.48k],],	[3150 [2.17k 2.36k 2.48k],],	[3175 [2.17k 2.36k 2.48k],],
[3200 [2.17k 2.36k 2.48k],],	[3225 [2.17k 2.36k 2.48k],],	[3250 [2.17k 2.36k 2.48k],],
[3275 [2.17k 2.36k 2.48k],],	[3300 [2.18k 2.41k 2.6k],],	[3325 [2.2k 2.41k 2.69k],],
[3350 [2.2k 2.41k 2.69k],],	[3375 [2.29k 2.43k 3.04k],],	[3400 [2.34k 2.52k 3.17k],],
[3425 [2.42k 2.78k 3.24k],],	[3450 [2.42k 2.78k 3.24k],],	[3475 [2.42k 2.78k 3.24k],],
[3500 [2.42k 2.78k 3.24k],],	[3525 [2.42k 2.78k 3.24k],],	[3550 [2.42k 2.78k 3.24k],],
[3575 [2.42k 2.78k 3.24k],],	[3600 [2.42k 2.96k 3.23k],],	[3625 [2.42k 2.96k 3.23k],],
[3650 [2.56k 3.12k 3.31k],],	[3675 [2.56k 3.12k 3.31k],],	[3700 [2.87k 3.23k 3.43k],],
[3725 [2.99k 3.26k 3.66k],],	[3750 [2.99k 3.26k 3.66k],],	[3775 [2.99k 3.26k 3.66k],],
[3800 [2.99k 3.26k 3.66k],],	[3825 [2.99k 3.26k 3.66k],],	[3850 [2.99k 3.26k 3.66k],],
[3875 [2.99k 3.26k 3.66k],],	[3900 [2.99k 3.26k 3.66k],],	[3925 [2.99k 3.26k 3.66k],],
[3950 [2.99k 3.26k 3.66k],],	[3975 [2.99k 3.26k 3.66k],],	[4000 [2.99k 3.26k 3.66k],],
[4025 [2.99k 3.26k 3.66k],],	[4050 [2.99k 3.26k 3.66k],],	[4075 [2.99k 3.26k 3.66k],],
[4100 [2.99k 3.26k 3.66k],],	[4125 [2.99k 3.26k 3.66k],],	[4150 [2.99k 3.26k 3.66k],],
[4175 [2.99k 3.26k 3.66k],],	[4200 [3.0k 3.29k 3.69k],],	[4225 [3.17k 3.33k 3.69k],],
[4250 [3.17k 3.33k 3.69k],],	[4275 [3.17k 3.33k 3.69k],],	[4300 [3.17k 3.33k 3.69k],],
[4325 [3.17k 3.33k 3.69k],],	[4350 [3.23k 3.36k 3.7k],],	[4375 [3.23k 3.36k 3.7k],],
[4400 [3.23k 3.5k 3.81k],],	[4425 [3.23k 3.5k 3.81k],],	[4450 [3.23k 3.5k 3.81k],],

[4475 [3.23k 3.5k 3.81k],],	[4500 [3.23k 3.5k 3.81k],],	[4525 [3.23k 3.67k 4.16k],],
[4550 [3.23k 3.67k 4.16k],],	[4575 [3.23k 3.67k 4.16k],],	[4600 [3.23k 3.69k 4.27k],],
[4625 [3.23k 3.7k 4.32k],],	[4650 [3.23k 3.7k 4.32k],],	[4675 [3.23k 3.7k 4.48k],],
[4700 [3.54k 3.91k 4.49k],],	[4725 [3.54k 3.91k 4.49k],],	[4750 [3.68k 4.2k 4.56k],],
[4775 [3.68k 4.2k 4.56k],],	[4800 [3.68k 4.2k 4.56k],],	[4825 [3.68k 4.2k 4.56k],],
[4850 [3.68k 4.2k 4.56k],],	[4875 [3.68k 4.2k 4.56k],],	[4900 [3.68k 4.2k 4.56k],],
[4925 [3.68k 4.2k 4.56k],],	[4950 [3.68k 4.2k 4.56k],],	[4975 [3.7k 4.37k 4.67k],],
[5000 [3.7k 4.37k 4.67k],],	[5025 [3.7k 4.37k 4.67k],],	[5050 [3.7k 4.37k 4.67k],],
[5075 [4.02k 4.5k 4.71k],],	[5100 [4.02k 4.5k 4.71k],],	[5125 [4.02k 4.5k 4.71k],],
[5150 [4.02k 4.5k 4.71k],],	[5175 [4.02k 4.5k 4.71k],],	[5200 [4.02k 4.5k 4.71k],],
[5225 [4.02k 4.5k 4.71k],],	[5250 [4.02k 4.5k 4.71k],],	[5275 [4.02k 4.5k 4.71k],],
[5300 [4.02k 4.5k 4.71k],],	[5325 [4.02k 4.5k 4.71k],],	[5350 [4.02k 4.5k 4.71k],],
[5375 [4.02k 4.5k 4.71k],],	[5400 [4.02k 4.5k 4.71k],],	[5425 [4.02k 4.5k 4.71k],],
[5450 [4.02k 4.5k 4.71k],],	[5475 [4.02k 4.5k 4.71k],],	[5500 [4.02k 4.5k 4.71k],],
[5525 [4.02k 4.5k 4.71k],],	[5550 [4.02k 4.5k 4.71k],],	[5575 [4.02k 4.5k 4.71k],],
[5600 [4.02k 4.5k 4.71k],],	[5625 [4.23k 4.59k 4.87k],],	[5650 [4.23k 4.59k 4.87k],],
[5675 [4.43k 4.67k 5.21k],],	[5700 [4.43k 4.67k 5.21k],],	[5725 [4.43k 4.67k 5.21k],],
[5750 [4.43k 4.67k 5.21k],],	[5775 [4.43k 4.67k 5.21k],],	[5800 [4.43k 4.67k 5.21k],],
[5825 [4.43k 4.67k 5.21k],],	[5850 [4.43k 4.67k 5.21k],],	[5875 [4.43k 4.67k 5.21k],],
[5900 [4.63k 4.94k 5.67k],],	[5925 [4.67k 5.35k 5.74k],],	[5950 [4.67k 5.35k 5.74k],],
[5975 [4.67k 5.35k 5.74k],],	[6000 [4.67k 5.35k 5.74k],],	[6025 [4.67k 5.35k 5.74k],],
[6050 [4.67k 5.35k 5.74k],],	[6075 [4.67k 5.35k 5.74k],],	[6100 [4.67k 5.35k 5.74k],],
[6125 [4.67k 5.35k 5.74k],],	[6150 [4.67k 5.35k 5.74k],],	[6175 [4.67k 5.35k 5.74k],],
[6200 [4.67k 5.35k 5.74k],],	[6225 [4.67k 5.35k 5.74k],],	[6250 [4.67k 5.35k 5.74k],],
[6275 [4.67k 5.35k 5.74k],],	[6300 [4.67k 5.35k 5.74k],],	[6325 [4.67k 5.35k 5.74k],],
[6350 [4.67k 5.35k 5.74k],],	[6375 [4.67k 5.35k 5.74k],],	[6400 [4.67k 5.35k 5.74k],],
[6425 [4.67k 5.35k 5.74k],],	[6450 [4.67k 5.35k 5.74k],],	[6475 [4.67k 5.35k 5.74k],],
[6500 [4.67k 5.35k 5.74k],],	[6525 [4.67k 5.35k 5.74k],],	[6550 [4.67k 5.35k 5.74k],],
[6575 [4.67k 5.35k 5.74k],],	[6600 [4.67k 5.35k 5.74k],],	[6625 [4.67k 5.35k 5.74k],],
[6650 [4.67k 5.35k 5.74k],],	[6675 [4.67k 5.35k 5.74k],],	[6700 [4.67k 5.35k 5.74k],],
[6725 [4.67k 5.35k 5.74k],],	[6750 [4.67k 5.35k 5.74k],],	[6775 [4.67k 5.35k 5.74k],],
[6800 [4.67k 5.35k 5.74k],],	[6825 [4.67k 5.35k 5.74k],],	[6850 [4.67k 5.35k 5.74k],],
[6875 [4.67k 5.35k 5.74k],],	[6900 [4.67k 5.35k 5.74k],],	[6925 [4.77k 5.64k 5.85k],],
[6950 [4.77k 5.64k 5.85k],],	[6975 [4.77k 5.64k 5.85k],],	[7000 [4.77k 5.64k 5.85k],],
[7025 [4.77k 5.64k 5.85k],],	[7050 [4.77k 5.64k 5.85k],],	[7075 [4.77k 5.64k 5.85k],],
[7100 [4.77k 5.64k 5.85k],],	[7125 [4.77k 5.64k 5.85k],],	[7150 [4.77k 5.64k 5.85k],],
[7175 [4.77k 5.64k 5.85k],],	[7200 [4.81k 5.65k 5.92k],],	[7225 [5.0k 5.68k 6.17k],],
[7250 [5.0k 5.68k 6.17k],],	[7275 [5.0k 5.68k 6.17k],],	[7300 [5.07k 5.71k 6.91k],],
[7325 [5.49k 5.77k 6.94k],],	[7350 [5.49k 5.77k 6.94k],],	[7375 [5.49k 5.77k 6.94k],],
[7400 [5.49k 5.77k 6.94k],],	[7425 [5.49k 5.77k 6.94k],],	[7450 [5.49k 5.77k 6.94k],],
[7475 [5.49k 5.77k 6.94k],],	[7500 [5.49k 5.77k 6.94k],],	[7525 [5.49k 5.77k 6.94k],],
[7550 [5.49k 5.77k 6.94k],],	[7575 [5.49k 5.77k 6.94k],],	[7600 [5.49k 5.77k 6.94k],],
[7625 [5.49k 5.77k 6.94k],],	[7650 [5.49k 5.77k 6.94k],],	[7675 [5.49k 5.77k 6.94k],],
[7700 [5.49k 5.77k 6.94k],],	[7725 [5.49k 5.77k 6.94k],],	[7750 [5.49k 5.77k 6.94k],],
[7775 [5.49k 5.77k 6.94k],],	[7800 [5.49k 5.77k 6.94k],],	[7825 [5.49k 5.77k 6.94k],],
[7850 [5.49k 5.77k 6.94k],],	[7875 [5.64k 5.88k 6.94k],],	[7900 [5.65k 5.92k 7.02k],],
[7925 [5.7k 6.29k 7.08k],],	[7950 [5.7k 6.29k 7.08k],],	[7975 [5.7k 6.29k 7.08k],],
[8000 [5.7k 6.29k 7.08k],],	[8025 [5.7k 6.29k 7.08k],],	[8050 [5.7k 6.29k 7.08k],],
[8075 [5.7k 6.29k 7.08k],],	[8100 [5.7k 6.29k 7.08k],],	[8125 [5.7k 6.29k 7.08k],],
[8150 [5.7k 6.29k 7.08k],],	[8175 [5.7k 6.29k 7.08k],],	[8200 [5.7k 6.29k 7.08k],],
[8225 [5.7k 6.29k 7.08k],],	[8250 [5.7k 6.29k 7.08k],],	[8275 [5.7k 6.29k 7.08k],],


```

|[8300|[5.8k|6.78k|7.4k], ], |[8325|[5.9k|6.97k|7.95k], ], |[8350|[5.9k|6.97k|7.95k], ],
|[8375|[5.9k|6.97k|7.95k], ], |[8400|[5.9k|6.97k|7.95k], ], |[8425|[5.9k|6.97k|7.95k], ],
|[8450|[5.9k|6.97k|7.95k], ], |[8475|[5.9k|6.97k|7.95k], ], |[8500|[5.9k|6.97k|7.95k], ],
|[8525|[5.9k|6.97k|7.95k], ], |[8550|[5.9k|6.97k|7.95k], ], |[8575|[5.9k|6.97k|7.95k], ],
|[8600|[5.9k|6.97k|7.95k], ], |[8625|[5.9k|6.97k|7.95k], ], |[8650|[5.9k|6.97k|7.95k], ],
|[8675|[5.9k|6.97k|7.95k], ], |[8700|[5.9k|6.97k|7.95k], ], |[8725|[5.9k|6.97k|7.95k], ],
|[8750|[5.9k|6.97k|7.95k], ], |[8775|[5.9k|6.97k|7.95k], ], |[8800|[5.9k|6.97k|7.95k], ],
|[8825|[5.9k|6.97k|7.95k], ], |[8850|[5.9k|6.97k|7.95k], ], |[8875|[5.9k|6.97k|7.95k], ],
|[8900|[5.9k|6.97k|7.95k], ], |[8925|[5.9k|6.97k|7.95k], ], |[8950|[5.9k|6.97k|7.95k], ],
|[8975|[5.9k|6.97k|7.95k], ], |[9000|[5.92k|7.02k|8.17k], ], |[9025|[6.47k|7.13k|8.2k], ],
|[9050|[6.47k|7.13k|8.2k], ], |[9075|[6.47k|7.13k|8.2k], ], |[9100|[6.47k|7.13k|8.2k], ],
|[9125|[6.47k|7.13k|8.2k], ], |[9150|[6.47k|7.13k|8.2k], ], |[9175|[6.47k|7.13k|8.2k], ],
|[9200|[6.47k|7.13k|8.2k], ], |[9225|[6.65k|7.24k|8.29k], ], |[9250|[6.85k|7.56k|8.45k], ],
|[9275|[6.85k|7.56k|8.45k], ], |[9300|[6.85k|7.56k|8.45k], ], |[9325|[6.85k|7.56k|8.45k], ],
|[9350|[6.85k|7.56k|8.45k], ], |[9375|[6.85k|7.56k|8.45k], ], |[9400|[6.99k|8.02k|8.95k], ],
|[9425|[7.02k|8.17k|9.02k], ], |[9450|[7.18k|8.23k|9.11k], ], |[9475|[7.18k|8.23k|9.11k], ],
|[9500|[7.18k|8.23k|9.11k], ], |[9525|[7.18k|8.23k|9.11k], ], |[9550|[7.72k|8.61k|9.38k], ],
|[9575|[7.72k|8.61k|9.38k], ], |[9600|[7.72k|8.61k|9.38k], ], |[9625|[7.72k|8.61k|9.38k], ],
|[9650|[7.72k|8.61k|9.38k], ], |[9675|[7.72k|8.61k|9.38k], ], |[9700|[7.72k|8.61k|9.38k], ],
|[9725|[7.72k|8.61k|9.38k], ], |[9750|[7.72k|8.61k|9.38k], ], |[9775|[7.72k|8.61k|9.38k], ],
|[9800|[7.72k|8.61k|9.38k], ], |[9825|[7.72k|8.61k|9.38k], ], |[9850|[7.72k|8.61k|9.38k], ],
|[9875|[7.72k|8.61k|9.38k], ], |[9900|[7.72k|8.61k|9.38k], ], |[9925|[7.72k|8.61k|9.38k], ],
|[9950|[7.72k|8.61k|9.38k], ], |[9975|[7.72k|8.61k|9.38k], ], |[10000|[7.72k|8.61k|9.38k], ], ],
//////////
//////////
//////////
[[25|[25.0|25.0|25.0], ], |[50|[50.0|50.0|50.0], ], |[75|[75.0|75.0|75.0], ],
|[100|[100.0|100.0|100.0], ], |[125|[79.8|125.0|125.0], ], |[150|[103.0|150.0|150.0], ],
|[175|[123.0|175.0|175.0], ], |[200|[147.0|200.0|200.0], ], |[225|[166.5|225.0|225.0], ],
|[250|[188.5|250.0|250.0], ], |[275|[209.8|275.0|275.0], ], |[300|[234.0|300.0|300.0], ],
|[325|[258.2|325.0|325.0], ], |[350|[283.0|350.0|350.0], ], |[375|[308.0|375.0|375.0], ],
|[400|[330.8|400.0|400.0], ], |[425|[351.2|425.0|425.0], ], |[450|[378.5|450.0|450.0], ],
|[475|[401.2|475.0|475.0], ], |[500|[433.0|500.0|500.0], ], |[525|[453.5|525.0|525.0], ],
|[550|[483.0|550.0|550.0], ], |[575|[509.0|575.0|575.0], ], |[600|[533.5|600.0|600.0], ],
|[625|[559.0|625.0|625.0], ], |[650|[583.5|650.0|650.0], ], |[675|[601.2|675.0|675.0], ],
|[700|[624.0|700.0|700.0], ], |[725|[645.2|725.0|725.0], ], |[750|[668.5|750.0|750.0], ],
|[775|[693.5|775.0|775.0], ], |[800|[718.5|800.0|800.0], ], |[825|[743.5|825.0|825.0], ],
|[850|[768.5|850.0|850.0], ], |[875|[793.0|875.0|875.0], ], |[900|[818.0|900.0|900.0], ],
|[925|[843.0|925.0|925.0], ], |[950|[867.0|950.0|950.0], ], |[975|[891.2|975.0|975.0], ],
|[1000|[914.0|1.0k|1.0k], ], |[1025|[939.0|1.02k|1.02k], ], |[1050|[964.0|1.05k|1.05k], ],
|[1075|[990.5|1.08k|1.08k], ], |[1100|[1.01k|1.1k|1.1k], ], |[1125|[1.04k|1.12k|1.12k], ],
|[1150|[1.07k|1.15k|1.15k], ], |[1175|[1.09k|1.18k|1.18k], ], |[1200|[1.11k|1.2k|1.2k], ],
|[1225|[1.13k|1.22k|1.22k], ], |[1250|[1.16k|1.25k|1.25k], ], |[1275|[1.18k|1.28k|1.28k], ],
|[1300|[1.21k|1.3k|1.3k], ], |[1325|[1.23k|1.32k|1.32k], ], |[1350|[1.25k|1.35k|1.35k], ],
|[1375|[1.27k|1.38k|1.38k], ], |[1400|[1.29k|1.4k|1.4k], ], |[1425|[1.32k|1.42k|1.42k], ],
|[1450|[1.34k|1.45k|1.45k], ], |[1475|[1.37k|1.48k|1.48k], ], |[1500|[1.41k|1.5k|1.5k], ],
|[1525|[1.43k|1.52k|1.52k], ], |[1550|[1.46k|1.55k|1.55k], ], |[1575|[1.48k|1.58k|1.58k], ],
|[1600|[1.51k|1.6k|1.6k], ], |[1625|[1.53k|1.62k|1.62k], ], |[1650|[1.56k|1.65k|1.65k], ],
|[1675|[1.58k|1.68k|1.68k], ], |[1700|[1.61k|1.7k|1.7k], ], |[1725|[1.63k|1.72k|1.72k], ],
|[1750|[1.66k|1.75k|1.75k], ], |[1775|[1.68k|1.78k|1.78k], ], |[1800|[1.71k|1.8k|1.8k], ],
|[1825|[1.73k|1.82k|1.82k], ], |[1850|[1.76k|1.85k|1.85k], ], |[1875|[1.78k|1.88k|1.88k], ],

```

|[1900|[1.81k|1.9k|1.9k],], |[1925|[1.83k|1.92k|1.92k],], |[1950|[1.86k|1.95k|1.95k],],
 |[1975|[1.88k|1.98k|1.98k],], |[2000|[1.91k|2.0k|2.0k],], |[2025|[1.93k|2.02k|2.02k],],
 |[2050|[1.96k|2.05k|2.05k],], |[2075|[1.98k|2.08k|2.08k],], |[2100|[2.01k|2.1k|2.1k],],
 |[2125|[2.04k|2.12k|2.12k],], |[2150|[2.07k|2.15k|2.15k],], |[2175|[2.09k|2.18k|2.18k],],
 |[2200|[2.12k|2.2k|2.2k],], |[2225|[2.14k|2.22k|2.22k],], |[2250|[2.17k|2.25k|2.25k],],
 |[2275|[2.19k|2.28k|2.28k],], |[2300|[2.22k|2.3k|2.3k],], |[2325|[2.25k|2.32k|2.32k],],
 |[2350|[2.27k|2.35k|2.35k],], |[2375|[2.3k|2.38k|2.38k],], |[2400|[2.32k|2.4k|2.4k],],
 |[2425|[2.35k|2.42k|2.42k],], |[2450|[2.37k|2.45k|2.45k],], |[2475|[2.4k|2.48k|2.48k],],
 |[2500|[2.42k|2.5k|2.5k],], |[2525|[2.45k|2.52k|2.52k],], |[2550|[2.47k|2.55k|2.55k],],
 |[2575|[2.51k|2.58k|2.58k],], |[2600|[2.53k|2.6k|2.6k],], |[2625|[2.55k|2.62k|2.62k],],
 |[2650|[2.58k|2.65k|2.65k],], |[2675|[2.6k|2.68k|2.68k],], |[2700|[2.63k|2.7k|2.7k],],
 |[2725|[2.65k|2.72k|2.72k],], |[2750|[2.68k|2.75k|2.75k],], |[2775|[2.7k|2.78k|2.78k],],
 |[2800|[2.73k|2.8k|2.8k],], |[2825|[2.75k|2.82k|2.82k],], |[2850|[2.78k|2.85k|2.85k],],
 |[2875|[2.8k|2.88k|2.88k],], |[2900|[2.83k|2.9k|2.9k],], |[2925|[2.85k|2.92k|2.92k],],
 |[2950|[2.88k|2.95k|2.95k],], |[2975|[2.9k|2.98k|2.98k],], |[3000|[2.93k|3.0k|3.0k],],
 |[3025|[2.95k|3.02k|3.02k],], |[3050|[2.98k|3.05k|3.05k],], |[3075|[3.0k|3.08k|3.08k],],
 |[3100|[3.03k|3.1k|3.1k],], |[3125|[3.05k|3.12k|3.12k],], |[3150|[3.08k|3.15k|3.15k],],
 |[3175|[3.1k|3.18k|3.18k],], |[3200|[3.13k|3.2k|3.2k],], |[3225|[3.15k|3.22k|3.22k],],
 |[3250|[3.18k|3.25k|3.25k],], |[3275|[3.2k|3.28k|3.28k],], |[3300|[3.22k|3.3k|3.3k],],
 |[3325|[3.25k|3.32k|3.32k],], |[3350|[3.27k|3.35k|3.35k],], |[3375|[3.29k|3.38k|3.38k],],
 |[3400|[3.33k|3.4k|3.4k],], |[3425|[3.36k|3.42k|3.42k],], |[3450|[3.38k|3.45k|3.45k],],
 |[3475|[3.41k|3.48k|3.48k],], |[3500|[3.43k|3.5k|3.5k],], |[3525|[3.46k|3.52k|3.52k],],
 |[3550|[3.48k|3.55k|3.55k],], |[3575|[3.51k|3.58k|3.58k],], |[3600|[3.53k|3.6k|3.6k],],
 |[3625|[3.56k|3.62k|3.62k],], |[3650|[3.58k|3.65k|3.65k],], |[3675|[3.61k|3.68k|3.68k],],
 |[3700|[3.63k|3.7k|3.7k],], |[3725|[3.66k|3.72k|3.72k],], |[3750|[3.68k|3.75k|3.75k],],
 |[3775|[3.71k|3.78k|3.78k],], |[3800|[3.73k|3.8k|3.8k],], |[3825|[3.76k|3.82k|3.82k],],
 |[3850|[3.78k|3.85k|3.85k],], |[3875|[3.81k|3.88k|3.88k],], |[3900|[3.83k|3.9k|3.9k],],
 |[3925|[3.86k|3.92k|3.92k],], |[3950|[3.88k|3.95k|3.95k],], |[3975|[3.91k|3.98k|3.98k],],
 |[4000|[3.93k|4.0k|4.0k],], |[4025|[3.96k|4.02k|4.02k],], |[4050|[3.98k|4.05k|4.05k],],
 |[4075|[4.01k|4.08k|4.08k],], |[4100|[4.03k|4.1k|4.1k],], |[4125|[4.06k|4.12k|4.12k],],
 |[4150|[4.08k|4.15k|4.15k],], |[4175|[4.11k|4.18k|4.18k],], |[4200|[4.13k|4.2k|4.2k],],
 |[4225|[4.16k|4.22k|4.22k],], |[4250|[4.18k|4.25k|4.25k],], |[4275|[4.21k|4.28k|4.28k],],
 |[4300|[4.23k|4.3k|4.3k],], |[4325|[4.26k|4.32k|4.32k],], |[4350|[4.28k|4.35k|4.35k],],
 |[4375|[4.31k|4.38k|4.38k],], |[4400|[4.33k|4.4k|4.4k],], |[4425|[4.36k|4.42k|4.42k],],
 |[4450|[4.38k|4.45k|4.45k],], |[4475|[4.41k|4.48k|4.48k],], |[4500|[4.43k|4.5k|4.5k],],
 |[4525|[4.46k|4.52k|4.52k],], |[4550|[4.48k|4.55k|4.55k],], |[4575|[4.51k|4.58k|4.58k],],
 |[4600|[4.54k|4.6k|4.6k],], |[4625|[4.56k|4.62k|4.62k],], |[4650|[4.58k|4.65k|4.65k],],
 |[4675|[4.61k|4.68k|4.68k],], |[4700|[4.63k|4.7k|4.7k],], |[4725|[4.66k|4.72k|4.72k],],
 |[4750|[4.68k|4.75k|4.75k],], |[4775|[4.71k|4.78k|4.78k],], |[4800|[4.73k|4.8k|4.8k],],
 |[4825|[4.76k|4.82k|4.82k],], |[4850|[4.78k|4.85k|4.85k],], |[4875|[4.81k|4.88k|4.88k],],
 |[4900|[4.83k|4.9k|4.9k],], |[4925|[4.86k|4.92k|4.92k],], |[4950|[4.88k|4.95k|4.95k],],
 |[4975|[4.91k|4.98k|4.98k],], |[5000|[4.94k|5.0k|5.0k],], |[5025|[4.96k|5.02k|5.02k],],
 |[5050|[4.99k|5.05k|5.05k],], |[5075|[5.01k|5.08k|5.08k],], |[5100|[5.04k|5.1k|5.1k],],
 |[5125|[5.06k|5.12k|5.12k],], |[5150|[5.08k|5.15k|5.15k],], |[5175|[5.11k|5.18k|5.18k],],
 |[5200|[5.14k|5.2k|5.2k],], |[5225|[5.16k|5.22k|5.22k],], |[5250|[5.18k|5.25k|5.25k],],
 |[5275|[5.21k|5.28k|5.28k],], |[5300|[5.24k|5.3k|5.3k],], |[5325|[5.26k|5.32k|5.32k],],
 |[5350|[5.28k|5.35k|5.35k],], |[5375|[5.31k|5.38k|5.38k],], |[5400|[5.34k|5.4k|5.4k],],
 |[5425|[5.36k|5.42k|5.42k],], |[5450|[5.38k|5.45k|5.45k],], |[5475|[5.41k|5.48k|5.48k],],
 |[5500|[5.44k|5.5k|5.5k],], |[5525|[5.46k|5.52k|5.52k],], |[5550|[5.48k|5.55k|5.55k],],
 |[5575|[5.51k|5.58k|5.58k],], |[5600|[5.54k|5.6k|5.6k],], |[5625|[5.56k|5.62k|5.62k],],
 |[5650|[5.58k|5.65k|5.65k],], |[5675|[5.61k|5.68k|5.68k],], |[5700|[5.63k|5.7k|5.7k],],

|[5725|[5.66k|5.72k|5.72k],], |[5750|[5.68k|5.75k|5.75k],], |[5775|[5.71k|5.78k|5.78k],],
 |[5800|[5.73k|5.8k|5.8k],], |[5825|[5.76k|5.82k|5.82k],], |[5850|[5.78k|5.85k|5.85k],],
 |[5875|[5.81k|5.88k|5.88k],], |[5900|[5.84k|5.9k|5.9k],], |[5925|[5.86k|5.92k|5.92k],],
 |[5950|[5.89k|5.95k|5.95k],], |[5975|[5.91k|5.98k|5.98k],], |[6000|[5.94k|6.0k|6.0k],],
 |[6025|[5.96k|6.02k|6.02k],], |[6050|[5.99k|6.05k|6.05k],], |[6075|[6.01k|6.08k|6.08k],],
 |[6100|[6.04k|6.1k|6.1k],], |[6125|[6.06k|6.12k|6.12k],], |[6150|[6.09k|6.15k|6.15k],],
 |[6175|[6.11k|6.18k|6.18k],], |[6200|[6.14k|6.2k|6.2k],], |[6225|[6.16k|6.22k|6.22k],],
 |[6250|[6.19k|6.25k|6.25k],], |[6275|[6.21k|6.28k|6.28k],], |[6300|[6.24k|6.3k|6.3k],],
 |[6325|[6.26k|6.32k|6.32k],], |[6350|[6.29k|6.35k|6.35k],], |[6375|[6.31k|6.38k|6.38k],],
 |[6400|[6.34k|6.4k|6.4k],], |[6425|[6.36k|6.42k|6.42k],], |[6450|[6.39k|6.45k|6.45k],],
 |[6475|[6.41k|6.48k|6.48k],], |[6500|[6.44k|6.5k|6.5k],], |[6525|[6.46k|6.52k|6.52k],],
 |[6550|[6.49k|6.55k|6.55k],], |[6575|[6.51k|6.58k|6.58k],], |[6600|[6.54k|6.6k|6.6k],],
 |[6625|[6.56k|6.62k|6.62k],], |[6650|[6.59k|6.65k|6.65k],], |[6675|[6.61k|6.68k|6.68k],],
 |[6700|[6.64k|6.7k|6.7k],], |[6725|[6.66k|6.72k|6.72k],], |[6750|[6.69k|6.75k|6.75k],],
 |[6775|[6.71k|6.78k|6.78k],], |[6800|[6.74k|6.8k|6.8k],], |[6825|[6.76k|6.82k|6.82k],],
 |[6850|[6.79k|6.85k|6.85k],], |[6875|[6.81k|6.88k|6.88k],], |[6900|[6.84k|6.9k|6.9k],],
 |[6925|[6.86k|6.92k|6.92k],], |[6950|[6.89k|6.95k|6.95k],], |[6975|[6.91k|6.98k|6.98k],],
 |[7000|[6.94k|7.0k|7.0k],], |[7025|[6.96k|7.02k|7.02k],], |[7050|[6.99k|7.05k|7.05k],],
 |[7075|[7.01k|7.08k|7.08k],], |[7100|[7.04k|7.1k|7.1k],], |[7125|[7.06k|7.12k|7.12k],],
 |[7150|[7.09k|7.15k|7.15k],], |[7175|[7.11k|7.18k|7.18k],], |[7200|[7.14k|7.2k|7.2k],],
 |[7225|[7.16k|7.22k|7.22k],], |[7250|[7.19k|7.25k|7.25k],], |[7275|[7.21k|7.28k|7.28k],],
 |[7300|[7.24k|7.3k|7.3k],], |[7325|[7.26k|7.32k|7.32k],], |[7350|[7.29k|7.35k|7.35k],],
 |[7375|[7.31k|7.38k|7.38k],], |[7400|[7.34k|7.4k|7.4k],], |[7425|[7.36k|7.42k|7.42k],],
 |[7450|[7.39k|7.45k|7.45k],], |[7475|[7.41k|7.48k|7.48k],], |[7500|[7.44k|7.5k|7.5k],],
 |[7525|[7.46k|7.52k|7.52k],], |[7550|[7.49k|7.55k|7.55k],], |[7575|[7.51k|7.58k|7.58k],],
 |[7600|[7.54k|7.6k|7.6k],], |[7625|[7.56k|7.62k|7.62k],], |[7650|[7.59k|7.65k|7.65k],],
 |[7675|[7.61k|7.68k|7.68k],], |[7700|[7.64k|7.7k|7.7k],], |[7725|[7.66k|7.72k|7.72k],],
 |[7750|[7.69k|7.75k|7.75k],], |[7775|[7.71k|7.78k|7.78k],], |[7800|[7.74k|7.8k|7.8k],],
 |[7825|[7.76k|7.82k|7.82k],], |[7850|[7.79k|7.85k|7.85k],], |[7875|[7.82k|7.88k|7.88k],],
 |[7900|[7.84k|7.9k|7.9k],], |[7925|[7.86k|7.92k|7.92k],], |[7950|[7.89k|7.95k|7.95k],],
 |[7975|[7.92k|7.98k|7.98k],], |[8000|[7.94k|8.0k|8.0k],], |[8025|[7.96k|8.02k|8.02k],],
 |[8050|[7.99k|8.05k|8.05k],], |[8075|[8.02k|8.08k|8.08k],], |[8100|[8.04k|8.1k|8.1k],],
 |[8125|[8.06k|8.12k|8.12k],], |[8150|[8.09k|8.15k|8.15k],], |[8175|[8.12k|8.18k|8.18k],],
 |[8200|[8.14k|8.2k|8.2k],], |[8225|[8.16k|8.22k|8.22k],], |[8250|[8.19k|8.25k|8.25k],],
 |[8275|[8.22k|8.28k|8.28k],], |[8300|[8.24k|8.3k|8.3k],], |[8325|[8.27k|8.33k|8.33k],],
 |[8350|[8.30k|8.36k|8.36k],], |[8375|[8.33k|8.39k|8.39k],], |[8400|[8.36k|8.42k|8.42k],],
 |[8425|[8.39k|8.45k|8.45k],], |[8450|[8.42k|8.48k|8.48k],], |[8475|[8.45k|8.51k|8.51k],],
 |[8500|[8.48k|8.54k|8.54k],], |[8525|[8.51k|8.57k|8.57k],], |[8550|[8.54k|8.6k|8.6k],],
 |[8575|[8.57k|8.63k|8.63k],], |[8600|[8.6k|8.66k|8.66k],], |[8625|[8.63k|8.69k|8.69k],],
 |[8650|[8.66k|8.72k|8.72k],], |[8675|[8.69k|8.75k|8.75k],], |[8700|[8.72k|8.78k|8.78k],],
 |[8725|[8.75k|8.81k|8.81k],], |[8750|[8.78k|8.84k|8.84k],], |[8775|[8.81k|8.87k|8.87k],],
 |[8800|[8.84k|8.9k|8.9k],], |[8825|[8.87k|8.93k|8.93k],], |[8850|[8.9k|8.96k|8.96k],],
 |[8875|[8.93k|8.99k|8.99k],], |[8900|[8.96k|9.02k|9.02k],], |[8925|[8.99k|9.05k|9.05k],],
 |[8950|[9.02k|9.08k|9.08k],], |[8975|[9.05k|9.11k|9.11k],], |[9000|[9.08k|9.14k|9.14k],],
 |[9025|[9.11k|9.17k|9.17k],], |[9050|[9.14k|9.2k|9.2k],], |[9075|[9.17k|9.23k|9.23k],],
 |[9100|[9.2k|9.26k|9.26k],], |[9125|[9.23k|9.29k|9.29k],], |[9150|[9.26k|9.32k|9.32k],],
 |[9175|[9.29k|9.35k|9.35k],], |[9200|[9.32k|9.38k|9.38k],], |[9225|[9.35k|9.41k|9.41k],],
 |[9250|[9.38k|9.44k|9.44k],], |[9275|[9.41k|9.47k|9.47k],], |[9300|[9.44k|9.5k|9.5k],],
 |[9325|[9.47k|9.53k|9.53k],], |[9350|[9.5k|9.56k|9.56k],], |[9375|[9.53k|9.59k|9.59k],],
 |[9400|[9.56k|9.62k|9.62k],], |[9425|[9.59k|9.65k|9.65k],], |[9450|[9.62k|9.68k|9.68k],],
 |[9475|[9.65k|9.71k|9.71k],], |[9500|[9.68k|9.74k|9.74k],], |[9525|[9.71k|9.77k|9.77k],],
 |[9550|[9.74k|9.8k|9.8k],], |[9575|[9.77k|9.83k|9.83k],], |[9600|[9.8k|9.86k|9.86k],],
 |[9625|[9.83k|9.89k|9.89k],], |[9650|[9.86k|9.92k|9.92k],], |[9675|[9.89k|9.95k|9.95k],],
 |[9700|[9.92k|9.98k|9.98k],], |[9725|[9.95k|10.0k|10.0k],], |[9750|[9.98k|10.0k|10.0k],],
 |[9775|[10.0k|10.0k|10.0k],], |[9800|[10.0k|10.0k|10.0k],], |[9825|[10.0k|10.0k|10.0k],],
 |[9850|[10.0k|10.0k|10.0k],], |[9875|[10.0k|10.0k|10.0k],], |[9900|[10.0k|10.0k|10.0k],],
 |[9925|[10.0k|10.0k|10.0k],], |[9950|[10.0k|10.0k|10.0k],], |[9975|[10.0k|10.0k|10.0k],],
 |[10000|[10.0k|10.0k|10.0k],],

As a result of these tests, I decided to increase the damage taken by senescent animas and make animas gain mating drive each tick so that it will not be outscaled by hunger and then unable to be around similar levels. Apples also increase mating drive when consumed; I here reward animas able to find food and punish animas simply aiming to live long enough to reproduce without attempting to reach the other metric of fitness: finding food. From this, we can see in subsequent tests that there is more reproduction and variance (changes implemented during UI development phase)

Next: predators can boost if hungry

Memory- creatures can remember encountered cells and events, can influence goals and behaviour, remembered scents also mean can track specific creatures (beanstalk eaters can remember where the beanstalk is)

Should rename self.elements to self.life or eq

****This is the post-phase testing, but there has been abundant testing throughout development which can be seen in the previous section. The purpose of this section is to formalise testing which was completed after the phase's development was finalised in order to ensure the complete package functions as intended; a more holistic approach will be taken****

Test type	Test statements	Test data	Expected Result	Actual Result	Pass / Fail ?
	<code>print(len(testRealm.spatialhash.buckets))</code>	--	25	25	PASS
	<code>print(vars(self))</code>	--	Dictionary of animal object attribute names and values	EX: { 'realm': <realm.Realm object at 0x0000012585AFB0E0>, 'x': 56.427002356333915, 'y': 16.375869950633515, 'bucket': -1, 'id': 23.	PASS

				<pre> 'angle': 3.94312951922210 48, 'roam_angle': 4.60292595043898 1, 'dx': 0, 'dy': 0, 'status': [0, 0, 0, 0, 28.2666078857866], 'target': None, 'action': [<bound method Anima.do_nothing of <anima.Anima object at 0x0000012585AFB8 C0>>, <bound method Anima.do_nothing of <anima.Anima object at 0x0000012585AFB8 C0>>], 'rejectedList': [], 'eatenList': [], 'offspring': [], 'parents': [], 'goals': [], 'mode': 1, 'locked': False, 'genome': [[None], [6.5030346662758 55, 0.53044552015415 95, 0.29654988956303 24], 17.4148803184493 88, 8.59853305458999 6, 9.26606980307649 , 11.0493984278781 38, 28.2666078857866], 'diet': [None], 'speed': 6.50303466627585 5, 'amphibian_facto r': 0.53044552015415 95, 'ice_factor': 0.29654988956303 24, 'turnspeed': </pre>	
--	--	--	--	--	--

				17.4148803184493 88, 'vision': 8.59853305458999 6, 'strength': 9.26606980307649 , 'toughness': 11.0493984278781 38, 'size': 20.3154682309546 3, 'reach': 100, 'food_given': 'meat', 'food_effect': [- 20.3154682309546 3, 0, 0, 0]} genome	
	<pre> print([[anima.distance_to(anima.target), anima.goals, anima.target, (anima.vision+anima.speed+anima.target.speed>= anima.distance_to(anima.target))] for anima in testRealm.elements[1] if anima.target != None]) #print([[anima.round_status(),round (anima.x), round(anima.y), round(anima.dx), round(anima.dy), anima.mode, anima.target, anima.goals] for anima in testRealm.elements[1]]) #anima.status, anima.genome,anima.x, anima.y, anima.dx, anima.dy, anima.mode, anima.target, anima.goals #print([flora.fruitNo for flora in testRealm.elements[0]]) </pre>	--	List of acceptable values, boolean value should be True	EX [[0.3228746944837181, [4, 1, 0, 2, 3], <anima.Anima object at 0x000002077D1AC590>, True], [0.3228746944837181, [4], <anima.Anima object at 0x000002077D0FBCB0>, True]]	PA SS

Review

The program is now at a state in which animas can interact with the world's natural elements and each other. They can perform many basic actions such as eating, drinking, mating, fighting and sleeping using a logical decision flow which is intuitive based on the needs presented by their time-defined status. Animas can detect elements in their vision, moving in the world around them, and they can move naturally and smoothly while wandering, fleeing, or approaching another element.

When time is incremented, all animas act through several phases which coordinate all animas and ensure that the output is the same regardless of an anima's position in the list being iterated. These are the thinking, acting and response phases; an anima only interacts with the public realm outside of itself during the 'act' phase, and all actions are based on the results from the 'thinking' phase.

Animas mate and have offspring with a combination of their genes. This is important to my project's intent as the mating process is the cornerstone of evolution in animals. It drives a

population towards a more ideal set of traits by taking traits randomly from both parents to create a new mix of potentially fitness-increasing traits, to be tested by the environment

When animas die they become corpses edible by meat-eaters. When these fully decay the anima is moved to a 'ghostyard' in which they can no longer be accessed in-simulation.

The in-simulation logic is now complete, with further development only an extension for complexity.

I have **tested** the features of this development phase by running simulations using different inputs for the environment and for the initial anima traits. Print lines embedded in anima and realm methods can tell me what events are happening, and when. I observed the terminal console to see how the frequency and sequence of these lines vary with differing input, and to ensure consistency in output.

The totality of this phase takes a large step in meeting my **success criteria**, as I now have a fully developed simulation which abstracts natural selection in animals responding to an external environment. This simulation uses mathematical models to simulate interaction and evolutionary progress, and so far is simple enough that its UI presentation should be intuitive and clear.

PHASE 3~ UI

Design

In order to promote ease of use I will employ a simple UI and basic graphics; the stakeholders did not place too great an emphasis on dramatic visuals; clarity and communicability should be maximised.

I will need to use *pygame_gui* to develop the user interface; each element in the simulation is modelled as a circle for symmetry, with varying size.

The simulation will occupy a space on the window smaller than the total resolution, with the remaining space filled with UI elements.

The minimalistic UI will involve buttons and simple text informatics. For user friendliness there will not be an overload of text, but there will be an optional tutorial explaining the mechanics of how everything works, on both UI and simulation level.

Category	UI Element	Object implementation
Sound (SFX)	SFX Volume increase	Button
	SFX Volume decrease	Button
	SFX Volume control	Slider (in between 2 buttons)
	SFX Sound mute	Button
	SFX Sound unmute	Button
Sound (Music)	Music Pause/Play	Button (2x sprite)
	Music Volume increase	Button
	Music Volume decrease	Button
	Music Volume control	Slider (in between 2 buttons)
	Music Sound mute	Button
	Music Sound unmute	Button
	Music Next track	Button
	Music Previous track	Button
Display	Music Track display	Music display
	Brightness control	Slider
	Toggle hi-contrast	Button
	Language switch	Button xN
Help	Hide UI	Button
	Tutorial	Sequence of text boxes with arrows and coloured keywords for maximal effectiveness of help
Simulation	Anima	Each anima is a filled circle with hitbox checked every tick
	Anima vision	Vision is modelled as a circle of genetically defined radius and centre anima; hitbox checked every tick while anima awake
	Flora	Each plant is a filled circle with hitbox checked every tick
	NatElement	Each natural element is modelled as a circle

Meta-simulation	Speed x1	Button
	Speed x2	Button
	Speed x4	Button
	Speed x8	Button
	Pause/Play	Button (2x sprite)
	End simulation	Button
	Simulation set up	Frame with customisable options for the setup of the simulation; from here simulation is started, creating a new simulation object with parameters taking the values of user input

There will be a high-contrast mode for accessibility; this is a usability feature which will allow users with eyesight troubles to use the program to the same efficacy as anyone else. For non-English learners, there will be other language options available; I plan to include Mandarin and Tamil as alternative language options.

As another usability feature, the user will be able to select an anima they have taken an interest in, which will display its vision range as a background circle on the simulation map and create a display window with further information on the specific anima. This will require an algorithm to detect if the user has clicked in the simulation plane, close enough to any animas, and store the closest anima to the clicked pixel in the UI state shell.

The UI elements will have no causal bearing on the simulation logic; the in-simulation UI objects will supervene on the simulation game objects, and the extra-simulation UI objects will remain fixed in place, either around the border of the simulation window or inside it.

I researched the best ways to handle sound playback in python, with the basic requirements of playback, pause, restart, volume control and track handling. I found the *sounddevice* module which when paired with *pygame*, *numpy*, *threading*, *wave* and *time*, can fulfil these requirements. I considered using threading to allow the game loop and music loop to run independently in parallel, but if each game tick is kept consistently a single tight unit differentiated by states, then this should not be required. Real-time changing of volume introduces complexity beyond the other playback mechanics and I may have to scrap it if it poses too much of a challenge, as it also adds a good margin of processing to every tick beyond the simpler button checks, if volume may be real-time updated. After further research into pygame, it is clear that multiple updated areas in this case is not an issue and is a core tenet of the library. So each frame of the simulation will also come with updates to music, visual effects, and other UI responses.

Test Design

Testing will become much easier with this phase as bugs and output values will be observable from a dynamic simulation.

Testing will include checking if the following effects which should be in place due to developed features can be observed through the simulation:

	No	Yes
Animal eating food	☐	☐
Animal drinking water	☐	☐
Predator chasing a fleeing prey	☐	☐
Two animals fighting and taking damage	☐	☐
Plants bearing fruit	☐	☐
Animals mating and producing offspring	☐	☐
Animals dying from each of starvation, thirst and wounds	☐	☐
Population appearances and traits changing over time (evolution)	☐	☐

The various UI elements also need to be tested for functionality (the demo on page 1 of this document **<Error! Reference source not found.>** provides evidence of this functionality) These are the UI elements as listed here:

SFX Volume increase
SFX Volume decrease
SFX Volume control
SFX Sound mute
SFX Sound unmute
Music Pause/Play
Music Volume increase
Music Volume decrease
Music Volume control
Music Sound mute
Music Sound unmute
Music Next track
Music Previous track
Music Track display
Brightness control
Toggle hi-contrast
Language switch
Hide UI
Tutorial
Anima
Anima vision
Flora
NatElement
Speed x1
Speed x2
Speed x4

Speed x8
Pause/Play
End simulation
Simulation set up

Development & Testing

16/03:

I pip installed pygame and imported it to the relevant files. Using the official 'pygame.org' website I constructed my basic user interface, from the standard template of the game:

```
# Example file showing a basic pygame "game loop"
import pygame

# pygame setup
pygame.init()
screen = pygame.display.set_mode((1280, 720))
clock = pygame.time.Clock()
running = True

while running:
    # poll for events
    # pygame.QUIT event means the user clicked X to close your window
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    # fill the screen with a color to wipe away anything from last frame
    screen.fill("purple")

    # RENDER YOUR GAME HERE

    # flip() the display to put your work on screen
    pygame.display.flip()

    clock.tick(60) # limits FPS to 60

pygame.quit()
```

Here is my version:

```
pygame.init()
screen = pygame.display.set_mode((1280,720))
clock = pygame.time.Clock()
running = True

window = ((200, 200 + testRealm.width),(120, 120 + testRealm.height))
#vis_realm = Visual_Realm(window, testRealm)

blit = ui.draw_world_blit(testRealm, window)

while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    screen.fill("white")
    testRealm.database.update()

    mySim.tick()
    ui.tick_visual_realm(testRealm, screen, window, blit)
```

```

pygame.display.flip()
clock.tick(5)

pygame.quit()

```

Since I have written all my realm logic outside of pygame, all I need to do is to model it using pygame objects without requiring coding any of the internal behaviour.

20/03 & 21/03:

It turns out pygame is not supported by python 3.14. So I had to create a new virtual environment using python 3.11, updating pip as well and forcing a direct pygame implementation rather than building from scratch.

I wrote a working model of the code. At first I used placeholder colours (purple plants and green grass) and later pivoted to what can be seen below.

```

GRASS_COLOUR = "olivegreen"
FLORA_COLOUR = random.choice(["forestgreen"])

def draw_world_blit(realm, window):
    blit = pygame.Surface((window[0][1]-window[0][0], window[1][1]-
window[1][0]))
    blit.fill("green")
    for row in range(len(realm.environment)):
        for col in range(row):
            if realm.environment[row][col] == "water":
                pygame.draw.rect(blit, "blue", (col, row, 1, 1))
    for row in range(len(realm.environment)):
        for col in range(row):
            if realm.environment[row][col] == "ice":
                pygame.draw.rect(blit, "white", (col, row, 1, 1))
    return blit

def tick_visual_realm(realm, screen, window, blit):
    left = window[0][0]
    right = window[0][1]
    top = window[1][0]
    bottom = window[1][1]

    screen.blit(blit, (left,top))

    for flora in realm.elements[0]:
        pygame.draw.rect(screen, FLORA_COLOUR, (left + flora.x-1.5, top +
flora.y-1.5, 3, 3))

```

```

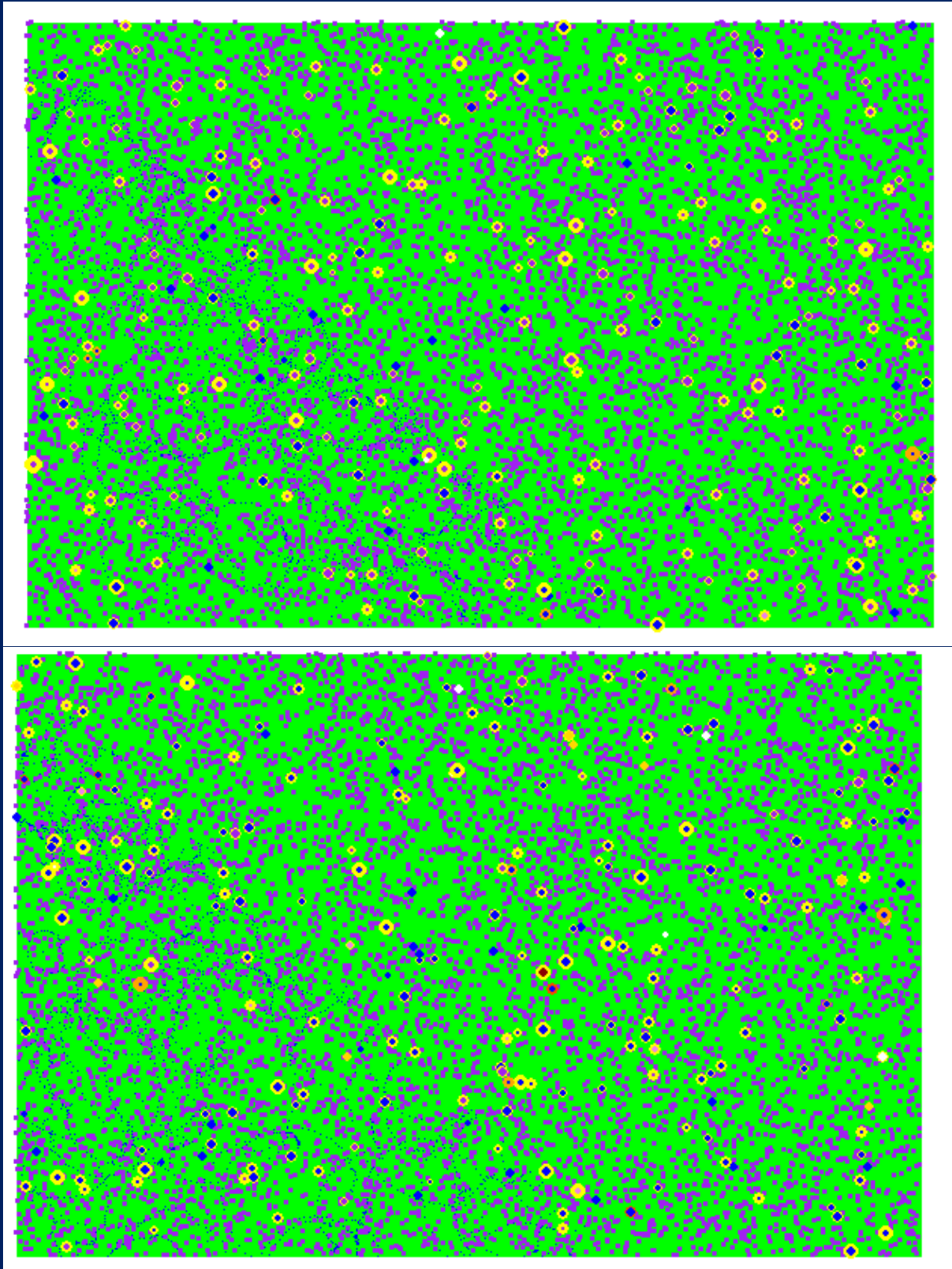
    for anima in realm.elements[1]:
        bodycolour = "orangered" if anima.is_injured() else "orange" if
anima.status[3] >= 25 else "white" if anima.is_senescent() else "turquoise1" if
not anima.is_mature() else "yellow"
        pygame.draw.circle(screen, bodycolour, (left + anima.x, top + anima.y),
min(max(sqrt(anima.size()), 1), 7))

        eyecolour = "gold" if anima.asleep else "black" if anima.mode == 2 and
"meat" in anima.diet and anima.action[1] == anima.eat else "purple" if
anima.mode == 2 else "white" if anima.mode == 3 else "violet" if 4 in anima.goals
else "blue"
        pygame.draw.circle(screen, eyecolour, (left + anima.x, top + anima.y),
min(max(sqrt(anima.vision/2), 1), min(max(sqrt(anima.size()), 1), 5)))
        #pygame.draw.circle(screen, "black", (left + flora.x, bottom +
flora.y), anima.vision, 1)

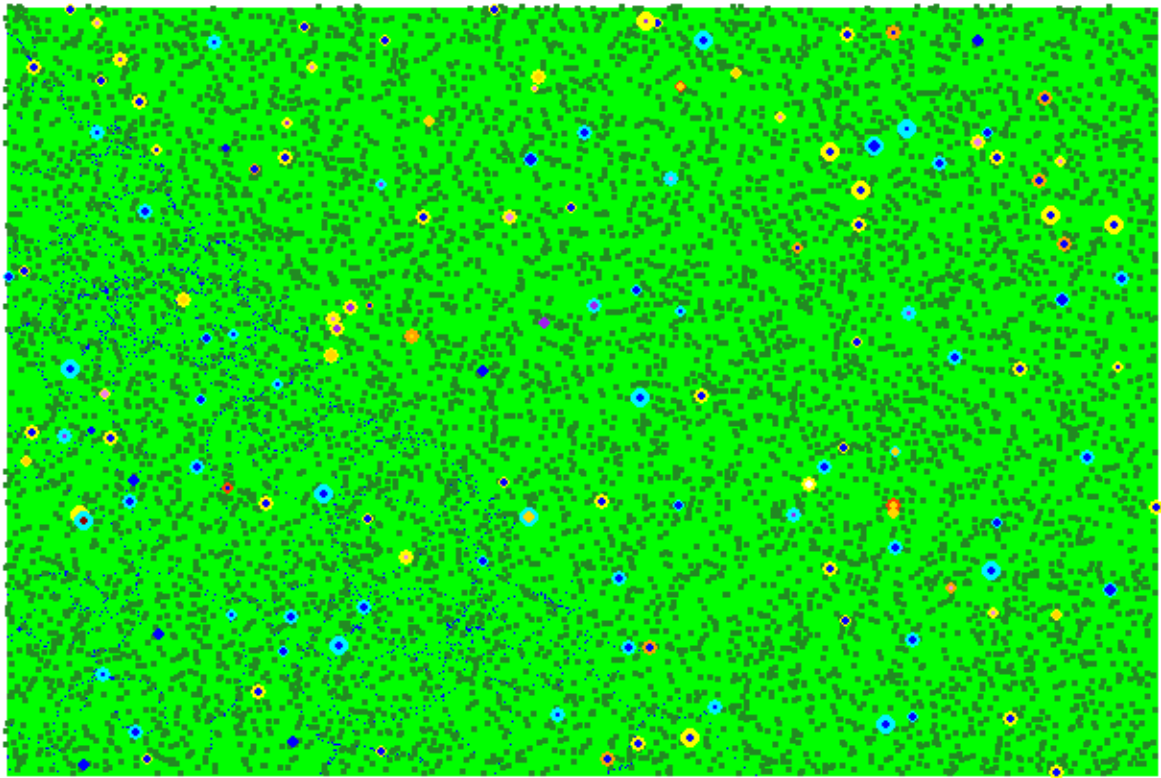
    for corpse in realm.graveyard[1]:
        pygame.draw.circle(screen, "red4", (left + anima.x, top + anima.y),
min(max(anima.food_chunks, 1), 5))

```

Placeholder version



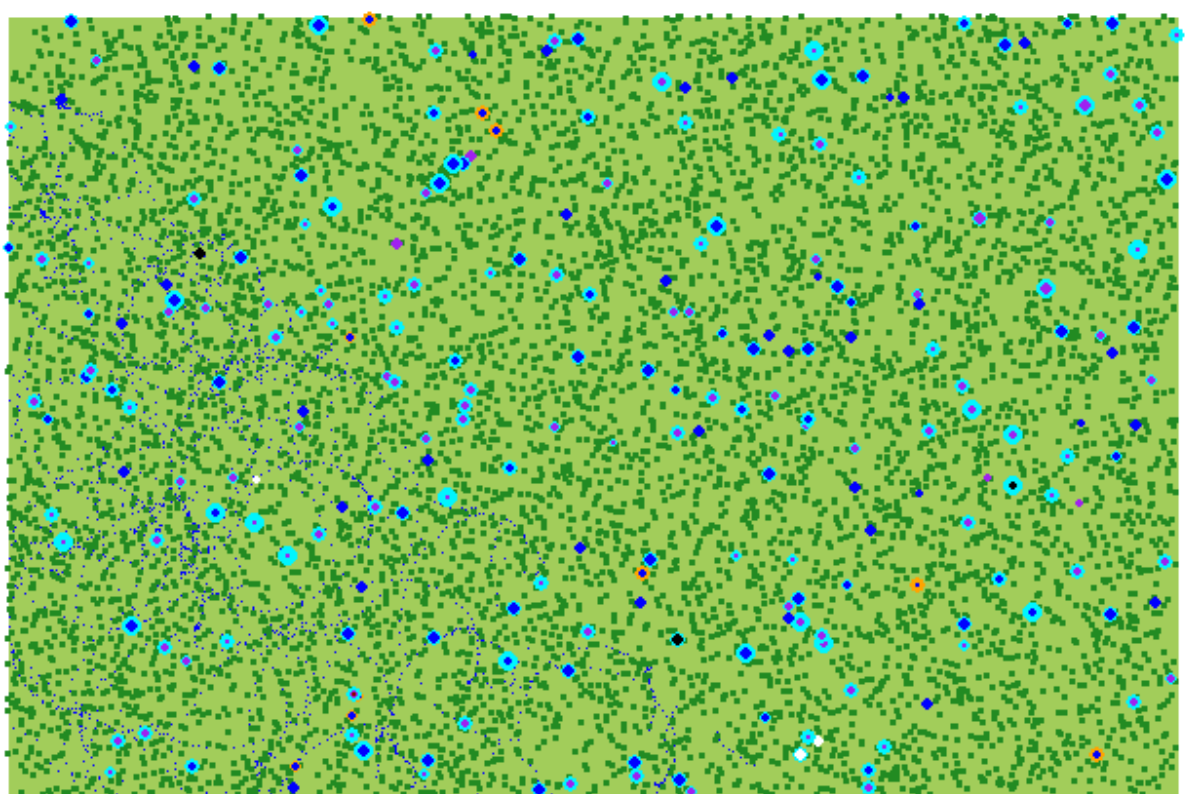
Current version:



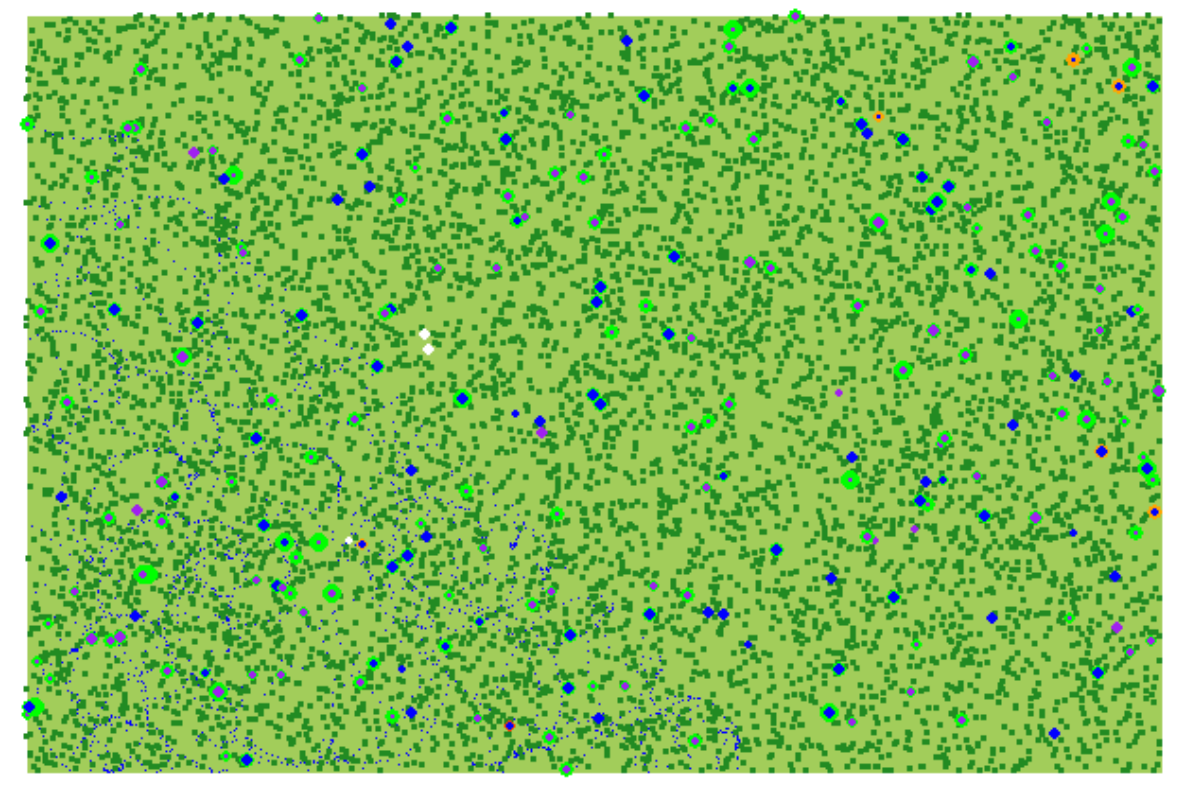
Since I like different colour combinations together, I created colour 'sets' as dictionaries which can be easily flipped between. For instance, I had an attachment to the original colour set and so kept it as 'og_set'.

There was an error caused by me mis-spelling turquoise as 'turquoise'; this was caught with some print messages which located the error.

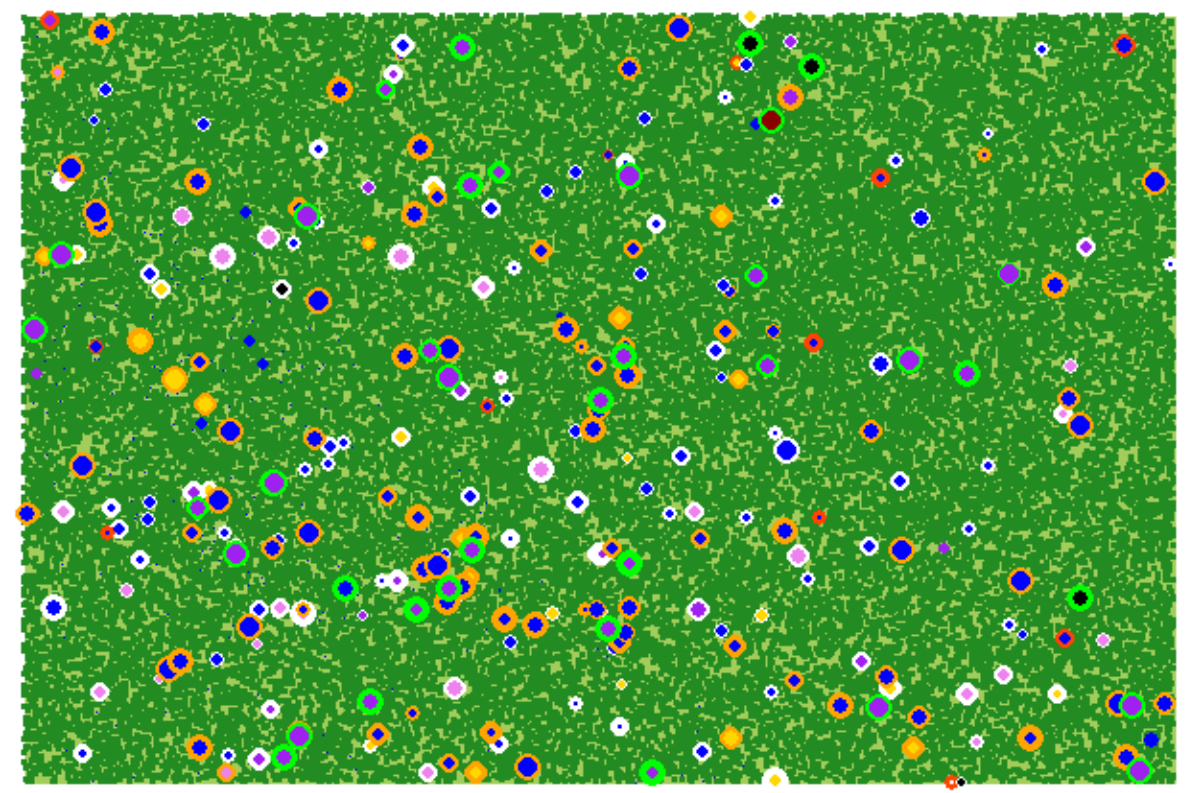
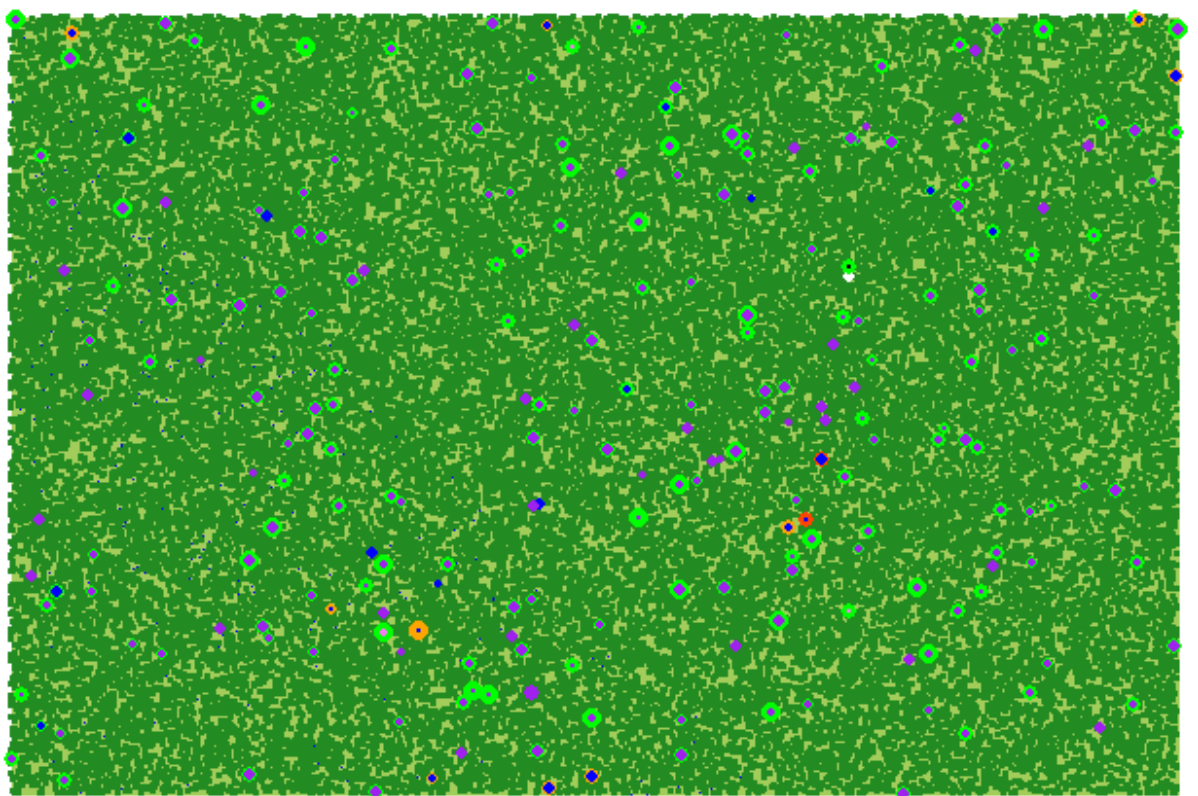
I accidentally created the mature colour set when I realised I hadn't set the grass colour to the grass variable, which I had set to 'darkolivegreen3'. When I changed it, it looked like this:

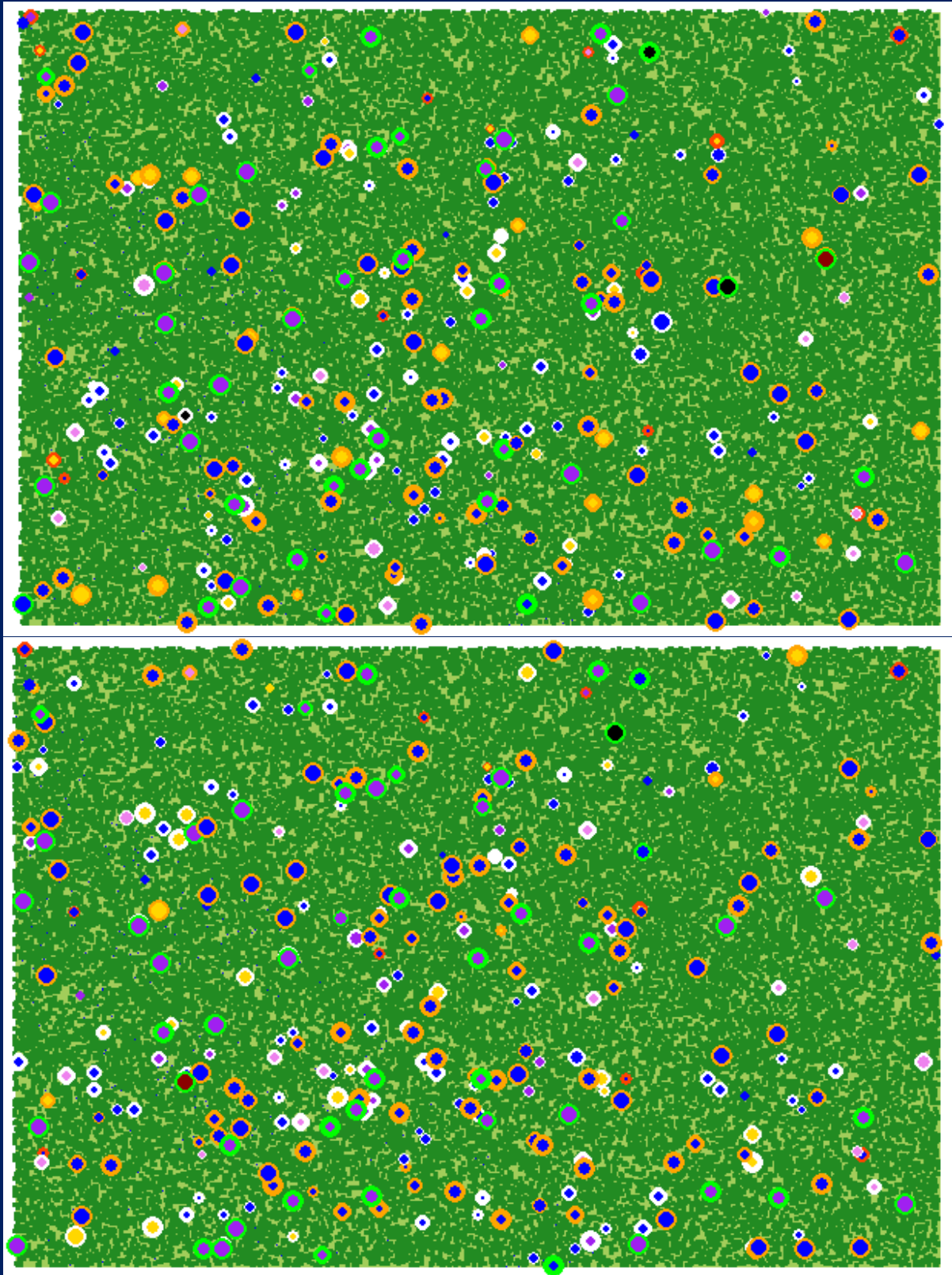


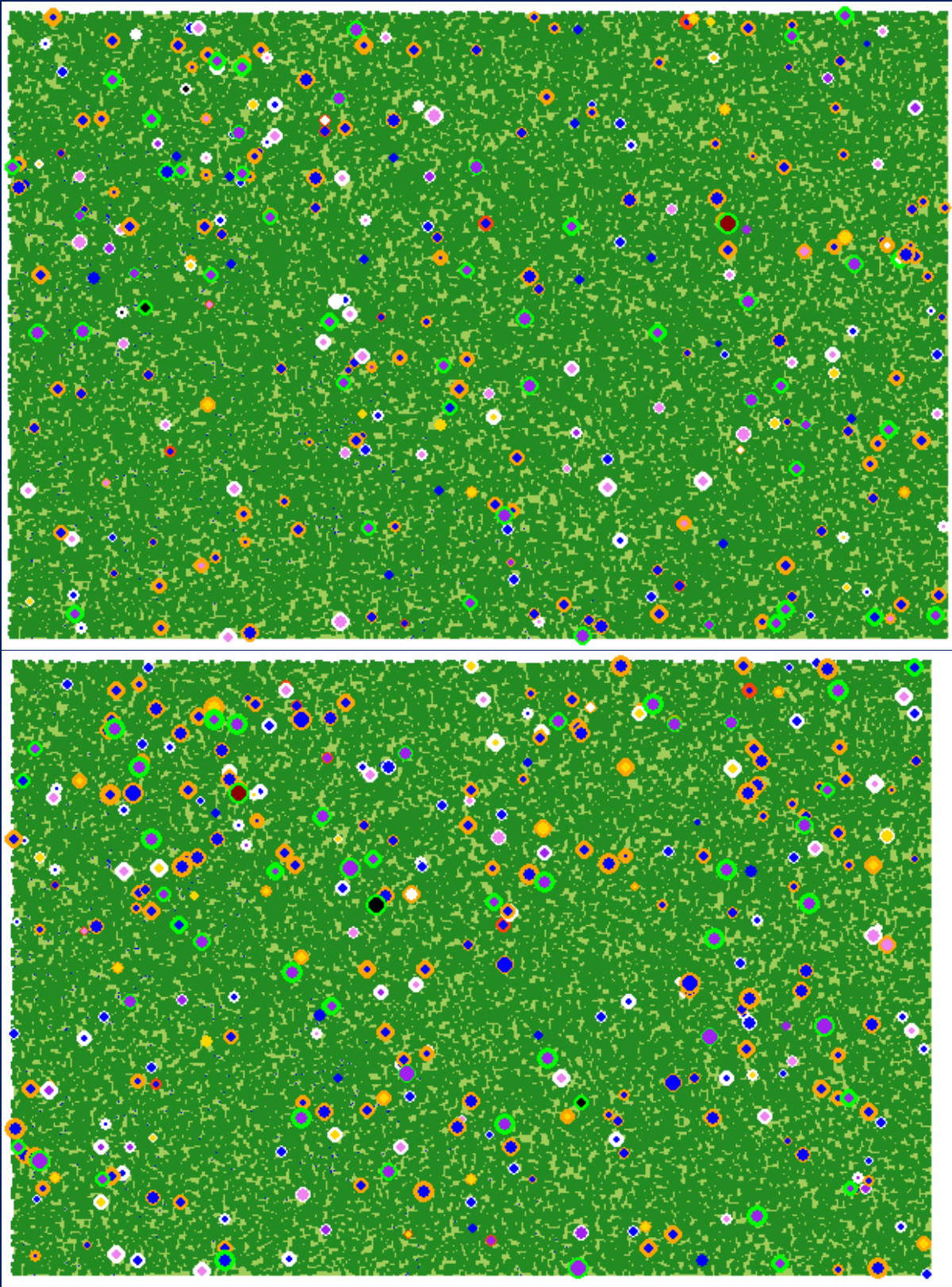
I tweaked a few things to make the mature colour set:

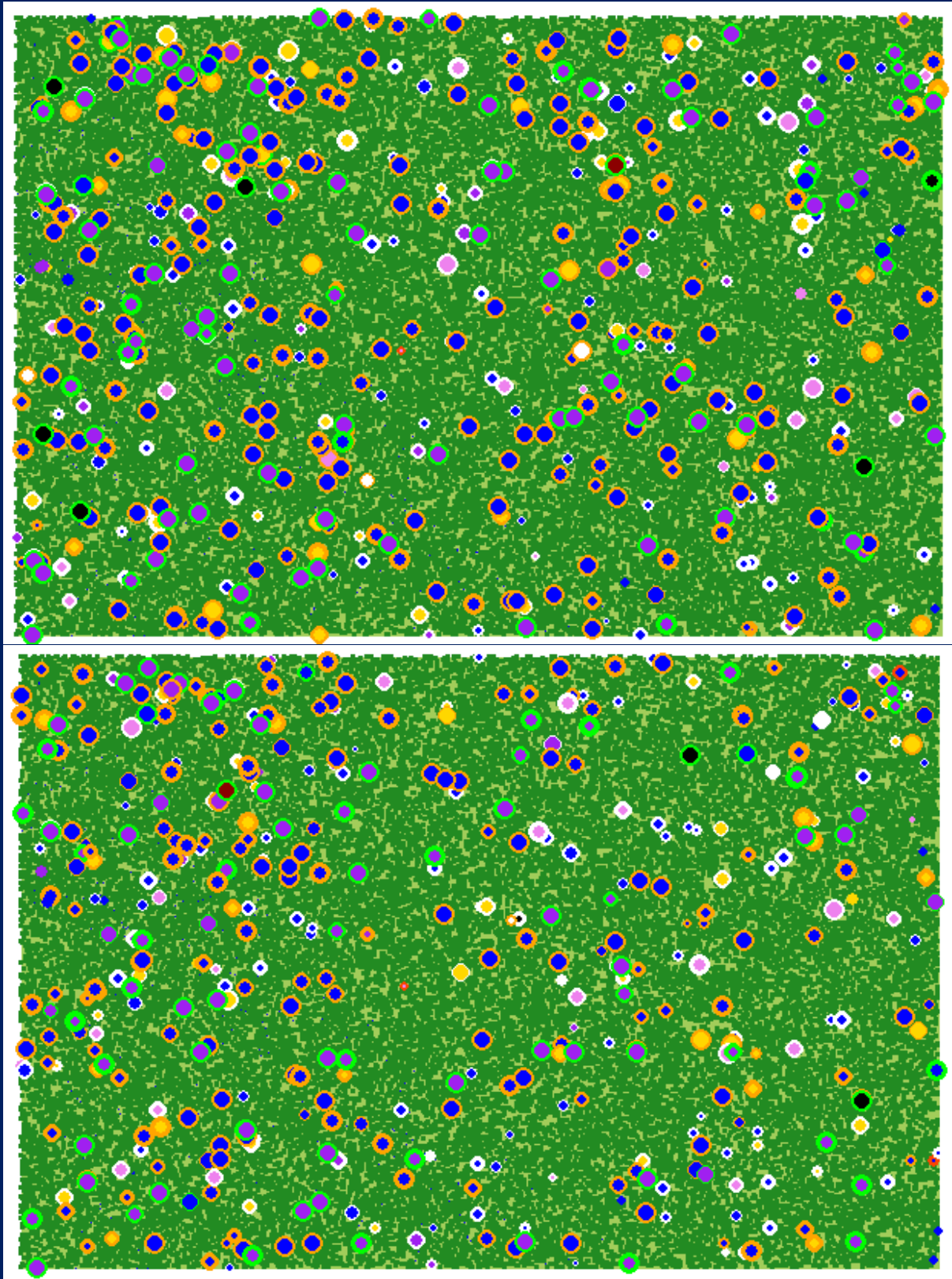


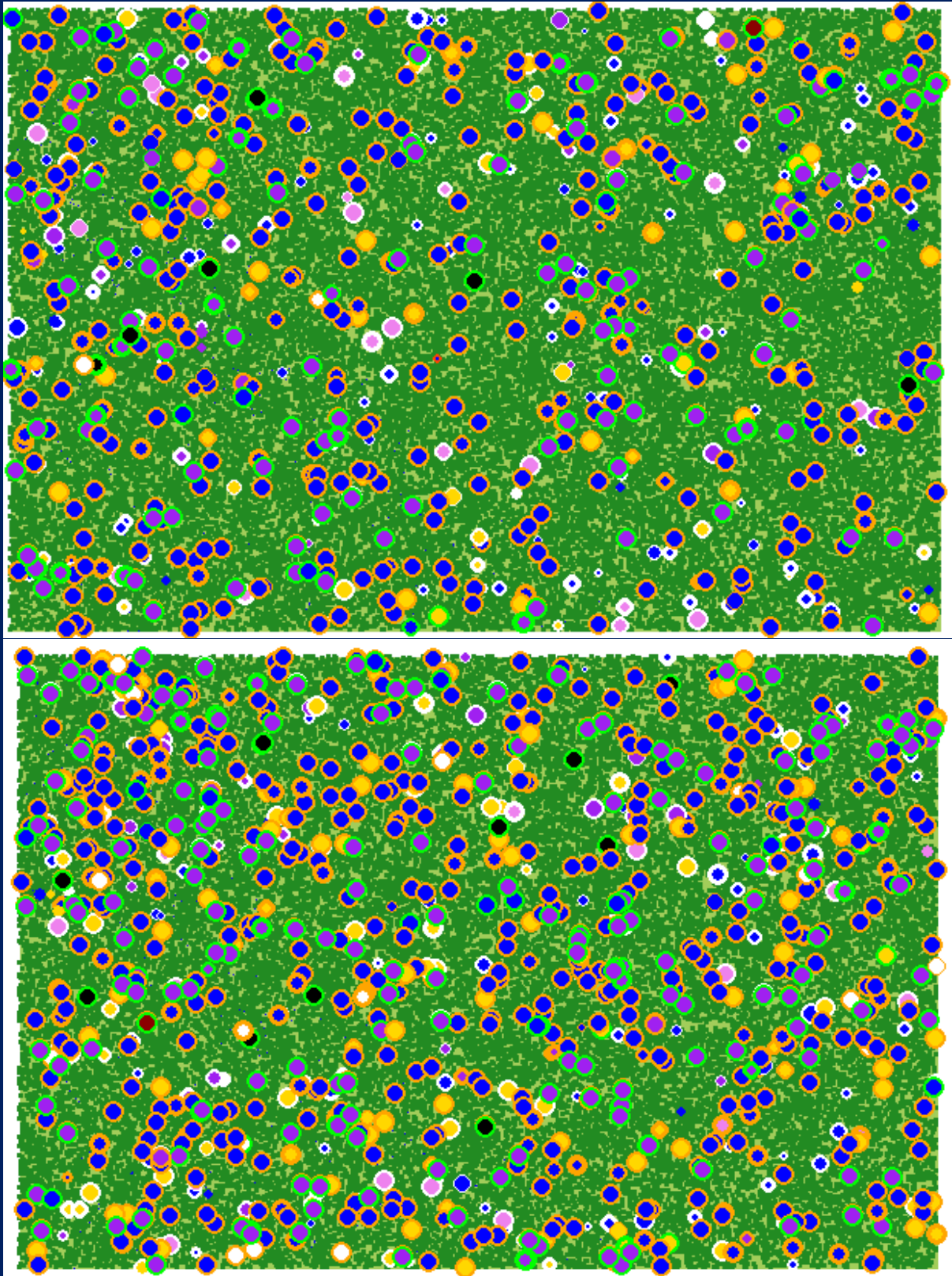
All creatures go extinct after a few hundred ticks. So, after increasing foliage density to be consistent with previous testing (multiplying all plant counts by 6, as the realm area was increased by a factor of 6):



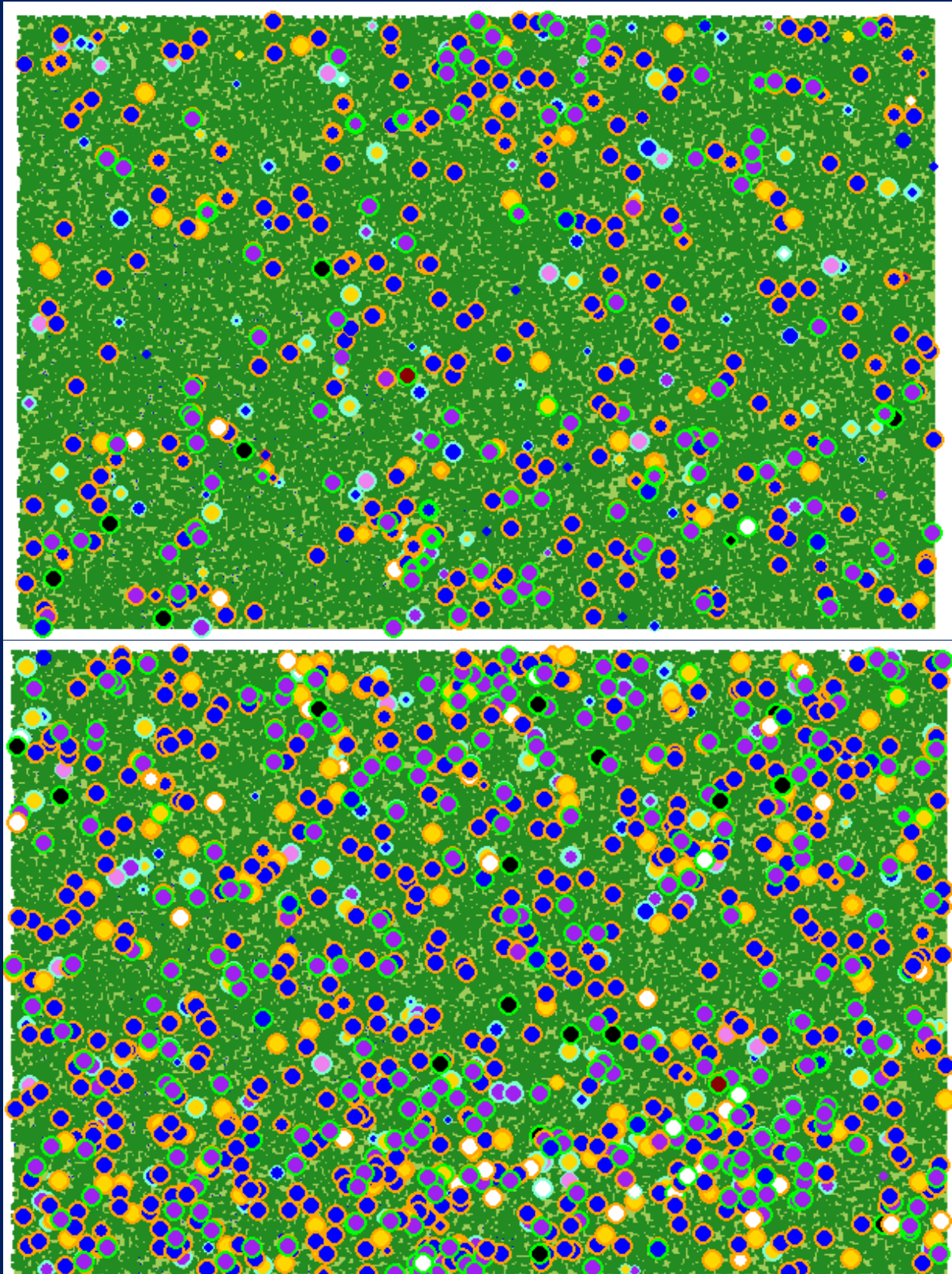








We can see from the progression of screenshots (all taken from a single simulation) that the population of creatures increases and the greatest indicator of fitness is the size of the creature. High vision is also seen. Since the population is so large, meat-eaters can also thrive, whereas they were the always amongst the first to go extinct when foliage was low. I ran the simulation again and the result is similar:



This suggests that the conditions of the realm are such that animas, by natural selection, evolve attributes towards an optimal setting of high size and vision, among other traits which are not immediately visible from the graphic. This is success! After months of work, I have visually demonstrated... *evolution*. The simulation can now be refined and more user features added, but the

core product has been delivered. Today is the day that terminal version 1.0 became complete! *(unfortunately at such a high population the program lags and the clock rate becomes very slow...)*

The previous iteration was successful but one of the key tenets of evolution is that the environment is the driving force... which means I need to show that changing the environment changes how the creatures end up evolving, and ideally using a relatively stable population.

In the last simulation the growth was rather extreme; the population boomed with creatures of strength and toughness that reached the cutoff wall I had placed (100)

```
Opweejeifo [melon|berry], 100.0 82.4 ate berry
Lajum [lavendar], 100.0 100.0 ate lavendar
Ofhooce [wildgrass|berry], 38.3 48.2 ate wildgrass
Faaz [apple|lavendar], 48.3 63.5 is boosting to get away from Fimop
at[lily], 86.1 100.0
Seth [apple|lavendar|meat], 99.3 100.0 ate apple
Ecy [apple|lily], 86.0 100.0 ate lily
Xeexaowwofh [meat], 100.0 100.0 is boosting to get away from Vounez:
n[meat], 100.0 100.0
Eenubbou [apple|wildgrass], 100.0 100.0 ate wildgrass
Vouzi [melon|lily], 57.1 64.6 is boosting to get away from Vounez:
meat], 100.0 100.0
Tlihhakkuc [melon|wildgrass|lily|lavendar], 100.0 100.0 ate melon
Ysaod [apple|meat], 97.8 100.0 ate apple
Mosooowvvo [lavendar], 100.0 100.0 ate lavendar
Mixx [lily], 80.8 100.0 is boosting to get away from Emaoze [meat].
0.0
Nepein [melon|wildgrass], 68.8 65.0 ate wildgrass
Ms Gag [apple|lily|meat], 100.0 100.0 struck Ewuimugr [lily|apple].
.0 and dealt 1 damage
Watradujoo [lily|apple], 100.0 100.0 ate apple
```

The cutoff was meant to prevent wacky outliers so having so many 100/100s is a sign of imbalance. I had set the mutation factor to 0.2 previously in order to speed up mutation, but I will now tune it down to 0.1. I removed the factor of $1/\sqrt{\text{anima.size()}}$ on energy movement expenses.

Previous realm origin:

```
for i in range(250):
    test_element_list.append(Anima(i, "name/diet/strength/toughness",
    [(random.choice(food_list)), [random.uniform(1, 5), 0.5, 0.3],
    random.uniform(15, 30), random.uniform(5, 25), random.uniform(5, 30),
    random.uniform(5, 30), random.uniform(20, 30), random.uniform(0, 1),
    random.uniform(30, 100), random.uniform(40, 200), random.uniform(5, 50)], None,
    random.uniform(0, 600), random.uniform(0, 400)))

    print("Generating realm...")
    testRealm = Realm(nickname.generate_nickname("realm"), 600, 400, 0.2, 0.05,
    [["tree", 3600], ["bush", 6000], ["vine", 1500], ["reed", 1200], ["lily", 3000],
```

```
["wildgrass", 3000]],test_element_list, "name/diet/strength/toughness")
#birth new test realm
```

Now with reduced foliage:

```
["tree", 1800], ["bush", 3000], ["vine", 750], ["reed", 1000], ["lily", 2000],
["wildgrass", 18000]
```

I analysed the data of the previous iteration:

```
70], [[berry|52], [[melon|47], [[lily|56], [[lavendar|52], [[meat|34], [[wildgrass|43], ], ],
[[259|[[apple|70], [[berry|52], [[melon|47], [[lily|56], [[lavendar|51], [[meat|34],
[[wildgrass|43], ], ], [[260|[[apple|70], [[berry|53], [[melon|47], [[lily|56], [[lavendar|51],
[[meat|34], [[wildgrass|43], ], ], [[261|[[apple|71], [[berry|53], [[melon|47], [[lily|58],
[[lavendar|52], [[meat|33], [[wildgrass|43], ], ], [[262|[[apple|71], [[berry|54], [[melon|47],
[[lily|58], [[lavendar|52], [[meat|33], [[wildgrass|43], ], ], [[263|[[apple|71], [[berry|55],
[[melon|47], [[lily|58], [[lavendar|52], [[meat|33], [[wildgrass|43], ], ], [[264|[[apple|73],
[[berry|55], [[melon|48], [[lily|61], [[lavendar|51], [[meat|33], [[wildgrass|43], ], ],
[[265|[[apple|73], [[berry|55], [[melon|48], [[lily|61], [[lavendar|50], [[meat|33],
[[wildgrass|43], ], ], [[266|[[apple|73], [[berry|56], [[melon|48], [[lily|62], [[lavendar|50],
[[meat|34], [[wildgrass|43], ], ], [[267|[[apple|73], [[berry|56], [[melon|48], [[lily|62],
[[lavendar|50], [[meat|34], [[wildgrass|43], ], ], [[268|[[apple|73], [[berry|57], [[melon|48],
[[lily|63], [[lavendar|50], [[meat|34], [[wildgrass|43], ], ], [[269|[[apple|74], [[berry|58],
[[melon|48], [[lily|63], [[lavendar|50], [[meat|34], [[wildgrass|43], ], ], [[270|[[apple|73],
[[berry|57], [[melon|48], [[lily|63], [[lavendar|50], [[meat|34], [[wildgrass|44], ], ],
[[271|[[apple|73], [[berry|57], [[melon|48], [[lily|63], [[lavendar|50], [[meat|34],
[[wildgrass|45], ], ], [[272|[[apple|73], [[berry|57], [[melon|48], [[lily|63], [[lavendar|50],
[[meat|34], [[wildgrass|45], ], ], [[273|[[apple|74], [[berry|58], [[melon|48], [[lily|63],
[[lavendar|50], [[meat|34], [[wildgrass|45], ], ], [[274|[[apple|74], [[berry|58], [[melon|48],
[[lily|63], [[lavendar|50], [[meat|34], [[wildgrass|45], ], ], [[275|[[apple|74], [[berry|58],
[[melon|48], [[lily|63], [[lavendar|50], [[meat|34], [[wildgrass|45], ], ], [[276|[[apple|75],
[[berry|58], [[melon|48], [[lily|63], [[lavendar|50], [[meat|34], [[wildgrass|45], ], ],
[[277|[[apple|77], [[berry|59], [[melon|50], [[lily|64], [[lavendar|50], [[meat|34],
[[wildgrass|45], ], ], [[278|[[apple|77], [[berry|59], [[melon|53], [[lily|64], [[lavendar|50],
[[meat|34], [[wildgrass|46], ], ], [[279|[[apple|79], [[berry|59], [[melon|53], [[lily|64],
[[lavendar|50], [[meat|35], [[wildgrass|47], ], ], [[280|[[apple|79], [[berry|59], [[melon|53],
[[lily|64], [[lavendar|50], [[meat|35], [[wildgrass|47], ], ], [[281|[[apple|78], [[berry|59],
[[melon|54], [[lily|65], [[lavendar|50], [[meat|35], [[wildgrass|48], ], ], [[282|[[apple|77],
[[berry|59], [[melon|54], [[lily|65], [[lavendar|50], [[meat|35], [[wildgrass|48], ], ],
[[283|[[apple|77], [[berry|61], [[melon|55], [[lily|66], [[lavendar|50], [[meat|35],
[[wildgrass|48], ], ], [[284|[[apple|78], [[berry|61], [[melon|58], [[lily|67], [[lavendar|50],
[[meat|35], [[wildgrass|48], ], ], [[285|[[apple|78], [[berry|61], [[melon|60], [[lily|67],
[[lavendar|50], [[meat|35], [[wildgrass|50], ], ], [[286|[[apple|79], [[berry|61], [[melon|60],
[[lily|70], [[lavendar|51], [[meat|35], [[wildgrass|51], ], ], [[287|[[apple|79], [[berry|63],
[[melon|60], [[lily|70], [[lavendar|50], [[meat|35], [[wildgrass|51], ], ], [[288|[[apple|79],
[[berry|63], [[melon|60], [[lily|70], [[lavendar|50], [[meat|35], [[wildgrass|51], ], ],
[[289|[[apple|79], [[berry|63], [[melon|60], [[lily|70], [[lavendar|49], [[meat|35],
[[wildgrass|51], ], ], [[290|[[apple|79], [[berry|63], [[melon|60], [[lily|71], [[lavendar|49],
[[meat|35], [[wildgrass|51], ], ], [[291|[[apple|79], [[berry|64], [[melon|60], [[lily|71],
[[lavendar|48], [[meat|35], [[wildgrass|51], ], ], [[292|[[apple|79], [[berry|65], [[melon|61],
[[lily|71], [[lavendar|48], [[meat|35], [[wildgrass|51], ], ], [[293|[[apple|79], [[berry|65],
[[melon|62], [[lily|71], [[lavendar|48], [[meat|35], [[wildgrass|51], ], ], [[294|[[apple|79],
```


[[lavendar|33], [[meat|27], [[wildgrass|57],],], [[382|[[apple|110], [[berry|69], [[melon|79],
 [[lily|78], [[lavendar|33], [[meat|27], [[wildgrass|57],],], [[383|[[apple|109], [[berry|69],
 [[melon|79], [[lily|78], [[lavendar|33], [[meat|26], [[wildgrass|57],],], [[384|[[apple|110],
 [[berry|69], [[melon|79], [[lily|78], [[lavendar|32], [[meat|27], [[wildgrass|58],],],
 [[385|[[apple|110], [[berry|69], [[melon|79], [[lily|78], [[lavendar|32], [[meat|27],
 [[wildgrass|58],],], [[386|[[apple|109], [[berry|69], [[melon|79], [[lily|77], [[lavendar|33],
 [[meat|26], [[wildgrass|57],],], [[387|[[apple|109], [[berry|69], [[melon|79], [[lily|77],
 [[lavendar|32], [[meat|26], [[wildgrass|57],],], [[388|[[apple|109], [[berry|69], [[melon|79],
 [[lily|78], [[lavendar|32], [[meat|26], [[wildgrass|57],],], [[389|[[apple|109], [[berry|71],
 [[melon|80], [[lily|79], [[lavendar|31], [[meat|26], [[wildgrass|57],],], [[390|[[apple|110],
 [[berry|72], [[melon|81], [[lily|80], [[lavendar|30], [[meat|26], [[wildgrass|57],],],
 [[391|[[apple|112], [[berry|72], [[melon|82], [[lily|81], [[lavendar|30], [[meat|26],
 [[wildgrass|57],],], [[392|[[apple|112], [[berry|71], [[melon|82], [[lily|81], [[lavendar|29],
 [[meat|26], [[wildgrass|57],],], [[393|[[apple|112], [[berry|70], [[melon|83], [[lily|80],
 [[lavendar|28], [[meat|26], [[wildgrass|57],],], [[394|[[apple|113], [[berry|70], [[melon|83],
 [[lily|82], [[lavendar|28], [[meat|26], [[wildgrass|58],],], [[395|[[apple|115], [[berry|71],
 [[melon|83], [[lily|85], [[lavendar|29], [[meat|26], [[wildgrass|59],],], [[396|[[apple|115],
 [[berry|71], [[melon|83], [[lily|86], [[lavendar|30], [[meat|26], [[wildgrass|60],],],
 [[397|[[apple|115], [[berry|71], [[melon|84], [[lily|86], [[lavendar|30], [[meat|26],
 [[wildgrass|60],],], [[398|[[apple|117], [[berry|72], [[melon|84], [[lily|89], [[lavendar|30],
 [[meat|26], [[wildgrass|60],],], [[399|[[apple|117], [[berry|72], [[melon|84], [[lily|89],
 [[lavendar|30], [[meat|26], [[wildgrass|60],],], [[400|[[apple|119], [[berry|74], [[melon|84],
 [[lily|89], [[lavendar|29], [[meat|26], [[wildgrass|60],],], [[401|[[apple|120], [[berry|75],
 [[melon|84], [[lily|89], [[lavendar|29], [[meat|26], [[wildgrass|60],],], [[402|[[apple|123],
 [[berry|76], [[melon|85], [[lily|89], [[lavendar|30], [[meat|26], [[wildgrass|60],],],
 [[403|[[apple|125], [[berry|76], [[melon|86], [[lily|90], [[lavendar|31], [[meat|27],
 [[wildgrass|60],],], [[404|[[apple|127], [[berry|77], [[melon|86], [[lily|90], [[lavendar|33],
 [[meat|27], [[wildgrass|60],],], [[405|[[apple|129], [[berry|77], [[melon|87], [[lily|90],
 [[lavendar|33], [[meat|27], [[wildgrass|60],],], [[406|[[apple|129], [[berry|76], [[melon|87],
 [[lily|91], [[lavendar|33], [[meat|27], [[wildgrass|60],],], [[407|[[apple|129], [[berry|77],
 [[melon|87], [[lily|90], [[lavendar|33], [[meat|28], [[wildgrass|61],],], [[408|[[apple|129],
 [[berry|76], [[melon|87], [[lily|91], [[lavendar|33], [[meat|29], [[wildgrass|61],],],
 [[409|[[apple|129], [[berry|76], [[melon|87], [[lily|92], [[lavendar|33], [[meat|30],
 [[wildgrass|62],],], [[410|[[apple|130], [[berry|76], [[melon|87], [[lily|92], [[lavendar|33],
 [[meat|30], [[wildgrass|63],],], [[411|[[apple|132], [[berry|76], [[melon|87], [[lily|96],
 [[lavendar|35], [[meat|31], [[wildgrass|63],],], [[412|[[apple|134], [[berry|77], [[melon|86],
 [[lily|97], [[lavendar|36], [[meat|32], [[wildgrass|63],],], [[413|[[apple|136], [[berry|77],
 [[melon|86], [[lily|100], [[lavendar|37], [[meat|32], [[wildgrass|63],],], [[414|[[apple|136],
 [[berry|77], [[melon|85], [[lily|100], [[lavendar|37], [[meat|32], [[wildgrass|64],],],
 [[415|[[apple|137], [[berry|78], [[melon|85], [[lily|102], [[lavendar|38], [[meat|32],
 [[wildgrass|65],],], [[416|[[apple|138], [[berry|79], [[melon|85], [[lily|102], [[lavendar|38],
 [[meat|32], [[wildgrass|65],],], [[417|[[apple|139], [[berry|81], [[melon|85], [[lily|104],
 [[lavendar|39], [[meat|32], [[wildgrass|65],],], [[418|[[apple|140], [[berry|83], [[melon|85],
 [[lily|106], [[lavendar|40], [[meat|32], [[wildgrass|65],],], [[419|[[apple|141], [[berry|85],
 [[melon|87], [[lily|106], [[lavendar|40], [[meat|32], [[wildgrass|66],],], [[420|[[apple|143],
 [[berry|86], [[melon|90], [[lily|108], [[lavendar|41], [[meat|32], [[wildgrass|66],],],
 [[421|[[apple|143], [[berry|87], [[melon|90], [[lily|108], [[lavendar|41], [[meat|32],
 [[wildgrass|67],],], [[422|[[apple|145], [[berry|87], [[melon|89], [[lily|109], [[lavendar|41],
 [[meat|31], [[wildgrass|67],],], [[423|[[apple|145], [[berry|88], [[melon|89], [[lily|109],
 [[lavendar|41], [[meat|31], [[wildgrass|67],],], [[424|[[apple|146], [[berry|88], [[melon|89],
 [[lily|111], [[lavendar|41], [[meat|31], [[wildgrass|67],],], [[425|[[apple|148], [[berry|89],

[[melon|90], [[lily|113], [[lavendar|41], [[meat|32], [[wildgrass|67],],], [[426|[[apple|149],
 [[berry|90], [[melon|90], [[lily|114], [[lavendar|41], [[meat|32], [[wildgrass|67],],],
 [[427|[[apple|150], [[berry|90], [[melon|90], [[lily|115], [[lavendar|41], [[meat|32],
 [[wildgrass|68],],], [[428|[[apple|151], [[berry|91], [[melon|90], [[lily|115], [[lavendar|41],
 [[meat|32], [[wildgrass|68],],], [[429|[[apple|153], [[berry|91], [[melon|89], [[lily|116],
 [[lavendar|43], [[meat|32], [[wildgrass|68],],], [[430|[[apple|153], [[berry|90], [[melon|89],
 [[lily|116], [[lavendar|43], [[meat|32], [[wildgrass|68],],], [[431|[[apple|153], [[berry|90],
 [[melon|89], [[lily|121], [[lavendar|47], [[meat|32], [[wildgrass|71],],], [[432|[[apple|154],
 [[berry|90], [[melon|89], [[lily|123], [[lavendar|47], [[meat|32], [[wildgrass|71],],],
 [[433|[[apple|154], [[berry|90], [[melon|89], [[lily|123], [[lavendar|47], [[meat|33],
 [[wildgrass|71],],], [[434|[[apple|153], [[berry|89], [[melon|89], [[lily|122], [[lavendar|47],
 [[meat|33], [[wildgrass|71],],], [[435|[[apple|154], [[berry|90], [[melon|88], [[lily|122],
 [[lavendar|47], [[meat|33], [[wildgrass|71],],], [[436|[[apple|155], [[berry|90], [[melon|87],
 [[lily|123], [[lavendar|47], [[meat|33], [[wildgrass|71],],], [[437|[[apple|157], [[berry|90],
 [[melon|90], [[lily|123], [[lavendar|47], [[meat|33], [[wildgrass|72],],], [[438|[[apple|158],
 [[berry|91], [[melon|91], [[lily|125], [[lavendar|48], [[meat|33], [[wildgrass|73],],],
 [[439|[[apple|156], [[berry|94], [[melon|91], [[lily|127], [[lavendar|48], [[meat|32],
 [[wildgrass|73],],], [[440|[[apple|158], [[berry|95], [[melon|92], [[lily|130], [[lavendar|49],
 [[meat|32], [[wildgrass|75],],], [[441|[[apple|158], [[berry|94], [[melon|91], [[lily|131],
 [[lavendar|49], [[meat|32], [[wildgrass|75],],], [[442|[[apple|159], [[berry|96], [[melon|91],
 [[lily|132], [[lavendar|50], [[meat|33], [[wildgrass|75],],], [[443|[[apple|158], [[berry|98],
 [[melon|91], [[lily|134], [[lavendar|50], [[meat|33], [[wildgrass|75],],], [[444|[[apple|161],
 [[berry|98], [[melon|93], [[lily|135], [[lavendar|50], [[meat|33], [[wildgrass|75],],],
 [[445|[[apple|161], [[berry|98], [[melon|93], [[lily|138], [[lavendar|50], [[meat|32],
 [[wildgrass|75],],], [[446|[[apple|162], [[berry|98], [[melon|93], [[lily|138], [[lavendar|50],
 [[meat|32], [[wildgrass|76],],], [[447|[[apple|162], [[berry|98], [[melon|94], [[lily|139],
 [[lavendar|51], [[meat|32], [[wildgrass|75],],], [[448|[[apple|163], [[berry|97], [[melon|93],
 [[lily|140], [[lavendar|51], [[meat|31], [[wildgrass|74],],], [[449|[[apple|163], [[berry|99],
 [[melon|94], [[lily|142], [[lavendar|51], [[meat|31], [[wildgrass|74],],], [[450|[[apple|165],
 [[berry|99], [[melon|94], [[lily|143], [[lavendar|51], [[meat|31], [[wildgrass|74],],],
 [[451|[[apple|168], [[berry|99], [[melon|96], [[lily|142], [[lavendar|52], [[meat|30],
 [[wildgrass|75],],], [[452|[[apple|168], [[berry|99], [[melon|101], [[lily|143], [[lavendar|52],
 [[meat|30], [[wildgrass|75],],], [[453|[[apple|168], [[berry|100], [[melon|101], [[lily|144],
 [[lavendar|52], [[meat|30], [[wildgrass|75],],], [[454|[[apple|170], [[berry|100], [[melon|103],
 [[lily|147], [[lavendar|53], [[meat|30], [[wildgrass|76],],], [[455|[[apple|172], [[berry|100],
 [[melon|104], [[lily|149], [[lavendar|53], [[meat|30], [[wildgrass|76],],], [[456|[[apple|171],
 [[berry|100], [[melon|105], [[lily|152], [[lavendar|53], [[meat|30], [[wildgrass|76],],],
 [[457|[[apple|173], [[berry|101], [[melon|105], [[lily|154], [[lavendar|54], [[meat|29],
 [[wildgrass|77],],], [[458|[[apple|172], [[berry|100], [[melon|105], [[lily|154], [[lavendar|54],
 [[meat|29], [[wildgrass|78],],], [[459|[[apple|172], [[berry|100], [[melon|105], [[lily|154],
 [[lavendar|53], [[meat|29], [[wildgrass|78],],], [[460|[[apple|174], [[berry|100], [[melon|105],
 [[lily|156], [[lavendar|53], [[meat|29], [[wildgrass|78],],], [[461|[[apple|175], [[berry|102],
 [[melon|106], [[lily|158], [[lavendar|53], [[meat|29], [[wildgrass|78],],], [[462|[[apple|177],
 [[berry|104], [[melon|106], [[lily|158], [[lavendar|52], [[meat|29], [[wildgrass|78],],],
 [[463|[[apple|177], [[berry|106], [[melon|107], [[lily|160], [[lavendar|52], [[meat|30],
 [[wildgrass|78],],], [[464|[[apple|178], [[berry|104], [[melon|107], [[lily|161], [[lavendar|52],
 [[meat|30], [[wildgrass|78],],], [[465|[[apple|179], [[berry|104], [[melon|106], [[lily|162],
 [[lavendar|52], [[meat|30], [[wildgrass|78],],], [[466|[[apple|180], [[berry|105], [[melon|106],
 [[lily|163], [[lavendar|53], [[meat|31], [[wildgrass|79],],], [[467|[[apple|182], [[berry|106],
 [[melon|106], [[lily|165], [[lavendar|54], [[meat|30], [[wildgrass|80],],], [[468|[[apple|181],
 [[berry|106], [[melon|105], [[lily|166], [[lavendar|54], [[meat|31], [[wildgrass|79],],],

[[469|[[apple|182], [[berry|106], [[melon|106], [[lily|167], [[lavendar|53], [[meat|31],
 [[wildgrass|78],],], [[470|[[apple|182], [[berry|107], [[melon|107], [[lily|166], [[lavendar|53],
 [[meat|31], [[wildgrass|78],],], [[471|[[apple|185], [[berry|109], [[melon|108], [[lily|170],
 [[lavendar|54], [[meat|31], [[wildgrass|78],],], [[472|[[apple|185], [[berry|111], [[melon|110],
 [[lily|170], [[lavendar|55], [[meat|31], [[wildgrass|78],],], [[473|[[apple|189], [[berry|113],
 [[melon|111], [[lily|174], [[lavendar|55], [[meat|31], [[wildgrass|76],],], [[474|[[apple|193],
 [[berry|113], [[melon|113], [[lily|176], [[lavendar|59], [[meat|31], [[wildgrass|76],],],
 [[475|[[apple|195], [[berry|114], [[melon|115], [[lily|176], [[lavendar|60], [[meat|30],
 [[wildgrass|77],],], [[476|[[apple|197], [[berry|114], [[melon|115], [[lily|178], [[lavendar|60],
 [[meat|30], [[wildgrass|78],],], [[477|[[apple|197], [[berry|114], [[melon|116], [[lily|178],
 [[lavendar|61], [[meat|30], [[wildgrass|80],],], [[478|[[apple|198], [[berry|112], [[melon|118],
 [[lily|179], [[lavendar|63], [[meat|30], [[wildgrass|83],],], [[479|[[apple|197], [[berry|113],
 [[melon|118], [[lily|178], [[lavendar|63], [[meat|30], [[wildgrass|84],],], [[480|[[apple|197],
 [[berry|114], [[melon|119], [[lily|179], [[lavendar|63], [[meat|30], [[wildgrass|84],],],
 [[481|[[apple|198], [[berry|116], [[melon|119], [[lily|181], [[lavendar|63], [[meat|31],
 [[wildgrass|84],],], [[482|[[apple|200], [[berry|117], [[melon|122], [[lily|181], [[lavendar|64],
 [[meat|30], [[wildgrass|84],],], [[483|[[apple|200], [[berry|117], [[melon|123], [[lily|181],
 [[lavendar|65], [[meat|30], [[wildgrass|86],],], [[484|[[apple|200], [[berry|117], [[melon|125],
 [[lily|181], [[lavendar|65], [[meat|30], [[wildgrass|86],],], [[485|[[apple|201], [[berry|118],
 [[melon|126], [[lily|183], [[lavendar|66], [[meat|30], [[wildgrass|86],],], [[486|[[apple|203],
 [[berry|118], [[melon|125], [[lily|186], [[lavendar|66], [[meat|30], [[wildgrass|86],],],
 [[487|[[apple|206], [[berry|118], [[melon|127], [[lily|189], [[lavendar|66], [[meat|30],
 [[wildgrass|90],],], [[488|[[apple|207], [[berry|119], [[melon|128], [[lily|191], [[lavendar|66],
 [[meat|30], [[wildgrass|90],],], [[489|[[apple|210], [[berry|122], [[melon|128], [[lily|191],
 [[lavendar|66], [[meat|30], [[wildgrass|91],],], [[490|[[apple|211], [[berry|124], [[melon|131],
 [[lily|194], [[lavendar|65], [[meat|30], [[wildgrass|90],],], [[491|[[apple|213], [[berry|126],
 [[melon|131], [[lily|196], [[lavendar|66], [[meat|30], [[wildgrass|90],],], [[492|[[apple|216],
 [[berry|127], [[melon|132], [[lily|196], [[lavendar|67], [[meat|30], [[wildgrass|90],],],
 [[493|[[apple|218], [[berry|127], [[melon|132], [[lily|199], [[lavendar|68], [[meat|30],
 [[wildgrass|91],],], [[494|[[apple|218], [[berry|127], [[melon|132], [[lily|199], [[lavendar|68],
 [[meat|30], [[wildgrass|91],],], [[495|[[apple|216], [[berry|128], [[melon|131], [[lily|197],
 [[lavendar|67], [[meat|31], [[wildgrass|93],],], [[496|[[apple|216], [[berry|129], [[melon|131],
 [[lily|197], [[lavendar|67], [[meat|30], [[wildgrass|92],],], [[497|[[apple|218], [[berry|132],
 [[melon|132], [[lily|198], [[lavendar|69], [[meat|32], [[wildgrass|95],],], [[498|[[apple|218],
 [[berry|132], [[melon|131], [[lily|197], [[lavendar|69], [[meat|32], [[wildgrass|97],],],
 [[499|[[apple|219], [[berry|131], [[melon|131], [[lily|203], [[lavendar|70], [[meat|33],
 [[wildgrass|96],],], [[500|[[apple|219], [[berry|131], [[melon|132], [[lily|202], [[lavendar|69],
 [[meat|32], [[wildgrass|98],],], [[501|[[apple|219], [[berry|131], [[melon|133], [[lily|204],
 [[lavendar|69], [[meat|32], [[wildgrass|101],],], [[502|[[apple|222], [[berry|132],
 [[melon|134], [[lily|206], [[lavendar|69], [[meat|32], [[wildgrass|100],],], [[503|[[apple|223],
 [[berry|131], [[melon|134], [[lily|208], [[lavendar|70], [[meat|32], [[wildgrass|102],],],
 [[504|[[apple|224], [[berry|132], [[melon|137], [[lily|209], [[lavendar|71], [[meat|32],
 [[wildgrass|102],],], [[505|[[apple|227], [[berry|135], [[melon|138], [[lily|209], [[lavendar|74],
 [[meat|32], [[wildgrass|102],],], [[506|[[apple|229], [[berry|137], [[melon|139], [[lily|210],
 [[lavendar|74], [[meat|32], [[wildgrass|102],],], [[507|[[apple|230], [[berry|138],
 [[melon|138], [[lily|212], [[lavendar|76], [[meat|32], [[wildgrass|102],],], [[508|[[apple|235],
 [[berry|138], [[melon|140], [[lily|214], [[lavendar|78], [[meat|32], [[wildgrass|102],],],
 [[509|[[apple|240], [[berry|140], [[melon|144], [[lily|218], [[lavendar|79], [[meat|33],
 [[wildgrass|106],],], [[510|[[apple|240], [[berry|141], [[melon|144], [[lily|219], [[lavendar|79],
 [[meat|33], [[wildgrass|107],],], [[511|[[apple|240], [[berry|141], [[melon|144], [[lily|216],
 [[lavendar|79], [[meat|34], [[wildgrass|108],],], [[512|[[apple|240], [[berry|142],

[[melon|146], [[lily|216], [[lavendar|82], [[meat|35], [[wildgrass|110],],], [[513|[[apple|240],
 [[berry|141], [[melon|147], [[lily|218], [[lavendar|82], [[meat|35], [[wildgrass|110],],],
 [[514|[[apple|243], [[berry|143], [[melon|147], [[lily|223], [[lavendar|83], [[meat|35],
 [[wildgrass|110],],], [[515|[[apple|246], [[berry|143], [[melon|149], [[lily|227], [[lavendar|83],
 [[meat|35], [[wildgrass|111],],], [[516|[[apple|250], [[berry|144], [[melon|150], [[lily|227],
 [[lavendar|82], [[meat|34], [[wildgrass|111],],], [[517|[[apple|251], [[berry|146],
 [[melon|151], [[lily|230], [[lavendar|82], [[meat|34], [[wildgrass|112],],], [[518|[[apple|253],
 [[berry|147], [[melon|154], [[lily|232], [[lavendar|84], [[meat|35], [[wildgrass|112],],],
 [[519|[[apple|254], [[berry|150], [[melon|156], [[lily|235], [[lavendar|85], [[meat|33],
 [[wildgrass|114],],], [[520|[[apple|254], [[berry|151], [[melon|157], [[lily|234], [[lavendar|86],
 [[meat|33], [[wildgrass|115],],], [[521|[[apple|259], [[berry|153], [[melon|159], [[lily|237],
 [[lavendar|87], [[meat|34], [[wildgrass|118],],], [[522|[[apple|260], [[berry|155],
 [[melon|160], [[lily|241], [[lavendar|90], [[meat|36], [[wildgrass|120],],], [[523|[[apple|260],
 [[berry|155], [[melon|163], [[lily|243], [[lavendar|92], [[meat|35], [[wildgrass|123],],],
 [[524|[[apple|266], [[berry|157], [[melon|163], [[lily|245], [[lavendar|95], [[meat|35],
 [[wildgrass|122],],], [[525|[[apple|268], [[berry|159], [[melon|163], [[lily|248], [[lavendar|95],
 [[meat|35], [[wildgrass|123],],], [[526|[[apple|273], [[berry|158], [[melon|165], [[lily|250],
 [[lavendar|97], [[meat|36], [[wildgrass|125],],], [[527|[[apple|274], [[berry|159],
 [[melon|165], [[lily|254], [[lavendar|99], [[meat|36], [[wildgrass|126],],], [[528|[[apple|274],
 [[berry|160], [[melon|165], [[lily|256], [[lavendar|99], [[meat|36], [[wildgrass|126],],],
 [[529|[[apple|277], [[berry|161], [[melon|169], [[lily|259], [[lavendar|100], [[meat|37],
 [[wildgrass|127],],], [[530|[[apple|280], [[berry|162], [[melon|170], [[lily|263],
 [[lavendar|102], [[meat|36], [[wildgrass|131],],], [[531|[[apple|282], [[berry|163],
 [[melon|171], [[lily|265], [[lavendar|104], [[meat|35], [[wildgrass|131],],], [[532|[[apple|291],
 [[berry|169], [[melon|175], [[lily|272], [[lavendar|108], [[meat|37], [[wildgrass|133],],],
 [[533|[[apple|298], [[berry|173], [[melon|176], [[lily|279], [[lavendar|109], [[meat|36],
 [[wildgrass|135],],], [[534|[[apple|297], [[berry|172], [[melon|175], [[lily|279],
 [[lavendar|108], [[meat|35], [[wildgrass|136],],], [[535|[[apple|300], [[berry|172],
 [[melon|178], [[lily|285], [[lavendar|107], [[meat|36], [[wildgrass|135],],], [[536|[[apple|303],
 [[berry|170], [[melon|179], [[lily|288], [[lavendar|109], [[meat|38], [[wildgrass|136],],],
 [[537|[[apple|305], [[berry|170], [[melon|179], [[lily|289], [[lavendar|112], [[meat|38],
 [[wildgrass|137],],], [[538|[[apple|308], [[berry|172], [[melon|180], [[lily|293],
 [[lavendar|112], [[meat|39], [[wildgrass|138],],], [[539|[[apple|312], [[berry|173],
 [[melon|183], [[lily|299], [[lavendar|114], [[meat|39], [[wildgrass|140],],], [[540|[[apple|317],
 [[berry|175], [[melon|185], [[lily|301], [[lavendar|115], [[meat|39], [[wildgrass|140],],],
 [[541|[[apple|322], [[berry|178], [[melon|191], [[lily|305], [[lavendar|114], [[meat|39],
 [[wildgrass|141],],], [[542|[[apple|325], [[berry|178], [[melon|193], [[lily|306],
 [[lavendar|115], [[meat|39], [[wildgrass|142],],], [[543|[[apple|327], [[berry|180],
 [[melon|194], [[lily|309], [[lavendar|115], [[meat|39], [[wildgrass|144],],], [[544|[[apple|330],
 [[berry|184], [[melon|195], [[lily|314], [[lavendar|115], [[meat|39], [[wildgrass|145],],],
 [[545|[[apple|331], [[berry|190], [[melon|197], [[lily|315], [[lavendar|117], [[meat|40],
 [[wildgrass|147],],], [[546|[[apple|331], [[berry|190], [[melon|198], [[lily|317],
 [[lavendar|119], [[meat|40], [[wildgrass|149],],], [[547|[[apple|332], [[berry|192],
 [[melon|200], [[lily|318], [[lavendar|119], [[meat|41], [[wildgrass|150],],], [[548|[[apple|335],
 [[berry|194], [[melon|204], [[lily|322], [[lavendar|120], [[meat|43], [[wildgrass|150],],],
 [[549|[[apple|337], [[berry|194], [[melon|206], [[lily|325], [[lavendar|121], [[meat|45],
 [[wildgrass|150],],], [[550|[[apple|345], [[berry|196], [[melon|205], [[lily|330],
 [[lavendar|124], [[meat|46], [[wildgrass|151],],], [[551|[[apple|349], [[berry|197],
 [[melon|205], [[lily|335], [[lavendar|125], [[meat|47], [[wildgrass|152],],], [[552|[[apple|351],
 [[berry|198], [[melon|205], [[lily|337], [[lavendar|126], [[meat|47], [[wildgrass|154],],],
 [[553|[[apple|355], [[berry|200], [[melon|206], [[lily|339], [[lavendar|127], [[meat|47],

```

[[wildgrass|155], ], ], |[554|[[apple|358], |[berry|202], |[melon|211], |[lily|343],
|[lavendar|129], |[meat|47], |[wildgrass|156], ], ], |[555|[[apple|361], |[berry|203],
|[melon|211], |[lily|346], |[lavendar|131], |[meat|48], |[wildgrass|157], ], ], |[556|[[apple|364],
|[berry|202], |[melon|210], |[lily|349], |[lavendar|131], |[meat|48], |[wildgrass|157], ], ],
|[557|[[apple|367], |[berry|205], |[melon|209], |[lily|350], |[lavendar|130], |[meat|48],
|[wildgrass|157], ], ], |[558|[[apple|370], |[berry|205], |[melon|211], |[lily|357],
|[lavendar|130], |[meat|48], |[wildgrass|157], ], ], |[559|[[apple|373], |[berry|208],
|[melon|210], |[lily|359], |[lavendar|130], |[meat|48], |[wildgrass|159], ], ], |[560|[[apple|378],
|[berry|209], |[melon|211], |[lily|360], |[lavendar|131], |[meat|48], |[wildgrass|161], ], ],
|[561|[[apple|381], |[berry|212], |[melon|211], |[lily|362], |[lavendar|131], |[meat|48],
|[wildgrass|163], ], ], |[562|[[apple|390], |[berry|213], |[melon|215], |[lily|368],
|[lavendar|131], |[meat|50], |[wildgrass|163], ], ], |[563|[[apple|398], |[berry|216],
|[melon|215], |[lily|377], |[lavendar|133], |[meat|49], |[wildgrass|165], ], ], |[564|[[apple|401],
|[berry|219], |[melon|217], |[lily|383], |[lavendar|134], |[meat|50], |[wildgrass|168], ], ],
|[565|[[apple|405], |[berry|222], |[melon|218], |[lily|387], |[lavendar|135], |[meat|51],
|[wildgrass|169], ], ], |[566|[[apple|407], |[berry|221], |[melon|221], |[lily|389],
|[lavendar|135], |[meat|53], |[wildgrass|170], ], ], |[567|[[apple|408], |[berry|226],
|[melon|221], |[lily|395], |[lavendar|138], |[meat|56], |[wildgrass|173], ], ], |[568|[[apple|413],
|[berry|229], |[melon|223], |[lily|400], |[lavendar|140], |[meat|57], |[wildgrass|172], ], ],
|[569|[[apple|417], |[berry|232], |[melon|225], |[lily|404], |[lavendar|141], |[meat|57],
|[wildgrass|175], ], ], |[570|[[apple|421], |[berry|232], |[melon|226], |[lily|411],
|[lavendar|141], |[meat|57], |[wildgrass|174], ], ], |[571|[[apple|422], |[berry|234],
|[melon|228], |[lily|414], |[lavendar|142], |[meat|57], |[wildgrass|174], ], ], |[572|[[apple|424],
|[berry|232], |[melon|230], |[lily|420], |[lavendar|143], |[meat|57], |[wildgrass|177], ], ],
|[573|[[apple|427], |[berry|233], |[melon|233], |[lily|422], |[lavendar|145], |[meat|58],
|[wildgrass|179], ], ], |[574|[[apple|427], |[berry|236], |[melon|233], |[lily|426],
|[lavendar|145], |[meat|59], |[wildgrass|180], ], ], |[575|[[apple|428], |[berry|236],
|[melon|237], |[lily|428], |[lavendar|146], |[meat|59], |[wildgrass|181], ], ], |[576|[[apple|433],
|[berry|239], |[melon|244], |[lily|433], |[lavendar|145], |[meat|60], |[wildgrass|183], ], ],
|[577|[[apple|434], |[berry|241], |[melon|249], |[lily|436], |[lavendar|146], |[meat|60],
|[wildgrass|183], ], ], |[578|[[apple|441], |[berry|244], |[melon|252], |[lily|442],
|[lavendar|148], |[meat|60], |[wildgrass|185], ], ], |[579|[[apple|442], |[berry|244],
|[melon|254], |[lily|445], |[lavendar|150], |[meat|60], |[wildgrass|190], ], ], |[580|[[apple|447],
|[berry|246], |[melon|255], |[lily|449], |[lavendar|152], |[meat|60], |[wildgrass|194], ], ],
|[581|[[apple|449], |[berry|247], |[melon|256], |[lily|452], |[lavendar|153], |[meat|62],
|[wildgrass|195], ], ], |[582|[[apple|453], |[berry|248], |[melon|261], |[lily|457],
|[lavendar|154], |[meat|61], |[wildgrass|195], ], ], |[583|[[apple|455], |[berry|256],
|[melon|264], |[lily|463], |[lavendar|155], |[meat|61], |[wildgrass|197], ], ], |[584|[[apple|460],
|[berry|256], |[melon|267], |[lily|468], |[lavendar|157], |[meat|62], |[wildgrass|196], ], ],
|[585|[[apple|466], |[berry|256], |[melon|270], |[lily|475], |[lavendar|161], |[meat|63],
|[wildgrass|198], ], ], |[586|[[apple|469], |[berry|258], |[melon|272], |[lily|480],
|[lavendar|161], |[meat|65], |[wildgrass|198], ], ], |[587|[[apple|471], |[berry|261],
|[melon|275], |[lily|483], |[lavendar|162], |[meat|65], |[wildgrass|198], ], ], |[588|[[apple|478],
|[berry|265], |[melon|277], |[lily|489], |[lavendar|163], |[meat|66], |[wildgrass|200], ], ],
|[589|[[apple|478], |[berry|267], |[melon|284], |[lily|491], |[lavendar|165], |[meat|69],
|[wildgrass|200], ], ], |[590|[[apple|477], |[berry|266], |[melon|285], |[lily|487],
|[lavendar|164], |[meat|68], |[wildgrass|199], ], ],
////////////////////
////////////////////
////////////////////

```

[[0 N/A],	[[1 N/A],	[[2 N/A],	[[3 3.0],	[[4 3.0],	[[5 3.7],	[[6 3.7],	[[7 4.5],	[[8 4.5],	[[9 5.4],
[[10 5.4],	[[11 5.4],	[[12 6.5],	[[13 6.5],	[[14 6.5],	[[15 8.2],	[[16 9.8],	[[17 12.3],	[[18 12.3],	[[19 12.3],
[[20 12.3],	[[21 12.3],	[[22 12.3],	[[23 12.3],	[[24 12.3],	[[25 12.3],	[[26 12.3],	[[27 12.3],	[[28 12.3],	[[29 12.3],
[[30 12.3],	[[31 12.3],	[[32 12.3],	[[33 12.3],	[[34 12.3],	[[35 12.3],	[[36 12.3],	[[37 12.3],	[[38 12.3],	[[39 12.3],
[[40 12.3],	[[41 12.3],	[[42 12.3],	[[43 12.3],	[[44 12.3],	[[45 12.3],	[[46 12.3],	[[47 12.3],	[[48 12.3],	[[49 12.3],
[[50 12.3],	[[51 12.3],	[[52 12.3],	[[53 12.3],	[[54 12.3],	[[55 12.3],	[[56 12.3],	[[57 12.3],	[[58 12.3],	[[59 37.5],
[[60 37.5],	[[61 37.5],	[[62 37.5],	[[63 37.5],	[[64 64.0],	[[65 64.0],	[[66 24.2],	[[67 24.2],	[[68 24.2],	[[69 24.2],
[[70 24.2],	[[71 24.2],	[[72 24.2],	[[73 24.2],	[[74 24.2],	[[75 31.4],	[[76 31.4],	[[77 31.4],	[[78 38.9],	[[79 38.9],
[[80 38.9],	[[81 38.9],	[[82 42.8],	[[83 49.9],	[[84 56.9],	[[85 59.2],	[[86 63.8],	[[87 69.0],	[[88 70.7],	[[89 76.2],
[[90 76.2],	[[91 76.2],	[[92 77.6],	[[93 79.5],	[[94 80.8],	[[95 82.5],	[[96 83.7],	[[97 85.7],	[[98 85.7],	[[99 85.7],
[[100 85.7],	[[101 85.7],	[[102 85.7],	[[103 85.7],	[[104 85.7],	[[105 85.7],	[[106 85.7],	[[107 87.6],	[[108 88.9],	[[109 88.9],
[[110 90.8],	[[111 93.8],	[[112 96.6],	[[113 97.8],	[[114 97.8],	[[115 99.2],	[[116 102.0],	[[117 102.0],	[[118 103.7],	[[119 103.7],
[[120 106.9],	[[121 106.9],	[[122 108.3],	[[123 111.1],	[[124 112.8],	[[125 113.9],	[[126 116.6],	[[127 118.5],	[[128 120.1],	[[129 120.9],
[[130 120.9],	[[131 122.9],	[[132 125.0],	[[133 126.9],	[[134 128.0],	[[135 130.2],	[[136 130.8],	[[137 130.8],	[[138 131.3],	[[139 127.5],
[[140 127.6],	[[141 130.5],	[[142 131.5],	[[143 131.5],	[[144 131.5],	[[145 131.5],	[[146 131.5],	[[147 131.5],	[[148 132.8],	[[149 132.8],
[[150 132.8],	[[151 132.9],	[[152 129.6],	[[153 129.3],	[[154 127.7],	[[155 127.0],	[[156 127.0],	[[157 116.8],	[[158 119.9],	[[159 125.4],
[[160 124.0],	[[161 124.0],	[[162 124.0],	[[163 126.0],	[[164 128.0],	[[165 128.0],	[[166 128.0],	[[167 131.2],	[[168 134.1],	[[169 133.5],
[[170 132.2],	[[171 132.2],	[[172 132.2],	[[173 132.2],	[[174 132.2],	[[175 132.2],	[[176 132.2],	[[177 139.8],	[[178 139.8],	[[179 139.8],
[[180 143.2],	[[181 143.2],	[[182 143.2],	[[183 143.2],	[[184 146.3],	[[185 149.1],	[[186 149.1],	[[187 149.1],	[[188 149.1],	[[189 150.3],
[[190 150.3],	[[191 153.2],	[[192 155.8],	[[193 155.8],	[[194 155.8],	[[195 160.4],	[[196 160.8],	[[197 165.5],	[[198 165.5],	[[199 165.5],
[[200 167.6],	[[201 170.3],	[[202 170.3],	[[203 171.1],	[[204 171.6],	[[205 171.6],	[[206 171.6],	[[207 171.6],	[[208 174.1],	[[209 174.1],
[[210 174.1],	[[211 174.1],	[[212 174.1],	[[213 174.1],	[[214 174.1],	[[215 177.5],	[[216 177.5],	[[217 182.1],	[[218 183.0],	[[219 185.1],
[[220 186.2],	[[221 186.2],	[[222 186.9],	[[223 186.9],	[[224 186.9],	[[225 186.9],	[[226 186.9],	[[227 186.9],	[[228 186.9],	[[229 186.9],
[[230 189.9],	[[231 189.5],	[[232 189.5],	[[233 189.5],	[[234 189.5],	[[235 189.5],	[[236 195.3],	[[237 198.1],	[[238 198.0],	[[239 197.8],
[[240 200.6],	[[241 196.5],	[[242 195.1],	[[243 197.1],	[[244 197.1],	[[245 195.8],	[[246 200.6],	[[247 199.5],	[[248 213.5],	[[249 212.2],
[[250 212.2],	[[251 212.2],	[[252 210.2],	[[253 210.2],	[[254 213.6],	[[255 221.4],	[[256 220.3],	[[257 220.3],	[[258 213.1],	[[259 217.1],
[[260 217.1],	[[261 219.7],	[[262 217.3],	[[263 217.3],	[[264 220.2],	[[265 210.6],	[[266 210.6],	[[267 210.6],	[[268 212.7],	[[269 212.7],
[[270 219.4],	[[271 217.8],	[[272 215.9],	[[273 223.2],	[[274 223.2],	[[275 223.2],	[[276 223.2],	[[277 223.2],	[[278 223.2],	[[279 220.5],
[[280 220.5],	[[281 225.2],	[[282 227.3],	[[283 227.3],	[[284 230.9],	[[285 230.9],	[[286 230.9],	[[287 235.2],	[[288 235.2],	[[289 239.1],
[[290 239.1],	[[291 242.5],	[[292 242.5],	[[293 242.5],	[[294 242.5],	[[295 251.3],	[[296 250.7],	[[297 250.1],	[[298 249.8],	[[299 253.5],
[[300 253.5],	[[301 257.7],	[[302 257.7],	[[303 260.1],	[[304 262.2],	[[305 262.1],	[[306 266.7],	[[307 266.7],	[[308 271.4],	[[309 261.9],
[[310 266.7],	[[311 263.9],	[[312 255.1],	[[313 257.7],	[[314 257.7],	[[315 258.0],	[[316 249.3],	[[317 250.7],	[[318 254.0],	[[319 257.0],
[[320 250.9],	[[321 247.3],	[[322 241.1],	[[323 231.2],	[[324 237.9],	[[325 241.1],	[[326 229.6],	[[327 234.5],	[[328 226.6],	[[329 225.3],
[[330 225.1],	[[331 233.0],	[[332 229.8],	[[333 236.4],	[[334 237.3],	[[335 237.7],				

[[336 243.3],	[[337 244.6],	[[338 247.6],	[[339 251.0],	[[340 250.8],	[[341 257.0],
[[342 264.4],	[[343 266.0],	[[344 265.6],	[[345 267.9],	[[346 270.7],	[[347 269.9],
[[348 266.1],	[[349 262.4],	[[350 261.3],	[[351 264.3],	[[352 262.2],	[[353 260.0],
[[354 262.7],	[[355 260.9],	[[356 258.1],	[[357 252.3],	[[358 259.7],	[[359 255.2],
[[360 254.1],	[[361 255.5],	[[362 260.4],	[[363 256.8],	[[364 258.9],	[[365 258.9],
[[366 261.4],	[[367 264.5],	[[368 263.5],	[[369 265.3],	[[370 265.3],	[[371 264.8],
[[372 262.0],	[[373 262.0],	[[374 265.3],	[[375 268.5],	[[376 267.7],	[[377 267.7],
[[378 268.4],	[[379 276.0],	[[380 276.0],	[[381 287.3],	[[382 288.8],	[[383 290.0],
[[384 292.0],	[[385 294.9],	[[386 287.8],	[[387 290.7],	[[388 288.4],	[[389 291.2],
[[390 297.0],	[[391 296.3],	[[392 288.1],	[[393 284.3],	[[394 274.6],	[[395 268.2],
[[396 269.3],	[[397 269.2],	[[398 269.7],	[[399 267.5],	[[400 270.8],	[[401 275.0],
[[402 275.0],	[[403 275.3],	[[404 275.7],	[[405 273.4],	[[406 281.9],	[[407 285.9],
[[408 278.0],	[[409 271.9],	[[410 267.4],	[[411 270.9],	[[412 262.4],	[[413 262.1],
[[414 253.6],	[[415 253.6],	[[416 263.4],	[[417 263.4],	[[418 267.4],	[[419 261.2],
[[420 261.2],	[[421 261.2],	[[422 262.4],	[[423 258.3],	[[424 258.3],	[[425 253.6],
[[426 253.6],	[[427 248.5],	[[428 242.4],	[[429 248.6],	[[430 233.9],	[[431 229.0],
[[432 233.5],	[[433 226.7],	[[434 227.7],	[[435 220.8],	[[436 225.9],	[[437 231.6],
[[438 227.9],	[[439 227.6],	[[440 219.9],	[[441 221.4],	[[442 219.0],	[[443 219.3],
[[444 223.0],	[[445 220.9],	[[446 222.1],	[[447 235.7],	[[448 239.3],	[[449 251.4],
[[450 247.0],	[[451 246.4],	[[452 246.4],	[[453 242.5],	[[454 242.9],	[[455 237.2],
[[456 250.3],	[[457 256.1],	[[458 256.8],	[[459 263.7],	[[460 264.7],	[[461 259.9],
[[462 255.0],	[[463 255.0],	[[464 247.2],	[[465 233.1],	[[466 233.1],	[[467 239.8],
[[468 233.5],	[[469 234.3],	[[470 227.2],	[[471 222.8],	[[472 214.4],	[[473 226.1],
[[474 216.5],	[[475 209.5],	[[476 204.9],	[[477 187.6],	[[478 184.7],	[[479 180.1],
[[480 179.6],	[[481 182.7],	[[482 186.8],	[[483 183.8],	[[484 184.6],	[[485 190.3],
[[486 184.2],	[[487 188.2],	[[488 185.7],	[[489 191.7],	[[490 195.1],	[[491 194.1],
[[492 185.7],	[[493 187.8],	[[494 194.9],	[[495 182.0],	[[496 186.9],	[[497 185.0],
[[498 185.2],	[[499 178.6],	[[500 177.4],	[[501 179.7],	[[502 178.2],	[[503 181.6],
[[504 176.0],	[[505 175.5],	[[506 175.5],	[[507 173.1],	[[508 170.1],	[[509 164.2],
[[510 166.1],	[[511 161.2],	[[512 162.3],	[[513 165.7],	[[514 161.1],	[[515 162.3],
[[516 155.7],	[[517 156.9],	[[518 159.0],	[[519 159.9],	[[520 150.4],	[[521 138.9],
[[522 138.9],	[[523 137.4],	[[524 135.6],	[[525 146.9],	[[526 143.9],	[[527 137.2],
[[528 129.6],	[[529 132.8],	[[530 135.9],	[[531 142.2],	[[532 142.2],	[[533 140.5],
[[534 142.6],	[[535 147.8],	[[536 166.8],	[[537 166.8],	[[538 167.9],	[[539 165.9],
[[540 166.7],	[[541 166.0],	[[542 170.5],	[[543 168.9],	[[544 170.1],	[[545 170.5],
[[546 163.6],	[[547 157.7],	[[548 157.6],	[[549 149.8],	[[550 144.3],	[[551 141.6],
[[552 137.2],	[[553 140.4],	[[554 139.1],	[[555 141.5],	[[556 143.3],	[[557 142.7],
[[558 141.2],	[[559 139.5],	[[560 141.3],	[[561 135.3],	[[562 140.5],	[[563 140.7],
[[564 136.3],	[[565 133.4],	[[566 136.8],	[[567 137.5],	[[568 137.4],	[[569 137.4],
[[570 129.3],	[[571 123.6],	[[572 127.3],	[[573 139.0],	[[574 140.0],	[[575 142.2],
[[576 140.9],	[[577 145.4],	[[578 138.6],	[[579 139.2],	[[580 137.4],	[[581 141.8],
[[582 141.8],	[[583 149.3],	[[584 154.1],	[[585 154.3],	[[586 148.8],	[[587 147.7],
[[588 139.4],	[[589 143.2],	[[590 145.3],	],		
////////////////////////////////////					
////////////////////////////////////					
////////					
[[0 N/A],	[[1 N/A],	[[2 N/A],	[[3 [3.0 3.0 3.0],	],	[[4 [3.0 3.0 3.0],
],	],	],	],	],	[[5 [3.0 3.0 5.0],
],	],	],	],	],	[[6 [3.0 3.0 5.0],
],	],	],	],	],	[[7 [3.0 4.0 6.5],
],	],	],	],	],	[[8 [3.0 4.0 6.5],
],	],	],	],	],	[[9 [3.0 5.0 8.0],
],	],	],	],	],	[[10 [3.0 5.0 8.0],
],	],	],	],	],	[[11 [3.0 5.0 8.0],
],	],	],	],	],	[[12 [3.0 6.0 9.8],
],	],	],	],	],	[[13 [3.0 6.0 9.8],
],	],	],	],	],	[[14 [3.0 6.0 9.8],
],	],	],	],	],	[[15 [5.5 8.0 11.2],
],	],	],	],	],	[[16 [6.0 9.0 14.0],
],	],	],	],	],	[[17 [9.0 12.0 16.0],
],	],	],	],	],	[[18 [9.0 12.0 16.0],
],	],	],	],	],	[[19 [9.0 12.0 16.0],
],	],	],	],	],	[[20 [9.0 12.0 16.0],
],	],	],	],	],	[[21 [9.0 12.0 16.0],
],	],	],	],	],	],

[22 [9.0 12.0 16.0],],	[23 [9.0 12.0 16.0],],	[24 [9.0 12.0 16.0],],	[25 [9.0 12.0 16.0],],
[26 [9.0 12.0 16.0],],	[27 [9.0 12.0 16.0],],	[28 [9.0 12.0 16.0],],	[29 [9.0 12.0 16.0],],
[30 [9.0 12.0 16.0],],	[31 [9.0 12.0 16.0],],	[32 [9.0 12.0 16.0],],	[33 [9.0 12.0 16.0],],
[34 [9.0 12.0 16.0],],	[35 [9.0 12.0 16.0],],	[36 [9.0 12.0 16.0],],	[37 [9.0 12.0 16.0],],
[38 [9.0 12.0 16.0],],	[39 [9.0 12.0 16.0],],	[40 [9.0 12.0 16.0],],	[41 [9.0 12.0 16.0],],
[42 [9.0 12.0 16.0],],	[43 [9.0 12.0 16.0],],	[44 [9.0 12.0 16.0],],	[45 [9.0 12.0 16.0],],
[46 [9.0 12.0 16.0],],	[47 [9.0 12.0 16.0],],	[48 [9.0 12.0 16.0],],	[49 [9.0 12.0 16.0],],
[50 [9.0 12.0 16.0],],	[51 [9.0 12.0 16.0],],	[52 [9.0 12.0 16.0],],	[53 [9.0 12.0 16.0],],
[54 [9.0 12.0 16.0],],	[55 [9.0 12.0 16.0],],	[56 [9.0 12.0 16.0],],	[57 [9.0 12.0 16.0],],
[58 [9.0 12.0 16.0],],	[59 [5.2 37.5 69.8],],	[60 [5.2 37.5 69.8],],	[61 [5.2 37.5 69.8],],
[62 [5.2 37.5 69.8],],	[63 [5.2 37.5 69.8],],	[64 [64.0 64.0 64.0],],	[65 [64.0 64.0 64.0],],
[66 [4.5 10.5 60.2],],	[67 [4.5 10.5 60.2],],	[68 [4.5 10.5 60.2],],	[69 [4.5 10.5 60.2],],
[70 [4.5 10.5 60.2],],	[71 [4.5 10.5 60.2],],	[72 [4.5 10.5 60.2],],	[73 [4.5 10.5 60.2],],
[74 [4.5 10.5 60.2],],	[75 [6.5 14.0 64.0],],	[76 [6.5 14.0 64.0],],	[77 [6.5 14.0 64.0],],
[78 [8.5 37.5 66.8],],	[79 [8.5 37.5 66.8],],	[80 [8.5 37.5 66.8],],	[81 [8.5 37.5 66.8],],
[82 [9.0 59.0 75.0],],	[83 [12.0 64.0 78.0],],	[84 [16.0 64.0 82.0],],	[85 [26.8 69.5 82.8],],
[86 [59.0 75.0 83.0],],	[87 [62.8 76.5 83.2],],	[88 [64.0 78.0 84.0],],	[89 [64.0 80.0 84.2],],
[90 [64.0 80.0 84.2],],	[91 [64.0 80.0 84.2],],	[92 [64.0 82.0 85.0],],	[93 [72.2 82.5 85.8],],
[94 [75.0 83.0 88.0],],	[95 [77.2 83.5 89.0],],	[96 [78.0 84.0 92.0],],	[97 [81.0 84.5 92.5],],
[98 [81.0 84.5 92.5],],	[99 [81.0 84.5 92.5],],	[100 [81.0 84.5 92.5],],	[101 [81.0 84.5 92.5],],
[102 [81.0 84.5 92.5],],	[103 [81.0 84.5 92.5],],	[104 [81.0 84.5 92.5],],	[105 [81.0 84.5 92.5],],
[106 [81.0 84.5 92.5],],	[107 [82.0 85.0 94.0],],	[108 [82.8 86.5 94.5],],	[109 [82.8 86.5 94.5],],
[110 [83.0 88.0 96.0],],	[111 [84.0 92.0 107.0],],	[112 [84.5 94.0 110.5],],	[113 [85.8 95.0 110.8],],
[114 [85.8 95.0 110.8],],	[115 [86.5 96.0 111.5],],	[116 [92.0 107.0 112.0],],	[117 [92.0 107.0 112.0],],
[118 [93.5 108.5 112.0],],	[119 [93.5 108.5 112.0],],	[120 [95.5 110.5 112.8],],	[121 [95.5 110.5 112.8],],
[122 [96.0 111.0 115.0],],	[123 [107.0 112.0 120.0],],	[124 [109.2 112.0 120.5],],	[125 [110.0 112.0 122.0],],
[126 [111.0 115.0 123.0],],	[127 [112.0 120.0 125.0],],	[128 [112.0 122.0 126.0],],	[129 [114.2 122.5 126.2],],
[130 [114.2 122.5 126.2],],	[131 [118.8 124.0 127.2],],	[132 [121.0 126.0 131.5],],	[133 [122.8 127.5 132.0],],
[134 [124.0 128.0 132.0],],	[135 [126.5 132.0 133.0],],	[136 [127.2 132.0 133.0],],	[137 [127.2 132.0 133.0],],
[138 [128.0 132.0 133.0],],	[139 [127.5 132.0 134.0],],	[140 [128.8 132.0 134.5],],	[141 [131.8 133.0 139.5],],
[142 [132.0 135.0 141.0],],	[143 [132.2 135.0 141.0],],	[144 [132.2 135.0 141.0],],	[145 [133.0 135.0 141.0],],
[146 [133.0 135.0 141.0],],	[147 [133.0 135.0 141.0],],	[148 [133.0 137.0 141.0],],	[149 [133.0 137.0 141.0],],
[150 [133.0 137.0 141.0],],	[151 [133.0 139.0 141.0],],	[152 [133.0 137.0 141.0],],	[153 [133.0 139.0 141.0],],
[154 [103.0 140.0 141.8],],	[155 [93.0 141.0 142.0],],	[156 [93.0 141.0 142.0],],	[157 [91.5 140.0 141.8],],
[158 [92.0 141.0 145.0],],	[159 [103.0 141.0 152.5],],	[160 [93.0 141.0 154.0],],	[161 [93.0 141.0 154.0],],
[162 [93.0 141.0 154.0],],	[163 [93.0 142.0 158.0],],	[164 [93.0 148.0 159.0],],	[165 [93.0 148.0 159.0],],
[166 [93.0 148.0 159.0],],	[167 [103.0 151.0 162.0],],	[168 [113.0 154.0 163.5],],	[169 [103.0 156.0 163.8],],
[170 [93.0 158.0 164.0],],	[171 [93.0 158.0 164.0],],	[172 [93.0 158.0 164.0],],	[173 [93.0 158.0 164.0],],
[174 [93.0 158.0 164.0],],	[175 [93.0 158.0 164.0],],	[176 [93.0 158.0 164.0],],	[177 [133.0 159.0 167.0],],
[178 [133.0 159.0 167.0],],	[179 [133.0 159.0 167.0],],	[180 [138.2 161.0 167.8],],	[181 [138.2 161.0 167.8],],
[182 [138.2 161.0 167.8],],	[183 [138.2 161.0 167.8],],	[184 [143.5 163.0 172.5],],	[185 [148.8 163.5 177.8],],
[186 [148.8 163.5 177.8],],	[187 [148.8 163.5 177.8],],	[188 [148.8 163.5 177.8],],	[189 [156.0 164.0 178.5],],
[190 [156.0 164.0 178.5],],			

[[191 [157.0 165.5 181.0],],	[[192 [158.0 167.0 184.0],],	[[193 [158.0 167.0 184.0],],
[[194 [158.0 167.0 184.0],],	[[195 [158.5 168.0 188.0],],	[[196 [160.0 172.5 189.5],],
[[197 [163.0 177.0 191.0],],	[[198 [163.0 177.0 191.0],],	[[199 [163.0 177.0 191.0],],
[[200 [163.2 178.5 191.8],],	[[201 [164.8 182.0 194.2],],	[[202 [164.8 182.0 194.2],],
[[203 [167.0 184.0 195.0],],	[[204 [167.8 184.5 195.0],],	[[205 [167.8 184.5 195.0],],
[[206 [167.8 184.5 195.0],],	[[207 [167.8 184.5 195.0],],	[[208 [168.0 185.0 195.0],],
[[209 [168.0 185.0 195.0],],	[[210 [168.0 185.0 195.0],],	[[211 [168.0 185.0 195.0],],
[[212 [168.0 185.0 195.0],],	[[213 [168.0 185.0 195.0],],	[[214 [168.0 185.0 195.0],],
[[215 [177.0 191.0 200.0],],	[[216 [177.0 191.0 200.0],],	[[217 [178.5 192.0 204.5],],
[[218 [181.0 193.5 206.2],],	[[219 [182.0 195.0 211.5],],	[[220 [184.2 195.0 213.2],],
[[221 [184.2 195.0 213.2],],	[[222 [185.0 195.0 215.0],],	[[223 [185.0 195.0 215.0],],
[[224 [185.0 195.0 215.0],],	[[225 [185.0 195.0 215.0],],	[[226 [185.0 195.0 215.0],],
[[227 [185.0 195.0 215.0],],	[[228 [185.0 195.0 215.0],],	[[229 [185.0 195.0 215.0],],
[[230 [191.0 200.0 217.0],],	[[231 [189.2 200.5 217.0],],	[[232 [189.2 200.5 217.0],],
[[233 [189.2 200.5 217.0],],	[[234 [189.2 200.5 217.0],],	[[235 [189.2 200.5 217.0],],
[[236 [191.2 204.5 218.5],],	[[237 [192.0 211.5 227.2],],	[[238 [191.0 215.0 230.0],],
[[239 [189.2 216.0 231.5],],	[[240 [191.0 217.0 236.0],],	[[241 [184.0 217.0 236.0],],
[[242 [183.0 217.0 236.0],],	[[243 [181.0 217.0 236.0],],	[[244 [181.0 217.0 236.0],],
[[245 [180.0 217.0 236.0],],	[[246 [181.0 223.5 239.2],],	[[247 [180.0 230.0 240.0],],
[[248 [183.0 233.0 240.8],],	[[249 [182.0 236.0 241.5],],	[[250 [182.0 236.0 241.5],],
[[251 [182.0 236.0 241.5],],	[[252 [181.0 222.0 242.2],],	[[253 [181.0 222.0 242.2],],
[[254 [182.0 236.0 244.5],],	[[255 [186.8 238.5 252.0],],	[[256 [184.0 240.0 254.0],],
[[257 [184.0 240.0 254.0],],	[[258 [181.0 238.0 252.0],],	[[259 [181.0 241.5 254.8],],
[[260 [181.0 241.5 254.8],],	[[261 [182.0 243.0 255.0],],	[[262 [180.0 246.0 255.0],],
[[263 [180.0 246.0 255.0],],	[[264 [181.0 246.0 255.0],],	[[265 [179.2 246.0 255.0],],
[[266 [179.2 246.0 255.0],],	[[267 [179.2 246.0 255.0],],	[[268 [180.0 246.0 255.0],],
[[269 [180.0 246.0 255.0],],	[[270 [182.0 254.0 260.0],],	[[271 [181.0 254.5 260.5],],
[[272 [180.0 255.0 261.0],],	[[273 [182.0 255.0 262.5],],	[[274 [182.0 255.0 262.5],],
[[275 [182.0 255.0 262.5],],	[[276 [182.0 255.0 262.5],],	[[277 [182.0 255.0 262.5],],
[[278 [182.0 255.0 262.5],],	[[279 [181.0 255.0 263.2],],	[[280 [181.0 255.0 263.2],],
[[281 [182.0 255.0 267.0],],	[[282 [182.0 259.0 270.0],],	[[283 [182.0 259.0 270.0],],
[[284 [198.8 260.0 270.0],],	[[285 [198.8 260.0 270.0],],	[[286 [198.8 260.0 270.0],],
[[287 [217.5 261.0 275.5],],	[[288 [217.5 261.0 275.5],],	[[289 [236.2 262.5 281.2],],
[[290 [236.2 262.5 281.2],],	[[291 [255.0 264.0 282.0],],	[[292 [255.0 264.0 282.0],],
[[293 [255.0 264.0 282.0],],	[[294 [255.0 264.0 282.0],],	[[295 [255.0 270.0 289.5],],
[[296 [255.0 270.0 290.0],],	[[297 [255.0 281.0 293.0],],	[[298 [256.0 281.5 294.0],],
[[299 [259.0 282.0 295.0],],	[[300 [259.0 282.0 295.0],],	[[301 [258.0 284.5 295.0],],
[[302 [258.0 284.5 295.0],],	[[303 [259.0 287.0 295.0],],	[[304 [262.8 288.0 295.5],],
[[305 [270.0 289.0 296.0],],	[[306 [281.0 291.0 297.0],],	[[307 [281.0 291.0 297.0],],
[[308 [287.0 295.0 305.5],],	[[309 [273.0 295.0 306.0],],	[[310 [287.0 297.0 308.0],],
[[311 [273.0 303.0 308.0],],	[[312 [219.5 303.0 308.0],],	[[313 [239.2 303.5 308.2],],
[[314 [239.2 303.5 308.2],],	[[315 [232.0 303.0 308.5],],	[[316 [192.5 304.0 309.5],],
[[317 [198.8 298.5 310.0],],	[[318 [232.0 306.0 310.0],],	[[319 [245.5 307.0 313.5],],
[[320 [205.0 306.0 313.0],],	[[321 [205.0 291.0 313.0],],	[[322 [133.0 298.5 314.5],],
[[323 [109.0 289.0 315.0],],	[[324 [112.0 290.0 315.2],],	[[325 [113.0 291.0 316.0],],
[[326 [111.0 273.0 314.5],],	[[327 [111.0 289.0 317.5],],	[[328 [108.0 280.0 316.8],],
[[329 [107.0 289.0 317.5],],	[[330 [106.8 280.0 318.2],],	[[331 [107.8 290.0 319.0],],
[[332 [109.5 289.0 319.0],],	[[333 [113.0 291.0 322.0],],	[[334 [113.0 299.5 324.2],],
[[335 [113.0 299.5 326.5],],	[[336 [122.5 308.0 327.0],],	[[337 [130.8 308.0 327.0],],
[[338 [139.0 308.0 327.0],],	[[339 [142.8 308.5 330.2],],	[[340 [144.0 308.0 331.0],],
[[341 [195.5 301.0 330.8],],	[[342 [215.0 308.5 332.5],],	[[343 [227.0 308.5 333.0],],

[[344 [227.0 308.5 333.0],],	[[345 [233.0 309.0 333.0],],	[[346 [235.0 311.0 337.0],],
[[347 [227.0 314.5 339.0],],	[[348 [215.0 310.5 338.8],],	[[349 [215.0 310.5 338.8],],
[[350 [204.5 310.5 339.8],],	[[351 [204.5 314.5 342.2],],	[[352 [191.0 316.0 343.0],],
[[353 [178.8 314.5 343.5],],	[[354 [191.0 316.0 345.0],],	[[355 [178.8 314.5 345.0],],
[[356 [154.2 314.5 345.8],],	[[357 [139.8 310.5 345.0],],	[[358 [140.5 316.0 346.5],],
[[359 [139.0 313.0 351.0],],	[[360 [138.0 308.0 349.0],],	[[361 [138.5 302.0 348.0],],
[[362 [186.0 308.0 349.0],],	[[363 [138.8 302.0 348.0],],	[[364 [138.8 310.5 352.0],],
[[365 [138.8 310.5 352.0],],	[[366 [154.0 313.0 353.0],],	[[367 [161.5 314.5 355.2],],
[[368 [154.0 313.0 353.0],],	[[369 [154.0 316.0 355.5],],	[[370 [154.0 316.0 355.5],],
[[371 [138.8 317.5 356.0],],	[[372 [138.5 316.0 355.5],],	[[373 [138.5 316.0 355.5],],
[[374 [138.5 316.0 355.5],],	[[375 [138.8 317.5 356.0],],	[[376 [138.5 319.0 356.0],],
[[377 [138.5 319.0 356.0],],	[[378 [138.5 319.0 358.5],],	[[379 [146.5 326.0 359.8],],
[[380 [146.5 326.0 359.8],],	[[381 [261.0 339.0 367.0],],	[[382 [261.0 342.0 369.0],],
[[383 [240.5 342.0 371.5],],	[[384 [240.5 343.0 374.5],],	[[385 [261.0 343.0 375.0],],
[[386 [240.5 342.0 374.5],],	[[387 [250.8 342.5 375.8],],	[[388 [240.5 342.0 376.5],],
[[389 [262.0 340.5 377.2],],	[[390 [264.0 342.5 379.5],],	[[391 [263.0 343.0 380.0],],
[[392 [261.0 342.0 379.0],],	[[393 [245.5 340.5 379.0],],	[[394 [178.5 339.0 380.0],],
[[395 [141.8 308.5 377.2],],	[[396 [130.0 340.5 379.0],],	[[397 [123.0 342.0 380.0],],
[[398 [123.0 342.0 381.0],],	[[399 [109.0 342.0 384.0],],	[[400 [123.0 342.0 385.5],],
[[401 [120.8 342.5 386.2],],	[[402 [120.8 342.5 386.2],],	[[403 [132.5 342.0 385.5],],
[[404 [132.5 342.0 385.5],],	[[405 [109.0 342.0 387.0],],	[[406 [144.2 347.0 387.8],],
[[407 [156.0 351.0 390.0],],	[[408 [143.0 343.0 390.0],],	[[409 [126.0 342.0 390.0],],
[[410 [117.5 340.5 388.5],],	[[411 [126.0 342.0 387.0],],	[[412 [126.0 339.0 381.0],],
[[413 [113.0 339.0 384.0],],	[[414 [109.0 285.0 384.0],],	[[415 [109.0 285.0 384.0],],
[[416 [120.5 313.5 388.5],],	[[417 [120.5 313.5 388.5],],	[[418 [113.0 342.0 390.0],],
[[419 [100.2 313.5 388.5],],	[[420 [100.2 313.5 388.5],],	[[421 [100.2 313.5 388.5],],
[[422 [104.5 342.0 390.0],],	[[423 [100.2 310.0 390.0],],	[[424 [100.2 310.0 390.0],],
[[425 [100.2 303.5 390.0],],	[[426 [100.2 303.5 390.0],],	[[427 [96.0 265.0 390.0],],
[[428 [93.5 256.0 385.5],],	[[429 [91.0 265.0 391.5],],	[[430 [82.5 247.0 387.0],],
[[431 [86.0 247.0 384.0],],	[[432 [84.2 247.0 385.5],],	[[433 [82.5 247.0 377.5],],
[[434 [80.8 247.0 370.5],],	[[435 [78.5 216.0 369.5],],	[[436 [79.2 218.0 370.5],],
[[437 [77.0 247.0 371.0],],	[[438 [73.8 218.0 374.2],],	[[439 [70.5 216.0 377.5],],
[[440 [63.2 202.5 370.5],],	[[441 [63.5 189.0 377.5],],	[[442 [63.0 189.0 384.0],],
[[443 [63.2 202.5 380.8],],	[[444 [63.2 202.5 390.8],],	[[445 [63.5 189.0 388.5],],
[[446 [63.2 202.5 390.8],],	[[447 [64.0 228.0 393.0],],	[[448 [75.0 228.0 393.0],],
[[449 [86.5 237.5 425.2],],	[[450 [77.8 237.5 425.2],],	[[451 [77.2 237.5 403.8],],
[[452 [77.2 237.5 403.8],],	[[453 [78.0 228.0 393.0],],	[[454 [78.0 228.0 393.0],],
[[455 [77.2 222.0 393.0],],	[[456 [78.0 247.0 437.0],],	[[457 [80.0 280.0 438.5],],
[[458 [82.5 247.0 438.0],],	[[459 [84.2 280.0 440.2],],	[[460 [82.5 313.0 441.5],],
[[461 [80.8 280.0 442.2],],	[[462 [80.8 251.0 426.0],],	[[463 [80.8 251.0 426.0],],
[[464 [85.0 236.0 393.0],],	[[465 [79.0 188.0 393.0],],	[[466 [79.0 188.0 393.0],],
[[467 [83.5 200.0 404.0],],	[[468 [85.2 177.0 393.0],],	[[469 [86.2 177.0 393.0],],
[[470 [85.0 165.0 393.0],],	[[471 [79.0 165.0 384.0],],	[[472 [79.0 159.0 374.2],],
[[473 [80.5 177.0 384.0],],	[[474 [79.0 153.0 384.0],],	[[475 [77.0 151.0 377.5],],
[[476 [79.0 139.0 370.5],],	[[477 [79.0 114.0 308.0],],	[[478 [76.0 113.5 296.8],],
[[479 [72.8 113.5 261.0],],	[[480 [72.8 112.0 261.0],],	[[481 [68.8 112.0 274.2],],
[[482 [72.0 113.0 308.0],],	[[483 [65.5 113.0 285.5],],	[[484 [62.2 112.0 296.8],],
[[485 [65.5 113.0 338.0],],	[[486 [55.0 104.0 308.0],],	[[487 [54.0 111.0 338.0],],
[[488 [54.5 107.5 323.0],],	[[489 [55.0 111.0 368.0],],	[[490 [55.0 104.0 382.0],],
[[491 [53.0 104.0 382.0],],	[[492 [50.0 88.0 368.0],],	[[493 [49.2 96.0 371.5],],
[[494 [49.2 108.5 382.5],],	[[495 [43.5 88.0 338.0],],	[[496 [47.0 104.0 368.0],],

```

|[497|[47.0|104.0|368.0], ], |[498|[45.2|96.0|370.5], ], |[499|[45.2|93.0|323.0], ],
|[500|[45.2|93.0|323.0], ], |[501|[48.5|98.0|338.0], ], |[502|[47.8|93.0|353.0], ],
|[503|[48.5|98.0|373.0], ], |[504|[47.0|88.0|368.0], ], |[505|[47.8|93.0|353.0], ],
|[506|[51.5|98.0|338.0], ], |[507|[47.5|93.0|323.0], ], |[508|[45.0|88.0|281.5], ],
|[509|[42.5|87.5|253.8], ], |[510|[45.0|88.0|261.5], ], |[511|[50.0|88.0|250.0], ],
|[512|[50.8|93.0|246.5], ], |[513|[47.5|93.0|251.2], ], |[514|[45.0|88.0|243.0], ],
|[515|[50.0|98.0|250.0], ], |[516|[42.5|93.0|235.8], ], |[517|[40.0|98.0|236.0], ],
|[518|[40.0|98.0|236.0], ], |[519|[42.5|101.0|235.8], ], |[520|[40.0|87.5|233.0], ],
|[521|[39.0|85.0|227.0], ], |[522|[39.0|85.0|227.0], ], |[523|[39.8|86.0|215.0], ],
|[524|[40.0|85.0|211.0], ], |[525|[40.0|87.0|231.0], ], |[526|[39.2|86.0|223.0], ],
|[527|[39.2|85.5|204.8], ], |[528|[39.0|84.0|170.2], ], |[529|[39.2|85.5|180.8], ],
|[530|[39.5|86.0|178.5], ], |[531|[39.8|86.5|192.2], ], |[532|[39.8|86.5|192.2], ],
|[533|[43.8|86.5|192.2], ], |[534|[43.8|85.5|222.5], ], |[535|[47.5|87.0|227.5], ],
|[536|[52.2|100.0|236.0], ], |[537|[53.0|104.0|236.0], ], |[538|[52.8|104.0|238.5], ],
|[539|[52.8|104.0|236.0], ], |[540|[53.0|105.0|236.0], ], |[541|[53.0|106.0|236.0], ],
|[542|[53.8|109.0|236.0], ], |[543|[53.8|109.0|235.8], ], |[544|[53.0|109.0|238.5], ],
|[545|[53.0|111.5|238.5], ], |[546|[52.8|109.0|237.5], ], |[547|[52.8|106.5|235.2], ],
|[548|[40.0|106.0|236.0], ], |[549|[39.0|104.0|235.0], ], |[550|[39.0|103.0|196.5], ],
|[551|[39.8|100.0|184.5], ], |[552|[39.0|85.0|184.0], ], |[553|[39.0|84.0|184.5], ],
|[554|[39.0|83.0|184.0], ], |[555|[39.2|84.0|195.0], ], |[556|[40.0|85.0|214.0], ],
|[557|[41.5|91.5|224.5], ], |[558|[41.5|84.0|224.5], ], |[559|[39.5|83.0|221.0], ],
|[560|[40.0|80.0|214.0], ], |[561|[39.2|77.5|197.8], ], |[562|[47.0|80.5|210.0], ],
|[563|[48.0|81.0|221.0], ], |[564|[47.0|82.0|210.0], ], |[565|[43.0|80.0|205.5], ],
|[566|[43.0|80.0|205.5], ], |[567|[47.0|82.0|209.8], ], |[568|[50.0|83.0|214.0], ],
|[569|[50.0|83.0|214.0], ], |[570|[44.0|80.0|191.5], ], |[571|[44.0|78.0|169.5], ],
|[572|[45.0|79.0|201.2], ], |[573|[47.5|82.0|228.5], ], |[574|[48.0|81.0|214.0], ],
|[575|[50.8|86.0|217.5], ], |[576|[50.5|83.0|205.5], ], |[577|[49.0|88.0|229.0], ],
|[578|[50.0|88.0|228.0], ], |[579|[50.0|89.0|228.0], ], |[580|[49.0|89.0|212.5], ],
|[581|[48.5|88.5|229.5], ], |[582|[50.5|88.0|232.5], ], |[583|[51.0|89.0|236.0], ],
|[584|[51.2|98.0|240.5], ], |[585|[52.0|100.0|235.8], ], |[586|[51.5|98.0|232.5], ],
|[587|[51.2|93.0|229.5], ], |[588|[51.0|88.0|211.0], ], |[589|[51.2|93.0|207.5], ],
|[590|[52.2|100.0|207.5], ], ],
////////////////////////////////////
////////////////////////////////////
////////////////////////////////////
[[0|[0.0|0.0|0.0], ], |[1|[1.0|1.0|1.0], ], |[2|[2.0|2.0|2.0], ], |[3|[3.0|3.0|3.0], ],
|[4|[4.0|4.0|4.0], ], |[5|[5.0|5.0|5.0], ], |[6|[6.0|6.0|6.0], ], |[7|[7.0|7.0|7.0], ],
|[8|[8.0|8.0|8.0], ], |[9|[9.0|9.0|9.0], ], |[10|[10.0|10.0|10.0], ], |[11|[11.0|11.0|11.0], ],
|[12|[12.0|12.0|12.0], ], |[13|[13.0|13.0|13.0], ], |[14|[14.0|14.0|14.0], ],
|[15|[15.0|15.0|15.0], ], |[16|[16.0|16.0|16.0], ], |[17|[17.0|17.0|17.0], ],
|[18|[18.0|18.0|18.0], ], |[19|[19.0|19.0|19.0], ], |[20|[20.0|20.0|20.0], ],
|[21|[21.0|21.0|21.0], ], |[22|[22.0|22.0|22.0], ], |[23|[23.0|23.0|23.0], ],
|[24|[24.0|24.0|24.0], ], |[25|[25.0|25.0|25.0], ], |[26|[26.0|26.0|26.0], ],
|[27|[27.0|27.0|27.0], ], |[28|[28.0|28.0|28.0], ], |[29|[29.0|29.0|29.0], ],
|[30|[30.0|30.0|30.0], ], |[31|[31.0|31.0|31.0], ], |[32|[32.0|32.0|32.0], ],
|[33|[33.0|33.0|33.0], ], |[34|[34.0|34.0|34.0], ], |[35|[35.0|35.0|35.0], ],
|[36|[36.0|36.0|36.0], ], |[37|[37.0|37.0|37.0], ], |[38|[38.0|38.0|38.0], ],
|[39|[39.0|39.0|39.0], ], |[40|[40.0|40.0|40.0], ], |[41|[41.0|41.0|41.0], ],
|[42|[42.0|42.0|42.0], ], |[43|[43.0|43.0|43.0], ], |[44|[44.0|44.0|44.0], ],
|[45|[45.0|45.0|45.0], ], |[46|[46.0|46.0|46.0], ], |[47|[47.0|47.0|47.0], ],
|[48|[48.0|48.0|48.0], ], |[49|[49.0|49.0|49.0], ], |[50|[50.0|50.0|50.0], ],

```

[51 [51.0 51.0 51.0],	],	[52 [52.0 52.0 52.0],	],	[53 [53.0 53.0 53.0],	],
[54 [54.0 54.0 54.0],	],	[55 [55.0 55.0 55.0],	],	[56 [56.0 56.0 56.0],	],
[57 [57.0 57.0 57.0],	],	[58 [58.0 58.0 58.0],	],	[59 [59.0 59.0 59.0],	],
[60 [60.0 60.0 60.0],	],	[61 [61.0 61.0 61.0],	],	[62 [62.0 62.0 62.0],	],
[63 [63.0 63.0 63.0],	],	[64 [64.0 64.0 64.0],	],	[65 [65.0 65.0 65.0],	],
[66 [66.0 66.0 66.0],	],	[67 [67.0 67.0 67.0],	],	[68 [68.0 68.0 68.0],	],
[69 [69.0 69.0 69.0],	],	[70 [70.0 70.0 70.0],	],	[71 [71.0 71.0 71.0],	],
[72 [72.0 72.0 72.0],	],	[73 [73.0 73.0 73.0],	],	[74 [74.0 74.0 74.0],	],
[75 [75.0 75.0 75.0],	],	[76 [76.0 76.0 76.0],	],	[77 [77.0 77.0 77.0],	],
[78 [78.0 78.0 78.0],	],	[79 [79.0 79.0 79.0],	],	[80 [80.0 80.0 80.0],	],
[81 [81.0 81.0 81.0],	],	[82 [82.0 82.0 82.0],	],	[83 [83.0 83.0 83.0],	],
[84 [84.0 84.0 84.0],	],	[85 [85.0 85.0 85.0],	],	[86 [86.0 86.0 86.0],	],
[87 [87.0 87.0 87.0],	],	[88 [88.0 88.0 88.0],	],	[89 [89.0 89.0 89.0],	],
[90 [90.0 90.0 90.0],	],	[91 [91.0 91.0 91.0],	],	[92 [92.0 92.0 92.0],	],
[93 [93.0 93.0 93.0],	],	[94 [94.0 94.0 94.0],	],	[95 [95.0 95.0 95.0],	],
[96 [96.0 96.0 96.0],	],	[97 [97.0 97.0 97.0],	],	[98 [98.0 98.0 98.0],	],
[99 [99.0 99.0 99.0],	],	[100 [100.0 100.0 100.0],	],	[101 [101.0 101.0 101.0],	],
[102 [102.0 102.0 102.0],	],	[103 [103.0 103.0 103.0],	],	[104 [104.0 104.0 104.0],	],
[105 [105.0 105.0 105.0],	],	[106 [106.0 106.0 106.0],	],	[107 [107.0 107.0 107.0],	],
[108 [108.0 108.0 108.0],	],	[109 [109.0 109.0 109.0],	],	[110 [110.0 110.0 110.0],	],
[111 [111.0 111.0 111.0],	],	[112 [112.0 112.0 112.0],	],	[113 [113.0 113.0 113.0],	],
[114 [114.0 114.0 114.0],	],	[115 [115.0 115.0 115.0],	],	[116 [116.0 116.0 116.0],	],
[117 [117.0 117.0 117.0],	],	[118 [118.0 118.0 118.0],	],	[119 [119.0 119.0 119.0],	],
[120 [120.0 120.0 120.0],	],	[121 [121.0 121.0 121.0],	],	[122 [122.0 122.0 122.0],	],
[123 [123.0 123.0 123.0],	],	[124 [124.0 124.0 124.0],	],	[125 [125.0 125.0 125.0],	],
[126 [126.0 126.0 126.0],	],	[127 [127.0 127.0 127.0],	],	[128 [128.0 128.0 128.0],	],
[129 [129.0 129.0 129.0],	],	[130 [130.0 130.0 130.0],	],	[131 [131.0 131.0 131.0],	],
[132 [132.0 132.0 132.0],	],	[133 [133.0 133.0 133.0],	],	[134 [134.0 134.0 134.0],	],
[135 [135.0 135.0 135.0],	],	[136 [136.0 136.0 136.0],	],	[137 [137.0 137.0 137.0],	],
[138 [138.0 138.0 138.0],	],	[139 [139.0 139.0 139.0],	],	[140 [140.0 140.0 140.0],	],
[141 [141.0 141.0 141.0],	],	[142 [142.0 142.0 142.0],	],	[143 [143.0 143.0 143.0],	],
[144 [144.0 144.0 144.0],	],	[145 [145.0 145.0 145.0],	],	[146 [146.0 146.0 146.0],	],
[147 [147.0 147.0 147.0],	],	[148 [148.0 148.0 148.0],	],	[149 [149.0 149.0 149.0],	],
[150 [150.0 150.0 150.0],	],	[151 [151.0 151.0 151.0],	],	[152 [152.0 152.0 152.0],	],
[153 [153.0 153.0 153.0],	],	[154 [154.0 154.0 154.0],	],	[155 [155.0 155.0 155.0],	],
[156 [156.0 156.0 156.0],	],	[157 [157.0 157.0 157.0],	],	[158 [158.0 158.0 158.0],	],
[159 [159.0 159.0 159.0],	],	[160 [160.0 160.0 160.0],	],	[161 [161.0 161.0 161.0],	],
[162 [161.8 162.0 162.0],	],	[163 [162.5 163.0 163.0],	],	[164 [159.0 164.0 164.0],	],
[165 [160.0 165.0 165.0],	],	[166 [161.0 166.0 166.0],	],	[167 [157.0 167.0 167.0],	],
[168 [157.0 168.0 168.0],	],	[169 [158.0 169.0 169.0],	],	[170 [159.0 170.0 170.0],	],
[171 [160.0 171.0 171.0],	],	[172 [158.8 172.0 172.0],	],	[173 [159.8 173.0 173.0],	],
[174 [160.0 174.0 174.0],	],	[175 [161.0 175.0 175.0],	],	[176 [162.0 176.0 176.0],	],
[177 [151.0 177.0 177.0],	],	[178 [139.0 178.0 178.0],	],	[179 [140.0 179.0 179.0],	],
[180 [140.0 180.0 180.0],	],	[181 [139.0 181.0 181.0],	],	[182 [140.0 182.0 182.0],	],
[183 [141.0 183.0 183.0],	],	[184 [141.5 184.0 184.0],	],	[185 [141.0 185.0 185.0],	],
[186 [142.0 186.0 186.0],	],	[187 [143.0 187.0 187.0],	],	[188 [144.0 188.0 188.0],	],
[189 [142.5 189.0 189.0],	],	[190 [143.5 190.0 190.0],	],	[191 [143.5 191.0 191.0],	],
[192 [144.2 192.0 192.0],	],	[193 [145.2 193.0 193.0],	],	[194 [145.5 194.0 194.0],	],
[195 [146.0 195.0 195.0],	],	[196 [147.0 196.0 196.0],	],	[197 [148.0 197.0 197.0],	],
[198 [148.5 198.0 198.0],	],	[199 [149.5 199.0 199.0],	],	[200 [150.2 200.0 200.0],	],
[201 [151.0 201.0 201.0],	],	[202 [152.0 202.0 202.0],	],	[203 [150.0 203.0 203.0],	],

[204 [149.5 204.0 204.0],],	[205 [149.2 205.0 205.0],],	[206 [148.0 206.0 206.0],],
[207 [149.0 207.0 207.0],],	[208 [148.0 208.0 208.0],],	[209 [147.0 209.0 209.0],],
[210 [147.0 210.0 210.0],],	[211 [147.2 211.0 211.0],],	[212 [148.2 212.0 212.0],],
[213 [149.2 213.0 213.0],],	[214 [149.0 214.0 214.0],],	[215 [149.5 215.0 215.0],],
[216 [149.0 216.0 216.0],],	[217 [147.5 217.0 217.0],],	[218 [143.5 218.0 218.0],],
[219 [142.0 219.0 219.0],],	[220 [143.0 220.0 220.0],],	[221 [139.0 221.0 221.0],],
[222 [134.0 222.0 222.0],],	[223 [133.2 223.0 223.0],],	[224 [124.5 224.0 224.0],],
[225 [123.0 225.0 225.0],],	[226 [118.5 226.0 226.0],],	[227 [119.5 227.0 227.0],],
[228 [119.8 228.0 228.0],],	[229 [120.8 229.0 229.0],],	[230 [120.0 230.0 230.0],],
[231 [121.0 231.0 231.0],],	[232 [122.0 232.0 232.0],],	[233 [123.0 233.0 233.0],],
[234 [123.2 234.0 234.0],],	[235 [123.5 235.0 235.0],],	[236 [124.0 236.0 236.0],],
[237 [125.0 237.0 237.0],],	[238 [126.0 238.0 238.0],],	[239 [127.0 239.0 239.0],],
[240 [127.2 240.0 240.0],],	[241 [128.0 241.0 241.0],],	[242 [129.0 242.0 242.0],],
[243 [128.5 243.0 243.0],],	[244 [128.0 244.0 244.0],],	[245 [126.5 245.0 245.0],],
[246 [126.0 246.0 246.0],],	[247 [127.0 247.0 247.0],],	[248 [125.0 248.0 248.0],],
[249 [126.0 249.0 249.0],],	[250 [122.5 250.0 250.0],],	[251 [120.0 251.0 251.0],],
[252 [118.0 252.0 252.0],],	[253 [110.5 253.0 253.0],],	[254 [107.5 254.0 254.0],],
[255 [106.0 255.0 255.0],],	[256 [106.2 256.0 256.0],],	[257 [105.5 257.0 257.0],],
[258 [98.0 258.0 258.0],],	[259 [98.8 259.0 259.0],],	[260 [99.0 260.0 260.0],],
[261 [98.0 261.0 261.0],],	[262 [99.0 262.0 262.0],],	[263 [99.2 263.0 263.0],],
[264 [90.2 264.0 264.0],],	[265 [93.0 265.0 265.0],],	[266 [86.8 266.0 266.0],],
[267 [87.8 267.0 267.0],],	[268 [85.5 268.0 268.0],],	[269 [85.0 269.0 269.0],],
[270 [83.5 270.0 270.0],],	[271 [83.0 271.0 271.0],],	[272 [84.0 272.0 272.0],],
[273 [85.0 273.0 273.0],],	[274 [86.0 274.0 274.0],],	[275 [87.0 275.0 275.0],],
[276 [87.0 276.0 276.0],],	[277 [80.0 277.0 277.0],],	[278 [75.2 277.5 278.0],],
[279 [74.8 278.0 279.0],],	[280 [75.8 279.0 280.0],],	[281 [74.0 276.0 281.0],],
[282 [74.8 274.5 282.0],],	[283 [75.0 272.5 283.0],],	[284 [75.0 270.0 284.0],],
[285 [73.5 271.0 285.0],],	[286 [71.5 244.0 286.0],],	[287 [69.2 244.0 287.0],],
[288 [70.2 245.0 288.0],],	[289 [70.5 245.0 289.0],],	[290 [69.8 246.0 290.0],],
[291 [70.0 246.0 291.0],],	[292 [71.0 246.0 292.0],],	[293 [72.0 246.5 293.0],],
[294 [73.0 247.5 294.0],],	[295 [73.5 247.0 295.0],],	[296 [73.8 247.5 296.0],],
[297 [74.0 248.0 297.0],],	[298 [75.0 249.0 298.0],],	[299 [76.0 250.0 299.0],],
[300 [77.0 250.5 300.0],],	[301 [77.0 251.0 301.0],],	[302 [77.8 250.0 302.0],],
[303 [78.0 248.5 303.0],],	[304 [78.5 247.5 304.0],],	[305 [76.0 245.0 305.0],],
[306 [77.0 245.0 306.0],],	[307 [78.0 246.0 307.0],],	[308 [72.8 244.0 308.0],],
[309 [70.0 244.0 309.0],],	[310 [71.0 240.5 310.0],],	[311 [70.5 239.0 311.0],],
[312 [70.8 242.5 312.0],],	[313 [69.5 231.0 313.0],],	[314 [70.0 232.0 314.0],],
[315 [70.2 230.0 315.0],],	[316 [71.8 231.0 316.0],],	[317 [71.5 225.5 317.0],],
[318 [72.0 223.0 318.0],],	[319 [72.0 222.0 319.0],],	[320 [73.0 224.0 320.0],],
[321 [73.5 225.0 321.0],],	[322 [75.0 226.0 322.0],],	[323 [74.5 227.0 323.0],],
[324 [75.0 227.0 324.0],],	[325 [74.8 223.0 325.0],],	[326 [71.8 221.0 326.0],],
[327 [71.5 218.0 327.0],],	[328 [72.5 219.0 328.0],],	[329 [72.8 219.0 329.0],],
[330 [73.0 219.0 330.0],],	[331 [74.0 219.0 331.0],],	[332 [75.0 220.5 332.0],],
[333 [73.0 220.0 333.0],],	[334 [73.8 220.5 334.0],],	[335 [73.8 220.0 335.0],],
[336 [74.8 221.0 336.0],],	[337 [74.0 218.5 337.0],],	[338 [75.0 219.0 338.0],],
[339 [76.0 215.0 339.0],],	[340 [77.0 216.0 340.0],],	[341 [76.0 207.0 341.0],],
[342 [76.0 203.0 342.0],],	[343 [76.0 196.5 343.0],],	[344 [76.2 196.0 344.0],],
[345 [73.0 185.0 345.0],],	[346 [70.5 183.0 346.0],],	[347 [71.0 184.0 347.0],],
[348 [72.0 185.0 348.0],],	[349 [73.0 186.0 349.0],],	[350 [74.0 187.0 350.0],],
[351 [75.0 188.0 351.0],],	[352 [76.0 189.0 352.0],],	[353 [77.0 190.0 353.0],],
[354 [78.0 189.5 354.0],],	[355 [78.5 188.0 355.0],],	[356 [79.0 184.0 356.0],],

```

|[357|[80.0|185.0|357.0], ], |[358|[78.0|177.0|358.0], ], |[359|[79.0|177.0|359.0], ],
|[360|[77.2|173.0|360.0], ], |[361|[78.0|168.5|361.0], ], |[362|[79.0|169.5|362.0], ],
|[363|[80.0|166.0|363.0], ], |[364|[80.0|164.5|364.0], ], |[365|[80.0|161.5|365.0], ],
|[366|[80.2|161.5|366.0], ], |[367|[77.8|159.5|367.0], ], |[368|[76.2|159.5|368.0], ],
|[369|[77.0|160.0|369.0], ], |[370|[78.0|161.0|370.0], ], |[371|[77.5|162.0|371.0], ],
|[372|[74.0|159.0|372.0], ], |[373|[73.0|158.0|373.0], ], |[374|[72.8|158.0|374.0], ],
|[375|[72.0|154.0|375.0], ], |[376|[72.2|155.0|376.0], ], |[377|[70.0|156.0|377.0], ],
|[378|[70.0|155.0|378.0], ], |[379|[70.5|156.0|379.0], ], |[380|[68.5|156.5|380.0], ],
|[381|[67.0|152.0|381.0], ], |[382|[63.0|151.0|382.0], ], |[383|[60.8|146.5|383.0], ],
|[384|[60.8|145.0|384.0], ], |[385|[61.8|146.0|385.0], ], |[386|[61.5|147.0|386.0], ],
|[387|[62.2|148.0|387.0], ], |[388|[63.0|149.0|388.0], ], |[389|[63.8|146.5|389.0], ],
|[390|[60.0|146.0|390.0], ], |[391|[59.0|146.0|391.0], ], |[392|[60.0|147.0|392.0], ],
|[393|[60.0|146.0|393.0], ], |[394|[57.0|144.0|393.0], ], |[395|[55.0|140.0|389.0], ],
|[396|[56.0|140.0|385.0], ], |[397|[56.2|140.5|385.2], ], |[398|[54.0|141.0|360.0], ],
|[399|[55.0|14, ], |[545|[27.0|71.0|152.0], ], |[546|[26.8|72.0|152.0], ],
|[547|[27.0|72.0|153.0], ], |[548|[27.0|71.5|153.0], ], |[549|[27.0|72.0|152.5], ],
|[550|[27.0|72.0|153.0], ], |[551|[28.0|71.0|152.0], ], |[552|[28.0|72.0|153.0], ],
|[553|[28.0|71.5|152.8], ], |[554|[28.0|71.0|152.0], ], |[555|[27.0|71.0|151.8], ],
|[556|[28.0|71.0|152.0], ], |[557|[28.0|71.0|149.8], ], |[558|[27.5|71.0|149.0], ],
|[559|[28.0|71.0|149.0], ], |[560|[29.0|71.0|148.0], ], |[561|[29.0|71.0|149.0], ],
|[562|[29.5|70.0|148.0], ], |[563|[28.0|67.0|146.0], ], |[564|[28.0|67.0|146.0], ],
|[565|[28.5|67.0|147.0], ], |[566|[29.0|67.0|147.0], ], |[567|[28.0|66.0|147.0], ],
|[568|[28.0|65.0|145.5], ], |[569|[28.0|66.0|145.8], ], |[570|[28.0|66.0|146.0], ],
|[571|[28.0|66.0|146.8], ], |[572|[29.0|67.0|146.2], ], |[573|[29.0|66.0|143.0], ],
|[574|[29.0|67.0|144.0], ], |[575|[29.0|67.0|144.0], ], |[576|[29.0|67.5|140.2], ],
|[577|[29.0|67.0|140.5], ], |[578|[29.0|66.0|140.0], ], |[579|[29.0|65.5|140.0], ],
|[580|[29.0|66.0|140.5], ], |[581|[29.0|66.0|140.0], ], |[582|[29.0|65.0|139.2], ],
|[583|[29.0|65.0|138.8], ], |[584|[28.0|64.0|137.2], ], |[585|[28.0|64.0|135.5], ],
|[586|[28.0|64.5|136.0], ], |[587|[28.0|65.0|136.0], ], |[588|[28.0|65.0|136.0], ],
|[589|[28.0|65.0|136.0], ], |[590|[29.0|66.0|137.0], ], ],
////////////////////
////////////////////
////////
211.8

```

This data is the output of these lines in main.py:

```

print(to_display(testRealm.database.death_causes))
flood()
print(to_display(testRealm.database.avg_genomes))
flood()
print(to_display(testRealm.database.diets))
flood()
print(to_display(testRealm.database.avg_lifespans))
flood()
display(testRealm.database.lifespan_quartiles)
flood()
display(testRealm.database.age_quartiles)
flood()
display(testRealm.database.calc_total_avg_lifespan())

```

I analysed the data for key insights:

Unfortunately I could not see genomes or death causes because there were so many printed lines that anything before halfway through the diets cannot be seen in the terminal, as it only shows a set number of the most recent lines.

I was happy to see that meat eaters were the least frequent, but importantly stable. They fell in numbers quickly before increasing again, showing that meat eaters required the most specialisation (high strength and toughness in order to win fights; high speed in order to catch prey). Once the fittest meat eaters survived and reproduce, it was a viable diet option. Plant eaters, especially apple eaters, were more numerous, and this is also true to real life; the biomass of organisms lower in a food chain is greater than the biomass of those above them in the food chain; survival is harder for carnivores so the stable population range is one lower and thus with less competition.

The toxic nature of the previous iteration is shown clearly with the lavender eaters, who increased in number quite quickly when the big boom occurred. Lavendar does not reduce hunger so these animas were reproducing having never eaten (unless they were multi-diet). They received the benefits of reduced injury, but they would all have suffered guaranteed death by hunger as quickly as it was possible. This should not be so viable a strategy, and yet they thrived (they were the least populous of the plant-eaters, yet still seeing exponential increase). This shows the extent to which the time bought by giant-ness decreasing the hunger gain rate (ie abstracted metabolism) was enough for animas to meet the optimal strategy of simply making it to adulthood, mating as many times as possible, and having more oversized babies. Hopefully the new changes will cause greater diversity in sizes, or a less extreme optimum

Lifespan and age means and quartiles seem in order; however the realm was only 590 ticks old.

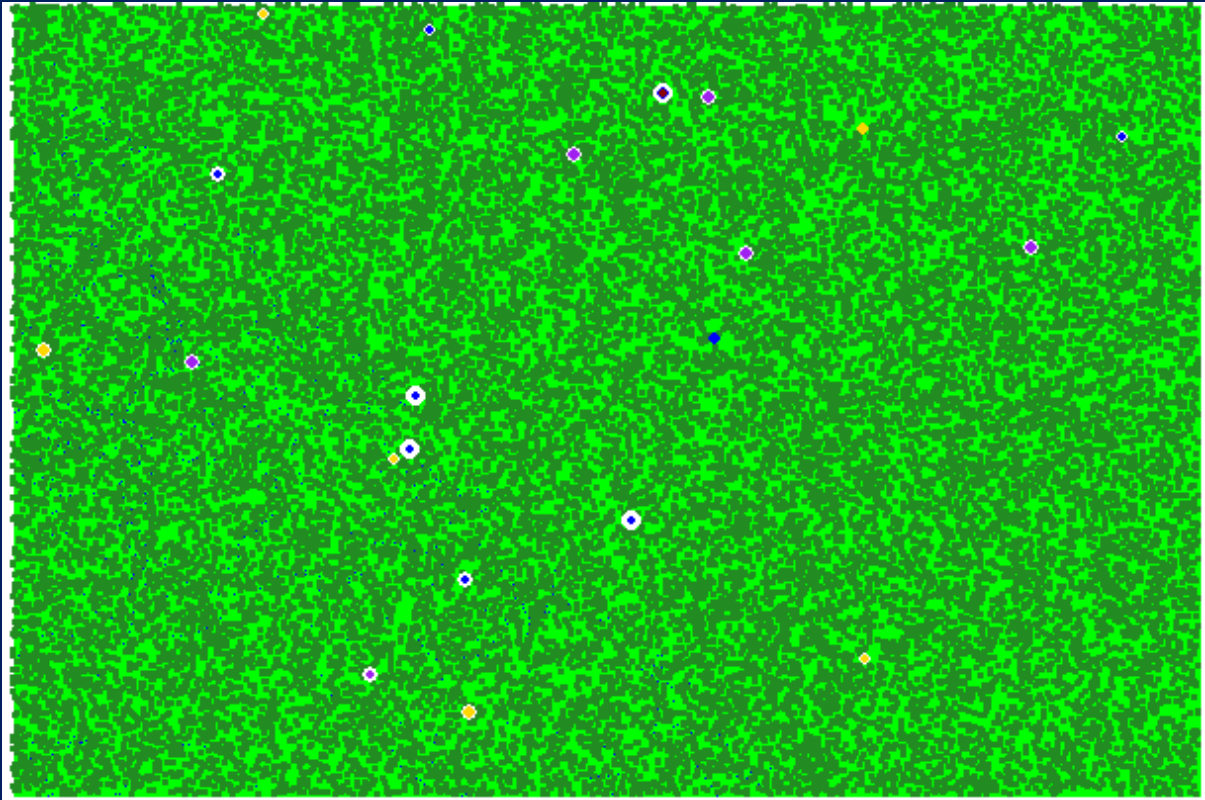
Now the database will update only once every ten ticks.

I added population numbers to the database.

I also added a single print line showing multi-diet animals at the end of the simulation.

With these few changes, I ran the simulation again.

Result:



Oops, I ran the simulation again without copying data; I'll run it again but first increase terminal scrollbar to 2500 lines, so all data can be captured. Data, starting from final database update (I later removed the print line in simulationdata.py causing the regular food list prints).

```
[['apple', 0], ['berry', 0], ['melon', 0], ['lily', 0], ['lavendar', 0], ['meat', 0], ['wildgrass', 0]]
['apple', 'berry', 'melon', 'lily', 'lavendar', 'meat', 'wildgrass']
Axag [wildgrass], 6.0 13.0 ate wildgrass
Baom [lily], 11.7 6.2 ate lily
Oodoo [melon], 16.7 15.3 ate melon
Cleli [wildgrass], 22.0 10.2 ate wildgrass
Uibifuny [wildgrass], 5.2 21.1 ate wildgrass
Wepy [wildgrass], 10.6 27.7 ate wildgrass
Ubuwuz [wildgrass], 6.7 5.8 ate wildgrass
Lefa [berry], 18.4 39.4 ate berry
Uidaddomool [apple], 21.0 22.9 ate apple
Axag [wildgrass], 6.0 13.0 ate wildgrass
Uinookik [apple], 22.1 26.8 ate apple
Blue Moum [melon|apple], 40.6 36.1 ate apple
Sooffuhaj [wildgrass], 29.4 6.0 ate wildgrass
Axag [wildgrass], 6.0 13.0 ate wildgrass
Eiwaw [wildgrass], 15.0 19.1 ate wildgrass
Eimov [lily], 7.6 26.3 ate lily
Egi [apple], 41.3 58.6 ate apple
Blue Moum [melon|apple], 40.6 36.1 ate melon
[[0|250], [10|246], [20|244], [30|244], [40|244], [50|245], [60|247], [70|256], [80|255],
[90|257], [100|257], [110|257], [120|250], [130|249], [140|249], [150|245], [160|242],
[170|234], [180|229], [190|226], [200|225], [210|224], [220|220], [230|215], [240|206],
[250|190], [260|184], [270|178], [280|175], [290|169], [300|162], [310|153], [320|150],
[330|145], [340|144], [350|141], [360|133], [370|131], [380|129], [390|128], [400|126],
```

```

[[410|126], [[420|124], [[430|125], [[440|121], [[450|122], [[460|119], [[470|118], [[480|115],
[[490|114], [[500|114], [[510|105], [[520|106], [[530|104], [[540|101], [[550|96], [[560|93],
[[570|90], [[580|91], [[590|88], [[600|87], [[610|87], [[620|86], [[630|87], [[640|83],
[[650|85], [[660|85], [[670|84], [[680|84], [[690|82], [[700|76], [[710|75], [[720|74],
[[730|74], [[740|69], [[750|67], [[760|66], [[770|63], [[780|63], [[790|63], [[800|62],
[[810|59], [[820|59], [[830|58], [[840|58], [[850|58], [[860|58], [[870|58], [[880|57],
[[890|57], [[900|56], [[910|55], [[920|55], [[930|54], [[940|54], [[950|54], [[960|54],
[[970|53], [[980|53], [[990|53], [[1000|52], [[1010|52], [[1020|52], [[1030|52], [[1040|51],
[[1050|51], [[1060|51], [[1070|50], [[1080|50], [[1090|50], [[1100|50], [[1110|49], [[1120|49],
[[1130|48], [[1140|47], [[1150|46], [[1160|46], [[1170|45], [[1180|45], [[1190|45], [[1200|44],
[[1210|44], [[1220|44], [[1230|43], [[1240|43], [[1250|43], [[1260|43], [[1270|43], [[1280|43],
[[1290|43], [[1300|43], [[1310|43], [[1320|43], [[1330|42], [[1340|42], [[1350|42], [[1360|42],
[[1370|42], [[1380|42], [[1390|42], [[1400|42], [[1410|41], [[1420|41], [[1430|41], [[1440|41],
[[1450|41], [[1460|41], [[1470|40], [[1480|40], [[1490|40], [[1500|40], [[1510|40], [[1520|40],
[[1530|40], [[1540|40], [[1550|40], [[1560|39], [[1570|38], [[1580|38], [[1590|38], [[1600|38],
[[1610|38], [[1620|38], [[1630|38], [[1640|38], [[1650|38], [[1660|38], [[1670|37], [[1680|37],
[[1690|36], [[1700|36], [[1710|36], [[1720|36], [[1730|36], [[1740|36], [[1750|36], [[1760|36],
[[1770|36], [[1780|36], [[1790|36], [[1800|36], ],
////////////////////////////////////
////////////////////////////////////
//////////
[228|12|0|102|0],
////////////////////////////////////
////////////////////////////////////
//////////
[[0|[[wildgrass],      [2.9|0.5|0.3],      [22.8|14.7|17.2|17.8|24.9|0.51|65.7|120.5|27.5],      ],
[[10|[[wildgrass],     [3.0|0.51|0.3],      [23.2|14.7|17.3|18.4|25.2|0.53|66.6|119.0|28.1],      ],
[[20|[[wildgrass],     [3.0|0.51|0.3],      [23.1|14.7|17.3|18.4|25.2|0.53|66.8|119.1|28.1],      ],
[[30|[[wildgrass],     [3.0|0.51|0.3],      [23.1|14.7|17.3|18.4|25.2|0.53|66.8|119.1|28.1],      ],
[[40|[[wildgrass],     [3.0|0.51|0.3],      [23.1|14.7|17.3|18.4|25.2|0.53|66.8|119.1|28.1],      ],
[[50|[[wildgrass],     [3.0|0.51|0.31],      [23.2|14.8|17.3|18.5|25.3|0.53|66.8|119.1|28.1],      ],
[[60|[[wildgrass],     [3.0|0.51|0.31],      [23.3|14.9|17.5|18.6|25.5|0.54|67.4|119.1|28.2],      ],
[[70|[[wildgrass],     [3.1|0.52|0.31],      [23.7|15.1|17.9|19.3|26.1|0.55|68.2|118.5|29.4],      ],
[[80|[[wildgrass],     [3.1|0.53|0.32],      [24.0|15.4|18.0|19.6|26.5|0.56|68.7|118.1|30.0],      ],
[[90|[[wildgrass],     [3.2|0.53|0.32],      [24.2|15.5|18.1|19.7|26.8|0.56|68.8|117.9|30.2],      ],
[[100|[[wildgrass],    [3.2|0.54|0.32],      [24.4|15.6|18.3|19.8|27.0|0.57|69.3|118.0|30.3],      ],
[[110|[[wildgrass],    [3.2|0.55|0.33],      [24.7|15.7|18.5|20.1|27.4|0.58|69.8|118.0|30.7],      ],
[[120|[[wildgrass],    [3.3|0.55|0.33],      [24.9|15.9|18.6|20.3|27.6|0.58|70.2|118.5|31.1],      ],
[[130|[[wildgrass],    [3.3|0.56|0.33],      [25.0|16.0|18.8|20.3|27.9|0.59|70.4|118.3|31.1],      ],
[[140|[[wildgrass],    [3.3|0.56|0.33],      [25.2|16.1|18.8|20.4|28.0|0.59|70.8|117.8|31.2],      ],
[[150|[[wildgrass],    [3.3|0.56|0.33],      [25.2|16.1|18.8|20.5|28.1|0.6|71.0|117.2|31.2],      ],
[[160|[[wildgrass],    [3.3|0.56|0.33],      [25.5|16.3|18.9|20.8|28.4|0.6|71.4|117.1|31.6],      ],
[[170|[[wildgrass],    [3.4|0.58|0.34],      [26.1|16.7|19.4|21.2|29.1|0.62|72.3|116.1|32.3],      ],
[[180|[[wildgrass],    [3.4|0.59|0.35],      [26.2|16.9|19.5|21.4|29.4|0.63|72.5|116.5|32.6],      ],
[[190|[[wildgrass],    [3.4|0.6|0.35],       [26.8|17.2|19.9|22.0|30.0|0.65|72.8|116.2|32.9],      ],
[[200|[[wildgrass],    [3.5|0.61|0.36],      [27.3|17.4|20.2|22.4|30.8|0.66|73.2|115.9|33.3],      ],
[[210|[[wildgrass],    [3.6|0.63|0.37],      [28.1|17.9|20.7|23.2|31.6|0.68|73.9|116.0|33.6],      ],
[[220|[[wildgrass],    [3.6|0.64|0.38],      [28.6|18.2|20.8|23.8|32.2|0.7|74.4|115.6|34.5],      ],
[[230|[[wildgrass],    [3.7|0.65|0.38],      [29.0|18.4|21.0|24.0|32.8|0.71|74.8|114.0|35.0],      ],
[[240|[[wildgrass],    [3.8|0.68|0.39],      [29.9|18.9|21.6|24.8|33.8|0.74|75.4|113.9|36.2],      ],
[[250|[[wildgrass],    [3.8|0.68|0.4],       [30.3|19.3|22.0|25.3|34.2|0.75|76.3|115.2|36.5],      ],

```

[[260 [[wildgrass],	[[3.8 0.69 0.4],	[30.5 19.4 22.1 25.5 34.4 0.75 76.3 114.1 36.7],	,
[[270 [[wildgrass],	[[3.9 0.7 0.41],	[31.1 19.9 22.5 25.8 35.1 0.76 77.2 113.8 36.8],	,
[[280 [[wildgrass],	[[3.9 0.71 0.41],	[31.4 20.2 22.8 26.3 35.4 0.77 77.2 113.6 37.2],	,
[[290 [[wildgrass],	[[3.9 0.71 0.41],	[31.5 20.4 22.9 26.2 35.5 0.77 77.5 113.1 37.3],	,
[[300 [[wildgrass],	[[4.0 0.72 0.42],	[31.8 20.8 23.1 26.7 36.1 0.79 77.6 113.3 37.7],	,
[[310 [[wildgrass],	[[4.1 0.74 0.43],	[32.4 21.7 23.5 26.4 36.7 0.79 77.8 114.4 37.9],	,
[[320 [[wildgrass],	[[4.1 0.74 0.43],	[32.5 21.7 23.5 26.5 36.8 0.8 78.3 114.4 38.1],	,
[[330 [[wildgrass],	[[4.3 0.75 0.43],	[32.7 22.2 24.0 26.2 37.3 0.81 78.0 113.3 38.8],	,
[[340 [[wildgrass],	[[4.3 0.76 0.44],	[33.5 23.2 24.6 26.2 38.2 0.82 78.0 112.2 39.7],	,
[[350 [[wildgrass],	[[4.3 0.76 0.44],	[33.4 23.3 24.5 26.2 38.1 0.83 77.5 112.4 40.1],	,
[[360 [[wildgrass],	[[4.2 0.75 0.43],	[32.9 23.3 24.4 25.5 37.6 0.81 76.7 113.2 39.7],	,
[[370 [[wildgrass],	[[4.2 0.76 0.44],	[33.3 23.6 24.7 25.6 38.0 0.82 76.8 112.7 40.2],	,
[[380 [[wildgrass],	[[4.2 0.77 0.44],	[33.6 24.1 24.8 25.6 38.4 0.81 76.8 112.9 40.3],	,
[[390 [[wildgrass],	[[4.3 0.77 0.44],	[33.7 24.1 24.9 25.8 38.5 0.82 76.7 113.1 40.3],	,
[[400 [[wildgrass],	[[4.3 0.8 0.45],	[34.5 25.3 25.5 26.2 39.9 0.84 77.2 112.8 41.2],	,
[[410 [[wildgrass],	[[4.4 0.81 0.46],	[35.0 25.6 26.0 26.6 40.3 0.84 77.2 112.8 41.9],	,
[[420 [[wildgrass],	[[4.3 0.8 0.46],	[34.8 25.4 25.7 26.2 40.1 0.84 76.9 112.8 41.6],	,
[[430 [[wildgrass],	[[4.3 0.82 0.46],	[35.3 25.9 26.4 26.4 40.7 0.84 77.1 112.7 41.9],	,
[[440 [[wildgrass],	[[4.1 0.8 0.45],	[34.6 25.2 25.6 25.8 39.5 0.82 76.5 113.1 41.4],	,
[[450 [[wildgrass],	[[4.2 0.81 0.46],	[35.4 25.9 26.5 26.3 40.5 0.82 76.9 113.0 42.3],	,
[[460 [[wildgrass],	[[4.1 0.81 0.46],	[35.2 25.8 26.3 26.1 40.1 0.81 76.7 113.8 42.4],	,
[[470 [[wildgrass],	[[4.1 0.82 0.46],	[35.7 26.3 26.8 26.4 40.9 0.81 76.9 113.2 42.5],	,
[[480 [[wildgrass],	[[4.1 0.83 0.47],	[36.0 26.6 27.1 26.5 41.1 0.82 77.0 113.4 42.6],	,
[[490 [[wildgrass],	[[4.0 0.83 0.47],	[36.2 27.1 27.3 26.2 41.5 0.8 77.1 113.0 43.2],	,
[[500 [[wildgrass],	[[4.0 0.84 0.48],	[36.6 27.5 27.8 26.6 42.1 0.8 77.5 112.8 43.7],	,
[[510 [[wildgrass],	[[4.0 0.82 0.47],	[35.8 27.0 27.3 26.6 41.4 0.77 77.0 112.9 43.8],	,
[[520 [[wildgrass],	[[4.0 0.84 0.48],	[36.3 27.4 28.0 27.2 42.1 0.77 77.2 112.7 44.5],	,
[[530 [[wildgrass],	[[4.0 0.84 0.48],	[36.0 27.4 27.6 27.2 41.9 0.76 76.7 113.0 45.1],	,
[[540 [[wildgrass],	[[4.0 0.82 0.47],	[35.4 26.6 27.1 26.9 41.0 0.76 76.2 112.4 44.2],	,
[[550 [[wildgrass],	[[4.0 0.8 0.45],	[34.4 25.6 26.0 26.5 39.6 0.76 75.6 113.4 42.4],	,
[[560 [[wildgrass],	[[4.0 0.8 0.46],	[34.9 26.2 26.5 26.9 40.2 0.74 75.3 113.9 42.9],	,
[[570 [[wildgrass],	[[3.9 0.78 0.45],	[34.1 25.7 25.6 26.3 39.4 0.72 74.5 114.3 41.9],	,
[[580 [[wildgrass],	[[4.0 0.81 0.46],	[35.2 26.4 26.9 27.4 40.6 0.71 74.8 114.2 43.4],	,
[[590 [[apple],	[[3.8 0.79 0.45],	[34.4 26.0 26.3 26.9 39.8 0.68 74.7 114.6 42.8],	,
[[600 [[apple],	[[3.8 0.8 0.46],	[34.6 26.1 26.6 27.5 40.0 0.66 75.0 115.3 43.5],	,
[[610 [[apple],	[[3.9 0.81 0.46],	[35.2 26.5 27.2 28.0 40.7 0.66 75.0 115.3 44.0],	,
[[620 [[wildgrass],	[[3.9 0.83 0.47],	[35.8 26.8 27.8 28.9 41.2 0.65 74.9 115.3 44.3],	,
[[630 [[wildgrass],	[[4.0 0.86 0.49],	[36.6 27.4 28.9 30.0 41.9 0.66 75.2 115.1 44.9],	,
[[640 [[apple],	[[4.2 0.89 0.5],	[37.5 27.7 29.9 31.4 42.5 0.66 75.2 115.7 45.4],	,
[[650 [[apple],	[[4.3 0.94 0.52],	[38.8 29.0 31.5 32.7 43.8 0.67 75.8 115.3 46.6],	,
[[660 [[wildgrass],	[[4.5 0.99 0.55],	[40.0 30.2 33.0 34.5 44.8 0.68 76.3 114.2 47.7],	,
[[670 [[wildgrass],	[[4.5 1.0 0.55],	[40.2 30.4 33.4 34.6 44.9 0.68 76.0 114.4 48.0],	,
[[680 [[wildgrass],	[[4.6 1.0 0.56],	[40.7 31.0 34.0 35.2 45.3 0.68 76.0 114.4 48.2],	,
[[690 [[wildgrass],	[[4.4 0.99 0.54],	[39.2 30.1 32.5 34.0 44.0 0.67 75.4 114.7 46.9],	,
[[700 [[wildgrass],	[[4.3 0.93 0.52],	[37.2 28.3 29.9 32.0 41.4 0.67 74.0 115.7 44.3],	,
[[710 [[wildgrass],	[[4.2 0.91 0.5],	[36.3 27.7 29.0 31.2 40.7 0.66 73.7 115.9 43.5],	,
[[720 [[wildgrass],	[[4.2 0.91 0.51],	[36.5 27.9 29.3 31.4 40.9 0.66 73.5 115.3 43.6],	,
[[730 [[wildgrass],	[[4.2 0.91 0.51],	[36.5 27.9 29.3 31.4 40.9 0.66 73.5 115.3 43.6],	,
[[740 [[wildgrass],	[[3.9 0.85 0.48],	[34.3 26.2 26.9 29.9 38.3 0.66 72.3 115.0 41.9],	,
[[750 [[wildgrass],	[[3.8 0.81 0.46],	[32.7 25.0 25.0 28.6 36.5 0.65 71.5 115.4 40.1],	,
[[760 [[wildgrass],	[[3.8 0.8 0.45],	[32.4 24.9 24.8 28.3 36.0 0.64 71.1 115.7 40.0],	,

[770 [[wildgrass],	[3.5 0.72 0.41],	29.7 22.6 21.6 25.7 33.0 0.63 69.7 116.4 37.1],	,
[780 [[wildgrass],	[3.5 0.72 0.41],	29.7 22.6 21.6 25.7 33.0 0.63 69.7 116.4 37.1],	,
[790 [[wildgrass],	[3.5 0.72 0.41],	29.7 22.6 21.6 25.7 33.0 0.63 69.7 116.4 37.1],	,
[800 [[wildgrass],	[3.3 0.67 0.39],	28.5 21.4 20.3 24.5 31.9 0.61 69.2 116.7 36.1],	,
[810 [[wildgrass],	[3.0 0.58 0.35],	25.9 19.2 17.8 21.8 29.5 0.59 67.7 117.5 33.9],	,
[820 [[wildgrass],	[3.0 0.58 0.35],	25.9 19.2 17.8 21.8 29.5 0.59 67.7 117.5 33.9],	,
[830 [[wildgrass],	[3.0 0.58 0.35],	26.0 19.2 17.8 22.1 29.5 0.6 68.0 118.5 34.1],	,
[840 [[wildgrass],	[3.0 0.58 0.35],	26.0 19.2 17.8 22.1 29.5 0.6 68.0 118.5 34.1],	,
[850 [[wildgrass],	[3.0 0.58 0.35],	26.0 19.2 17.8 22.1 29.5 0.6 68.0 118.5 34.1],	,
[860 [[wildgrass],	[3.0 0.58 0.35],	26.0 19.2 17.8 22.1 29.5 0.6 68.0 118.5 34.1],	,
[870 [[wildgrass],	[3.0 0.58 0.35],	26.0 19.2 17.8 22.1 29.5 0.6 68.0 118.5 34.1],	,
[880 [[apple],	[3.1 0.59 0.36],	26.0 19.1 18.0 22.1 29.6 0.6 67.5 117.4 34.4],	,
[890 [[apple],	[3.1 0.59 0.36],	26.0 19.1 18.0 22.1 29.6 0.6 67.5 117.4 34.4],	,
[900 [[apple],	[3.0 0.59 0.36],	25.9 19.1 17.9 22.2 29.7 0.61 67.6 117.3 34.4],	,
[910 [[apple],	[3.0 0.59 0.36],	25.9 19.0 17.8 22.2 29.8 0.61 68.1 118.2 34.8],	,
[920 [[apple],	[3.0 0.59 0.36],	25.9 19.0 17.8 22.2 29.8 0.61 68.1 118.2 34.8],	,
[930 [[apple],	[3.1 0.59 0.36],	26.0 19.2 17.6 22.3 29.9 0.61 68.8 116.8 35.2],	,
[940 [[apple],	[3.1 0.59 0.36],	26.0 19.2 17.6 22.3 29.9 0.61 68.8 116.8 35.2],	,
[950 [[apple],	[3.1 0.59 0.36],	26.0 19.2 17.6 22.3 29.9 0.61 68.8 116.8 35.2],	,
[960 [[apple],	[3.1 0.59 0.36],	26.0 19.2 17.6 22.3 29.9 0.61 68.8 116.8 35.2],	,
[970 [[apple],	[3.0 0.59 0.36],	26.2 19.3 17.6 22.4 29.9 0.6 68.2 116.1 35.5],	,
[980 [[apple],	[3.0 0.59 0.36],	26.2 19.3 17.6 22.4 29.9 0.6 68.2 116.1 35.5],	,
[990 [[apple],	[3.0 0.59 0.36],	26.2 19.3 17.6 22.4 29.9 0.6 68.2 116.1 35.5],	,
[1000 [[apple],	[3.0 0.59 0.36],	26.2 19.3 17.6 22.1 29.7 0.59 68.0 116.4 35.1],	,
[1010 [[apple],	[3.0 0.59 0.36],	26.2 19.3 17.6 22.1 29.7 0.59 68.0 116.4 35.1],	,
[1020 [[apple],	[3.0 0.59 0.36],	26.2 19.3 17.6 22.1 29.7 0.59 68.0 116.4 35.1],	,
[1030 [[apple],	[3.0 0.59 0.36],	26.2 19.3 17.6 22.1 29.7 0.59 68.0 116.4 35.1],	,
[1040 [[apple],	[3.0 0.58 0.35],	25.9 19.1 17.0 21.8 29.2 0.6 67.3 116.8 34.4],	,
[1050 [[apple],	[3.0 0.58 0.35],	25.9 19.1 17.0 21.8 29.2 0.6 67.3 116.8 34.4],	,
[1060 [[apple],	[3.0 0.58 0.35],	25.9 19.1 17.0 21.8 29.2 0.6 67.3 116.8 34.4],	,
[1070 [[apple],	[3.0 0.58 0.35],	25.8 19.0 16.6 21.7 29.0 0.59 67.4 117.1 34.0],	,
[1080 [[apple],	[3.0 0.58 0.35],	25.8 19.0 16.6 21.7 29.0 0.59 67.4 117.1 34.0],	,
[1090 [[apple],	[3.0 0.58 0.35],	25.8 19.0 16.6 21.7 29.0 0.59 67.4 117.1 34.0],	,
[1100 [[apple],	[3.0 0.58 0.35],	25.8 19.0 16.6 21.7 29.0 0.59 67.4 117.1 34.0],	,
[1110 [[apple],	[3.0 0.58 0.35],	26.0 18.9 16.7 21.5 29.2 0.59 67.6 116.3 33.8],	,
[1120 [[apple],	[3.0 0.58 0.35],	26.0 18.9 16.7 21.5 29.2 0.59 67.6 116.3 33.8],	,
[1130 [[apple],	[3.0 0.58 0.35],	26.1 19.0 16.7 21.8 29.2 0.59 67.0 115.5 33.6],	,
[1140 [[apple],	[3.0 0.58 0.36],	26.0 19.0 16.7 21.8 29.2 0.6 66.5 116.5 33.5],	,
[1150 [[apple],	[2.9 0.58 0.36],	25.9 19.0 16.8 21.6 29.0 0.59 66.6 116.9 33.2],	,
[1160 [[apple],	[2.9 0.58 0.36],	25.9 19.0 16.8 21.6 29.0 0.59 66.6 116.9 33.2],	,
[1170 [[apple],	[2.9 0.57 0.35],	25.6 18.8 16.1 21.4 28.7 0.57 66.1 117.3 33.7],	,
[1180 [[apple],	[2.9 0.57 0.35],	25.6 18.8 16.1 21.4 28.7 0.57 66.1 117.3 33.7],	,
[1190 [[apple],	[2.9 0.57 0.35],	25.6 18.8 16.1 21.4 28.7 0.57 66.1 117.3 33.7],	,
[1200 [[apple],	[3.0 0.57 0.35],	25.6 18.9 16.2 21.4 28.7 0.58 65.6 117.3 34.3],	,
[1210 [[apple],	[3.0 0.57 0.35],	25.6 18.9 16.2 21.4 28.7 0.58 65.6 117.3 34.3],	,
[1220 [[apple],	[3.0 0.57 0.35],	25.6 18.9 16.2 21.4 28.7 0.58 65.6 117.3 34.3],	,
[1230 [[apple],	[3.0 0.58 0.35],	25.5 19.0 16.1 21.2 28.8 0.57 66.1 115.9 34.2],	,
[1240 [[apple],	[3.0 0.58 0.35],	25.5 19.0 16.1 21.2 28.8 0.57 66.1 115.9 34.2],	,
[1250 [[apple],	[3.0 0.58 0.35],	25.5 19.0 16.1 21.2 28.8 0.57 66.1 115.9 34.2],	,
[1260 [[apple],	[3.0 0.58 0.35],	25.5 19.0 16.1 21.2 28.8 0.57 66.1 115.9 34.2],	,
[1270 [[apple],	[3.0 0.58 0.35],	25.5 19.0 16.1 21.2 28.8 0.57 66.1 115.9 34.2],	,


```

|[1790|[[apple|, |[3.0|0.58|0.36], |[25.7|19.2|16.5|21.1|29.3|0.58|65.2|116.2|34.1], ],
|[1800|[[apple|, |[3.0|0.58|0.36], |[25.7|19.2|16.5|21.1|29.3|0.58|65.2|116.2|34.1], ], ],
//////////
//////////
//////////
[[0|[[apple|40], |[berry|29], |[melon|37], |[lily|31], |[lavendar|30], |[meat|36], |[wildgrass|47],
], ], |[10|[[apple|40], |[berry|29], |[melon|36], |[lily|31], |[lavendar|27], |[meat|38],
|[wildgrass|46], ], ], |[20|[[apple|40], |[berry|28], |[melon|36], |[lily|30], |[lavendar|27],
|[meat|38], |[wildgrass|46], ], ], |[30|[[apple|40], |[berry|28], |[melon|36], |[lily|30],
|[lavendar|27], |[meat|38], |[wildgrass|46], ], ], |[40|[[apple|40], |[berry|28], |[melon|36],
|[lily|30], |[lavendar|27], |[meat|38], |[wildgrass|46], ], ], |[50|[[apple|40], |[berry|28],
|[melon|37], |[lily|30], |[lavendar|27], |[meat|38], |[wildgrass|46], ], ], |[60|[[apple|41],
|[berry|28], |[melon|37], |[lily|30], |[lavendar|28], |[meat|37], |[wildgrass|47], ], ],
|[70|[[apple|44], |[berry|27], |[melon|40], |[lily|33], |[lavendar|28], |[meat|37],
|[wildgrass|50], ], ], |[80|[[apple|41], |[berry|28], |[melon|39], |[lily|33], |[lavendar|28],
|[meat|37], |[wildgrass|52], ], ], |[90|[[apple|42], |[berry|29], |[melon|39], |[lily|33],
|[lavendar|27], |[meat|37], |[wildgrass|54], ], ], |[100|[[apple|42], |[berry|29], |[melon|38],
|[lily|33], |[lavendar|26], |[meat|38], |[wildgrass|56], ], ], |[110|[[apple|43], |[berry|29],
|[melon|38], |[lily|32], |[lavendar|25], |[meat|38], |[wildgrass|57], ], ], |[120|[[apple|43],
|[berry|25], |[melon|38], |[lily|32], |[lavendar|23], |[meat|38], |[wildgrass|56], ], ],
|[130|[[apple|42], |[berry|24], |[melon|38], |[lily|33], |[lavendar|23], |[meat|38],
|[wildgrass|56], ], ], |[140|[[apple|43], |[berry|24], |[melon|39], |[lily|32], |[lavendar|23],
|[meat|38], |[wildgrass|55], ], ], |[150|[[apple|42], |[berry|24], |[melon|39], |[lily|32],
|[lavendar|21], |[meat|38], |[wildgrass|54], ], ], |[160|[[apple|43], |[berry|24], |[melon|40],
|[lily|31], |[lavendar|20], |[meat|36], |[wildgrass|54], ], ], |[170|[[apple|44], |[berry|23],
|[melon|39], |[lily|32], |[lavendar|14], |[meat|35], |[wildgrass|55], ], ], |[180|[[apple|43],
|[berry|21], |[melon|39], |[lily|32], |[lavendar|11], |[meat|36], |[wildgrass|55], ], ],
|[190|[[apple|44], |[berry|21], |[melon|39], |[lily|33], |[lavendar|10], |[meat|33],
|[wildgrass|56], ], ], |[200|[[apple|44], |[berry|21], |[melon|39], |[lily|33], |[lavendar|10],
|[meat|33], |[wildgrass|56], ], ], |[210|[[apple|43], |[berry|22], |[melon|40], |[lily|32],
|[lavendar|10], |[meat|31], |[wildgrass|57], ], ], |[220|[[apple|42], |[berry|21], |[melon|40],
|[lily|33], |[lavendar|9], |[meat|29], |[wildgrass|58], ], ], |[230|[[apple|42], |[berry|21],
|[melon|39], |[lily|34], |[lavendar|7], |[meat|26], |[wildgrass|58], ], ], |[240|[[apple|40],
|[berry|21], |[melon|39], |[lily|33], |[lavendar|5], |[meat|19], |[wildgrass|61], ], ],
|[250|[[apple|39], |[berry|20], |[melon|37], |[lily|29], |[lavendar|4], |[meat|16],
|[wildgrass|57], ], ], |[260|[[apple|39], |[berry|19], |[melon|37], |[lily|28], |[lavendar|4],
|[meat|14], |[wildgrass|55], ], ], |[270|[[apple|38], |[berry|16], |[melon|38], |[lily|27],
|[lavendar|3], |[meat|13], |[wildgrass|57], ], ], |[280|[[apple|38], |[berry|16], |[melon|36],
|[lily|27], |[lavendar|2], |[meat|13], |[wildgrass|57], ], ], |[290|[[apple|36], |[berry|16],
|[melon|35], |[lily|25], |[lavendar|2], |[meat|12], |[wildgrass|57], ], ], |[300|[[apple|36],
|[berry|16], |[melon|31], |[lily|24], |[lavendar|1], |[meat|11], |[wildgrass|57], ], ],
|[310|[[apple|38], |[berry|16], |[melon|28], |[lily|21], |[lavendar|0], |[meat|6], |[wildgrass|58],
], ], |[320|[[apple|38], |[berry|16], |[melon|28], |[lily|20], |[lavendar|0], |[meat|4],
|[wildgrass|58], ], ], |[330|[[apple|37], |[berry|15], |[melon|26], |[lily|19], |[lavendar|1],
|[meat|3], |[wildgrass|58], ], ], |[340|[[apple|39], |[berry|14], |[melon|26], |[lily|18],
|[lavendar|1], |[meat|2], |[wildgrass|59], ], ], |[350|[[apple|39], |[berry|14], |[melon|26],
|[lily|17], |[lavendar|1], |[meat|2], |[wildgrass|56], ], ], |[360|[[apple|39], |[berry|14],
|[melon|22], |[lily|17], |[lavendar|1], |[meat|2], |[wildgrass|53], ], ], |[370|[[apple|39],
|[berry|13], |[melon|22], |[lily|17], |[lavendar|1], |[meat|2], |[wildgrass|52], ], ],
|[380|[[apple|39], |[berry|14], |[melon|21], |[lily|18], |[lavendar|1], |[meat|1], |[wildgrass|51],
], ], |[390|[[apple|39], |[berry|14], |[melon|21], |[lily|18], |[lavendar|1], |[meat|1],

```

ETHAN CHENG 5045 | Evolution Simulator

[illegible]

```

|[melon|3], |[lily|4], |[lavendar|0], |[meat|0], |[wildgrass|9], ], ], |[1790|[apple|14], |[berry|7],
|[melon|3], |[lily|4], |[lavendar|0], |[meat|0], |[wildgrass|9], ], ], |[1800|[apple|14], |[berry|7],
|[melon|3], |[lily|4], |[lavendar|0], |[meat|0], |[wildgrass|9], ], ],
////////////////////
////////////////////
////////////////
[[0|N/A], |[10|5.0], |[20|7.5], |[30|7.5], |[40|7.5], |[50|7.5], |[60|17.8], |[70|35.7], |[80|68.6],
|[90|74.1], |[100|83.4], |[110|97.3], |[120|112.6], |[130|118.8], |[140|122.5], |[150|131.6],
|[160|134.7], |[170|145.0], |[180|153.8], |[190|162.2], |[200|175.5], |[210|182.1],
|[220|194.7], |[230|206.7], |[240|219.0], |[250|224.0], |[260|225.7], |[270|234.9],
|[280|234.2], |[290|235.1], |[300|239.7], |[310|249.2], |[320|253.7], |[330|256.2],
|[340|252.6], |[350|254.3], |[360|244.6], |[370|240.1], |[380|242.8], |[390|244.3],
|[400|265.4], |[410|268.9], |[420|267.1], |[430|268.0], |[440|260.9], |[450|250.1],
|[460|259.7], |[470|263.1], |[480|299.6], |[490|292.3], |[500|307.6], |[510|304.9],
|[520|305.7], |[530|291.8], |[540|248.0], |[550|247.5], |[560|266.3], |[570|267.4],
|[580|279.0], |[590|278.1], |[600|242.5], |[610|255.5], |[620|277.5], |[630|277.7],
|[640|316.0], |[650|316.3], |[660|324.2], |[670|326.4], |[680|307.8], |[690|294.1],
|[700|285.9], |[710|270.8], |[720|258.8], |[730|244.4], |[740|278.7], |[750|265.8],
|[760|264.6], |[770|249.7], |[780|249.7], |[790|254.5], |[800|251.1], |[810|251.6],
|[820|235.7], |[830|259.9], |[840|266.7], |[850|248.8], |[860|251.6], |[870|240.1],
|[880|273.4], |[890|254.9], |[900|288.6], |[910|300.4], |[920|280.1], |[930|322.9],
|[940|336.4], |[950|348.0], |[960|372.0], |[970|430.7], |[980|453.1], |[990|453.1],
|[1000|423.2], |[1010|444.2], |[1020|444.2], |[1030|444.2], |[1040|475.2], |[1050|475.2],
|[1060|475.2], |[1070|511.4], |[1080|511.4], |[1090|511.2], |[1100|507.5], |[1110|523.5],
|[1120|523.5], |[1130|618.7], |[1140|661.9], |[1150|741.5], |[1160|741.5], |[1170|758.0],
|[1180|811.6], |[1190|811.6], |[1200|841.2], |[1210|841.2], |[1220|831.0], |[1230|861.1],
|[1240|861.1], |[1250|919.5], |[1260|919.5], |[1270|919.5], |[1280|919.5], |[1290|918.7],
|[1300|918.7], |[1310|918.7], |[1320|918.7], |[1330|952.7], |[1340|962.3], |[1350|962.3],
|[1360|962.3], |[1370|962.3], |[1380|962.3], |[1390|962.3], |[1400|962.3], |[1410|999.2],
|[1420|999.2], |[1430|1.0k], |[1440|1.0k], |[1450|1.0k], |[1460|1.0k], |[1470|1.03k],
|[1480|1.03k], |[1490|1.12k], |[1500|1.12k], |[1510|1.12k], |[1520|1.12k], |[1530|1.12k],
|[1540|1.12k], |[1550|1.12k], |[1560|1.15k], |[1570|1.19k], |[1580|1.19k], |[1590|1.2k],
|[1600|1.2k], |[1610|1.2k], |[1620|1.2k], |[1630|1.2k], |[1640|1.2k], |[1650|1.2k], |[1660|1.2k],
|[1670|1.24k], |[1680|1.24k], |[1690|1.28k], |[1700|1.28k], |[1710|1.28k], |[1720|1.28k],
|[1730|1.3k], |[1740|1.3k], |[1750|1.3k], |[1760|1.3k], |[1770|1.3k], |[1780|1.3k], |[1790|1.3k],
|[1800|1.3k], ],
////////////////////
////////////////////
////////////////
[[0|N/A], |[10|[3.0|5.0|7.2], ], |[20|[4.5|6.5|9.5], ], |[30|[4.5|6.5|9.5], ], |[40|[4.5|6.5|9.5], ],
|[50|[4.5|6.5|9.5], ], |[60|[5.8|8.5|26.2], ], |[70|[8.8|34.5|63.0], ], |[80|[61.0|70.5|77.0], ],
|[90|[66.8|76.0|79.5], ], |[100|[75.5|80.0|94.0], ], |[110|[91.2|100.0|105.2], ],
|[120|[108.2|114.0|118.0], ], |[130|[115.0|118.0|122.5], ], |[140|[118.0|120.5|128.5], ],
|[150|[120.5|130.0|147.0], ], |[160|[129.2|149.0|151.2], ], |[170|[150.5|161.0|165.5], ],
|[180|[152.0|167.0|174.0], ], |[190|[159.5|174.0|187.0], ], |[200|[169.5|187.0|190.0], ],
|[210|[172.2|190.5|202.0], ], |[220|[187.0|202.0|213.0], ], |[230|[194.0|213.0|224.0], ],
|[240|[207.0|226.0|236.0], ], |[250|[202.0|236.5|243.2], ], |[260|[192.0|240.0|249.8], ],
|[270|[211.2|248.0|262.0], ], |[280|[204.0|249.0|267.0], ], |[290|[191.2|252.5|273.5], ],
|[300|[186.0|266.0|290.0], ], |[310|[194.0|272.0|299.2], ], |[320|[212.0|284.0|305.0], ],
|[330|[212.5|294.0|308.8], ], |[340|[184.0|296.0|309.0], ], |[350|[212.0|292.0|309.0], ],
|[360|[172.0|255.5|309.0], ], |[370|[164.0|247.0|309.0], ], |[380|[166.0|245.5|309.0], ],

```



```

|[390|[157.0|244.0|330.0], ], |[400|[160.8|277.5|371.0], ], |[410|[154.8|282.0|374.8], ],
|[420|[152.5|255.0|385.0], ], |[430|[152.5|255.0|385.0], ], |[440|[153.0|244.0|380.0], ],
|[450|[150.5|239.0|361.5], ], |[460|[146.8|241.5|383.5], ], |[470|[151.8|241.5|366.5], ],
|[480|[180.5|323.0|401.0], ], |[490|[165.5|282.5|409.0], ], |[500|[173.0|337.0|417.0], ],
|[510|[173.0|267.0|476.0], ], |[520|[164.0|267.0|476.0], ], |[530|[164.0|263.0|417.0], ],
|[540|[159.0|191.5|316.0], ], |[550|[153.0|205.0|309.0], ], |[560|[157.0|208.0|315.5], ],
|[570|[153.0|208.0|323.0], ], |[580|[159.0|251.0|378.8], ], |[590|[161.0|239.0|376.0], ],
|[600|[153.0|205.0|309.0], ], |[610|[143.0|208.0|322.0], ], |[620|[161.5|233.0|322.8], ],
|[630|[161.5|233.0|322.8], ], |[640|[159.5|253.0|553.0], ], |[650|[159.5|247.0|554.5], ],
|[660|[162.5|227.0|605.5], ], |[670|[164.0|227.0|595.0], ], |[680|[164.0|208.0|525.0], ],
|[690|[146.5|195.0|542.5], ], |[700|[121.0|179.0|525.0], ], |[710|[119.0|173.0|417.5], ],
|[720|[118.5|169.0|309.8], ], |[730|[117.2|163.0|232.5], ], |[740|[118.5|169.5|473.2], ],
|[750|[117.2|163.0|363.8], ], |[760|[116.5|165.0|418.5], ], |[770|[115.8|152.5|287.2], ],
|[780|[115.8|152.5|287.2], ], |[790|[115.0|161.0|309.0], ], |[800|[115.8|160.5|287.2], ],
|[810|[117.2|160.5|243.8], ], |[820|[116.5|160.0|221.5], ], |[830|[116.5|161.0|265.5], ],
|[840|[115.0|160.0|309.0], ], |[850|[114.8|152.0|243.8], ], |[860|[114.5|144.0|265.5], ],
|[870|[114.2|142.0|221.8], ], |[880|[115.8|152.0|287.2], ], |[890|[115.0|144.0|222.0], ],
|[900|[114.8|142.0|369.8], ], |[910|[114.5|144.0|430.5], ], |[920|[114.0|140.0|309.0], ],
|[930|[115.0|144.0|552.0], ], |[940|[117.2|152.0|633.5], ], |[950|[116.5|160.0|715.0], ],
|[960|[118.0|160.0|878.0], ], |[970|[123.0|172.0|891.0], ], |[980|[130.5|240.5|893.8], ],
|[990|[130.5|240.5|893.8], ], |[1000|[128.0|172.0|913.5], ], |[1010|[125.5|234.5|919.2], ],
|[1020|[125.5|234.5|919.2], ], |[1030|[125.5|234.5|919.2], ], |[1040|[128.0|309.0|913.5], ],
|[1050|[128.0|309.0|913.5], ], |[1060|[128.0|309.0|913.5], ], |[1070|[130.5|430.5|925.8], ],
|[1080|[130.5|430.5|925.8], ], |[1090|[135.8|430.5|927.2], ], |[1100|[133.0|309.0|928.0], ],
|[1110|[133.0|309.0|963.0], ], |[1120|[133.0|309.0|963.0], ], |[1130|[144.0|847.0|982.0], ],
|[1140|[148.0|887.5|1.07k], ], |[1150|[197.2|945.5|1.1k], ], |[1160|[197.2|945.5|1.1k], ],
|[1170|[234.5|956.0|1.09k], ], |[1180|[443.5|959.5|1.1k], ], |[1190|[443.5|959.5|1.1k], ],
|[1200|[578.0|963.0|1.11k], ], |[1210|[578.0|963.0|1.11k], ], |[1220|[443.5|969.0|1.12k], ],
|[1230|[578.0|982.0|1.13k], ], |[1240|[578.0|982.0|1.13k], ], |[1250|[867.2|1.04k|1.13k], ],
|[1260|[867.2|1.04k|1.13k], ], |[1270|[867.2|1.04k|1.13k], ], |[1280|[867.2|1.04k|1.13k], ],
|[1290|[847.0|1.09k|1.14k], ], |[1300|[847.0|1.09k|1.14k], ], |[1310|[847.0|1.09k|1.14k], ],
|[1320|[847.0|1.09k|1.14k], ], |[1330|[874.2|1.09k|1.18k], ], |[1340|[956.0|1.1k|1.2k], ],
|[1350|[956.0|1.1k|1.2k], ], |[1360|[956.0|1.1k|1.2k], ], |[1370|[956.0|1.1k|1.2k], ],
|[1380|[956.0|1.1k|1.2k], ], |[1390|[956.0|1.1k|1.2k], ], |[1400|[956.0|1.1k|1.2k], ],
|[1410|[962.5|1.11k|1.22k], ], |[1420|[962.5|1.11k|1.22k], ], |[1430|[956.0|1.12k|1.22k], ],
|[1440|[956.0|1.12k|1.22k], ], |[1450|[956.0|1.12k|1.22k], ], |[1460|[956.0|1.12k|1.22k], ],
|[1470|[989.0|1.13k|1.3k], ], |[1480|[989.0|1.13k|1.3k], ], |[1490|[1.09k|1.14k|1.33k], ],
|[1500|[1.09k|1.14k|1.33k], ], |[1510|[1.09k|1.14k|1.33k], ], |[1520|[1.09k|1.14k|1.33k], ],
|[1530|[1.09k|1.14k|1.33k], ], |[1540|[1.09k|1.14k|1.33k], ], |[1550|[1.09k|1.14k|1.33k], ],
|[1560|[1.09k|1.17k|1.39k], ], |[1570|[1.1k|1.21k|1.41k], ], |[1580|[1.1k|1.21k|1.41k], ],
|[1590|[1.09k|1.22k|1.41k], ], |[1600|[1.09k|1.22k|1.41k], ], |[1610|[1.09k|1.22k|1.41k], ],
|[1620|[1.09k|1.22k|1.41k], ], |[1630|[1.09k|1.22k|1.41k], ], |[1640|[1.09k|1.22k|1.41k], ],
|[1650|[1.09k|1.22k|1.41k], ], |[1660|[1.09k|1.22k|1.41k], ], |[1670|[1.09k|1.33k|1.55k], ],
|[1680|[1.09k|1.33k|1.55k], ], |[1690|[1.12k|1.37k|1.56k], ], |[1700|[1.12k|1.37k|1.56k], ],
|[1710|[1.12k|1.37k|1.56k], ], |[1720|[1.12k|1.37k|1.56k], ], |[1730|[1.2k|1.41k|1.57k], ],
|[1740|[1.2k|1.41k|1.57k], ], |[1750|[1.2k|1.41k|1.57k], ], |[1760|[1.2k|1.41k|1.57k], ],
|[1770|[1.2k|1.41k|1.57k], ], |[1780|[1.2k|1.41k|1.57k], ], |[1790|[1.2k|1.41k|1.57k], ],
|[1800|[1.2k|1.41k|1.57k], ], ],
////////////////////
////////////////////
////////////////////

```



```

[[0|[0.0|0.0|0.0], ], |[10|[10.0|10.0|10.0], ], |[20|[20.0|20.0|20.0], ], |[30|[30.0|30.0|30.0], ],
|[40|[40.0|40.0|40.0], ], |[50|[50.0|50.0|50.0], ], |[60|[60.0|60.0|60.0], ],
|[70|[70.0|70.0|70.0], ], |[80|[80.0|80.0|80.0], ], |[90|[90.0|90.0|90.0], ],
|[100|[100.0|100.0|100.0], ], |[110|[110.0|110.0|110.0], ], |[120|[120.0|120.0|120.0], ],
|[130|[130.0|130.0|130.0], ], |[140|[140.0|140.0|140.0], ], |[150|[150.0|150.0|150.0], ],
|[160|[160.0|160.0|160.0], ], |[170|[170.0|170.0|170.0], ], |[180|[180.0|180.0|180.0], ],
|[190|[190.0|190.0|190.0], ], |[200|[197.0|200.0|200.0], ], |[210|[151.5|210.0|210.0], ],
|[220|[159.2|220.0|220.0], ], |[230|[167.0|230.0|230.0], ], |[240|[163.8|240.0|240.0], ],
|[250|[162.8|250.0|250.0], ], |[260|[172.2|260.0|260.0], ], |[270|[169.0|270.0|270.0], ],
|[280|[179.0|280.0|280.0], ], |[290|[186.5|290.0|290.0], ], |[300|[193.8|300.0|300.0], ],
|[310|[182.0|310.0|310.0], ], |[320|[180.2|320.0|320.0], ], |[330|[176.5|330.0|330.0], ],
|[340|[181.8|340.0|340.0], ], |[350|[189.0|350.0|350.0], ], |[360|[202.5|360.0|360.0], ],
|[370|[205.0|370.0|370.0], ], |[380|[196.5|380.0|380.0], ], |[390|[205.8|390.0|390.0], ],
|[400|[207.5|400.0|400.0], ], |[410|[214.8|410.0|410.0], ], |[420|[226.5|420.0|420.0], ],
|[430|[233.5|430.0|430.0], ], |[440|[250.0|440.0|440.0], ], |[450|[249.2|450.0|450.0], ],
|[460|[261.0|460.0|460.0], ], |[470|[263.2|470.0|470.0], ], |[480|[271.0|480.0|480.0], ],
|[490|[276.8|490.0|490.0], ], |[500|[273.0|500.0|500.0], ], |[510|[307.5|510.0|510.0], ],
|[520|[313.2|520.0|520.0], ], |[530|[332.8|530.0|530.0], ], |[540|[350.0|540.0|540.0], ],
|[550|[391.8|550.0|550.0], ], |[560|[402.5|560.0|560.0], ], |[570|[430.2|570.0|570.0], ],
|[580|[429.0|580.0|580.0], ], |[590|[462.8|590.0|590.0], ], |[600|[464.0|600.0|600.0], ],
|[610|[459.0|610.0|610.0], ], |[620|[467.8|620.0|620.0], ], |[630|[474.0|630.0|630.0], ],
|[640|[484.0|640.0|640.0], ], |[650|[476.5|650.0|650.0], ], |[660|[466.5|660.0|660.0], ],
|[670|[473.8|670.0|670.0], ], |[680|[483.8|680.0|680.0], ], |[690|[523.8|690.0|690.0], ],
|[700|[561.5|700.0|700.0], ], |[710|[609.0|710.0|710.0], ], |[720|[606.5|720.0|720.0], ],
|[730|[616.5|730.0|730.0], ], |[740|[675.0|740.0|740.0], ], |[750|[690.0|750.0|750.0], ],
|[760|[707.5|760.0|760.0], ], |[770|[769.0|770.0|770.0], ], |[780|[779.0|780.0|780.0], ],
|[790|[789.0|790.0|790.0], ], |[800|[799.8|800.0|800.0], ], |[810|[810.0|810.0|810.0], ],
|[820|[820.0|820.0|820.0], ], |[830|[830.0|830.0|830.0], ], |[840|[840.0|840.0|840.0], ],
|[850|[850.0|850.0|850.0], ], |[860|[860.0|860.0|860.0], ], |[870|[870.0|870.0|870.0], ],
|[880|[880.0|880.0|880.0], ], |[890|[890.0|890.0|890.0], ], |[900|[900.0|900.0|900.0], ],
|[910|[910.0|910.0|910.0], ], |[920|[920.0|920.0|920.0], ], |[930|[930.0|930.0|930.0], ],
|[940|[940.0|940.0|940.0], ], |[950|[950.0|950.0|950.0], ], |[960|[960.0|960.0|960.0], ],
|[970|[970.0|970.0|970.0], ], 8k|1.48k], ], |[1490|[1.49k|1.49k|1.49k], ],
|[1500|[1.5k|1.5k|1.5k], ], |[1510|[1.51k|1.51k|1.51k], ], |[1520|[1.52k|1.52k|1.52k], ],
|[1530|[1.53k|1.53k|1.53k], ], |[1540|[1.54k|1.54k|1.54k], ], |[1550|[1.55k|1.55k|1.55k], ],
|[1560|[1.56k|1.56k|1.56k], ], |[1570|[1.57k|1.57k|1.57k], ], |[1580|[1.58k|1.58k|1.58k], ],
|[1590|[1.59k|1.59k|1.59k], ], |[1600|[1.6k|1.6k|1.6k], ], |[1610|[1.61k|1.61k|1.61k], ],
|[1620|[1.62k|1.62k|1.62k], ], |[1630|[1.63k|1.63k|1.63k], ], |[1640|[1.64k|1.64k|1.64k], ],
|[1650|[1.65k|1.65k|1.65k], ], |[1660|[1.66k|1.66k|1.66k], ], |[1670|[1.67k|1.67k|1.67k], ],
|[1680|[1.68k|1.68k|1.68k], ], |[1690|[1.69k|1.69k|1.69k], ], |[1700|[1.7k|1.7k|1.7k], ],
|[1710|[1.71k|1.71k|1.71k], ], |[1720|[1.72k|1.72k|1.72k], ], |[1730|[1.73k|1.73k|1.73k], ],
|[1740|[1.74k|1.74k|1.74k], ], |[1750|[1.75k|1.75k|1.75k], ], |[1760|[1.76k|1.76k|1.76k], ],
|[1770|[1.77k|1.77k|1.77k], ], |[1780|[1.78k|1.78k|1.78k], ], |[1790|[1.79k|1.79k|1.79k], ],
|[1800|[1.8k|1.8k|1.8k], ], ],
////////////////////////////////////
////////////////////////////////////
/////////
294.4
////////////////////////////////////
////////////////////////////////////
/////////

```

```
[[melon|apple], ],
```

We can see from the data that the opposite result happened: barely any creatures mated; the population became stagnant and continually dwindled; the genomes barely change with the reduced mutation and mating. Wildgrass eaters fare best at first due to the high quantity of wildgrass, but then are superceded by apple eaters, as apple trees replenish faster. There is only one dual-diet creature, but at least there is one.

Things to note for tomorrow:

- Unknown bug: water only spawns in a triangle in the bottom left of realm
- Unknown bug: it seems that animas do not move when their mode is approach, even though this should only be the case when their target is in range, and vision is usually much higher than reach (2.5 cells)
- Increase foliage to encourage a greater but steady population
- Giants have disappeared since I removed the square root factor advantage and cut foliage; these likely had a joint impact
- I could add diseases and fires that spread between plants; forces evolution in certain directions
- Add ice once pools are fixed
- Bug: it looks as though some corpses are zombies? They persist and behave as normal animas but keep blinking and showing the colour of a corpse

The bugs I mentioned I have been investigating since yesterday, but I have been unable to find their causes in the code.

Tomorrow's to-do (and the other days to come):

- Fix the aforementioned bugs
- Complete pool generation function and ice terrain
- Resolve sleep vs mating drive conflict
- Simulate a stable population of size 200-500
- Simulate predation naturally arising from a population of herbivores
- Add UI for setting up a simulation
- Add UI for controlling the simulation
- Add UI to convey data

22/03:

I ran another simulation and when two creatures survived for almost 20000 ticks I realised senescence had not been implemented correctly.

Ytagl and Ounukool had both reached an age of 19600 before I ended the simulation realising they might never die. An apple eater and a berry eater, their friend whose name began with Z had survived to the 16000 tick mark before dying of thirst. Such senescent creatures should have died from status[4] long before they reached this point. I located the error; the condition preventing animas from healing when old also prevented them from gaining damage. So I corrected this.

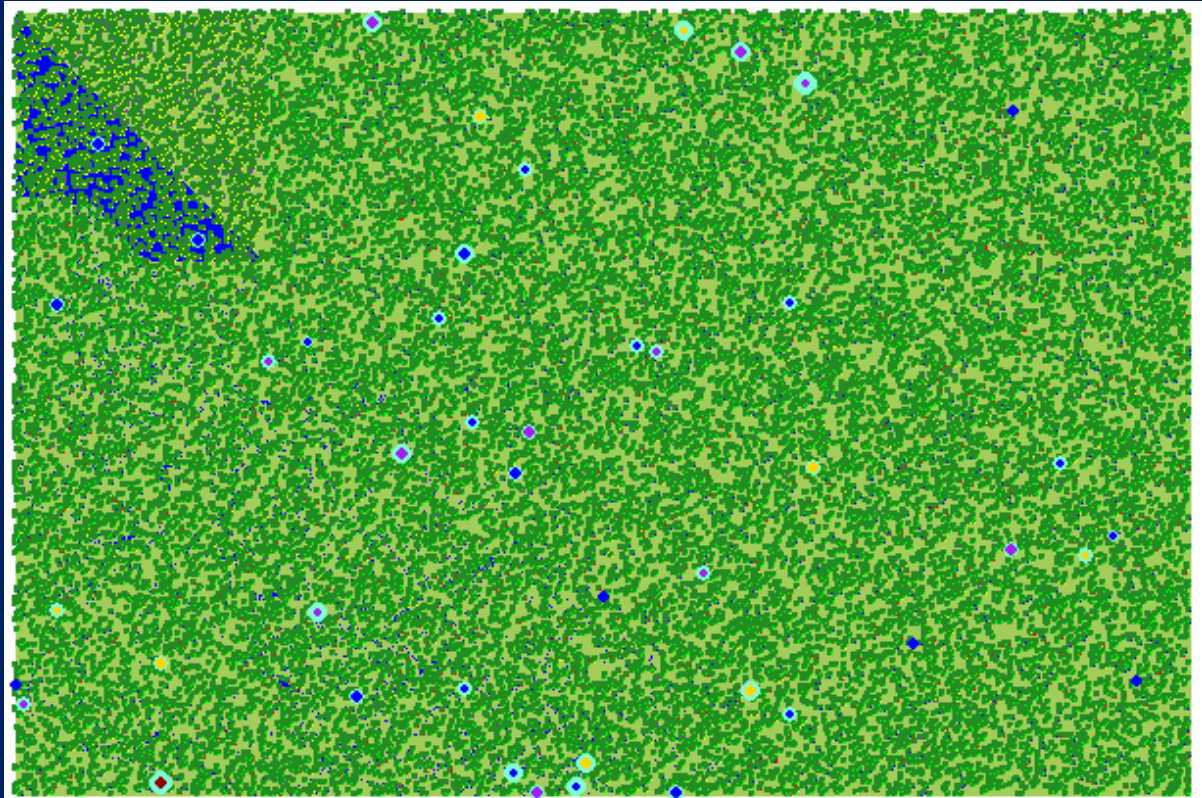
I increased the scrollbar to 5000 as with such a long simulation the terminal was still unable to capture all the data at once.

After another simulation I realised the senescence limit was still broken, because I had not used the syntax of `update(3,x,-1)` for reducing health; I had been using `update(3,-x)`. So I changed the condition to check if `quantity*sign` was below 0.

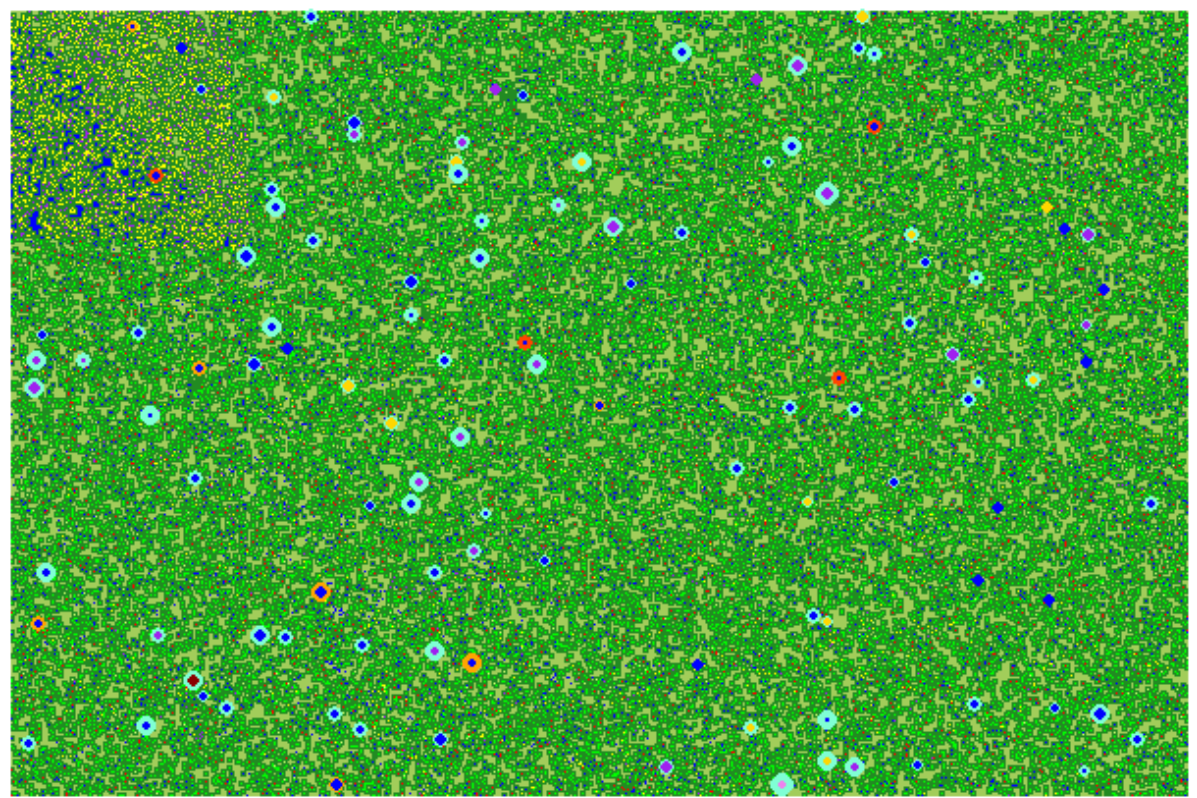
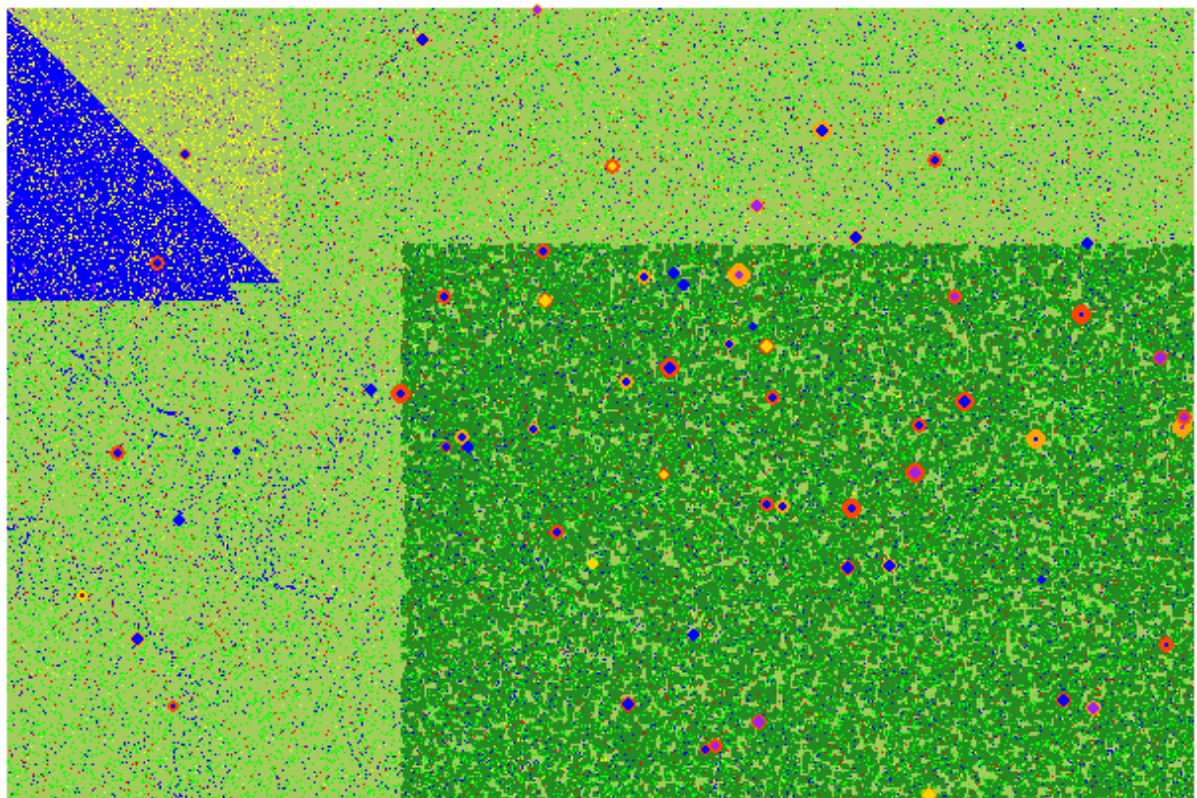
I changed energy expense to:

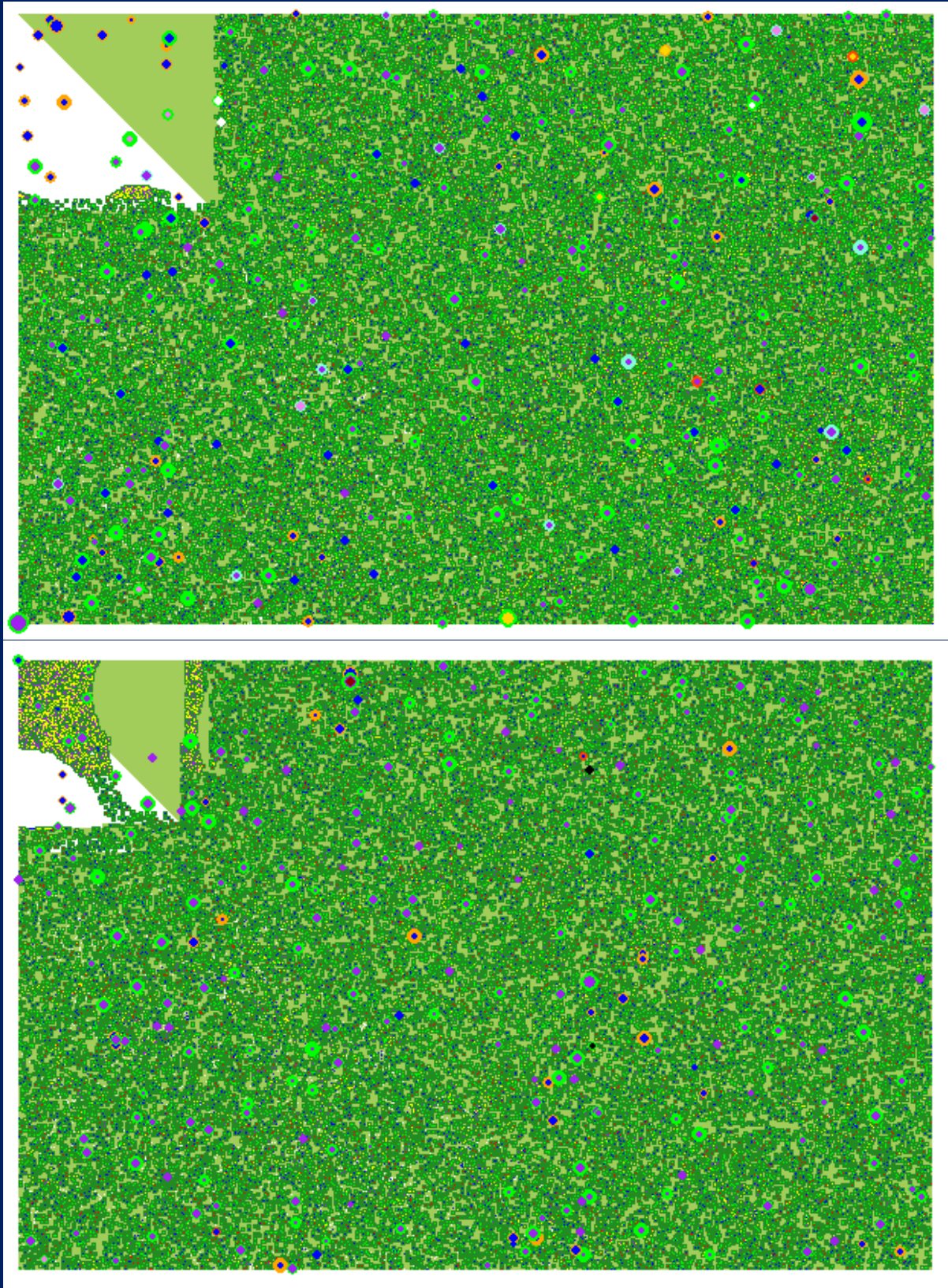
```
(0.01 * self.vision**2)/sqrt(self.birth_size) + hypot(self.dx, self.dy) + 1
```

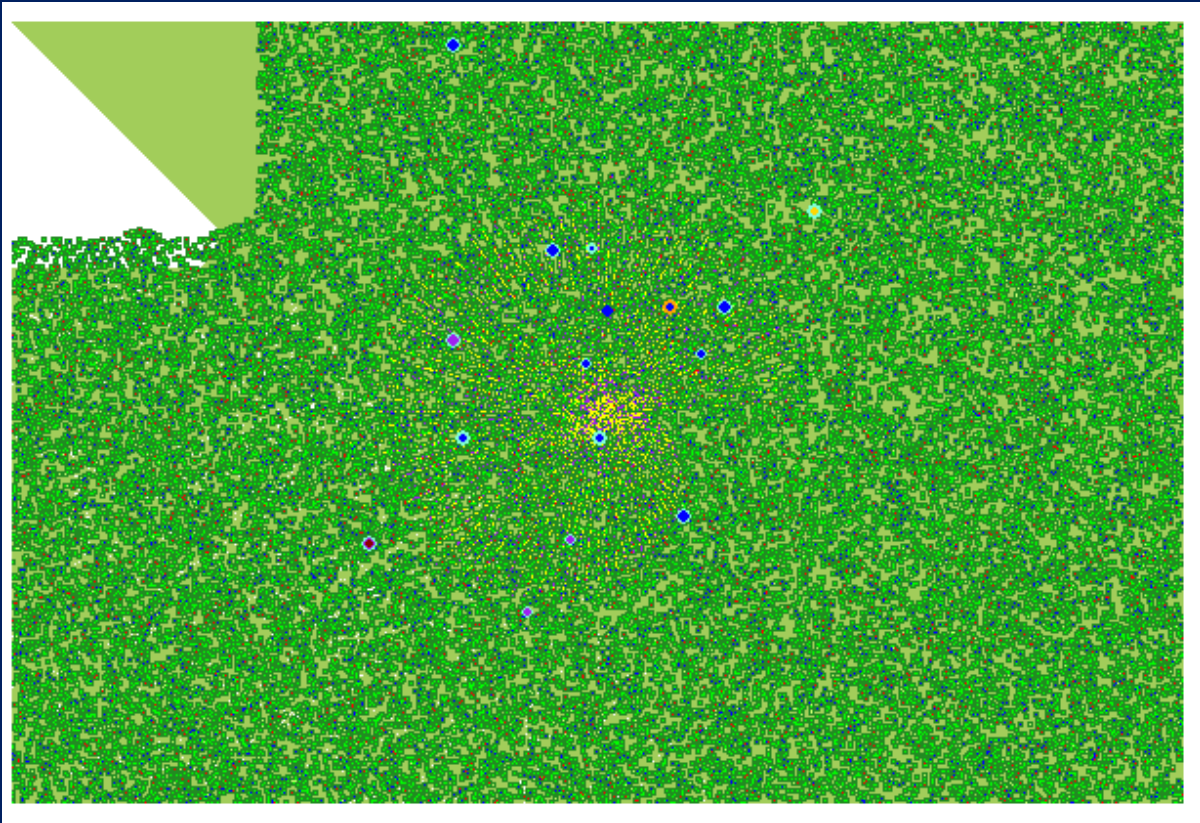
I added fruit colour sets so it is now possible to tell the species of a plant; I may later add colour indicators for abundance of fruit.



After some adjustments:







Finally fixed one of the bugs!

Amended code from :

```
if self.is_target_in_reach:
```

To:

```
if self.is_target_in_reach():
```

The previous condition of course always expresses as True. This one took a long time to find but now anima behaviour is much better, and animas do not have infinite reach in their vision.

Fixed corpses displaying from their own coordinates instead of the last displayed anima (this is what was causing the lonely zombie error; cause became apparent with an error which displayed when there were zero living animas and thus no reference for the corpses)

Applied the appropriate getters to the following code:

```
if self.target != None and (self.y + self.dy - self.get_target_y()) * (self.y
- self.get_target_y()) < 0:
    self.dy = self.get_target_y - self.y - MIN_DISTANCE_TO_TARGET
    if self.target != None and (self.x + self.dx - self.get_target_x())
* (self.x - self.get_target_x()) < 0:
        self.dx = self.get_target_x - self.x - MIN_DISTANCE_TO_TARGET
```

Used new 'angle_to' function to update old code:

```
"""dsx = self.get_target_x() - self.x
    dsy = self.get_target_y() - self.y
    true_angle = atan2(dsy, dsx)"""
```

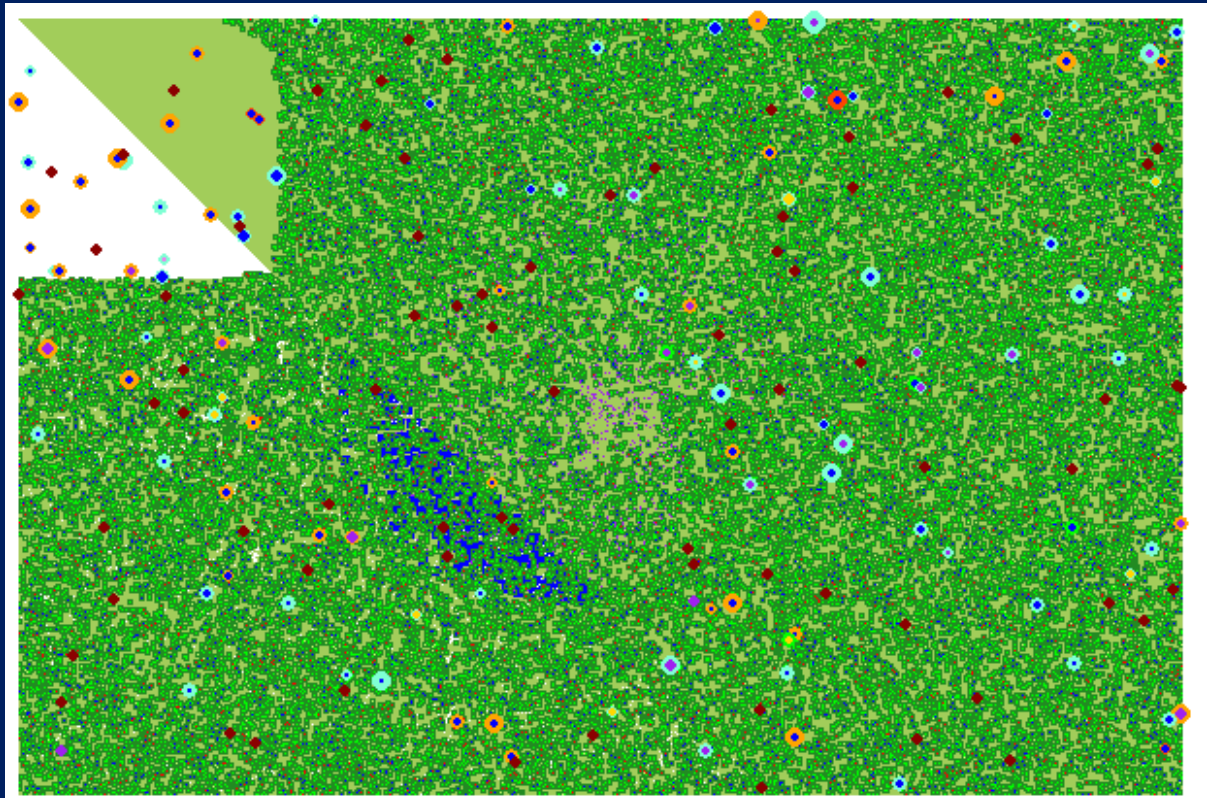


```

        true_angle = self.angle_to(self.target)
"""dsx = self.get_target_x() - self.x
    dsy = self.get_target_y() - self.y
    true_angle = (atan2(dsy, dsx) + math.pi) % (2 * math.pi)"""
    true_angle = self.target.angle_to(self)

```

Maybe the most common programming error I have made throughout this project is forgetting the brackets on function calls; it has happened again above ^



Current errors:

- Weird diagonal shapes for ice and water; has been constantly puzzling me (wetness checks shows about 1.6% and iciness is 0.8%; both these values should be much higher)
- Water and ice do not display correctly; this is proven by the fact that lilies can spawn in area where water does not explicitly show (I removed the lilies and made water rects 5x5 for the purposes of the above screenshot)
- Corpse sizes behaving erratically and changing as one

This is such that there seem to be errors both on the realm's generation side and the ui display side.

Corpse sizes fixed (error was that size was tied to anima variable instead of corpse)

The newest changes made it too difficult for animas to survive; I doubled all nutrition values (for hunger replenishment) and changed melon thirst replenishment from 20 to 100. I also halved the rate of hunger gain.

Fixed some of the conditions for simulation data functions; mostly to do with edge cases where there are zero or only one anima.

Blue Ghedda [wildgrass], 25.3 19.2 died of thirst aged 400. Its final status was [92.55951527471123, 100.0, 97.85712594978078, 0.3, 74.8381806384965] and it was the parent of []

Iletb [apple], 9.3 27.2 died of thirst aged 400. Its final status was [90.10799909681499, 100.0, 97.55957288981035, 0.3, 74.82399961361202] and it was the parent of []

Eximui [apple], 18.7 29.7 died of thirst aged 400. Its final status was [79.12308809555497, 100.0, 100.24616199110969, 0.3, 74.80240232050924] and it was the parent of []

Ouhexozoukk [lily], 5.1 12.0 died of thirst aged 400. Its final status was [76.75204316420003, 100.0, 97.26175443179724, 0.3, 74.94968163960354] and it was the parent of []

Ounuthusnuifr [apple], 20.4 13.6 died of thirst aged 400. Its final status was [97.47589819899214, 100.0, 98.98314203185578, 0.3, 72.093012988801518] and it was the parent of []

Oduf [apple], 27.0 29.1 died of thirst aged 400. Its final status was [90.89073920698814, 100.0, 97.92612126612232, 0.3, 69.00900646199202] and it was the parent of []

Mov [berry], 8.9 27.9 died of thirst aged 400. Its final status was [94.79361749161762, 100.0, 98.31597264079755, 0.3, 66.61641171562712] and it was the parent of []

Ulaggeixx [berry], 16.7 14.3 died of thirst aged 400. Its final status was [94.50867701697203, 100.0, 98.75764851476477, 0.3, 73.48985254692103] and it was the parent of []

Ughegbeith [berry], 19.8 22.5 died of thirst aged 400. Its final status was [76.41457966406388, 100.0, 99.34333714525866, 0.3, 74.82036099256956] and it was the parent of []

Fajuk [apple], 5.8 15.9 died of thirst aged 400. Its final status was [67.55557152537763, 100.0, 100.09047504475002, 2.7999999999995557, 74.98242486507448] and it was the parent of []

Beghuigeimwou [melon], 22.7 16.1 died of thirst aged 400. Its final status was [79.6388501187113, 100.0, 99.27770023742254, 0.3, 74.86465945789236] and it was the parent of []

Etrozo [wildgrass], 14.5 12.5 died of thirst aged 400. Its final status was [95.54598048834131, 100.0, 98.0530608419445, 0.3, 74.82191973750335] and it was the parent of []

Zobui [wildgrass], 5.9 8.6 died of thirst aged 400. Its final status was [93.3310059776515, 100.0, 100.10339434018826, 0.3, 74.83180593042047] and it was the parent of []

Toowih [berry], 27.9 16.8 died of thirst aged 400. Its final status was [75.22026344764068, 100.0, 98.61498967738596, 36.03076576758226, 74.8759021078169] and it was the parent of []

Ezewotee [wildgrass], 29.9 22.9 died of thirst aged 400. Its final status was [92.03019523460888, 100.0, 98.89912829964744, 0.3, 74.87758237460798] and it was the parent of []

Xupn [berry], 25.4 20.2 died of thirst aged 400. Its final status was [84.79797889493582, 100.0, 99.74127489345351, 0.3, 74.97138272838912] and it was the parent of []

Irumme [wildgrass], 14.8 20.6 died of thirst aged 400. Its final status was [94.56464601590824, 100.0, 98.13854243194348, 0.3, 74.85014397599173] and it was the parent of []

Udi [apple], 28.8 22.7 died of starvation aged 400. Its final status was [100.28087454451072, 100.0, 98.58537839318775, 0.3, 74.99679539765923] and it was the parent of []

Hullei [berry], 18.8 7.5 died of thirst aged 400. Its final status was [95.19839597457894, 100.0, 98.66056811729885, 0.3, 74.96273748889294] and it was the parent of []

Lil' Anillao [lavendar], 9.1 20.3 died of thirst aged 400. Its final status was [87.70594296311769, 100.0, 97.41188592623404, 0.3, 74.95223901574647] and it was the parent of []

Umouvzy [apple], 14.3 28.3 died of thirst aged 400. Its final status was [95.54223701728513, 100.0, 99.95816627662974, 0.3, 74.83440895082286] and it was the parent of []

Uixovui [wildgrass], 19.1 16.3 died of thirst aged 400. Its final status was [96.06297964003096, 100.0, 99.63069647559453, 0.3, 69.7860991351085] and it was the parent of []

Thuiui [wildgrass], 9.8 18.7 died of thirst aged 400. Its final status was [94.08374551155997, 100.0, 99.47788054087715, 0.3, 68.06088975075227] and it was the parent of []

Vevevo [lavendar], 12.5 26.7 died of thirst aged 400. Its final status was [81.96345825518087, 100.0, 99.92691651036192, 4.499999999999726, 69.60880803236945] and it was the parent of []

Uikih [apple], 25.8 28.1 died of thirst aged 400. Its final status was [92.10616289633508, 100.0, 98.3449272688139, 0.3, 72.88458890837475] and it was the parent of []

Boutheitowu [lavendar], 29.2 9.4 died of thirst aged 400. Its final status was [99.43786677499835, 100.0, 99.87573354999607, 0.3, 73.33890084379226] and it was the parent of []

Veiktuif [apple], 17.3 9.7 died of thirst aged 400. Its final status was [97.91431933330364, 100.0, 99.3112253359504, 0.3, 74.82890844840375] and it was the parent of []

Uw [lily], 18.4 7.2 died of thirst aged 400. Its final status was [93.73295050904726, 100.0, 97.46500101809382, 23.293656648462758, 74.80668376117872] and it was the parent of []

Reicbooxahy [wildgrass], 6.3 28.4 died of thirst aged 400. Its final status was [96.29533340378406, 100.0, 98.61821699229425, 0.3, 65.54397448903285] and it was the parent of []

Mrs Uikahh [apple], 14.1 17.2 died of thirst aged 400. Its final status was [95.71650516600796, 100.0, 100.40322287215162, 0.3, 74.98124809898624] and it was the parent of []

Khoodui [wildgrass], 26.2 20.2 died of thirst aged 400. Its final status was [94.42032547707704, 100.0, 97.23131251321536, 0.3, 72.33963612314827] and it was the parent of []

Ywuisuupv [meat], 22.9 14.2 died of thirst aged 400. Its final status was [99.90048622870239, 100.0, 97.74586752747801, 33.86973953582011, 74.95273262939662] and it was the parent of []

Eide [berry], 9.6 17.4 died of thirst aged 400. Its final status was [92.72625262297538, 100.0, 99.52567980105079, 0.3, 74.9360694078001] and it was the parent of []

Oolaofa [meat/wildgrass], 27.7 29.3 ate wildgrass

Wezzal [apple], 5.9 13.9 died of thirst aged 401. Its final status was [90.6458287846349, 100.25, 100.34188951313085, 0.3, 70.2858459505985] and it was the parent of []

Hethevi [apple], 22.3 25.9 died of starvation aged 401. Its final status was [100.33365860252046, 100.25, 100.07661112919595, 0.3, 66.11264056325125] and it was the parent of []

Eijecnouhuix [wildgrass], 14.9 8.9 died of thirst aged 401. Its final status was [96.54505664359294, 100.25, 100.69591168435583, 0.3, 74.90645002840853] and it was the parent of []

Naodazeijjoo [meat], 26.1 17.3 died of thirst aged 401. Its final status was [82.98684724990179, 100.25, 99.43264150054996, 52.38806981191226, 74.92882658847586] and it was the parent of []

Eneiccel [wildgrass], 27.4 19.2 died of starvation aged 401. Its final status was [100.54072008765661, 100.25, 99.9286434915859, 0.3, 74.95886782031573] and it was the parent of []

Itlaorofou [berry], 29.7 24.6 died of thirst aged 401. Its final status was [83.80048441775024, 100.25, 100.488653879677, 0.3, 74.53107694914537] and it was the parent of []

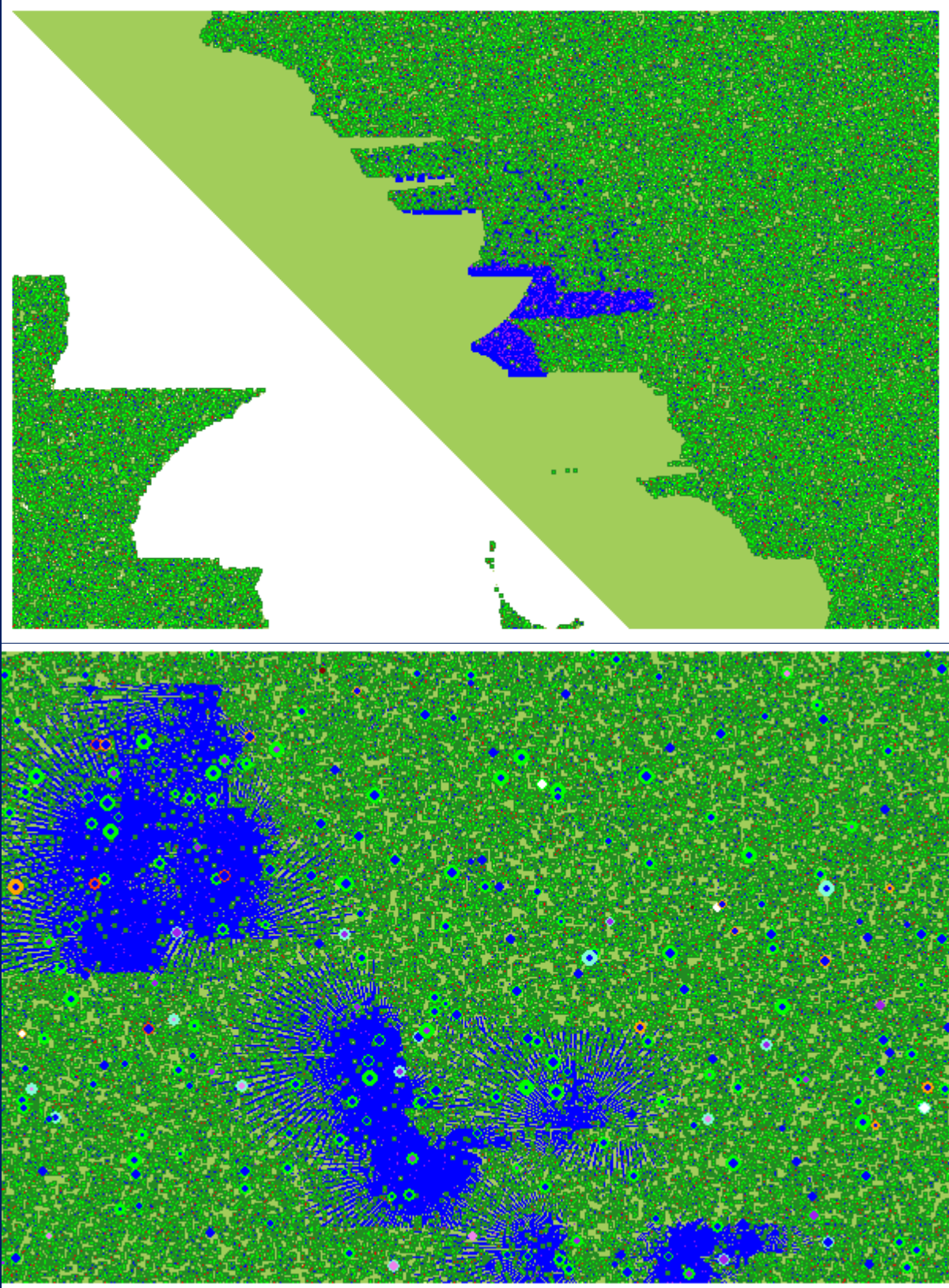
Eivehh [apple], 19.9 17.2 died of thirst aged 401. Its final status was [80.2141903171247, 100.25, 100.10548255054002, 0.3, 74.80357097814355] and it was the parent of []

Ajjujuithoo [lavendar], 18.4 29.1 died of thirst aged 401. Its final status was [88.83662182160396, 100.25, 98.67324364320771, 0.3, 74.9877328983083] and it was the parent of []

Luivvex [apple], 20.0 21.8 died of starvation aged 401. Its final status was [100.03874510397989, 100.25, 100.59027035820328, 0.3, 74.8333270916127] and it was the parent of []

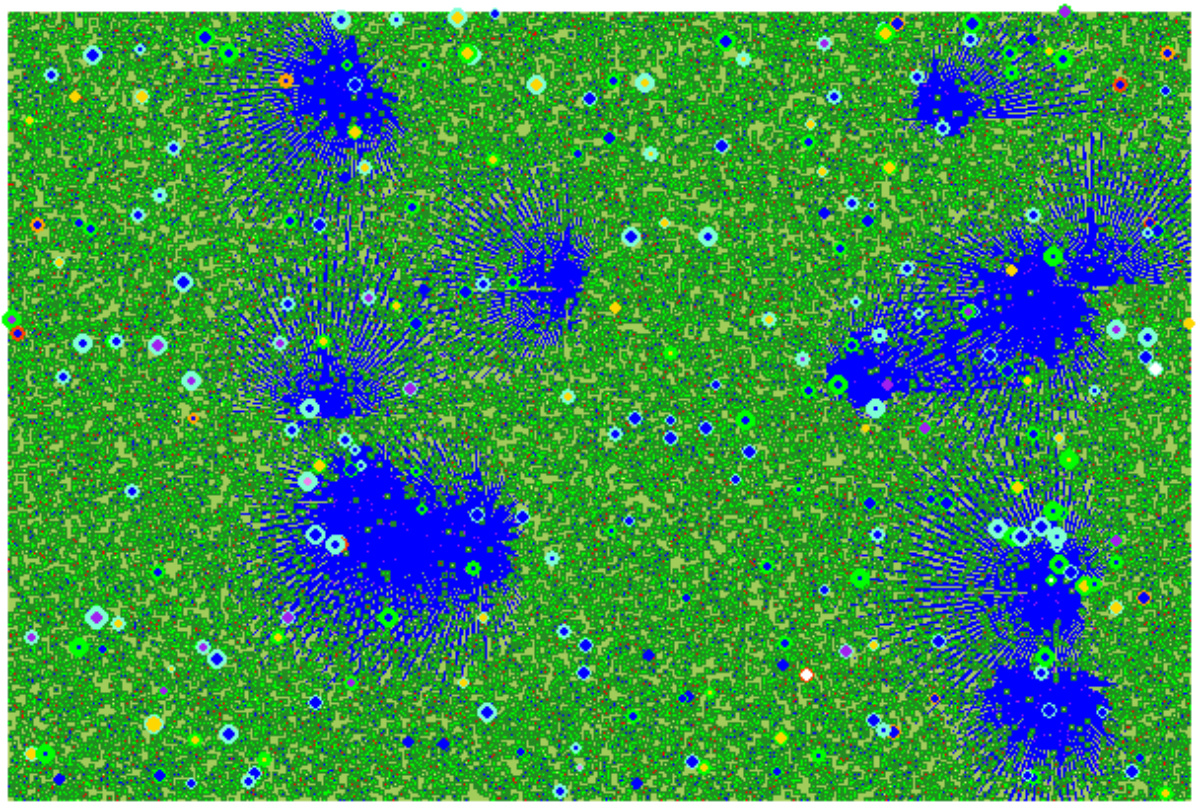
Lavime [wildgrass], 25.1 29.8 died of thirst aged 401. Its final status was [96.38163070480022, 100.25, 99.22876019029594, 0.3, 74.97080913348977] and it was the parent of []

Clearly water is an issue here. I continued to test the water generation and display functions.



Above: both row and col look within the same range (using x values)

Fixed below:



I halved energy expenditure again.

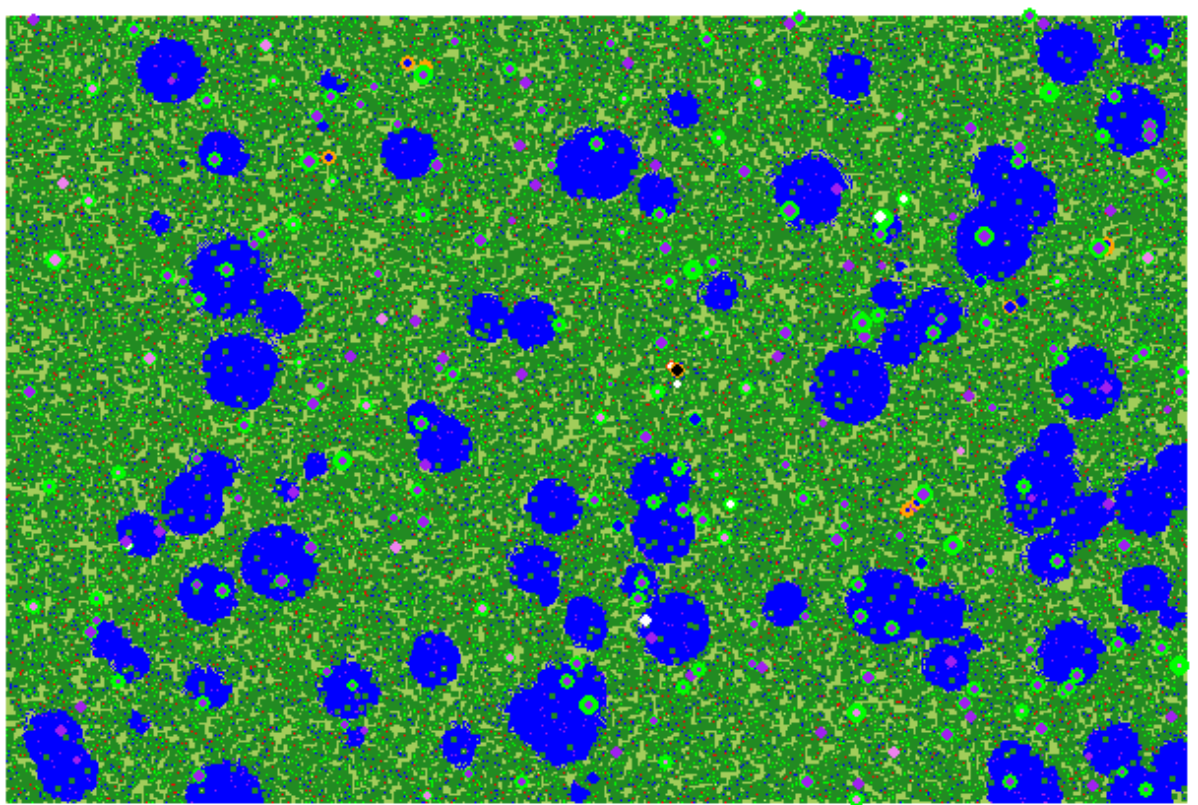
Changed polar tracing line from:

$$r = r + \text{random.randint}(-3, 3) * \sin(t) + 3 * \cos(t)$$

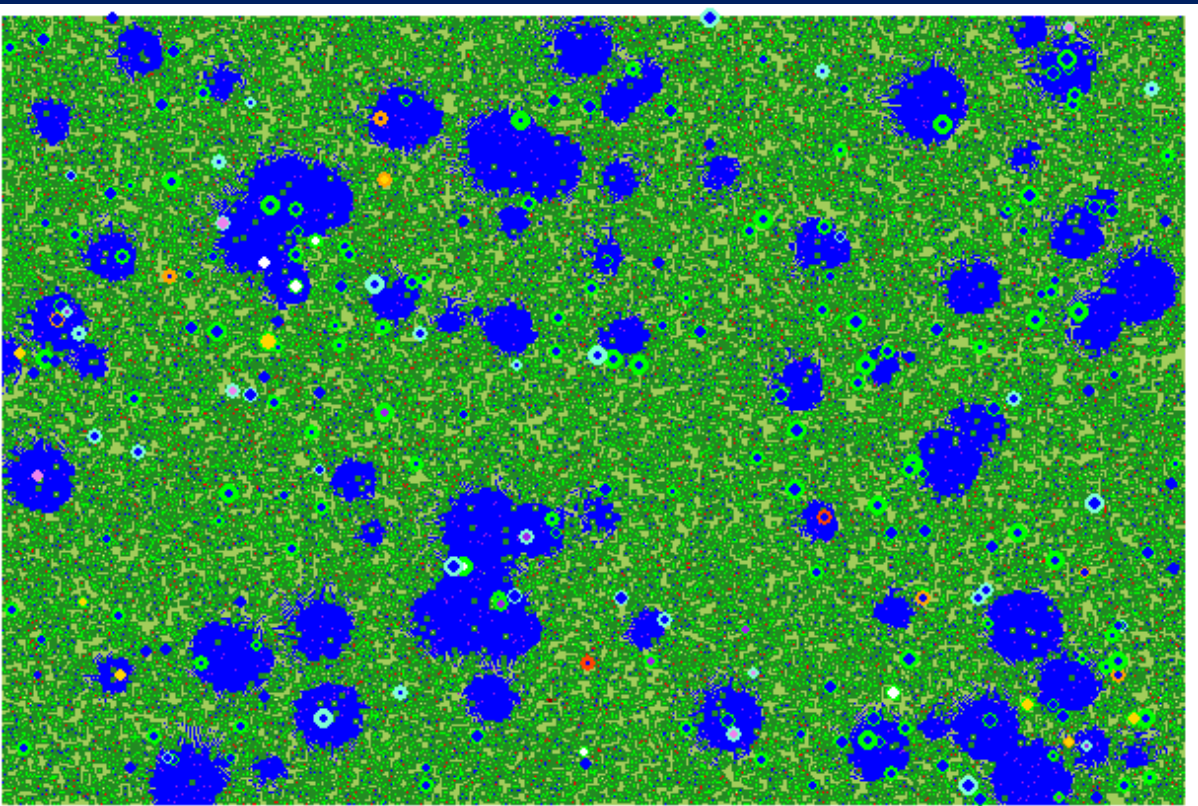
To:

```
r = max(r + random.uniform(-1, 1) * sin(t), 1)
```

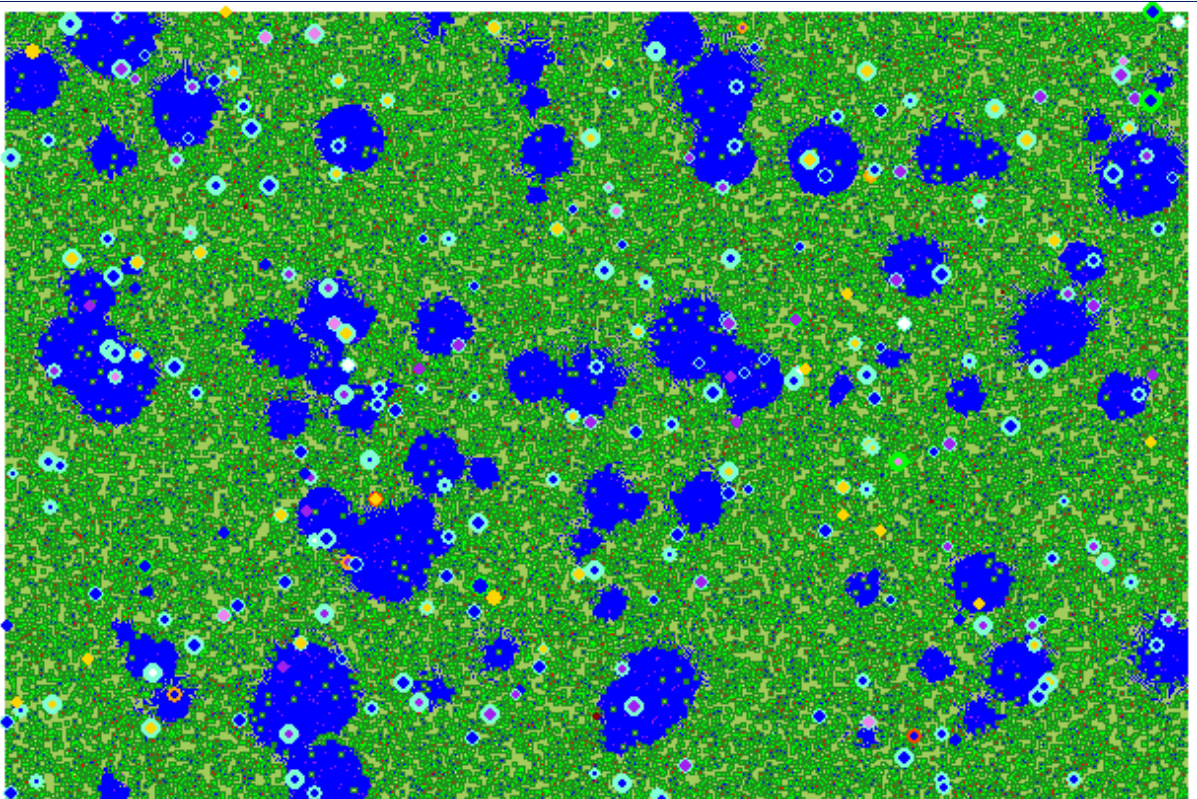
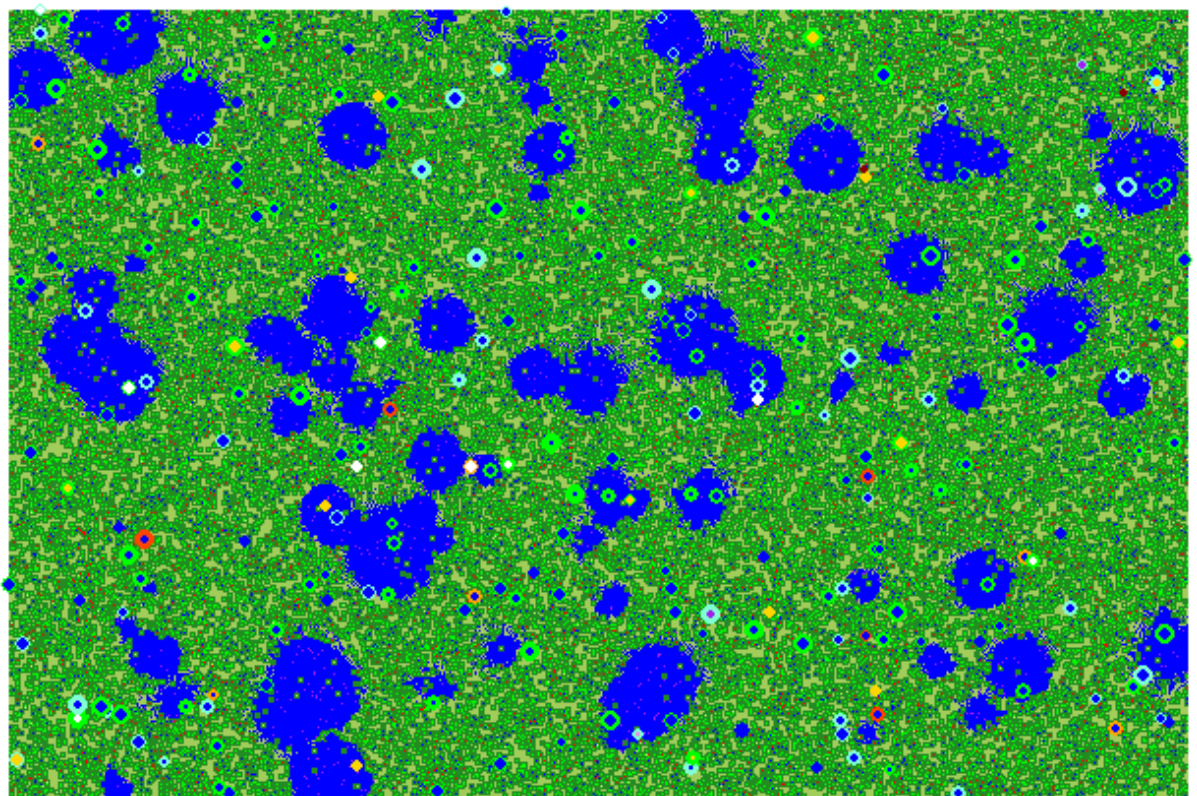
Success!:



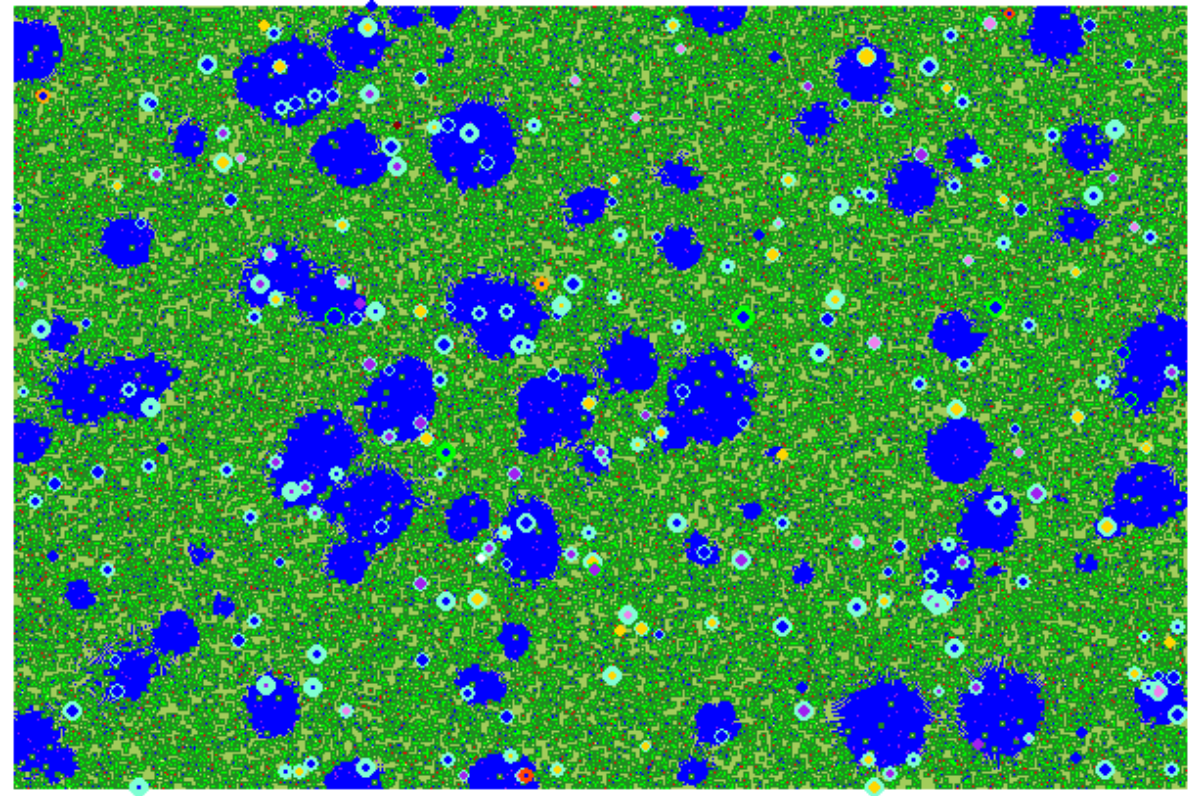
With a cosine term:



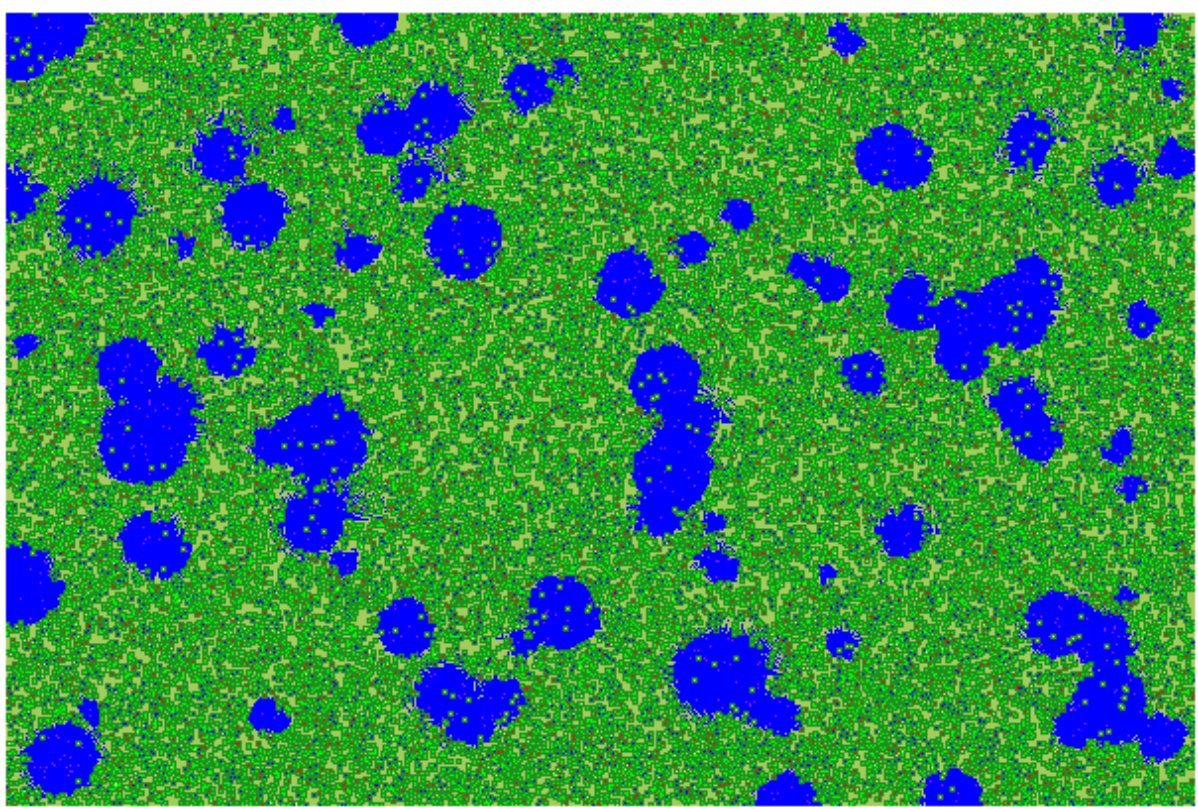
With doubled sine term and no cosine term:



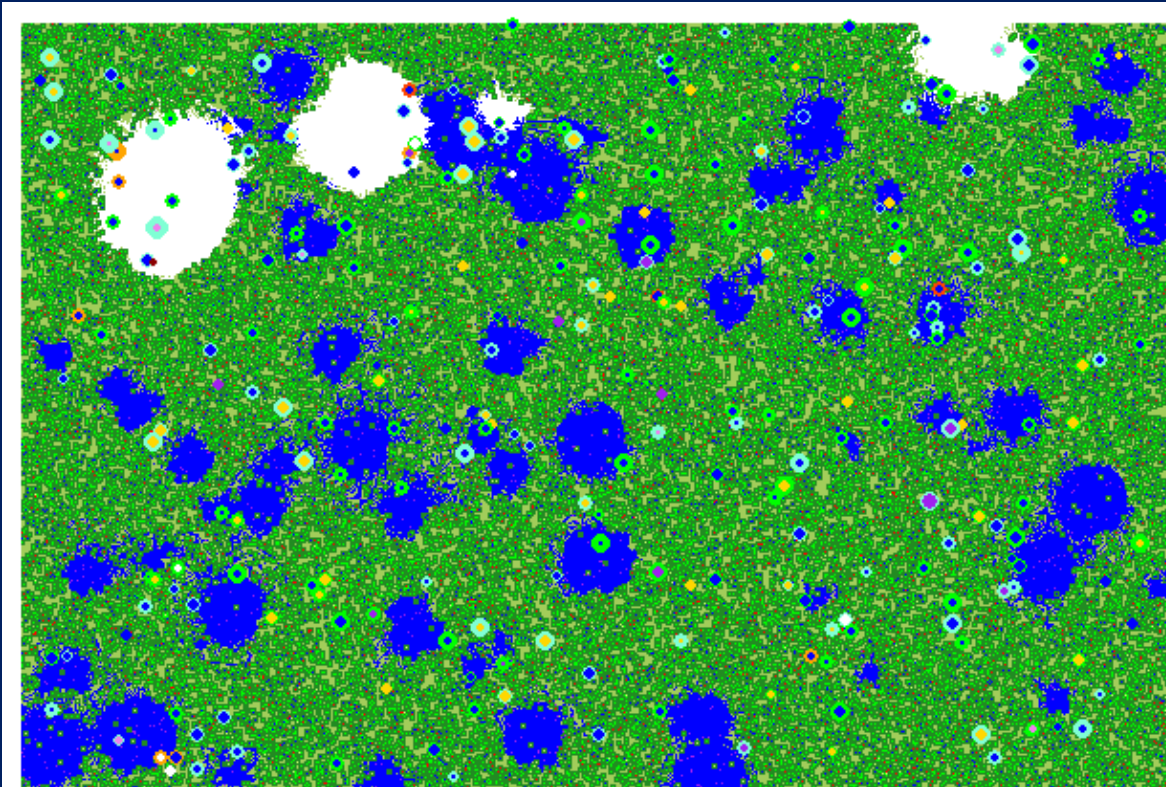
Thirst rate increase 0.2 $\rightarrow$ 0.1 (earlier today 0.25)



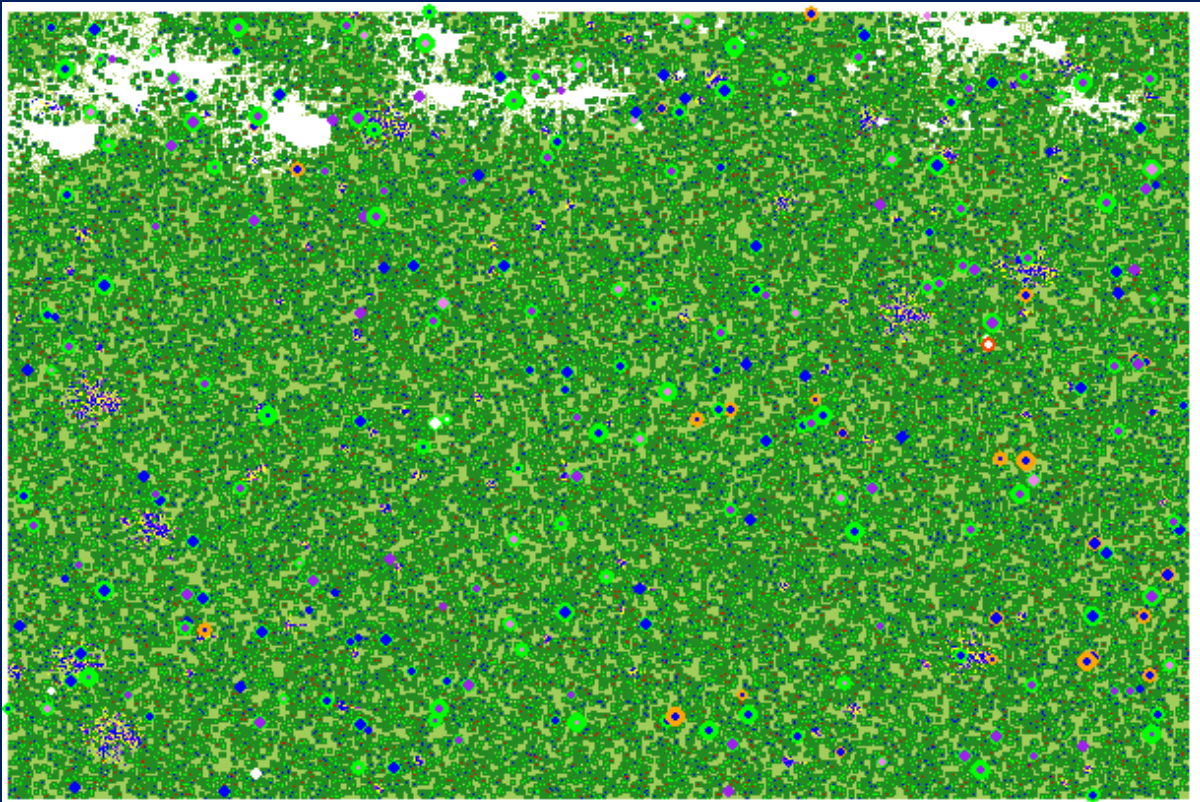
Pool angle gradation 100 -> 50



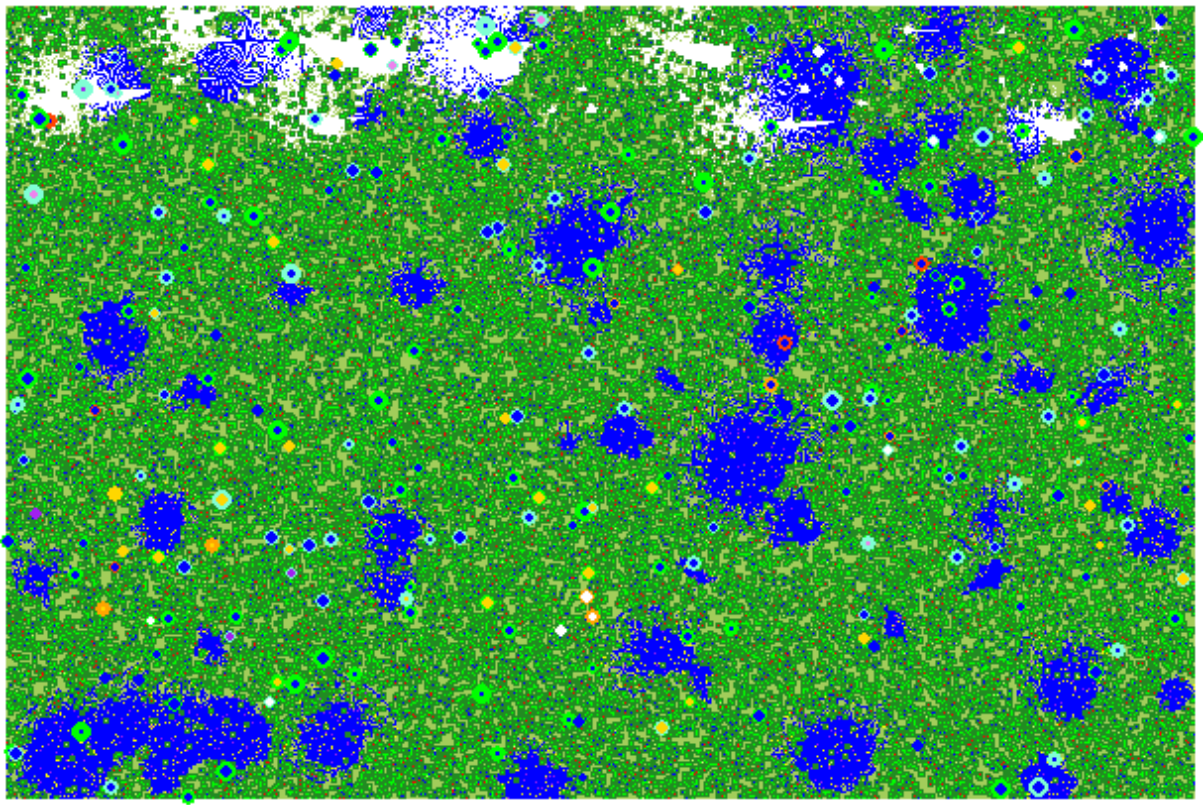
Added ice and pool angle gradation -> 360



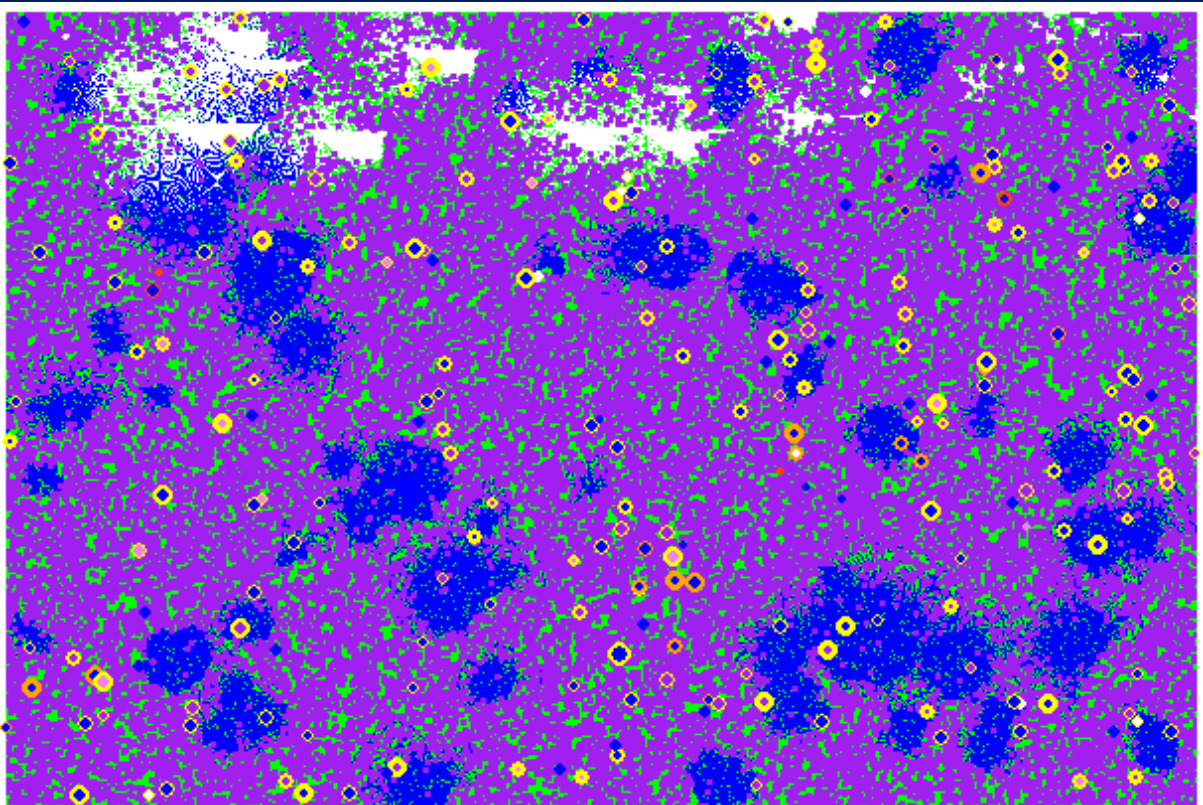
With cosine flattening:



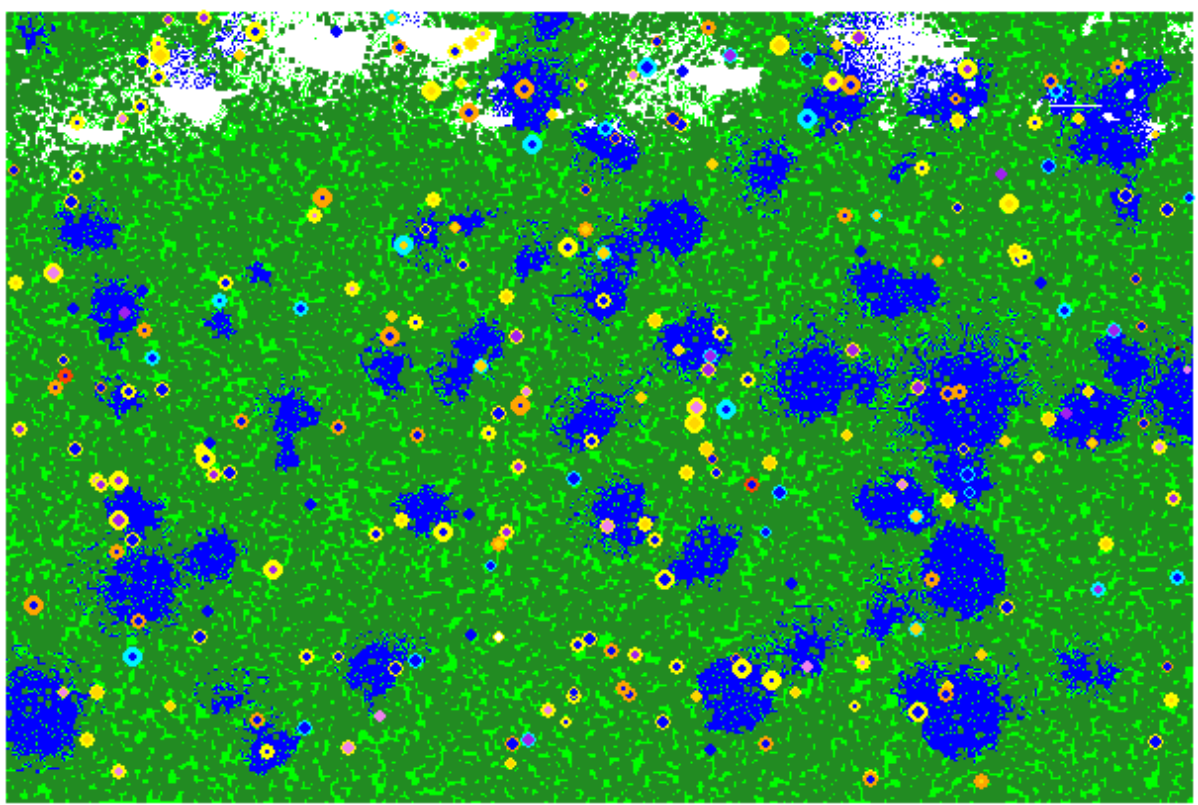
With flattened ice and rounded water:



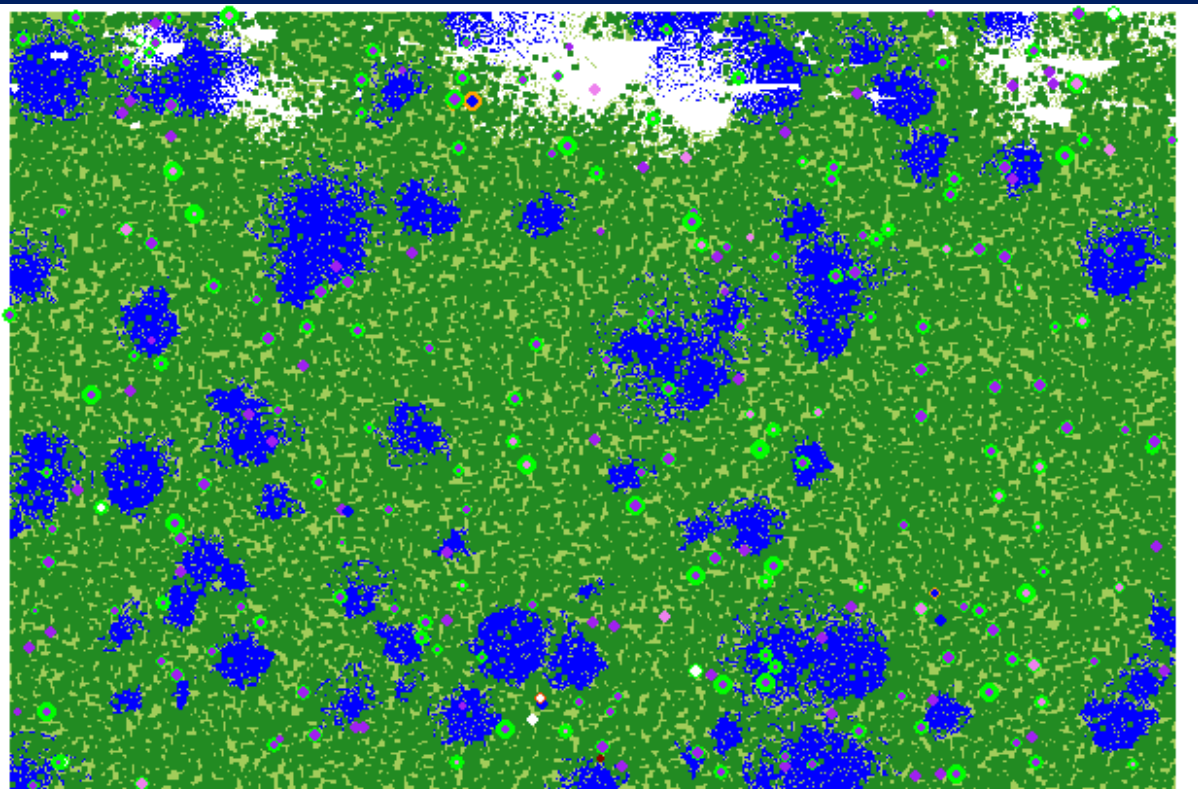
Og set:



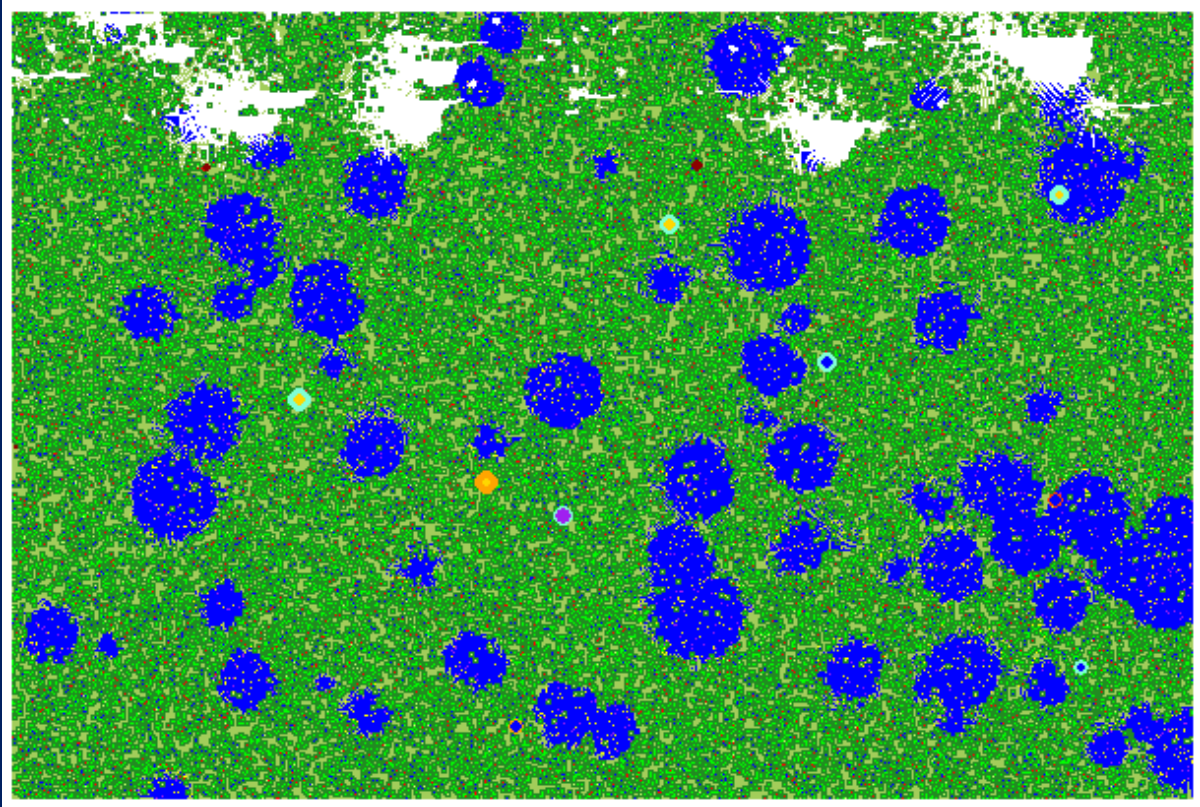
Declutter classic set:



Declutter mature set:



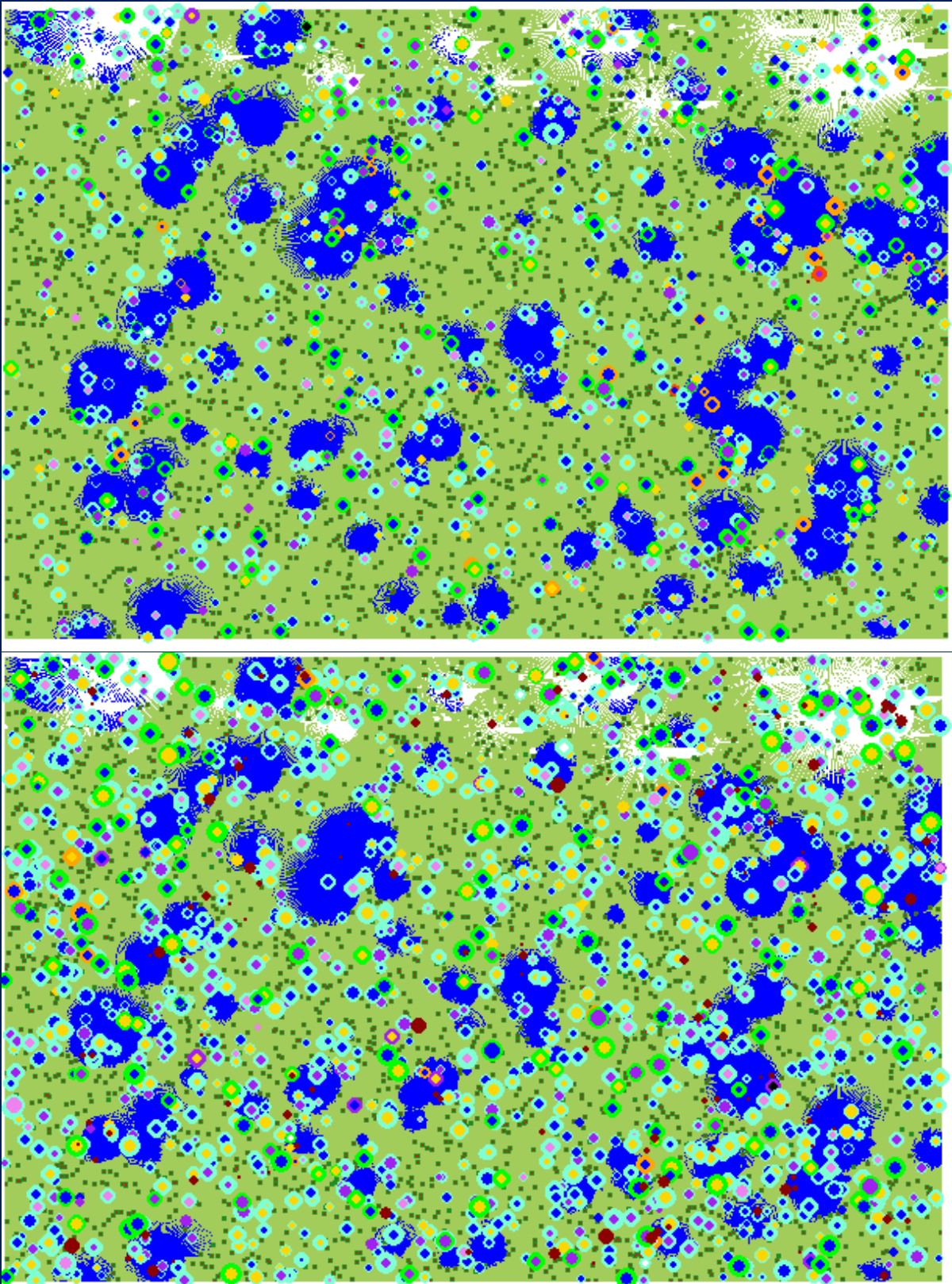
Perhaps: remove foliage from wildgrass; uncluttered mature with yellow adults



Made some adjustments to values.

Test conditions:

700 animas with diet ("apple") and hybrid diets off. 2000 trees



Hardcore apple survival (intent is for evolution for fitness is more extreme conditions)

Diet preservation factor 0.95 -> 1

Diet mutation plus factor 0

700 animas and only 750 trees

Increased energy requirements

All animas died.

30/03:

I set up a new venv with the new version of the program. I had to bug fix many of the issues which could not be detected without a terminal (on the school computer).

31/03:

I have reworked a lot of things since last update, with a plan to move into the final stage of UI-building tomorrow. The main changes are as follows (these were implemented between the apple experiment and 30/03, excepting world_facts.py being implemented on 31/03, though biomes were likewise created during this timeframe):

- Added biomes as fixed realm objects to a new file named 'world_facts.py'; this also contains terrain_index and any shared realm constants; created to avoid circular imports with cell.py and other files
- Added temperature as a realm attribute; this affects fruiting and animal energy expenditure
- Changed anima adaptation to different terrains; these are affected by a set terrain bias so that a weighted sum of anima terrain speed factors always sums to 1. Grass factor is also implemented such that movement in grass is increased from base speed by the weighted grass factor, and amphibian factor is also implemented now such that movement in water is increased from one-fifth of base speed by the weighted amphibian factor

Since pools still had the unnatural finger-like patterns at the edges, I tried implementing a velocity-based approach rather than simple displacement. Here acceleration at each angle is randomised and scaled by a trigonometric function of time.

```
def generate_pool(self, cx, cy, radius, type, shapetype):
    #using parametric polar curves to generate curved pools of varying shape

    angle_start = random.randint(0, POOL_ANGLE_GRADATION)
    r = radius
    pool_points = []
    maxr = 0
    velocity = 0
    for u_theta in range (POOL_ANGLE_GRADATION):
        normalising_factor = 2 * PI / POOL_ANGLE_GRADATION
        n_theta = u_theta * normalising_factor
        t = (angle_start + u_theta) * normalising_factor

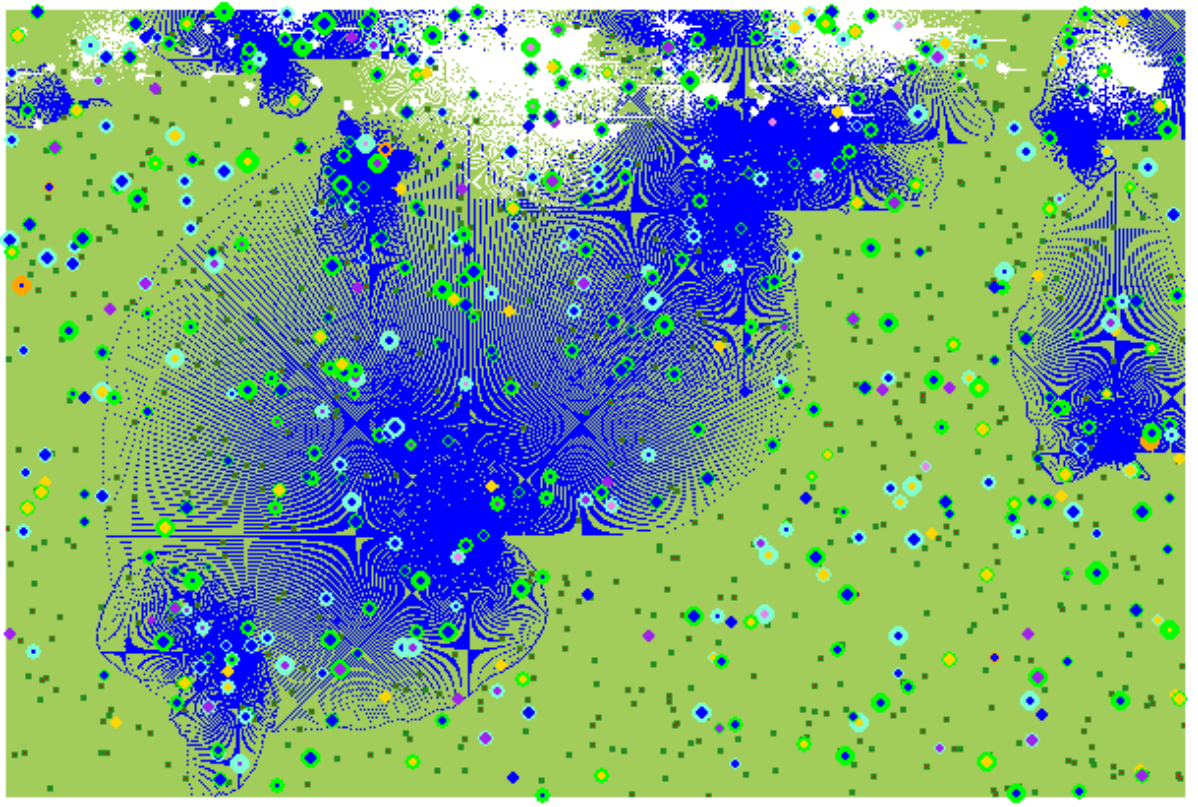
        velocity += random.uniform(-1, 1) * sin(t)
        velocity *= 0.95
        if shapetype == "rounded":
            r = max(r + velocity, 1)
        elif shapetype == "flattened":
            r = max(r * cos(t) + random.uniform(-2, 2) * sin(t), 1)
        # -> collect points using rsintheta and rcostheta
```

```

        if r > maxr:
            maxr = r
        pool_points.append((cx + r*cos(n_theta), cy + r*sin(n_theta)))
        #print(theta)
    for point in pool_points:
        #print(point)
        if 0 <= point[0] < self.width and 0 <= point[1] < self.height:
            self.environment[int(point[1])][int(point[0])] =
terrain_index[type] #temporary code, need fill in possible solution:
keep choosing adjacent tile closest to next (polar?) function result until back
to original tile, then closed object can be easily filled in using edge of pool
as boundaries for fill in, for each row start filling in when reach one and
stop when reach next, but need to cover edge cases of pools that go off map,
cant then use polarity, so how?
            #print((int(point[0]), int(point[1])))
    # -> draw inside polygon of points, checking for edge of world
    centre = Cell(self, cx, cy)
    maxr = int(maxr+1)
    for row in range(cy-maxr, cy + maxr+1):
        for col in range(cx-maxr, cx + maxr+1):
            if 0 <= col < self.width and 0 <= row < self.height:
                cell = Cell(self, col, row)
                arg = centre.angle_to(cell)
                u_theta = arg / normalising_factor
                p1 = pool_points[floor(u_theta)]
                p2 = pool_points[ceil(u_theta % (2*PI))]
                d1 = centre.distance_to(p1)
                d2 = centre.distance_to(p2)
                lerpdist = d1 + (d2-d1) * (u_theta-floor(u_theta))
                if cell.is_within(centre, lerpdist):
                    self.environment[row][col] = terrain_index[type]
                    #print((col,row))

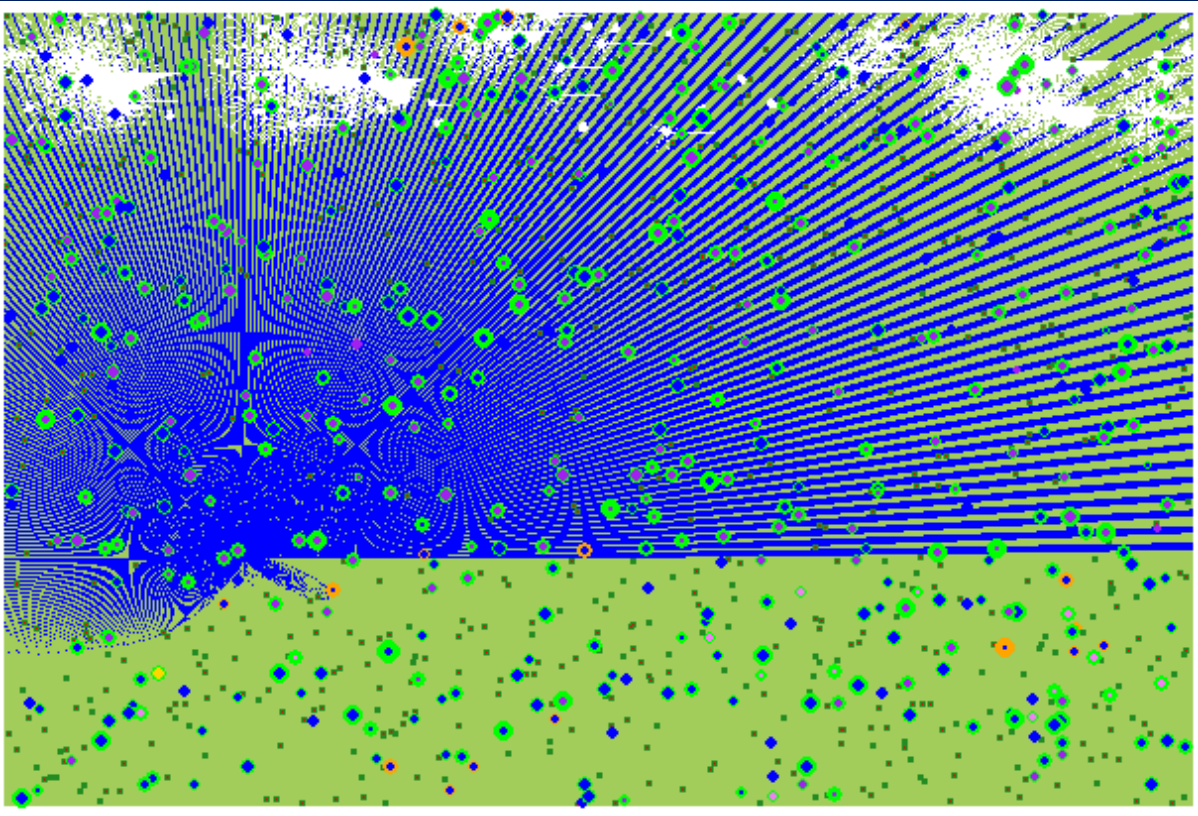
```

The result was not at all what I intended:

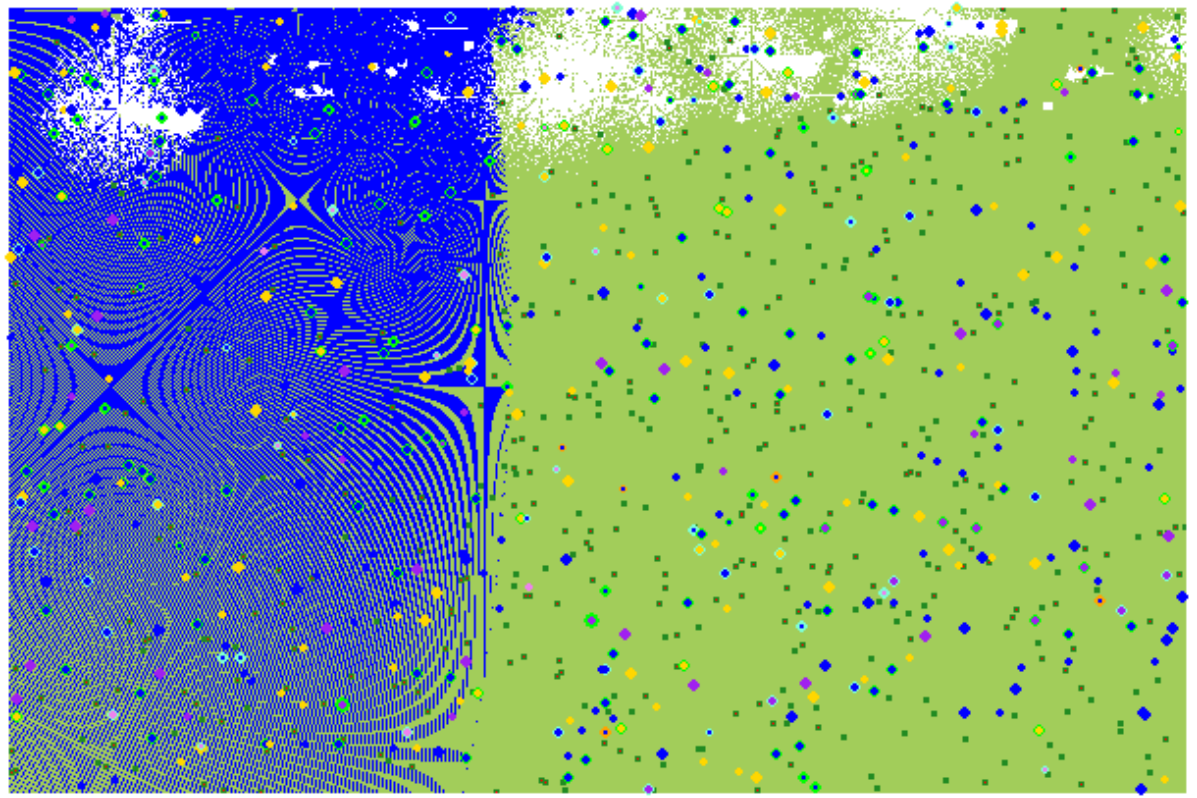


I tried making acceleration a random variable which is re-generated at random intervals.

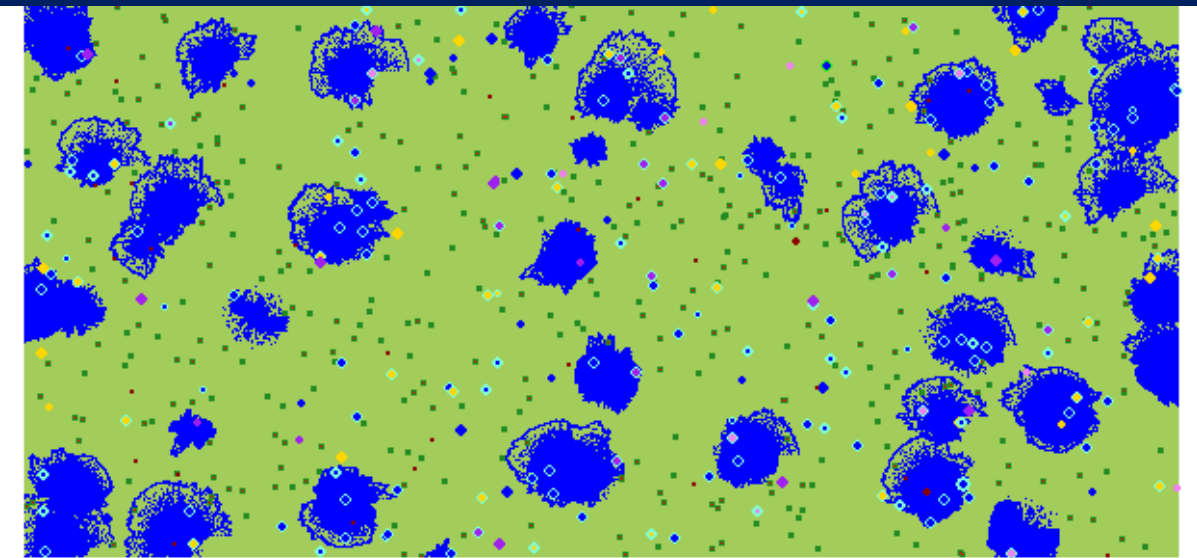
Result:



Finally, I tried increasing the pool angle gradation from 360 to 1200:

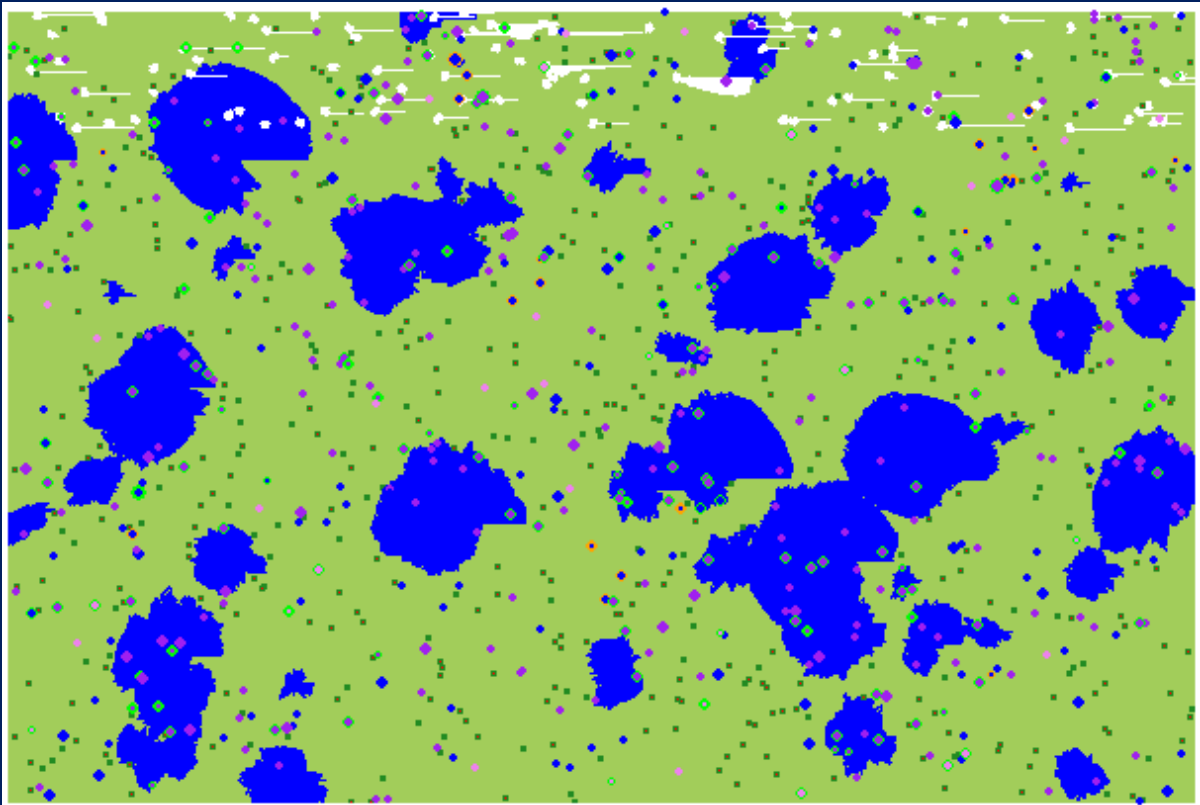


For now, I will revert to the previous pool generation algorithm, and revisit this later as I am yet unable to come up with a satisfactory solution.

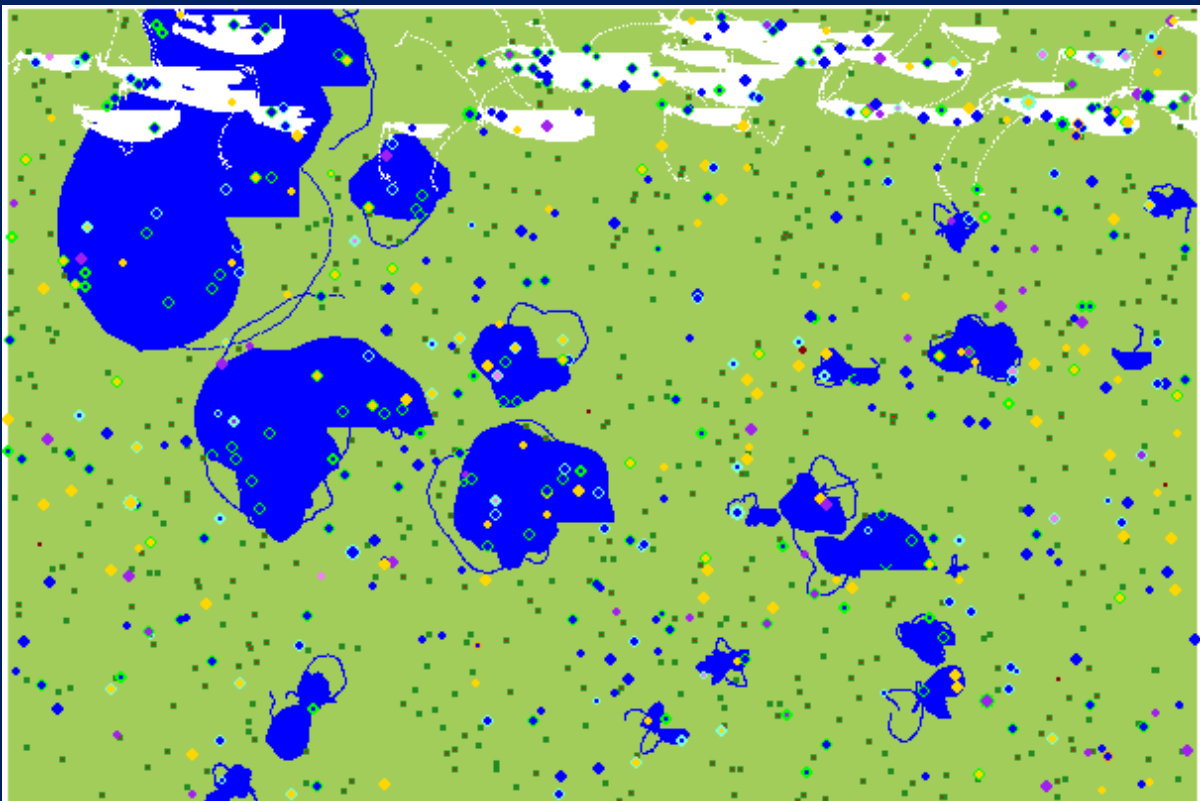


I realised that since `u_theta` in my linear interpolation code is in fact the integer unnormalised angle, `floor` and `ceil` will both return that same angle. So instead:

```
p1 = pool_points[floor(u_theta)]
p2 = pool_points[(floor(u_theta) + 1) % POOL_ANGLE_GRADATION]
```

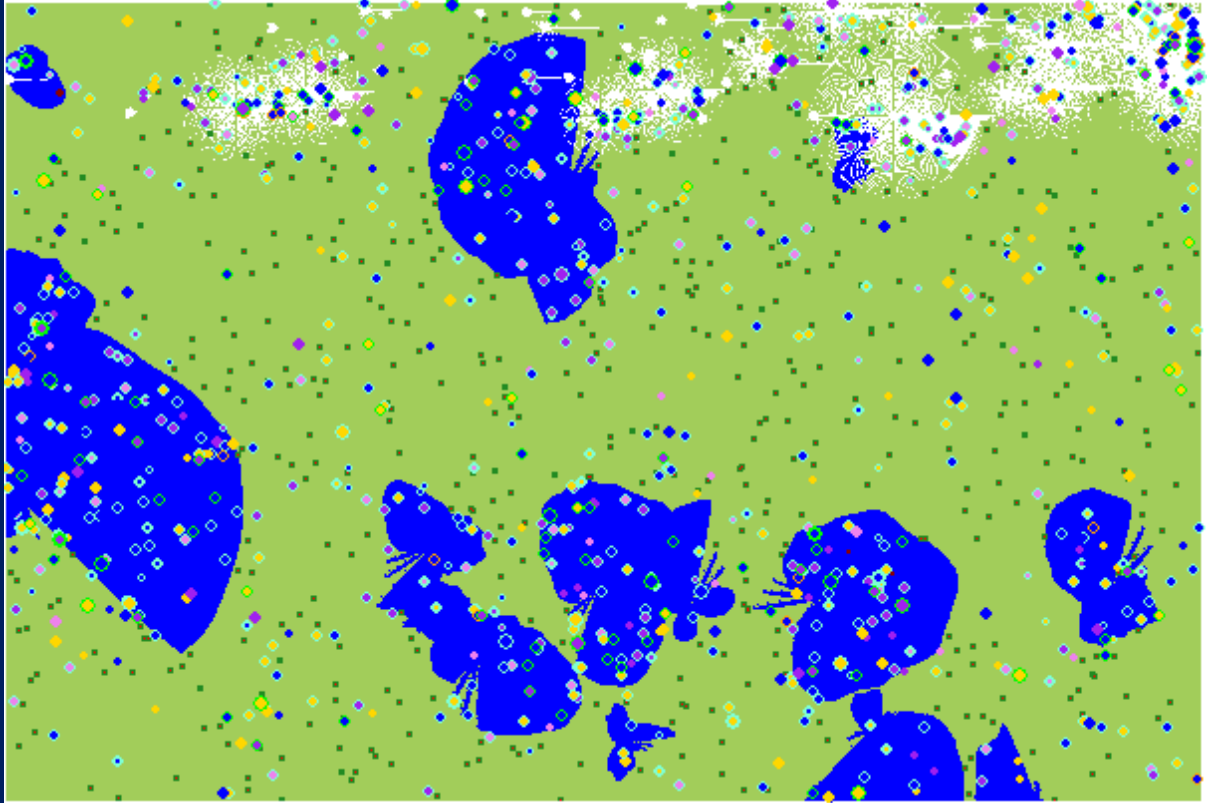



Now the main issue is that r at angle 2π does not match up with r at angle 0 .



I changed plotting to use t instead of n_theta so that the shape's orientation is randomised instead of always starting from the horizontal direction. I also added an accelerating term which I intended to smoothen the gap between r values at the discontinuity, but only had the effect of creating strange double or triple fingers at this angle range. The algorithm will be revisited later.

I re-attempted the apple experiment with 500 trees and tweaked anima and realm constants:



It is difficult to create a simulation in which animas do not either go extinct within a couple generations, or grow to a very large population, overeat, and then go extinct anyway

Example of a population boom which then causes mass starvation and extinction:

```
[[0|500], [[10|541], [[20|570], [[30|590], [[40|606], [[50|618], [[60|633], [[70|645], [[80|656],
[[90|665], [[100|671], [[110|677], [[120|686], [[130|692], [[140|698], [[150|701], [[160|711],
[[170|723], [[180|735], [[190|743], [[200|755], [[210|767], [[220|773], [[230|787], [[240|799],
[[250|812], [[260|818], [[270|827], [[280|841], [[290|845], [[300|854], [[310|863], [[320|879],
[[330|888], [[340|903], [[350|919], [[360|937], [[370|945], [[380|964], [[390|981], [[400|999],
[[410|1025], [[420|1045], [[430|1073], [[440|1090], [[450|1119], [[460|1148], [[470|1170],
[[480|1198], [[490|1217], [[500|1240], [[510|1260], [[520|1277], [[530|1299], [[540|1321],
[[550|1333], [[560|1340], [[570|1349], [[580|1367], [[590|1360], [[600|1349], [[610|1350],
[[620|1353], [[630|1339], [[640|1333], [[650|1326], [[660|1313], [[670|1291], [[680|1276],
[[690|1269], [[700|1242], [[710|1205], [[720|1188], [[730|1153], [[740|1118], [[750|1075],
[[760|1045], [[770|1031], [[780|1014], [[790|994], [[800|971], [[810|929], [[820|896],
[[830|864], [[840|834], [[850|799], [[860|754], [[870|725], [[880|694], [[890|653], [[900|616],
[[910|589], [[920|558], [[930|534], [[940|518], [[950|493], [[960|480], [[970|454], [[980|436],
[[990|415], [[1000|399], [[1010|278], [[1020|255], [[1030|238], [[1040|233], [[1050|221],
[[1060|205], [[1070|198], [[1080|188], [[1090|177], [[1100|164], [[1110|155], [[1120|148],
[[1130|136], [[1140|124], [[1150|117], [[1160|113], [[1170|106], [[1180|99], [[1190|90],
[[1200|82], [[1210|75], [[1220|67], [[1230|62], [[1240|60], [[1250|58], [[1260|55], [[1270|49],
[[1280|46], [[1290|45], [[1300|42], [[1310|40], [[1320|39], [[1330|37], [[1340|35], [[1350|32],
[[1360|31], [[1370|27], [[1380|26], [[1390|26], [[1400|25], [[1410|24], [[1420|24], [[1430|20],
[[1440|17], [[1450|16], [[1460|14], [[1470|12], [[1480|12], [[1490|12], [[1500|12], [[1510|12],
[[1520|11], [[1530|10], [[1540|10], [[1550|10], [[1560|10], [[1570|9], [[1580|9], [[1590|8],
[[1600|8], [[1610|6], [[1620|6], [[1630|5], [[1640|5], [[1650|5], [[1660|5], [[1670|4],
[[1680|4], [[1690|4], [[1700|3], [[1710|3], [[1720|2], [[1730|2], [[1740|2], [[1750|2],
[[1760|2], [[1770|1], [[1780|1], [[1790|1], [[1800|1], [[1810|0], [[1820|0], [[1830|0],
```

```

[[1840|0], [[1850|0], [[1860|0], [[1870|0], [[1880|0], [[1890|0], [[1900|0], [[1910|0],
[[1920|0], [[1930|0], [[1940|0], [[1950|0], [[1960|0], [[1970|0], [[1980|0], [[1990|0],
[[2000|0], [[2010|0], [[2020|0], [[2030|0], [[2040|0], [[2050|0], [[2060|0], [[2070|0],
[[2080|0], [[2090|0], [[2100|0], [[2110|0], [[2120|0], [[2130|0], [[2140|0], [[2150|0],
[[2160|0], ],
//////////
//////////
//////////
[2516|116|0|0|0],
//////////
//////////
//////////
[[0|[[apple], [[3.6|[1.0|0.5|0.3], ], |22.7|10.1|7.0|7.0|25.0|0.51|64.8|122.3|27.0|[], ], ],
[[10|[[apple], [[3.7|[1.0|0.52|0.31], ], |23.6|10.4|7.3|7.3|25.9|0.53|66.7|120.6|28.1|[], ], ],
[[20|[[apple], [[3.8|[1.1|0.53|0.32], ], |24.2|10.7|7.5|7.5|26.6|0.54|67.8|119.5|29.1|[], ], ],
[[30|[[apple], [[3.9|[1.1|0.54|0.33], ], |24.7|10.9|7.6|7.7|27.2|0.55|68.8|118.8|29.8|[], ], ],
[[40|[[apple], [[3.9|[1.1|0.55|0.34], ], |25.0|11.0|7.7|7.8|27.6|0.56|69.5|118.3|30.2|[], ], ],
[[50|[[apple], [[4.0|[1.1|0.56|0.34], ], |25.3|11.1|7.8|7.9|27.9|0.56|70.0|117.9|30.5|[], ], ],
[[60|[[apple], [[4.0|[1.1|0.56|0.34], ], |25.6|11.2|7.9|8.0|28.1|0.57|70.5|117.4|30.7|[], ], ],
[[70|[[apple], [[4.0|[1.1|0.57|0.35], ], |25.7|11.3|7.9|8.0|28.4|0.58|71.0|117.0|31.0|[], ], ],
[[80|[[apple], [[4.0|[1.1|0.57|0.35], ], |25.9|11.4|8.0|8.1|28.6|0.58|71.3|116.7|31.1|[], ], ],
[[90|[[apple], [[4.1|[1.1|0.57|0.35], ], |26.1|11.4|8.0|8.1|28.8|0.58|71.7|116.5|31.2|[], ], ],
[[100|[[apple], [[4.1|[1.2|0.58|0.35], ], |26.2|11.5|8.1|8.2|29.0|0.59|71.8|116.3|31.4|[], ], ],
[[110|[[apple], [[4.1|[1.2|0.58|0.35], ], |26.4|11.5|8.1|8.2|29.1|0.59|72.0|116.2|31.6|[], ], ],
[[120|[[apple], [[4.1|[1.2|0.58|0.35], ], |26.5|11.6|8.2|8.3|29.3|0.59|72.3|115.9|31.8|[], ], ],
[[130|[[apple], [[4.1|[1.2|0.59|0.36], ], |26.5|11.6|8.2|8.3|29.4|0.6|72.4|115.8|31.9|[], ], ],
[[140|[[apple], [[4.1|[1.2|0.59|0.36], ], |26.7|11.7|8.2|8.3|29.5|0.6|72.6|115.7|31.9|[], ], ],
[[150|[[apple], [[4.2|[1.2|0.59|0.36], ], |26.7|11.7|8.3|8.3|29.5|0.6|72.6|115.6|32.0|[], ], ],
[[160|[[apple], [[4.2|[1.2|0.59|0.36], ], |26.9|11.8|8.3|8.4|29.7|0.6|72.9|115.3|32.3|[], ], ],
[[170|[[apple], [[4.2|[1.2|0.6|0.37], ], |27.2|11.9|8.4|8.5|30.1|0.61|73.3|115.0|32.6|[], ], ],
[[180|[[apple], [[4.3|[1.2|0.61|0.37], ], |27.3|11.9|8.5|8.5|30.3|0.61|73.7|114.8|32.9|[], ], ],
[[190|[[apple], [[4.3|[1.2|0.61|0.37], ], |27.5|12.0|8.5|8.6|30.5|0.62|74.0|114.6|33.1|[], ], ],
[[200|[[apple], [[4.3|[1.2|0.61|0.37], ], |27.7|12.1|8.6|8.7|30.8|0.62|74.2|114.4|33.4|[], ], ],
[[210|[[apple], [[4.3|[1.2|0.62|0.38], ], |28.0|12.2|8.7|8.8|31.1|0.63|74.5|114.1|33.8|[], ], ],
[[220|[[apple], [[4.3|[1.3|0.62|0.38], ], |28.1|12.3|8.7|8.8|31.3|0.63|74.6|114.0|33.8|[], ], ],
[[230|[[apple], [[4.4|[1.3|0.63|0.38], ], |28.4|12.3|8.8|8.9|31.5|0.63|75.0|113.8|34.1|[], ], ],
[[240|[[apple], [[4.4|[1.3|0.63|0.39], ], |28.6|12.4|8.8|8.9|31.8|0.64|75.3|113.6|34.2|[], ], ],
[[250|[[apple], [[4.4|[1.3|0.64|0.39], ], |28.9|12.5|8.9|9.0|32.2|0.64|75.6|113.3|34.6|[], ], ],
[[260|[[apple], [[4.4|[1.3|0.64|0.39], ], |29.0|12.6|9.0|9.0|32.3|0.65|75.8|113.1|34.6|[], ], ],
[[270|[[apple], [[4.5|[1.3|0.65|0.4], ], |29.4|12.7|9.1|9.1|32.7|0.65|76.1|112.9|35.1|[], ], ],
[[280|[[apple], [[4.5|[1.3|0.66|0.4], ], |29.6|12.8|9.2|9.2|33.0|0.66|76.4|112.7|35.4|[], ], ],
[[290|[[apple], [[4.5|[1.3|0.66|0.4], ], |29.9|12.9|9.2|9.2|33.2|0.67|76.6|112.5|35.7|[], ], ],
[[300|[[apple], [[4.5|[1.3|0.66|0.4], ], |30.1|13.0|9.3|9.3|33.5|0.68|77.0|112.2|36.0|[], ], ],
[[310|[[apple], [[4.5|[1.4|0.67|0.41], ], |30.3|13.1|9.4|9.3|33.7|0.68|77.2|112.1|36.3|[], ], ],
[[320|[[apple], [[4.6|[1.4|0.68|0.41], ], |30.8|13.2|9.5|9.5|34.2|0.69|77.5|111.9|36.8|[], ], ],
[[330|[[apple], [[4.6|[1.4|0.68|0.41], ], |31.0|13.3|9.6|9.5|34.5|0.7|77.9|111.6|37.0|[], ], ],
[[340|[[apple], [[4.6|[1.4|0.69|0.42], ], |31.3|13.5|9.6|9.6|34.9|0.7|78.1|111.4|37.3|[], ], ],
[[350|[[apple], [[4.7|[1.4|0.7|0.43], ], |31.9|13.7|9.8|9.8|35.5|0.71|78.5|111.1|37.8|[], ], ],
[[360|[[apple], [[4.7|[1.4|0.71|0.43], ], |32.3|13.8|9.9|9.9|35.9|0.72|78.9|110.8|38.2|[], ], ],
[[370|[[apple], [[4.8|[1.5|0.71|0.44], ], |32.6|13.9|10.0|9.9|36.2|0.73|79.2|110.7|38.6|[], ], ],
[[380|[[apple], [[4.8|[1.5|0.73|0.44], ], |33.0|14.2|10.1|10.1|36.8|0.74|79.6|110.4|39.1|[], ], ],

```

[390 [apple], 4.9 [1.5 0.74 0.45], , 33.5 14.4 10.2 10.3 37.4 0.75 80.2 110.1 39.5 [], , ,
[400 [apple], 4.9 [1.5 0.75 0.46], , 34.0 14.5 10.4 10.4 37.9 0.76 80.6 109.8 39.9 [], , ,
[410 [apple], 5.0 [1.5 0.75 0.46], , 34.4 14.7 10.5 10.5 38.3 0.76 81.1 109.4 40.1 [], , ,
[420 [apple], 5.0 [1.6 0.76 0.47], , 34.8 14.8 10.7 10.6 38.8 0.77 81.4 109.1 40.6 [], , ,
[430 [apple], 5.1 [1.6 0.77 0.47], , 35.2 15.0 10.8 10.7 39.3 0.78 81.8 108.9 41.0 [], , ,
[440 [apple], 5.2 [1.6 0.78 0.48], , 35.6 15.1 10.9 10.8 39.7 0.79 82.1 108.6 41.3 [], , ,
[450 [apple], 5.2 [1.6 0.79 0.48], , 36.1 15.3 11.1 10.9 40.2 0.8 82.6 108.4 41.7 [], , ,
[460 [apple], 5.2 [1.6 0.8 0.49], , 36.5 15.5 11.2 11.1 40.6 0.81 83.0 108.2 42.0 [], , ,
[470 [apple], 5.3 [1.7 0.81 0.5], , 37.0 15.8 11.4 11.3 41.3 0.82 83.4 108.1 42.5 [], , ,
[480 [apple], 5.3 [1.7 0.82 0.5], , 37.6 16.0 11.5 11.4 41.9 0.83 83.8 107.9 43.1 [], , ,
[490 [apple], 5.4 [1.7 0.83 0.51], , 38.1 16.2 11.7 11.6 42.4 0.84 84.2 107.7 43.4 [], , ,
[500 [apple], 5.5 [1.7 0.84 0.52], , 38.6 16.4 11.8 11.7 42.9 0.85 84.6 107.5 43.8 [], , ,
[510 [apple], 5.5 [1.8 0.85 0.52], , 39.0 16.6 11.9 11.9 43.5 0.86 84.8 107.3 44.0 [], , ,
[520 [apple], 5.6 [1.8 0.87 0.53], , 39.6 16.8 12.1 12.1 44.2 0.88 85.3 107.0 44.6 [], , ,
[530 [apple], 5.6 [1.8 0.88 0.54], , 40.3 17.1 12.3 12.3 44.9 0.9 85.7 106.7 45.3 [], , ,
[540 [apple], 5.7 [1.8 0.89 0.54], , 40.6 17.2 12.4 12.4 45.3 0.91 86.0 106.6 45.5 [], , ,
[550 [apple], 5.7 [1.8 0.9 0.55], , 40.9 17.4 12.6 12.5 45.7 0.91 86.2 106.6 45.7 [], , ,
[560 [apple], 5.7 [1.9 0.9 0.55], , 41.2 17.5 12.7 12.6 46.1 0.92 86.5 106.5 46.0 [], , ,
[570 [apple], 5.7 [1.9 0.91 0.55], , 41.4 17.6 12.8 12.6 46.5 0.93 86.7 106.3 45.9 [], , ,
[580 [apple], 5.8 [1.9 0.93 0.56], , 42.0 17.9 13.0 12.8 47.1 0.94 87.2 106.0 46.6 [], , ,
[590 [apple], 5.8 [1.9 0.93 0.57], , 42.2 17.9 13.1 12.9 47.4 0.95 87.4 105.9 46.7 [], , ,
[600 [apple], 5.8 [1.9 0.93 0.57], , 42.3 18.0 13.1 12.9 47.5 0.95 87.4 105.8 46.6 [], , ,
[610 [apple], 5.8 [1.9 0.94 0.57], , 42.6 18.0 13.2 13.0 47.8 0.95 87.5 105.7 46.9 [], , ,
[620 [apple], 5.8 [1.9 0.95 0.57], , 42.9 18.2 13.3 13.1 48.3 0.96 87.7 105.4 47.3 [], , ,
[630 [apple], 5.8 [1.9 0.95 0.58], , 43.0 18.2 13.3 13.2 48.4 0.96 87.9 105.3 47.3 [], , ,
[640 [apple], 5.8 [2.0 0.96 0.58], , 43.3 18.3 13.4 13.3 48.8 0.97 87.9 105.3 47.4 [], , ,
[650 [apple], 5.8 [2.0 0.97 0.59], , 43.8 18.5 13.6 13.4 49.3 0.99 88.1 105.1 47.7 [], , ,
[660 [apple], 5.8 [2.0 0.98 0.59], , 44.1 18.6 13.7 13.5 49.6 1.0 88.2 105.1 47.9 [], , ,
[670 [apple], 5.8 [2.0 0.99 0.59], , 44.2 18.7 13.8 13.6 49.8 1.0 88.2 105.2 47.8 [], , ,
[680 [apple], 5.9 [2.0 1.0 0.6], , 44.7 18.9 13.9 13.7 50.4 1.0 88.3 105.2 48.1 [], , ,
[690 [apple], 5.9 [2.0 1.0 0.61], , 45.0 19.0 14.1 13.8 50.8 1.0 88.4 105.2 48.4 [], , ,
[700 [apple], 5.8 [2.0 1.0 0.61], , 45.2 19.1 14.1 13.9 50.9 1.0 88.4 105.3 48.4 [], , ,
[710 [apple], 5.8 [2.0 1.0 0.61], , 45.1 19.1 14.1 13.9 51.0 1.0 88.6 105.2 48.4 [], , ,
[720 [apple], 5.8 [2.0 1.0 0.61], , 45.2 19.1 14.2 13.9 51.1 1.0 88.6 105.4 48.5 [], , ,
[730 [apple], 5.8 [2.1 1.0 0.61], , 45.6 19.2 14.3 14.0 51.5 1.0 88.7 105.5 48.9 [], , ,
[740 [apple], 5.7 [2.1 1.0 0.62], , 45.7 19.3 14.3 14.1 51.6 1.0 88.6 105.6 49.0 [], , ,
[750 [apple], 5.7 [2.1 1.0 0.62], , 45.5 19.2 14.3 14.0 51.5 1.0 88.4 105.8 48.8 [], , ,
[760 [apple], 5.6 [2.1 1.0 0.62], , 45.6 19.3 14.3 14.0 51.7 1.0 88.2 106.0 48.9 [], , ,
[770 [apple], 5.6 [2.1 1.0 0.62], , 46.1 19.4 14.4 14.1 52.2 1.0 88.6 106.0 49.1 [], , ,
[780 [apple], 5.6 [2.1 1.0 0.63], , 46.3 19.5 14.5 14.1 52.4 1.1 88.5 106.1 49.1 [], , ,
[790 [apple], 5.6 [2.1 1.0 0.63], , 46.4 19.6 14.6 14.2 52.6 1.1 88.7 106.0 49.1 [], , ,
[800 [apple], 5.5 [2.1 1.0 0.64], , 47.0 19.8 14.7 14.3 53.1 1.1 88.7 106.2 49.7 [], , ,
[810 [apple], 5.5 [2.1 1.0 0.64], , 47.1 19.9 14.7 14.4 53.2 1.1 88.8 106.3 49.9 [], , ,
[820 [apple], 5.5 [2.2 1.0 0.64], , 47.2 19.9 14.8 14.4 53.4 1.1 88.9 106.1 49.9 [], , ,
[830 [apple], 5.4 [2.2 1.0 0.64], , 47.2 19.9 14.8 14.4 53.4 1.1 89.0 106.0 49.6 [], , ,
[840 [apple], 5.4 [2.2 1.1 0.64], , 47.3 20.0 14.8 14.4 53.4 1.1 89.0 106.0 49.6 [], , ,
[850 [apple], 5.3 [2.1 1.0 0.64], , 46.9 19.8 14.6 14.3 53.0 1.1 88.8 106.1 48.9 [], , ,
[860 [apple], 5.2 [2.1 1.0 0.63], , 46.7 19.7 14.5 14.2 52.9 1.1 88.9 106.1 48.5 [], , ,
[870 [apple], 5.2 [2.1 1.0 0.63], , 46.6 19.6 14.5 14.2 52.8 1.1 88.7 106.2 48.3 [], , ,
[880 [apple], 5.1 [2.1 1.0 0.63], , 46.5 19.5 14.5 14.1 52.7 1.1 88.8 106.4 48.0 [], , ,
[890 [apple], 5.1 [2.1 1.0 0.63], , 46.6 19.6 14.5 14.1 52.8 1.1 88.9 106.4 47.8 [], , ,

```

|[900|[[apple], |[5.1|[2.1|1.0|0.63], ], |46.6|19.6|14.5|14.2|52.8|1.1|89.0|106.4|47.7|[], ], ],
|[910|[[apple], |[5.0|[2.1|1.0|0.62], ], |46.2|19.3|14.3|14.1|52.3|1.0|88.8|106.8|47.4|[], ], ],
|[920|[[apple], |[4.9|[2.1|1.0|0.61], ], |45.7|19.1|14.2|13.9|51.7|1.0|88.6|106.8|47.1|[], ], ],
|[930|[[apple], |[4.9|[2.1|1.0|0.61], ], |45.6|19.1|14.2|13.9|51.6|1.0|88.5|106.6|46.9|[], ], ],
|[940|[[apple], |[4.8|[2.1|1.0|0.61], ], |45.6|19.1|14.1|13.8|51.5|1.0|88.4|106.7|46.5|[], ], ],
|[950|[[apple], |[4.8|[2.1|1.0|0.6], ], |45.2|18.9|14.0|13.7|51.1|1.0|88.3|106.7|46.3|[], ], ],
|[960|[[apple], |[4.7|[2.1|1.0|0.6], ], |45.2|18.9|14.0|13.7|51.0|1.0|88.2|106.6|45.8|[], ], ],
|[970|[[apple], |[4.7|[2.1|0.99|0.6], ], |45.1|18.9|14.1|13.7|50.9|1.0|88.4|106.4|45.4|[], ], ],
|[980|[[apple], |[4.7|[2.1|0.99|0.6], ], |45.1|18.8|14.0|13.7|50.9|1.0|88.5|106.2|45.2|[], ], ],
|[990|[[apple], |[4.6|[2.1|0.98|0.6], ], |44.8|18.8|13.9|13.6|50.6|1.0|88.4|106.6|44.8|[], ], ],
|[1000|[[apple], |[4.6|[2.1|1.0|0.61], ], |45.5|19.0|14.1|13.8|51.3|1.0|88.5|106.6|45.0|[], ], ],
|[1010|[[apple], |[5.6|[2.5|1.2|0.72], ], |54.1|22.6|16.8|16.5|61.2|1.2|96.6|99.7|51.9|[], ], ],
|[1020|[[apple], |[5.5|[2.5|1.2|0.72], ], |54.0|22.5|16.7|16.4|60.9|1.2|96.6|99.7|51.5|[], ], ],
|[1030|[[apple], |[5.5|[2.5|1.2|0.72], ], |53.7|22.3|16.6|16.4|60.5|1.2|96.6|99.7|50.7|[], ], ],
|[1040|[[apple], |[5.4|[2.5|1.2|0.71], ], |53.6|22.3|16.5|16.3|60.3|1.2|96.5|99.7|50.5|[], ], ],
|[1050|[[apple], |[5.4|[2.5|1.2|0.71], ], |53.8|22.3|16.5|16.4|60.3|1.2|96.5|99.7|50.1|[], ], ],
|[1060|[[apple], |[5.4|[2.5|1.2|0.71], ], |53.4|22.2|16.4|16.3|59.9|1.2|96.5|99.7|50.1|[], ], ],
|[1070|[[apple], |[5.4|[2.5|1.2|0.71], ], |53.3|22.1|16.4|16.4|60.0|1.2|96.7|99.7|50.0|[], ], ],
|[1080|[[apple], |[5.5|[2.5|1.2|0.72], ], |53.9|22.4|16.6|16.6|60.7|1.2|96.5|99.7|51.0|[], ], ],
|[1090|[[apple], |[5.4|[2.5|1.2|0.71], ], |53.3|22.0|16.3|16.5|60.0|1.2|96.3|99.7|50.2|[], ], ],
|[1100|[[apple], |[5.3|[2.5|1.1|0.71], ], |53.0|21.8|16.2|16.3|59.7|1.2|96.2|99.7|49.2|[], ], ],
|[1110|[[apple], |[5.3|[2.5|1.1|0.71], ], |53.0|21.7|16.1|16.3|59.5|1.2|96.3|99.7|48.3|[], ], ],
|[1120|[[apple], |[5.3|[2.4|1.1|0.71], ], |52.9|21.6|16.1|16.2|59.3|1.1|96.3|99.8|47.7|[], ], ],
|[1130|[[apple], |[5.3|[2.4|1.1|0.71], ], |53.0|21.5|16.2|16.2|59.2|1.1|96.4|99.7|47.3|[], ], ],
|[1140|[[apple], |[5.3|[2.4|1.1|0.71], ], |52.9|21.7|16.3|16.2|59.3|1.1|96.6|99.7|46.6|[], ], ],
|[1150|[[apple], |[5.2|[2.4|1.1|0.71], ], |53.0|21.6|16.2|16.1|58.9|1.1|96.5|99.7|45.5|[], ], ],
|[1160|[[apple], |[5.2|[2.4|1.1|0.71], ], |52.9|21.6|16.2|16.1|58.9|1.1|96.5|99.7|45.4|[], ], ],
|[1170|[[apple], |[5.2|[2.4|1.1|0.71], ], |52.5|21.3|16.0|16.0|58.4|1.1|96.5|99.7|44.7|[], ], ],
|[1180|[[apple], |[5.2|[2.4|1.1|0.71], ], |52.3|21.1|15.8|15.8|57.9|1.1|96.5|99.6|44.6|[], ], ],
|[1190|[[apple], |[5.2|[2.4|1.1|0.71], ], |52.4|21.2|15.9|16.1|58.2|1.1|96.3|99.6|45.8|[], ], ],
|[1200|[[apple], |[5.2|[2.4|1.1|0.7], ], |51.8|20.9|15.6|15.8|57.4|1.1|96.3|99.6|44.0|[], ], ],
|[1210|[[apple], |[5.3|[2.4|1.1|0.7], ], |52.4|20.9|15.8|15.9|57.8|1.1|96.0|99.5|44.8|[], ], ],
|[1220|[[apple], |[5.3|[2.4|1.1|0.71], ], |52.8|21.0|16.1|16.1|58.3|1.1|96.1|99.5|45.3|[], ], ],
|[1230|[[apple], |[5.2|[2.4|1.1|0.7], ], |51.7|20.4|15.5|15.8|57.3|1.1|96.0|99.4|45.4|[], ], ],
|[1240|[[apple], |[5.1|[2.4|1.1|0.7], ], |51.0|20.2|15.3|15.7|56.6|1.1|95.9|99.4|44.9|[], ], ],
|[1250|[[apple], |[5.1|[2.4|1.1|0.69], ], |50.5|20.2|15.2|15.6|56.1|1.1|95.7|99.4|44.7|[], ], ],
|[1260|[[apple], |[5.0|[2.4|1.1|0.69], ], |50.4|20.2|15.2|15.6|55.9|1.1|95.9|99.3|44.6|[], ], ],
|[1270|[[apple], |[4.8|[2.3|1.0|0.64], ], |47.4|19.1|14.4|14.7|52.7|1.0|95.4|99.3|44.1|[], ], ],
|[1280|[[apple], |[4.7|[2.2|1.0|0.64], ], |47.1|18.9|14.2|14.5|52.3|1.0|95.1|99.2|43.2|[], ], ],
|[1290|[[apple], |[4.7|[2.2|1.0|0.64], ], |47.2|18.9|14.2|14.6|52.5|1.0|95.1|99.2|43.5|[], ], ],
|[1300|[[apple], |[4.6|[2.2|0.99|0.63], ], |46.5|18.7|14.0|14.4|51.6|1.0|94.8|99.1|42.9|[], ], ],
|[1310|[[apple], |[4.3|[2.1|0.93|0.61], ], |43.8|17.7|13.2|13.8|49.2|1.0|94.5|99.1|42.1|[], ], ],
|[1320|[[apple], |[4.3|[2.1|0.92|0.6], ], |43.1|17.5|13.0|13.7|48.6|1.0|94.4|99.1|41.4|[], ], ],
|[1330|[[apple], |[4.2|[2.0|0.91|0.59], ], |42.9|17.3|13.0|13.6|48.2|1.0|94.1|99.0|41.9|[], ], ],
|[1340|[[apple], |[4.0|[1.9|0.88|0.56], ], |41.2|16.8|12.6|13.1|46.7|0.98|93.8|99.0|40.0|[], ], ],
|[1350|[[apple], |[3.9|[1.9|0.86|0.54], ], |40.9|16.4|12.3|12.8|45.5|0.94|93.2|98.9|40.0|[], ], ],
|[1360|[[apple], |[3.9|[1.9|0.86|0.54], ], |41.1|16.5|12.3|12.9|45.8|0.93|93.0|98.8|40.3|[], ], ],
|[1370|[[apple], |[3.9|[1.9|0.83|0.54], ], |39.5|15.9|11.8|12.6|44.5|0.89|92.9|98.8|38.2|[], ], ],
|[1380|[[apple], |[3.8|[1.8|0.82|0.53], ], |38.8|15.8|11.7|12.4|43.8|0.9|92.6|98.8|37.9|[], ], ],
|[1390|[[apple], |[3.8|[1.8|0.82|0.53], ], |38.8|15.8|11.7|12.4|43.8|0.9|92.6|98.8|37.9|[], ], ],
|[1400|[[apple], |[3.7|[1.8|0.82|0.53], ], |38.5|15.6|11.7|12.5|43.7|0.87|93.2|98.7|37.5|[], ], ],

```



```

|[1410|[[apple], |[3.7|[1.8|0.82|0.53], ], |39.1|15.8|11.8|12.6|44.1|0.87|94.0|99.4|38.1|[], ], ],
|[1420|[[apple], |[3.7|[1.8|0.82|0.53], ], |39.1|15.8|11.8|12.6|44.1|0.87|94.0|99.4|38.1|[], ], ],
|[1430|[[apple], |[3.7|[1.8|0.82|0.56], ], |40.0|15.8|12.0|13.0|44.8|0.87|92.8|99.3|40.8|[], ], ],
|[1440|[[apple], |[3.6|[1.7|0.8|0.56], ], |39.5|16.0|11.5|12.6|44.1|0.89|95.1|99.1|38.5|[], ], ],
|[1450|[[apple], |[3.3|[1.7|0.77|0.53], ], |38.8|15.2|11.1|12.3|42.9|0.87|94.8|99.1|38.2|[], ], ],
|[1460|[[apple], |[3.4|[1.8|0.77|0.54], ], |39.5|15.4|11.3|12.5|43.4|0.88|94.1|98.9|40.7|[], ], ],
|[1470|[[apple], |[3.4|[1.8|0.78|0.54], ], |39.9|15.7|11.2|12.7|43.3|0.83|96.0|100.0|37.3|[], ], ],
|[1480|[[apple], |[3.4|[1.8|0.78|0.54], ], |39.9|15.7|11.2|12.7|43.3|0.83|96.0|100.0|37.3|[], ], ],
|[1490|[[apple], |[3.4|[1.8|0.78|0.54], ], |39.9|15.7|11.2|12.7|43.3|0.83|96.0|100.0|37.3|[], ], ],
|[1500|[[apple], |[3.4|[1.8|0.78|0.54], ], |39.9|15.7|11.2|12.7|43.3|0.83|96.0|100.0|37.3|[], ], ],
|[1510|[[apple], |[3.4|[1.8|0.78|0.54], ], |39.9|15.7|11.2|12.7|43.3|0.83|96.0|100.0|37.3|[], ], ],
|[1520|[[apple], |[3.3|[1.7|0.78|0.54], ], |39.7|15.8|11.2|12.7|42.9|0.85|95.6|100.0|38.0|[], ], ],
|[1530|[[apple], |[3.2|[1.7|0.77|0.54], ], |39.7|15.8|11.1|12.5|42.2|0.84|95.2|100.0|34.2|[], ], ],
|[1540|[[apple], |[3.2|[1.7|0.77|0.54], ], |39.7|15.8|11.1|12.5|42.2|0.84|95.2|100.0|34.2|[], ], ],
|[1550|[[apple], |[3.2|[1.7|0.77|0.54], ], |39.7|15.8|11.1|12.5|42.2|0.84|95.2|100.0|34.2|[], ], ],
|[1560|[[apple], |[3.2|[1.7|0.77|0.54], ], |39.7|15.8|11.1|12.5|42.2|0.84|95.2|100.0|34.2|[], ], ],
|[1570|[[apple], |[3.2|[1.7|0.74|0.53], ], |38.8|15.7|11.3|12.5|41.4|0.83|94.6|100.0|35.1|[], ], ],
|[1580|[[apple], |[3.2|[1.7|0.74|0.53], ], |38.8|15.7|11.3|12.5|41.4|0.83|94.6|100.0|35.1|[], ], ],
|[1590|[[apple], |[3.4|[1.6|0.73|0.54], ], |39.0|16.6|11.4|12.8|41.6|0.84|94.0|100.0|36.4|[], ], ],
|[1600|[[apple], |[3.4|[1.6|0.73|0.54], ], |39.0|16.6|11.4|12.8|41.6|0.84|94.0|100.0|36.4|[], ], ],
|[1610|[[apple], |[3.3|[1.8|0.76|0.54], ], |40.6|17.3|11.5|12.7|42.1|0.82|97.7|100.0|38.5|[], ], ],
|[1620|[[apple], |[3.3|[1.8|0.76|0.54], ], |40.6|17.3|11.5|12.7|42.1|0.82|97.7|100.0|38.5|[], ], ],
|[1630|[[apple], |[3.3|[1.8|0.79|0.53], ], |42.4|18.2|12.6|13.5|42.9|0.81|97.3|100.0|44.0|[], ], ],
|[1640|[[apple], |[3.3|[1.8|0.79|0.53], ], |42.4|18.2|12.6|13.5|42.9|0.81|97.3|100.0|44.0|[], ], ],
|[1650|[[apple], |[3.3|[1.8|0.79|0.53], ], |42.4|18.2|12.6|13.5|42.9|0.81|97.3|100.0|44.0|[], ], ],
|[1660|[[apple], |[3.3|[1.8|0.79|0.53], ], |42.4|18.2|12.6|13.5|42.9|0.81|97.3|100.0|44.0|[], ], ],
|[1670|[[apple], |[3.2|[1.8|0.77|0.52], ], |38.7|16.3|11.9|13.6|41.1|0.84|96.6|100.0|39.8|[], ], ],
|[1680|[[apple], |[3.2|[1.8|0.77|0.52], ], |38.7|16.3|11.9|13.6|41.1|0.84|96.6|100.0|39.8|[], ], ],
|[1690|[[apple], |[3.2|[1.8|0.77|0.52], ], |38.7|16.3|11.9|13.6|41.1|0.84|96.6|100.0|39.8|[], ], ],
|[1700|[[apple], |[3.0|[2.0|0.72|0.55], ], |39.7|15.1|11.9|13.8|41.6|0.8|95.5|100.0|46.6|[], ], ],
|[1710|[[apple], |[3.0|[2.0|0.72|0.55], ], |39.7|15.1|11.9|13.8|41.6|0.8|95.5|100.0|46.6|[], ], ],
|[1720|[[apple], |[3.5|[2.5|0.74|0.56], ], |41.4|14.8|12.0|14.2|46.5|0.83|100.0|100.0|51.5|[], ], ],
|[1730|[[apple], |[3.5|[2.5|0.74|0.56], ], |41.4|14.8|12.0|14.2|46.5|0.83|100.0|100.0|51.5|[], ], ],
|[1740|[[apple], |[3.5|[2.5|0.74|0.56], ], |41.4|14.8|12.0|14.2|46.5|0.83|100.0|100.0|51.5|[], ], ],
|[1750|[[apple], |[3.5|[2.5|0.74|0.56], ], |41.4|14.8|12.0|14.2|46.5|0.83|100.0|100.0|51.5|[], ], ],
|[1760|[[apple], |[3.5|[2.5|0.74|0.56], ], |41.4|14.8|12.0|14.2|46.5|0.83|100.0|100.0|51.5|[], ], ],
|[1770|[[apple], |[1.9|[2.0|0.65|0.42], ], |35.2|8.1|11.6|14.4|41.2|1.2|100.0|100.0|56.6|[], ], ],
|[1780|[[apple], |[1.9|[2.0|0.65|0.42], ], |35.2|8.1|11.6|14.4|41.2|1.2|100.0|100.0|56.6|[], ], ],
|[1790|[[apple], |[1.9|[2.0|0.65|0.42], ], |35.2|8.1|11.6|14.4|41.2|1.2|100.0|100.0|56.6|[], ], ],
|[1800|[[apple], |[1.9|[2.0|0.65|0.42], ], |35.2|8.1|11.6|14.4|41.2|1.2|100.0|100.0|56.6|[], ], ],
|[1810|[], ], |1820|[], ], |1830|[], ], |1840|[], ], |1850|[], ], |1860|[], ], |1870|[], ], |1880|[], ], |1890|[], ], |1900|[], ], |1910|[], ], |1920|[], ], |1930|[], ], |1940|[], ], |1950|[], ], |1960|[], ], |1970|[], ], |1980|[], ], |1990|[], ], |2000|[], ], |2010|[], ], |2020|[], ], |2030|[], ], |2040|[], ], |2050|[], ], |2060|[], ], |2070|[], ], |2080|[], ], |2090|[], ], |2100|[], ], |2110|[], ], |2120|[], ], |2130|[], ], |2140|[], ], |2150|[], ], |2160|[], ], ],

```

```

////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////
[[0|N/A], |[10|N/A], |[20|N/A], |[30|N/A], |[40|N/A], |[50|N/A], |[60|N/A], |[70|N/A],
|[80|N/A], |[90|N/A], |[100|N/A], |[110|N/A], |[120|N/A], |[130|N/A], |[140|N/A], |[150|N/A],
|[160|N/A], |[170|N/A], |[180|148.0], |[190|148.0], |[200|154.5], |[210|154.5], |[220|185.5],
|[230|185.5], |[240|185.5], |[250|201.0], |[260|219.6], |[270|218.9], |[280|222.4],
|[290|241.4], |[300|250.5], |[310|262.2], |[320|264.8], |[330|271.7], |[340|275.0],
|[350|283.0], |[360|283.2], |[370|280.9], |[380|289.4], |[390|317.3], |[400|316.4],
|[410|299.4], |[420|297.8], |[430|279.9], |[440|291.2], |[450|304.4], |[460|299.4],
|[470|317.4], |[480|343.6], |[490|324.9], |[500|338.6], |[510|322.9], |[520|307.3],
|[530|323.5], |[540|298.5], |[550|289.9], |[560|290.8], |[570|287.0], |[580|291.1],
|[590|277.6], |[600|278.1], |[610|269.9], |[620|271.3], |[630|268.9], |[640|275.1],
|[650|280.8], |[660|274.7], |[670|263.4], |[680|262.4], |[690|256.7], |[700|249.3],
|[710|254.3], |[720|249.2], |[730|263.1], |[740|270.6], |[750|252.9], |[760|251.2],
|[770|262.2], |[780|243.5], |[790|242.7], |[800|242.3], |[810|258.2], |[820|253.2],
|[830|270.8], |[840|268.5], |[850|258.2], |[860|280.9], |[870|282.5], |[880|286.0],
|[890|299.2], |[900|313.2], |[910|292.6], |[920|275.3], |[930|282.0], |[940|293.0],
|[950|303.9], |[960|308.3], |[970|321.6], |[980|295.2], |[990|306.2], |[1000|319.8],
|[1010|657.2], |[1020|575.0], |[1030|479.4], |[1040|429.3], |[1050|419.9], |[1060|378.3],
|[1070|357.7], |[1080|363.3], |[1090|348.5], |[1100|358.8], |[1110|350.6], |[1120|332.9],
|[1130|336.3], |[1140|338.6], |[1150|335.9], |[1160|333.3], |[1170|338.0], |[1180|329.6],
|[1190|356.9], |[1200|368.3], |[1210|365.5], |[1220|402.7], |[1230|401.8], |[1240|390.0],
|[1250|385.1], |[1260|411.7], |[1270|384.3], |[1280|366.6], |[1290|377.7], |[1300|383.0],
|[1310|389.9], |[1320|363.9], |[1330|412.8], |[1340|409.7], |[1350|422.7], |[1360|461.5],
|[1370|490.4], |[1380|495.7], |[1390|521.7], |[1400|546.6], |[1410|569.3], |[1420|582.5],
|[1430|655.1], |[1440|678.1], |[1450|660.3], |[1460|681.6], |[1470|711.9], |[1480|732.2],
|[1490|733.5], |[1500|727.7], |[1510|709.5], |[1520|724.6], |[1530|746.4], |[1540|746.4],
|[1550|775.9], |[1560|779.0], |[1570|783.6], |[1580|783.6], |[1590|787.6], |[1600|787.6],
|[1610|798.6], |[1620|798.6], |[1630|813.2], |[1640|807.3], |[1650|807.3], |[1660|807.3],
|[1670|808.5], |[1680|808.5], |[1690|813.4], |[1700|819.7], |[1710|819.7], |[1720|831.0],
|[1730|854.0], |[1740|854.0], |[1750|854.0], |[1760|854.0], |[1770|857.7], |[1780|857.7],
|[1790|857.7], |[1800|851.1], |[1810|861.1], |[1820|854.9], |[1830|854.9], |[1840|854.9],
|[1850|843.7], |[1860|843.7], |[1870|843.7], |[1880|843.7], |[1890|843.7], |[1900|843.7],
|[1910|843.7], |[1920|843.7], |[1930|843.7], |[1940|843.7], |[1950|843.7], |[1960|843.7],
|[1970|843.7], |[1980|843.7], |[1990|843.7], |[2000|843.7], |[2010|843.7], |[2020|843.7],
|[2030|843.7], |[2040|843.7], |[2050|843.7], |[2060|843.7], |[2070|843.7], |[2080|843.7],
|[2090|843.7], |[2100|843.7], |[2110|843.7], |[2120|843.7], |[2130|843.7], |[2140|843.7],
|[2150|843.7], |[2160|843.7], ],
////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////
[[0|N/A], |[10|N/A], |[20|N/A], |[30|N/A], |[40|N/A], |[50|N/A], |[60|N/A], |[70|N/A],
|[80|N/A], |[90|N/A], |[100|N/A], |[110|N/A], |[120|N/A], |[130|N/A], |[140|N/A], |[150|N/A],
|[160|N/A], |[170|N/A], |[180|N/A], |[190|N/A], |[200|[144.8|154.5|164.2], ],
|[210|[144.8|154.5|164.2], ], |[220|[157.8|194.5|205.8], ], |[230|[157.8|194.5|205.8], ],
|[240|[157.8|194.5|205.8], ], |[250|[183.5|202.0|219.8], ], |[260|[191.0|210.0|247.0], ],
|[270|[200.0|207.0|247.0], ], |[280|[202.5|209.5|249.0], ], |[290|[207.0|246.0|274.5], ],
|[300|[209.5|267.0|285.5], ], |[310|[236.2|280.0|287.8], ], |[320|[231.5|284.5|295.5], ],
|[330|[236.2|288.0|303.2], ], |[340|[244.5|287.0|315.5], ], |[350|[246.0|290.0|322.0], ],
|[360|[245.5|289.5|322.5], ], |[370|[236.0|290.0|342.0], ], |[380|[232.2|302.0|357.2], ],
|[390|[271.2|348.5|376.0], ], |[400|[242.0|355.0|381.5], ], |[410|[226.0|308.0|377.5], ],

```

```

|[420|[217.0|308.0|386.0], ], |[430|[195.2|275.5|375.5], ], |[440|[197.5|282.0|379.0], ],
|[450|[224.0|300.0|396.0], ], |[460|[211.8|270.5|398.5], ], |[470|[219.0|300.0|448.0], ],
|[480|[229.0|368.0|454.0], ], |[490|[196.5|324.5|465.5], ], |[500|[225.0|340.0|473.0], ],
|[510|[204.0|301.0|473.5], ], |[520|[177.2|273.0|465.8], ], |[530|[177.2|294.5|484.0], ],
|[540|[167.0|253.0|449.0], ], |[550|[193.0|247.0|369.8], ], |[560|[189.8|241.5|376.8], ],
|[570|[189.2|234.5|349.8], ], |[580|[187.0|226.0|349.5], ], |[590|[160.5|216.0|347.0], ],
|[600|[155.5|216.0|369.0], ], |[610|[145.0|211.0|334.0], ], |[620|[144.8|210.5|331.8], ],
|[630|[134.0|211.0|331.0], ], |[640|[142.5|215.5|360.8], ], |[650|[138.5|218.0|370.5], ],
|[660|[135.5|222.5|348.5], ], |[670|[134.0|210.0|316.0], ], |[680|[138.2|213.5|314.2], ],
|[690|[139.0|201.0|302.0], ], |[700|[138.0|199.0|301.0], ], |[710|[134.8|196.5|313.2], ],
|[720|[134.0|188.0|301.5], ], |[730|[141.5|198.0|303.5], ], |[740|[145.2|209.5|308.8], ],
|[750|[143.2|204.0|295.0], ], |[760|[142.0|203.0|299.0], ], |[770|[141.0|206.0|305.0], ],
|[780|[139.5|200.0|289.0], ], |[790|[133.0|193.0|282.0], ], |[800|[138.5|195.5|280.8], ],
|[810|[140.0|200.5|289.0], ], |[820|[133.2|192.0|289.0], ], |[830|[140.5|195.0|297.5], ],
|[840|[140.0|205.0|297.5], ], |[850|[140.0|200.0|299.0], ], |[860|[147.0|202.0|331.0], ],
|[870|[147.8|209.5|342.2], ], |[880|[147.5|209.0|347.0], ], |[890|[150.0|220.0|361.0], ],
|[900|[151.2|222.5|373.8], ], |[910|[144.0|211.0|358.0], ], |[920|[150.0|213.0|296.0], ],
|[930|[151.0|214.0|292.0], ], |[940|[155.5|216.0|304.2], ], |[950|[166.5|217.5|324.0], ],
|[960|[166.0|217.0|327.0], ], |[970|[172.0|228.5|324.0], ], |[980|[169.0|225.0|312.5], ],
|[990|[170.8|216.0|344.5], ], |[1000|[172.2|218.0|369.0], ], |[1010|[236.0|992.0|1.0k], ],
|[1020|[203.5|414.0|1.0k], ], |[1030|[198.5|306.0|1.0k], ], |[1040|[195.0|276.0|544.5], ],
|[1050|[192.0|284.0|494.0], ], |[1060|[188.5|283.0|443.5], ], |[1070|[187.2|278.0|424.0], ],
|[1080|[216.0|298.0|416.0], ], |[1090|[206.5|284.0|417.5], ], |[1100|[209.0|295.0|442.0], ],
|[1110|[208.5|300.0|443.5], ], |[1120|[198.0|289.0|419.0], ], |[1130|[192.0|291.0|428.5], ],
|[1140|[188.0|292.0|422.5], ], |[1150|[187.0|300.0|408.0], ], |[1160|[178.8|301.0|397.5], ],
|[1170|[171.5|289.5|446.0], ], |[1180|[171.5|285.5|448.0], ], |[1190|[197.0|300.0|523.0], ],
|[1200|[208.5|316.5|532.8], ], |[1210|[215.5|317.0|523.0], ], |[1220|[219.0|335.0|604.0], ],
|[1230|[211.5|296.0|607.0], ], |[1240|[201.8|283.0|586.5], ], |[1250|[201.8|283.0|533.8], ],
|[1260|[212.0|335.0|610.0], ], |[1270|[201.5|289.0|532.5], ], |[1280|[200.8|279.5|441.2], ],
|[1290|[202.2|279.5|513.8], ], |[1300|[200.0|317.5|513.8], ], |[1310|[191.0|309.0|546.0], ],
|[1320|[187.0|260.0|400.0], ], |[1330|[205.5|309.0|593.0], ], |[1340|[200.5|299.0|607.8], ],
|[1350|[205.2|317.5|602.2], ], |[1360|[200.5|360.0|672.5], ], |[1370|[205.2|477.5|696.8], ],
|[1380|[223.5|521.0|694.5], ], |[1390|[260.0|597.0|699.0], ], |[1400|[289.0|603.0|699.0], ],
|[1410|[311.8|603.5|752.2], ], |[1420|[281.8|629.5|794.8], ], |[1430|[597.0|690.0|826.0], ],
|[1440|[598.5|694.5|871.0], ], |[1450|[515.5|690.0|856.0], ], |[866.5|952.0|1.0k], ],
|[1990|[866.5|952.0|1.0k], ], |[2000|[866.5|952.0|1.0k], ], |[2010|[866.5|952.0|1.0k], ],
|[2020|[866.5|952.0|1.0k], ], |[2030|[866.5|952.0|1.0k], ], |[2040|[866.5|952.0|1.0k], ],
|[2050|[866.5|952.0|1.0k], ], |[2060|[866.5|952.0|1.0k], ], |[2070|[866.5|952.0|1.0k], ],
|[2080|[866.5|952.0|1.0k], ], |[2090|[866.5|952.0|1.0k], ], |[2100|[866.5|952.0|1.0k], ],
|[2110|[866.5|952.0|1.0k], ], |[2120|[866.5|952.0|1.0k], ], |[2130|[866.5|952.0|1.0k], ],
|[2140|[866.5|952.0|1.0k], ], |[2150|[866.5|952.0|1.0k], ], |[2160|[866.5|952.0|1.0k], ], ],
////////
////////
////////
[[0|[0.0|0.0|0.0], ], |[10|[10.0|10.0|10.0], ], |[20|[20.0|20.0|20.0], ], |[30|[30.0|30.0|30.0], ],
|[40|[40.0|40.0|40.0], ], |[50|[50.0|50.0|50.0], ], |[60|[60.0|60.0|60.0], ],
|[70|[70.0|70.0|70.0], ], |[80|[80.0|80.0|80.0], ], |[90|[90.0|90.0|90.0], ],
|[100|[100.0|100.0|100.0], ], |[110|[108.0|110.0|110.0], ], |[120|[116.0|120.0|120.0], ],
|[130|[125.2|130.0|130.0], ], |[140|[134.8|140.0|140.0], ], |[150|[143.0|150.0|150.0], ],
|[160|[152.0|160.0|160.0], ], |[170|[160.0|170.0|170.0], ], |[180|[166.0|180.0|180.0], ],
|[190|[174.0|190.0|190.0], ], |[200|[181.0|200.0|200.0], ], |[210|[189.0|210.0|210.0], ],

```

[220 [193.0 220.0 220.0],],	[230 [197.0 230.0 230.0],],	[240 [201.0 240.0 240.0],],
[250 [198.5 250.0 250.0],],	[260 [204.8 260.0 260.0],],	[270 [207.0 270.0 270.0],],
[280 [205.0 280.0 280.0],],	[290 [202.0 290.0 290.0],],	[300 [185.0 300.0 300.0],],
[310 [181.0 310.0 310.0],],	[320 [162.0 320.0 320.0],],	[330 [159.2 330.0 330.0],],
[340 [154.0 340.0 340.0],],	[350 [145.0 350.0 350.0],],	[360 [140.0 360.0 360.0],],
[370 [131.5 370.0 370.0],],	[380 [124.5 376.0 380.0],],	[390 [118.0 381.0 390.0],],
[400 [108.0 382.0 400.0],],	[410 [98.5 378.0 410.0],],	[420 [99.5 372.0 420.0],],
[430 [93.5 356.0 430.0],],	[440 [94.0 355.5 440.0],],	[450 [90.0 298.0 450.0],],
[460 [87.0 286.0 460.0],],	[470 [86.0 262.5 470.0],],	[480 [86.0 248.0 480.0],],
[490 [86.0 241.0 490.0],],	[500 [84.5 226.5 500.0],],	[510 [83.0 216.0 510.0],],
[520 [80.0 205.0 520.0],],	[530 [82.0 203.0 530.0],],	[540 [81.0 195.0 540.0],],
[550 [81.0 192.0 550.0],],	[560 [82.0 185.5 560.0],],	[570 [80.0 180.0 570.0],],
[580 [79.0 181.0 580.0],],	[590 [83.0 182.5 590.0],],	[600 [82.0 182.0 600.0],],
[610 [84.0 181.5 610.0],],	[620 [85.0 178.0 620.0],],	[630 [85.0 185.0 630.0],],
[640 [86.0 185.0 640.0],],	[650 [87.0 184.0 650.0],],	[660 [88.0 185.0 660.0],],
[670 [91.0 185.0 665.0],],	[680 [87.0 182.0 666.8],],	[690 [86.0 185.0 673.0],],
[700 [88.8 186.0 688.5],],	[710 [92.0 189.0 672.5],],	[720 [94.0 193.0 671.8],],
[730 [95.5 194.0 672.0],],	[740 [99.0 195.0 685.2],],	[750 [103.0 196.0 740.0],],
[760 [103.5 200.0 751.5],],	[770 [105.0 203.0 712.0],],	[780 [107.0 208.5 725.2],],
[790 [110.8 214.0 715.2],],	[800 [111.0 211.0 724.0],],	[810 [110.0 215.0 709.0],],
[820 [111.0 219.0 744.8],],	[830 [115.0 223.0 737.5],],	[840 [115.5 227.5 758.2],],
[850 [119.0 237.0 841.0],],	[860 [120.0 243.0 802.0],],	[870 [120.0 249.0 870.0],],
[880 [125.8 258.0 880.0],],	[890 [128.0 262.0 890.0],],	[900 [134.2 261.0 900.0],],
[910 [141.0 273.0 910.0],],	[920 [145.8 287.5 920.0],],	[930 [153.0 299.0 930.0],],
[940 [160.0 307.5 940.0],],	[950 [167.5 318.0 950.0],],	[960 [174.0 326.5 960.0],],
[970 [178.0 336.5 970.0],],	[980 [184.5 339.5 980.0],],	[990 [190.0 356.0 990.0],],
[1000 [181.0 359.0 1.0k],],	[1010 [142.8 243.0 401.5],],	[1020 [142.0 250.0 408.0],],
[1030 [149.8 259.5 418.8],],	[1040 [149.0 270.0 429.5],],	[1050 [150.5 278.0 433.0],],
[1060 [150.5 288.0 447.5],],	[1070 [152.0 298.5 457.2],],	[1080 [150.0 299.5 466.2],],
[1090 [153.0 310.0 477.5],],	[1100 [162.5 321.5 487.8],],	[1110 [162.0 333.0 501.0],],
[1120 [167.5 341.5 509.2],],	[1130 [174.5 351.5 513.0],],	[1140 [177.5 357.5 523.0],],
[1150 [188.0 373.0 538.5],],	[1160 [196.5 383.0 556.0],],	[1170 [205.0 395.5 570.5],],
[1180 [216.0 408.0 585.0],],	[1190 [217.5 415.5 590.5],],	[1200 [235.0 425.5 594.5],],
[1210 [238.0 454.0 603.0],],	[1220 [238.0 436.0 583.0],],	[1230 [255.5 467.0 597.2],],
[1240 [269.0 490.0 615.8],],	[1250 [281.0 505.0 631.0],],	[1260 [292.0 516.0 640.0],],
[1270 [323.0 554.0 666.0],],	[1280 [341.0 564.0 680.5],],	[1290 [343.0 574.0 692.0],],
[1300 [456.2 597.5 709.2],],	[1310 [492.0 626.5 727.8],],	[1320 [514.0 647.0 742.0],],
[1330 [530.5 657.0 743.5],],	[1340 [547.0 672.0 762.0],],	[1350 [586.0 691.5 781.8],],
[1360 [588.0 697.0 782.0],],	[1370 [598.0 731.0 805.0],],	[1380 [626.8 742.0 822.8],],
[1390 [636.8 752.0 832.8],],	[1400 [645.5 763.0 850.5],],	[1410 [636.2 772.0 841.8],],
[1420 [646.2 782.0 851.8],],	[1430 [627.2 784.5 861.8],],	[1440 [640.5 803.0 868.5],],
[1450 [676.2 821.5 881.8],],	[1460 [663.8 831.5 885.2],],	[1470 [667.2 824.5 872.0],],
[1480 [677.2 834.5 882.0],],	[1490 [687.2 844.5 892.0],],	[1500 [697.2 854.5 902.0],],
[1510 [707.2 864.5 912.0],],	[1520 [714.0 866.0 913.0],],	[1530 [715.5 845.0 913.2],],
[1540 [725.5 855.0 923.2],],	[1550 [735.5 865.0 933.2],],	[1560 [745.5 875.0 943.2],],
[1570 [747.0 854.0 948.0],],	[1580 [757.0 864.0 958.0],],	[1590 [758.5 835.5 948.8],],
[1600 [768.5 845.5 958.8],],	[1610 [766.2 810.5 913.8],],	[1620 [776.2 820.5 923.8],],
[1630 [782.5 824.0 875.5],],	[1640 [792.5 834.0 885.5],],	[1650 [802.5 844.0 895.5],],
[1660 [812.5 854.0 905.5],],	[1670 [827.2 870.5 934.8],],	[1680 [837.2 880.5 944.8],],
[1690 [847.2 890.5 954.8],],	[1700 [845.0 894.0 984.0],],	[1710 [855.0 904.0 994.0],],

They went extinct with initial mean genome:

At time 400 (high population count)

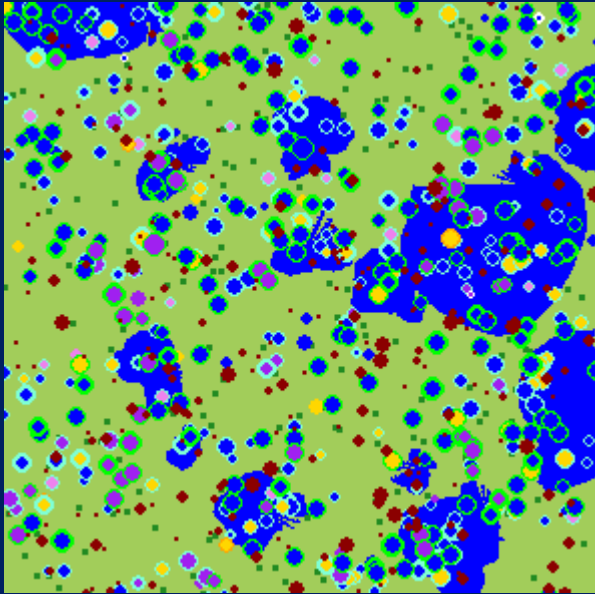
At time 760 (the population peak):

Mean genome at time 1390:

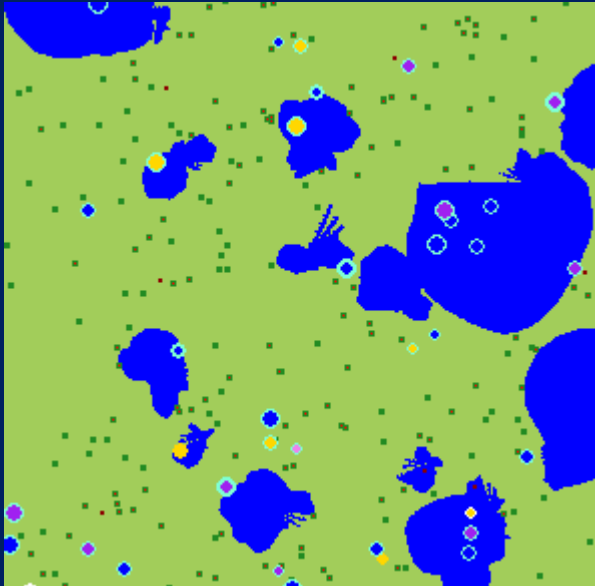
01/04:

With corrected movement, I attempted the simulation again.

Population boom:



Post mass-starvation:



All animas die eventually but the simulation is starting to become more promising with the improved movement.

These are the mean genomes of the endgame survivors:

```
[[2330|[[apple], |[6.4|[2.8|1.6|0.89], ], |72.3|29.1|20.4|21.2|82.5|1.6|100.0|100.0|71.3|[ ], ], ],
|[2340|[[apple], |[6.3|[2.8|1.6|0.85], ], |71.9|29.0|20.0|21.0|81.4|1.5|100.0|100.0|69.7|[ ], ], ],
|[2350|[[apple], |[6.3|[2.8|1.6|0.81], ], |70.6|28.8|19.8|20.9|80.6|1.5|100.0|100.0|67.7|[ ], ], ],
|[2360|[[apple], |[6.3|[2.8|1.6|0.81], ], |70.6|28.8|19.8|20.9|80.6|1.5|100.0|100.0|67.7|[ ], ], ],
|[2370|[[apple], |[6.3|[2.8|1.6|0.81], ], |70.6|28.8|19.8|20.9|80.6|1.5|100.0|100.0|67.7|[ ], ], ],
|[2380|[[apple], |[6.3|[2.8|1.6|0.81], ], |70.6|28.8|19.8|20.9|80.6|1.5|100.0|100.0|67.7|[ ], ], ],
|[2390|[[apple], |[6.3|[2.8|1.6|0.8], ], |71.6|28.6|20.2|20.8|80.9|1.5|100.0|100.0|66.6|[ ], ], ],
|[2400|[[apple], |[6.3|[2.8|1.6|0.8], ], |71.6|28.6|20.2|20.8|80.9|1.5|100.0|100.0|66.6|[ ], ], ],
|[2410|[[apple], |[6.3|[2.8|1.6|0.8], ], |71.6|28.6|20.2|20.8|80.9|1.5|100.0|100.0|66.6|[ ], ], ],
|[2420|[[apple], |[6.3|[2.8|1.6|0.8], ], |71.6|28.6|20.2|20.8|80.9|1.5|100.0|100.0|66.6|[ ], ], ],
|[2430|[[apple], |[6.3|[2.8|1.6|0.8], ], |71.6|28.6|20.2|20.8|80.9|1.5|100.0|100.0|66.6|[ ], ], ],
```



```
|62.3|24.8|16.6|18.1|69.7|0.97|100.0|100.0|63.2|[], [], |, |, |[2900|[[apple], |[5.6|[2.5|1.5|0.67],
|, |62.3|24.8|16.6|18.1|69.7|0.97|100.0|100.0|63.2|[], [], |, |[2910|[], |, |[2920|[], |, |[2930|[], |,
|]
```

I was surprised to find that the terrain adaptation genes universally increased from their base of [1, 0.5, 0.3], considering that if my code logic was correct, only the ratios between them matter. I will have to bear in mind that this likely shows that either my terrain movement logic is wrong, or my mutation logic favours increasing values rather than decreasing values enough to have produced this result. Most surprising is that ice terrain adaptation increases, considering there was no ice terrain in this iteration of the experiment.

In fact it seems the latter hypothesis the case when comparing the final mean genomes to the original:

```
[0|[[apple], |[3.4|[1.0|0.5|0.3], |, |22.8|9.8|6.8|7.1|25.3|0.52|64.9|117.6|29.6|[], |]
```

All values with the capacity to increase did so.

I amended the max genotypelimit to 500. Limits should be more natural rather than this awkward fixed constant limit for all genes, which makes no sense since all genes have very different ranges.

The population count throughout the simulation and death causes were as follows:

```
[[0|200], |[10|200], |[20|200], |[30|200], |[40|200], |[50|202], |[60|202], |[70|205], |[80|209],
|[90|221], |[100|224], |[110|231], |[120|237], |[130|240], |[140|245], |[150|253], |[160|256],
|[170|265], |[180|270], |[190|278], |[200|284], |[210|292], |[220|299], |[230|307], |[240|311],
|[250|322], |[260|324], |[270|333], |[280|343], |[290|356], |[300|364], |[310|375], |[320|394],
|[330|410], |[340|421], |[350|433], |[360|448], |[370|460], |[380|476], |[390|495], |[400|511],
|[410|522], |[420|538], |[430|563], |[440|576], |[450|588], |[460|619], |[470|644], |[480|652],
|[490|677], |[500|695], |[510|721], |[520|755], |[530|776], |[540|794], |[550|810], |[560|831],
|[570|846], |[580|861], |[590|862], |[600|891], |[610|915], |[620|912], |[630|918], |[640|938],
|[650|956], |[660|972], |[670|977], |[680|980], |[690|992], |[700|996], |[710|1009], |[720|1010],
|[730|1011], |[740|1008], |[750|1006], |[760|1003], |[770|999], |[780|1016], |[790|1009],
|[800|1029], |[810|1019], |[820|1024], |[830|1023], |[840|1008], |[850|1011], |[860|994],
|[870|996], |[880|962], |[890|955], |[900|923], |[910|919], |[920|894], |[930|861], |[940|849],
|[950|826], |[960|797], |[970|780], |[980|750], |[990|726], |[1000|696], |[1010|663],
|[1020|633], |[1030|602], |[1040|560], |[1050|540], |[1060|526], |[1070|499], |[1080|467],
|[1090|437], |[1100|422], |[1110|401], |[1120|373], |[1130|356], |[1140|332], |[1150|322],
|[1160|306], |[1170|280], |[1180|260], |[1190|249], |[1200|237], |[1210|217], |[1220|197],
|[1230|186], |[1240|179], |[1250|165], |[1260|143], |[1270|135], |[1280|136], |[1290|130],
|[1300|124], |[1310|116], |[1320|111], |[1330|108], |[1340|105], |[1350|99], |[1360|94],
|[1370|88], |[1380|84], |[1390|78], |[1400|72], |[1410|69], |[1420|68], |[1430|67], |[1440|60],
|[1450|58], |[1460|58], |[1470|57], |[1480|52], |[1490|50], |[1500|47], |[1510|43], |[1520|43],
|[1530|42], |[1540|42], |[1550|41], |[1560|40], |[1570|40], |[1580|40], |[1590|40], |[1600|40],
|[1610|40], |[1620|39], |[1630|39], |[1640|39], |[1650|38], |[1660|38], |[1670|38], |[1680|38],
|[1690|38], |[1700|38], |[1710|38], |[1720|38], |[1730|38], |[1740|38], |[1750|38], |[1760|37],
|[1770|36], |[1780|36], |[1790|33], |[1800|33], |[1810|33], |[1820|33], |[1830|33], |[1840|33],
|[1850|33], |[1860|31], |[1870|31], |[1880|31], |[1890|31], |[1900|31], |[1910|31], |[1920|31],
|[1930|31], |[1940|31], |[1950|31], |[1960|29], |[1970|29], |[1980|29], |[1990|29], |[2000|29],
|[2010|29], |[2020|29], |[2030|29], |[2040|28], |[2050|27], |[2060|27], |[2070|26], |[2080|26],
|[2090|26], |[2100|25], |[2110|25], |[2120|25], |[2130|25], |[2140|24], |[2150|24], |[2160|24],
|[2170|24], |[2180|23], |[2190|23], |[2200|23], |[2210|23], |[2220|22], |[2230|21], |[2240|21],
|[2250|21], |[2260|17], |[2270|17], |[2280|17], |[2290|17], |[2300|16], |[2310|15], |[2320|14],
|[2330|13], |[2340|12], |[2350|11], |[2360|11], |[2370|11], |[2380|11], |[2390|9], |[2400|9],
```

[2966|41|0|3|0],

The high death count from hunger is to be expected due to the mass starvation. The death count list does not give much idea of how senescence affected the population; it could be helpful to improve the database system to include this.

Dr He [apple], [], 19.5 15.7 ate apple

I adjusted movement to follow a smoother path and be less abrupt, now mostly incrementing by smaller amounts:

I noticed that animas were dying from hunger at younger ages than I would have thought likely, so I will investigate the energy expenditure function.

```
|1225|[apple], |5.0|[2.0|1.0|0.61], |, |48.1|20.4|13.5|13.9|50.9|1.1|129.1|220.2|48.6|[[], |, |,
|1250|[apple], |4.9|[2.0|1.0|0.61], |, |47.9|20.3|13.6|13.9|50.8|1.1|129.2|221.3|48.2|[[], |,
```

```
[0|[apple], |[3.4|[1.0|0.5|0.3], ], |22.8|9.8|6.8|7.1|25.3|0.52|64.9|117.6|29.6|[ ], ]
```

ETHAN CHENG 5045 | Evolution Simulator

I sourced the error and corrected it:

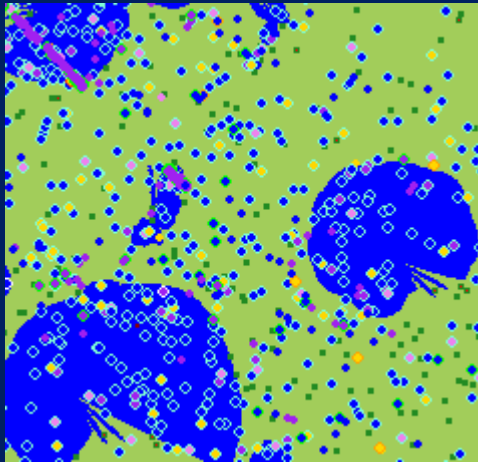
```
random.gauss((A+B)/2), (MUTATION_FACTOR*(A+B)/2))
```

->

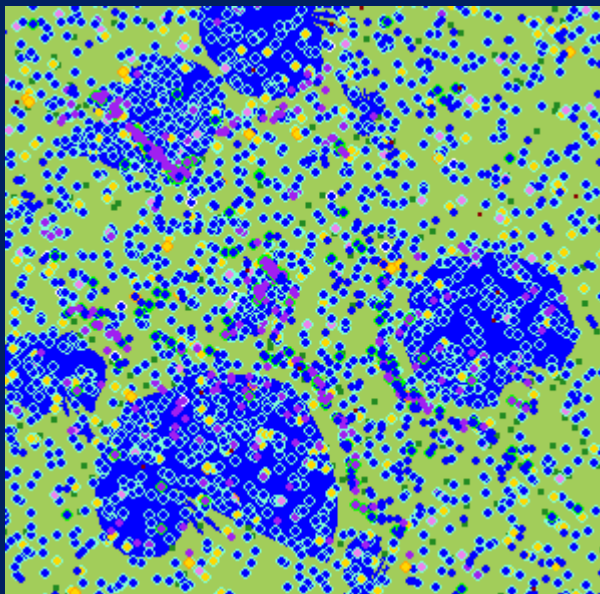
```
random.gauss(((A+B)/2), (MUTATION_FACTOR*(A+B)/2))
```

Now the mean taken for the normal distribution should be the mean of the two parent genes.

In a new simulation, I noticed that animas in approach mode will sometimes still jiggle, oscillating between two cells. I also noticed that animas in approach mode strangely tend to line up along the “y=-x” diagonal, as the water cut once did, specifically when in water pools.



With no natural predators, small creatures can survive and thrive due to their low energy expenditure:



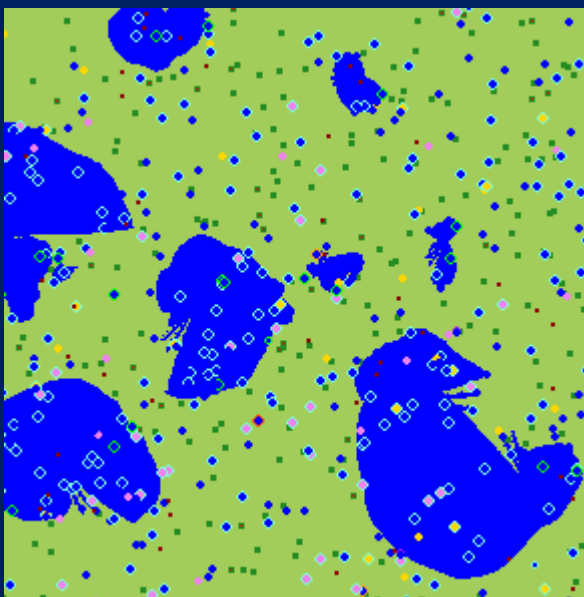
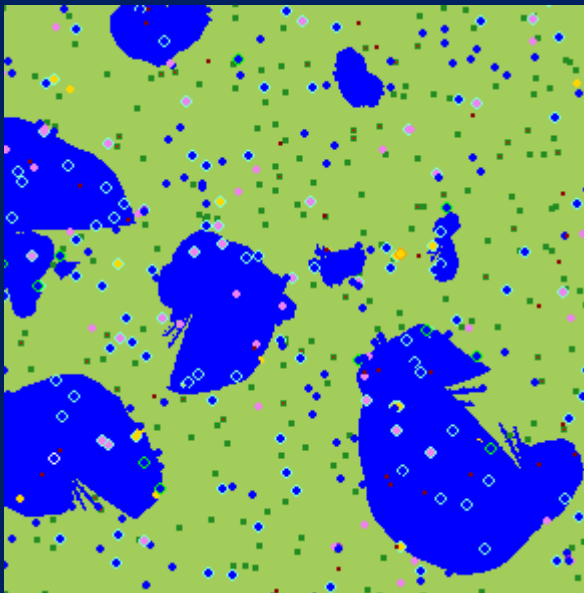
I fixed the diagonal clustering, by swapping the x and y values in a tuple expression in the water search method. I also fixed animas mating while immature due to apples now giving mating drive, by adding the missing parentheses to a function call in the update method so that it did not always return true (as existing methods always return true).

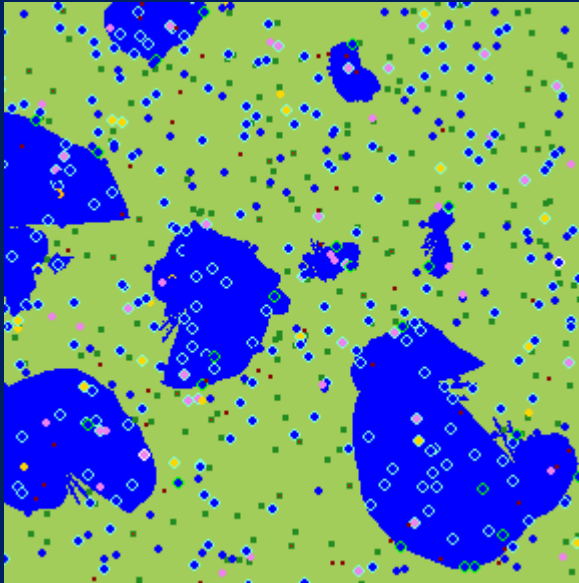
Now, to tune the simulation so that it was more balanced, and long term stable populations could be sustained.

- Energy consumption each tick was increased (from 0.025 multiplied by the energy function's output to 0.1 multiplied by the same)
- Mating drive each tick was decreased from 0.2 to 0.1, so that it is always less than the food increase rate
- Instead, eating apples now increases mating drive to better reward animas that are able to find food

Under the previous simulation, animas could live for thousands of ticks without ever finding food, and still reproducing. This is not an accurate representation of natural selection.

Under the new simulation, animas which are more fit, and more able to find food, are more likely to reproduce, and reproduce more times, than animas which are unable to find food.





For the first time, I have created a simulation with a stable population able to last thousands of ticks, and I assume, would have continued ad infinitum.

When I ended the simulation, these were the age quartiles:

[11250|[103.0|201.0|348.0]

This is very balanced and shows that while some animas were able to live to hundreds of ticks, many would have died before that point. The first quartile being 103 shows there were a large number of immature animas; there was a healthy amount of mating. This did not change much from some arbitrary other point in the middle of the simulation (4075|[106.0|211.0|346.5), showing the demographics were sustainable.

Lifespans were similar; these were the quartiles at the end:

[[11075|[255.0|388.0|642.0],], [[11100|[266.5|373.0|524.5],], [[11125|[248.0|338.5|572.2],],
[[11150|[269.5|361.0|585.0],], [[11175|[259.0|330.5|510.0],], [[11200|[267.0|380.0|561.0],],
[[11225|[261.8|338.5|511.0],], [[11250|[255.0|341.0|529.0]

Here are the lifespans near the start of the simulation:

[[2300|[228.0|286.0|561.0],], [[2325|[242.0|359.0|607.0],], [[2350|[262.0|427.0|611.5],],
[[2375|[266.0|427.0|541.0],], [[2400|[220.5|345.0|516.0],], [[2425|[264.8|377.0|615.8],]

The numbers are consistent between both timeframes; once again this shows the sustainability of the simulation.

The mean age increased to the high 500s near the start of the simulation as fruits were abundant, then fall to below 400 during a mass starvation event. After this, however, it was stable between about 400 and 480 between tick 3625 onwards to the end of the simulation at tick 11250. The mean was always between the median and the upper quartile, suggesting a negative skew; this makes sense as some animas which were fitter or luckier ate and survived for much longer, whereas, due to no predation, animas would always live to at least a certain age cutoff before starving. This would create the skew effect as a smaller proportion of animas lived disproportionately longer.

This was the mean genome at the end of the simulation:

[11250|[[apple], |[2.2|[1.0|0.53|0.31],], |22.9|11.9|8.4|7.1|27.8|0.39|68.9|78.1|24.1

Here is the mean genome in the middle of the simulation:

[6850|[[apple], |[2.4|[1.0|0.51|0.3],], |22.7|11.9|8.0|6.8|27.8|0.39|72.9|82.8|25.3|[],

About one quarter through:

2950|[[apple], |[2.7|[1.0|0.48|0.3],], |22.3|11.9|7.9|6.5|26.0|0.39|69.4|92.7|26.2|[],

After a few generations:

1425|[[apple], |[2.9|[1.0|0.5|0.31],], |22.3|11.8|7.5|6.5|25.6|0.38|66.2|97.7|25.7|[],],

Compare with the starting mean genome:

[0|[[apple], |[2.9|[1.0|0.5|0.3],], |22.1|11.3|7.5|6.5|25.6|0.43|65.6|116.6|25.4|[],

I used ChatGPT to generate a table to better view the results, and gave it the trait names to use as column headings:

Stage	Pop	Spd	G	W	I	Vis	Turn	Str	Tgh	Mate	Boost	InjThr	MatAge	Refr
Start	0	2.9	1.0	0.50	0.30	22.1	11.3	7.5	6.5	25.6	0.43	65.6	116.6	25.4
Early	1425	2.9	1.0	0.50	0.31	22.3	11.8	7.5	6.5	25.6	0.38	66.2	97.7	25.7
%	2950	2.7	1.0	0.48	0.30	22.3	11.9	7.9	6.5	26.0	0.39	69.4	92.7	26.2
Mid	6850	2.4	1.0	0.51	0.30	22.7	11.9	8.0	6.8	27.8	0.39	72.9	82.8	25.3
End	11250	2.2	1.0	0.53	0.31	22.9	11.9	8.4	7.1	27.8	0.39	68.9	78.1	24.1

ChatGPT confused turn speed and vision here; the two should be swapped.

We can see natural selection in action here.

I had run the simulation for a long time in order to get maximum process of natural selection, especially since I could see that sizes were not changing much.

We can see that the following trends emerged:

- Speed decreased from mean 2.9 to 2.2
- Terrain adaptation did not change much but an amphibian nature was slightly favoured (in reality this may just be due to the lower bound of 0.05 given to genes which skews them upwards at low values; if I cared much about this then I would implement an inverse scale which would allow low values to function as if there was no lower bound, when combining genes, but when expressed in the world take some curbed value, eg if an exponential function is taken, x is the mutation values and y is the expressed values; this would keep mutation logic unchanged and consistent but prevent logic from breaking at low values)
- Turning speed was not of much importance; perhaps I should make the starting genotypelower in this regard so that it can be selected for more; however for now I shall treat it as a kind of control
- Vision increased, but not to as drastic of a degree as speed's change. The convergence was fast
- Size continually increased throughout the simulation
- Mating drive increased a little
- Boost factor decreased and converged at 0.39; I am unsure why this even changed considering I don't think any animas were boosting; this one requires investigation

- Ditto with injury threshold; I am not sure why it changed, but it increased then decreased again
- Maturity age steadily decreased as with low opportunities for food the optimal strategy was to simply survive until able to mate without requiring any or much food (also there were no predators so the benefits of growth were greatly diminished); this genotype had the biggest decrease and is the biggest indicator that this was the dominant strategy, which allowed the population to continue to slowly expand

These changes suggest a general convergence towards animas which did not require food to reproduce; due to the intense scarcity of apples (the realm was usually barren with food being instantly swooped up as it was borne; this rewarded simply being lucky enough to be in the same place as food as it spawned, and the benefit of speed, as we see, is outweighed by the energy cost which made it more likely for hunger to outscale mating drive, and death come before an apple).

Of note were the genes with no real-world expression, injury threshold and boost factor.

Deaths were mostly from starvation, suggesting I could lower the realm wetness, or that it plays a more supporting role as a survival condition currently:

[9856|4|0|30|0]

There are then at least 30 deaths that can be attributed to a combination of childbirth and old age (childbirth unlikely to have much impact as it is a set 25 damage likely given to a healthy anima)

Here is the population count data:

[[0|50], [25|50], [50|50], [75|52], [100|52], [125|54], [150|59], [175|61], [200|64], [225|68], [250|73], [275|74], [300|75], [325|76], [350|73], [375|72], [400|76], [425|79], [450|83], [475|89], [500|92], [525|93], [550|100], [575|101], [600|102], [625|108], [650|112], [675|118], [700|121], [725|131], [750|137], [775|145], [800|152], [825|155], [850|170], [875|178], [900|186], [925|190], [950|200], [975|204], [1000|210], [1025|221], [1050|228], [1075|233], [1100|245], [1125|253], [1150|255], [1175|258], [1200|264], [1225|263], [1250|267], [1275|270], [1300|276], [1325|282], [1350|282], [1375|277], [1400|269], [1425|269], [1450|273], [1475|277], [1500|294], [1525|305], [1550|316], [1575|327], [1600|335], [1625|329], [1650|335], [1675|343], [1700|347], [1725|337], [1750|327], [1775|331], [1800|336], [1825|340], [1850|333], [1875|332], [1900|341], [1925|335], [1950|340], [1975|332], [2000|332], [2025|343], [2050|358], [2075|355], [2100|359], [2125|353], [2150|359], [2175|358], [2200|355], [2225|362], [2250|361], [2275|368], [2300|364], [2325|371], [2350|368], [2375|360], [2400|351], [2425|342], [2450|337], [2475|334], [2500|333], [2525|336], [2550|334], [2575|328], [2600|326], [2625|328], [2650|333], [2675|325], [2700|332], [2725|327], [2750|322], [2775|325], [2800|323], [2825|316], [2850|325], [2875|325], [2900|324], [2925|325], [2950|332], [2975|328], [3000|327], [3025|329], [3050|335], [3075|334], [3100|337], [3125|346], [3150|342], [3175|348], [3200|351], [3225|357], [3250|367], [3275|379], [3300|383], [3325|397], [3350|405], [3375|404], [3400|402], [3425|405], [3450|404], [3475|397], [3500|383], [3525|377], [3550|374], [3575|372], [3600|370], [3625|364], [3650|364], [3675|357], [3700|346], [3725|346], [3750|367], [3775|368], [3800|362], [3825|372], [3850|379], [3875|378], [3900|382], [3925|387], [3950|387], [3975|393], [4000|399], [4025|406], [4050|403], [4075|413], [4100|408], [4125|403], [4150|404], [4175|405], [4200|403], [4225|401], [4250|393], [4275|385], [4300|384], [4325|387], [4350|385], [4375|380], [4400|377], [4425|374], [4450|382], [4475|366], [4500|367], [4525|373], [4550|364], [4575|358], [4600|369], [4625|373], [4650|373], [4675|371], [4700|375],

[[4725|374], [[4750|371], [[4775|369], [[4800|363], [[4825|367], [[4850|357], [[4875|359],
 [[4900|354], [[4925|361], [[4950|360], [[4975|367], [[5000|362], [[5025|355], [[5050|355],
 [[5075|359], [[5100|356], [[5125|355], [[5150|357], [[5175|365], [[5200|360], [[5225|361],
 [[5250|365], [[5275|369], [[5300|381], [[5325|383], [[5350|385], [[5375|384], [[5400|388],
 [[5425|387], [[5450|399], [[5475|402], [[5500|412], [[5525|413], [[5550|416], [[5575|423],
 [[5600|419], [[5625|424], [[5650|423], [[5675|430], [[5700|425], [[5725|414], [[5750|414],
 [[5775|401], [[5800|388], [[5825|383], [[5850|379], [[5875|370], [[5900|361], [[5925|354],
 [[5950|357], [[5975|357], [[6000|359], [[6025|354], [[6050|357], [[6075|368], [[6100|376],
 [[6125|381], [[6150|384], [[6175|391], [[6200|397], [[6225|398], [[6250|396], [[6275|391],
 [[6300|395], [[6325|397], [[6350|404], [[6375|400], [[6400|398], [[6425|401], [[6450|414],
 [[6475|422], [[6500|418], [[6525|415], [[6550|413], [[6575|416], [[6600|403], [[6625|402],
 [[6650|392], [[6675|386], [[6700|379], [[6725|375], [[6750|386], [[6775|392], [[6800|392],
 [[6825|393], [[6850|400], [[6875|402], [[6900|411], [[6925|421], [[6950|428], [[6975|431],
 [[7000|431], [[7025|418], [[7050|426], [[7075|431], [[7100|439], [[7125|423], [[7150|411],
 [[7175|418], [[7200|415], [[7225|417], [[7250|410], [[7275|401], [[7300|403], [[7325|397],
 [[7350|401], [[7375|390], [[7400|384], [[7425|379], [[7450|378], [[7475|386], [[7500|402],
 [[7525|412], [[7550|416], [[7575|420], [[7600|432], [[7625|434], [[7650|438], [[7675|445],
 [[7700|444], [[7725|454], [[7750|447], [[7775|449], [[7800|435], [[7825|443], [[7850|440],
 [[7875|436], [[7900|434], [[7925|434], [[7950|443], [[7975|452], [[8000|445], [[8025|440],
 [[8050|439], [[8075|437], [[8100|438], [[8125|437], [[8150|435], [[8175|443], [[8200|454],
 [[8225|440], [[8250|444], [[8275|430], [[8300|436], [[8325|425], [[8350|426], [[8375|435],
 [[8400|432], [[8425|426], [[8450|440], [[8475|443], [[8500|451], [[8525|459], [[8550|469],
 [[8575|477], [[8600|479], [[8625|494], [[8650|493], [[8675|503], [[8700|492], [[8725|473],
 [[8750|463], [[8775|460], [[8800|452], [[8825|446], [[8850|434], [[8875|441], [[8900|426],
 [[8925|425], [[8950|423], [[8975|418], [[9000|419], [[9025|428], [[9050|425], [[9075|415],
 [[9100|428], [[9125|436], [[9150|440], [[9175|448], [[9200|454], [[9225|459], [[9250|459],
 [[9275|472], [[9300|468], [[9325|468], [[9350|473], [[9375|468], [[9400|468], [[9425|461],
 [[9450|465], [[9475|459], [[9500|459], [[9525|462], [[9550|466], [[9575|465], [[9600|476],
 [[9625|482], [[9650|485], [[9675|496], [[9700|502], [[9725|499], [[9750|503], [[9775|516],
 [[9800|512], [[9825|515], [[9850|500], [[9875|506], [[9900|500], [[9925|487], [[9950|480],
 [[9975|490], [[10000|502], [[10025|492], [[10050|498], [[10075|497], [[10100|507],
 [[10125|495], [[10150|504], [[10175|494], [[10200|495], [[10225|499], [[10250|486],
 [[10275|497], [[10300|488], [[10325|493], [[10350|485], [[10375|480], [[10400|476],
 [[10425|480], [[10450|480], [[10475|477], [[10500|478], [[10525|483], [[10550|491],
 [[10575|495], [[10600|501], [[10625|487], [[10650|492], [[10675|482], [[10700|471],
 [[10725|473], [[10750|465], [[10775|466], [[10800|462], [[10825|468], [[10850|474],
 [[10875|476], [[10900|482], [[10925|479], [[10950|485], [[10975|494], [[11000|499],
 [[11025|502], [[11050|496], [[11075|508], [[11100|500], [[11125|499], [[11150|493],
 [[11175|501], [[11200|506], [[11225|494], [[11250|499],],

////////////////////////////////////

Population numbers were gradually trending upwards for the entirety of the simulation all the way to the stopping point, suggesting that there were enough animas able to mate after eating only one apple or even none, to drive an increase in numbers even in food scarcity.

The starting fruit ratio is lowered and fruit amounts are now generated semi-probabilistically (some integer amount equal to the floor of total capacity multiplied by starting fruit ratio realm attribute, with probability, equal to the remainder of this, of one extra fruit).

The min water pool radius was reduced from 5 to 3, and the max from 20 to 15. Wetness was halved from 0.2 to 0.1. Apple food amount given was reduced from 300 to 100, but each tree now produces apples in batches of 3 at a rate of 0.05 and a capacity of 4.

150 trees with batch 3 at a rate of 0.03*

Animas are born with an initial hunger and thirst of 20 each.

Animas grow by 0.025 in both strength and toughness per tick while immature, was 0.02.

These changes should promote a greater drive toward fitness in terms of finding resources.

And so, as I move on to the next iteration of the simulation, I say goodbye to Mehh...

Mehh [apple], [], 9.2 7.3 died of starvation aged 391. Its final status was [100.66476423562301, 32.700000000000195, 68.86956143484143, 0.1, 72.0134937981872] and it was the parent of []

03/04 continued:

There was in fact yet another massive issue which was heavily impacting results: in the setGoals method which I had developed months ago, sleepiness was treated as a need just like the others, with a priority which would stifle other needs, even though sleep itself would only come when sleepiness reached 100. This was bad design because it meant that once sleepiness reached a certain level above hunger and thirst, animas would aimlessly wander until they were able to sleep.

I fixed this by removing sleepiness from the list of statuses affecting prioritised needs I did not remove damage though I will bear its impact in mind for when predation is introduced later.

With this change, animas once again flourished and increased without bound. I doubled the hunger and thirst, reduced the tree count to 100, increased the sleepiness rate by 4/3, and restarted the simulation. All the while I have also been adjusting the colour set "mature_set"; for instance there is now a difference in eye colour between when a target is food, and when it is water, and for water it matches the water colour itself, for visual intuitiveness; also, the water colour itself has been changed due to being the same as the idle eye colour, which made some small creatures invisible in water for the simulation observer. The animas now died very quickly, so I increased the tree amount back to 150 and decreased the hunger rate to 0.15 multiplied by the energy expense return.

The simulation now looked very good, but I had concerns that natural selection for resource finding still had an underplayed role and that creatures were able to compete with one another at a high variance of fitness, due to the capacity for mating with low food being quite high, and the capacity for mating multiple times even with high fitness being moderately low, in my hypothesis.

To test my hypothesis, I added offspring count distributions to the database, as well as average statuses.

After running a simulation of over 7000 ticks, I had... no data, due to the fact that I had never actually implemented any code adding to the offspring and parent lists. I now did so.

Also, mating drive status was heavily skewed by all the '-999' values given to immature animas, so I set the floor of values passed to be 0 (if I ever add more status points, this may require further updates, if I wish to use negative values as an extension of the positive ones).

Without too much analysis of the data, I noticed that the population over time seemed very stable:

[[0|50], [[25|50], [[50|49], [[75|47], [[100|46], [[125|48], [[150|48], [[175|50], [[200|56],
 [[225|62], [[250|70], [[275|74], [[300|81], [[325|87], [[350|98], [[375|105], [[400|112],
 [[425|121], [[450|127], [[475|142], [[500|153], [[525|173], [[550|195], [[575|206], [[600|199],
 [[625|206], [[650|203], [[675|200], [[700|200], [[725|203], [[750|199], [[775|190], [[800|184],
 [[825|164], [[850|165], [[875|162], [[900|164], [[925|171], [[950|174], [[975|174], [[1000|177],
 [[1025|193], [[1050|185], [[1075|190], [[1100|184], [[1125|179], [[1150|183], [[1175|189],
 [[1200|188], [[1225|183], [[1250|190], [[1275|195], [[1300|203], [[1325|218], [[1350|220],
 [[1375|220], [[1400|221], [[1425|223], [[1450|220], [[1475|215], [[1500|198], [[1525|192],
 [[1550|176], [[1575|169], [[1600|161], [[1625|163], [[1650|165], [[1675|162], [[1700|168],
 [[1725|170], [[1750|164], [[1775|168], [[1800|180], [[1825|187], [[1850|197], [[1875|205],
 [[1900|211], [[1925|221], [[1950|219], [[1975|223], [[2000|225], [[2025|215], [[2050|188],
 [[2075|173], [[2100|167], [[2125|176], [[2150|176], [[2175|176], [[2200|183], [[2225|188],
 [[2250|181], [[2275|185], [[2300|181], [[2325|190], [[2350|190], [[2375|198], [[2400|190],
 [[2425|173], [[2450|173], [[2475|176], [[2500|171], [[2525|183], [[2550|190], [[2575|188],
 [[2600|184], [[2625|189], [[2650|201], [[2675|199], [[2700|210], [[2725|218], [[2750|227],
 [[2775|224], [[2800|237], [[2825|239], [[2850|244], [[2875|238], [[2900|231], [[2925|226],
 [[2950|217], [[2975|195], [[3000|191], [[3025|190], [[3050|194], [[3075|201], [[3100|197],
 [[3125|199], [[3150|196], [[3175|209], [[3200|195], [[3225|184], [[3250|185], [[3275|197],
 [[3300|194], [[3325|198], [[3350|210], [[3375|215], [[3400|229], [[3425|239], [[3450|218],
 [[3475|206], [[3500|211], [[3525|206], [[3550|227], [[3575|227], [[3600|233], [[3625|240],
 [[3650|244], [[3675|237], [[3700|235], [[3725|233], [[3750|234], [[3775|221], [[3800|214],
 [[3825|193], [[3850|179], [[3875|176], [[3900|176], [[3925|184], [[3950|184], [[3975|193],
 [[4000|188], [[4025|184], [[4050|189], [[4075|182], [[4100|182], [[4125|205], [[4150|207],
 [[4175|214], [[4200|216], [[4225|217], [[4250|206], [[4275|200], [[4300|193], [[4325|192],
 [[4350|193], [[4375|197], [[4400|199], [[4425|208], [[4450|224], [[4475|227], [[4500|236],
 [[4525|252], [[4550|269], [[4575|266], [[4600|269], [[4625|267], [[4650|258], [[4675|232],
 [[4700|220], [[4725|224], [[4750|231], [[4775|245], [[4800|240], [[4825|238], [[4850|236],
 [[4875|228], [[4900|220], [[4925|221], [[4950|226], [[4975|219], [[5000|223], [[5025|225],
 [[5050|228], [[5075|231], [[5100|230], [[5125|229], [[5150|231], [[5175|236], [[5200|234],
 [[5225|244], [[5250|244], [[5275|238], [[5300|231], [[5325|233], [[5350|234], [[5375|239],
 [[5400|255], [[5425|262], [[5450|257], [[5475|257], [[5500|259], [[5525|274], [[5550|263],
 [[5575|249], [[5600|257], [[5625|251], [[5650|250], [[5675|230], [[5700|231], [[5725|217],
 [[5750|230], [[5775|243], [[5800|244], [[5825|259], [[5850|272], [[5875|272], [[5900|264],
 [[5925|253], [[5950|235], [[5975|202], [[6000|190], [[6025|179], [[6050|180], [[6075|194],
 [[6100|210], [[6125|211], [[6150|229], [[6175|222], [[6200|238], [[6225|240], [[6250|233],
 [[6275|226], [[6300|233], [[6325|231], [[6350|242], [[6375|245], [[6400|245], [[6425|260],
 [[6450|276], [[6475|282], [[6500|296], [[6525|295], [[6550|310], [[6575|286], [[6600|280],
 [[6625|271], [[6650|256], [[6675|240], [[6700|233], [[6725|235], [[6750|242], [[6775|253],
 [[6800|248], [[6825|239], [[6850|231], [[6875|233], [[6900|217], [[6925|220], [[6950|211],
 [[6975|209], [[7000|212], [[7025|226], [[7050|242], [[7075|253], [[7100|262], [[7125|273],
 [[7150|279], [[7175|267], [[7200|266], [[7225|259], [[7250|258], [[7275|268], [[7300|274],
 [[7325|270], [[7350|292], [[7375|295], [[7400|301], [[7425|291], [[7450|292], [[7475|277],
 [[7500|251], [[7525|245], [[7550|255],]

This was a good sign that food scarcity was a more effective limiter than before.

I ran the simulation again with the new updates.

Here was the offspring distribution at the end:

```
[1775|[[0|164], |[1|16], |[2|20], |[3|21], |[4|23], |[5|12], |[6|10], |[7|5], |[8|6], |[9|2], |[10|1],
|[11|2], |[12|0], |[13|0], |[14|0], |[15|0], |[16|0], |[17|1], |[18|0], |[19|1], ], ], |[1800|[[0|169],
|[1|19], |[2|23], |[3|16], |[4|26], |[5|11], |[6|11], |[7|3], |[8|5], |[9|2], |[10|1], |[11|1], |[12|0],
|[13|0], |[14|0], |[15|0], |[16|0], |[17|1], |[18|0], |[19|0], |[20|0], |[21|1], ], ], |[1825|[[0|194],
|[1|15], |[2|31], |[3|18], |[4|19], |[5|9], |[6|13], |[7|6], |[8|5], |[9|7], |[10|1], |[11|0], |[12|0],
|[13|0], |[14|0], |[15|0], |[16|0], |[17|1], |[18|0], |[19|0], |[20|0], |[21|1]
```

I halved the food gain from apples and set hunger gain rate to 0.1 multiplied by the energy expense return. The apple fruit is now: [-50, 0, 0, -50, 50]; I think this will be a good default fruit to base others around.

The population spiralled out of control again. I realised that my efforts to control mating had all been largely in vain due to the line:

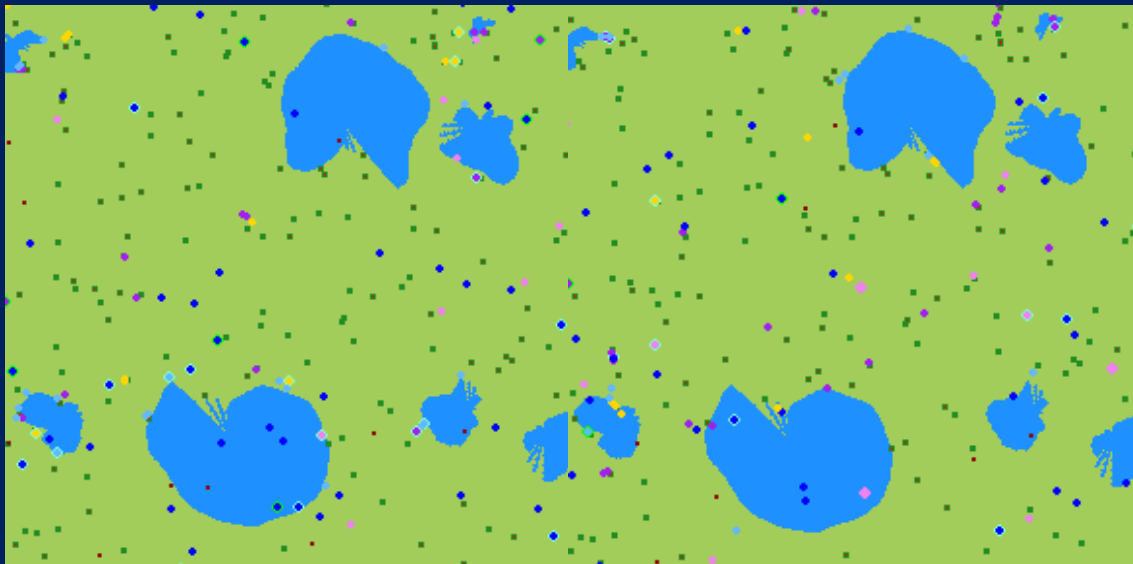
```
if self.status[4] < self.mating_drive:
```

```
    self.status[4] = min(self.mating_drive, MAXIMUM_MATING_DRIVE)
```

This code meant that regardless of not being able to eat, animas which had evolved a high enough genetic mating drive were simply able to mate and keep mating as long as they were not asleep, as soon as they had matured. This meant that a combination of low maturity age and high mating drive meant the anima population would balloon out of control, with little regard to any other selective pressures.

So I removed this line and created a similar line setting mating drive status to a fixed 10 if mature. Mating drive as a genotype will be implemented differently in future but for now I am shelving it.

Is the simulation now stable?



Here is the mean genotype after over 7000 ticks:

```
[7625|[[apple], |[2.3|[1.0|0.49|0.27], ], |27.0|18.3|6.4|7.0|28.8|0.6|61.3|75.6|22.2|[],
```

Compare to at 0 ticks:

```
[0|[[apple], |[3.1|[1.0|0.5|0.3], ], |23.0|9.6|6.8|6.9|25.2|0.5|67.2|122.1|29.1|[],
```

We can see that vision increased twofold while speed dropped dramatically.

Starting hunger and thirst 20 -> 25

Since a stable iteration of the simulation had been reached with this more limited realm, I created a larger realm of size 600x500 and scaled up the animas and trees. The next step will be to introduce other flora, and predation.

The population showed an upwards trend but did was mostly very stable:

```
[[0|400], [[25|400], [[50|395], [[75|376], [[100|351], [[125|322], [[150|298], [[175|288],
[[200|279], [[225|291], [[250|295], [[275|304], [[300|324], [[325|326], [[350|341], [[375|358],
[[400|375], [[425|392], [[450|399], [[475|415], [[500|446], [[525|453], [[550|470], [[575|471],
[[600|479], [[625|477], [[650|448], [[675|426], [[700|402], [[725|394], [[750|399], [[775|372],
[[800|384], [[825|385], [[850|395], [[875|409], [[900|413], [[925|419], [[950|435], [[975|415],
[[1000|409], [[1025|418], [[1050|435], [[1075|438], [[1100|424], [[1125|400], [[1150|405],
[[1175|410], [[1200|398], [[1225|389], [[1250|383], [[1275|385], [[1300|376], [[1325|393],
[[1350|408], [[1375|431], [[1400|443], [[1425|446], [[1450|450], [[1475|459], [[1500|456],
[[1525|465], [[1550|472], [[1575|484], [[1600|490], [[1625|466], [[1650|448], [[1675|418],
[[1700|398], [[1725|370], [[1750|356], [[1775|343], [[1800|357], [[1825|356], [[1850|366],
[[1875|369], [[1900|382], [[1925|397], [[1950|416], [[1975|443], [[2000|459], [[2025|478],
[[2050|487], [[2075|504], [[2100|506], [[2125|491], [[2150|487], [[2175|476], [[2200|466],
[[2225|442], [[2250|442], [[2275|418], [[2300|392], [[2325|378], [[2350|376], [[2375|374],
[[2400|369], [[2425|366], [[2450|374], [[2475|376], [[2500|385], [[2525|392], [[2550|414],
[[2575|427], [[2600|428], [[2625|436], [[2650|428], [[2675|416], [[2700|408], [[2725|419],
[[2750|413], [[2775|406], [[2800|392], [[2825|393], [[2850|400], [[2875|396], [[2900|386],
[[2925|382], [[2950|371], [[2975|376], [[3000|377], [[3025|385], [[3050|395], [[3075|402],
[[3100|420], [[3125|438], [[3150|433], [[3175|424], [[3200|414], [[3225|404], [[3250|387],
[[3275|387], [[3300|387], [[3325|383], [[3350|377], [[3375|388], [[3400|403], [[3425|404],
[[3450|422], [[3475|438], [[3500|437], [[3525|434], [[3550|424], [[3575|426], [[3600|419],
[[3625|398], [[3650|381], [[3675|364], [[3700|342], [[3725|353], [[3750|370], [[3775|384],
[[3800|393], [[3825|415], [[3850|438], [[3875|459], [[3900|468], [[3925|480], [[3950|481],
[[3975|461], [[4000|451], [[4025|447], [[4050|439], [[4075|417], [[4100|389], [[4125|378],
[[4150|391], [[4175|390], [[4200|395], [[4225|405], [[4250|396], [[4275|395], [[4300|410],
[[4325|416], [[4350|417], [[4375|397], [[4400|390], [[4425|390], [[4450|389], [[4475|384],
[[4500|391], [[4525|383], [[4550|378], [[4575|382], [[4600|399], [[4625|424], [[4650|434],
[[4675|444], [[4700|457], [[4725|451], [[4750|422], [[4775|390], [[4800|374], [[4825|355],
[[4850|342], [[4875|323], [[4900|307], [[4925|323], [[4950|323], [[4975|334], [[5000|356],
[[5025|367], [[5050|392], [[5075|409], [[5100|407], [[5125|430], [[5150|441], [[5175|437],
[[5200|449], [[5225|451], [[5250|453], [[5275|461], [[5300|480], [[5325|463], [[5350|443],
[[5375|421], [[5400|426], [[5425|410], [[5450|399], [[5475|381], [[5500|378], [[5525|386],
[[5550|398], [[5575|415], [[5600|451], [[5625|463], [[5650|480], [[5675|463], [[5700|458],
[[5725|435], [[5750|407], [[5775|396], [[5800|377], [[5825|360], [[5850|333], [[5875|325],
[[5900|320], [[5925|323], [[5950|348], [[5975|374], [[6000|398], [[6025|419], [[6050|427],
[[6075|438], [[6100|450], [[6125|458], [[6150|469], [[6175|456], [[6200|436], [[6225|432],
[[6250|441], [[6275|456], [[6300|467], [[6325|442], [[6350|420], [[6375|415], [[6400|404],
[[6425|394], [[6450|401], [[6475|393], [[6500|406], [[6525|418], [[6550|424], [[6575|429],
[[6600|448], [[6625|432], [[6650|416], [[6675|409], [[6700|400], [[6725|405], [[6750|402],
[[6775|407], [[6800|407], [[6825|413], [[6850|405], [[6875|428], [[6900|448], [[6925|466],
[[6950|478], [[6975|480], [[7000|452], [[7025|425], [[7050|400], [[7075|370], [[7100|347],
[[7125|354], [[7150|358], [[7175|369], [[7200|379], [[7225|391], [[7250|412], [[7275|413],
```

```
[
  [7300|423], [7325|434], [7350|447], [7375|439], [7400|462], [7425|472], [7450|471],
  [7475|468], [7500|474], [7525|458], [7550|429], [7575|402], [7600|391], [7625|364],
  [7650|360], [7675|381], [7700|380], [7725|386], [7750|389], [7775|405], [7800|426],
  [7825|453], [7850|460], [7875|454], [7900|442], [7925|413], [7950|398], [7975|357],
  [8000|349], [8025|351], [8050|377], [8075|376], [8100|414], [8125|437], [8150|466],
  [8175|491], [8200|499], [8225|496], [8250|488], [8275|484], [8300|467], [8325|451],
  [8350|434], [8375|414], [8400|407], [8425|395], [8450|405], [8475|421], [8500|400],
  [8525|419], [8550|431], [8575|454], [8600|449], [8625|438], [8650|453], [8675|457],
  [8700|456], [8725|457], [8750|446], [8775|458], [8800|439], [8825|433], [8850|413],
  [8875|408], [8900|404], [8925|424], [8950|427], [8975|459], [9000|475], [9025|488],
  [9050|491], [9075|490], [9100|467], [9125|442], [9150|419], [9175|383], [9200|366],
  [9225|357], [9250|369], [9275|375], [9300|396], [9325|388], [9350|415], [9375|437],
  [9400|455], [9425|488], [9450|509], [9475|517], [9500|495], [9525|474], [9550|461],
  [9575|439], [9600|411], [9625|394], [9650|401], [9675|422], [9700|426], [9725|431],
  [9750|414], [9775|415], [9800|444], [9825|466], [9850|477], [9875|480], [9900|492],
  [9925|489], [9950|473], [9975|479], [10000|488], [10025|473], [10050|492], [10075|470],
]
```

Once again thirst influenced behaviour but did not cause many deaths:

```
[18558|12|0|0|0]
```

No animas died purely from senescence-caused damage.

Mean genotypes at certain times:

```
[0|[[apple], [3.0|[1.0|0.5|0.3], ], 22.9|10.1|7.0|6.9|24.6|0.5|65.1|121.3|27.6|[], ], ]
[500|[[apple], [2.6|[1.0|0.5|0.3], ], 22.0|11.7|7.1|7.0|24.7|0.51|62.2|98.5|28.9|[], ], ]
[2000|[[apple], [2.5|[1.0|0.52|0.29], ], 22.9|13.7|7.2|6.9|24.7|0.53|61.2|80.3|28.0|[], ], ]
[5000|[[apple], [2.3|[1.0|0.51|0.31], ], 20.8|17.2|6.8|7.3|25.9|0.5|64.7|64.8|26.8|[], ], ]
[7500|[[apple], [2.4|[1.0|0.53|0.32], ], 20.1|17.5|6.9|7.1|25.5|0.52|67.8|54.0|25.1|[], ], ]
[10075|[[apple], [2.4|[1.0|0.57|0.31], ], 20.1|16.8|6.7|6.7|25.7|0.52|71.5|44.0|23.0|[], ], ]
```

In table format (I used ChatGPT again here to make the table; the columns are now corrected):

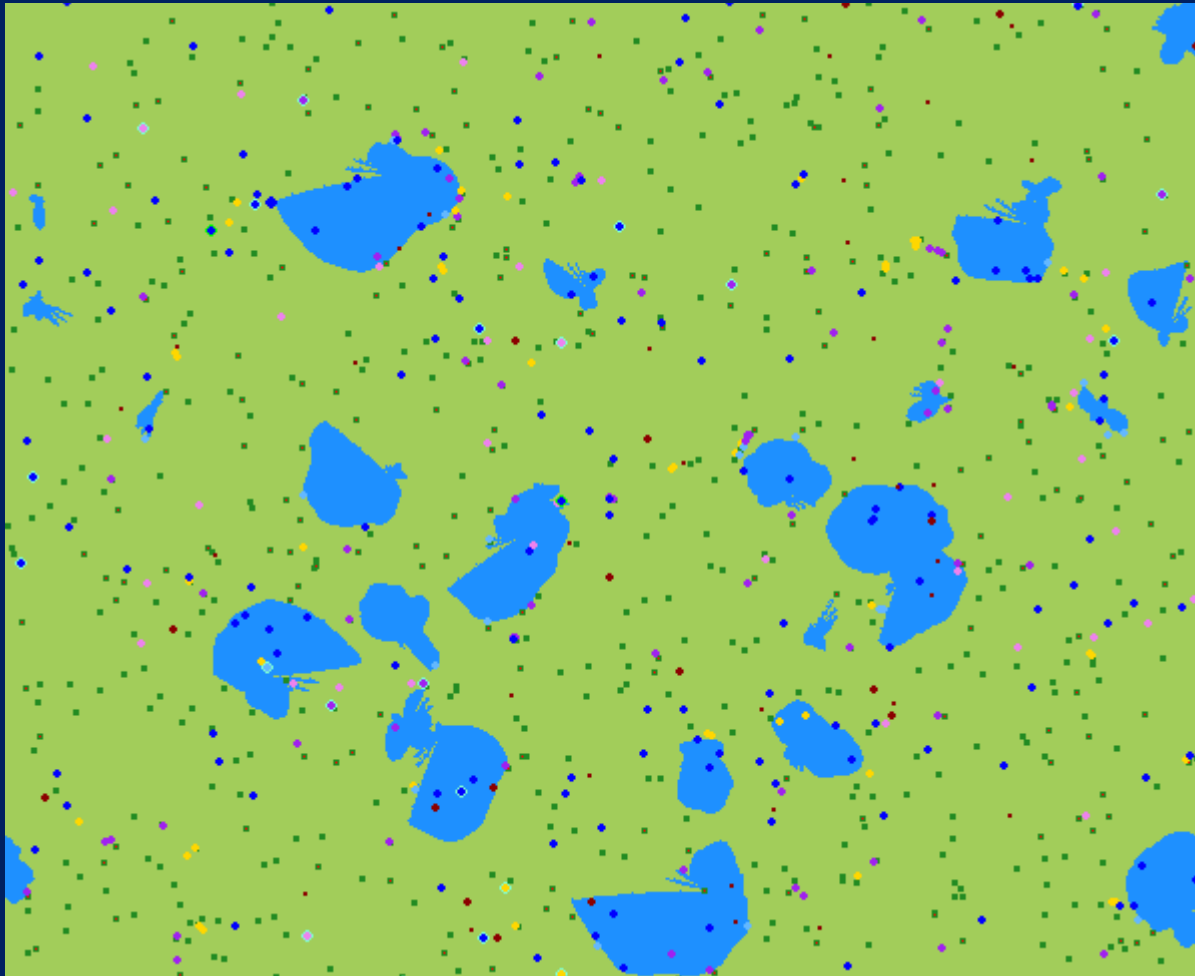
Stage	Pop	Spd	G	W	I	Turn	Vis	Str	Tgh	Mate	Boost	InjThr	MatAge	Refr
Start	0	3.0	1.0	0.50	0.30	22.9	10.1	7.0	6.9	24.6	0.50	65.1	121.3	27.6
500	500	2.6	1.0	0.50	0.30	22.0	11.7	7.1	7.0	24.7	0.51	62.2	98.5	28.9
2000	2000	2.5	1.0	0.52	0.29	22.9	13.7	7.2	6.9	24.7	0.53	61.2	80.3	28.0
5000	5000	2.3	1.0	0.51	0.31	20.8	17.2	6.8	7.3	25.9	0.50	64.7	64.8	26.8
7500	7500	2.4	1.0	0.53	0.32	20.1	17.5	6.9	7.1	25.5	0.52	67.8	54.0	25.1
End	10075	2.4	1.0	0.57	0.31	20.1	16.8	6.7	6.7	25.7	0.52	71.5	44.0	23.0

Vision converged at a higher value than the starting mean, of around 17, showing that the quadratic energy cost of $17^2 * 0.05$ (14.45) energy expense was worth paying for the better detection. Meanwhile the comparatively smaller expense paid for movement was less worth it, as speed fell from the starting value of 3 to 2.4; the energy expense would only have been about 3.7 in grass, almost 4 times cheaper than the mean energy cost for vision. With no predators, it seems the benefits provided

by speed are minimal compared to the benefits provided by being able to detect food from a greater distance, when competing with others for that same food.

Ah! I erred in my previous paragraph. Vision cost is inverse with the size; so, with a mean size of 13.4, animas were paying only about $14.45/13.4$ (1.05) energy expense for their vision of 17. This means that movement remained the greatest expense. Maturity age once again saw the most dramatic reduction over time as growth offered no benefit with the lack of predators, and had a considerable cost.

Next I added predators.



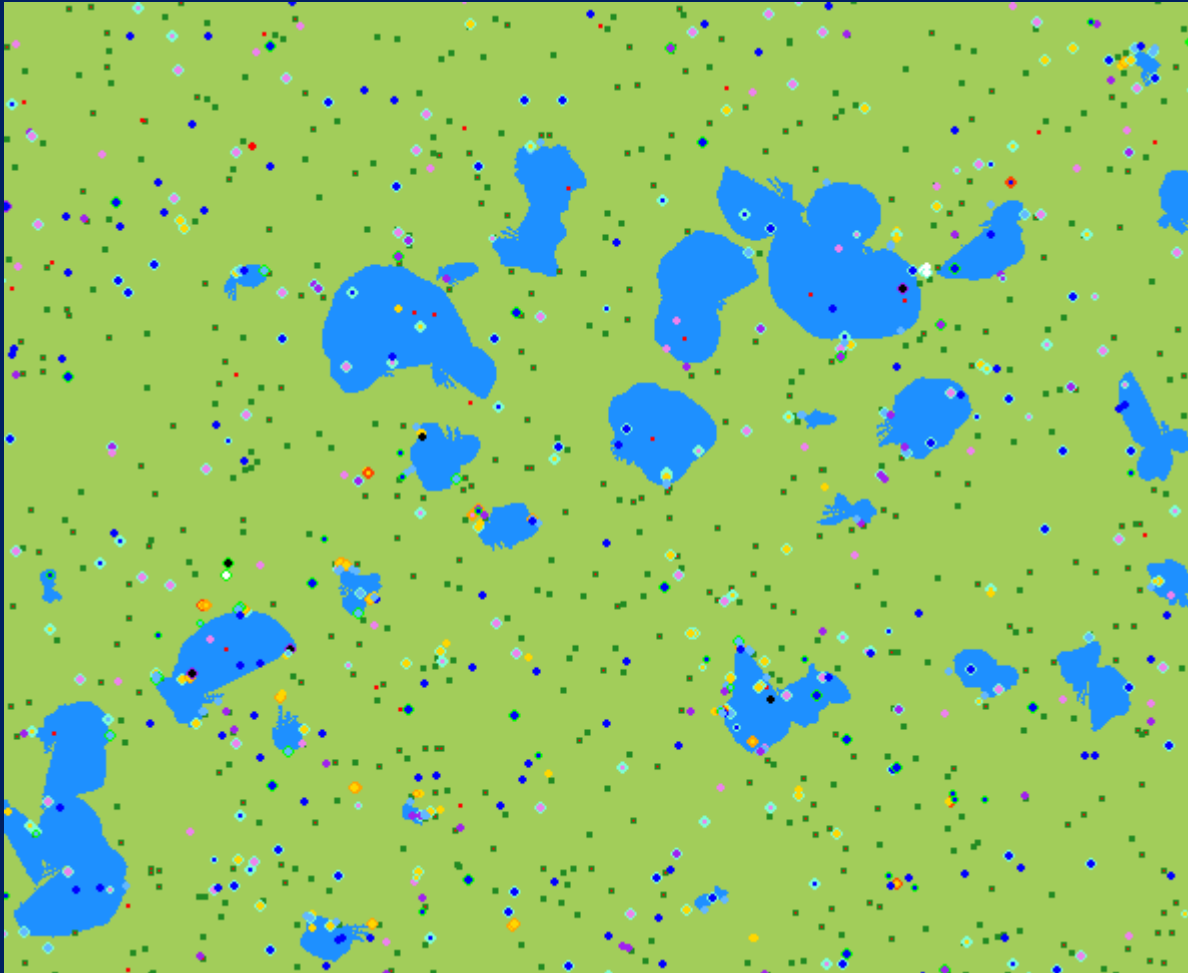
Within the first few generations the predators had died out.

08/04:

There have been many fixes and changes, including:

- Added attributes for fleeing and hunting to make evaluating other anima states easier within each anima
- Fixed hunters not being able to use the `eat()` method; this was because of unfinished `set_hunting_action()` implementation not including `eat()` as part of a conditional block
- Fixed nicknames sometimes losing a batch due to batch being set to “quen” instead of it being concatenated to nickname; also reduced the odds of “quen” from 0.025 to 0.01
- Added more colours for visual clarity and improved aesthetic

- Changed the square root factor of birth size in consumption to a constant power factor which is currently set to 0.9; increased fruit and meat values accordingly (ie meat 30 -> 100 and apple 50 -> 200); simulations were relatively stable with the old system and will now require recalibration; however this change was implemented as for predators there was no disadvantage to increasing size to match prey size, except when scavenging, as the consumption scaled at a root factor of size and meat given by an anima scales proportionally with size. The other way to fix this would be to make anima meat scale by the same factor as consumption; this could also work.



```

Leekaocewoo [apple], [], 6.7 9.4 was born to Noos [meat], [], 8.0 8.6 and Ajaoz [meat], [], 7.4 8.7!
Ougoothisu [apple], [], 6.2 7.4 ate apple
Mrs Imuxerruj [meat], [], 6.6 9.0 struck Eijouzeh [apple], [], 8.4 7.1 and dealt 3.953932869284025 damage
Cyr [apple], [], 8.0 7.8 ate apple
Kahh [apple], [], 7.1 6.9 was born to Vaoj [apple], [], 6.6 6.1 and Apwarraobbao [apple], [], 8.0 6.5!
Trythyjo [apple], [], 5.2 8.5 was born to Ynur [meat], [], 5.9 8.5 and Ms Soocuwetha [apple], [], 4.5 8.5!
Uiwitheisrou [apple], [], 6.1 6.1 is boosting to get away from Ourudam [meat], [], 5.5 7.8
Girewwee [apple], [], 5.8 6.2 is boosting to get away from Aohaov [meat], [], 6.7 9.8
Lil' Aokruthup [apple], [], 5.6 6.2 ate apple
Ekwynyrree [apple], [], 7.4 6.8 is boosting to get away from Aodui [meat], [], 7.2 10.0
Ewou [apple], [], 7.1 7.6 ate apple
Ajaoz [meat], [], 7.4 8.7 struck Ledeezuijjui [apple], [], 7.6 5.9 and dealt 5.323129742391679 damage
Mr Oovoubopy [apple], [], 6.5 8.0 is boosting to get away from Mr Weikuvid [meat], [], 8.2 8.3
Sloozugikt [apple], [], 5.5 7.0 ate apple
Ledeezuijjui [apple], [], 7.6 5.9 is boosting to get away from Ajaoz [meat], [], 7.4 8.7
Oolas [apple], [], 7.8 8.0 ate apple
Agiflube [apple], [], 5.4 6.2 is boosting to get away from Aojeic [meat], [], 6.7 7.9
Xevoukeezo [apple], [], 7.4 7.5 is boosting to get away from Aodui [meat], [], 7.2 10.0
Dituictewi [apple], [], 5.2 8.2 is boosting to get away from Tyw [meat], [], 8.7 7.1
Ugopittym [apple], [], 7.1 8.7 ate apple
Uina [apple], [], 7.3 7.7 is boosting to get away from Rivupo [meat], [], 6.9 6.5
Rofee [apple], [], 5.7 8.5 is boosting to get away from Outydi [meat], [], 7.9 8.5
Pivvysaksoo [apple], [], 7.0 7.0 ate apple
Moutudud [apple], [], 5.4 6.0 ate apple
Uinysa [apple], [], 7.4 6.8 is boosting to get away from Wat [meat], [], 5.9 6.7
Yjouzz [apple], [], 6.8 6.9 is boosting to get away from Xim [meat], [], 8.1 10.0
Eijouzeh [apple], [], 8.4 7.1 is boosting to get away from Mrs Imuxerruj [meat], [], 6.6 9.0
Oucavos [apple], [], 6.8 7.9 ate apple
Nige [apple], [], 5.5 6.4 is boosting to get away from Reikullevao [meat], [], 5.7 8.3
Ouletseeraoh [apple], [], 7.8 6.5 is boosting to get away from Jurtheigsapy [meat], [], 5.3 7.5
Raggeejj [apple], [], 7.3 5.7 is boosting to get away from Reikullevao [meat], [], 5.7 8.3
Nulonnak [meat], [], 7.6 9.5 struck Mrs Ouwyhheetb [apple], [], 8.4 7.9 and dealt 4.115865708889906 damage
Ado [meat], [], 7.9 7.4 is boosting to get away from Ijaxx [meat], [], 7.8 8.8
Aoduvycwam [apple], [], 7.8 9.2 is boosting to get away from Daothuixoutt [meat], [], 8.7 8.5
Ynuikethao [meat], [], 5.8 7.7 is boosting to get away from Luwyllou [meat], [], 6.3 7.1
Evabizz [apple], [], 4.9 7.9 is boosting to get away from Nulonnak [meat], [], 7.6 9.5
Mrs Ouwyhheetb [apple], [], 8.4 7.9 is boosting to get away from Nulonnak [meat], [], 7.6 9.5
Lil' Azaoxannopn [meat], [], 6.5 5.4 was startled awake!
Frufy [apple], [], 7.7 8.6 died of starvation aged 276. Its final status was [100.3101617893512, 75.20000000000066, 67.6319363474201, 0.1, 54.3723671190
017] and it was the parent of []
Nywaovo [apple], [], 7.2 9.0 died of starvation aged 78. Its final status was [100.57348566331596, 25.599999999999945, 23.14229350302573, 0.1, -999] and
it was the parent of []
Soubirrv [apple], [], 6.2 6.4 died of starvation aged 69. Its final status was [100.28486282250076, 33.799999999999998, 23.32167482914934, 0.1, -999] and
it was the parent of []
Seitaceib [apple], [], 5.1 9.5's body has fully decayed.
Avezokook [apple], [], 7.7 7.0's body has fully decayed.
Einiff [apple], [], 4.8 6.5's body has fully decayed.
Copuzoo [apple], [], 7.7 6.5's body has fully decayed.
Zenof [apple], [], 7.4 6.3's body has fully decayed.
[]

```

Extra traits implemented as a list element of the genome list.

Bleed and poison have been implemented as conditions; animas which are bleeding or poisoned will suffer status updates each tick, and have a probability of dispelling the condition.

Poison is implemented via a closure which allows for adjustment of the poison effect factor value.

Condition functions exist outside the class and are called with the afflicted anima as an argument at the end of each tick (the anima iterates through a copy of the condition list which is one of its attributes).

```

def bleed(this):
    this.take_damage(1)
    if random.random() < 0.15: #self.bloodclottingrate, also gene for passive healing?
        this.conditions.remove(bleed)
    bleed.effect_type = "bleed"

def define_poison(severity, stronggut):
    def suffer_poison(this):
        nonlocal severity
        this.take_damage(severity)
        this.update(0, severity/this.size())
        this.update(1, severity/this.size())
        if random.random() < 0.15 if not stronggut else 0.3:
            this.conditions.remove(suffer_poison)
            if severity >= 2:
                this.conditions.append(define_poison(int(severity/2)))
    suffer_poison.effect_type = "poison"
    return suffer_poison

```

11/04:

On this day I began creating the (extra-simulation) input UI; this was the interface by which factors of colour, music, sim control, etc could be interacted with. The two main classes expressing this interface are the Button and Slider.

I made the button class, which is as follows:

```

class Button():
    button_list = []

    def __init__(self, rect, text="", toggleable=False, off_colour=(200,200,200), hover_colour = (200,150,200), on_colour=(200,100,200), border_colour = "grey", text_colour="white"):
        self.rect = pygame.Rect(rect)
        self.text = text
        self.toggleable = toggleable
        self.status = False
        self.hovered = False
        self.clicked = False
        self.off_colour = off_colour
        self.hover_colour = hover_colour
        self.on_colour = on_colour
        self.border_colour = border_colour
        self.text_colour = text_colour
        Button.button_list.append(self)

    @classmethod
    def get_button_list(cls):
        return cls.button_list

    def handle_event(self, event):
        self.clicked = False
        if event.type == pygame.MOUSEBUTTONDOWN:
            if self.rect.collidepoint(event.pos):
                self.clicked = True
                if self.toggleable:
                    self.status = not self.status

    def draw(self, screen, font):
        if self.status and (not self.rect.collidepoint(pygame.mouse.get_pos()) or pygame.MOUSEBUTTONDOWN):
            colour = self.on_colour
        elif self.rect.collidepoint(pygame.mouse.get_pos()):
            colour = self.hover_colour
        else:
            colour = self.off_colour

        pygame.draw.rect(screen, colour, self.rect)
        pygame.draw.rect(screen, self.border_colour, self.rect, 2)

        if self.text:
            txt = font.render(self.text, True, self.text_colour)
            screen.blit(txt, txt.get_rect(center = self.rect.center))

```

This class relies on pygame and represents one input-output unit with a user-controlled effect. Due to the OOP design of the project, buttons can be made as modular objects with the collective of all buttons having events which are tallied and responded to each pygame clock tick. Encapsulation also means that states and attributes belonging to each button are bundled together with it. The other main UI unit will be the slider, which will have a slightly more complex behaviour.

Appearance-wise, a button will have 3 visual states; on, off and hover. The conditions are designed such that hovering the cursor over the button will trigger the 'hover' state, unless the button has been turned on and the mouse is still being held down (this, as well as the button's state being ON while the cursor is not colliding with it, would trigger the 'on' state). This is meant to give an intuitive feel to the button design which guides users and increases the quality or comfort of experience.

12/04:

```
class Slider():
    slider_list = []

    def __init__(self, rect, value_range=(0,100), default=0, slider_colour = (0,50,150), knob_colour = (100,100,150)):
        self.rect = pygame.Rect(rect)
        self.min = value_range[0]
        self.max = value_range[1]
        self.status = max(self.min, min(self.max, default)) #status is int slider value
        self.dragging = False # is currently being dragged?
        self.slider_colour = slider_colour
        self.knob_colour = knob_colour
        Slider.slider_list.append(self)

    @classmethod
    def get_slider_list(cls):
        return cls.slider_list

    def set_status(self, value):
        self.status = max(self.min, min(self.max, value))

    def handle_event(self, event):
        if event.type == pygame.MOUSEBUTTONDOWN:
            if self.rect.collidepoint(event.pos):
                self.dragging = True

        elif event.type == pygame.MOUSEBUTTONUP:
            self.dragging = False

        elif event.type == pygame.MOUSEMOTION and self.dragging:
            self.value = self.min + max(0, min(1, (event.pos[0]-self.rect.x)/self.rect.width)) * (self.max-self.min)

    def draw(self, screen):
        pygame.draw.rect(screen, self.slider_colour, self.rect)
        pygame.draw.circle(screen, self.knob_colour, (self.rect.x + int(self.rect.width * (self.status-self.min)/(self.max-self.min)), self.rect.centery), self.rect.height * 1.15)
```

I have created the rudimentary UI; at this point the sliders do not function. Most of the buttons and sliders required have been added but do not currently function as persistent state cannot be modified (so, only button effects which are not state changes will proceed, eg next music track or music pause/unpause. Currently there is no music; toggleable buttons can be toggled and colour changes apply as required when hovering over any button.

13/04:

Music tracks have been added and the UI edited. The issue of persistent state is fixed with the solution of the Shell class:


```
class Shell():
    def __init__(self):

        self.sfx_volume = 0.5
        self.music_volume = 0.5
        self.sfx_muted = False
        self.music_muted = False

        self.track_index = 0

        self.ui_visible = True
        self.sim_speed = 1
        self.paused = False
        self.running = True
        self.brightness = 1

        self.tutorial_index = 0

        self.anima_to_view = None
```

The shell stores the data of the current state of the UI; the shell is accessible by the main function and so the persistent state of the UI can be modified during runtime.

Each of the persistent variables such that the volume of music or the track number are now stored within the shell which acts as an intermediary between the 'ui_design' file and the main program.

So `handle_ui` now primarily alters shell attributes and executes music procedures:

```

def handle_ui(event, shell, sim_window, realm):
    if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        left = sim_window[0][0]
        right = sim_window[0][1]
        top = sim_window[1][0]
        bottom = sim_window[1][1]

        mx, my = pygame.mouse.get_pos()
        if left < mx < right and top < my < bottom:
            shell.anima_to_view = None

            nearest_anima = None
            nearest_dist = anima_selection_range

            for anima in realm.elements[1] + realm.graveyard[1]:
                dist = anima.distance_to_point(mx, my)
                if dist <= nearest_dist:
                    nearest_anima = anima
                    nearest_dist = dist
            shell.anima_to_view = nearest_anima

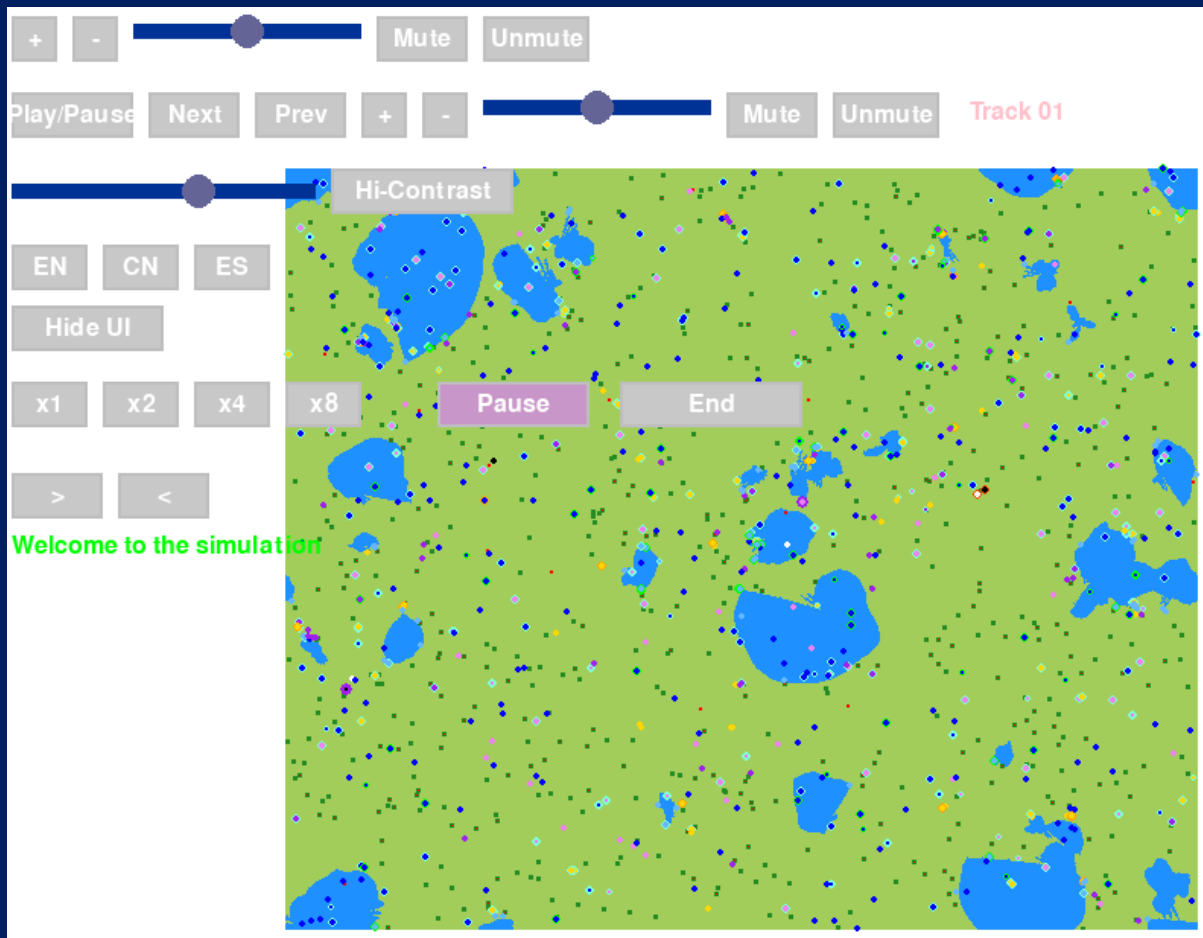
    for b in Button.button_list:
        b.handle_event(event)

    for b in lang_buttons + speed_buttons:
        b.handle_event(event)

    for s in Slider.slider_list:
        s.handle_event(event)

```

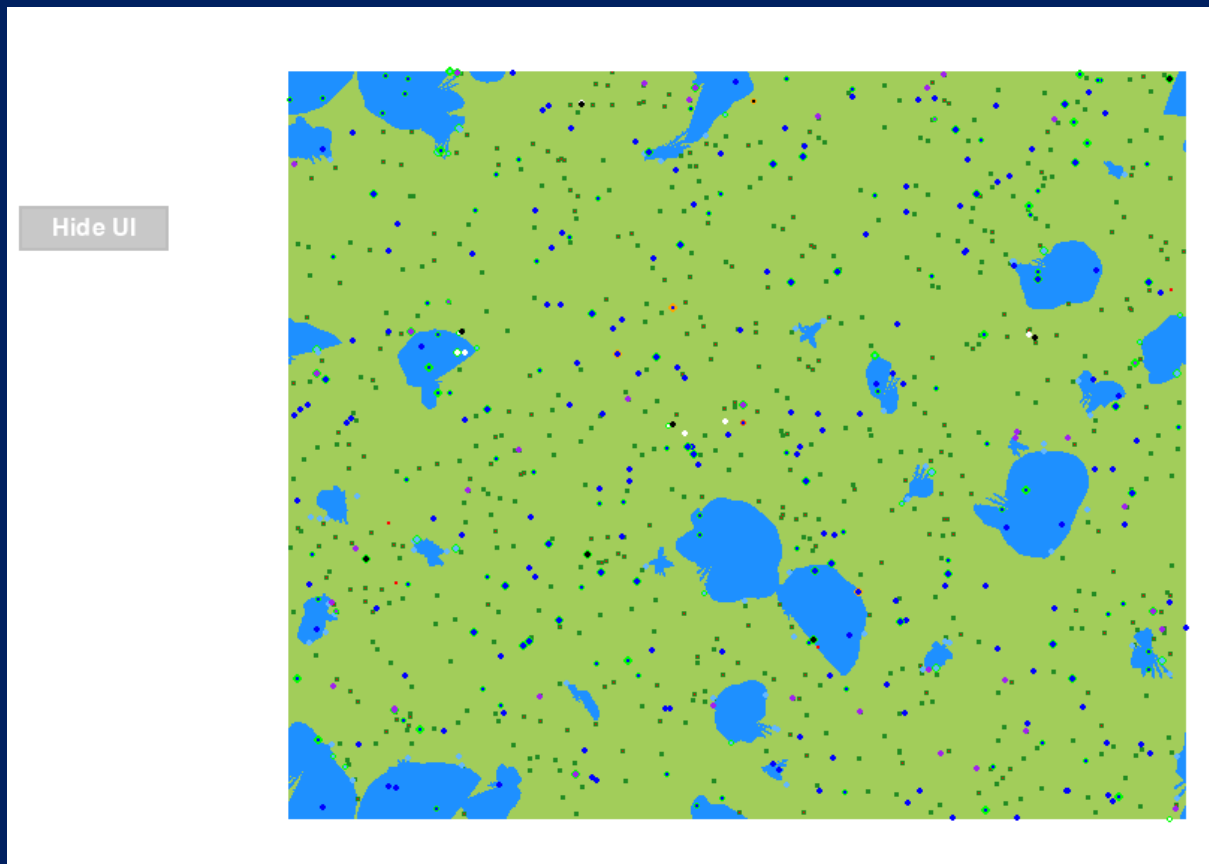
I tested all the buttons for the correct outputs and each test was positive for the correct result.



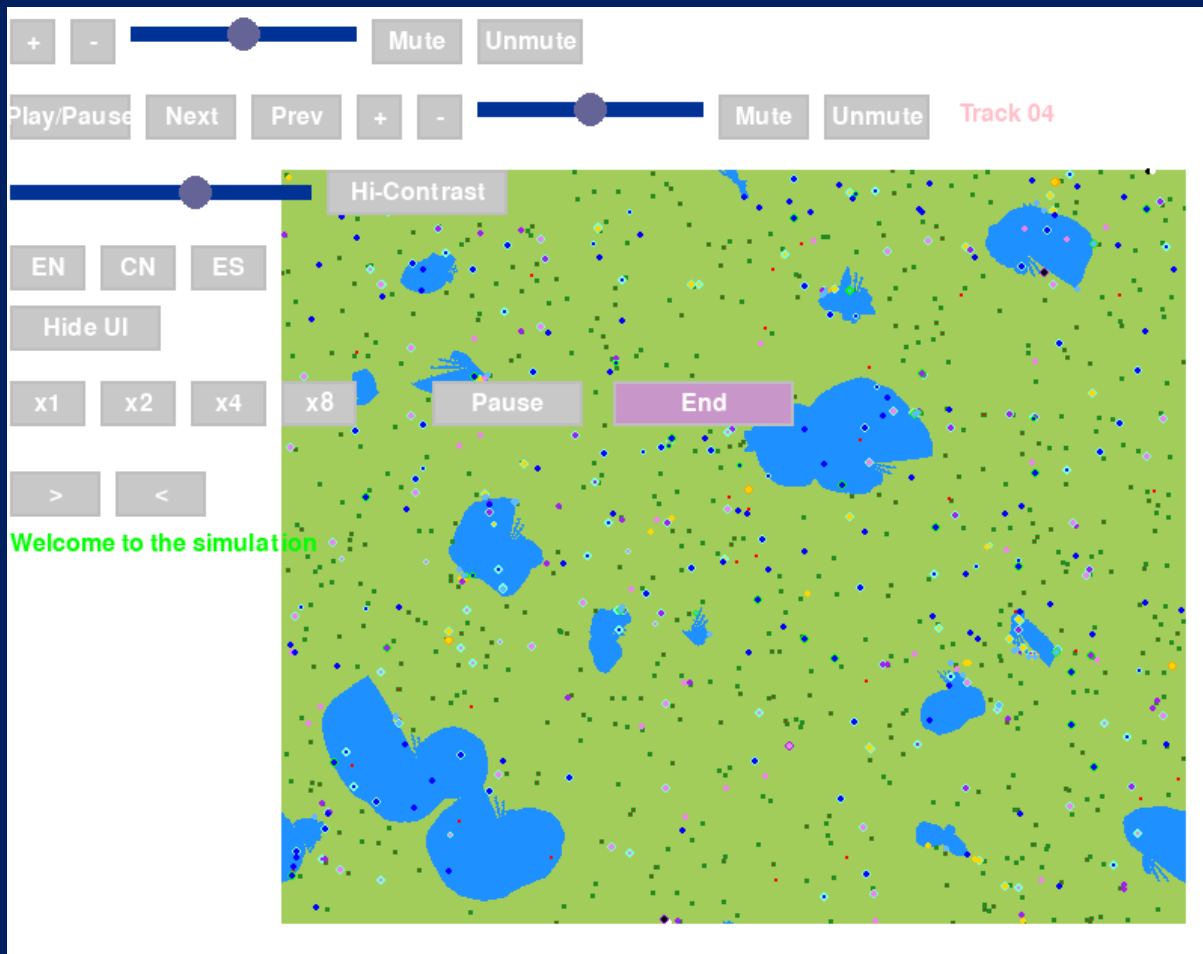
In the example above the UI displays correctly as it is not hidden; buttons are the correct colour- most are grey save for the pause button as the mouse is currently hovering over it causing it to be filled with the purple-tinted hover colour.

The speed buttons work as intended as the maximum frame rate scales proportionally with the value displayed on the button selected (of course, the processor performance upper limit can be a bottleneck for high numbers of anima)

However the sliders still do not function properly. They clearly do update each tick as using the buttons attached to the same effect will update the position of the slider knob; however the knob cannot be dragged.



Here is the screen when the UI is hidden.



Pressing the 'end' button closes the pygame window and ends the simulation.

I made the ExtraTraits class which belongs in the file 'extratraits.py'. Genome extra traits will now be stored not as a list of strings, but an ExtraTraits object with a self.traits set attribute; I use a set here because the order of elements in the data structure does not matter. Special methods are used so that the same code that was used before still works on ExtraTraits objects as they did on the list. The method 'correct_traits' has the role of limiting animas to only one of each extra trait type; currently the only such type is attacks; these do not include the 'inky-shot' trait but I could adjust this. More extra traits have been implemented such as 'inky-shot', 'sand-attack' and 'spiritful-child'.


```

extratraits.py x
1  import random
2
3  class ExtraTraits():
4      def __init__(self, trait_set):
5          self.traits = trait_set
6
7      def __contains__(self, trait):
8          return trait in self.traits
9
10     def __str__(self):
11         return str(self.traits)
12
13     def add(self, trait):
14         self.traits.add(trait)
15
16     def correct_traits(self):
17         attacks = self.traits & {"claws", "pincers", "venomous", "sand-attack"}
18
19         while len(attacks) > 1:
20             cull = random.choice(attacks)
21             self.traits.remove(cull)
22             attacks.remove(cull)
23

```

Thots [apple], 7.7 7.2 is boosting to get away from Brurro [meat], 12.2 8.7

I fixed various algorithms to account for realm grid looping around at each edge: this includes pool generation, calculation of angle to target, and calculation of distance between cells (now distance and displacement are both implemented such that they are always calculated taking the shortest route between cells including looping around the edge).

I began implementing the code allowing for individual anima selection.

16/04:

More extra traits have been introduced, alongside new anima conditions and state attributes.

22/04:

There was an error when changing the music track:

```

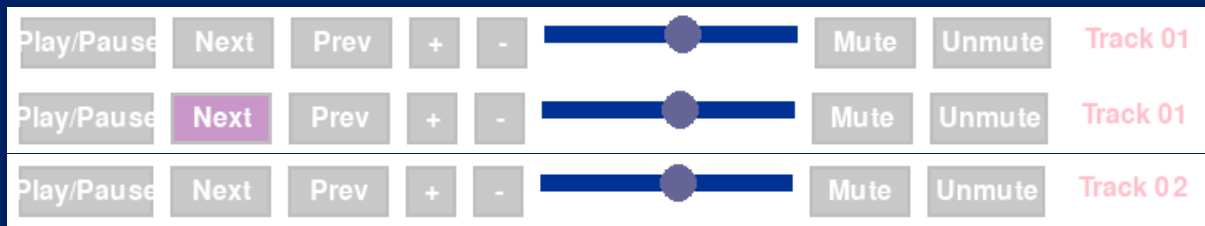
Traceback (most recent call last):
  File "C:\Users\ethan\OneDrive\Documents\projectsim 2\projectsim 2\main.py", line 146, in <module>
    main()
  File "C:\Users\ethan\OneDrive\Documents\projectsim 2\projectsim 2\main.py", line 98, in main
    ui.handle_ui(event, ui.myShell, window, testRealm)
  File "C:\Users\ethan\OneDrive\Documents\projectsim 2\projectsim 2\ui_design.py", line 138, in handle_ui
    play_music(shell)
  File "C:\Users\ethan\OneDrive\Documents\projectsim 2\projectsim 2\ui_design.py", line 83, in play_music
    pygame.mixer.music.load(tracks[shell.track_index])
pygame.error: No file '00664805mp3' found in working directory 'C:\Users\ethan\OneDrive\Documents\projectsim 2\projectsim 2'.

```

This was fixed by correcting the stored filename from "00664805mp3" to "00664805.mp3".

Now, the music track can be changed with the track change buttons.

When pressing "next", the next track begins to play:



Likewise pressing “prev” causes the previous track to play:

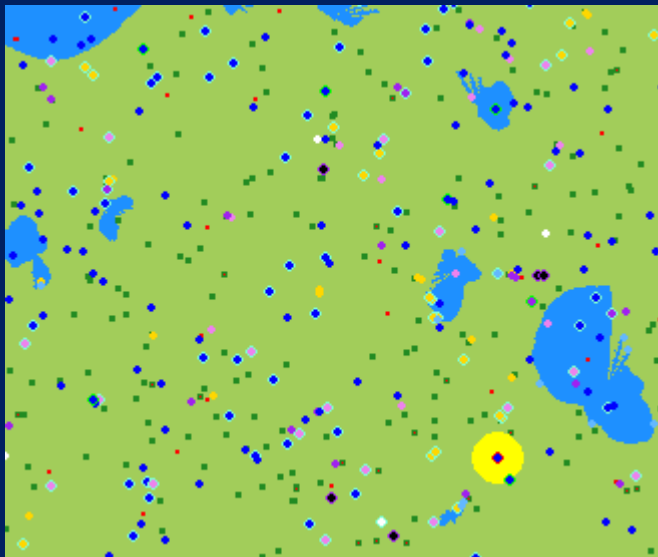


This further shows that the frame-by-frame event handling works with this implementation. The only issue is that the adjusting the speed of the simulation likewise adjusts the rate of UI handling; this is very minor however and will hardly be felt by users considering even the slowest rates operate at one frame each fifth-of-a-second.

This is what a highlighted anima looks like with a red highlight colour:



The selected anima was incorrect; in the following case I pressed the screen with my cursor far to the left and above of the selected anima:



In fact, selected animas were offset by some fixed horizontal and vertical components. In order to fix this bug I had to revisit the selection algorithm:

```
mx, my = pygame.mouse.get_pos()
    if left < mx < right and top < my < bottom:
        shell.anima_to_view = None
```

```

nearest_anima = None
nearest_dist = anima_selection_range

for anima in realm.elements[1] + realm.graveyard[1]:
    dist = anima.distance_to_point(mx, my)
    if dist <= nearest_dist:
        nearest_anima = anima
        nearest_dist = dist
shell.anima_to_view = nearest_anima

```

The issue was that the arguments passed to `anima.distance_to_point()` were the relative x and y values of the cursor on the screen, while `anima.x` and `anima.y` were relative to the smaller simulation window which does not begin at (0, 0) on the larger screen. Therefore the offset of the logic was equal to the offset of the window from the origin (which is 200 in the x direction and 120 in the y direction). So I simply had to correct the offset by changing the offending line from:

```
dist = anima.distance_to_point(mx, my)
```

To:

```
dist = anima.distance_to_point(mx-200, my-120)
```

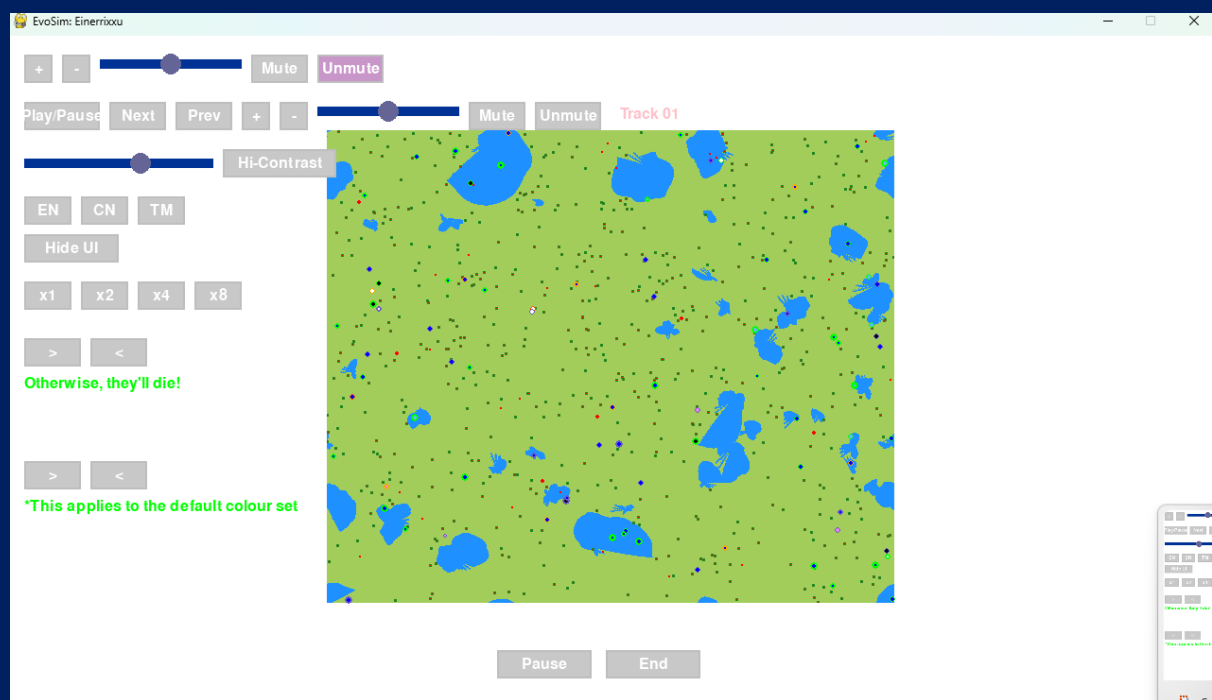
After this, however, I would adjust these numbers as I moved some things around the layout.

I made the fact file rect for the individually displayed anima.

The fact file is very useful both a usability feature for the user which allows them to perform both light and in-depth analysis on the creatures, and as a development feature for myself, to use when testing; up until now I had the database outputs for general population data, but now I could look at individual animas and see how they squared up to one another as individuals with unique traits.

This is how the fact file looks:

Apog
 ID 794
 LOC : [367.9|176.5],
 AGE : 9
 STATUS : [4 6.6|28.8|50.5|0.1|-999],
 GOALS : [0],
 WANDERING
 0.53|0.23],], [32.5|7.5|6.5|4.8|40.5|
 CNK : Apog [apple], set() 6.5 4.8
 GOALS : [0],



(The UI after a few minor improvements : now 27/04)

30/04:

Here is an error:

File "C:\Users\echeng_b19\OneDrive - King Edward VI Grammar School\2024-2026\CompSci NEA Y13\projectsim+\projectsim 2\anima.py", line 846, in mix_genes

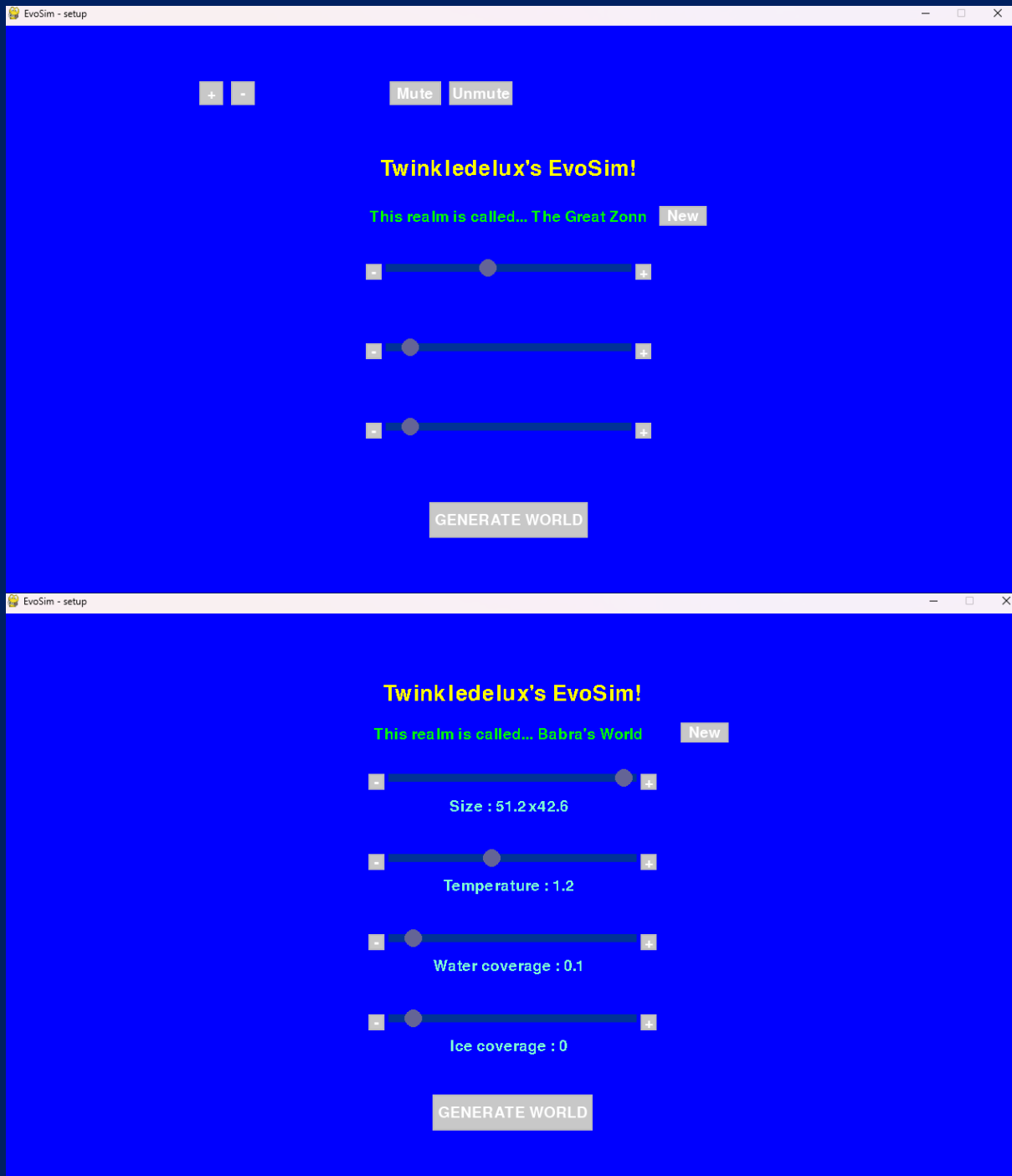
```
new_gene = new_gene + (random.choice([food for food in self.realm_foods() if food not in
new_gene]),)
```

```
^^^^
```

NameError: name 'self' is not defined

I fixed this by passing the food list into the function so the original functionality could be preserved alongside the new one. By default, the original applies as the anima module food_list is passed in.

02/05:



The following video demonstrates how the realm name can be generated using the random name generator which I built (during anima development):



EvoSim - setup 2026-05-02 22-29-23.mp4

See attached video : "Setup screen"

Post-Testing

****This is the post-phase testing, but there has been abundant testing throughout development which can be seen in the previous section. The purpose of this section is to formalise testing which was completed after the phase's development was finalised in order to ensure the complete package functions as intended; a more holistic approach will be taken****

Test realm with default starting conditions:

```
Extra Traits [[egg-laying|17], |[claws|21], |[flippers|16], |[abrupt-wanderer|21], |[poisonous|43],
|[venomous|117], |[pincers|40], |[snowycloak|35], |[strong-gut|123], |[sand-attack|18],
|[spiritful-child|20], |[inky-shot|3], |[berserker|402], |[vampirism|117], |[zombie|19],
|[gluttonous-child|2], |[necro-mimicry|231], |[absorb-victim|113], |[mimic-attack|11],
|[leviathan|33], |[play-dead|47], |[plate-armour|30], |[growth-syndrome|6], ],
//////////
//////////
//////////
[[0|400|30], |[25|396|29], |[50|384|23], |[75|377|21], |[100|374|16], |[125|377|15],
|[150|394|11], |[175|433|11], |[200|477|9], |[225|516|9], |[250|586|9], |[275|640|8],
|[300|703|6], |[325|748|6], |[350|782|4], |[375|819|3], |[400|867|4], |[425|922|4],
|[450|962|3], |[475|941|3], |[500|901|3], |[525|825|3], |[550|773|3], |[575|716|3],
|[600|675|3], |[625|635|3], |[650|626|5], |[675|640|4], |[700|659|4], |[725|684|3],
|[750|708|4], |[775|762|3], |[800|802|2], |[825|809|2], |[850|822|2], |[875|835|1],
|[900|862|1], |[925|872|1], |[950|880|1], |[975|890|1], |[1000|894|1], |[1025|882|1],
|[1050|878|1], |[1075|825|1], |[1100|838|0], |[1125|839|0], |[1150|817|0], |[1175|800|0],
|[1200|788|0], |[1225|787|0], |[1250|797|0], |[1275|794|0], |[1300|821|0], |[1325|835|0],
|[1350|876|0], |[1375|924|0], |[1400|954|0], |[1425|996|0], |[1450|1004|0], |[1475|1007|0],
|[1500|984|0], |[1525|972|0], |[1550|959|0], |[1575|945|0], |[1600|926|0], |[1625|908|0],
|[1650|896|0], |[1675|911|0], |[1700|904|0], |[1725|927|0], |[1750|924|0], |[1775|917|0],
|[1800|906|0], |[1825|884|0], |[1850|888|0], |[1875|854|0], |[1900|869|0], |[1925|863|0],
|[1950|881|0], |[1975|901|0], |[2000|880|0], |[2025|888|0], |[2050|906|0], |[2075|889|0],
|[2100|908|0], |[2125|907|0], |[2150|901|0], |[2175|887|0], |[2200|876|0], |[2225|871|0],
|[2250|862|0], |[2275|863|0], |[2300|864|0], |[2325|863|0], |[2350|870|0], |[2375|889|0],
|[2400|898|0], |[2425|927|0], ],
//////////
//////////
//////////
[8005|17|0|17|0],
//////////
//////////
//////////
[[0|[[apple], |[3.0|[1.0|0.5|0.3], ], |[22.5|9.9|7.0|6.9|53.9|0.5|64.2|120.0|27.4|N/A], |[meat],
|[3.8|[1.0|0.5|0.3], ], |[22.3|9.4|9.1|10.0|60.6|0.49|47.3|127.1|31.9|N/A], ], |[25|[[apple],
|[2.9|[1.0|0.5|0.3], ], |[22.6|9.9|7.0|6.9|53.9|0.5|64.4|119.9|27.3|N/A], |[meat],
|[3.9|[1.0|0.5|0.3], ], |[22.4|9.3|8.9|10.2|61.1|0.48|46.5|127.2|31.5|N/A], ], |[50|[[apple],
|[2.9|[1.0|0.5|0.3], ], |[22.6|9.9|7.0|7.0|53.8|0.5|64.1|119.4|27.5|N/A], |[meat],
|[3.2|[1.0|0.5|0.3], ], |[22.7|9.1|9.1|10.2|63.7|0.46|44.8|125.1|29.9|N/A], ], |[75|[[apple],
|[2.9|[1.0|0.5|0.3], ], |[22.6|10.0|7.0|6.9|53.7|0.5|64.4|117.4|27.4|N/A], |[meat],
|[3.0|[1.0|0.5|0.3], ], |[22.1|8.8|9.4|10.0|64.3|0.45|44.1|124.8|30.5|N/A], ], |[100|[[apple],
|[2.9|[1.0|0.5|0.3], ], |[22.6|10.1|7.0|7.0|53.9|0.49|64.8|114.7|27.2|N/A], |[meat],
|[2.6|[1.0|0.5|0.3], ], |[21.2|8.5|9.7|10.2|63.8|0.48|41.7|123.4|29.2|N/A], ], |[125|[[apple],
```

[2.9 [1.0 0.5 0.3],	],	22.7 10.2 7.1 6.9 54.3 0.49 64.8 112.5 27.0 N/A ,	[[[meat],	
[2.7 [1.0 0.51 0.3],	],	21.2 8.5 9.6 9.9 65.8 0.48 44.1 126.9 30.4 N/A ,	],	[150 [[apple],
[2.9 [1.0 0.5 0.3],	],	22.7 10.2 7.0 7.0 54.5 0.49 65.4 109.3 26.8 N/A ,	[[[meat],	
[2.6 [1.0 0.51 0.3],	],	20.1 9.0 9.6 10.7 64.8 0.5 40.8 118.6 30.0 N/A ,	],	[175 [[apple],
[2.9 [1.0 0.5 0.3],	],	22.7 10.4 7.1 7.0 54.3 0.48 65.3 107.5 27.0 N/A ,	[[[meat],	
[2.7 [1.0 0.51 0.3],	],	20.8 9.8 8.9 10.4 62.4 0.57 38.6 121.2 30.8 N/A ,	],	[200 [[apple],
[2.9 [1.0 0.5 0.3],	],	22.8 10.6 7.1 7.0 54.0 0.49 65.0 108.1 26.8 N/A ,	[[[meat],	
[2.7 [1.0 0.51 0.3],	],	20.6 10.1 8.3 10.5 59.1 0.49 37.3 107.2 29.5 N/A ,	],	[225 [[apple],
[2.9 [1.0 0.5 0.3],	],	22.8 10.6 7.1 7.0 54.0 0.48 64.6 107.3 26.8 N/A ,	[[[meat],	
[2.4 [1.0 0.5 0.3],	],	21.4 9.8 8.6 10.3 59.2 0.48 39.2 115.7 30.2 N/A ,	],	[250 [[apple],
[2.9 [1.0 0.5 0.3],	],	22.8 10.6 7.1 7.0 54.1 0.48 64.8 106.9 26.4 N/A ,	[[[meat],	
[2.4 [1.0 0.5 0.3],	],	21.4 9.8 8.6 10.3 59.2 0.48 39.2 115.7 30.2 N/A ,	],	[275 [[apple],
[2.9 [1.0 0.5 0.3],	],	22.8 10.6 7.1 7.0 53.9 0.48 64.7 106.5 26.4 N/A ,	[[[meat],	
[2.1 [1.0 0.5 0.3],	],	20.6 10.3 9.0 10.1 57.9 0.5 41.4 107.9 29.4 N/A ,	],	[300 [[apple],
[2.9 [1.0 0.5 0.3],	],	22.8 10.6 7.1 7.0 53.9 0.47 64.7 106.1 26.5 N/A ,	[[[meat],	
[2.1 [1.0 0.5 0.3],	],	22.2 10.8 7.6 10.3 59.9 0.6 43.1 115.5 34.0 N/A ,	],	[325 [[apple],
[2.9 [1.0 0.5 0.3],	],	22.8 10.7 7.1 7.0 54.0 0.47 64.9 105.3 26.4 N/A ,	[[[meat],	
[2.1 [1.0 0.5 0.3],	],	22.2 10.8 7.6 10.3 59.9 0.6 43.1 115.5 34.0 N/A ,	],	[350 [[apple],
[2.9 [1.0 0.5 0.3],	],	22.8 10.8 7.1 7.0 54.1 0.48 65.3 104.9 26.3 N/A ,	[[[meat],	
[1.7 [1.0 0.49 0.3],	],	20.6 10.7 7.4 10.9 55.8 0.55 41.1 113.9 32.5 N/A ,	],	[375 [[apple],
[2.9 [1.0 0.5 0.3],	],	22.7 10.9 7.1 7.0 54.1 0.48 65.3 105.4 26.1 N/A ,	[[[meat],	
[1.7 [1.0 0.49 0.3],	],	22.4 12.1 7.4 9.5 59.3 0.68 43.1 123.8 38.6 N/A ,	],	[400 [[apple],
[2.9 [1.0 0.5 0.3],	],	22.6 10.9 7.1 7.0 53.5 0.48 65.3 104.7 26.3 N/A ,	[[[meat],	
[2.0 [1.0 0.51 0.32],	],	22.6 13.2 7.0 8.9 55.3 0.66 47.5 111.7 34.5 N/A ,	],	[425 [[apple],
[2.9 [1.0 0.5 0.3],	],	22.5 11.0 7.1 7.0 53.4 0.48 65.3 104.1 26.4 N/A ,	[[[meat],	
[2.0 [1.0 0.51 0.32],	],	22.6 13.2 7.0 8.9 55.3 0.66 47.5 111.7 34.5 N/A ,	],	[450 [[apple],
[2.8 [1.0 0.5 0.3],	],	22.5 11.1 7.1 7.0 53.4 0.48 65.3 103.0 26.4 N/A ,	[[[meat],	
[2.3 [1.0 0.51 0.32],	],	24.8 12.9 6.8 8.3 56.8 0.64 44.1 111.9 31.1 N/A ,	],	[475 [[apple],
[2.8 [1.0 0.5 0.3],	],	22.4 11.1 7.1 7.0 53.4 0.48 65.2 102.0 26.4 N/A ,	[[[meat],	
[2.3 [1.0 0.51 0.32],	],	24.8 12.9 6.8 8.3 56.8 0.64 44.1 111.9 31.1 N/A ,	],	[500 [[apple],
[2.8 [1.0 0.5 0.3],	],	22.4 11.1 7.1 7.0 53.0 0.48 65.2 102.0 26.7 N/A ,	[[[meat],	
[2.3 [1.0 0.51 0.32],	],	24.8 12.9 6.8 8.3 56.8 0.64 44.1 111.9 31.1 N/A ,	],	[525 [[apple],
[2.8 [1.0 0.5 0.3],	],	22.4 11.1 7.1 7.0 52.8 0.48 65.5 101.1 26.7 N/A ,	[[[meat],	
[2.3 [1.0 0.51 0.32],	],	24.8 12.9 6.8 8.3 56.8 0.64 44.1 111.9 31.1 N/A ,	],	[550 [[apple],
[2.8 [1.0 0.5 0.3],	],	22.4 11.2 7.2 7.0 52.8 0.48 65.5 100.3 26.7 N/A ,	[[[meat],	
[2.3 [1.0 0.51 0.32],	],	24.8 12.9 6.8 8.3 56.8 0.64 44.1 111.9 31.1 N/A ,	],	[575 [[apple],
[2.8 [1.0 0.5 0.3],	],	22.3 11.2 7.1 7.0 52.5 0.48 65.6 99.8 26.6 N/A ,	[[[meat],	
[2.3 [1.0 0.51 0.32],	],	24.8 12.9 6.8 8.3 56.8 0.64 44.1 111.9 31.1 N/A ,	],	[600 [[apple],
[2.8 [1.0 0.5 0.3],	],	22.3 11.3 7.1 6.9 52.9 0.48 66.0 98.9 26.6 N/A ,	[[[meat],	
[2.3 [1.0 0.51 0.32],	],	24.8 12.9 6.8 8.3 56.8 0.64 44.1 111.9 31.1 N/A ,	],	[625 [[apple],
[2.8 [1.0 0.5 0.3],	],	22.1 11.4 7.0 6.9 52.6 0.48 66.3 98.0 26.5 N/A ,	[[[meat],	
[2.5 [1.0 0.53 0.33],	],	24.0 13.1 7.1 7.3 52.9 0.57 60.2 113.9 28.7 N/A ,	],	[650 [[apple],
[2.8 [1.0 0.5 0.3],	],	22.1 11.5 7.0 6.9 52.5 0.47 66.6 97.4 26.5 N/A ,	[[[meat],	
[2.8 [1.0 0.5 0.35],	],	22.8 13.9 7.1 7.3 48.2 0.55 64.6 106.4 27.6 N/A ,	],	[675 [[apple],
[2.8 [1.0 0.49 0.3],	],	22.0 11.6 7.0 6.9 52.7 0.48 66.4 96.7 26.5 N/A ,	[[[meat],	
[2.7 [1.0 0.49 0.36],	],	22.8 13.5 7.3 7.7 50.8 0.57 57.7 107.3 28.6 N/A ,	],	[700 [[apple],
[2.8 [1.0 0.49 0.3],	],	22.0 11.6 7.0 6.9 52.3 0.48 66.7 95.9 26.4 N/A ,	[[[meat],	
[2.7 [1.0 0.49 0.36],	],	22.8 13.5 7.3 7.7 50.8 0.57 57.7 107.3 28.6 N/A ,	],	[725 [[apple],
[2.8 [1.0 0.49 0.3],	],	22.2 11.6 7.0 6.9 52.4 0.48 67.1 95.6 26.3 N/A ,	[[[meat],	
[2.6 [1.0 0.51 0.35],	],	24.3 12.5 7.3 7.5 52.6 0.6 54.6 114.6 28.4 N/A ,	],	[750 [[apple],
[2.8 [1.0 0.49 0.29],	],	22.1 11.6 6.9 6.9 52.3 0.47 67.3 95.6 26.1 N/A ,	[[[meat],	

```

|[2.7|[1.0|0.48|0.34], ], |24.4|12.9|7.3|7.9|52.6|0.57|57.6|108.8|27.8|N/A], ], |[775|[[apple],
|[2.8|[1.0|0.49|0.29], ], |22.1|11.6|6.9|6.9|52.4|0.47|67.3|95.8|26.0|N/A], |[[meat],
|[3.1|[1.0|0.48|0.35], ], |23.7|14.8|6.7|7.4|44.4|0.57|67.6|90.6|23.6|N/A], ], |[800|[[apple],
|[2.8|[1.0|0.49|0.29], ], |22.1|11.6|6.9|6.9|52.3|0.48|67.4|95.9|26.2|N/A], |[[meat],
|[3.0|[1.0|0.51|0.38], ], |23.4|15.2|6.5|6.6|40.1|0.6|68.1|90.2|22.5|N/A], ], |[825|[[apple],
|[2.8|[1.0|0.49|0.3], ], |22.1|11.6|6.9|6.9|52.2|0.48|67.3|95.1|26.1|N/A], |[[meat],
|[3.0|[1.0|0.51|0.38], ], |23.4|15.2|6.5|6.6|40.1|0.6|68.1|90.2|22.5|N/A], ], |[850|[[apple],
|[2.8|[1.0|0.49|0.29], ], |22.1|11.7|6.9|6.9|52.1|0.47|67.3|95.2|26.0|N/A], |[[meat],
|[3.0|[1.0|0.51|0.38], ], |23.4|15.2|6.5|6.6|40.1|0.6|68.1|90.2|22.5|N/A], ], |[875|[[apple],
|[2.7|[1.0|0.49|0.3], ], |22.2|11.7|6.9|6.8|52.2|0.47|67.0|95.4|26.0|N/A], |[[meat],
|[3.2|[1.0|0.46|0.38], ], |23.5|13.8|7.3|6.3|36.9|0.58|75.4|105.1|22.9|N/A], ], |[900|[[apple],
|[2.8|[1.0|0.49|0.29], ], |22.2|11.7|6.9|6.8|52.4|0.47|66.9|95.3|25.8|N/A], |[[meat],
|[3.2|[1.0|0.46|0.38], ], |23.5|13.8|7.3|6.3|36.9|0.58|75.4|105.1|22.9|N/A], ], |[925|[[apple],
|[2.8|[1.0|0.49|0.29], ], |22.1|11.7|6.9|6.8|52.4|0.47|66.8|94.6|25.6|N/A], |[[meat],
|[3.2|[1.0|0.46|0.38], ], |23.5|13.8|7.3|6.3|36.9|0.58|75.4|105.1|22.9|N/A], ], |[950|[[apple],
|[2.8|[1.0|0.49|0.29], ], |22.2|11.7|6.9|6.8|52.4|0.47|66.2|94.1|25.5|N/A], |[[meat],
|[3.2|[1.0|0.46|0.38], ], |23.5|13.8|7.3|6.3|36.9|0.58|75.4|105.1|22.9|N/A], ], |[975|[[apple],
|[2.8|[1.0|0.49|0.29], ], |22.2|11.8|6.9|6.9|52.6|0.47|66.1|93.6|25.5|N/A], |[[meat],
|[3.2|[1.0|0.46|0.38], ], |23.5|13.8|7.3|6.3|36.9|0.58|75.4|105.1|22.9|N/A], ], |[1000|[[apple],
|[2.8|[1.0|0.49|0.29], ], |22.2|11.8|6.9|6.9|52.5|0.47|66.2|93.6|25.5|N/A], |[[meat],
|[3.2|[1.0|0.46|0.38], ], |23.5|13.8|7.3|6.3|36.9|0.58|75.4|105.1|22.9|N/A], ], |[1025|[[apple],
|[2.8|[1.0|0.49|0.29], ], |22.2|11.8|6.9|6.9|52.4|0.47|65.9|93.1|25.4|N/A], |[[meat],
|[3.2|[1.0|0.46|0.38], ], |23.5|13.8|7.3|6.3|36.9|0.58|75.4|105.1|22.9|N/A], ], |[1050|[[apple],
|[2.7|[1.0|0.49|0.29], ], |22.2|11.8|6.9|6.8|52.5|0.47|65.9|92.9|25.4|N/A], |[[meat],
|[3.2|[1.0|0.46|0.38], ], |23.5|13.8|7.3|6.3|36.9|0.58|75.4|105.1|22.9|N/A], ], |[1075|[[apple],
|[2.7|[1.0|0.49|0.29], ], |22.2|11.8|7.0|6.8|52.4|0.47|65.6|92.3|25.4|N/A], |[[meat],
|[3.2|[1.0|0.46|0.38], ], |23.5|13.8|7.3|6.3|36.9|0.58|75.4|105.1|22.9|N/A], ], |[1100|[[apple],
|[2.7|[1.0|0.49|0.29], ], |22.1|11.9|7.0|6.8|52.3|0.47|66.0|92.0|25.3|N/A], |[ ], ],
|[1125|[[apple], |[2.7|[1.0|0.49|0.29], ], |22.1|11.9|7.0|6.8|52.2|0.47|65.8|92.3|25.4|N/A],
|[ ], ], |[1150|[[apple], |[2.7|[1.0|0.49|0.29], ], |22.2|11.9|7.0|6.8|52.2|0.47|65.6|92.2|25.2|N/A],
|[ ], ], |[1175|[[apple], |[2.7|[1.0|0.49|0.29], ], |22.3|11.9|7.0|6.8|52.0|0.47|65.6|91.7|25.1|N/A],
|[ ], ], |[1200|[[apple], |[2.7|[1.0|0.49|0.29], ], |22.1|11.9|7.0|6.8|51.8|0.47|65.8|92.0|25.1|N/A],
|[ ], ], |[1225|[[apple], |[2.7|[1.0|0.49|0.29], ], |22.0|11.9|7.0|6.7|51.8|0.47|65.8|91.7|25.1|N/A],
|[ ], ], |[1250|[[apple], |[2.7|[1.0|0.49|0.29], ], |22.1|11.9|7.0|6.7|51.6|0.47|66.0|91.6|25.0|N/A],
|[ ], ], |[1275|[[apple], |[2.7|[1.0|0.49|0.29], ], |22.0|12.0|7.0|6.7|51.7|0.47|66.0|91.7|25.0|N/A],
|[ ], ], |[1300|[[apple], |[2.7|[1.0|0.49|0.29], ], |22.1|12.0|7.0|6.7|51.6|0.47|66.0|91.4|24.9|N/A],
|[ ], ], |[1325|[[apple], |[2.7|[1.0|0.49|0.3], ], |22.1|12.0|7.0|6.7|51.6|0.47|66.0|91.1|24.9|N/A],
|[ ], ], |[1350|[[apple], |[2.7|[1.0|0.49|0.29], ], |22.0|12.0|7.0|6.7|51.3|0.47|66.1|90.9|25.1|N/A],
|[ ], ], |[1375|[[apple], |[2.7|[1.0|0.49|0.29], ], |22.0|12.0|7.0|6.7|51.3|0.47|66.5|91.0|25.2|N/A],
|[ ], ], |[1400|[[apple], |[2.7|[1.0|0.49|0.29], ], |22.0|12.1|7.0|6.6|51.4|0.47|66.3|90.4|25.2|N/A],
|[ ], ], |[1425|[[apple], |[2.7|[1.0|0.49|0.29], ], |22.0|12.1|7.0|6.6|51.4|0.48|66.4|90.4|25.2|N/A],
|[ ], ], |[1450|[[apple], |[2.7|[1.0|0.49|0.29], ], |22.0|12.1|6.9|6.6|51.3|0.48|66.2|90.4|25.0|N/A],
|[ ], ], |[1475|[[apple], |[2.7|[1.0|0.49|0.3], ], |22.0|12.1|6.9|6.6|51.5|0.48|66.4|89.9|25.1|N/A],
|[ ], ], |[1500|[[apple], |[2.7|[1.0|0.49|0.3], ], |21.9|12.2|6.9|6.6|51.6|0.48|66.3|90.3|24.9|N/A],
|[ ], ], |[1525|[[apple], |[2.7|[1.0|0.49|0.29], ], |21.8|12.2|6.9|6.6|51.5|0.48|66.3|90.2|24.9|N/A],
|[ ], ], |[1550|[[apple], |[2.7|[1.0|0.49|0.3], ], |21.9|12.3|6.9|6.6|51.7|0.48|66.5|89.9|24.9|N/A], |[ ],

```



```

], |[1575|[[apple], |[2.7|[1.0|0.49|0.3], ], |22.0|12.4|6.9|6.7|51.7|0.48|66.7|89.6|24.9|N/A],
|[], ], |[1600|[[apple], |[2.7|[1.0|0.49|0.3], ], |21.9|12.4|6.8|6.7|51.9|0.48|66.5|90.0|24.9|N/A], |[], ], |[1625|[[apple], |[2.7|[1.0|0.49|0.3],
], |22.0|12.4|6.8|6.7|51.9|0.48|66.8|90.0|24.9|N/A], |[], ], |[1650|[[apple], |[2.7|[1.0|0.49|0.3], ], |22.0|12.5|6.8|6.6|51.9|0.48|66.8|89.8|24.8|N/A], |[], ],
|[1675|[[apple], |[2.7|[1.0|0.49|0.29], ], |21.9|12.4|6.8|6.6|52.2|0.48|66.8|89.2|24.9|N/A], |[], ], |[1700|[[apple], |[2.7|[1.0|0.49|0.29], ], |22.0|12.5|6.8|6.6|52.2|0.48|66.7|89.0|24.9|N/A], |[], ], |[1725|[[apple], |[2.7|[1.0|0.49|0.3],
], |21.9|12.5|6.7|6.6|52.3|0.48|66.6|89.0|24.8|N/A], |[], ], |[1750|[[apple], |[2.7|[1.0|0.49|0.3], ], |21.8|12.5|6.7|6.6|52.2|0.49|66.4|89.1|24.8|N/A], |[], ],
|[1775|[[apple], |[2.7|[1.0|0.49|0.29], ], |21.8|12.6|6.7|6.6|52.3|0.49|66.5|88.6|24.8|N/A], |[], ], |[1800|[[apple], |[2.6|[1.0|0.49|0.29], ], |21.9|12.7|6.8|6.6|52.5|0.49|66.5|88.6|24.7|N/A], |[], ], |[1825|[[apple], |[2.7|[1.0|0.49|0.3],
], |21.9|12.7|6.7|6.6|52.6|0.49|66.8|88.5|24.9|N/A], |[], ], |[1850|[[apple], |[2.7|[1.0|0.49|0.3], ], |21.9|12.8|6.8|6.6|52.5|0.49|66.5|88.3|24.9|N/A], |[], ],
|[1875|[[apple], |[2.7|[1.0|0.49|0.3], ], |21.9|12.8|6.8|6.6|52.4|0.49|66.4|87.9|25.0|N/A], |[], ], |[1900|[[apple], |[2.7|[1.0|0.49|0.3], ], |21.9|12.9|6.8|6.6|52.4|0.49|66.4|88.0|25.1|N/A],
|[], ], |[1925|[[apple], |[2.7|[1.0|0.49|0.3], ], |21.9|12.9|6.8|6.6|52.5|0.49|66.1|88.1|25.2|N/A], |[], ], |[1950|[[apple], |[2.6|[1.0|0.49|0.3], ], |21.9|13.0|6.8|6.6|52.4|0.49|66.0|87.8|25.1|N/A], |[], ], |[1975|[[apple],
|[2.6|[1.0|0.49|0.3], ], |21.9|13.0|6.8|6.6|52.4|0.49|66.0|87.5|25.1|N/A], |[], ], |[2000|[[apple], |[2.6|[1.0|0.49|0.3], ], |22.0|13.0|6.8|6.6|52.3|0.49|66.0|87.6|25.1|N/A], |[], ],
|[2025|[[apple], |[2.6|[1.0|0.49|0.29], ], |21.9|13.1|6.7|6.6|52.3|0.49|66.1|87.7|25.1|N/A], |[], ], |[2050|[[apple], |[2.6|[1.0|0.49|0.29], ], |22.0|13.2|6.7|6.6|52.4|0.49|66.1|87.1|25.1|N/A], |[], ], |[2075|[[apple],
|[2.6|[1.0|0.49|0.29], ], |21.9|13.2|6.7|6.6|52.5|0.49|66.1|87.3|25.0|N/A], |[], ], |[2100|[[apple], |[2.6|[1.0|0.49|0.29], ], |21.8|13.2|6.7|6.6|52.6|0.48|66.2|87.4|25.0|N/A],
|[], ], |[2125|[[apple], |[2.6|[1.0|0.49|0.29], ], |21.8|13.3|6.7|6.6|52.5|0.48|66.0|87.2|25.1|N/A], |[], ], |[2150|[[apple], |[2.6|[1.0|0.49|0.29], ], |21.8|13.3|6.7|6.6|52.9|0.48|66.1|87.1|25.0|N/A], |[], ],
|[2175|[[apple], |[2.6|[1.0|0.49|0.29], ], |21.8|13.3|6.7|6.6|53.1|0.48|66.1|86.9|25.0|N/A], |[], ], |[2200|[[apple], |[2.6|[1.0|0.49|0.29], ], |21.8|13.3|6.7|6.6|53.3|0.48|66.2|86.5|25.0|N/A], |[], ], |[2225|[[apple],
|[2.6|[1.0|0.49|0.29], ], |21.9|13.4|6.6|6.5|53.3|0.48|66.1|86.1|25.0|N/A], |[], ], |[2250|[[apple], |[2.6|[1.0|0.49|0.29], ], |21.8|13.4|6.6|6.5|53.4|0.48|66.2|86.1|25.0|N/A],
|[], ], |[2275|[[apple], |[2.6|[1.0|0.5|0.29], ], |21.9|13.5|6.6|6.5|53.5|0.48|66.0|85.5|25.0|N/A], |[], ], |[2300|[[apple], |[2.6|[1.0|0.5|0.29], ], |21.9|13.5|6.5|6.5|53.4|0.48|65.7|85.3|25.0|N/A], |[], ], |[2325|[[apple],
|[2.6|[1.0|0.5|0.29], ], |21.9|13.5|6.5|6.5|53.5|0.48|65.7|85.2|24.9|N/A], |[], ], |[2350|[[apple], |[2.6|[1.0|0.5|0.29], ], |22.0|13.6|6.5|6.5|53.5|0.48|65.7|85.1|25.0|N/A], |[], ],
|[2375|[[apple], |[2.6|[1.0|0.5|0.29], ], |22.0|13.5|6.5|6.5|53.3|0.48|65.8|84.9|24.8|N/A], |[], ], |[2400|[[apple], |[2.6|[1.0|0.5|0.29], ], |21.9|13.6|6.5|6.5|53.3|0.48|65.8|84.7|24.8|N/A], |[], ], |[2425|[[apple], |[2.6|[1.0|0.5|0.29],
], |21.9|13.7|6.5|6.5|53.3|0.48|65.5|84.8|24.7|N/A], |[], ], ],
////////////////////////////////////
////////////////////////////////////
////////////////////////////////////
////////////////////////////////////
[[0|[[apple|400], |[berry|0], |[melon|0], |[lily|0], |[lavendar|0], |[meat|30], |[wildgrass|0], ], ],
|[25|[[apple|396], |[berry|0], |[melon|0], |[lily|0], |[lavendar|0], |[meat|29], |[wildgrass|0], ], ],
|[50|[[apple|384], |[berry|0], |[melon|0], |[lily|0], |[lavendar|0], |[meat|23], |[wildgrass|0], ], ],
|[75|[[apple|377], |[berry|0], |[melon|0], |[lily|0], |[lavendar|0], |[meat|21], |[wildgrass|0], ], ],

```

|[100|[[apple|374], |[berry|0], |[melon|0], |[lily|0], |[lavendar|0], |[meat|16], |[wildgrass|0],],
], |[125|[[apple|377], |[berry|0], |[melon|0], |[lily|0], |[lavendar|0], |[meat|15], |[wildgrass|0],
],], |[150|[[apple|392], |[berry|0], |[melon|0], |[lily|2], |[lavendar|0], |[meat|11],
 |[wildgrass|0],],], |[175|[[apple|430], |[berry|0], |[melon|0], |[lily|3], |[lavendar|0],
 |[meat|11], |[wildgrass|0],],], |[200|[[apple|472], |[berry|1], |[melon|0], |[lily|4],
 |[lavendar|0], |[meat|9], |[wildgrass|0],],], |[225|[[apple|510], |[berry|1], |[melon|0], |[lily|5],
 |[lavendar|0], |[meat|9], |[wildgrass|0],],], |[250|[[apple|574], |[berry|4], |[melon|0], |[lily|8],
 |[lavendar|0], |[meat|9], |[wildgrass|0],],], |[275|[[apple|629], |[berry|6], |[melon|0], |[lily|5],
 |[lavendar|0], |[meat|8], |[wildgrass|0],],], |[300|[[apple|691], |[berry|7], |[melon|0], |[lily|5],
 |[lavendar|0], |[meat|6], |[wildgrass|0],],], |[325|[[apple|736], |[berry|7], |[melon|0], |[lily|5],
 |[lavendar|0], |[meat|6], |[wildgrass|0],],], |[350|[[apple|769], |[berry|9], |[melon|0], |[lily|4],
 |[lavendar|0], |[meat|4], |[wildgrass|0],],], |[375|[[apple|806], |[berry|10], |[melon|0],
 |[lily|3], |[lavendar|0], |[meat|3], |[wildgrass|0],],], |[400|[[apple|852], |[berry|11],
 |[melon|0], |[lily|4], |[lavendar|0], |[meat|4], |[wildgrass|0],],], |[425|[[apple|905],
 |[berry|11], |[melon|0], |[lily|6], |[lavendar|0], |[meat|4], |[wildgrass|0],],],
 |[450|[[apple|943], |[berry|11], |[melon|0], |[lily|8], |[lavendar|0], |[meat|3], |[wildgrass|0],],
], |[475|[[apple|922], |[berry|11], |[melon|0], |[lily|8], |[lavendar|0], |[meat|3], |[wildgrass|0],
],], |[500|[[apple|880], |[berry|12], |[melon|0], |[lily|9], |[lavendar|0], |[meat|3],
 |[wildgrass|0],],], |[525|[[apple|801], |[berry|15], |[melon|0], |[lily|9], |[lavendar|0],
 |[meat|3], |[wildgrass|0],],], |[550|[[apple|746], |[berry|16], |[melon|0], |[lily|11],
 |[lavendar|0], |[meat|3], |[wildgrass|0],],], |[575|[[apple|690], |[berry|17], |[melon|0],
 |[lily|9], |[lavendar|0], |[meat|3], |[wildgrass|0],],], |[600|[[apple|647], |[berry|18],
 |[melon|0], |[lily|10], |[lavendar|0], |[meat|3], |[wildgrass|0],],], |[625|[[apple|611],
 |[berry|16], |[melon|0], |[lily|8], |[lavendar|0], |[meat|3], |[wildgrass|0],],],
 |[650|[[apple|603], |[berry|15], |[melon|0], |[lily|8], |[lavendar|0], |[meat|5], |[wildgrass|0],],
], |[675|[[apple|615], |[berry|16], |[melon|0], |[lily|9], |[lavendar|0], |[meat|4], |[wildgrass|0],
],], |[700|[[apple|633], |[berry|19], |[melon|0], |[lily|7], |[lavendar|0], |[meat|4],
 |[wildgrass|0],],], |[725|[[apple|658], |[berry|17], |[melon|0], |[lily|9], |[lavendar|0],
 |[meat|3], |[wildgrass|0],],], |[750|[[apple|682], |[berry|17], |[melon|0], |[lily|9],
 |[lavendar|0], |[meat|4], |[wildgrass|0],],], |[775|[[apple|732], |[berry|18], |[melon|0],
 |[lily|12], |[lavendar|0], |[meat|3], |[wildgrass|0],],], |[800|[[apple|770], |[berry|20],
 |[melon|0], |[lily|12], |[lavendar|0], |[meat|2], |[wildgrass|0],],], |[825|[[apple|774],
 |[berry|22], |[melon|0], |[lily|13], |[lavendar|0], |[meat|2], |[wildgrass|0],],],
 |[850|[[apple|783], |[berry|23], |[melon|0], |[lily|16], |[lavendar|0], |[meat|2], |[wildgrass|0],
],], |[875|[[apple|797], |[berry|22], |[melon|0], |[lily|16], |[lavendar|0], |[meat|1],
 |[wildgrass|0],],], |[900|[[apple|823], |[berry|23], |[melon|0], |[lily|16], |[lavendar|0],
 |[meat|1], |[wildgrass|0],],], |[925|[[apple|836], |[berry|23], |[melon|0], |[lily|13],
 |[lavendar|0], |[meat|1], |[wildgrass|0],],], |[950|[[apple|842], |[berry|24], |[melon|0],
 |[lily|14], |[lavendar|0], |[meat|1], |[wildgrass|0],],], |[975|[[apple|848], |[berry|26],
 |[melon|0], |[lily|16], |[lavendar|0], |[meat|1], |[wildgrass|0],],], |[1000|[[apple|848],
 |[berry|28], |[melon|0], |[lily|18], |[lavendar|0], |[meat|1], |[wildgrass|0],],],
 |[1025|[[apple|836], |[berry|31], |[melon|0], |[lily|15], |[lavendar|0], |[meat|1], |[wildgrass|0],
],], |[1050|[[apple|835], |[berry|30], |[melon|0], |[lily|13], |[lavendar|0], |[meat|1],
 |[wildgrass|0],],], |[1075|[[apple|781], |[berry|32], |[melon|0], |[lily|12], |[lavendar|0],
 |[meat|1], |[wildgrass|0],],], |[1100|[[apple|789], |[berry|35], |[melon|0], |[lily|14],
 |[lavendar|0], |[meat|0], |[wildgrass|0],],], |[1125|[[apple|787], |[berry|38], |[melon|0],
 |[lily|14], |[lavendar|0], |[meat|0], |[wildgrass|0],],], |[1150|[[apple|762], |[berry|41],
 |[melon|0], |[lily|14], |[lavendar|0], |[meat|0], |[wildgrass|0],],], |[1175|[[apple|744],
 |[berry|42], |[melon|0], |[lily|14], |[lavendar|0], |[meat|0], |[wildgrass|0],],],
 |[1200|[[apple|735], |[berry|40], |[melon|0], |[lily|13], |[lavendar|0], |[meat|0], |[wildgrass|0],
],], |[1225|[[apple|734], |[berry|40], |[melon|0], |[lily|13], |[lavendar|0], |[meat|0],

```

[[wildgrass|0], ], ], [[1250|[[apple|744], [[berry|39], [[melon|0], [[lily|14], [[lavendar|0],
[[meat|0], [[wildgrass|0], ], ], [[1275|[[apple|740], [[berry|38], [[melon|0], [[lily|16],
[[lavendar|0], [[meat|0], [[wildgrass|0], ], ], [[1300|[[apple|765], [[berry|41], [[melon|0],
[[lily|15], [[lavendar|0], [[meat|0], [[wildgrass|0], ], ], [[1325|[[apple|781], [[berry|40],
[[melon|0], [[lily|14], [[lavendar|0], [[meat|0], [[wildgrass|0], ], ], [[1350|[[apple|817],
[[berry|41], [[melon|0], [[lily|18], [[lavendar|0], [[meat|0], [[wildgrass|0], ], ],
[[1375|[[apple|861], [[berry|46], [[melon|0], [[lily|17], [[lavendar|0], [[meat|0], [[wildgrass|0],
], ], ], [[1400|[[apple|888], [[berry|48], [[melon|0], [[lily|18], [[lavendar|0], [[meat|0],
[[wildgrass|0], ], ], [[1425|[[apple|928], [[berry|49], [[melon|0], [[lily|19], [[lavendar|0],
[[meat|0], [[wildgrass|0], ], ], [[1450|[[apple|929], [[berry|54], [[melon|0], [[lily|21],
[[lavendar|0], [[meat|0], [[wildgrass|0], ], ], [[1475|[[apple|930], [[berry|59], [[melon|0],
[[lily|18], [[lavendar|0], [[meat|0], [[wildgrass|0], ], ], [[1500|[[apple|910], [[berry|58],
[[melon|0], [[lily|16], [[lavendar|0], [[meat|0], [[wildgrass|0], ], ], [[1525|[[apple|896],
[[berry|65], [[melon|0], [[lily|11], [[lavendar|0], [[meat|0], [[wildgrass|0], ], ],
[[1550|[[apple|882], [[berry|64], [[melon|0], [[lily|13], [[lavendar|0], [[meat|0], [[wildgrass|0],
], ], ], [[1575|[[apple|863], [[berry|69], [[melon|0], [[lily|13], [[lavendar|0], [[meat|0],
[[wildgrass|0], ], ], [[1600|[[apple|842], [[berry|71], [[melon|0], [[lily|13], [[lavendar|0],
[[meat|0], [[wildgrass|0], ], ], [[1625|[[apple|823], [[berry|73], [[melon|0], [[lily|12],
[[lavendar|0], [[meat|0], [[wildgrass|0], ], ], [[1650|[[apple|815], [[berry|70], [[melon|0],
[[lily|11], [[lavendar|0], [[meat|0], [[wildgrass|0], ], ], [[1675|[[apple|828], [[berry|69],
[[melon|0], [[lily|14], [[lavendar|0], [[meat|0], [[wildgrass|0], ], ], [[1700|[[apple|821],
[[berry|67], [[melon|0], [[lily|16], [[lavendar|0], [[meat|0], [[wildgrass|0], ], ],
[[1725|[[apple|841], [[berry|67], [[melon|0], [[lily|19], [[lavendar|0], [[meat|0], [[wildgrass|0],
], ], ], [[1750|[[apple|841], [[berry|65], [[melon|0], [[lily|18], [[lavendar|0], [[meat|0],
[[wildgrass|0], ], ], ], [[1775|[[apple|836], [[berry|63], [[melon|0], [[lily|18], [[lavendar|0],
[[meat|0], [[wildgrass|0], ], ], ], [[1800|[[apple|824], [[berry|63], [[melon|0], [[lily|19],
[[lavendar|0], [[meat|0], [[wildgrass|0], ], ], ], [[1825|[[apple|804], [[berry|60], [[melon|0],
[[lily|20], [[lavendar|0], [[meat|0], [[wildgrass|0], ], ], ], [[1850|[[apple|811], [[berry|60],
[[melon|0], [[lily|17], [[lavendar|0], [[meat|0], [[wildgrass|0], ], ], ], [[1875|[[apple|783],
[[berry|55], [[melon|0], [[lily|16], [[lavendar|0], [[meat|0], [[wildgrass|0], ], ], ],
[[1900|[[apple|798], [[berry|58], [[melon|0], [[lily|13], [[lavendar|0], [[meat|0], [[wildgrass|0],
], ], ], [[1925|[[apple|798], [[berry|54], [[melon|0], [[lily|11], [[lavendar|0], [[meat|0],
[[wildgrass|0], ], ], ], [[1950|[[apple|810], [[berry|56], [[melon|0], [[lily|15], [[lavendar|0],
[[meat|0], [[wildgrass|0], ], ], ], [[1975|[[apple|831], [[berry|56], [[melon|0], [[lily|14],
[[lavendar|0], [[meat|0], [[wildgrass|0], ], ], ], [[2000|[[apple|811], [[berry|56], [[melon|0],
[[lily|13], [[lavendar|0], [[meat|0], [[wildgrass|0], ], ], ], [[2025|[[apple|820], [[berry|57],
[[melon|0], [[lily|11], [[lavendar|0], [[meat|0], [[wildgrass|0], ], ], ], [[2050|[[apple|840],
[[berry|55], [[melon|0], [[lily|11], [[lavendar|0], [[meat|0], [[wildgrass|0], ], ], ],
[[2075|[[apple|825], [[berry|55], [[melon|0], [[lily|9], [[lavendar|0], [[meat|0], [[wildgrass|0],
], ], ], [[2100|[[apple|835], [[berry|64], [[melon|0], [[lily|9], [[lavendar|0], [[meat|0],
[[wildgrass|0], ], ], ], [[2125|[[apple|839], [[berry|60], [[melon|0], [[lily|8], [[lavendar|0],
[[meat|0], [[wildgrass|0], ], ], ], [[2150|[[apple|835], [[berry|59], [[melon|0], [[lily|7],
[[lavendar|0], [[meat|0], [[wildgrass|0], ], ], ], [[2175|[[apple|817], [[berry|62], [[melon|0],
[[lily|8], [[lavendar|0], [[meat|0], [[wildgrass|0], ], ], ], [[2200|[[apple|801], [[berry|66],
[[melon|0], [[lily|9], [[lavendar|0], [[meat|0], [[wildgrass|0], ], ], ], [[2225|[[apple|795],
[[berry|65], [[melon|0], [[lily|11], [[lavendar|0], [[meat|0], [[wildgrass|0], ], ], ],
[[2250|[[apple|773], [[berry|71], [[melon|0], [[lily|18], [[lavendar|0], [[meat|0], [[wildgrass|0],
], ], ], [[2275|[[apple|765], [[berry|78], [[melon|0], [[lily|20], [[lavendar|0], [[meat|0],
[[wildgrass|0], ], ], ], [[2300|[[apple|772], [[berry|77], [[melon|0], [[lily|15], [[lavendar|0],
[[meat|0], [[wildgrass|0], ], ], ], [[2325|[[apple|767], [[berry|80], [[melon|0], [[lily|16],
[[lavendar|0], [[meat|0], [[wildgrass|0], ], ], ], [[2350|[[apple|766], [[berry|86], [[melon|0],

```

```

|[lily|18], |[lavendar|0], |[meat|0], |[wildgrass|0], ], ], |[2375|[[apple|784], |[berry|86],
|[melon|0], |[lily|19], |[lavendar|0], |[meat|0], |[wildgrass|0], ], ], |[2400|[[apple|788],
|[berry|93], |[melon|0], |[lily|17], |[lavendar|0], |[meat|0], |[wildgrass|0], ], ],
|[2425|[[apple|816], |[berry|96], |[melon|0], |[lily|15], |[lavendar|0], |[meat|0], |[wildgrass|0],
], ], ],

```

```

////////////////////////////////////
////////////////////////////////////
////////////////////////////////////

```

```

[[0|N/A], |[25|19.2], |[50|35.1], |[75|59.6], |[100|80.7], |[125|104.7], |[150|124.9],
|[175|147.0], |[200|178.2], |[225|191.6], |[250|191.8], |[275|189.7], |[300|192.0],
|[325|234.2], |[350|236.2], |[375|227.0], |[400|237.9], |[425|216.8], |[450|239.4],
|[475|214.6], |[500|222.1], |[525|217.7], |[550|242.1], |[575|225.9], |[600|240.8],
|[625|237.7], |[650|253.0], |[675|254.3], |[700|263.3], |[725|250.8], |[750|254.1],
|[775|258.9], |[800|232.3], |[825|217.0], |[850|246.1], |[875|222.0], |[900|229.2],
|[925|234.2], |[950|226.8], |[975|197.7], |[1000|191.3], |[1025|202.9], |[1050|218.9],
|[1075|227.6], |[1100|232.6], |[1125|219.2], |[1150|229.2], |[1175|231.4], |[1200|238.1],
|[1225|230.8], |[1250|208.0], |[1275|206.3], |[1300|228.5], |[1325|215.8], |[1350|226.6],
|[1375|239.0], |[1400|232.4], |[1425|224.4], |[1450|205.4], |[1475|210.4], |[1500|226.5],
|[1525|200.1], |[1550|216.9], |[1575|228.7], |[1600|227.3], |[1625|217.3], |[1650|216.9],
|[1675|222.7], |[1700|217.1], |[1725|209.9], |[1750|222.4], |[1775|229.1], |[1800|234.0],
|[1825|204.6], |[1850|212.9], |[1875|224.6], |[1900|221.8], |[1925|230.0], |[1950|219.6],
|[1975|222.9], |[2000|222.0], |[2025|240.5], |[2050|231.5], |[2075|230.2], |[2100|220.6],
|[2125|211.1], |[2150|222.9], |[2175|219.6], |[2200|223.0], |[2225|226.6], |[2250|220.0],
|[2275|208.7], |[2300|207.6], |[2325|229.6], |[2350|225.5], |[2375|229.5], |[2400|210.0],
|[2425|220.7], ],

```

```

////////////////////////////////////
////////////////////////////////////
////////////////////////////////////

```

```

[[0|N/A], |[25|[13.0|21.0|24.5], ], |[50|[28.0|34.5|40.5], ], |[75|[51.0|61.0|69.0], ],
|[100|[72.0|86.5|94.2], ], |[125|[95.0|108.0|119.0], ], |[150|[119.0|134.0|144.0], ],
|[175|[140.2|150.5|167.2], ], |[200|[175.5|184.0|188.0], ], |[225|[186.5|204.5|216.2], ],
|[250|[171.0|212.5|232.2], ], |[275|[109.0|232.0|259.5], ], |[300|[85.0|246.5|288.5], ],
|[325|[99.5|297.0|312.0], ], |[350|[111.0|287.0|340.0], ], |[375|[116.0|235.0|351.0], ],
|[400|[129.5|228.0|375.0], ], |[425|[112.0|196.0|313.0], ], |[450|[137.0|215.0|337.0], ],
|[475|[95.0|179.0|305.5], ], |[500|[109.5|195.0|296.0], ], |[525|[109.5|173.0|300.5], ],
|[550|[131.0|195.0|313.0], ], |[575|[127.0|189.0|302.0], ], |[600|[125.0|185.0|335.0], ],
|[625|[126.8|188.5|324.2], ], |[650|[143.2|225.0|330.5], ], |[675|[133.5|206.0|358.5], ],
|[700|[138.8|206.5|363.0], ], |[725|[122.0|220.0|328.0], ], |[750|[132.5|220.5|320.0], ],
|[775|[139.0|223.5|340.2], ], |[800|[131.0|199.0|334.0], ], |[825|[98.0|185.0|313.0], ],
|[850|[106.0|200.0|331.0], ], |[875|[119.5|191.0|293.0], ], |[900|[120.2|185.5|322.5], ],
|[925|[105.5|180.5|333.2], ], |[950|[114.8|182.0|304.0], ], |[975|[99.0|141.0|266.5], ],
|[1000|[102.2|155.0|255.0], ], |[1025|[107.0|165.0|277.0], ], |[1050|[114.0|180.0|299.0], ],
|[1075|[108.0|180.0|323.0], ], |[1100|[118.5|196.0|315.0], ], |[1125|[107.0|168.0|313.0], ],
|[1150|[101.0|172.0|313.0], ], |[1175|[111.2|197.5|305.8], ], |[1200|[111.8|197.5|309.5], ],
|[1225|[120.0|195.0|311.5], ], |[1250|[111.0|169.5|272.8], ], |[1275|[111.2|182.0|268.2], ],
|[1300|[116.0|186.0|296.0], ], |[1325|[108.2|164.5|297.5], ], |[1350|[105.8|173.0|300.5], ],
|[1375|[122.0|215.0|313.0], ], |[1400|[106.5|196.5|313.2], ], |[1425|[100.2|193.0|322.5], ],
|[1450|[95.0|165.0|281.5], ], |[1475|[95.0|154.5|270.8], ], |[1500|[108.5|179.0|281.0], ],
|[1525|[111.0|168.0|266.5], ], |[1550|[107.0|159.0|308.0], ], |[1575|[106.0|176.0|320.0], ],
|[1600|[122.0|187.0|298.0], ], |[1625|[106.2|202.5|296.8], ], |[1650|[114.0|177.0|304.0], ],
|[1675|[123.0|177.0|313.0], ], |[1700|[101.0|171.0|287.5], ], |[1725|[108.8|163.5|272.5], ],

```

```

|[1750|[101.0|164.0|313.0], ], |[1775|[111.2|168.0|316.0], ], |[1800|[109.5|185.5|316.0], ],
|[1825|[96.5|166.0|269.0], ], |[1850|[104.5|175.0|282.5], ], |[1875|[116.2|178.5|293.0], ],
|[1900|[109.0|169.0|296.0], ], |[1925|[116.0|194.0|309.5], ], |[1950|[112.2|175.5|275.8], ],
|[1975|[123.0|189.0|299.0], ], |[2000|[116.8|191.0|295.2], ], |[2025|[110.0|195.0|319.0], ],
|[2050|[110.0|181.5|320.0], ], |[2075|[113.0|179.5|322.0], ], |[2100|[109.8|181.0|296.2], ],
|[2125|[98.8|182.0|272.8], ], |[2150|[105.0|177.0|275.0], ], |[2175|[94.0|167.0|296.0], ],
|[2200|[104.8|176.0|273.2], ], |[2225|[111.5|170.0|296.8], ], |[2250|[100.2|177.0|292.8], ],
|[2275|[107.0|168.0|272.0], ], |[2300|[93.0|148.0|284.0], ], |[2325|[119.2|163.5|311.5], ],
|[2350|[116.2|175.5|312.2], ], |[2375|[117.5|179.0|273.5], ], |[2400|[110.5|180.5|258.2], ],
|[2425|[111.5|180.5|296.5], ], ],
////////////////////////////////////
////////////////////////////////////
//////////
|[0|[0.0|0.0|0.0], ], |[25|[25.0|25.0|25.0], ], |[50|[50.0|50.0|50.0], ], |[75|[75.0|75.0|75.0], ],
|[100|[100.0|100.0|100.0], ], |[125|[125.0|125.0|125.0], ], |[150|[150.0|150.0|150.0], ],
|[175|[83.0|175.0|175.0], ], |[200|[56.0|200.0|200.0], ], |[225|[53.0|225.0|225.0], ],
|[250|[49.0|131.0|250.0], ], |[275|[50.0|121.0|275.0], ], |[300|[52.0|118.0|300.0], ],
|[325|[58.0|125.0|325.0], ], |[350|[64.0|126.5|266.5], ], |[375|[61.0|128.0|233.2], ],
|[400|[58.0|131.0|234.0], ], |[425|[52.8|132.0|230.2], ], |[450|[51.0|123.0|226.0], ],
|[475|[60.0|126.0|236.8], ], |[500|[67.2|128.0|241.0], ], |[525|[73.2|135.0|253.5], ],
|[550|[68.2|135.0|256.5], ], |[575|[71.0|146.0|259.0], ], |[600|[69.0|152.0|260.0], ],
|[625|[68.0|150.5|257.0], ], |[650|[63.0|142.0|260.0], ], |[675|[55.0|135.0|254.8], ],
|[700|[57.0|133.0|260.0], ], |[725|[54.0|125.0|250.0], ], |[750|[54.0|119.0|246.0], ],
|[775|[52.0|120.0|233.0], ], |[800|[54.2|119.0|234.0], ], |[825|[59.0|125.0|235.0], ],
|[850|[61.2|126.0|218.0], ], |[875|[61.0|127.0|230.8], ], |[900|[55.0|129.0|228.0], ],
|[925|[57.0|130.0|230.5], ], |[950|[59.0|123.0|238.0], ], |[975|[56.0|126.0|242.0], ],
|[1000|[57.0|125.0|248.0], ], |[1025|[58.0|128.0|255.0], ], |[1050|[59.0|126.0|254.0], ],
|[1075|[62.0|126.5|232.8], ], |[1100|[59.0|129.0|233.0], ], |[1125|[61.0|133.0|242.0], ],
|[1150|[58.0|134.0|247.5], ], |[1175|[57.0|129.5|233.0], ], |[1200|[60.2|129.0|242.8], ],
|[1225|[60.0|129.0|239.0], ], |[1250|[57.0|129.0|253.5], ], |[1275|[56.0|125.5|247.2], ],
|[1300|[54.5|127.0|244.5], ], |[1325|[58.0|124.0|241.0], ], |[1350|[53.2|119.5|238.8], ],
|[1375|[51.0|120.0|227.0], ], |[1400|[54.0|124.0|232.0], ], |[1425|[56.0|123.5|227.0], ],
|[1450|[57.2|125.0|233.0], ], |[1475|[61.0|126.0|236.0], ], |[1500|[56.2|123.5|241.5], ],
|[1525|[56.0|121.0|242.0], ], |[1550|[59.0|125.0|237.0], ], |[1575|[63.0|129.0|236.0], ],
|[1600|[65.8|127.0|239.2], ], |[1625|[57.0|125.5|234.8], ], |[1650|[55.0|130.0|239.0], ],
|[1675|[62.0|125.0|239.0], ], |[1700|[59.0|125.0|243.8], ], |[1725|[56.0|125.0|244.0], ],
|[1750|[61.0|132.0|247.0], ], |[1775|[58.0|122.0|242.0], ], |[1800|[58.0|124.5|237.0], ],
|[1825|[58.0|127.5|240.0], ], |[1850|[57.2|129.0|237.8], ], |[1875|[59.0|129.5|239.5], ],
|[1900|[59.0|130.0|239.0], ], |[1925|[59.0|131.0|243.0], ], |[1950|[58.0|125.0|244.0], ],
|[1975|[54.0|130.0|247.0], ], |[2000|[62.0|124.0|243.8], ], |[2025|[56.0|120.5|236.8], ],
|[2050|[54.0|124.0|228.2], ], |[2075|[56.5|124.0|231.5], ], |[2100|[56.0|125.0|223.8], ],
|[2125|[53.0|122.0|227.0], ], |[2150|[59.0|123.0|229.0], ], |[2175|[61.0|130.0|237.0], ],
|[2200|[56.0|118.5|229.8], ], |[2225|[53.0|119.0|225.0], ], |[2250|[55.8|121.0|228.2], ],
|[2275|[55.0|119.0|227.0], ], |[2300|[56.0|124.0|228.0], ], |[2325|[57.0|122.0|227.0], ],
|[2350|[56.8|123.0|233.2], ], |[2375|[54.5|117.0|237.0], ], |[2400|[53.8|119.5|243.0], ],
|[2425|[54.0|122.0|234.0], ], ],
////////////////////////////////////
////////////////////////////////////
//////////
228.6

```



```

|[1450|[45.6|25.5|41.7|4.4|15.4], |[], |, |[1475|[46.4|25.7|42.4|4.2|15.8], |[], |,
|[1500|[46.4|25.9|43.0|4.5|15.6], |[], |, |[1525|[44.3|26.3|42.7|3.9|15.4], |[], |,
|[1550|[45.9|26.9|41.8|3.4|15.9], |[], |, |[1575|[47.4|27.2|43.5|3.9|16.0], |[], |,
|[1600|[47.5|27.2|43.6|4.0|16.5], |[], |, |[1625|[45.5|27.1|44.3|4.5|16.0], |[], |,
|[1650|[44.9|27.2|42.6|3.6|16.6], |[], |, |[1675|[45.7|27.4|44.2|4.4|16.4], |[], |,
|[1700|[44.5|27.6|44.3|4.5|16.2], |[], |, |[1725|[45.0|28.0|43.9|4.3|16.1], |[], |,
|[1750|[45.9|27.9|43.2|3.9|16.3], |[], |, |[1775|[44.9|27.3|44.5|5.0|16.0], |[], |,
|[1800|[45.0|27.2|44.1|4.8|16.0], |[], |, |[1825|[46.5|27.5|44.3|4.5|15.9], |[], |,
|[1850|[45.3|27.4|44.9|5.0|16.1], |[], |, |[1875|[45.7|27.6|43.6|3.9|17.1], |[], |,
|[1900|[45.6|27.4|44.4|4.4|17.1], |[], |, |[1925|[45.3|27.9|44.8|4.7|17.1], |[], |,
|[1950|[44.0|28.2|45.0|4.2|17.2], |[], |, |[1975|[44.8|28.0|45.0|4.4|16.5], |[], |,
|[2000|[45.6|27.6|43.5|3.9|16.7], |[], |, |[2025|[45.2|27.3|43.6|4.2|16.7], |[], |,
|[2050|[46.3|27.2|44.5|4.3|16.3], |[], |, |[2075|[45.3|26.9|43.4|3.5|16.3], |[], |,
|[2100|[45.8|26.8|44.7|4.6|16.2], |[], |, |[2125|[45.2|27.3|44.4|4.4|15.8], |[], |,
|[2150|[47.2|27.3|43.5|3.6|16.5], |[], |, |[2175|[46.8|26.7|45.0|3.8|16.7], |[], |,
|[2200|[45.6|27.0|44.2|4.0|16.1], |[], |, |[2225|[44.5|26.2|44.6|4.8|15.5], |[], |,
|[2250|[45.8|26.2|44.6|4.5|15.8], |[], |, |[2275|[44.5|26.4|44.7|4.9|15.8], |[], |,
|[2300|[44.5|26.7|44.2|4.5|16.2], |[], |, |[2325|[43.4|26.8|44.4|4.5|16.3], |[], |,
|[2350|[44.3|26.7|43.8|4.3|17.1], |[], |, |[2375|[43.3|26.6|45.2|5.2|16.1], |[], |,
|[2400|[42.2|26.9|44.4|4.7|16.1], |[], |, |[2425|[42.7|26.9|44.2|4.4|15.9], |[], |, ],
////////////////////
////////////////////
////////////////
[[0|[[0|400], |, |[[0|30], |, |, |[25|[[0|396], |, |[[0|29], |, |, |[50|[[0|384], |, |[[0|23], |, |,
|[75|[[0|362], |[1|10], |[2|5], |, |[[0|21], |, |, |[100|[[0|344], |[1|10], |[2|20], |, |[[0|16], |, |,
|[125|[[0|317], |[1|22], |[2|37], |[3|1], |, |[[0|14], |[1|1], |, |, |[150|[[0|302], |[1|33], |[2|50],
|[3|5], |[4|4], |, |[[0|9], |[1|2], |, |, |[175|[[0|310], |[1|40], |[2|52], |[3|17], |[4|10], |[5|3],
|[6|1], |, |[[0|9], |[1|1], |[2|1], |, |, |[200|[[0|303], |[1|57], |[2|68], |[3|24], |[4|14], |[5|8],
|[6|2], |[7|1], |, |[[0|7], |[1|1], |[2|1], |, |, |[225|[[0|299], |[1|75], |[2|71], |[3|35], |[4|18],
|[5|9], |[6|8], |[7|1], |, |[[0|7], |[1|1], |[2|1], |, |, |[250|[[0|337], |[1|61], |[2|81], |[3|46],
|[4|31], |[5|11], |[6|13], |[7|4], |[8|2], |, |[[0|7], |[1|1], |[2|1], |, |, |[275|[[0|362], |[1|64],
|[2|75], |[3|54], |[4|44], |[5|18], |[6|13], |[7|7], |[8|2], |[9|1], |, |[[0|5], |[1|1], |[2|2], |, |,
|[300|[[0|399], |[1|64], |[2|73], |[3|53], |[4|54], |[5|23], |[6|21], |[7|9], |[8|5], |[9|1], |[10|0],
|[11|1], |, |[[0|3], |[1|1], |[2|2], |, |, |[325|[[0|424], |[1|67], |[2|72], |[3|49], |[4|68], |[5|29],
|[6|19], |[7|6], |[8|9], |[9|4], |[10|0], |[11|1], |, |[[0|3], |[1|1], |[2|2], |, |, |[350|[[0|450],
|[1|58], |[2|81], |[3|50], |[4|62], |[5|26], |[6|27], |[7|11], |[8|10], |[9|3], |[10|2], |[11|2], |,
|[0|2], |[1|1], |[2|1], |, |, |[375|[[0|467], |[1|59], |[2|93], |[3|49], |[4|58], |[5|27], |[6|34],
|[7|11], |[8|10], |[9|6], |[10|2], |[11|1], |[12|1], |[13|1], |, |[[0|1], |[1|1], |[2|1], |, |,
|[400|[[0|489], |[1|63], |[2|105], |[3|50], |[4|53], |[5|37], |[6|30], |[7|10], |[8|15], |[9|6],
|[10|1], |[11|5], |[12|1], |[13|2], |, |[[0|1], |[1|2], |[2|1], |, |, |[425|[[0|521], |[1|53], |[2|120],
|[3|54], |[4|57], |[5|35], |[6|30], |[7|14], |[8|19], |[9|8], |[10|3], |[11|4], |[12|1], |[13|2],
|[14|1], |, |[[0|1], |[1|2], |[2|1], |, |, |[450|[[0|551], |[1|58], |[2|110], |[3|65], |[4|56], |[5|29],
|[6|33], |[7|14], |[8|17], |[9|11], |[10|7], |[11|5], |[12|1], |[13|1], |[14|1], |[15|3], |, |[[0|1],
|[1|2], |, |, |[475|[[0|536], |[1|56], |[2|100], |[3|75], |[4|54], |[5|22], |[6|32], |[7|16], |[8|16],
|[9|14], |[10|6], |[11|6], |[12|2], |[13|1], |[14|1], |[15|4], |, |[[0|1], |[1|2], |, |, |[500|[[0|523],
|[1|52], |[2|90], |[3|68], |[4|54], |[5|23], |[6|25], |[7|12], |[8|16], |[9|13], |[10|10], |[11|6],
|[12|2], |[13|1], |[14|1], |[15|3], |[16|0], |[17|2], |, |[[0|1], |[1|1], |[2|1], |, |, |[525|[[0|474],
|[1|50], |[2|77], |[3|61], |[4|49], |[5|27], |[6|21], |[7|13], |[8|15], |[9|9], |[10|13], |[11|8],
|[12|2], |[13|1], |[14|1], |[15|1], |[16|0], |[17|3], |, |[[0|1], |[1|1], |[2|1], |, |, |[550|[[0|435],
|[1|56], |[2|73], |[3|54], |[4|45], |[5|21], |[6|27], |[7|10], |[8|13], |[9|11], |[10|9], |[11|8],
|[12|5], |[13|1], |[14|0], |[15|1], |[16|1], |[17|2], |[18|0], |[19|1], |, |[[0|0], |[1|2], |[2|1], |, |,

```

|575|[[0|396], |[1|56], |[2|71], |[3|42], |[4|36], |[5|26], |[6|23], |[7|16], |[8|10], |[9|9],
 |[10|10], |[11|8], |[12|7], |[13|1], |[14|0], |[15|1], |[16|1], |[17|1], |[18|0], |[19|2],], |[0|0],
 |[1|2], |[2|1],],], |[600|[[0|371], |[1|57], |[2|73], |[3|33], |[4|38], |[5|15], |[6|26], |[7|11],
 |[8|15], |[9|8], |[10|7], |[11|8], |[12|7], |[13|1], |[14|0], |[15|1], |[16|1], |[17|1], |[18|0],
 |[19|1], |[20|0], |[21|1],], |[0|0], |[1|2], |[2|1],],], |[625|[[0|340], |[1|53], |[2|71], |[3|33],
 |[4|36], |[5|19], |[6|21], |[7|14], |[8|15], |[9|6], |[10|8], |[11|6], |[12|6], |[13|2], |[14|0],
 |[15|1], |[16|0], |[17|2], |[18|0], |[19|1], |[20|0], |[21|1],], |[0|1], |[1|1], |[2|1],],],
 |[650|[[0|350], |[1|48], |[2|59], |[3|35], |[4|31], |[5|18], |[6|21], |[7|15], |[8|18], |[9|5],
 |[10|6], |[11|6], |[12|7], |[13|1], |[14|1], |[15|1], |[16|1], |[17|1], |[18|0], |[19|0], |[20|1],
 |[21|0], |[22|1],], |[0|3], |[1|1], |[2|0], |[3|0], |[4|1],],], |[675|[[0|352], |[1|51], |[2|72],
 |[3|36], |[4|30], |[5|15], |[6|25], |[7|14], |[8|12], |[9|7], |[10|7], |[11|4], |[12|6], |[13|2],
 |[14|1], |[15|2], |[16|1], |[17|0], |[18|1], |[19|0], |[20|0], |[21|1], |[22|0], |[23|1],], |[0|2],
 |[1|1], |[2|0], |[3|0], |[4|1],],], |[700|[[0|370], |[1|49], |[2|73], |[3|36], |[4|27], |[5|22],
 |[6|21], |[7|11], |[8|13], |[9|10], |[10|5], |[11|4], |[12|5], |[13|6], |[14|1], |[15|1], |[16|2],
 |[17|0], |[18|1], |[19|0], |[20|0], |[21|1], |[22|0], |[23|1],], |[0|2], |[1|1], |[2|0], |[3|0],
 |[4|1],],], |[725|[[0|389], |[1|51], |[2|67], |[3|31], |[4|42], |[5|24], |[6|15], |[7|15], |[8|12],
 |[9|10], |[10|7], |[11|4], |[12|1], |[13|7], |[14|2], |[15|1], |[16|2], |[17|1], |[18|1], |[19|0],
 |[20|0], |[21|0], |[22|0], |[23|1], |[24|0], |[25|1],], |[0|1], |[1|1], |[2|0], |[3|0], |[4|1],],],
 |[750|[[0|405], |[1|53], |[2|66], |[3|29], |[4|44], |[5|26], |[6|16], |[7|15], |[8|17], |[9|8],
 |[10|5], |[11|4], |[12|5], |[13|7], |[14|2], |[15|2], |[16|1], |[17|1], |[18|0], |[19|0], |[20|0],
 |[21|0], |[22|0], |[23|1], |[24|0], |[25|0], |[26|1],], |[0|2], |[1|1], |[2|0], |[3|0], |[4|0], |[5|1],
],], |[775|[[0|439], |[1|45], |[2|89], |[3|28], |[4|42], |[5|31], |[6|15], |[7|18], |[8|17], |[9|11],
 |[10|7], |[11|3], |[12|3], |[13|7], |[14|1], |[15|2], |[16|0], |[17|1], |[18|1], |[19|0], |[20|0],
 |[21|0], |[22|0], |[23|0], |[24|0], |[25|1], |[26|1],], |[0|2], |[1|0], |[2|0], |[3|0], |[4|0], |[5|1],
],], |[800|[[0|455], |[1|53], |[2|94], |[3|42], |[4|38], |[5|33], |[6|18], |[7|15], |[8|14], |[9|13],
 |[10|5], |[11|4], |[12|2], |[13|4], |[14|5], |[15|2], |[16|0], |[17|2], |[18|1], |[19|0], |[20|0],
 |[21|0], |[22|0], |[23|0], |[24|0], |[25|1], |[26|1],], |[0|1], |[1|0], |[2|0], |[3|0], |[4|0], |[5|1],
],], |[825|[[0|461], |[1|59], |[2|86], |[3|41], |[4|44], |[5|26], |[6|21], |[7|11], |[8|17], |[9|13],
 |[10|8], |[11|2], |[12|5], |[13|4], |[14|4], |[15|1], |[16|1], |[17|0], |[18|2], |[19|1], |[20|0],
 |[21|0], |[22|0], |[23|0], |[24|0], |[25|0], |[26|1], |[27|1],], |[0|0], |[1|1], |[2|0], |[3|0],
 |[4|0], |[5|1],],], |[850|[[0|469], |[1|62], |[2|88], |[3|46], |[4|43], |[5|18], |[6|30], |[7|8],
 |[8|18], |[9|11], |[10|7], |[11|4], |[12|4], |[13|3], |[14|3], |[15|1], |[16|2], |[17|0], |[18|2],
 |[19|1], |[20|0], |[21|0], |[22|0], |[23|0], |[24|0], |[25|0], |[26|1], |[27|1],], |[0|0], |[1|1],
 |[2|0], |[3|0], |[4|0], |[5|1],],], |[875|[[0|466], |[1|67], |[2|92], |[3|45], |[4|46], |[5|25],
 |[6|26], |[7|10], |[8|19], |[9|11], |[10|8], |[11|4], |[12|1], |[13|4], |[14|4], |[15|0], |[16|1],
 |[17|1], |[18|1], |[19|2], |[20|0], |[21|0], |[22|0], |[23|0], |[24|0], |[25|0], |[26|1], |[27|1],],
 |[0|0], |[1|1],],], |[900|[[0|487], |[1|57], |[2|102], |[3|40], |[4|49], |[5|31], |[6|26], |[7|15],
 |[8|14], |[9|8], |[10|11], |[11|5], |[12|1], |[13|1], |[14|7], |[15|1], |[16|0], |[17|0], |[18|2],
 |[19|3], |[20|0], |[21|0], |[22|0], |[23|0], |[24|0], |[25|0], |[26|1], |[27|1],], |[0|0], |[1|1],],],
 |[925|[[0|495], |[1|52], |[2|105], |[3|36], |[4|56], |[5|35], |[6|23], |[7|16], |[8|17], |[9|8],
 |[10|8], |[11|9], |[12|0], |[13|2], |[14|5], |[15|1], |[16|1], |[17|0], |[18|1], |[19|1], |[20|0],
 |[21|0], |[22|0], |[23|0], |[24|0], |[25|0], |[26|1],], |[0|0], |[1|1],],], |[950|[[0|496], |[1|50],
 |[2|98], |[3|45], |[4|63], |[5|31], |[6|25], |[7|20], |[8|16], |[9|6], |[10|9], |[11|9], |[12|2],
 |[13|2], |[14|3], |[15|1], |[16|2], |[17|0], |[18|0], |[19|2],], |[0|0], |[1|1],],], |[975|[[0|509],
 |[1|41], |[2|90], |[3|49], |[4|63], |[5|36], |[6|30], |[7|20], |[8|14], |[9|7], |[10|10], |[11|10],
 |[12|2], |[13|2], |[14|3], |[15|0], |[16|1], |[17|0], |[18|1], |[19|0], |[20|1], |[21|1],], |[0|0],
 |[1|1],],], |[1000|[[0|507], |[1|53], |[2|75], |[3|53], |[4|57], |[5|40], |[6|30], |[7|23], |[8|17],
 |[9|7], |[10|9], |[11|8], |[12|5], |[13|3], |[14|1], |[15|2], |[16|0], |[17|0], |[18|2], |[19|0],
 |[20|1], |[21|1],], |[0|0], |[1|1],],], |[1025|[[0|496], |[1|53], |[2|74], |[3|55], |[4|55], |[5|39],
 |[6|28], |[7|22], |[8|21], |[9|9], |[10|4], |[11|8], |[12|6], |[13|5], |[14|1], |[15|2], |[16|0],
 |[17|0], |[18|2], |[19|0], |[20|1], |[21|1],], |[0|0], |[1|1],],], |[1050|[[0|492], |[1|55], |[2|76],

|[3|54], |[4|49], |[5|38], |[6|31], |[7|20], |[8|22], |[9|9], |[10|6], |[11|6], |[12|7], |[13|6],
 |[14|1], |[15|2], |[16|0], |[17|0], |[18|2], |[19|0], |[20|1], |[21|0], |[22|1],], |[0|0], |[1|1],],],
 |[1075|[[0|467], |[1|52], |[2|81], |[3|43], |[4|37], |[5|35], |[6|28], |[7|21], |[8|18], |[9|12],
 |[10|8], |[11|5], |[12|3], |[13|6], |[14|3], |[15|2], |[16|1], |[17|0], |[18|1], |[19|0], |[20|1],
 |[21|0], |[22|0], |[23|1],], |[0|0], |[1|1],],], |[1100|[[0|478], |[1|47], |[2|90], |[3|35], |[4|35],
 |[5|34], |[6|38], |[7|16], |[8|23], |[9|11], |[10|7], |[11|6], |[12|3], |[13|5], |[14|4], |[15|3],
 |[16|1], |[17|0], |[18|0], |[19|0], |[20|1], |[21|0], |[22|0], |[23|1],], |N/A], |[1125|[[0|470],
 |[1|59], |[2|84], |[3|42], |[4|36], |[5|38], |[6|24], |[7|18], |[8|22], |[9|15], |[10|10], |[11|4],
 |[12|4], |[13|3], |[14|4], |[15|3], |[16|1], |[17|1], |[18|0], |[19|0], |[20|0], |[21|0], |[22|0],
 |[23|1],], |N/A], |[1150|[[0|451], |[1|55], |[2|89], |[3|46], |[4|36], |[5|31], |[6|23], |[7|18],
 |[8|22], |[9|14], |[10|13], |[11|5], |[12|4], |[13|1], |[14|4], |[15|2], |[16|1], |[17|0], |[18|1],
 |[19|0], |[20|0], |[21|0], |[22|0], |[23|0], |[24|1],], |N/A], |[1175|[[0|451], |[1|51], |[2|95],
 |[3|35], |[4|33], |[5|25], |[6|29], |[7|19], |[8|16], |[9|14], |[10|12], |[11|6], |[12|2], |[13|2],
 |[14|5], |[15|2], |[16|1], |[17|0], |[18|1], |[19|0], |[20|0], |[21|0], |[22|0], |[23|0], |[24|1],],
 |N/A], |[1200|[[0|448], |[1|46], |[2|91], |[3|32], |[4|36], |[5|28], |[6|24], |[7|20], |[8|15],
 |[9|10], |[10|17], |[11|5], |[12|3], |[13|4], |[14|3], |[15|1], |[16|2], |[17|1], |[18|1], |[19|0],
 |[20|0], |[21|0], |[22|0], |[23|0], |[24|0], |[25|1],], |N/A], |[1225|[[0|445], |[1|60], |[2|80],
 |[3|34], |[4|44], |[5|24], |[6|20], |[7|18], |[8|11], |[9|10], |[10|14], |[11|9], |[12|4], |[13|3],
 |[14|3], |[15|2], |[16|3], |[17|1], |[18|1], |[19|0], |[20|0], |[21|0], |[22|0], |[23|0], |[24|0],
 |[25|1],], |N/A], |[1250|[[0|440], |[1|54], |[2|93], |[3|43], |[4|42], |[5|21], |[6|20], |[7|17],
 |[8|12], |[9|13], |[10|11], |[11|11], |[12|4], |[13|3], |[14|5], |[15|3], |[16|3], |[17|1], |[18|0],
 |[19|0], |[20|0], |[21|0], |[22|0], |[23|0], |[24|0], |[25|1],], |N/A], |[1275|[[0|443], |[1|45],
 |[2|100], |[3|44], |[4|32], |[5|26], |[6|26], |[7|8], |[8|15], |[9|12], |[10|9], |[11|10], |[12|7],
 |[13|3], |[14|6], |[15|1], |[16|5], |[17|0], |[18|1], |[19|0], |[20|0], |[21|0], |[22|0], |[23|0],
 |[24|0], |[25|0], |[26|1],], |N/A], |[1300|[[0|454], |[1|61], |[2|87], |[3|51], |[4|38], |[5|24],
 |[6|24], |[7|12], |[8|15], |[9|11], |[10|8], |[11|7], |[12|11], |[13|6], |[14|2], |[15|3], |[16|4],
 |[17|1], |[18|1], |[19|0], |[20|0], |[21|0], |[22|0], |[23|0], |[24|0], |[25|0], |[26|0], |[27|1],],
 |N/A], |[1325|[[0|459], |[1|66], |[2|87], |[3|47], |[4|44], |[5|26], |[6|21], |[7|14], |[8|17],
 |[9|9], |[10|12], |[11|4], |[12|8], |[13|8], |[14|4], |[15|2], |[16|3], |[17|1], |[18|2], |[19|0],
 |[20|0], |[21|0], |[22|0], |[23|0], |[24|0], |[25|0], |[26|0], |[27|1],], |N/A], |[1350|[[0|486],
 |[1|67], |[2|89], |[3|47], |[4|53], |[5|32], |[6|21], |[7|11], |[8|18], |[9|9], |[10|9], |[11|5],
 |[12|7], |[13|10], |[14|4], |[15|3], |[16|2], |[17|1], |[18|0], |[19|1], |[20|0], |[21|0], |[22|0],
 |[23|0], |[24|0], |[25|0], |[26|0], |[27|1],], |N/A], |[1375|[[0|527], |[1|65], |[2|83], |[3|49],
 |[4|62], |[5|31], |[6|24], |[7|8], |[8|17], |[9|12], |[10|10], |[11|8], |[12|5], |[13|8], |[14|3],
 |[15|6], |[16|1], |[17|2], |[18|1], |[19|0], |[20|1], |[21|0], |[22|0], |[23|0], |[24|0], |[25|0],
 |[26|0], |[27|0], |[28|1],], |N/A], |[1400|[[0|544], |[1|65], |[2|96], |[3|37], |[4|66], |[5|32],
 |[6|33], |[7|9], |[8|16], |[9|12], |[10|7], |[11|7], |[12|7], |[13|5], |[14|4], |[15|5], |[16|3],
 |[17|3], |[18|0], |[19|1], |[20|1], |[21|0], |[22|0], |[23|0], |[24|0], |[25|0], |[26|0], |[27|0],
 |[28|0], |[29|1],], |N/A], |[1425|[[0|575], |[1|57], |[2|111], |[3|37], |[4|63], |[5|36], |[6|36],
 |[7|11], |[8|10], |[9|9], |[10|8], |[11|12], |[12|6], |[13|5], |[14|7], |[15|4], |[16|3], |[17|3],
 |[18|0], |[19|0], |[20|1], |[21|1], |[22|0], |[23|0], |[24|0], |[25|0], |[26|0], |[27|0], |[28|0],
 |[29|1],], |N/A], |[1450|[[0|578], |[1|61], |[2|110], |[3|38], |[4|63], |[5|32], |[6|33], |[7|16],
 |[8|11], |[9|11], |[10|9], |[11|8], |[12|6], |[13|4], |[14|4], |[15|8], |[16|3], |[17|4], |[18|1],
 |[19|1], |[20|0], |[21|0], |[22|2], |[23|0], |[24|0], |[25|0], |[26|0], |[27|0], |[28|0], |[29|1],],
 |N/A], |[1475|[[0|579], |[1|62], |[2|103], |[3|46], |[4|61], |[5|26], |[6|39], |[7|17], |[8|17],
 |[9|10], |[10|7], |[11|9], |[12|4], |[13|5], |[14|3], |[15|7], |[16|3], |[17|4], |[18|2], |[19|1],
 |[20|0], |[21|0], |[22|2],], |N/A], |[1500|[[0|570], |[1|54], |[2|104], |[3|46], |[4|61], |[5|32],
 |[6|29], |[7|19], |[8|15], |[9|9], |[10|8], |[11|7], |[12|5], |[13|4], |[14|4], |[15|2], |[16|2],
 |[17|6], |[18|4], |[19|0], |[20|0], |[21|1], |[22|1], |[23|0], |[24|1],], |N/A], |[1525|[[0|558],
 |[1|57], |[2|100], |[3|42], |[4|57], |[5|33], |[6|27], |[7|19], |[8|19], |[9|16], |[10|6], |[11|9],
 |[12|4], |[13|3], |[14|5], |[15|1], |[16|3], |[17|6], |[18|3], |[19|1], |[20|0], |[21|1], |[22|1],

|[23|0], |[24|0], |[25|1],], |N/A], |[1550|[[0|552], |[1|57], |[2|96], |[3|40], |[4|58], |[5|32],
 |[6|26], |[7|21], |[8|18], |[9|15], |[10|6], |[11|8], |[12|5], |[13|5], |[14|3], |[15|0], |[16|4],
 |[17|3], |[18|2], |[19|4], |[20|2], |[21|1], |[22|0], |[23|0], |[24|0], |[25|0], |[26|1],], |N/A],
 |[1575|[[0|531], |[1|64], |[2|105], |[3|40], |[4|45], |[5|40], |[6|27], |[7|21], |[8|20], |[9|11],
 |[10|4], |[11|6], |[12|6], |[13|5], |[14|2], |[15|0], |[16|5], |[17|0], |[18|2], |[19|6], |[20|2],
 |[21|2], |[22|0], |[23|0], |[24|0], |[25|0], |[26|0], |[27|0], |[28|0], |[29|1],], |N/A],
 |[1600|[[0|524], |[1|63], |[2|104], |[3|45], |[4|39], |[5|37], |[6|28], |[7|15], |[8|24], |[9|4],
 |[10|5], |[11|8], |[12|3], |[13|3], |[14|2], |[15|2], |[16|6], |[17|1], |[18|1], |[19|3], |[20|4],
 |[21|2], |[22|2], |[23|0], |[24|0], |[25|0], |[26|0], |[27|0], |[28|0], |[29|1],], |N/A],
 |[1625|[[0|517], |[1|67], |[2|96], |[3|39], |[4|43], |[5|33], |[6|28], |[7|18], |[8|22], |[9|6],
 |[10|4], |[11|6], |[12|3], |[13|5], |[14|1], |[15|2], |[16|4], |[17|2], |[18|0], |[19|2], |[20|4],
 |[21|3], |[22|1], |[23|0], |[24|0], |[25|1], |[26|0], |[27|0], |[28|0], |[29|0], |[30|0], |[31|1],],
 |N/A], |[1650|[[0|503], |[1|72], |[2|100], |[3|35], |[4|46], |[5|24], |[6|27], |[7|20], |[8|21],
 |[9|6], |[10|6], |[11|5], |[12|4], |[13|3], |[14|3], |[15|3], |[16|3], |[17|2], |[18|1], |[19|2],
 |[20|2], |[21|5], |[22|1], |[23|0], |[24|0], |[25|1], |[26|0], |[27|0], |[28|0], |[29|0], |[30|0],
 |[31|1],], |N/A], |[1675|[[0|514], |[1|58], |[2|101], |[3|49], |[4|43], |[5|28], |[6|26], |[7|19],
 |[8|18], |[9|11], |[10|7], |[11|3], |[12|8], |[13|4], |[14|2], |[15|3], |[16|3], |[17|1], |[18|0],
 |[19|1], |[20|4], |[21|3], |[22|2], |[23|1], |[24|0], |[25|0], |[26|0], |[27|1], |[28|0], |[29|0],
 |[30|0], |[31|1],], |N/A], |[1700|[[0|512], |[1|56], |[2|101], |[3|43], |[4|46], |[5|34], |[6|29],
 |[7|13], |[8|20], |[9|10], |[10|6], |[11|3], |[12|10], |[13|2], |[14|2], |[15|2], |[16|3], |[17|0],
 |[18|0], |[19|1], |[20|4], |[21|3], |[22|1], |[23|1], |[24|0], |[25|0], |[26|0], |[27|1], |[28|0],
 |[29|0], |[30|0], |[31|0], |[32|0], |[33|1],], |N/A], |[1725|[[0|525], |[1|59], |[2|107], |[3|41],
 |[4|45], |[5|41], |[6|20], |[7|14], |[8|18], |[9|13], |[10|6], |[11|5], |[12|10], |[13|2], |[14|4],
 |[15|2], |[16|4], |[17|0], |[18|0], |[19|1], |[20|3], |[21|1], |[22|0], |[23|4], |[24|0], |[25|0],
 |[26|0], |[27|1], |[28|0], |[29|0], |[30|0], |[31|0], |[32|0], |[33|1],], |N/A], |[1750|[[0|515],
 |[1|67], |[2|101], |[3|48], |[4|47], |[5|40], |[6|22], |[7|16], |[8|13], |[9|13], |[10|9], |[11|4],
 |[12|9], |[13|0], |[14|5], |[15|2], |[16|3], |[17|0], |[18|0], |[19|1], |[20|2], |[21|1], |[22|0],
 |[23|3], |[24|1], |[25|0], |[26|0], |[27|1], |[28|0], |[29|0], |[30|0], |[31|0], |[32|0], |[33|1],],
 |N/A], |[1775|[[0|508], |[1|70], |[2|102], |[3|56], |[4|39], |[5|37], |[6|26], |[7|13], |[8|9],
 |[9|12], |[10|9], |[11|5], |[12|7], |[13|4], |[14|2], |[15|3], |[16|5], |[17|0], |[18|1], |[19|0],
 |[20|2], |[21|0], |[22|1], |[23|3], |[24|1], |[25|0], |[26|0], |[27|1], |[28|0], |[29|0], |[30|0],
 |[31|0], |[32|0], |[33|1],], |N/A], |[1800|[[0|503], |[1|69], |[2|112], |[3|49], |[4|41], |[5|28],
 |[6|28], |[7|15], |[8|11], |[9|11], |[10|8], |[11|4], |[12|5], |[13|5], |[14|4], |[15|1], |[16|3],
 |[17|1], |[18|1], |[19|0], |[20|1], |[21|1], |[22|1], |[23|2], |[24|0], |[25|1], |[26|0], |[27|1],],
 |N/A], |[1825|[[0|502], |[1|60], |[2|98], |[3|43], |[4|42], |[5|32], |[6|30], |[7|15], |[8|15],
 |[9|10], |[10|6], |[11|5], |[12|5], |[13|3], |[14|4], |[15|2], |[16|3], |[17|3], |[18|0], |[19|0],
 |[20|1], |[21|1], |[22|0], |[23|1], |[24|1], |[25|1], |[26|0], |[27|0], |[28|1],], |N/A],
 |[1850|[[0|507], |[1|52], |[2|93], |[3|51], |[4|50], |[5|29], |[6|28], |[7|16], |[8|15], |[9|13],
 |[10|6], |[11|2], |[12|4], |[13|4], |[14|3], |[15|3], |[16|2], |[17|3], |[18|1], |[19|0], |[20|1],
 |[21|1], |[22|0], |[23|1], |[24|1], |[25|1], |[26|0], |[27|0], |[28|0], |[29|1],], |N/A],
 |[1875|[[0|484], |[1|52], |[2|92], |[3|45], |[4|52], |[5|28], |[6|27], |[7|16], |[8|14], |[9|11],
 |[10|8], |[11|2], |[12|5], |[13|3], |[14|1], |[15|1], |[16|4], |[17|3], |[18|1], |[19|0], |[20|0],
 |[21|1], |[22|1], |[23|0], |[24|2], |[25|0], |[26|0], |[27|0], |[28|0], |[29|1],], |N/A],
 |[1900|[[0|481], |[1|63], |[2|89], |[3|52], |[4|54], |[5|34], |[6|21], |[7|14], |[8|17], |[9|11],
 |[10|4], |[11|3], |[12|10], |[13|3], |[14|1], |[15|1], |[16|2], |[17|3], |[18|2], |[19|0], |[20|0],
 |[21|0], |[22|1], |[23|0], |[24|2], |[25|0], |[26|0], |[27|0], |[28|0], |[29|1],], |N/A],
 |[1925|[[0|479], |[1|62], |[2|93], |[3|46], |[4|55], |[5|24], |[6|28], |[7|11], |[8|21], |[9|15],
 |[10|2], |[11|3], |[12|9], |[13|5], |[14|1], |[15|0], |[16|1], |[17|2], |[18|1], |[19|1], |[20|0],
 |[21|0], |[22|1], |[23|0], |[24|2], |[25|0], |[26|0], |[27|0], |[28|0], |[29|1],], |N/A],
 |[1950|[[0|486], |[1|59], |[2|106], |[3|48], |[4|50], |[5|25], |[6|29], |[7|13], |[8|15], |[9|17],
 |[10|8], |[11|1], |[12|7], |[13|4], |[14|6], |[15|0], |[16|1], |[17|2], |[18|1], |[19|1], |[20|0],

|[21|0], |[22|0], |[23|0], |[24|1], |[25|0], |[26|0], |[27|0], |[28|0], |[29|1],], |N/A|,
 |[1975|[[0|495], |[1|59], |[2|113], |[3|41], |[4|49], |[5|34], |[6|30], |[7|17], |[8|15], |[9|10],
 |[10|11], |[11|3], |[12|7], |[13|6], |[14|3], |[15|2], |[16|2], |[17|2], |[18|0], |[19|1], |[20|0],
 |[21|0], |[22|0], |[23|0], |[24|1],], |N/A|, |[2000|[[0|493], |[1|51], |[2|93], |[3|60], |[4|44],
 |[5|38], |[6|22], |[7|14], |[8|15], |[9|10], |[10|13], |[11|3], |[12|6], |[13|5], |[14|4], |[15|2],
 |[16|3], |[17|0], |[18|1], |[19|2], |[20|0], |[21|0], |[22|0], |[23|0], |[24|0], |[25|1],], |N/A|,
 |[2025|[[0|493], |[1|62], |[2|96], |[3|54], |[4|47], |[5|35], |[6|22], |[7|16], |[8|13], |[9|12],
 |[10|10], |[11|4], |[12|8], |[13|3], |[14|3], |[15|4], |[16|2], |[17|0], |[18|2], |[19|0], |[20|1],
 |[21|0], |[22|0], |[23|0], |[24|0], |[25|0], |[26|0], |[27|1],], |N/A|, |[2050|[[0|504], |[1|59],
 |[2|108], |[3|51], |[4|50], |[5|32], |[6|30], |[7|16], |[8|8], |[9|10], |[10|12], |[11|6], |[12|6],
 |[13|1], |[14|2], |[15|4], |[16|2], |[17|1], |[18|1], |[19|1], |[20|1], |[21|0], |[22|0], |[23|0],
 |[24|0], |[25|0], |[26|0], |[27|1],], |N/A|, |[2075|[[0|499], |[1|54], |[2|110], |[3|52], |[4|44],
 |[5|32], |[6|31], |[7|17], |[8|9], |[9|6], |[10|12], |[11|4], |[12|6], |[13|2], |[14|1], |[15|1],
 |[16|3], |[17|1], |[18|1], |[19|1], |[20|2], |[21|0], |[22|0], |[23|0], |[24|0], |[25|0], |[26|0],
 |[27|1],], |N/A|, |[2100|[[0|513], |[1|55], |[2|105], |[3|54], |[4|48], |[5|27], |[6|32], |[7|21],
 |[8|15], |[9|4], |[10|9], |[11|4], |[12|5], |[13|3], |[14|3], |[15|1], |[16|3], |[17|1], |[18|1],
 |[19|1], |[20|2], |[21|0], |[22|0], |[23|0], |[24|0], |[25|0], |[26|0], |[27|0], |[28|1],], |N/A|,
 |[2125|[[0|509], |[1|53], |[2|101], |[3|55], |[4|49], |[5|32], |[6|26], |[7|28], |[8|15], |[9|7],
 |[10|5], |[11|5], |[12|6], |[13|2], |[14|5], |[15|1], |[16|1], |[17|2], |[18|1], |[19|1], |[20|1],
 |[21|1], |[22|0], |[23|0], |[24|0], |[25|0], |[26|0], |[27|0], |[28|1],], |N/A|, |[2150|[[0|497],
 |[1|62], |[2|95], |[3|52], |[4|51], |[5|39], |[6|30], |[7|19], |[8|19], |[9|8], |[10|4], |[11|4],
 |[12|6], |[13|3], |[14|5], |[15|1], |[16|0], |[17|2], |[18|0], |[19|1], |[20|1], |[21|1], |[22|0],
 |[23|0], |[24|0], |[25|0], |[26|0], |[27|0], |[28|1],], |N/A|, |[2175|[[0|497], |[1|62], |[2|85],
 |[3|50], |[4|55], |[5|33], |[6|36], |[7|15], |[8|15], |[9|12], |[10|6], |[11|2], |[12|2], |[13|6],
 |[14|3], |[15|3], |[16|1], |[17|2], |[18|0], |[19|0], |[20|1], |[21|0], |[22|0], |[23|1],], |N/A|,
 |[2200|[[0|492], |[1|56], |[2|90], |[3|50], |[4|57], |[5|33], |[6|37], |[7|9], |[8|14], |[9|11],
 |[10|7], |[11|4], |[12|2], |[13|4], |[14|2], |[15|1], |[16|3], |[17|1], |[18|1], |[19|0], |[20|0],
 |[21|1], |[22|0], |[23|1],], |N/A|, |[2225|[[0|485], |[1|53], |[2|101], |[3|53], |[4|50], |[5|29],
 |[6|40], |[7|10], |[8|12], |[9|13], |[10|5], |[11|4], |[12|2], |[13|2], |[14|3], |[15|1], |[16|4],
 |[17|0], |[18|2], |[19|0], |[20|0], |[21|1], |[22|0], |[23|0], |[24|0], |[25|1],], |N/A|,
 |[2250|[[0|482], |[1|55], |[2|96], |[3|47], |[4|56], |[5|29], |[6|30], |[7|12], |[8|13], |[9|18],
 |[10|4], |[11|3], |[12|2], |[13|3], |[14|2], |[15|1], |[16|5], |[17|0], |[18|1], |[19|0], |[20|1],
 |[21|1], |[22|0], |[23|0], |[24|0], |[25|1],], |N/A|, |[2275|[[0|484], |[1|48], |[2|105], |[3|44],
 |[4|53], |[5|35], |[6|23], |[7|17], |[8|15], |[9|12], |[10|6], |[11|4], |[12|2], |[13|3], |[14|3],
 |[15|0], |[16|5], |[17|0], |[18|0], |[19|1], |[20|1], |[21|1], |[22|0], |[23|0], |[24|0], |[25|1],],
 |N/A|, |[2300|[[0|474], |[1|54], |[2|106], |[3|52], |[4|51], |[5|31], |[6|22], |[7|20], |[8|15],
 |[9|11], |[10|7], |[11|3], |[12|4], |[13|3], |[14|2], |[15|1], |[16|2], |[17|1], |[18|1], |[19|1],
 |[20|0], |[21|1], |[22|1], |[23|0], |[24|0], |[25|0], |[26|0], |[27|1],], |N/A|, |[2325|[[0|476],
 |[1|53], |[2|99], |[3|60], |[4|48], |[5|32], |[6|21], |[7|20], |[8|17], |[9|8], |[10|5], |[11|5],
 |[12|4], |[13|4], |[14|2], |[15|1], |[16|2], |[17|1], |[18|1], |[19|1], |[20|0], |[21|1], |[22|1],
 |[23|0], |[24|0], |[25|0], |[26|0], |[27|0], |[28|1],], |N/A|, |[2350|[[0|483], |[1|56], |[2|105],
 |[3|53], |[4|46], |[5|26], |[6|28], |[7|20], |[8|15], |[9|9], |[10|7], |[11|3], |[12|2], |[13|7],
 |[14|1], |[15|1], |[16|0], |[17|2], |[18|1], |[19|1], |[20|1], |[21|1], |[22|0], |[23|0], |[24|1],
 |[25|0], |[26|0], |[27|0], |[28|1],], |N/A|, |[2375|[[0|492], |[1|51], |[2|113], |[3|51], |[4|48],
 |[5|29], |[6|27], |[7|20], |[8|19], |[9|10], |[10|7], |[11|5], |[12|1], |[13|5], |[14|2], |[15|3],
 |[16|0], |[17|2], |[18|1], |[19|0], |[20|1], |[21|0], |[22|0], |[23|0], |[24|0], |[25|0], |[26|1],
 |[27|0], |[28|0], |[29|0], |[30|1],], |N/A|, |[2400|[[0|490], |[1|51], |[2|120], |[3|55], |[4|45],
 |[5|38], |[6|22], |[7|24], |[8|17], |[9|8], |[10|7], |[11|3], |[12|3], |[13|2], |[14|3], |[15|4],
 |[16|0], |[17|2], |[18|1], |[19|0], |[20|1], |[21|0], |[22|0], |[23|0], |[24|0], |[25|0], |[26|1],
 |[27|0], |[28|0], |[29|0], |[30|1],], |N/A|, |[2425|[[0|513], |[1|57], |[2|99], |[3|64], |[4|56],
 |[5|35], |[6|24], |[7|21], |[8|17], |[9|13], |[10|7], |[11|5], |[12|1], |[13|2], |[14|1], |[15|6],

Review

The program is now at a state at which the user can interact with the simulation and the meta elements to enhance their own experience. This phase focused on presentation to the user and included the creation of most usability features.

There is now a pygame window for simulation which contains the simulation view, buttons and sliders for user input, a fact file view which can be displayed if an anima is clicked on, music, and tutorials to guide the user.

There is also an initial pygame setup window which allows the user to adjust the attributes of the realm to their liking to vary the evolutionary effects.

I have **tested** this phase by running many simulations and observing how the animas move, change states and interact with each other and the natural elements, seeing if this checks out with how I imagined it to be implemented and ensuring all basic functions are completed in an intuitive fashion in relation to goals.

The phase goes a long way toward meeting my **success criteria** because I have now create a clear and intuitive graphical representation of the mathematics through the simulation view, which is criterion 2. I have implemented a customisable simulation set up allowing the user to test many different scenarios, which is criterion 4. From the basic function of the simulation and the informative tutorials I have created a program which enhances user understanding of evolution, which is criterion 5.

The following phase will wrap up the success criteria by presenting the population data in a more tangible fashion which allows for further analysis of the patterns, trends and potential outliers.

PHASE 4~ DATA

Design

The data tab is one of the chief essential features considering the project's educational nature.

It's important to maximise the genuinely insightful information given to the user.

The data tab will include information on the starting position of the simulation, with the distribution of traits, population counts, etc

Some trends and insights which I will implement on account of their informational value are as follows:

Each of these can be represented by bar graphs where each bar represents a day and is broken down into key-coloured subcategories; those marked (*) are represented also by line graphs which show continuous change and update with each tick of the simulation (or perhaps at a lower frequency should this cause performance issues):

- (*) Population data (moving average, variance) of each individual variable value, ie trait values, activity and age
- (*) Population count
- (*) Biodiversity
- Distribution of cause of death

These can be represented by a visual map of the simulation space:

- Geographical distribution of creatures
- Distribution of trait values across the space

I will need to use *openpyxl* to access and modify an Excel file from within the program; this file will store the same data as the tab but represented in Excel's spreadsheet form. The idea is for the user to be able to download the file should they wish to save a copy of the data from a single simulation. The file will be downloadable as many times as necessary and will be cleared every time the user starts a new simulation.

I have verified that *openpyxl* can perform the tasks required of it, namely:

- Create new Excel files without the need to install Excel
- Write to Excel files
- Read from Excel files
- Manipulate Excel cells
- Generate Excel charts and tables
- Support the downloading of the Excel files

First I will need to install *openpyxl* using PIP; this will be required on every device I use to program the project.

Development

I pip installed openpyxl onto the relevant computers.

I decided to implement the interface of the realm database, which was already created for this future use as well as testing throughout phases 2 and 3, as an integrated terminal space in the pygame window.

The terminal is realised as an object with its content stored as an attribute which is updated with each print line with a new row from the bottom, meaning the earliest row is deleted and all row indexes are decremented. All print lines have data cleaned by my 'todisplay' function; this is especially helpful for making long float values which may have a long string of repeated numbers due to floating point imprecision (ie 0.777777777) easy to read for the user (the example case becomes simply 0.78).

The creation of this terminal was made easier due to the work I have already done in the earlier modules such as todisplay, ui, and realmdatabase (none of these are themselves imported into the ui_terminal file, but they are used implicitly as they are involved heavily in what is passed into the content attribute through the print lines). This is a clear example of how the modularity of the program helps with further development and maintainability; I have used the OOP paradigm well here.

In order to make print lines connect directly to the terminal, I had to overwrite the system standard output (sys.__stdout__).

The terminal had a maximum width to keep it compact in the bottom left corner of the window. Some lines had a length which exceeded this maximum, and I handled this by breaking these lines down into smaller lines which went up until the limit before wrapping down to the line below. This is made easier due to the fact I opted to use consolas, a monospaced font. The monospacing means I can set the terminal width limit to be some arbitrary number which fits the aesthetic requirement and all terminal lines will be able to uniformly wrap around with each exceeding line reaching exactly up to the limit as each character has uniform width. This creates an effect of neatness, seen in the example screenshot below:

```
Krapottyc [apple], {'growth-syndrome', 'strong-gut', 'claws'} 6.4 7.1
was born to Veimmosymm [apple], {'claws'} 7.2 6.8 and Ulegladuip [ap
ple], {'strong-gut'} 7.1 7.3!
Eivuxogazz [apple], {'abrupt-wanderer'} 5.9 7.9 ate apple
Anaxx [apple], {'abrupt-wanderer', 'poisonous'} 5.9 7.1 ate apple
Eimiz [apple], {'poisonous', 'strong-gut'} 4.7 7.1 ate apple
Vekkaov [apple], {'plate-armour', 'strong-gut', 'play-dead'} 5.3 5.5
ate apple
Uithidole [apple], {'abrupt-wanderer'} 7.2 6.8 ate apple
Aokwubeb [apple], {'claws'} 8.2 6.2 ate apple
Hazetoott [apple], {'strong-gut'} 5.6 5.7 died of starvation aged 397
```


Testing

44

10000

None

[210|0|0|120|0],

```

[[25|[[lily], |[2.0|0.5|0.3], |[22.4|10.0|10.1|10.0|25.1|0.5|64.8|82.6|22.5], ], |[50|[[lily],
|[2.0|0.51|0.31], |[23.1|10.2|10.5|10.3|25.8|0.51|66.4|82.7|23.0], ], |[75|[[melon],
|[2.1|0.54|0.32], |[24.0|10.6|10.9|10.9|27.0|0.54|68.7|83.7|23.9], ], |[100|[[lavendar],
|[2.1|0.56|0.33], |[24.7|10.7|11.2|11.3|27.7|0.55|70.7|84.4|24.7], ], |[125|[[melon],
|[2.2|0.57|0.34], |[25.2|10.8|11.2|11.5|28.3|0.56|71.9|86.3|24.7], ], |[150|[[wildgrass],
|[2.2|0.57|0.34], |[25.3|10.8|11.3|11.6|28.4|0.57|72.5|86.1|25.4], ], |[175|[[wildgrass],
|[2.2|0.58|0.35], |[26.0|10.8|11.6|11.9|29.2|0.58|73.7|86.1|25.7], ], |[200|[[wildgrass],
|[2.3|0.59|0.36], |[26.4|10.8|11.7|12.1|29.7|0.58|74.2|86.7|25.4], ], |[225|[[melon],
|[2.3|0.61|0.36], |[27.2|10.7|11.9|12.4|30.4|0.61|75.3|87.4|25.1], ], |[250|[[melon],
|[2.3|0.62|0.37], |[27.7|10.7|12.1|12.6|30.9|0.62|75.9|88.7|24.9], ], |[275|[[melon],
|[2.4|0.63|0.37], |[27.7|10.7|12.2|12.7|31.2|0.64|77.2|89.8|24.8], ], |[300|[[melon],
|[2.4|0.64|0.38], |[28.4|10.8|12.3|12.9|31.8|0.65|78.0|90.6|24.8], ], |[325|[[melon],
|[2.5|0.65|0.38], |[29.0|11.0|12.3|13.1|32.3|0.67|78.5|91.4|24.4], ], |[350|[[melon],
|[2.5|0.65|0.38], |[29.5|11.3|12.3|13.0|32.7|0.68|79.6|91.3|23.9], ], |[375|[[melon],
|[2.6|0.67|0.39], |[30.4|11.5|12.4|13.5|33.5|0.7|80.5|91.6|23.9], ], |[400|[[lily], |[2.7|0.68|0.39],
|[30.7|11.7|12.2|13.6|33.6|0.72|81.0|91.6|24.2], ], |[425|[[lily], |[2.7|0.67|0.39],
|[30.4|11.8|12.2|13.7|33.4|0.72|81.0|91.5|24.3], ], |[450|[[lily], |[2.7|0.68|0.39],
|[30.9|11.9|12.2|13.9|33.7|0.73|81.1|91.7|24.1], ], |[475|[[lily], |[2.7|0.68|0.39],
|[30.8|11.8|12.2|13.9|33.7|0.72|80.9|91.6|23.8], ], |[500|[[lily], |[2.7|0.68|0.39],
|[31.0|11.9|12.0|13.9|33.8|0.74|81.1|91.6|23.7], ], |[525|[[lily], |[2.7|0.66|0.39],
|[30.6|11.9|11.8|13.7|33.5|0.72|80.9|92.1|23.0], ], |[550|[[lily], |[2.7|0.66|0.39],
|[30.4|11.8|11.7|13.5|33.2|0.72|80.7|92.0|22.8], ], |[575|[[lily], |[2.8|0.66|0.39],
|[30.8|11.9|11.7|13.7|33.5|0.74|81.0|92.2|22.4], ], |[600|[[lily], |[2.8|0.67|0.39],
|[30.9|11.9|11.7|13.8|33.6|0.74|81.1|92.3|22.3], ], |[625|[[lily], |[2.8|0.66|0.39],
|[31.0|12.0|11.6|13.8|33.6|0.74|81.1|92.3|21.6], ], |[650|[[lily], |[2.8|0.67|0.39],
|[31.1|12.0|11.5|13.8|33.7|0.75|81.1|92.6|21.4], ], |[675|[[lily], |[2.9|0.67|0.39],
|[31.4|12.0|11.6|14.0|33.8|0.76|81.5|92.7|21.0], ], |[700|[[lily], |[2.8|0.67|0.39],
|[30.9|11.9|11.5|13.8|33.4|0.74|81.1|92.5|21.4], ], |[725|[[lily], |[2.8|0.67|0.39],
|[30.6|11.9|11.5|13.7|33.3|0.74|81.3|92.6|21.4], ], |[750|[[lily], |[2.8|0.66|0.38],
|[30.1|11.8|11.5|13.5|33.0|0.73|80.8|92.5|21.8], ], |[775|[[lily], |[2.7|0.65|0.38],
|[29.7|11.8|11.3|13.2|32.5|0.71|80.5|92.2|21.0], ], |[800|[[lily], |[2.6|0.64|0.37],
|[28.8|11.6|11.2|12.8|31.8|0.68|80.1|91.9|21.7], ], |[825|[[lily], |[2.6|0.64|0.37],
|[28.8|11.6|11.2|12.8|31.8|0.68|80.1|91.9|21.7], ], |[850|[[lily], |[2.6|0.64|0.37],
|[28.5|11.5|11.2|12.7|31.6|0.68|79.9|91.8|21.9], ], |[875|[[lily], |[2.6|0.63|0.36],
|[27.9|11.4|11.3|12.5|31.1|0.66|79.8|91.4|22.4], ], |[900|[[lily], |[2.6|0.63|0.36],
|[28.0|11.5|11.2|12.5|31.2|0.66|80.0|91.8|22.2], ], |[925|[[berry], |[2.5|0.63|0.36],
|[27.7|11.4|11.2|12.3|31.0|0.65|79.8|91.7|22.4], ], |[950|[[berry], |[2.5|0.63|0.36],
|[27.7|11.3|11.1|12.4|30.9|0.65|79.4|91.6|22.7], ], |[975|[[berry], |[2.5|0.63|0.36],
|[27.7|11.3|11.1|12.4|30.9|0.65|79.4|91.6|22.7], ], |[1000|[[berry], |[2.6|0.63|0.36],
|[27.8|11.4|11.2|12.5|31.0|0.66|79.5|92.2|22.6], ], |[1025|[[berry], |[2.5|0.63|0.36],
|[27.7|11.4|11.0|12.4|30.8|0.66|79.2|92.3|22.5], ], |[1050|[[berry], |[2.6|0.63|0.36],
|[27.7|11.4|11.1|12.5|30.8|0.66|79.0|92.7|22.6], ], |[1075|[[berry], |[2.6|0.63|0.35],

```

27.7 11.4 11.1 12.4 30.8 0.66 78.8 92.6 22.6],	,	[1100 [[berry],	[2.6 0.63 0.35],
27.7 11.4 11.1 12.4 30.8 0.66 78.8 92.6 22.6],	,	[1125 [[berry],	[2.6 0.63 0.36],
27.8 11.5 11.1 12.4 30.9 0.66 78.7 92.4 22.4],	,	[1150 [[berry],	[2.6 0.63 0.36],
27.8 11.5 11.1 12.4 30.9 0.66 78.7 92.4 22.4],	,	[1175 [[berry],	[2.5 0.62 0.35],
27.6 11.4 11.0 12.3 30.8 0.66 78.5 92.4 22.1],	,	[1200 [[berry],	[2.5 0.62 0.35],
27.7 11.4 11.1 12.3 30.8 0.66 78.8 92.4 22.2],	,	[1225 [[berry],	[2.6 0.63 0.35],
27.8 11.5 11.1 12.4 30.9 0.67 78.5 92.3 22.2],	,	[1250 [[berry],	[2.6 0.63 0.35],
27.8 11.5 11.1 12.4 30.9 0.67 78.5 92.3 22.2],	,	[1275 [[berry],	[2.6 0.63 0.35],
27.8 11.5 11.1 12.4 30.9 0.67 78.5 92.3 22.2],	,	[1300 [[berry],	[2.6 0.63 0.35],
27.8 11.5 11.1 12.4 30.9 0.67 78.5 92.3 22.2],	,	[1325 [[berry],	[2.6 0.63 0.35],
27.8 11.5 11.1 12.4 30.9 0.67 78.5 92.3 22.2],	,	[1350 [[berry],	[2.5 0.62 0.35],
27.4 11.4 11.1 12.2 30.6 0.65 78.3 92.3 22.5],	,	[1375 [[berry],	[2.5 0.61 0.35],
27.4 11.4 11.0 12.1 30.5 0.65 78.0 92.2 21.7],	,	[1400 [[berry],	[2.5 0.61 0.35],
27.4 11.4 11.0 12.1 30.5 0.65 78.0 92.2 21.7],	,	[1425 [[berry],	[2.5 0.61 0.35],
27.4 11.4 11.0 12.1 30.5 0.65 78.0 92.2 21.7],	,	[1450 [[berry],	[2.5 0.61 0.35],
27.4 11.4 11.0 12.1 30.5 0.65 78.0 92.2 21.7],	,	[1475 [[berry],	[2.5 0.62 0.35],
27.6 11.5 11.0 12.2 30.5 0.65 77.9 92.1 21.6],	,	[1500 [[berry],	[2.5 0.62 0.35],
27.6 11.5 11.0 12.2 30.5 0.65 77.9 92.1 21.6],	,	[1525 [[berry],	[2.5 0.61 0.35],
27.1 11.4 11.0 12.0 30.2 0.64 77.6 92.0 21.7],	,	[1550 [[berry],	[2.5 0.61 0.35],
27.1 11.4 11.0 12.0 30.2 0.64 77.6 92.0 21.7],	,	[1575 [[berry],	[2.5 0.61 0.35],
27.0 11.3 10.9 11.9 30.0 0.64 77.3 91.9 21.7],	,	[1600 [[berry],	[2.5 0.61 0.35],
27.0 11.3 10.9 11.9 30.0 0.64 77.3 91.9 21.7],	,		
[1625 [[berry],	[2.5 0.61 0.35],	27.0 11.3 10.9 11.9 30.0 0.64 77.3 91.9 21.7],	,
[1650 [[berry],	[2.5 0.61 0.35],	27.0 11.4 11.0 12.0 30.1 0.64 77.5 91.6 21.9],	,
[1675 [[berry],	[2.5 0.61 0.35],	27.1 11.4 11.0 12.0 30.1 0.64 78.0 91.9 21.9],	,
[1700 [[berry],	[2.5 0.61 0.35],	27.2 11.3 10.9 12.0 30.1 0.65 77.8 92.3 22.1],	,
[1725 [[berry],	[2.5 0.61 0.35],	27.2 11.3 10.9 12.0 30.1 0.65 77.8 92.3 22.1],	,
[1750 [[berry],	[2.5 0.61 0.35],	27.2 11.3 10.9 12.0 30.1 0.65 77.8 92.3 22.1],	,
[1775 [[berry],	[2.5 0.61 0.35],	27.2 11.3 10.9 12.0 30.1 0.65 77.8 92.3 22.1],	,
[1800 [[berry],	[2.5 0.61 0.35],	27.2 11.3 10.9 12.0 30.1 0.65 77.8 92.3 22.1],	,
[1825 [[berry],	[2.5 0.61 0.35],	27.2 11.3 10.9 12.0 30.1 0.65 77.8 92.3 22.1],	,
[1850 [[berry],	[2.5 0.61 0.35],	27.2 11.3 10.9 12.0 30.1 0.65 77.8 92.3 22.1],	,
[1875 [[berry],	[2.5 0.61 0.35],	27.2 11.3 10.9 12.0 30.1 0.65 77.8 92.3 22.1],	,
[1900 [[berry],	[2.5 0.61 0.35],	27.2 11.3 10.9 12.0 30.1 0.65 77.8 92.3 22.1],	,
[1925 [[berry],	[2.5 0.61 0.35],	27.2 11.3 10.9 12.0 30.1 0.65 77.8 92.3 22.1],	,
[1950 [[berry],	[2.5 0.61 0.35],	27.2 11.3 10.9 12.0 30.1 0.65 77.8 92.3 22.1],	,
[1975 [[berry],	[2.5 0.61 0.35],	27.2 11.3 10.9 12.0 30.1 0.65 77.8 92.3 22.1],	,
[2000 [[berry],	[2.5 0.61 0.35],	27.1 11.4 11.0 12.1 30.2 0.65 77.5 92.3 22.2],	,
[2025 [[berry],	[2.5 0.62 0.35],	27.2 11.4 11.0 12.1 30.3 0.64 77.9 92.0 22.0],	,
[2050 [[berry],	[2.5 0.62 0.35],	27.2 11.4 11.0 12.1 30.3 0.64 77.9 92.0 22.0],	,
[2075 [[berry],	[2.5 0.62 0.35],	27.2 11.4 11.0 12.1 30.3 0.64 77.9 92.0 22.0],	,
[2100 [[berry],	[2.5 0.62 0.35],	27.2 11.4 11.0 12.1 30.3 0.64 77.9 92.0 22.0],	,
[2125 [[berry],	[2.5 0.62 0.35],	27.2 11.4 11.0 12.1 30.3 0.64 77.9 92.0 22.0],	,
[2150 [[berry],	[2.5 0.62 0.35],	27.2 11.4 11.0 12.1 30.3 0.64 77.9 92.0 22.0],	,
[2175 [[berry],	[2.5 0.62 0.35],	27.2 11.4 11.0 12.1 30.3 0.64 77.9 92.0 22.0],	,
[2200 [[berry],	[2.5 0.62 0.35],	27.2 11.4 11.0 12.1 30.3 0.64 77.9 92.0 22.0],	,
[2225 [[berry],	[2.5 0.62 0.35],	27.2 11.4 11.1 12.0 30.4 0.64 78.0 92.1 21.7],	,
[2250 [[berry],	[2.5 0.62 0.35],	27.2 11.4 11.1 12.0 30.4 0.64 78.0 92.1 21.7],	,

Review

The phase focused on the presentation of simulation data to the user.

There is now an integrated terminal logging many events simulation-wide. The user can trigger the dumping of database records into the terminal.

I **tested** this phase by running the simulation and moving the terminal around to ensure correct grid spacing.

This phase helps achieve the success criteria by handling criterion

The program is now complete.

EVALUATION

On evaluation of the project I find the central task achieved to sufficient precision. The program is able to be used by students and curious individuals in an exercise in how natural selection drives evolution. The unpredictable complexities and patterns which emerge from the simulation are both entertaining to observe and analyse, and illuminating from an educational standpoint.

After completion of testing, I showed the project to numerous stakeholders, among which were those I had initially interviewed during the analysis phase, and the feedback was overwhelmingly positive. Stakeholders described how they were able to use the striking visuals-focused simulation in order to conduct and play around with their own explorations on evolution. Due to the high customisability and combinatorically high-infinite potential of the simulated world, stakeholders were able to adjust any few variables they wanted to see how it would affect the simulation, and the results, while often in line with expectations, often also subverted them in good teaching moments that stakeholders could then correct their hypotheses from; stakeholders claimed that it was satisfying to see how tweaking a couple variables could cause new species or archetypes of creatures to emerge.

I conducted a stakeholder poll and these were the results:

Evolution Simulator Feedback Survey

To extract stakeholder thoughts and feedback on the computational evolution simulator

- To what degree is the visual logic sound and graphics appropriate for **representing** the world, plants and animals the interactions between them in the simulation?

nothing makes sense

I can see and understand what is going on
 (how the world and dynamics between elements are portrayed via the simulation objects)
- Explain your answer to the first question in as many words as necessary

Enter your answer
- Do you think the program is acceptable for school use? (ie no explicit content, focus on educational value, not over-stimulating or under-stimulating)

Enter your answer
- Was the program **easy/intuitive** to use? Did you understand how/Were you able to **adjust the simulation** in order to produce effects on the world and animals?

Enter your answer

5. How **strongly** do you agree or disagree with the following? ***

	Disagree	Somewhat disagree	Neutral	Somewhat agree	Agree
The UI was intuitive	☐	☐	☐	☐	☐
The tutorial texts were helpful	☐	☐	☐	☐	☐
The music added to the experience	☐	☐	☐	☐	☐
The UI did not overwhelm me	☐	☐	☐	☐	☐
The simulation was aesthetically nice	☐	☐	☐	☐	☐
I developed my knowledge of evolution	☐	☐	☐	☐	☐
I had a good experience with the program	☐	☐	☐	☐	☐
I would recommend this program to students as a tool for studying evolution	☐	☐	☐	☐	☐

6. Were you able to identify each of the following in the simulation?

	No	Yes
Animal eating food	☐	☐
Predator hunting prey	☐	☐
Animal drinking water	☐	☐
Plants bearing fruit	☐	☐
Animals mating and producing offspring	☐	☐
Animal dies	☐	☐
Population appearances and traits changing over time (evolution)	☐	☐

7. Did you encounter any issues or bugs?

Enter your answer

8. Please share any additional suggestions or comments

Enter your answer

Achievement of Success Criteria

Here is the success criteria which was formulated in the analysis phase, re-printed here for convenience of cross-checking:

《Success Criteria》

The computational solution will be successful if it fulfils the following criteria, composed of the requirements as outlined in the previous section:

1. Mathematically simulates how organisms can evolve under selection pressure
 - A population of organisms has traits which change each generation
 - Organisms inherit traits as some combination of their parents' traits and random mutation
 - Different combinations of traits in an individual affect its tendency to survive and to reproduce
 - Traits correlating with tendency to survive and to reproduce are likely to be better represented in each passing generation, as opposed to less represented
2. Provides a clear and intuitive visualisation of the mathematics
 - There is a stakeholder consensus that the visual logic is sound and graphics appropriate
 - Organisms with differing traits have corresponding* differences in their visual representation (*these may not be 1-to-1, or actual in some cases)
3. Is compatible with a school environment
 - The software can be run on KEGS school monitors
 - There is a stakeholder consensus that the program remains within the bounds of an appropriate educational school project
4. Has highly interactive and customisable setup *so users can test many different scenarios*
 - The starting population, initial traits of organisms, and initial environment layout are determined by user inputs
 - Users can modify organism appearance to create their own 'species' for ease of visual following and differentiation
 - There are at least three distinct trait combinations which populations can converge to, depending on the starting position
 - There is at least one situation in which all organisms become extinct
 - The stable range of population differs depending on the environment setup
 - The environment can be altered in such a way that 'speciation occurs', where two populations converge differently in a single simulation
5. Enhances user understanding of evolution
 - The program is shown to be conducive to greater understanding of evolutionary mechanics
 - The program includes written information explaining evolutionary concepts
6. Presents well-organised and informative data regarding the simulation
 - The program features a section where data and trends can be viewed
 - Each trait and population variable is featured in this section; the average value is graphed alongside individual values over time, with key values such as the maximum and minimum, and variance, made clear
 - Data on a simulation can be downloaded onto the user's computer, where it may be stored as either a screen capture, flat file, or text document detailing key values

Link to where outlined in the analysis section: *Success Criteria*

In the following section I will give my honest reflection on what I believe is the extent to which the success criteria has been achieved, taking each criterion in isolation and providing evidence to support my claims.

There are six general criteria, each with its own sub-criteria which together are indicative of, but do not exhaust, the ways in which the key point of the criterion may be demonstrated. I will go into detail addressing each sub-criteria and then provide a final evaluation for each criterion

1. Mathematically simulates how organisms can evolve under selection pressure

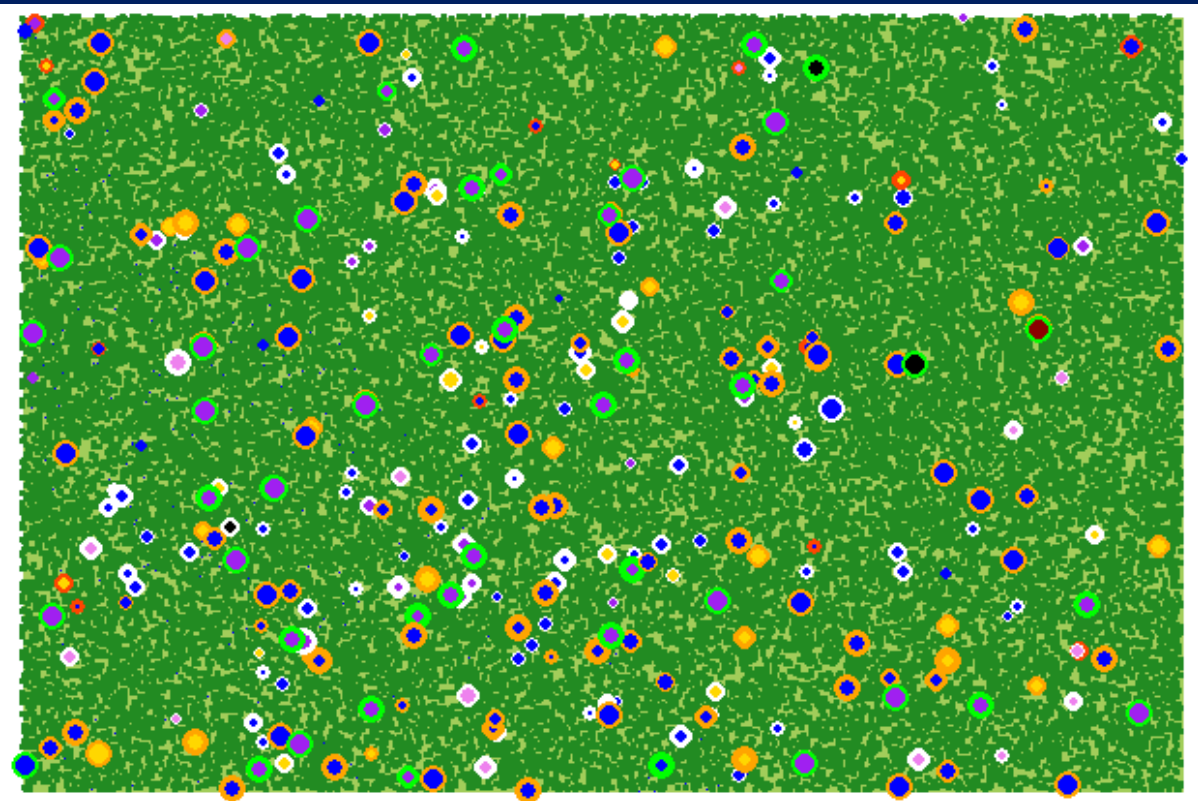
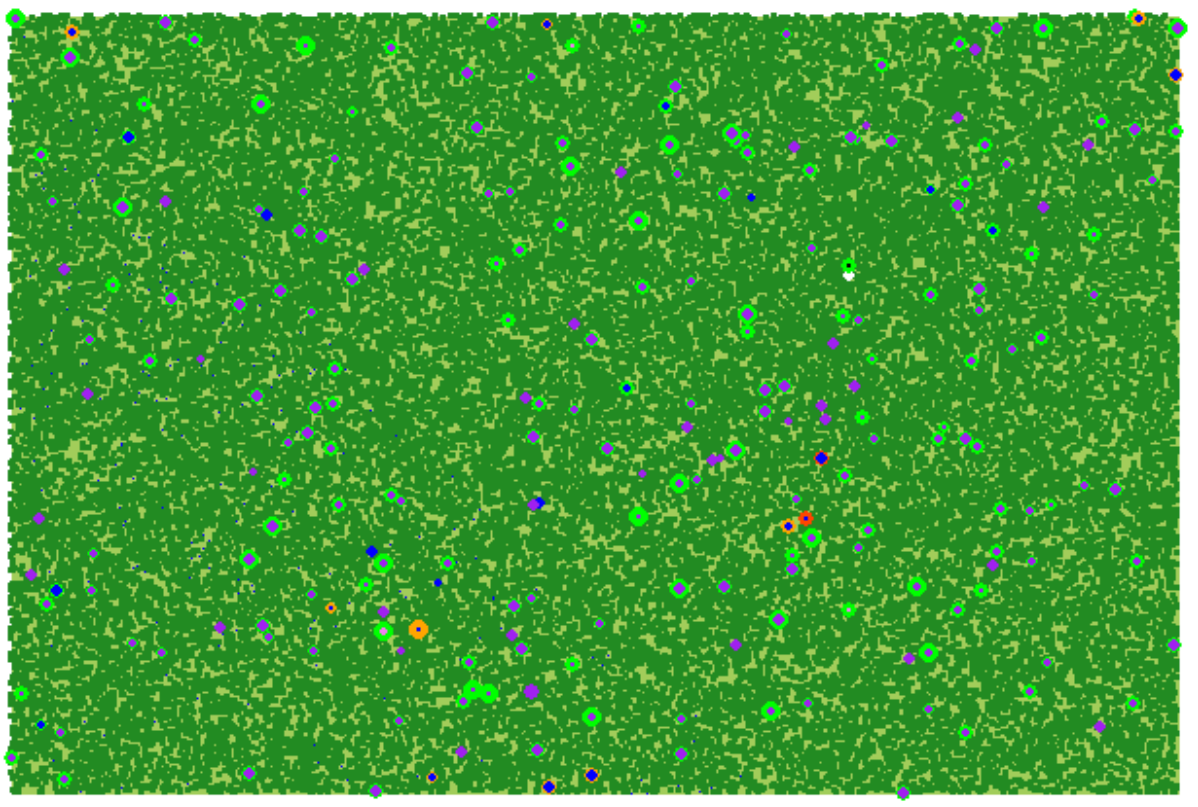
- A population of organisms has traits which change each generation ✓
- Organisms inherit traits as some combination of their parents' traits and random mutation ✓
- Different combinations of traits in an individual affect its tendency to survive and to reproduce ✓
- Traits correlating with tendency to survive and to reproduce are likely to be better represented in each passing generation, as opposed to less represented ✓

I believe this criterion has been fully achieved. There are many ways in which these sub-criteria could have been achieved in a simulation; since I have access to both the visual pygame representation of the simulation and the terminal in which lines describing simulation events are printed in bulk (alongside the realm database, at the termination of the program), I can provide evidence towards each of these points at any given tick of the thousands of ticks which occur during a simulation run. I have created a system in which evolution and the selection of traits occurs systematically as per implicit mathematical laws, and though chance is involved there is on the whole a predictable mathematical model driving anima traits toward fitness; this is evident in the fact that over the many simulations I have run, traits slowly converge toward some value over time (though this value can also change throughout the simulation in response to environmental changes); and through multiple simulations it is evident that traits converge similarly in most, with major variance between simulations unlikely when environmental variables are kept constant.

A population of organisms has traits which change each generation.



It is clear from the following photos, taken at different times during a simulation, that a population of organisms can have traits which change from one generation to the next:



Each circle on the plane represents an organism; the collective at a given point in time is the population; one example of a trait which can be seen to change between the two populations is the size of the anima; the circles in the second population appear to be in general larger than those in the

first; the size of the anima in the simulation view is fixed directly to their size trait in the simulation logic.

There is the objection that perhaps traits may change over many generations, but what of the wording ‘each generation’ which implies the more specific case that every pair of consecutive generations must see a change in organism traits.

For a more concrete piece of evidence towards this, the following data taken as a snippet from the displayed realm database during a simulation run spans a couple generations, and the average genome can be seen to slowly drift during this time:

```
[1400|[[apple], |[3.7|[1.8|0.82|0.53], ], |38.5|15.6|11.7|12.5|43.7|0.87|93.2|98.7|37.5|[], ], ],
[1410|[[apple], |[3.7|[1.8|0.82|0.53], ], |39.1|15.8|11.8|12.6|44.1|0.87|94.0|99.4|38.1|[], ], ],
[1420|[[apple], |[3.7|[1.8|0.82|0.53], ], |39.1|15.8|11.8|12.6|44.1|0.87|94.0|99.4|38.1|[], ], ],
[1430|[[apple], |[3.7|[1.8|0.82|0.56], ], |40.0|15.8|12.0|13.0|44.8|0.87|92.8|99.3|40.8|[], ], ],
[1440|[[apple], |[3.6|[1.7|0.8|0.56], ], |39.5|16.0|11.5|12.6|44.1|0.89|95.1|99.1|38.5|[], ], ],
[1450|[[apple], |[3.3|[1.7|0.77|0.53], ], |38.8|15.2|11.1|12.3|42.9|0.87|94.8|99.1|38.2|[], ], ],
[1460|[[apple], |[3.4|[1.8|0.77|0.54], ], |39.5|15.4|11.3|12.5|43.4|0.88|94.1|98.9|40.7|[], ], ],
[1470|[[apple], |[3.4|[1.8|0.78|0.54], ], |39.9|15.7|11.2|12.7|43.3|0.83|96.0|100.0|37.3|[], ], ],
[1480|[[apple], |[3.4|[1.8|0.78|0.54], ], |39.9|15.7|11.2|12.7|43.3|0.83|96.0|100.0|37.3|[], ], ],
[1490|[[apple], |[3.4|[1.8|0.78|0.54], ], |39.9|15.7|11.2|12.7|43.3|0.83|96.0|100.0|37.3|[], ], ],
[1500|[[apple], |[3.4|[1.8|0.78|0.54], ], |39.9|15.7|11.2|12.7|43.3|0.83|96.0|100.0|37.3|[], ], ],
[1510|[[apple], |[3.4|[1.8|0.78|0.54], ], |39.9|15.7|11.2|12.7|43.3|0.83|96.0|100.0|37.3|[], ], ],
[1520|[[apple], |[3.3|[1.7|0.78|0.54], ], |39.7|15.8|11.2|12.7|42.9|0.85|95.6|100.0|38.0|[], ], ],
[1530|[[apple], |[3.2|[1.7|0.77|0.54], ], |39.7|15.8|11.1|12.5|42.2|0.84|95.2|100.0|34.2|[], ], ],
[1540|[[apple], |[3.2|[1.7|0.77|0.54], ], |39.7|15.8|11.1|12.5|42.2|0.84|95.2|100.0|34.2|[], ], ],
[1550|[[apple], |[3.2|[1.7|0.77|0.54], ], |39.7|15.8|11.1|12.5|42.2|0.84|95.2|100.0|34.2|[], ], ],
[1560|[[apple], |[3.2|[1.7|0.77|0.54], ], |39.7|15.8|11.1|12.5|42.2|0.84|95.2|100.0|34.2|[], ], ],
[1570|[[apple], |[3.2|[1.7|0.74|0.53], ], |38.8|15.7|11.3|12.5|41.4|0.83|94.6|100.0|35.1|[], ], ],
[1580|[[apple], |[3.2|[1.7|0.74|0.53], ], |38.8|15.7|11.3|12.5|41.4|0.83|94.6|100.0|35.1|[], ], ],
[1590|[[apple], |[3.4|[1.6|0.73|0.54], ], |39.0|16.6|11.4|12.8|41.6|0.84|94.0|100.0|36.4|[], ], ],
```

Organisms inherit traits as some combination of their parents’ traits and random mutation. 

When two animas have an offspring, the value of each of its (real number) traits can be modelled using a gaussian distribution with mean equal to the mean of its parents values for the same trait, and variance equal to the standard realm mutation standard deviation, which is customisable by the user (and by default 10% of the mean)

A mock reproductive simulation is performed below to show how the mean and variance of a trait should vary over time; this is a standard Gaussian random walk.

```
0 99.94564697465093 9.9423140981101
20 100.44658968707903 13.823677833518126
40 100.62775068798749 14.267388532403055
60 99.81446074141728 13.2576476735998
80 98.07030106543284 14.413621217404065
100 101.20643243333001 14.48088955271968
120 101.77401021930217 14.600515943595205
140 105.63736432104503 14.73907612734889
160 101.17302381131206 14.924740457944289
180 100.31571279586234 14.045218004889222200 100.99302544533072 13.484650595679147
220 100.01328329178625 14.318600041854431
240 99.07003248366142 14.104490031943238
260 100.0001899731424 14.00025791336324
280 97.68346151804263 13.813710592641678
300 94.59460077668925 13.184446056476956
320 93.9990010266584 12.914200075778387
340 91.70319376839196 12.82507454140902
360 90.9517794893492 12.81289905363239
```

```

380 92.7741777761976 13.202371466160162
400 96.16691535848899 13.559362493854913
420 93.83446734406837 13.166050814018641
440 93.73799218840823 13.638633939867628
460 93.83148839303043 12.867120118638239
480 94.5834294313853 13.627782183798958
500 91.96816272909591 12.638877555399743
520 92.17946921106042 14.109718708292073
540 90.10298476238673 12.881788262147456
560 86.16750401250388 12.111924799571172
580 84.8770183844489 11.852771615245423
600 85.1405095002845 12.18614717885423
620 83.02981523163163 11.794944782200544
640 83.69494562369354 11.936442403682165
660 86.05523322609167 12.390346327765737
680 87.93279682810626 12.07324322613115
700 87.82490459753518 12.688462831537688
720 87.58627222211878 12.061159979139413
740 88.2691380551624 13.286905734803664
760 90.00127252435807 12.704341402014897
780 91.62484891480239 13.373329238604997
800 90.11152861899247 12.840983765816642
820 93.23486746521677 13.213924019474131
840 93.38942698883535 12.915274940018811
860 94.44199916237669 13.287930073825429
880 95.93589361862121 13.417847695131867
900 95.40597761563538 13.891634205074698
920 96.76401733264602 13.383745411975037
940 93.90688903657392 13.941939527798088
960 93.26213077404319 12.933999044933634
980 92.43008091670461 13.0197239040004
1000 94.15855646104754 13.824399761304154
...
...
...
19000 201.232154929619 29.704317649674163
19020 204.47651124035792 28.7911073869536
19040 209.67348227275664 29.798328736251424
19060 216.9618626733856 29.90879594664581
19080 221.7667479071135 30.153032294602454
19100 222.2496994569982 31.215406503389378
19120 220.66347373444998 31.33682281166524
19140 217.99411842691123 31.533466193497727
19160 219.296051765844 32.58367815118836
19180 221.05412898841993 30.621672538996023
19200 220.00458030090618 32.18991512418776
19220 216.8918945770787 30.325258418057047
19240 210.48399727685367 31.082226633813015
19260 205.5284407835331 29.64097922077969
19280 204.46149795955694 28.900310472660916
19300 211.5708663247172 28.878638428576114
19320 200.85146505803192 27.968875519544785
19340 200.09068333348793 28.952348073234624
19360 191.53251394952176 27.286680187062473
19380 189.35906349832044 28.41316736842405
19400 183.22506720302647 26.191486093229052
19420 186.56045458867524 25.292398249125633
19440 183.74704943646012 26.485097494855336
19460 182.36256369660114 25.79129222338385
19480 178.58607573466315 25.11763742207515
19500 178.93558926858768 26.261445896681106
19520 185.83582554720616 26.24201804598396
19540 191.72009352792048 27.274495830661472
19560 192.61302852969783 26.18858977138576
19580 197.88464091415787 29.66403196838989
19600 196.321077396338 28.49058930160555
19620 198.4055004763364 27.95719055536224
19640 203.25325451514104 29.35757180994612
19660 194.88562235378436 27.295657407514064
19680 194.38972287605282 27.256287344511435
19700 195.4362312720472 26.886405851883445
19720 192.05886010726044 26.684401088116452
19740 192.01695910270453 27.12900693192495
19760 190.60937295141696 26.80514201034847
19780 192.1190592494418 26.983746262408513
19800 190.78925816891908 27.066110617730807
19820 185.24359031306946 27.175143681227016
19840 186.23426862872216 26.06861736392868
19860 194.0199902408736 25.62341634166895
19880 194.72077882588272 28.991721797667697
19900 190.7899851939415 28.785315477743552
19920 193.38441092166502 27.32250837187874
19940 191.44260243173267 26.54476366032762
19960 190.0567465231423 27.403739787630155
19980 191.9414600566804 26.978126932546942

```

The walk is not predictable; it never converges toward any fixed number and changes are random and arbitrary. When comparing it with terminal database outputs over the many simulation test runs (see ex: those in development phase), we see a distinctly different behaviour. Animas interacting in an environment have traits which behave one of two ways:

1. Gaussian random walk: this is expressed by control traits with little to no bearing on selection- for instance, a trait which I have not written any code regarding as yet. Since they have no impact on selection, they will largely follow the random walk; increases or decreases are both logically undifferentiated here.
2. Initially fast rate of change into a steady convergence toward some optimal value: this is the expression of traits which follow natural selection.

Whichever of the two behaviours a trait mean follows, it can be said that trait values are a combination of the parents' trait values offset by a random factor.

Different combinations of traits in an individual affect its tendency to survive and to reproduce

A starting population with anima given as {

```
Anima(i, "name/diet/strength/toughness", [("meat",), [random.uniform(1, 8),
[1,0.5,0.3]], random.uniform(15, 30), random.uniform(5,15), random.uniform(4,
15), random.uniform(4,15), random.uniform(30,80), random.uniform(0, 1),
random.uniform(20,80), random.uniform(40,200), random.uniform(5,50),
ExtraTraits(set())], None, random.uniform(0, 600) , random.uniform(0, 500))
} thrives in the apple experiment with adequate prey.
```

However, the same genome can exist with one trait altered to cause non-viability.

Such alterations can include:

- Single diet type which is not 'apple' or 'meat'
- Speed too low such that prey cannot be found or once found it cannot be caught
- Vision too low such that food cannot be found in an efficient timeframe
- Speed/vision too high such that energy requirements are unsustainable
- Size too low such that fights against prey cannot be won

Etc etc.

For a trivial case, animas with speed or vision of 0 will simply die.

For a single more generalisable case, a starting population of animas given as {

```
Anima(i, "name/diet/strength/toughness", [("apple",), [random.uniform(1, 5),
[1,0.5,0.3]], random.uniform(15, 30), random.uniform(5,15),
random.uniform(4,10), random.uniform(4,10), random.uniform(30,80),
random.uniform(0, 1), random.uniform(30,100), random.uniform(40,200),
random.uniform(5,50), ExtraTraits(set())], None, random.uniform(0, 600) ,
random.uniform(0, 500))
```

} sees far greater population increase than the population with only meat eaters, provided there are enough apple trees.

In the case of only meat eaters with diet mutation turned off, we get a Viltrumite-style purge where only the larger, faster animas can at first survive, but those too later die from starvation; only two remained after 325 ticks.

I ran the simulation again, later, in the newest version. Here is an excerpt of the data:

```
[[0|[], [[meat], [4.6|[1.0|0.5|0.3], ], |22.5|10.0|9.8|9.6|55.6|0.5|50.6|123.6|27.5|N/A], ],
|[25|[], [[meat], [4.1|[1.0|0.5|0.3], ], |22.6|9.9|10.3|10.1|55.9|0.44|49.6|125.8|27.2|N/A], ],
```

```

|[50|[], |[[meat], |[3.7|[1.0|0.5|0.3], ], |22.8|10.1|10.8|10.7|56.1|0.42|50.0|126.5|27.0|N/A], ],
|[75|[], |[[meat], |[3.4|[1.0|0.5|0.3], ], |22.4|10.1|11.0|10.8|55.3|0.43|49.6|127.1|27.8|N/A], ],
|[100|[], |[[meat], |[3.2|[1.0|0.5|0.3], ], |22.3|10.1|11.2|11.2|55.1|0.43|49.7|132.8|28.4|N/A], ],
|[125|[], |[[meat], |[3.0|[1.0|0.5|0.3], ], |22.1|10.0|11.4|11.4|54.9|0.4|49.5|132.6|27.7|N/A], ],
|[150|[], |[[meat], |[2.9|[1.0|0.5|0.3], ], |21.9|10.0|11.7|11.2|55.3|0.39|47.8|131.2|25.9|N/A], ],
|[175|[], |[[meat], |[2.8|[1.0|0.5|0.3], ], |21.7|10.1|11.8|10.8|55.3|0.39|49.2|131.5|25.5|N/A], ],
|[200|[], |[[meat], |[2.5|[1.0|0.5|0.3], ], |20.7|9.5|11.5|10.6|53.0|0.4|47.6|129.7|27.5|N/A], ],
|[225|[], |[[meat], |[2.5|[1.0|0.5|0.3], ], |20.2|9.9|11.7|10.3|51.9|0.38|50.6|135.1|29.6|N/A], ],
|[250|[], |[[meat], |[2.5|[1.0|0.5|0.3], ], |19.5|10.0|11.5|9.5|53.9|0.35|51.4|127.5|30.7|N/A], ],
|[275|[], |[[meat], |[2.1|[1.0|0.5|0.3], ], |20.3|10.2|11.7|9.7|56.0|0.42|60.3|135.6|28.4|N/A], ],
|[300|[], |[[meat], |[2.0|[1.0|0.5|0.3], ], |20.4|10.2|11.2|9.2|58.2|0.38|59.1|130.6|25.9|N/A], ],
|[325|[], |[[meat], |[2.1|[1.0|0.5|0.3], ], |19.7|10.2|11.4|9.6|56.6|0.4|62.8|129.1|19.2|N/A], ],
|[350|[], |[[meat], |[1.4|[1.0|0.5|0.3], ], |20.7|8.7|9.6|8.3|46.3|0.34|57.5|75.8|16.4|N/A], ],
|[375|[], |[[meat], |[1.3|[1.0|0.5|0.3], ], |17.1|6.1|5.9|7.3|34.2|0.26|73.2|60.9|15.4|N/A], ], ],

[[0|[[apple|0], |[[meat|400], ], ], |25|[[apple|0], |[[meat|285], ], ], |50|[[apple|0], |[[meat|175], ], ],
|[75|[[apple|0], |[[meat|137], ], ], |100|[[apple|0], |[[meat|101], ], ], |125|[[apple|0], |[[meat|77], ], ], ],
|[150|[[apple|0], |[[meat|59], ], ], |175|[[apple|0], |[[meat|43], ], ], ],
|[200|[[apple|0], |[[meat|25], ], ], |225|[[apple|0], |[[meat|17], ], ], |250|[[apple|0], |[[meat|13], ], ], ],
|[275|[[apple|0], |[[meat|9], ], ], |300|[[apple|0], |[[meat|8], ], ], |325|[[apple|0], |[[meat|6], ], ], ],
|[350|[[apple|0], |[[meat|3], ], ], |375|[[apple|0], |[[meat|1], ], ], ], ],

```

We can very barely call this natural selection. There was close to zero mating and it was essentially a contest of survival with little means for food. In these conditions, those with the lowest speeds and visions were more likely to survive longer, counterintuitively, as though they were less likely to find food, they also starved slower and thus lasted longer in the simulation. Thus animas with low viability but long survivability were selected for if lifespan is used as the metric of fitness.

Traits correlating with tendency to survive and to reproduce are likely to be better represented in each passing generation, as opposed to less represented 

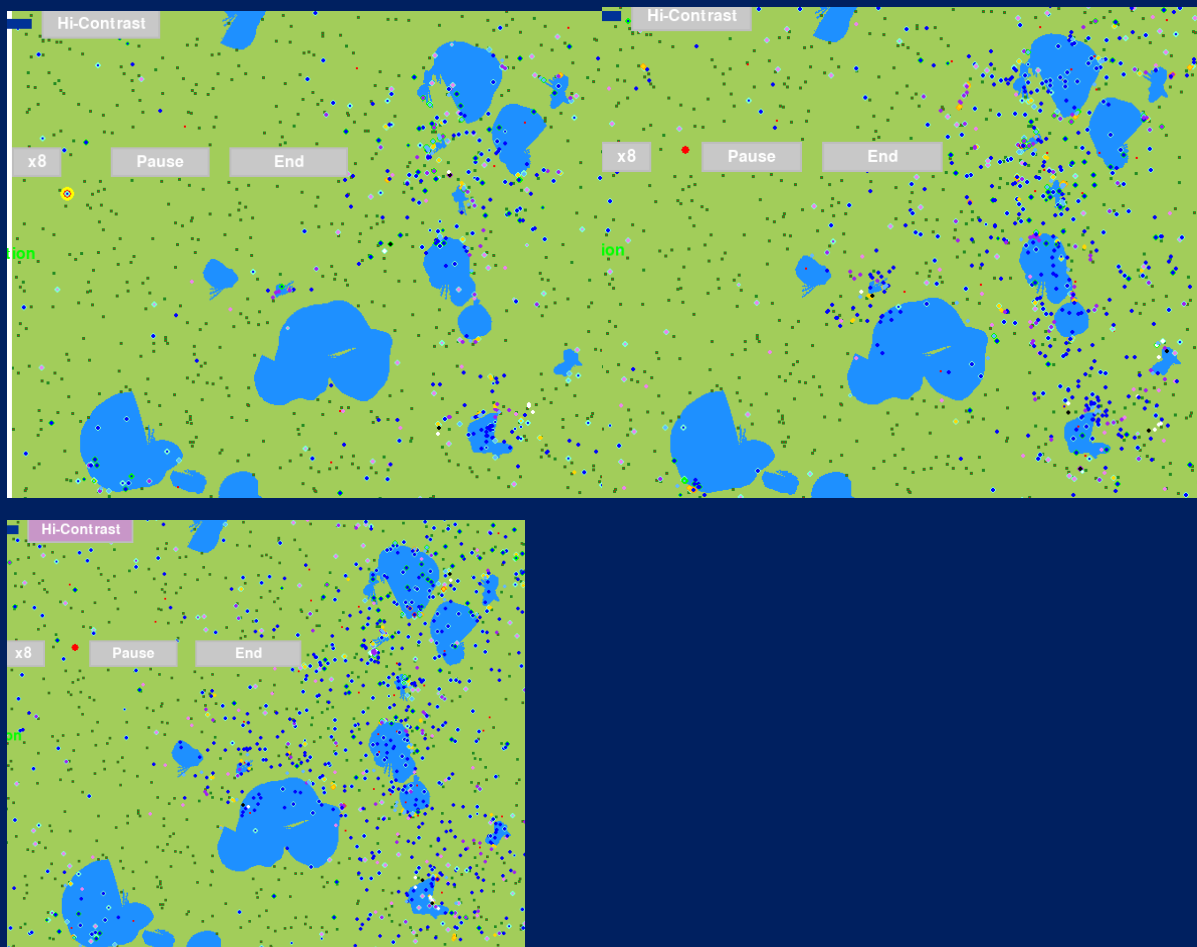
Traits should affect viability not simply in isolation, but in combination with the values of other traits.

For one concrete example, I ran a simulation with random speed distribution for predators set to `random.uniform(0, 1)`

Herbivorous animas still had a speed distribution of (1, 5).

Within only 750 ticks, the mean speed for predators had increased from 0.57 to >1.9, over tripling, as the creatures with sub-1 speed could not find food and died quickly.

I then ran the experiment with reversed settings; predators had a distribution of (1,8) while herbivores had a distribution of (0,1). Most of the animas quickly died save for those clustered around pools of water. These then began to slowly migrate outwards:



Since speed is so low, it was important for animas to be near to a body of water; if they were stranded in the open area on the left side of the map, they would starve.

Over 1325 ticks, prey mean speed increased from 0.47 to >0.74 , and predator mean speed decreased from 4.9 all the way down to <1.7 . We can see these responses taking place as a result of the environment. In the previous test run, prey was fast, so predators had to increase speed in order to catch up; in this run prey was slow and plentiful, so there was no incentive for predators to have a high speed; this would only cause insurmountable energy demands. So predator mean speed ended up being lower in this test run despite starting at over 8x

("<" and ">" used because of incorrect grouping logic on wrong program version causing groups to include all past and current members, instead of only alive animas)

Final evaluation (1):

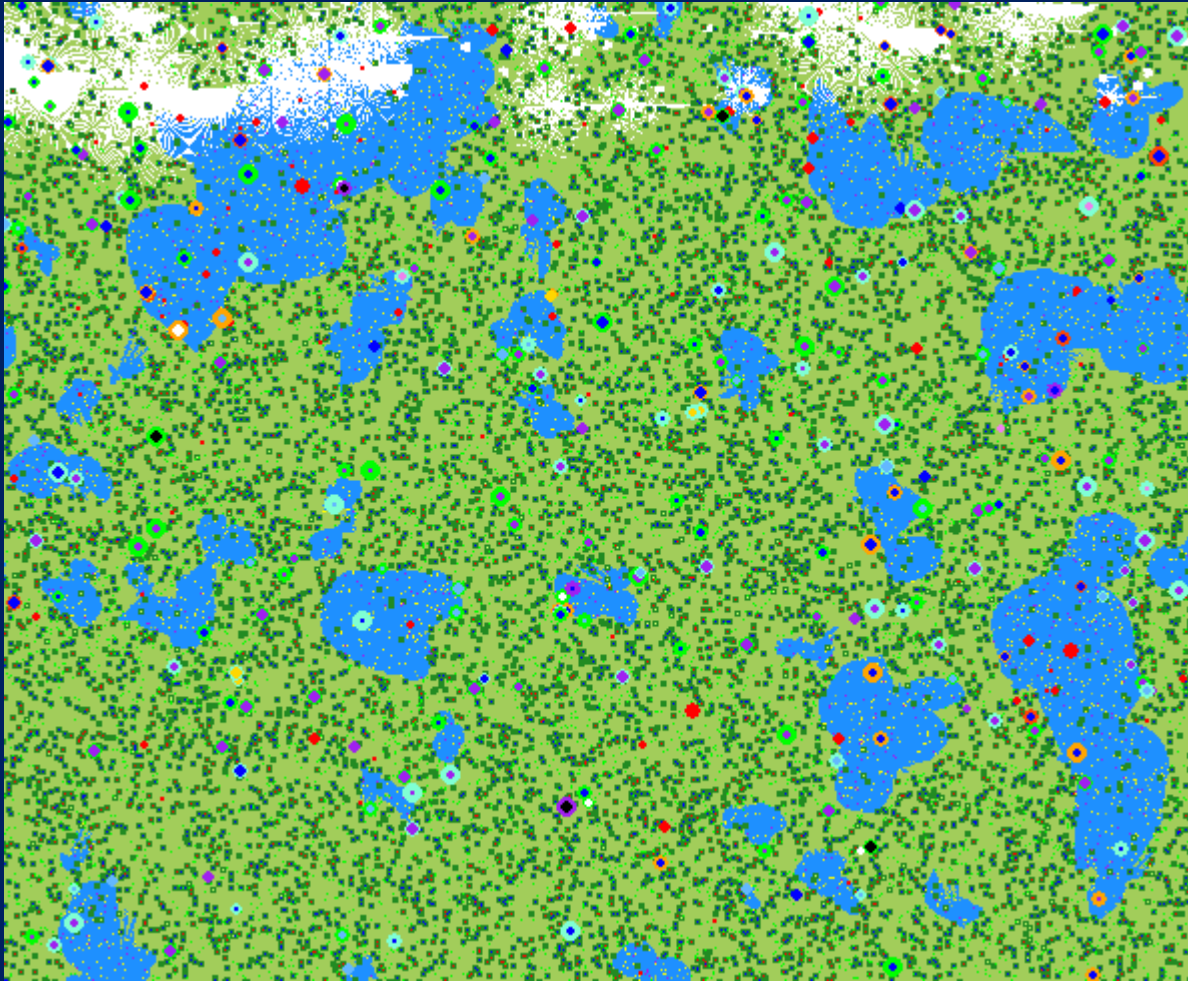
I have added so many things to the anima and realm logics to increase diversity, complexity, and realism, yet there are still many, many opportunities for future updates, such as a mating selection focus update implementing (selective mating; male and female sexes; childcare and pregnancy; eggs), or an anima perception update implementing (camouflage; smell, hearing and other senses; terrain manipulation; memory of certain perceptions, which could be hereditary -> genetic or taught in childcare). More conditions, terrains, traits and attributes could always be added; nature is an infinite source of inspiration and there isn't a limit with this type of modular project. This just goes to show what an exciting and vast premise an evolution simulator is; despite this I believe I have been ambitious and included enough to create a good, usable prototype showing off the concept, and the benchmarks I set for this criterion have been wholly achieved.

2. Provides a clear and intuitive visualisation of the mathematics

- There is a stakeholder consensus that the visual logic is sound and graphics appropriate 
- Organisms with differing traits have corresponding* differences in their visual representation (*these may not be 1-to-1, or actual in some cases) 

I believe this criterion is achieved to a sufficient degree for the project. While the presentation of the simulation view could be reworked and improved, it being a rather rough prototype, it is already true that the UI provides a clear and intuitive visualisation of the underlying mathematics. Animas are easy to keep track of; floras are simply presented as bearing at least one fruit or not, which is all that is required for interaction decisions; the user can choose between different colours sets to decide what they prefer most, which includes their preferences for visual clarity. Since the simulation can run in the tens of frames per second, there is a smooth user experience where they can watch as the animas interact and perform actions and exist in states which, while perhaps not immediately able to be intuited, are simple enough to be learned quickly, especially with the updated tutorial.

There is a stakeholder consensus that the visual logic is sound and graphics appropriate 



Organisms with differing traits have corresponding* differences in their visual representation (*these may not be 1-to-1, or actual in some cases) ☒

In the simulation, plants have different-coloured fruit as according to the fruit set:

```
fruit_set = {
    "apple" : "red",
    "berry" : "blue",
    "lily" : "yellow",
    "lavendar" : "purple",
    "melon" : "darkolivegreen1",
    "wildgrass" : "green",
}
```

Animas have different appearances according to their current state and the general colour set:

```
mature_set = {
    "grass" : "darkolivegreen3",
    "flora" : "forestgreen",
    "water" : "dodgerblue",
    "fruit" : fruit_set,
    "child" : "green",
}
```



```

"old" : "white",
"corpse" : "red",
"adult" : "aquamarine",
"meateater" : "purple",
"found_food" : "purple",
"found_water" : "steelblue1",
"roam" : "blue",
"vision" : "yellow",
"highlight" : "red",
}

```

However, this sub-criterion focuses on how differing *traits* cause differing appearances.

Currently, three traits directly relate to the expressed appearance in the simulation view: size, vision and diet.

The code relating to the display of each anima is thus:

```

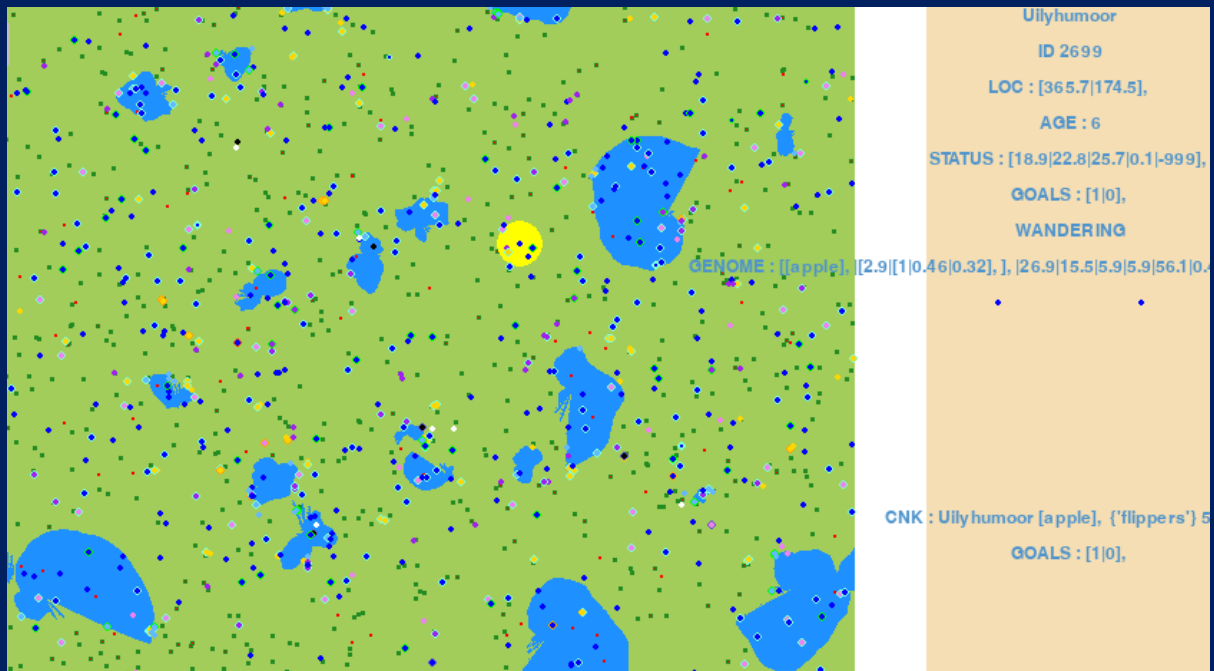
bodycolour = "purple" if anima.is_poisoned() else "orangered" if
anima.is_injured() else "orange" if anima.status[3] >= 25 else SENESCENT_COLOUR
if anima.is_senescent() else CHILD_COLOUR if not anima.is_mature() else
MEAT_EATER_COLOUR if "meat" in anima.diet else ADULT_COLOUR
    pygame.draw.circle(screen, bodycolour, (round(left + anima.x),
round(top + anima.y)), min(max(sqrt(anima.size()), 1), 7))

    eyecolour = "gold" if anima.asleep else "black" if anima.hunting else
FOUND_WATER_COLOUR if anima.mode == 2 and isinstance(anima.target, tuple) else
FOUND_FOOD_COLOUR if anima.mode == 2 else "white" if anima.mode == 3 else
"violet" if 4 in anima.goals else ROAM_COLOUR
    pygame.draw.circle(screen, eyecolour, (round(left + anima.x), round(top
+ anima.y)), min(max(sqrt(anima.vision/2), 1), min(max(sqrt(anima.size()), 1),
5)))

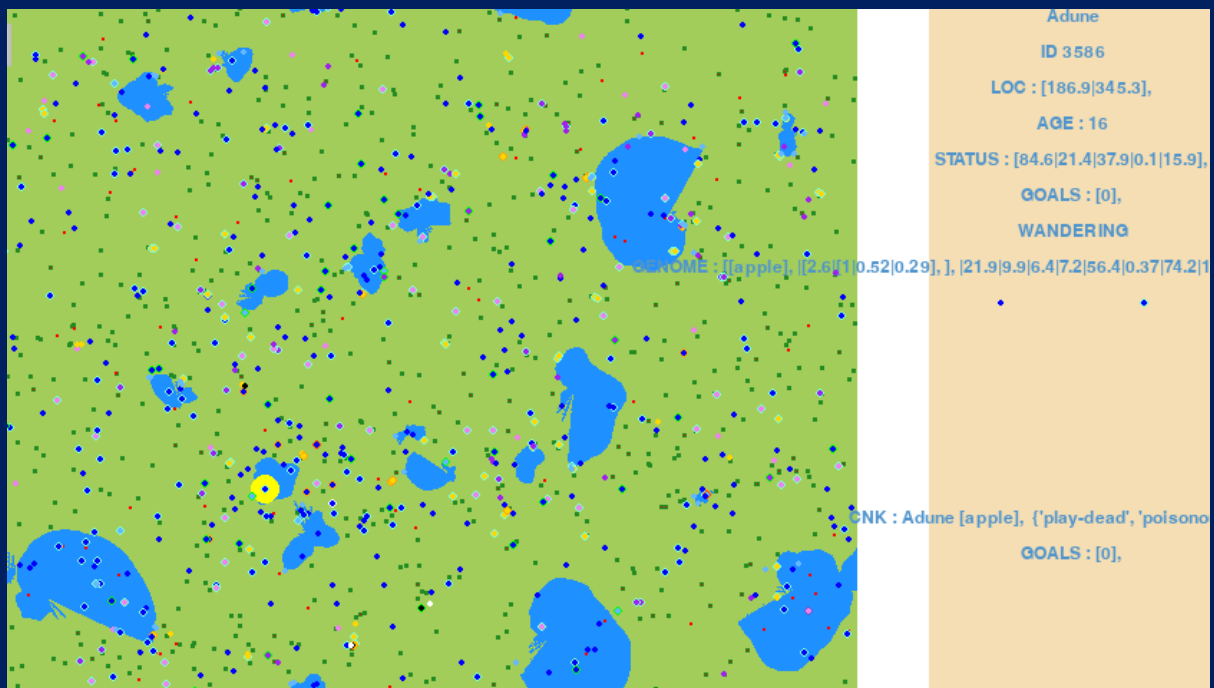
```

To prove that this relation works as intended, I can use the simulation view anima fact file display to compare various animas' genomes with their expressed appearance.

Here are some examples of different animas where their differences in genomes can be seen in their representation in the simulation view.



This anima has a large eye, and when selected it has displayed a large vision radius and high vision at over 15 (the starting range is only 5-15)



This anima has a smaller eye, and when selected it displays a much smaller vision radius; indeed its vision is less than 2/3 that of Uilyhumoor



Goohou

DIET : [apple],

SPEED : [1.6|[1|0.6|0.25],],

TURNING SPEED : 16.6

VISION RANGE : 10.3

STRENGTH : 7.1(12.0)

TOUGHNESS : 12.8(17.6)

MATING DRIVE : 59.0

BOOST FACTOR : 0.65

INJURY THRESHOLD : 101.5

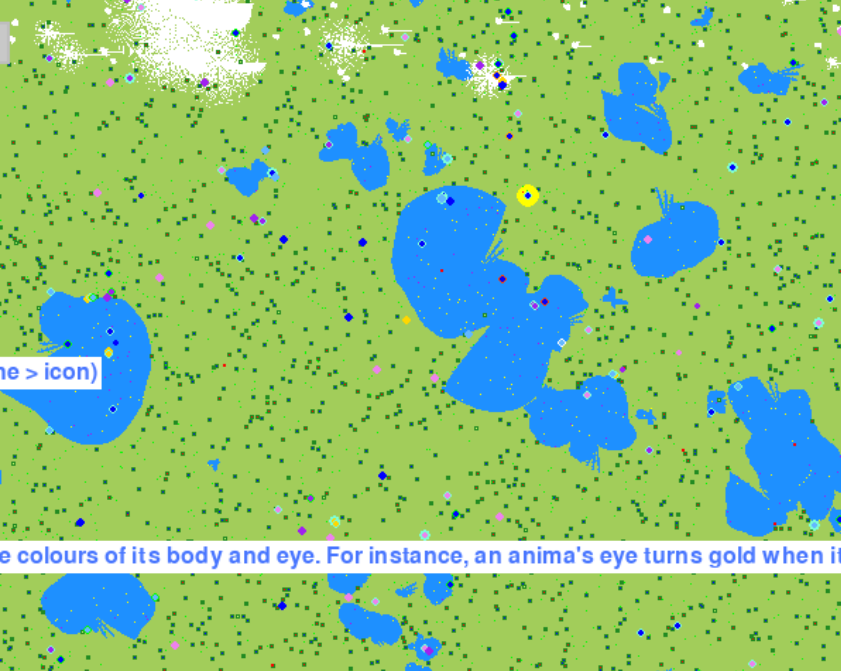
MATURITY AGE : 194.0

REFRACTORY : 38.5

EXTRA TRAITS : {'necro-mimicry'}

[VIEW A NIMA](#)

This anima is larger than average on the sim view; its fact file corroborates this with a size of over 12



Yweex

DIET : [apple],

SPEED : [1.5|[1|0.55|0.29],],

TURNING SPEED : 26.5

VISION RANGE : 7.1

STRENGTH : 7.6(10.1)

TOUGHNESS : 9.4(11.9)

MATING DRIVE : 93.1

BOOST FACTOR : 0.8

INJURY THRESHOLD : 41.7

MATURITY AGE : 97.5

REFRACTORY : 11.9

EXTRA TRAITS : {}

[VIEW A NIMA](#)

this anima is smaller than average with a low body circle radius, and its fact file corroborates this with a size of just over 10.

To increase the expressivity of genotypes to the user, I can add more visual indicators, for instance new colours or new shapes beyond the simple two-circle structure of an anima.

I could also eventually rework the anima appearance completely. I could keep drawing animas using shapes and colours with `pygame.draw`, but with much more complex appearances: perhaps not every anima is composed of the same shapes, for instance an anima with the “flippers” extra trait could have a few pixels dedicated to representing this in their appearance. Alternatively, I could load PNG or JPEG images from within the project folder into the program to represent individual animas. Then, the appearance of an anima could be one of various sprites representing the idea of different species; an anima’s species would be the summary of their traits. For instance, an anima which was fast, had flippers, and ate meat could be given a dolphin sprite; an anima which was slow, large and ate wildgrass could be given an elephant sprite. This would make understanding the differences between species easier at a glance but finer nuances of genomes could be lost, as there are very many possible differing trait combinations, such that animas within the same species, and thus having the same appearance, could still have very distinct behaviours. A combined approach could be taken with species acting as a template and some traits (e.g. vision) still being represented separately or independently.

我还是觉得不需要全部的基因都会在模拟的图片里表示的起来。

I think it is more important that the most impactful traits such as speed and vision can be quickly understood from either the design or behaviour of the anima: speed does not require an appearance change since it can be quickly understood from simply watching the anima move around the simulation, while traits like vision and strength, despite their large impact, cannot be simply read from only watching an anima for a couple of seconds in the same way speed can, so these have their own visual imprints in the anima appearance. More importantly, anima vision without an indicator would have to be estimated from each anima by watching it individually, either through guessing at its needs and how its vision would allow or prevent it to target certain elements, or through the individual anima selection feature. The speed of animas, however, can be gleaned through the mean kinetic motion of animas on the simulation plane, and one can easily observe the speed of many animas at once.

Then there are some other traits which do not directly factor into anima interactions, such as maturity age. Thus there is no need to give a visual indicator as the components of these traits which interactions do depend on (in the case of maturity age, these are the size and maturity of the anima), are displayed in the anima appearance. Traits which are fall into this class, or are otherwise not important enough to factor into appearance lest animas appear too cluttered, can still be inspected by a curious user using the genome view within a selected anima’s fact file.

Final evaluation (2):

The simulation art is rudimentary and if this were to be taken to the next level as a project it could be argued that it would require an overhaul. That being said, it certainly achieves its purpose and there is no requirement for complex animations and design in a non-game project which is more focused on prediction and data. In fact, more complicated art and animation would likely detract from the focus and might overcomplicate the simulation for the user; my aim was to minimise user overload and make events and interactions as clear as possible. In that regard the use of `pygame` allowed for very efficient simulation drawing; beyond that, further polish was never a key component of this project.

One further improvement which could be completed within a reasonable timeframe are the greater integration of sound into the simulation; audio cues can be used just as well as visual cues to inform users about what is happening; the only danger is that audio cues can become more overwhelming

when there are a large number of elements. Audio cues could be used to effectively communicate the rate of deaths, births, and other probabilistic events.

All in all, I am satisfied with how users are able to understand the most important traits of an animal, or even a larger group of animals, at a quick glance. This is visually intuitive design in action, and helps streamline the experience which is after all supposed to be a learning one.

3. Is compatible with a school environment

- The software can be run on KEGS school monitors ●
- There is a stakeholder consensus that the program remains within the bounds of an appropriate educational school project ✓

I believe that this criterion has been mostly achieved, at least to the extent that is plausible with any exe file.

This is a qualitative criterion which I can measure only using stakeholder opinions, alongside the correct function of the application on hardware. The latter has been mostly achieved with the exe file being able to be run on all KEGS monitors; the former has also largely been hit as my stakeholders vastly agree that the content and focus of the program is fitting to the task.

I have added usability features and tailored the UI towards being easily picked up and used by all students; it is a mature program as my target audience of secondary school students can be trusted to be mature (by this I mean that the UI does not grab at your attention and is very simplistic with natural or dull colours). There is no explicit content; all information about evolution is accurate and pertains to a UK secondary school biology curriculum.

The software can be run on KEGS school monitors ●

It is true that I was able to develop my program and run it on VS Code editor on the KEGS school computers.

However, the program requires pygame to be installed, and some schools may have network policies preventing such installations or use of external libraries. In my case, I was able to run the program on the main CR computers, but not on computers in the 6th form study which had different user restrictions in place.

There are no complex hardware requirements on software installations required to facilitate this program, but many computers will disallow the execution of unknown executables, including my program, and an exception would be required to be made; this should not generally be difficult but must be noted.

There is a stakeholder consensus that the program remains within the bounds of an appropriate educational school project ✓

The stakeholders unanimously agreed that the content of the program was appropriate and suitable and as was the structure of how information was conveyed. The simplistic design and intuitive use of the simulation is perfect for a school environment; everything is packaged safely and simply- the program is very sanitised; it is difficult to use the program to perform anything other than its intended function. There are music tracks which feature lyrics appropriate to a school setting.

Since the simulation is a sandbox, it can sometimes be difficult to adjust setup settings to create an intended effect in the realm. This will require the user to spend a bit of time familiarising themselves

with how the system generally works; arbitrary values which create certain effects will not be known unless experimented with; however the tutorial and default realm options help with this.

Final evaluation (3):

My project is school-friendly and does not require complex setups or installations, only the download of the program itself. It would maybe be more suitable if it were web-based as schools can more easily allow a single website through a firewall than update policies to allow an unknown executable to run; antimalware might also identify the exe as a threat though I would not know for what reason. The criterion has been hit to a satisfactory degree.

4. Has highly interactive and customisable setup so users can test many different scenarios

- The starting population, initial traits of organisms, and initial environment layout are determined by user inputs ●
- Users can modify organism appearance to create their own 'species' for ease of visual following and differentiation ✗
- There are at least three distinct trait combinations which populations can converge to, depending on the starting position ●
- There is at least one situation in which all organisms become extinct ✓
- The stable range of population differs depending on the environment setup ✓
- The environment can be altered in such a way that 'speciation occurs', where two populations converge differently in a single simulation ✓

I believe this criterion has been partially achieved. The unmet or partially unmet sub-criteria are this way due to either time constraints or change in vision as I moved from analysis and design to development.

There is a variance in what stage of development each of these sub-criteria came to be tackled. As early as the early anima stage, it could be plausible that the population-based sub-criteria were able to be achieved, although I did not always test for them explicitly. It is possible with these sub-criteria to have false negatives, as there are so many possible permutations of the simulation I cannot test every combination but rather change values in broader strokes; however since for each I would imagine an infinite number of permutations providing a positive result, obtaining a single such result should not be overly difficult. The question is to what extent the scenario shapes evolution; if there is a general convergence or over-factorisation of one or two important factors, then that would be unfortunate, as I have spent a lot of time trying to make the simulation complex; if simplified in practice it would be a shame, as nature and evolution in real life exists in a complex world with many pertinent factors. If the problem arises, then I would imagine that sometimes simply tweaking numbers would solve it at least partially, but sometimes more underlying problems can be present. For the purposes of this analysis, I will simply try to achieve the positive results as listed in the sub-criteria above, but bearing in mind all the while that while they are good indicators of a successful product, they do not mean perfection in the finer intricacies of what I have just stated.

In the later UI stage I added increasing layers of input options and customisability for the user. It could be said that this stage has not been fully developed, with many more finer optimisations and features to add; however what has been added is generally enough to contribute towards each sub-criterion and show the general outline for how the criterion would be tackled, so that further development would be streamlined and natural from what I have already produced (and therefore for the purposes

of this evaluation I would give myself the benefit of the doubt to say that I would be able to without difficulty produce these developments).

The starting population, initial traits of organisms, and initial environment layout are determined by user inputs ●

The user is able to set values to each of the attributes of the realm before the realm is generated. This means the user can create and test a near-infinite number of permutations of the realm with different size, temperature, terrain coverage, and starting element values.

However, I have not yet implemented the ability to adjust the initial traits of organisms. This would not be too difficult to implement, but I have been short of doing so due to time constraints.

If I were to implement this, I would simply apply a similar UI as with the realm attribute settings; the only difference is there are a higher number of organism traits so I might have to make two distinct pages to cover all of them, or else make a scroll bar which allows the user to vertically scroll through them.

My initial vision for user input on environment layout was to have an image of the realm displayed during setup, which would update as settings were changed, so the changing appearance of the realm could be seen by users as they made each change. The user would then also be able to manually add elements such as trees and additional water pools, by clicking on the realm image where they wanted to add them. This UI structure would be perfect for an educational project; it shows in real time what changes are being made with direct feedback. However I did not end up implementing the setup setting in this way, opting for the simpler text and button approach, to save development time as I had already implemented most of my UI in this way. To improve the program I could revert to my original vision, but it is not essential that I should do so.

The greater issue is that there are in fact many variables in the Realm module which are not realm object parameters, but could and perhaps should be. Such variables I experimented with the values of in testing to reach constants which produced a simulation satisfying to me. However, by not including these in simulation set up I am not allowing users to perform the same experimentation. There are benefits and disadvantages to this. On the one hand most of these are more technical and finicky and by only including the more simple and broader attributes, each with large, tangible impact, I am simplifying the product for the user and allowing them to create many different permutations of a realm without cluttering the UI and confusing them with more obscure or harder-to-explain attributes; the user can change a few core values and dive into the simulation, using values for variables which I have already fine-tuned for a good simulation, instead of variables with user-tuned values which might cause out-of-proportion adverse effects, or have little effect at all. Programs of a similar nature sometimes have an 'advanced settings' option I could also implement for this role so users have a clear split between the important realm attributes and more obscure ones. However another roadblock for implementing advanced settings is that I have already implemented (not much, but some) code using realm as it is in the current format, and diluting with many more attributes would force me to edit and refactor what I have already added.

Therefore I am almost satisfied with my progress towards this sub-criterion, but given more time I would add the anima trait adjustment at the very least. I would also add tutorial text explaining each realm and anima attribute simplistically so users know what they are actually adjusting.

Users can modify organism appearance to create their own 'species' for ease of visual following and differentiation ✖

This criterion was unmet as the feature of anima sprite customisation for users is fun and slightly helpful but not necessary for the main purpose of showing a scientific simulation, and would have taken too much time to implement. Coding UI elements generally takes longer than underlying logic code, as elements must be adjusted and ensured to fit in and work with other elements without overlapping, independent of each others' variable size. Making UI elements coherent was very time consuming.

It was also possible this feature would end up making things more confusing. There is already a learning curve to recognising what each colour represents in an anima's body or eye, and this would complicate things further. Some colours chosen by the user could be the same or very similar to colours used for certain effects; I am not sure what exactly what would be customised as animas would obviously have to be more complicated than the simple two-circle structure currently used; originally when I wrote this criterion I thought body shape, colour, etc could all be made customisable, but this would only muddy the waters on what colour and shape meant in regards to representing anima traits and status.

Making all animas drawn using the same universal algorithm promotes consistency and better guides the user on what each anima's traits and status is at a glance; the different species idea never came into fruition the way I expected. For one, most animas are able to breed with one another, and there is otherwise a gradient of compatibility rather than strict species-like groups separated from one another genetically. So different species art would often be visually misleading, as different 'species' might be more similar than expected; I would have to rework mating selection, which I could do but is not currently in the scope of the project.

Due to the fact it would require heavy time investment, make the user experience more complex, and create false categorisation of animas which tend to be of a universal (or at most bi-categorised) species, I decided to abandon the feature required by this criterion. It is something I could consider later down the line, but was never a necessary component and the benefit it provides I generally believe is outweighed by the risks and time cost.

There are at least three distinct trait combinations which populations can converge to, depending on the starting position ●

It is true that there are over three trait combinations a population can converge to, over all permutations of setup.

The ambiguity in the success of this sub-criterion lies in how 'distinct' is defined, as well as the range of 'starting positions' that the dependence is on.

I would allow two distinct trait combinations as two combinations of traits wherein the relative values of one (or more) trait(s) are outside of a 25% margin of error from one another.

With how many unique setup permutations there exists, it is all but a guarantee for there to be a high number of distinct trait combinations.

So, to show in a smaller scope three distinct combinations would be to show in all likelihood that an extrapolation to a larger scope would denote this high number of trait combinations.

This smaller scope would be that the environment is fixed, and only the starting traits of animas are to be varied, to beget three distinct trait combinations over three distinct starting positions wherein only the primordial list is varied.

There is at least one situation in which all organisms become extinct ✓

There are many unique positions which lead to total extinction; these are driven most often by a lack of food, due to either too little vegetation in the realm, or too many predators compared to herbivore animas causing prey to go extinct, with predators following due to an all-meateater population being unsustainable for more than one or two generations.

Here is an example of a starting primordial list which universally leads to extinction (a variant of the Viltrumite purge realm I discussed in one of the previous criterions):

```
for i in range(20):
test_element_list.append(Anima(i, "name/diet/strength/toughness", [("meat",),
[random.uniform(1, 8), [1,0.5,0.3]], random.uniform(15, 30),
random.uniform(5,15), random.uniform(4, 15), random.uniform(4,15),
random.uniform(30,80), random.uniform(0, 1), random.uniform(20,80),
random.uniform(40,200), random.uniform(5,50), ExtraTraits(set()), None,
random.uniform(0, 600) , random.uniform(0, 500)))
```

Here is an example of a starting primordial list which leads to extinction when vegetation is too little:

```
for i in range(400):
test_element_list.append(Anima(i, "name/diet/strength/toughness", [("apple",),
[random.uniform(1, 5), [1,0.5,0.3]], random.uniform(15, 30),
random.uniform(5,15), random.uniform(4,10), random.uniform(4,10),
random.uniform(30,80), random.uniform(0, 1), random.uniform(30,100),
random.uniform(40,200), random.uniform(5,50), ExtraTraits(set()), None,
random.uniform(0, 600) , random.uniform(0, 500)))
for i in range(20):
test_element_list.append(Anima(i, "name/diet/strength/toughness", [("meat",),
[random.uniform(1, 8), [1,0.5,0.3]], random.uniform(15, 30),
random.uniform(5,15), random.uniform(4, 15), random.uniform(4,15),
random.uniform(30,80), random.uniform(0, 1), random.uniform(20,80),
random.uniform(40,200), random.uniform(5,50), ExtraTraits(set()), None,
random.uniform(0, 600) , random.uniform(0, 500)))
```

The same list of animas will thrive and be sustainable with a sufficient number of trees (indeed, this is the testing list I have been using in my apple experiment tests: see those for screenshot evidence)

The stable range of population differs depending on the environment setup ✓

This is similarly trivial to prove.

Keeping all other unmentioned setup variables constant throughout tests --

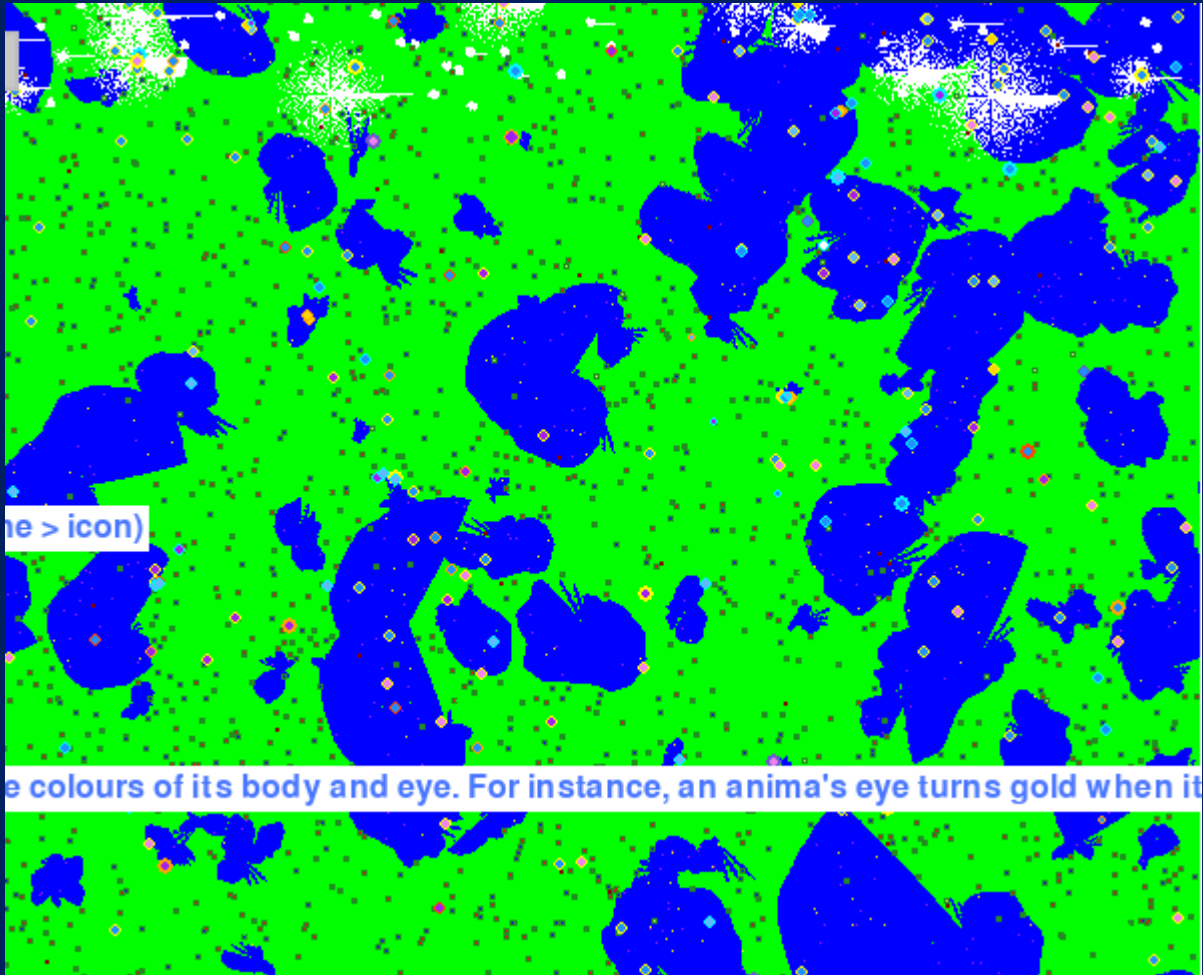
Case 1 (Default settings):



20260503-0055-01.2
617627.mp4

See attached video : “Default case sim”

Case 2 (0.96 wetness):



Case 3 (2x temperature, 1.2x vegetation density):



20260503-0125-25.5
078186.mp4

See attached video : “Utopia Sim”

So the number of trees, and the temperature, are both environmental setup variables which factor into determining the stable range of population. There are other factors (water coverage, ice coverage, anima consumption power factor, etc), the effects of which I observed throughout in-development testing; to prove the sub-criterion these two of tree number and temperature should suffice.

The environment can be altered in such a way that ‘speciation occurs’, where two populations converge differently in a single simulation ✓

In the following cases, the starting anima population have values generated randomly according to a uniform random function; there is initially a single homogenous population pool.

From a population of herbivores, it is possible for a carnivorous individual to mutate from a herbivore; from this individual a carnivorous population might emerge to survive alongside the herbivorous population.

Case:

[[225|225|2], [[250|245|2], [[275|259|3], [[300|277|3], [[325|318|2], [[350|363|2],
[[375|430|3], [[400|503|2], [[425|572|3], [[450|667|3], [[475|769|4], [[500|882|4],
[[525|1039|7], [[550|1179|7], [[575|1345|9], [[600|1529|11], [[625|1722|12], [[650|1990|12],
[[675|2286|14], [[700|2622|20], [[725|3041|24], [[750|3429|29], [[775|3845|35],
[[800|4399|44], [[825|4871|58], [[850|5432|63], [[875|5952|81],]

(Technically not mutated from pure herbivore population, but in a 600x500 grid it was close enough to be as if it were the case)

Otherwise, since all animas are part of a general mating pool (apart from predator/prey categorisation), it is hard for separate genetic populations to emerge. Generally, all animas of the same diet type will have numeric traits distributed normally; to have bimodal distributions would make little mathematical sense.

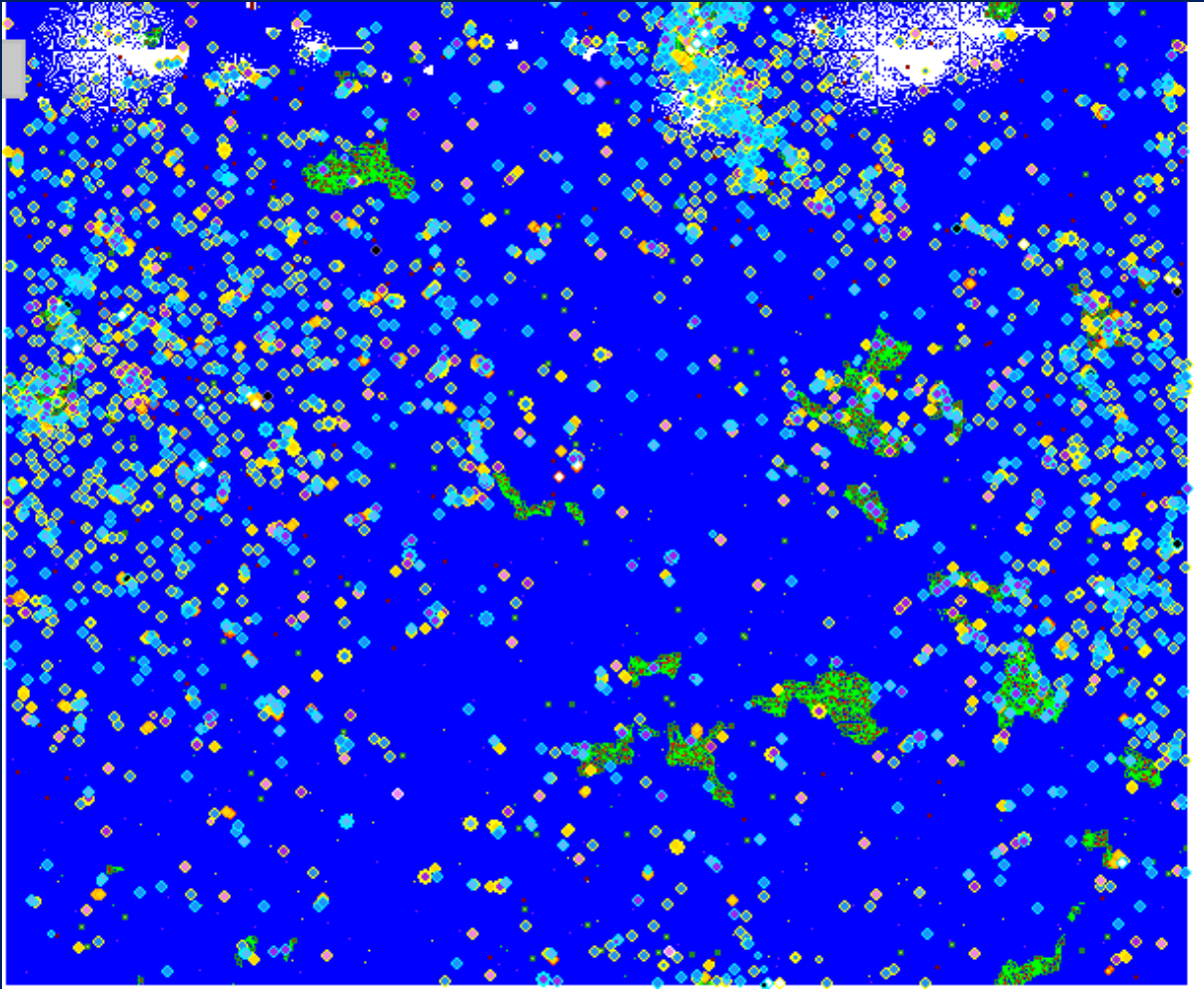
If there were more discrete categories which limited the mating pools then speciation would occur more readily; as is, even discrete value traits such as ‘extra traits’ do not introduce speciation as all animas with any values for these traits are no more likely to mate with similar animas than with others; possible exceptions could come from geographical isolation, which I could explore in a future update. As is in this current iteration of the program, a realm with a high water coverage could potentially see speciation between land-based and aquatic animas.

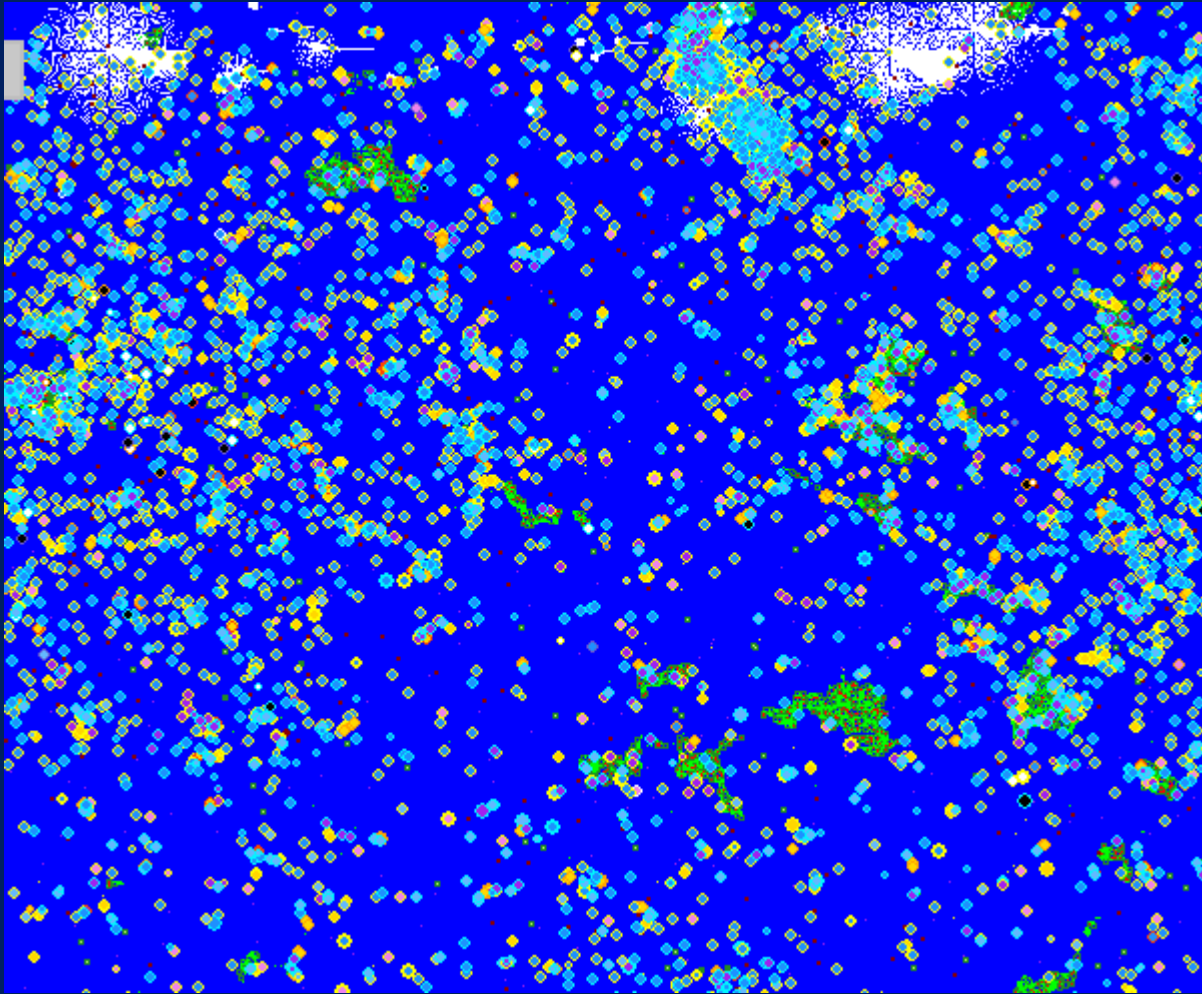
Case:



20260503-0114-29.0
663594.mp4

See attached video : “Aquatic Sim”





```

Extra Traits  [[egg-laying|835],  [[claws|55],  [[flippers|48],  [[abrupt-wanderer|203],
[[poisonous|95],  [[venomous|292],  [[pincers|193],  [[snowycloak|238],  [[strong-gut|211],
[[sand-attack|230],  [[spiritful-child|100],  [[inky-shot|771],  [[berserker|193],  [[vampirism|139],
[[zombie|392],  [[gluttonous-child|23],  [[necro-mimicry|57],  [[absorb-victim|720],  [[mimic-
attack|88],  [[leviathan|411],  [[play-dead|82],  [[plate-armour|83],  [[growth-syndrome|50], ],
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////
[[0|400|30],  [[25|395|29],  [[50|381|23],  [[75|313|15],  [[100|273|11],  [[125|233|9],
[[150|217|5],  [[175|201|4],  [[200|213|3],  [[225|225|2],  [[250|245|2],  [[275|259|3],
[[300|277|3],  [[325|318|2],  [[350|363|2],  [[375|430|3],  [[400|503|2],  [[425|572|3],
[[450|667|3],  [[475|769|4],  [[500|882|4],  [[525|1039|7],  [[550|1179|7],  [[575|1345|9],
[[600|1529|11],  [[625|1722|12],  [[650|1990|12],  [[675|2286|14],  [[700|2622|20],
[[725|3041|24],  [[750|3429|29],  [[775|3845|35],  [[800|4399|44],  [[825|4871|58],
[[850|5432|63],  [[875|5952|81], ],
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////
[3523|0|0|98|0],
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////

```

[[0 [[berry], [3.0 [1.0 0.5 0.3],], [22.9 10.0 6.9 6.9 55.3 0.5 65.7 119.2 27.3 N/A], [[meat],
[4.5 [1.0 0.5 0.3],], [21.9 9.8 9.9 9.3 55.9 0.42 52.2 126.7 26.3 N/A],], [25 [[berry],
[3.0 [1.0 0.5 0.3],], [22.9 10.0 6.9 6.9 55.4 0.49 65.8 119.2 27.3 N/A], [[meat],
[4.5 [1.0 0.5 0.3],], [21.9 9.9 9.8 9.3 56.4 0.42 51.3 125.5 27.0 N/A],], [50 [[berry],
[3.0 [1.0 0.5 0.3],], [22.9 9.8 6.9 6.9 55.5 0.49 65.8 119.7 27.3 N/A], [[meat],
[3.9 [1.0 0.5 0.3],], [22.0 10.1 10.8 9.2 57.3 0.44 49.2 126.8 25.8 N/A],], [75 [[lily],
[2.7 [1.0 0.5 0.3],], [22.8 9.8 7.0 6.8 55.5 0.49 65.2 120.6 26.8 N/A], [[meat],
[3.4 [1.0 0.5 0.3],], [22.6 10.2 11.0 9.1 56.5 0.53 48.7 130.2 26.2 N/A],], [100 [[lily],
[2.6 [1.0 0.5 0.3],], [22.8 9.7 6.9 6.8 55.6 0.5 65.5 121.5 26.5 N/A], [[meat],
[3.2 [1.0 0.5 0.3],], [22.8 10.2 11.0 9.4 58.4 0.57 47.4 122.2 24.5 N/A],], [125 [[lily],
[2.6 [1.0 0.5 0.3],], [22.8 9.8 6.9 6.8 55.9 0.51 65.6 120.4 26.8 N/A], [[meat],
[3.4 [1.0 0.5 0.3],], [23.1 9.6 11.7 9.4 61.3 0.59 50.6 119.8 26.4 N/A],], [150 [[lily],
[2.7 [1.0 0.5 0.3],], [22.9 9.7 6.9 6.8 55.9 0.51 65.6 116.7 26.2 N/A], [[meat],
[3.8 [1.0 0.5 0.3],], [24.7 10.1 11.9 10.7 62.8 0.64 48.0 118.5 29.1 N/A],], [175 [[lily],
[2.7 [1.0 0.5 0.3],], [22.8 9.8 6.9 6.9 55.3 0.5 66.1 115.4 26.6 N/A], [[meat],
[4.5 [1.0 0.5 0.3],], [25.8 9.7 11.1 9.8 60.6 0.62 54.0 117.9 29.7 N/A],], [200 [[lily],
[2.7 [1.0 0.5 0.3],], [23.1 9.9 6.9 6.9 54.4 0.5 65.8 111.6 26.7 N/A], [[meat],
[4.0 [1.0 0.5 0.3],], [25.1 10.6 10.2 9.2 56.3 0.75 59.7 107.6 32.2 N/A],], [225 [[lily],
[2.8 [1.0 0.5 0.3],], [23.0 9.9 6.9 6.9 54.8 0.51 64.8 111.7 25.9 N/A], [[meat],
[2.8 [1.0 0.5 0.3],], [26.6 12.8 8.9 9.1 46.8 0.77 53.2 94.2 42.2 N/A],], [250 [[lily],
[2.7 [1.0 0.49 0.3],], [23.1 9.8 6.8 6.8 54.8 0.51 64.3 107.6 25.5 N/A], [[meat],
[2.8 [1.0 0.5 0.3],], [26.6 12.8 8.9 9.1 46.8 0.77 53.2 94.2 42.2 N/A],], [275 [[lily],
[2.8 [1.0 0.49 0.3],], [22.9 9.6 6.8 6.9 54.0 0.5 63.7 105.0 25.0 N/A], [[meat],
[2.7 [1.0 0.49 0.31],], [23.6 11.8 8.6 9.2 42.8 0.74 65.4 73.1 40.8 N/A],], [300 [[lily],
[2.7 [1.0 0.49 0.3],], [23.0 9.4 6.8 6.9 53.9 0.51 64.5 101.2 24.1 N/A], [[meat],
[2.7 [1.0 0.49 0.31],], [23.6 11.8 8.6 9.2 42.8 0.74 65.4 73.1 40.8 N/A],], [325 [[lily],
[2.7 [1.0 0.49 0.3],], [23.1 9.4 6.9 6.9 53.6 0.51 64.6 96.8 23.5 N/A], [[meat],
[3.4 [1.0 0.48 0.32],], [21.1 11.1 10.4 8.3 39.7 0.76 61.5 72.3 38.4 N/A],], [350 [[lily],
[2.7 [1.0 0.49 0.3],], [23.1 9.3 6.8 6.8 54.7 0.52 65.2 92.1 22.5 N/A], [[meat],
[3.4 [1.0 0.48 0.32],], [21.1 11.1 10.4 8.3 39.7 0.76 61.5 72.3 38.4 N/A],], [375 [[lily],
[2.8 [1.0 0.49 0.3],], [23.3 9.3 6.9 6.8 54.8 0.51 65.4 90.4 22.5 N/A], [[meat],
[2.3 [1.0 0.48 0.35],], [16.9 10.7 8.2 9.3 41.1 0.5 95.6 34.2 33.8 N/A],], [400 [[lily],
[2.8 [1.0 0.48 0.3],], [23.4 9.2 6.9 6.8 55.1 0.51 66.1 85.2 21.8 N/A], [[meat],
[2.4 [1.0 0.48 0.35],], [16.9 9.8 8.8 9.3 44.8 0.45 96.1 32.3 38.1 N/A],], [425 [[lily],
[2.8 [1.0 0.48 0.3],], [23.5 9.3 6.9 6.8 55.2 0.51 65.9 83.2 21.9 N/A], [[meat],
[2.7 [1.0 0.47 0.31],], [20.4 9.2 8.6 8.3 47.6 0.36 82.2 76.1 27.0 N/A],], [450 [[lily],
[2.8 [1.0 0.48 0.3],], [23.4 9.4 6.9 6.8 55.1 0.51 65.7 81.3 21.5 N/A], [[meat],
[2.7 [1.0 0.47 0.31],], [20.4 9.2 8.6 8.3 47.6 0.36 82.2 76.1 27.0 N/A],], [475 [[lily],
[2.8 [1.0 0.48 0.3],], [23.4 9.4 6.9 6.8 54.9 0.51 65.4 80.1 21.2 N/A], [[meat],
[2.3 [1.0 0.45 0.3],], [18.3 9.4 8.8 8.3 63.6 0.47 73.0 67.2 33.4 N/A],], [500 [[lily],
[2.8 [1.0 0.48 0.3],], [23.3 9.4 6.9 6.8 54.5 0.51 64.8 78.8 21.4 N/A], [[meat],
[2.3 [1.0 0.45 0.3],], [18.3 9.4 8.8 8.3 63.6 0.47 73.0 67.2 33.4 N/A],], [525 [[lily],
[2.8 [1.0 0.48 0.3],], [23.0 9.5 6.8 6.8 54.8 0.51 65.1 77.9 21.4 N/A], [[meat],
[2.6 [1.0 0.44 0.31],], [20.2 8.6 7.4 7.2 59.4 0.43 71.4 66.3 27.6 N/A],], [550 [[lily],
[2.8 [1.0 0.48 0.3],], [22.9 9.5 6.8 6.8 54.7 0.51 64.6 76.5 21.5 N/A], [[meat],
[2.6 [1.0 0.44 0.31],], [20.2 8.6 7.4 7.2 59.4 0.43 71.4 66.3 27.6 N/A],], [575 [[lily],
[2.8 [1.0 0.48 0.3],], [22.9 9.5 6.7 6.8 54.7 0.51 64.7 75.0 21.5 N/A], [[meat],
[2.3 [1.0 0.42 0.31],], [19.0 9.1 7.1 7.3 59.2 0.48 71.9 65.5 24.7 N/A],], [600 [[lily],
[2.8 [1.0 0.48 0.3],], [22.8 9.5 6.7 6.8 54.9 0.51 64.4 73.5 21.5 N/A], [[meat],
[2.5 [1.0 0.4 0.29],], [20.1 9.3 6.5 6.9 58.5 0.5 71.0 60.8 20.6 N/A],], [625 [[lily],
[2.8 [1.0 0.48 0.3],], [22.7 9.5 6.7 6.8 54.8 0.51 64.7 72.6 21.4 N/A], [[meat],

```

|[2.7|[1.0|0.4|0.3], ], |21.5|9.0|6.1|6.4|58.4|0.43|73.1|66.4|18.7|N/A], ], |[650|[[lily],
|[2.8|[1.0|0.48|0.3], ], |22.5|9.6|6.7|6.8|55.0|0.52|64.6|70.8|21.5|N/A], |[meat],
|[2.8|[1.0|0.4|0.3], ], |21.6|9.3|6.2|6.5|57.7|0.46|70.6|63.0|17.6|N/A], ], |[675|[[lily],
|[2.8|[1.0|0.47|0.3], ], |22.4|9.6|6.7|6.8|55.3|0.53|64.5|69.6|21.4|N/A], |[meat],
|[2.5|[1.0|0.4|0.3], ], |21.1|9.3|6.1|6.5|60.6|0.56|68.0|66.2|19.0|N/A], ], |[700|[[lily],
|[2.8|[1.0|0.47|0.3], ], |22.4|9.6|6.7|6.8|55.6|0.54|64.5|69.1|21.4|N/A], |[meat],
|[2.5|[1.0|0.42|0.32], ], |20.7|10.1|6.3|6.8|57.3|0.53|64.0|64.5|19.7|N/A], ], |[725|[[lily],
|[2.8|[1.0|0.47|0.3], ], |22.5|9.6|6.7|6.9|55.6|0.55|64.5|67.5|21.2|N/A], |[meat],
|[2.6|[1.0|0.43|0.31], ], |20.2|9.7|6.2|6.8|56.4|0.55|61.3|74.3|18.9|N/A], ], |[750|[[lily],
|[2.8|[1.0|0.47|0.3], ], |22.5|9.6|6.7|6.9|55.5|0.54|64.0|67.4|21.2|N/A], |[meat],
|[2.6|[1.0|0.45|0.3], ], |20.4|8.8|6.5|7.0|56.2|0.56|66.0|69.8|19.6|N/A], ], |[775|[[lily],
|[2.8|[1.0|0.47|0.3], ], |22.4|9.7|6.7|6.9|55.7|0.55|64.1|66.3|21.2|N/A], |[meat],
|[2.6|[1.0|0.47|0.31], ], |20.8|9.0|6.3|7.2|53.8|0.56|63.6|71.1|20.6|N/A], ], |[800|[[lily],
|[2.8|[1.0|0.47|0.3], ], |22.4|9.7|6.7|6.9|55.4|0.55|64.0|65.1|21.3|N/A], |[meat],
|[2.5|[1.0|0.47|0.3], ], |21.3|9.7|6.4|7.3|54.7|0.59|62.6|71.2|22.4|N/A], ], |[825|[[lily],
|[2.8|[1.0|0.47|0.3], ], |22.3|9.7|6.7|7.0|55.3|0.55|64.0|64.6|21.4|N/A], |[meat],
|[2.5|[1.0|0.48|0.3], ], |21.0|9.4|6.2|7.3|54.5|0.63|64.9|65.1|21.0|N/A], ], |[850|[[lily],
|[2.8|[1.0|0.47|0.3], ], |22.4|9.7|6.6|7.0|55.4|0.54|63.8|63.7|21.3|N/A], |[meat],
|[2.5|[1.0|0.48|0.3], ], |21.1|9.4|6.1|7.5|55.8|0.66|65.2|62.8|20.0|N/A], ], |[875|[[lily],
|[2.7|[1.0|0.47|0.3], ], |22.4|9.7|6.6|7.0|55.4|0.54|63.5|62.7|21.3|N/A], |[meat],
|[2.6|[1.0|0.5|0.29], ], |20.9|9.4|5.9|7.5|54.6|0.67|63.6|61.2|19.8|N/A], ], ],
//////////
//////////
//////////
[[0|[[apple|68], |[berry|73], |[melon|61], |[lily|71], |[lavendar|62], |[wildgrass|65], ], ],
|[25|[[apple|65], |[berry|73], |[melon|60], |[lily|71], |[lavendar|62], |[wildgrass|64], ], ],
|[50|[[apple|64], |[berry|72], |[melon|54], |[lily|71], |[lavendar|62], |[wildgrass|58], ], ],
|[75|[[apple|46], |[berry|47], |[melon|42], |[lily|70], |[lavendar|59], |[wildgrass|49], ], ],
|[100|[[apple|39], |[berry|38], |[melon|31], |[lily|70], |[lavendar|51], |[wildgrass|44], ], ],
|[125|[[apple|32], |[berry|26], |[melon|25], |[lily|69], |[lavendar|40], |[wildgrass|41], ], ],
|[150|[[apple|32], |[berry|22], |[melon|19], |[lily|71], |[lavendar|32], |[wildgrass|41], ], ],
|[175|[[apple|34], |[berry|16], |[melon|12], |[lily|73], |[lavendar|28], |[wildgrass|38], ], ],
|[200|[[apple|34], |[berry|16], |[melon|10], |[lily|83], |[lavendar|25], |[wildgrass|45], ], ],
|[225|[[apple|34], |[berry|15], |[melon|10], |[lily|94], |[lavendar|22], |[wildgrass|50], ], ],
|[250|[[apple|35], |[berry|15], |[melon|9], |[lily|103], |[lavendar|22], |[wildgrass|61], ], ],
|[275|[[apple|32], |[berry|11], |[melon|7], |[lily|117], |[lavendar|22], |[wildgrass|70], ], ],
|[300|[[apple|40], |[berry|9], |[melon|5], |[lily|127], |[lavendar|17], |[wildgrass|79], ], ],
|[325|[[apple|46], |[berry|7], |[melon|6], |[lily|151], |[lavendar|18], |[wildgrass|90], ], ],
|[350|[[apple|56], |[berry|6], |[melon|6], |[lily|166], |[lavendar|18], |[wildgrass|111], ], ],
|[375|[[apple|64], |[berry|6], |[melon|7], |[lily|195], |[lavendar|20], |[wildgrass|138], ], ],
|[400|[[apple|88], |[berry|6], |[melon|8], |[lily|222], |[lavendar|17], |[wildgrass|162], ], ],
|[425|[[apple|101], |[berry|6], |[melon|7], |[lily|259], |[lavendar|13], |[wildgrass|186], ], ],
|[450|[[apple|112], |[berry|7], |[melon|9], |[lily|314], |[lavendar|10], |[wildgrass|215], ], ],
|[475|[[apple|130], |[berry|9], |[melon|7], |[lily|373], |[lavendar|9], |[wildgrass|241], ], ],
|[500|[[apple|140], |[berry|9], |[melon|7], |[lily|442], |[lavendar|10], |[wildgrass|274], ], ],
|[525|[[apple|162], |[berry|11], |[melon|7], |[lily|533], |[lavendar|10], |[wildgrass|316], ], ],
|[550|[[apple|175], |[berry|13], |[melon|8], |[lily|626], |[lavendar|11], |[wildgrass|346], ], ],
|[575|[[apple|214], |[berry|13], |[melon|9], |[lily|733], |[lavendar|11], |[wildgrass|365], ], ],
|[600|[[apple|244], |[berry|13], |[melon|12], |[lily|881], |[lavendar|7], |[wildgrass|372], ], ],
|[625|[[apple|270], |[berry|12], |[melon|14], |[lily|1017], |[lavendar|9], |[wildgrass|400], ], ],
|[650|[[apple|312], |[berry|9], |[melon|17], |[lily|1217], |[lavendar|11], |[wildgrass|424], ], ],

```



```

|[675|[[apple|359], |[berry|11], |[melon|16], |[lily|1433], |[lavendar|11], |[wildgrass|456], ], ],
|[700|[[apple|410], |[berry|11], |[melon|15], |[lily|1680], |[lavendar|14], |[wildgrass|492], ], ],
|[725|[[apple|496], |[berry|12], |[melon|14], |[lily|1946], |[lavendar|16], |[wildgrass|557], ], ],
|[750|[[apple|550], |[berry|14], |[melon|18], |[lily|2238], |[lavendar|19], |[wildgrass|590], ], ],
|[775|[[apple|612], |[berry|20], |[melon|21], |[lily|2540], |[lavendar|23], |[wildgrass|629], ], ],
|[800|[[apple|707], |[berry|18], |[melon|24], |[lily|2940], |[lavendar|24], |[wildgrass|686], ], ],
|[825|[[apple|797], |[berry|19], |[melon|28], |[lily|3261], |[lavendar|29], |[wildgrass|737], ], ],
|[850|[[apple|919], |[berry|25], |[melon|33], |[lily|3619], |[lavendar|32], |[wildgrass|804], ], ],
|[875|[[apple|1046], |[berry|30], |[melon|39], |[lily|3946], |[lavendar|36], |[wildgrass|855], ], ],
],
////////////////////////////////////
////////////////////////////////////
//////////
|[0|N/A], |[25|15.5], |[50|36.4], |[75|61.9], |[100|79.4], |[125|104.5], |[150|127.3],
|[175|152.6], |[200|169.1], |[225|192.0], |[250|204.8], |[275|227.3], |[300|237.2],
|[325|222.5], |[350|288.3], |[375|280.3], |[400|278.5], |[425|251.2], |[450|208.7],
|[475|187.9], |[500|174.2], |[525|181.4], |[550|162.3], |[575|159.5], |[600|160.3],
|[625|180.2], |[650|158.7], |[675|158.7], |[700|153.5], |[725|160.9], |[750|147.5],
|[775|146.5], |[800|161.5], |[825|156.4], |[850|153.6], |[875|153.5], ],
////////////////////////////////////
////////////////////////////////////
//////////
|[0|N/A], |[25|[9.0|16.0|22.8], ], |[50|[29.2|40.0|47.8], ], |[75|[55.0|62.0|69.0], ],
|[100|[68.2|83.5|90.0], ], |[125|[94.8|109.0|119.2], ], |[150|[116.0|129.0|138.0], ],
|[175|[141.5|159.0|164.0], ], |[200|[161.0|164.0|189.0], ], |[225|[177.0|200.5|214.5], ],
|[250|[188.2|220.5|235.5], ], |[275|[227.5|253.0|262.0], ], |[300|[241.0|274.5|286.0], ],
|[325|[103.2|286.0|311.8], ], |[350|[254.5|325.5|333.8], ], |[375|[217.0|333.0|350.0], ],
|[400|[212.0|344.0|376.0], ], |[425|[120.2|232.5|405.8], ], |[450|[90.0|142.0|408.0], ],
|[475|[84.0|150.0|239.0], ], |[500|[87.0|130.0|213.0], ], |[525|[75.5|128.0|229.5], ],
|[550|[93.0|142.0|184.0], ], |[575|[70.5|129.0|199.5], ], |[600|[79.0|131.5|194.5], ],
|[625|[70.0|144.0|209.0], ], |[650|[79.0|128.0|197.0], ], |[675|[81.2|132.5|199.5], ],
|[700|[83.0|128.0|189.0], ], |[725|[74.5|118.0|207.5], ], |[750|[72.0|117.0|197.0], ],
|[775|[74.0|115.5|185.2], ], |[800|[77.0|124.0|198.5], ], |[825|[81.0|129.0|205.8], ],
|[850|[75.5|127.0|190.5], ], |[875|[74.0|123.0|193.0], ], ],
////////////////////////////////////
////////////////////////////////////
//////////
|[0|[0.0|0.0|0.0], ], |[25|[25.0|25.0|25.0], ], |[50|[50.0|50.0|50.0], ], |[75|[75.0|75.0|75.0], ],
|[100|[100.0|100.0|100.0], ], |[125|[125.0|125.0|125.0], ], |[150|[150.0|150.0|150.0], ],
|[175|[175.0|175.0|175.0], ], |[200|[200.0|200.0|200.0], ], |[225|[81.0|225.0|225.0], ],
|[250|[59.0|250.0|250.0], ], |[275|[54.8|184.5|275.0], ], |[300|[53.2|133.0|300.0], ],
|[325|[47.0|113.5|325.0], ], |[350|[42.0|104.0|350.0], ], |[375|[41.5|98.0|224.0], ],
|[400|[40.0|92.0|200.5], ], |[425|[40.0|95.0|190.0], ], |[450|[37.8|88.5|181.8], ],
|[475|[39.0|89.0|180.5], ], |[500|[39.8|90.0|183.0], ], |[525|[36.8|89.0|180.2], ],
|[550|[40.0|91.0|182.0], ], |[575|[39.0|89.0|177.2], ], |[600|[38.0|89.5|175.8], ],
|[625|[39.0|91.0|178.0], ], |[650|[37.8|87.0|175.0], ], |[675|[36.0|86.0|173.0], ],
|[700|[37.0|84.0|172.2], ], |[725|[35.0|81.0|165.0], ], |[750|[35.0|81.0|165.0], ],
|[775|[37.0|83.0|165.0], ], |[800|[35.0|82.0|162.0], ], |[825|[37.0|82.0|159.0], ],
|[850|[36.0|81.0|162.0], ], |[875|[37.0|84.0|161.0], ], ],

```

```

/////////////////////////////////////////////////////////////////
/////////////////////////////////////////////////////////////////
/////////////////////////////////////////////////////////////////
153.5
/////////////////////////////////////////////////////////////////
/////////////////////////////////////////////////////////////////
/////////////////////////////////////////////////////////////////
[[0|[25.0|25.0|0.0|0.0|0.0], |[25.0|25.0|0.0|0.0|0.0], ], |[25|[36.9|12.2|7.6|1.2|0.0],
|[50.4|17.2|9.5|6.4|0.0], ], |[50|[47.9|14.5|15.0|1.9|0.64], |[59.0|18.9|14.7|10.5|0.46], ],
|[75|[46.9|15.3|21.2|2.7|2.3], |[61.9|22.2|18.4|12.3|0.87], ], |[100|[45.4|16.4|27.8|4.3|5.3],
|[63.2|21.4|22.0|8.1|3.6], ], |[125|[41.6|16.3|31.9|4.6|9.0], |[55.3|22.8|28.1|0.1|10.9], ],
|[150|[35.9|15.8|36.5|6.3|14.2], |[41.8|15.5|36.8|3.5|21.9], ],
|[175|[32.0|14.9|39.5|5.6|18.6], |[51.6|13.1|47.8|10.8|26.6], ],
|[200|[32.9|14.7|44.9|7.5|20.3], |[33.6|19.1|48.7|11.7|40.4], ],
|[225|[34.4|15.2|45.2|7.0|21.0], |[40.2|29.0|59.5|14.9|54.8], ],
|[250|[34.0|15.4|45.3|7.0|21.2], |[61.6|34.0|66.4|12.4|57.3], ],
|[275|[31.2|15.0|43.4|6.7|20.4], |[49.2|35.2|49.5|6.2|43.0], ],
|[300|[29.7|15.3|41.7|6.1|19.1], |[50.4|35.2|40.2|0.1|50.9], ],
|[325|[28.9|14.3|40.6|6.5|17.0], |[50.3|30.3|79.8|18.6|42.8], ],
|[350|[27.6|14.1|38.3|6.2|15.3], |[60.8|35.3|45.7|0.1|44.0], ],
|[375|[28.7|14.6|38.5|6.4|14.1], |[52.1|25.8|21.5|34.0|4.1], ],
|[400|[30.0|14.0|38.0|6.8|14.2], |[48.4|26.3|22.2|22.0|13.2], ],
|[425|[30.9|14.1|36.5|6.3|13.6], |[51.8|30.0|18.4|23.4|10.5], ],
|[450|[29.9|14.2|37.7|7.3|12.8], |[30.8|20.1|23.1|23.5|19.2], ],
|[475|[30.9|14.2|36.3|6.4|12.9], |[29.1|25.7|21.2|18.0|18.2], ],
|[500|[31.4|14.0|35.7|6.1|12.8], |[44.6|29.4|26.2|16.7|19.4], ],
|[525|[32.5|14.4|36.3|6.6|12.5], |[53.2|25.1|19.8|11.6|13.5], ],
|[550|[33.6|14.5|35.9|6.3|12.8], |[54.3|30.1|25.3|10.7|17.7], ],
|[575|[33.2|14.7|36.6|6.6|12.4], |[39.2|26.3|23.9|17.7|14.9], ],
|[600|[34.0|14.9|36.1|6.4|12.5], |[42.2|23.3|18.9|10.0|14.6], ],
|[625|[34.8|15.0|35.6|6.1|12.4], |[41.6|22.0|22.5|12.3|11.8], ],
|[650|[34.3|15.0|36.4|6.5|12.0], |[39.8|21.9|40.7|24.0|14.4], ],
|[675|[34.4|14.8|36.1|6.7|11.6], |[37.7|21.3|33.2|17.7|15.2], ],
|[700|[35.7|15.0|35.4|6.2|11.6], |[19.3|19.1|31.5|21.0|12.5], ],
|[725|[34.9|15.1|36.6|7.3|11.2], |[22.1|17.4|28.4|22.7|12.3], ],
|[750|[36.1|15.2|36.0|6.8|11.2], |[26.1|18.9|35.8|27.3|12.2], ],
|[775|[36.2|15.4|35.2|6.4|11.4], |[28.2|19.2|29.2|24.5|11.5], ],
|[800|[37.3|15.4|35.7|6.8|11.2], |[28.0|19.9|26.5|23.7|11.2], ],
|[825|[38.0|15.6|34.8|6.4|11.2], |[25.2|16.8|26.9|22.2|9.9], ],
|[850|[38.6|15.7|35.5|6.7|11.2], |[25.3|16.2|26.5|20.7|13.2], ],
|[875|[39.4|16.0|35.3|6.3|11.3], |[22.2|15.1|31.0|22.3|11.9], ], ],
/////////////////////////////////////////////////////////////////
/////////////////////////////////////////////////////////////////
/////////////////////////////////////////////////////////////////
[[0|[[0|400], ], |[0|30], ], ], |[25|[[0|395], ], |[0|29], ], ], |[50|[[0|381], ], |[0|23], ], ],
|[75|[[0|313], ], |[0|15], ], ], |[100|[[0|267], |[1|4], |[2|2], ], |[0|11], ], ], |[125|[[0|221], |[1|4],
|[2|8], ], |[0|9], ], ], |[150|[[0|196], |[1|8], |[2|9], |[3|0], |[4|4], ], |[0|5], ], ], |[175|[[0|174],
|[1|12], |[2|6], |[3|2], |[4|5], |[5|2], ], |[0|4], ], ], |[200|[[0|163], |[1|23], |[2|14], |[3|4],
|[4|5], |[5|1], |[6|3], ], |[0|3], ], ], |[225|[[0|159], |[1|23], |[2|28], |[3|3], |[4|6], |[5|2], |[6|2],
|[7|2], ], |[0|2], ], ], |[250|[[0|162], |[1|27], |[2|28], |[3|8], |[4|10], |[5|4], |[6|4], |[7|0],
|[8|1], |[9|1], ], |[0|2], ], ], |[275|[[0|164], |[1|27], |[2|24], |[3|14], |[4|18], |[5|3], |[6|4],

```

|[7|1], |[8|3], |[9|0], |[10|1],], |[0|3],],], |[300|[[0|167], |[1|30], |[2|29], |[3|15], |[4|18],
 |[5|3], |[6|7], |[7|2], |[8|3], |[9|0], |[10|2], |[11|1],], |[0|3],],], |[325|[[0|182], |[1|37],
 |[2|39], |[3|10], |[4|21], |[5|8], |[6|11], |[7|0], |[8|4], |[9|0], |[10|3], |[11|2], |[12|1],],
 |[0|1], |[1|0], |[2|1],],], |[350|[[0|210], |[1|30], |[2|44], |[3|19], |[4|25], |[5|3], |[6|13],
 |[7|5], |[8|8], |[9|0], |[10|1], |[11|2], |[12|1], |[13|2],], |[0|1], |[1|0], |[2|1],],],
 |[375|[[0|249], |[1|32], |[2|53], |[3|21], |[4|22], |[5|13], |[6|13], |[7|6], |[8|8], |[9|4], |[10|2],
 |[11|2], |[12|2], |[13|2], |[14|0], |[15|1],], |[0|2], |[1|0], |[2|0], |[3|0], |[4|1],],],
 |[400|[[0|291], |[1|31], |[2|63], |[3|21], |[4|33], |[5|14], |[6|16], |[7|5], |[8|10], |[9|6],
 |[10|5], |[11|1], |[12|2], |[13|2], |[14|2], |[15|0], |[16|1],], |[0|1], |[1|0], |[2|0], |[3|0],
 |[4|1],],], |[425|[[0|334], |[1|35], |[2|67], |[3|27], |[4|33], |[5|17], |[6|16], |[7|7], |[8|11],
 |[9|5], |[10|9], |[11|3], |[12|0], |[13|4], |[14|1], |[15|1], |[16|0], |[17|1], |[18|0], |[19|1],],
 |[0|2], |[1|0], |[2|0], |[3|0], |[4|1],],], |[450|[[0|382], |[1|42], |[2|78], |[3|23], |[4|37],
 |[5|30], |[6|21], |[7|11], |[8|12], |[9|8], |[10|8], |[11|2], |[12|5], |[13|2], |[14|1], |[15|1],
 |[16|2], |[17|0], |[18|1], |[19|1],], |[0|2], |[1|0], |[2|0], |[3|0], |[4|1],],], |[475|[[0|438],
 |[1|37], |[2|100], |[3|30], |[4|43], |[5|27], |[6|30], |[7|13], |[8|12], |[9|9], |[10|11], |[11|3],
 |[12|5], |[13|5], |[14|0], |[15|2], |[16|1], |[17|1], |[18|0], |[19|2],], |[0|3], |[1|0], |[2|0],
 |[3|0], |[4|1],],], |[500|[[0|503], |[1|46], |[2|106], |[3|40], |[4|50], |[5|29], |[6|28], |[7|19],
 |[8|18], |[9|8], |[10|10], |[11|7], |[12|5], |[13|3], |[14|4], |[15|0], |[16|1], |[17|2], |[18|1],
 |[19|1], |[20|1],], |[0|3], |[1|0], |[2|0], |[3|0], |[4|1],],], |[525|[[0|601], |[1|43], |[2|124],
 |[3|37], |[4|70], |[5|29], |[6|40], |[7|22], |[8|28], |[9|5], |[10|7], |[11|9], |[12|9], |[13|1],
 |[14|3], |[15|4], |[16|1], |[17|2], |[18|1], |[19|2], |[20|0], |[21|1],], |[0|6], |[1|0], |[2|0],
 |[3|0], |[4|1],],], |[550|[[0|685], |[1|49], |[2|124], |[3|49], |[4|75], |[5|45], |[6|37], |[7|22],
 |[8|30], |[9|16], |[10|10], |[11|6], |[12|12], |[13|4], |[14|2], |[15|3], |[16|2], |[17|2], |[18|2],
 |[19|3], |[20|0], |[21|0], |[22|1],], |[0|6], |[1|0], |[2|0], |[3|0], |[4|1],],], |[575|[[0|773],
 |[1|56], |[2|146], |[3|45], |[4|97], |[5|45], |[6|54], |[7|21], |[8|29], |[9|23], |[10|17], |[11|6],
 |[12|4], |[13|10], |[14|6], |[15|1], |[16|5], |[17|2], |[18|0], |[19|2], |[20|1], |[21|1], |[22|0],
 |[23|1],], |[0|8], |[1|0], |[2|0], |[3|0], |[4|1],],], |[600|[[0|883], |[1|60], |[2|159], |[3|57],
 |[4|95], |[5|55], |[6|62], |[7|37], |[8|29], |[9|24], |[10|23], |[11|9], |[12|6], |[13|8], |[14|6],
 |[15|3], |[16|3], |[17|1], |[18|2], |[19|3], |[20|2], |[21|1], |[22|0], |[23|0], |[24|0], |[25|1],],
 |[0|10], |[1|0], |[2|0], |[3|0], |[4|1],],], |[625|[[0|989], |[1|69], |[2|180], |[3|67], |[4|117],
 |[5|51], |[6|59], |[7|46], |[8|46], |[9|23], |[10|23], |[11|10], |[12|9], |[13|10], |[14|2], |[15|4],
 |[16|6], |[17|2], |[18|2], |[19|3], |[20|1], |[21|1], |[22|1], |[23|0], |[24|0], |[25|1],], |[0|11],
 |[1|0], |[2|0], |[3|0], |[4|0], |[5|1],],], |[650|[[0|1127], |[1|83], |[2|221], |[3|79], |[4|131],
 |[5|66], |[6|68], |[7|44], |[8|51], |[9|31], |[10|20], |[11|16], |[12|18], |[13|9], |[14|3], |[15|2],
 |[16|6], |[17|3], |[18|1], |[19|3], |[20|3], |[21|2], |[22|0], |[23|2], |[24|0], |[25|0], |[26|0],
 |[27|1],], |[0|9], |[1|0], |[2|1], |[3|1], |[4|0], |[5|0], |[6|1],],], |[675|[[0|1300], |[1|79],
 |[2|252], |[3|112], |[4|141], |[5|73], |[6|90], |[7|35], |[8|58], |[9|35], |[10|36], |[11|15],
 |[12|14], |[13|12], |[14|8], |[15|7], |[16|2], |[17|4], |[18|4], |[19|2], |[20|1], |[21|0], |[22|2],
 |[23|3], |[24|0], |[25|0], |[26|0], |[27|1],], |[0|10], |[1|0], |[2|2], |[3|1], |[4|0], |[5|0], |[6|1],
],], |[700|[[0|1481], |[1|90], |[2|296], |[3|126], |[4|176], |[5|83], |[6|87], |[7|48], |[8|65],
 |[9|34], |[10|45], |[11|20], |[12|19], |[13|11], |[14|8], |[15|7], |[16|4], |[17|7], |[18|5],
 |[19|0], |[20|2], |[21|1], |[22|3], |[23|1], |[24|0], |[25|2], |[26|0], |[27|1],], |[0|14], |[1|2],
 |[2|1], |[3|1], |[4|1], |[5|0], |[6|0], |[7|0], |[8|1],],], |[725|[[0|1736], |[1|112], |[2|314],
 |[3|146], |[4|196], |[5|107], |[6|106], |[7|65], |[8|66], |[9|41], |[10|32], |[11|22], |[12|34],
 |[13|12], |[14|9], |[15|10], |[16|8], |[17|8], |[18|5], |[19|2], |[20|2], |[21|1], |[22|1], |[23|1],
 |[24|1], |[25|3], |[26|0], |[27|1],], |[0|16], |[1|3], |[2|1], |[3|1], |[4|2], |[5|0], |[6|0], |[7|0],
 |[8|1],],], |[750|[[0|1962], |[1|112], |[2|352], |[3|159], |[4|239], |[5|115], |[6|126], |[7|73],
 |[8|76], |[9|39], |[10|44], |[11|14], |[12|33], |[13|20], |[14|17], |[15|9], |[16|6], |[17|13],
 |[18|6], |[19|4], |[20|1], |[21|1], |[22|1], |[23|0], |[24|3], |[25|1], |[26|0], |[27|3],], |[0|19],
 |[1|4], |[2|2], |[3|0], |[4|2], |[5|0], |[6|1], |[7|0], |[8|0], |[9|0], |[10|1],],], |[775|[[0|2202],
 |[1|133], |[2|373], |[3|189], |[4|253], |[5|118], |[6|158], |[7|91], |[8|79], |[9|49], |[10|46],

```

[[11|24], [[12|24], [[13|17], [[14|30], [[15|17], [[16|5], [[17|11], [[18|8], [[19|6], [[20|2],
[[21|2], [[22|1], [[23|0], [[24|2], [[25|2], [[26|0], [[27|2], [[28|1], ], [[0|24], [[1|3], [[2|4],
[[3|0], [[4|1], [[5|0], [[6|2], [[7|0], [[8|0], [[9|0], [[10|1], ], ], [[800|[[0|2507], [[1|152],
[[2|435], [[3|207], [[4|310], [[5|137], [[6|170], [[7|108], [[8|89], [[9|54], [[10|48], [[11|33],
[[12|31], [[13|19], [[14|23], [[15|19], [[16|13], [[17|13], [[18|9], [[19|7], [[20|4], [[21|2],
[[22|0], [[23|1], [[24|1], [[25|1], [[26|2], [[27|3], [[28|0], [[29|1], ], [[0|30], [[1|4], [[2|4],
[[3|2], [[4|0], [[5|0], [[6|3], [[7|0], [[8|0], [[9|0], [[10|1], ], ], [[825|[[0|2794], [[1|145],
[[2|499], [[3|218], [[4|330], [[5|157], [[6|198], [[7|110], [[8|106], [[9|65], [[10|50], [[11|42],
[[12|20], [[13|26], [[14|21], [[15|15], [[16|20], [[17|19], [[18|14], [[19|6], [[20|3], [[21|3],
[[22|0], [[23|2], [[24|1], [[25|1], [[26|1], [[27|2], [[28|2], [[29|0], [[30|1], ], [[0|42], [[1|3],
[[2|4], [[3|1], [[4|3], [[5|1], [[6|2], [[7|0], [[8|1], [[9|0], [[10|1], ], ], [[850|[[0|3100], [[1|177],
[[2|551], [[3|245], [[4|367], [[5|185], [[6|221], [[7|104], [[8|107], [[9|80], [[10|74], [[11|42],
[[12|26], [[13|25], [[14|29], [[15|18], [[16|12], [[17|21], [[18|14], [[19|9], [[20|6], [[21|6],
[[22|1], [[23|1], [[24|3], [[25|1], [[26|0], [[27|3], [[28|3], [[29|0], [[30|0], [[31|1], ], [[0|45],
[[1|2], [[2|6], [[3|2], [[4|1], [[5|1], [[6|3], [[7|0], [[8|2], [[9|0], [[10|1], ], ], [[875|[[0|3400],
[[1|172], [[2|626], [[3|249], [[4|399], [[5|214], [[6|221], [[7|115], [[8|122], [[9|94], [[10|76],
[[11|60], [[12|32], [[13|28], [[14|32], [[15|22], [[16|14], [[17|17], [[18|18], [[19|11], [[20|9],
[[21|6], [[22|3], [[23|1], [[24|2], [[25|2], [[26|0], [[27|3], [[28|2], [[29|0], [[30|0], [[31|1],
[[32|1], ], [[0|50], [[1|10], [[2|11], [[3|3], [[4|0], [[5|1], [[6|2], [[7|1], [[8|1], [[9|1], [[10|1],
], ], ],
////////////////////
////////////////////
////////////////
[],

```

We can note how the animas reliant on land-based plants to survive cluster on the islands and since these islands are small have low speeds and visions, while animas which can be sustained on an aquatic diet have much higher speeds and visions on average as there is a much larger connected area of land to find food. This is a kind of observed speciation.

Surprisingly the ‘flippers’ trait was not beneficial here; it seemed that since food and mates were so abundant, speed itself is not an advantage and only increases energy demands; being slower likely meant living longer and thus having more offspring.

Thus, even though I designed flippers to be effective in these high water coverage environments, the forces of evolution, while mathematically solid, do not always operate in the most intuitive fashion.

Final evaluation (4):

This criterion has been met to a satisfying degree. As evidenced by my simulation cases, the program is versatile and able to demonstrate many different scenarios through only a few limited controllable simulation realm settings. Various key targets have been demonstrated which align with the project’s goal of being educational; these targets are terms which come up in the foundational education of evolution (speciation, stable population, extinction)

To improve, I could simply add more customisation options, most obviously for the initial anima trait setup. More customisation usually means a better sandbox as long as what is being customised is communicated clearly to the user. To this end I could add a mini-tutorial or help section to the home screen instead of forcing the user to set up the simulation before receiving the tutorial.

I could also fully implement the partially finished biome feature, which gives the user example templates of realms which achieve certain things and have different convergent traits for anima

populations. This would be especially useful to new users as they might not know how to set up a realm the way they want it, having no reference point but for the default values.

5. Enhances user understanding of evolution

- The program is shown to be conducive to greater understanding of evolutionary mechanics ✓
- The program includes written information explaining evolutionary concepts ✓

Stakeholders agree that the project clearly communicated the mechanics and concepts behind evolution. One stakeholder even remarked that they became emotional upon discovering just how complex evolution was.

6. Presents well-organised and informative data regarding the simulation

- The program features a section where data and trends can be viewed ✓
- Each trait and population variable is featured in this section; the average value is graphed alongside individual values over time, with key values such as the maximum and minimum, and variance, made clear ✓
- Data on a simulation can be downloaded onto the user's computer, where it may be stored as either a screen capture, flat file, or text document detailing key values ✗

Data on a simulation can be downloaded onto the user's computer, where it may be stored as either a screen capture, flat file, or text document detailing key values ✗

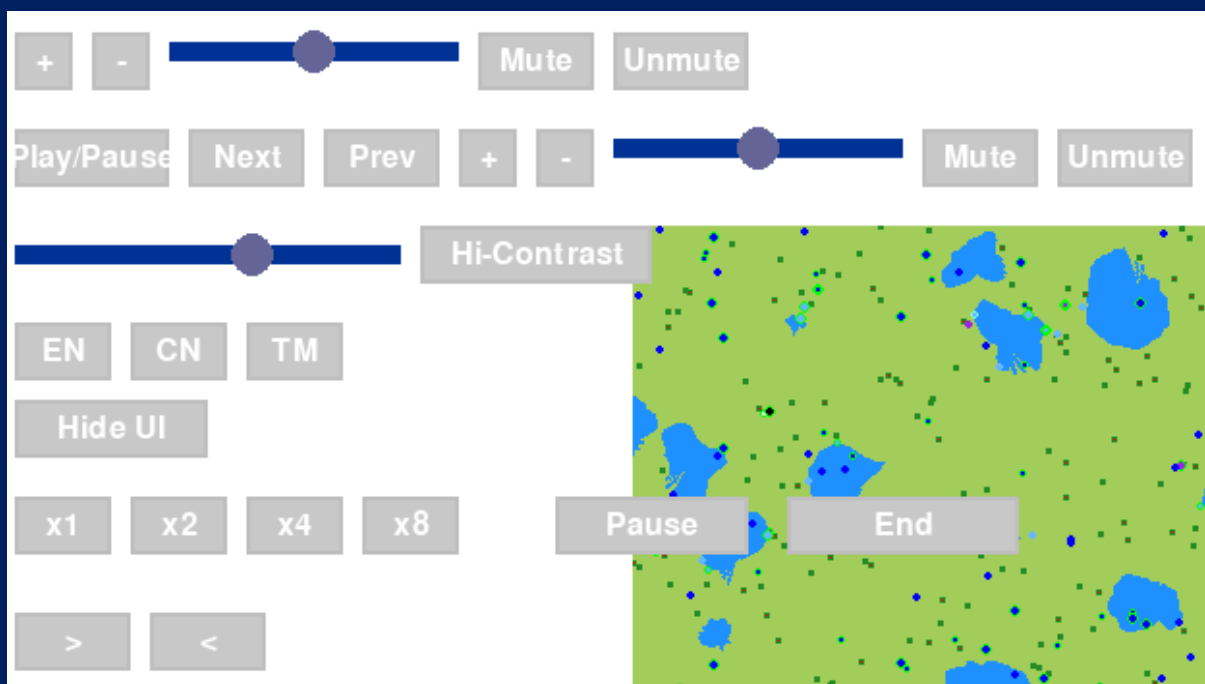
Simulation data is shown to the user through the integrated terminal I created for the pygame window. Therefore I did not use a graphing module and nor have I integrated Excel. This change in plans came from a decision based on convenience and time constraint. I had already been using the terminal to output data lines while testing, so by building the terminal directly into pygame, I was saving time and sticking to what was familiar and I had already designed. Therefore, Excel never came into the picture.

Usability Features

Of course, I have already gone over some usability features to the extent that they allow my project to reach the criteria in my success criteria, some of which are oriented around user benefit and experience.

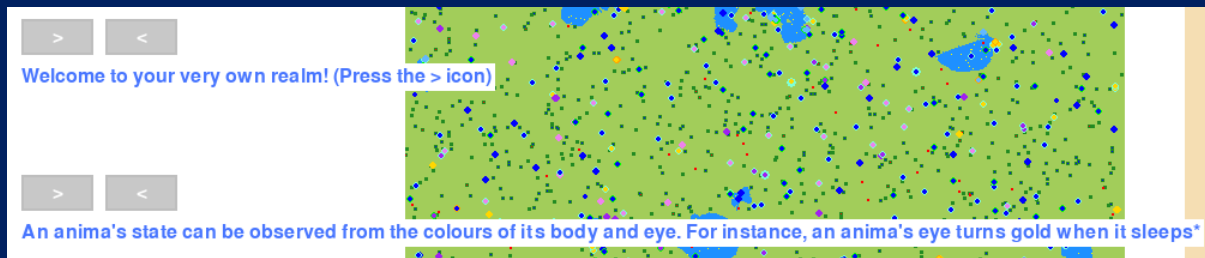
I will now be going over and evaluating my implementation and use of other miscellaneous features and other potential features I could implement to further improve my project. These features may be designed to be helpful to all users in the general case, or may be tailored towards a specific group of users, for instance the visually impaired. I have tried to make the program accessible to everyone as my stakeholder group is all mature students regardless of disability or language barrier. I have also tried to make the program as easy to use and understand as possible, for all users, and they can adjust the UI to their own needs and requirements.

The program features **buttons** and **sliders**, UI components which enhance usability by both tightening the user experience and the back end implementation. These are intuitive and easy to use, and would allow easily porting to touchscreen devices should I decide to do so. For a user, clicking simply on a button or sliding a slider is easier than typing in a text command or input value. With buttons and sliders validation is built in to the UI components and the limits of values are obvious to the user; the ends of the slider show their minimum and maximum values. Buttons and sliders condense information and input function into small components with a low amount of text to have to read. Example UI is shown below:



There are many UI elements, which may at first appear overwhelming, but they each have a very simple function which can be understood from the short text on them alone; there is also a 'Hide UI' option which can reduce the clutter so the user can focus on observing the simulation.

The program features several **tutorials** explaining various aspects of the program.



These tutorials concisely describe the purpose of the simulation and how it can be used, detailing the various UI elements and how the user can interface with the simulation via sim settings and the anima fact files. The colour tutorial details which anima states any given eye or body colour corresponds to.

The program has an **integrated terminal** which describes individual events to the user so that such events can be known from either observing them in the simulation, or by reading off the terminal:

```
Krapottyc [apple], {'growth-syndrome', 'strong-gut', 'claws'} 6.4 7.1
was born to Veimmosymm [apple], {'claws'} 7.2 6.8 and Ulegladuip [apple], {'strong-gut'} 7.1 7.3!
Eivuxogazz [apple], {'abrupt-wanderer'} 5.9 7.9 ate apple
Anaxx [apple], {'abrupt-wanderer', 'poisonous'} 5.9 7.1 ate apple
Eimiz [apple], {'poisonous', 'strong-gut'} 4.7 7.1 ate apple
Vekkaov [apple], {'plate-armour', 'strong-gut', 'play-dead'} 5.3 5.5
ate apple
Uithidole [apple], {'abrupt-wanderer'} 7.2 6.0 ate apple
Aokwubeb [apple], {'claws'} 8.2 6.2 ate apple
Hazetoott [apple], {'strong-gut'} 5.6 5.7 died of starvation aged 397
```

The terminal logs these events so that a user can know exactly what is happening (in regards to certain events) where they may not yet be fully familiar with the simulation UI. This also helps the user learn what each simulation UI event represents in the underlying logic, as they connect observed simulation phenomena with terminal transcribed events. The terminal makes use of anima nicknames to represent them, which is helpful as the information attached to nicknames (in the case above, the diet, extra traits, and genetic strength and toughness) are able to let the user know quickly about some key details regarding the anima. It is a good optimal point between just the name itself, which provides no in-simulation information, and the entire genome, which provides an overload of information. The alias then anchors the in-simulation information to a word more easily memorisable to the user.

There were some partially or unmet usability features which I will now provide comments on for how the issues surrounding them could be addressed in further development.

I intended to add a hi-contrast mode and brightness control; the UI elements are in the program as can be seen here:



However they currently have no functionality. If the program were to be developed further I would use the existing shell model to implement brightness control and apply mode setting to all colours to include the hi-contrast setting. This would be a good usability feature as it gives the user scope of choice on how they want the program to appear, which can vary depending on the circumstances of their room lighting/eyesight/colour-blindness.

I also intended to add alternate language options to English, which would benefit international users, since my stakeholder base is not specific to any one country. To implement this I would simply have a list of dictionaries mapping each language to the correct translation, for each line.

Limitations and Issues

To refer to the stated limitations on this project --

There are some certain limitations to the solution to be observed:

1. Limitation in scope
2. Limitation in accuracy
3. Limitation in computation and storage

This being a small-scale project by one person, there is an immediate limitation in scope [1]. The program must be basic, without many additional features; while this does not contradict with the highly abstracted nature of an ideal educational tool for evolution (as an all-detailed complex simulation would be little better at demonstrating the principles than this one abstracted), it does mean I may be forced not to include some features I otherwise would, or build the structure for a greater program that could conceive of more and greater cases to simulate.

Of course, the real-world mechanics of biology and evolution will not be accurately represented [2]; the program should not be used as a substitute for formal or alternate education, but as a supplement by which certain principles may be illustrated. Users should not rely on the program to provide an accurate depiction of evolution in any fine detail; there is a limitation in accuracy due to the scope [1*] as the real world is too fine and with too many factors non-extraneous to evolutionary development to create the accurate ideal (this is no severe limitation due to the lack of benefit in too great an adherence to an accurate model as it becomes inverse to simplicity which is paramount to the teaching tool).

The components of hardware and software impose a limitation on how much memory or processing power can be dedicated to the program [3]. Overloading the number of objects in the simulation or running it for too long may exhaust the computational or storage capabilities at the user end. This is of course proportional to the individual computer's specification. To address this limitation, I will impose a limit on the number of objects that can be present within the simulation at once, though this will sacrifice consistency and accuracy [2]. I will likewise limit the maximum duration of a simulation run.

--

The scope limitation can be seen with the failure to meet all described usability features and development goals. I believe I achieved a lot with the project; however some things had to be scrapped in order to stay on track and finish in a timely fashion.

The accuracy limitation I cannot say too much on as I am not a biologist myself; however from my research I attempted to be as accurate as possible for the important issues; some constants I made variable to fit the needs and the developed system of the program. Overall accuracy remains an issue due to the program's intent being educational; however in the task of describing evolution at its core I still believe it does a good job.

Performance did prove to be a significant limitation. The program never crashed from overloading, but high numbers of animas did cause considerable lag, such that it can be said that populations surpassing the ballpark of 1000 cause the program to become unusable.

The performance bottleneck could be addressed with optimisation and refactoring of code. For example, I could decrease the rate of database sampling and also reduce the number of operations performed under an anima's tick events each tick. I could also sacrifice database scope but increase performance by saving space on storing the data of anima once they reach a certain time limit in the

ghost yard. However, at this point in time, smaller populations of <1000 can still be run at decent speeds, and the simulation runs relatively smoothly at these smaller populations.

The other major performance bottleneck comes from the initial pygame setup of the simulation. This kind of bottleneck is largely built into how pygame works, but on some devices it could take a considerably long time to the point where a user might become anxious and quit the program (for instance, loading times have taken up to 2 minutes on my desktop computer). The window also sometimes throws a 'not responding' in the window bar. Unfortunately this does not seem like an issue that can be easily dealt with, and it makes it difficult to continuously run simulations back to back, as there will be semi-lengthy wait times in between.

Maintenance issues:

There are some implementations which, if I could go back and change how initially approached them, I would. A notable example is the anima genome. I did not properly leverage the OOP paradigm and ended up creating something messy and overly complex. The anima genome should have been its own class, which would have made all functions regarding them easy to implement as methods, and instead of hard coding a list with arbitrarily and redundantly defined index correspondents to genotypes, I could have easily implemented genotypes as attributes of the genome class. This would have increased maintainability and made it much easier to add, change, or delete any genotypes during further development. This is how I would fix the maintenance issue.

Another example is the realm attributes which have been similarly hard coded with redundancy. It is such that if I wish to add new attributes I would need to add repeated information throughout multiple different files in the program; in an ideally designed OOP program I should only need to add each datum once. So, in further development to fix this issue of maintainability I would add a dictionary linking each attribute to a number, and use the number to replace the attribute throughout the program, so that if the attribute were to be changed, either due to restructuring or due to the user getting some kind of customisation in some respect which would affect this ordering, only the dictionary would need to be changed.

A final issue is the lack of commenting for some complex algorithms and procedures. Comments were used throughout to make notes of ideas for future development and for brief descriptions of code blocks, but could have been employed more with directions on how functions could be used and what their in-context uses were in regards to the program. This would make maintenance and further development easier.

Other maintenance issues and their solutions were discussed in my evaluation of success criteria.

WORKS CITED

a. (n.d.).

Icoso, *I Made an Evolution Simulator (with silly little guys)*. 8 Feb 2025

[I Made an Evolution Simulator \(with silly little guys\)](#)

EightLittleBears, *various works*.

[EightLittleBears - YouTube](#)

Minutelabs, *Evolution Simulator*. 2020

[Evolution Simulator | MinuteLabs.io](#)

Pygame, *pygame documentation*

[Pygame Front Page — pygame v2.6.0 documentation](#)

APPENDIX - CODE LISTING

Given by alphabetical file name, python files only

ANGLE.PY

```
import math

PI = math.pi

def sin(theta):
    return math.sin(theta)

def cos(theta):
    return math.cos(theta)

def atan2(y, x):
    return math.atan2(y, x)

def lerple(current, true, max_turn):
    change = (true - current - PI) % (2 * PI) - PI
    change = max(-max_turn, min(max_turn, change))
    #return min(max_turn, max(-max_turn, change))
    return change
```

ANIMA.PY

```

from cell import Cell
from extratraits import ExtraTraits, trait_costs
from flora import Flora
from angle import *
from nickname import *
from todisplay import *
import random
from math import sqrt, hypot

#import numpy as np

MIN_DISTANCE_TO_TARGET = 0.05

TERRAIN_FACTOR_WEIGHTED_SUM = 1
GRASS_BIAS = 5
WATER_BIAS = 2

CONSUMPTION_POWER_FACTOR = 0.75

AWAKENING_COST = 2.5 #this is the extra sleep status amount added to sleep debt on being
awakened while still asleep

DAMAGE_FACTOR = 5
RANDOM_DAMAGE_FACTOR = 0.2
INK_SHOT_RANGE = 8

BOOST_EXPENSE = 1.2 #should scale as a ratio to normal food expense ***
BOOST_EQUILIBRIUM = 0.25

MAXIMUM_MATING_DRIVE = 75
TWIN_ODDS = 0.025
TWINS_IDENTICAL_ODDS = 0.25

INHERITANCE_TYPE = "binary"
MUTATION_FACTOR = 0.1 #average mutation factor ; default 0.05
DIET_MUTATION_PLUS_FACTOR = 0 #chance of newborn gaining new diet type given they
inherited at least one type from parents 0.1
DIET_PRESERVATION_FACTOR = 0.96 #chance of diet element being preserved to child, given
parent 0.95
HYBRID_DIETS_ON = False
MIN_GENE_LIMIT = 0.05
MAX_GENE_LIMIT = 500

EXTRA_TRAIT_MUTATION_ODDS = 0.02
EXTRA_TRAIT_ROLLS = 4
EXTRA_TRAIT_PRESERVATION_FACTOR = 0.8

```

```

SENESCENCE_AGE = 250 #was 1000 but now scales with square root of size

MEAT_CHUNK_SIZE = 3
MEAT_SIZE_SCALE_POWER_FACTOR = 1
CORPSE_DECAY_RATE = 0.1 # = probability of decay on each tick; first decay and decrease
until <1
#decay was 0.05

extra_traits = [
    "egg-laying",
    "claws",
    "flippers",
    "abrupt-wanderer",
    "poisonous",
    "venomous",
    "pincers",
    "snowycloak",
    "strong-gut",
    "sand-attack",
    "spiritful-child",
    "inky-shot",
    "berserker",
    "vampirism",
    "zombie",
    "gluttonous-child",
    "necro-mimicry",
    "absorb-victim",
    "mimic-attack",
    "leviathan",
    "play-dead",
    "plate-armour",
    "growth-syndrome",
]

food_list = [
    "apple",
    "berry",
    "melon",
    "lily",
    "lavendar",
    "meat",
    "wildgrass",
]

green_food_list = food_list
green_food_list.remove("meat")

class Anima(Cell):

    def __init__(self, id, naming_mode, genome, realm, spawnx, spawny):

```

```

super().__init__(realm, spawnx, spawny)

self.id = id

self.angle = random.uniform(0, 2 * PI)
self.roam_angle = random.uniform(0, 2 * PI)
self.dx = 0
self.dy = 0

self.age = 0
self.status = [25,25,0,0,-999] #hunger, thirst, sleepiness, damage, mating drive
(genetic) cannot be >80
self.target = None
self.action = [self.do_nothing,self.do_nothing]
self.rejectedList = []
self.eatenList = []
self.offspring = []
self.parents = []
self.goals = []
self.mode = 1 # 0 sleep 1 roam 2 approach 3 flee

self.locked = False
self.fleeing = False
self.hunting = False
self.post_hunt = False

self.conditions = []
self.responses = []

self.asleep = False
self.sleep_debt = 0
self.dream = self.do_nothing #***

self.genome = genome
self.diet = genome[0]
self.speed = genome[1][0]

self.terrain_factor_sum = genome[1][1][0] * GRASS_BIAS + genome[1][1][1] *
WATER_BIAS + genome[1][1][2]
self.grass_factor = genome[1][1][0]/self.terrain_factor_sum *
TERRAIN_FACTOR_WEIGHTED_SUM
self.amphibian_factor = genome[1][1][1]/self.terrain_factor_sum *
TERRAIN_FACTOR_WEIGHTED_SUM
self.ice_factor = genome[1][1][2]/self.terrain_factor_sum *
TERRAIN_FACTOR_WEIGHTED_SUM

self.turnspeed = genome[2]
self.vision = genome[3]
self.strength = genome[4]
self.toughness = genome[5]
self.birth_size = self.size()

```



```

self.mating_drive = genome[6]
self.boost_factor = genome[7]
self.injury_threshold = genome[8]
self.maturity_age = genome[9]
self.senescence_age = (SENESCENCE_AGE + self.maturity_age) * sqrt(self.size())
self.refractory = genome[10]
self.extra_traits = genome[11]
self.reach = 2.5

self.food_given = "meat"
self.food_effect = [(-MEAT_CHUNK_SIZE * 100), 0, 0, (-MEAT_CHUNK_SIZE *
3), MEAT_CHUNK_SIZE * 20]
self.food_chunks = int((self.size()-
1)**MEAT_SIZE_SCALE_POWER_FACTOR/MEAT_CHUNK_SIZE) + 1 #equivalent to fruitNo, could merge

self.alias = None
self.nickname = self.create_nickname(naming_mode) #dependent on realm

def tickThink(self):
    self.responses.append(self.unlock)
    self.setGoals()
    self.choose_mode()
    self.set_velocity()      #sets velocity and action

    #check if can mate

    #method sets private states: dx, dy, target, mode

def tickAct(self):
    self.asleep = False
    self.action[1](self.target)
    self.move()
    self.increment_age()

    #method sets public states: asleep, status

def tickRespond(self):
    for response in self.responses:
        response()
    self.responses = []

def increment_age(self):

    self.update(0, 0.125 * self.energy_expense())
    #print(self.energy_expense())
    self.update(1, 0.2) # *self.realm.temperature ?

```

```

self.update(2, 0.04 * self.energy_expense()) #need downsides for each trait
self.update(3,-0.1)

conditions = self.conditions
for condition in conditions:
    condition(self)

if not self.is_mature():
    self.grow()
    if "spiritful-child" in self.extra_traits:
        self.update(2,-0.2)
    if "gluttonous-child" in self.extra_traits:
        self.update(0,1)
        self.grow()
else:
    #if self.status[4] < self.mating_drive:
    #self.status[4] = min(self.mating_drive, MAXIMUM_MATING_DRIVE)
    if self.status[4] < 10:
        self.update(4, 10-self.status[4]) #should it be less?

    self.update(4, 0.1)

    self.update(3, -0.2)

    if "growth-syndrome" in self.extra_traits:
        self.update(0,1)
        self.grow()

if self.is_senescent:
    self.update(3, 0.1)

self.age += 1

def energy_expense(self):
    return (0.04 * self.vision**2)/sqrt(self.size()) + self.energy_expense_by_moving()
+ 1.5 + min(0.25/self.realm.temperature, 2) + self.energy_expense_by_extra_traits()
#should vision be sqrt or not?
    #energy is expended by 1. vision/senses 2. movement 3. general expenses 4. heating
5. extra traits

def energy_expense_by_moving(self):
    if self.realm.environment[int(self.y)][int(self.x)] in terrain_energy_expenditure:
        terrain_cost =
terrain_energy_expenditure[self.realm.environment[int(self.y)][int(self.x)]]
    else:
        #for floras
        terrain_cost = terrain_energy_expenditure["grass"]
    return hypot(self.dx, self.dy) * terrain_cost * sqrt(self.speed)

def energy_expense_by_extra_traits(self):
    return sum([trait_costs[trait] for trait in self.extra_traits.traits]) * (1.005 **
len(self.extra_traits)) #physical overload factor

```

```

def grow(self):
    self.strength += 0.025
    self.toughness += 0.025

def round_status(self):          #for displaying
    #status = [round(need) for need in self.status]
    return round_list(self.status)

def genome_to_display(self):
    return to_display(self.genome)

def size(self):
    return sqrt(self.strength * self.toughness)    #should size affect eating? various
implications

def diet_type(self):
    if "meat" in self.diet:
        if len(self.diet) == 1:
            return "carnivore"
        else:
            return "omnivore"
    return "herbivore"

def will_sleep(self):
    return self.status[2] >= 100 or (self.mode == 0 and self.status[2] >=
max(self.status[0], self.status[1]))

def setGoals(self):
    #this method updates the goals of the associated anima, creating a new empty list and
    appending at least one int element goal; the greatest need is appended first, then if the
    second greatest need is close enough in urgency then it is appended too, and so on
    #per tick

    self.goals = []
    needs = self.status[:]
    needs[2] = 0          #for now, sleepiness will not affect priority of other needs

    if self.post_hunt:
        if needs[0] >= 10 and isinstance(self.target, Anima):          #1 tick delay
            needs[0] = 100
        else:
            self.post_hunt = False
            print(f"{self.nickname} has finished its meal")

    prevGreatestNeed = 0

    for _ in range(len(needs)):
        greatestNeed = needs.index(max(needs))

```

```

        if needs[greatestNeed] > prevGreatestNeed - 1.5 * sqrt(abs(100-
prevGreatestNeed)):
            self.goals.append(greatestNeed)
            prevGreatestNeed = needs[greatestNeed]
            needs[greatestNeed] = -9999
    return

def choose_mode(self):
    #this method updates the mode of the associated anima based on anima logic, setting
the int variable mode to the correct value (0:sleep/1:roam/2:approach/3:flee)
    #per tick
    self.hunting = False          #hunting timer

    if self.will_sleep():
        self.mode = 0

    else:
        self.mode = 1

        none_means_fleeing = self.checkSurroundings()

        if self.target != None:
            self.mode = 2

        if none_means_fleeing == None and self.target != None:    #currently fleeing
ends as soon as cannot see predator
            self.mode = 3
            self.fleeing = True

def set_action(self, action=None): #why wasnt tiggerinbg mate eaelier before code fix
of simply moving to making set action to be less accessible?
    action_index = {
        None : [self.do_nothing,self.do_nothing],
        0 : [self.do_nothing,self.eat],
        1 : [self.do_nothing,self.drink],
        #2 : [self.do_nothing,self.sleep],
        4 : [self.lock_mate,self.mate],
    }

    if self.mode == 0:
        return [self.do_nothing,self.sleep]
    elif self.mode == 1:
        return [self.do_nothing,self.do_nothing]
    elif self.hunting:                                #subset of mode 2
where target is not in reach and is a prey food
        return [self.do_nothing,self.set_hunting_action()]
    elif self.mode == 2:
        return action_index[action]
    elif self.mode == 3:

```

```

        return [self.do_nothing, self.set_fight_or_flight()] #next: fight or flight
(speed boost)

    def set_fight_or_flight(self):
        if "inky-shot" in self.extra_traits and self.distance_to(self.target) <
INK_SHOT_RANGE:      #check inkbag also
            return self.ink_shot

        if (self.strength > self.target.toughness and not self.is_injured()) or
self.locked or "berserker" in self.extra_traits:
            if self.is_target_in_reach():
                return self.fight
            else:
                return self.do_nothing
        else:
            return self.boost

    def set_hunting_action(self):
        if self.is_target_in_reach():
            return self.eat
        elif self.is_starving():
            return self.boost
        return self.do_nothing

    def is_target_in_reach(self):
        tx, ty = self.get_target_x(), self.get_target_y()                #direct centre
targetting problem, with high resolution is negligible
        return self.is_within_point(tx,ty,self.reach)

    def checkSurroundings(self):                #checks surroundings and sets target
        vision_modifier = 1

        for _ in range(min(10, self.conditions.count(blind))):
            vision_modifier *= 0.8

        vision = self.vision * vision_modifier

        self.cells_to_check = []

        for dy in range(int(2*vision+1)):      #future: raycasting prevents animas seeing
through objects
            row = (int(self.y - vision) + dy) % self.realm.height
            for dx in range(2*int(vision)+1):
                col = (int(self.x - vision + 0.5) + dx) % self.realm.width
                if self.distance_to_point(col, row) <= vision:
                    self.cells_to_check.append((row, col))

        target = self.__check_for_danger(vision)
        if target == None:
            for goal in self.goals:

```



```

        pass #if visible

        if target == None:
            target = self.check_for(goal, vision)
            if target != None:
                self.target = target
                if goal == 0 and isinstance(target, Anima):
                    self.hunting = True
                return goal
            #should modes be changed outside choosemode?
        self.target = target
        return None
    #return False

def check_for(self, goal, vision):
    if goal == 0:
        target = self.__check_for_food(vision)
    elif goal == 1:
        target = self.__check_for_water(vision)
    elif goal == 2:
        #self.__check_for_shelter
        target = None
    elif goal == 3:
        #flee?
        target = None
    elif goal == 4:
        target = self.__check_for_mates(vision)
    return target

def __check_for_food(self, vision):
    nearest_food = None
    nearest_dist = vision

    for element in self.check_spatial_hash(vision):
        if not element in self.eatenList and element.food_given in self.diet and not
self.is_family(element) and not (isinstance(element, Flora) and element.fruitNo ==
0): #allow children to eat parents?
            dist = self.distance_to(element)
            if dist <= nearest_dist: #vision for emergency clamping
                if not isinstance(element, Anima) or self.is_threat_to(element) or
element.is_dead(): #logical OR short circuits //self.toughness > element.strength or
                    nearest_dist = dist
                    nearest_food = element

    #if nearest_food == None and "grass" in self.diet:

    return nearest_food

def __check_for_water(self, vision):

```

```

nearest_water = None
nearest_dist = vision + 1

for pos in self.cells_to_check:
    if self.realm.environment[pos[0]][pos[1]] == "water":
        dist = self.distance_to_point(pos[1], pos[0])
        if min(dist, nearest_dist, vision) == dist:    #vision for emergency
            nearest_dist = dist
            nearest_water = (pos[1],pos[0])
return nearest_water

def __check_for_mates(self, vision):
    nearest_mate = None
    nearest_dist = vision + 1

    for element in self.check_spatial_hash(vision):
        if not element in self.rejectedlist and isinstance(element, Anima) and not
element == self :    #prevent children and parents?
            dist = self.distance_to(element)
            if min(dist, nearest_dist, vision) == dist:    #vision for emergency
                nearest_dist = dist
                nearest_mate = element
            elif not (not self.is_threat_to(element) and not
self.is_threatened_by(element)) :
                self.rejectedList.append(element)
    return nearest_mate

def __check_for_danger(self, vision):
    nearest_danger = None
    nearest_dist = vision + 1

    for threat in self.check_spatial_hash(vision):
        if isinstance(threat, Anima):
            dist = self.distance_to(threat)
            if min(dist, nearest_dist, vision) == dist:    #vision for emergency
                nearest_dist = dist
                nearest_danger = threat
    return nearest_danger

def check_spatial_hash(self, vision):
    return self.realm.spatialhash.get_neighbours(self.x, self.y, vision)

def is_threatened_by(self, threat): #so there is a 1-tick wake up grace period
    return isinstance(threat, Anima) and not threat.is_dead() and not threat.asleep
and (not threat.is_injured() or threat.status[0] > threat.status[3]) and

```

```

((threat.strength > self.toughness) or self.is_injured()) and "meat" in threat.diet and
not self.is_family(threat)

def is_threat_to(self, other):
    return isinstance(other, Anima) and other.is_threatened_by(self)

def set_velocity(self):
    true_angle = 0
    speed_modifier = 1 - (self.status[3])/120
    self.action = self.set_action()

    if "plate-armour" in self.extra_traits:
        speed_modifier *= 0.75

    if self.locked:
        speed_modifier = 0

    elif self.mode == 0:
        speed_modifier = 0

    # can do slowing down when approaching object

    elif self.mode == 1:
        if "abrupt-wanderer" in self.extra_traits:
            if random.uniform(0,1) < 0.05:
                self.roam_angle = random.uniform(0, 2 * PI)
            else:
                self.roam_angle += random.uniform(-0.07, 0.07)
                if random.uniform(0,1) < 0.1:
                    self.roam_angle += random.uniform(-0.8, 0.8)
                if random.uniform(0,1) < 0.015:
                    self.roam_angle = random.uniform(0, 2 * PI)
                self.roam_angle = self.roam_angle % (2*PI)

        true_angle = self.roam_angle

        """
        dx_t = self.target.x - self.x
        dy_t = self.target.y - self.y
        dist = math.hypot(dx_t, dy_t)

        if dist > 0:
            speed = min(self.speed, dist)
            self.dx = speed * dx_t / dist
            self.dy = speed * dy_t / dist
        else:
            self.dx = self.dy = 0
            """

    elif self.mode == 2:

```

```

        if self.is_target_in_reach():
            speed_modifier = 0
            self.action = self.set_action(self.checkSurroundings())
            self.action[0](self.target)                                #should this
be here?***
        else:
            """dsx = self.get_target_x() - self.x
            dsy = self.get_target_y() - self.y
            true_angle = atan2(dsy, dsx)"""

            true_angle = self.angle_to(self.target)

    elif self.mode == 3: #flee
        """dsx = self.get_target_x() - self.x
        dsy = self.get_target_y() - self.y
        true_angle = (atan2(dsy, dsx) + math.pi) % (2 * math.pi)"""
        true_angle = self.target.angle_to(self)

        #reset roam angle upon fleeing (to prevent repeatedly running back toward
predator once out of vision range)
        self.roam_angle = random.uniform(0, 2 * PI)
        self.roam_angle = self.roam_angle % (2*PI)

    if self.realm.environment[int(self.y)][int(self.x)] == "grass":
        speed_modifier = speed_modifier * (1 + self.grass_factor)
        if "flippers" in self.extra_traits:
            speed_modifier *= 0.75

    if self.realm.environment[int(self.y)][int(self.x)] == "water": #should round it
        speed_modifier = speed_modifier * (0.5 + self.amphibian_factor)
        if "flippers" in self.extra_traits:
            speed_modifier *= 2

    if self.realm.environment[int(self.y)][int(self.x)] == "ice":
        speed_modifier = speed_modifier * self.ice_factor

    self.angle = self.angle + lerple(self.angle, true_angle, self.turnspeed)
    self.dx = self.speed * cos(self.angle) * speed_modifier
    self.dy = self.speed * sin(self.angle) * speed_modifier

    """if self.target != None and (self.y + self.dy - self.get_target_y()) * (self.y
- self.get_target_y()) < 0:
        self.dy = self.get_target_y() - self.y - MIN_DISTANCE_TO_TARGET *
(2*int(self.get_target_y() > self.y) - 1)
        if self.target != None and (self.x + self.dx - self.get_target_x()) * (self.x -
self.get_target_x()) < 0:
            self.dx = self.get_target_x() - self.x - MIN_DISTANCE_TO_TARGET *
(2*int(self.get_target_x() > self.x) - 1)"""

```

```

        if self.target != None and self.speed * speed_modifier >
self.distance_to(self.target):
            self.dy = self.get_target_y() - self.y - MIN_DISTANCE_TO_TARGET *
(2*int(self.get_target_y() > self.y) - 1)
            self.dx = self.get_target_x() - self.x - MIN_DISTANCE_TO_TARGET *
(2*int(self.get_target_x() > self.x) - 1)

    def get_target_x(self):
        if isinstance(self.target, Cell):
            tx = self.target.x
        elif isinstance(self.target, tuple):
            tx = self.target[0]                    #direct centre targetting problem, with
high resolution is negligible
        return tx

    def get_target_y(self):
        if isinstance(self.target, Cell):
            ty = self.target.y
        elif isinstance(self.target, tuple):
            ty = self.target[1]                    #direct centre targetting problem, with
high resolution is negligible
        return ty

    def move(self, factor = 1):
        self.x += self.dx
        self.y += self.dy
        #temporary solution to edge of map logic
        if not 0 <= self.x < self.realm.width:
            self.x = self.x % self.realm.width
        if not 0 <= self.y < self.realm.height:
            self.y = self.y % self.realm.height

    def boost(self, _ = None):
        self.move(self.boost_factor)
        self.update(0, max(BOOST_EXPENSE / BOOST_EQUILIBRIUM * (self.boost_factor**2) *
self.energy_expense_by_moving(), BOOST_EXPENSE * self.boost_factor *
self.energy_expense_by_moving()))
        print(f"{self.nickname} is boosting to get away from {self.target.nickname}")

    def fight(self, anima):
        #damage = DAMAGE_FACTOR * (1+random.uniform(-
2*RANDOM_DAMAGE_FACTOR,2*RANDOM_DAMAGE_FACTOR)) * (self.strength/anima.toughness + max(0,
(self.strength-anima.toughness))) #factor method??? returning this to use with all random
factors
        damage = max(1, DAMAGE_FACTOR * (1+random.uniform(-
2*RANDOM_DAMAGE_FACTOR,2*RANDOM_DAMAGE_FACTOR)) * (self.strength/anima.toughness))
        if anima.is_injured():
            damage = damage * 1.15 + 2
        if "plate-armour" in self.extra_traits:

```



```

        damage = damage * 0.5
        anima.take_damage(damage)
        print(f"{self.nickname} struck {self.target.nickname} and dealt
{to_display(damage)} damage")

    if anima.asleep:
        anima.responses.append(self.awaken)

    attacks = self.extra_traits #self.extra_traits.attack()

    if "mimic-attack" in self.extra_traits:
        attacks.extend(anima.extra_traits.traits)

    if "pincers" in attacks and self.strength > anima.toughness:
        self.grab(anima)

    if "claws" in attacks and random.random() < (self.strength -
anima.toughness)/anima.toughness:
        anima.conditions.append(bleed)
        anima.conditions.append(bleed)

    if "venomous" in attacks:
        anima.conditions.append(define_poison(5 * self.strength / anima.toughness,
"strong-gut" in anima.extra_traits))

    if "sand-attack" in attacks:
        anima.conditions.append(blind)

def take_damage(self, amount):
    self.update(3, amount)

def grab(self, target):
    angle = self.angle_to(target)
    r = self.reach * 0.9
    if self.distance_to(target) > r:
        x = self.x
        y = self.y
        def set_position():
            target.x = x + r * cos(angle)
            target.y = y + r * sin(angle)
        target.responses.append(set_position)
    target.responses.append(target.lock)
    self.responses.append(self.lock)

def ink_shot(self, target):
    if random.random() < 0.5:
        target.conditions.append(blind)

```

```

def is_family(self, anima):
    #check immediate family, ie direct children or parents
    return anima in self.offspring or anima in self.parents or anima == self

def is_satiated(self):
    return self.status[1] < 35

def is_starving(self):
    return self.status[1] > 72.5

def is_injured(self):
    return self.status[3] > self.injury_threshold

def is_bleeding(self):
    return any(getattr(condition, "effect_type", None) == "bleed" for condition in
self.conditions)

def is_poisoned(self):
    return any(getattr(condition, "effect_type", None) == "poison" for condition in
self.conditions)

#def sick():

def is_mature(self):
    return self.age >= self.maturity_age

def is_senescent(self):
    return self.age >= self.senescence_age

def is_dead(self):
    if any(need >= 100 for i,need in enumerate(self.status) if i != 2): #except
sleep?
        return True
    return False

def autopsy(self):
    for i in range(len(self.status)):
        if not i == 2 and self.status[i] >= 100:      #cannot die of sleepiness
            return cause_of_death_index[i]
    #age
    return "unknown causes"

def autopsy_index(self):
    for i in range(len(self.status)):
        if not i == 2 and self.status[i] >= 100:
            return i

    return None

#mortuary

```

```

def update(self, need, quantity, sign=1):
    if need == 3 and quantity*sign < 0 and self.is_senescent():
        return

    if need == 4 and not self.is_mature():
        return

    elif need == 4 and self.status[4] + quantity*sign > MAXIMUM_MATING_DRIVE:
        self.status[4] = MAXIMUM_MATING_DRIVE
        return

    self.status[need] += quantity * sign
    if self.status[need] < 0:
        self.status[need] = 0

def do_nothing(_ = None, __ = None):
    pass

def eat(self, source, quantity = 1): #currently eating takes only 1 tick
    if source in self.realm.elements[1]:
        self.fight(source)
        self.responses.append(self.check_preay)
    else:
        quantity_eaten = source.provide(quantity)
        effect_modifier = quantity_eaten / (self.birth_size**CONSUMPTION_POWER_FACTOR)
        / sqrt(len(self.diet)+0.25)
        for need in range(len(source.food_effect)):
            self.update(need,source.food_effect[need] * effect_modifier)    #should be
sqrt or not?
            if source.food_given == "meat":
                if "poisonous" in source.extra_traits:
                    if random.random() < 0.25:
                        self.conditions.append(define_poison(12, "strong-gut" in
self.extra_traits))
                if "vampirism" in self.extra_traits:
                    for i in range(4):
                        self.drink()
            else:
                self.eatenList.append(source)
                print(f"{self.nickname} ate " + source.food_given)

def drink(self, _ = None):
    self.update(1,-5)
    #print(f"{self.nickname} drank...")

def sleep(self, _ = None):
    self.asleep = True
    self.update(2, self.sleep_debt)

```

```

self.sleep_debt = 0

self.update(2,-3) #sleepiness is reduced inversely with vision
self.update(3,-1.5)
self.rejectedList = []
self.eatenList = []

self.dream()
#print(f"Shhh! {self.nickname} is sleeping...")

def get_sleepiness(self):
    return self.status[2]

def awaken(self):    #response
    self.sleep_debt = max(AWAKENING_COST, self.status[2] - max(self.status[0],
self.status[1], self.status[4]))
    self.update(2,-self.sleep_debt+AWAKENING_COST)
    print(f"{self.nickname} was startled awake!")

def check_preyl(self):    #response
    if self.target.is_dead():
        self.post_hunt = True
        if "absorb-victim" in self.extra_traits and self.target.extra_traits:
            self.extra_traits.add(random.choice(list(self.target.extra_traits)))
            self.extra_traits.correct_traits()

def mate(self, mate):
    new_genome = self.mix_genomes(mate)

    newborn = Anima(self.realm.anima_id(), self.realm.creature_naming_mode,
new_genome, self.realm, self.x, (self.y+1)%self.realm.height)    #random adjacent
cell? #sad that cannot isolate from realm for this line
    newborn.parents.extend([self, mate])
    self.offspring.append(newborn)
    mate.offspring.append(newborn)
    self.realm.add_element(newborn)

    while random.random() < TWIN_ODDS:
        if random.random() < TWINS_IDENTICAL_ODDS:
            identical_newborn = Anima(self.realm.anima_id(),
self.realm.creature_naming_mode, new_genome, self.realm, (self.x+1)%self.realm.width,
self.y)

            identical_newborn.parents.extend([self, mate])
            self.offspring.append(identical_newborn)
            mate.offspring.append(identical_newborn)
            self.realm.add_element(identical_newborn)
            print(f"{newborn.nickname} and {identical_newborn.nickname} are identical
twins!")
        else:
            new_genome = self.mix_genomes(mate)

```

```

        twin_newborn = Anima(self.realm.anima_id(),
self.realm.creature_naming_mode, new_genome, self.realm, (self.x+1)%self.realm.width,
(self.y+1)%self.realm.height)  #random adjacent cell?  #sad that cannot isolate from
realm for this line
        twin_newborn.parents.extend([self, mate])
        self.offspring.append(twin_newborn)
        mate.offspring.append(twin_newborn)
        self.realm.add_element(twin_newborn)
        print(f"{newborn.nickname} and {twin_newborn.nickname} are twins!")

    self.update(2,50)
    self.update(3,25)
    self.update(4,-self.refractory)
    mate.update(2,50)
    mate.update(3,25)
    mate.update(4,-mate.refractory)
    mate.locked = False

    print(f"{newborn.nickname} was born to {self.nickname} and {mate.nickname}!")

def lock_mate(self, mate):
    mate.locked = True

def unlock(self):
    self.locked = False

def lock(self):
    self.locked = True

def mix_genomes(self, mate):
    """new_genome = []
    for genotype in range(len(self.genome)):
        if isinstance(self.genome[genotype], list):
            new_genome.append([])
            for gene in range(len(self.genome[genotype])):
                #incompatible
with list diet and list speeds
                new_genome[genotype].append(mix_genes(type(self.genome[genotype][gene]
),self.genome[genotype][gene],mate.genome[genotype][gene]))
            else:
                new_genome.append(mix_genes(type(self.genome[genotype]),
self.genome[genotype], mate.genome[genotype])) """
        new_genome = mix_genes(list, self.genome[:], mate.genome[:],
realm_foods(self.realm))
        return new_genome

def reproduce(self):
    #append to self.offspring
    pass

def provide(self, quantity = 1):

```



```

quantity_provided = min(quantity, self.food_chunks)
self.food_chunks -= quantity_provided
return quantity_provided

def tickDecay(self):          #very basic for now
    corpse_to_decay = CORPSE_DECAY_RATE
    while corpse_to_decay >= 1:
        self.food_chunks -= 1
        corpse_to_decay -= 1
    if random.random() < corpse_to_decay:
        self.food_chunks -= 1

def create_nickname(self, naming_mode):
    nickname_structure = naming_mode.split("/")
    #print(nickname_structure)
    nickname = ""
    for mode in nickname_structure:
        #print(mode)
        #print(vars(self))
        if mode in ["", "nameless"]:
            nickname += "A creature"
        elif mode in ["random", "surprise", "name"]:
            self.alias = generate_nickname("anima")
            nickname += self.alias.get_alias()
        elif mode == "story":
            #nickname += random story name
            pass
        elif mode in vars(self):
            nickname += to_display(getattr(self, mode))
        else:
            pass
        nickname += " "
    nickname = nickname[:len(nickname)-1]

    return nickname

def get_alias(self):
    return self.alias.get_alias()

def update_nickname(self):
    pass

def name_me(self):
    pass

#should've made genome a class maybe
def mix_genes(gene_type, A, B, food_options = food_list):
    if gene_type == tuple:          #diet must be only tuple
        new_gene = ()

```

```

        if random.uniform(0,1) < 0.1 and HYBRID_DIETS_ON ==
True:                                #omnivore
            for item in A:
                if item not in new_gene and random.uniform(0,1) <
DIET_PRESERVATION_FACTOR:
                    new_gene = new_gene + (item,)
            for item in B:
                if item not in new_gene and random.uniform(0,1) <
DIET_PRESERVATION_FACTOR:
                    new_gene = new_gene + (item,)
            else:
                #elif random.uniform(0,1) < 0.95
                for item in random.choice([A, B]):
                    if item not in new_gene and random.uniform(0,1) <
DIET_PRESERVATION_FACTOR:
                        new_gene = new_gene + (item,)
                if new_gene == () or (random.uniform(0,1) < DIET_MUTATION_PLUS_FACTOR and
HYBRID_DIETS_ON):
                    new_gene = new_gene + (random.choice([food for food in food_options if food
not in new_gene]),)
                return new_gene

    elif gene_type == ExtraTraits:
        new_gene = ExtraTraits(set())
        atraits = list(A.traits)
        btraits = list(B.traits)
        traits = []
        while atraits or btraits:
            if random.random() < 0.5 and atraits:
                traits.append(atraits.pop())
            elif btraits:
                traits.append(btraits.pop())
        for trait in traits:
            if random.random() < EXTRA_TRAIT_PRESERVATION_FACTOR/(len(new_gene)+1) and
trait not in new_gene:
                new_gene.add(trait)
        for roll in range(EXTRA_TRAIT_ROLLS):
            if random.random() < EXTRA_TRAIT_MUTATION_ODDS:
                new_gene.add(random.choice(extra_traits))
        new_gene.correct_traits()
        return new_gene

    elif gene_type == int:
        #new_gene = (A+B)/2 * (1 + random.uniform(-2,2) * MUTATION_FACTOR)
        if INHERITANCE_TYPE == "mean":
            new_gene = random.gauss(((A+B)/2), (MUTATION_FACTOR*(A+B)/2))
        elif INHERITANCE_TYPE == "binary":
            G = random.choice((A,B))
            new_gene = random.gauss(G, MUTATION_FACTOR*G)
        new_gene = max(new_gene, MIN_GENE_LIMIT)

```

```

        new_gene = min(new_gene, MAX_GENE_LIMIT) #should
i set limits for each gene specifically? would require overhaul of this and use of a
dictionary
        return int(round(new_gene))

    elif gene_type == float:
        #new_gene = (A+B)/2 * (1 + random.uniform(-2,2) * MUTATION_FACTOR)
        if INHERITANCE_TYPE == "mean":
            new_gene = random.gauss(((A+B)/2), (MUTATION_FACTOR*(A+B)/2))
        elif INHERITANCE_TYPE == "binary":
            G = random.choice((A,B))
            new_gene = random.gauss(G, MUTATION_FACTOR*G)
        #new_gene = random.gauss((A+B/2), MUTATION_FACTOR)
        new_gene = max(new_gene, MIN_GENE_LIMIT)
        new_gene = min(new_gene, MAX_GENE_LIMIT) #should
i set limits for each gene specifically? would require overhaul of this and use of a
dictionary
        return float(new_gene)

    elif gene_type == list:
        return [mix_genes(type(a), a, b, food_options) for a, b in zip(A, B)]

    raise ValueError(f"Genotype must be one of the given data types; Invalid type
{gene_type}")

def bleed(this):
    this.take_damage(1)
    if random.random() < 0.15: #self.bloodclottingrate, also gene for passive healing?
        this.conditions.remove(bleed)
bleed.effect_type = "bleed"

def define_poison(severity, stronggut):
    def suffer_poison(this):
        nonlocal severity
        this.take_damage(severity)
        this.update(0,severity/this.size())
        this.update(1,severity/this.size())
        if random.random() < 0.15 if not stronggut else 0.3:
            this.conditions.remove(suffer_poison)
            if severity >= 2:
                this.conditions.append(define_poison(int(severity/2), "strong-gut" in
this.extra_traits))
        suffer_poison.effect_type = "poison"
        return suffer_poison

    def intoxicated(this):
        if random.random() < 0.15 if not "stronggut" in this.extra_traits else 0.3:
            this.conditions.remove(intoxicated)
    intoxicated.effect_type = "intoxicated"

def blind(this):

```

```
if random.random() < 0.15:
    this.conditions.remove(blind)
blind.effect_type = "blind"

def realm_foods(realm):
    return [fruit.name for fruit in realm.fruit_list if fruit.name in food_list] +
["meat"]

cause_of_death_index = {
    0 : "starvation",
    1 : "thirst",
    3 : "its wounds",
}

terrain_energy_expenditure = {
    "grass" : 1,
    "water" : 1.5,
    "ice" : 2,
}
```

BIOME.PY

```
class Biome():
    def __init__(self, facts):
        self.width_unit = facts[0][0]
        self.height_unit = facts[0][1]
        self.temperature = facts[1]
        self.wetness = facts[2][0]
        self.iciness = facts[2][1]
        self.vegetation = facts[3]
        #self.natural_elements = facts[4]
        #self.native_fauna = facts[5]

biomes = {
    "testk utopia" : Biome([[600, 400], 1, [0.2, 0.05], [{"tree", 3000}, {"bush", 5000},
{"vine", 1250}, {"reed", 1000}, {"lily", 2000}, {"wildgrass", 25000}]]),
    "testa apple experiment" : Biome([[600, 500], 1.25, [0.1, 0], [{"tree", 600}]]),
    "grassland" : Biome([[10, 10], 1, [0.1, 0], [{"tree", 100}, {"lily", 100},
{"wildgrass", 1500}]]),
    "forest" : Biome([[10, 10], 1, [0.1, 0], [{"tree", 500}, {"bush", 500}, {"lily",
250}]]),
    "jungle" : Biome([[10, 10], 1.5, [0.15, 0], [{"tree", 500}, {"bush", 500}, {"vine",
50}, {"lily", 250}]]),
    "mountains" : Biome([[10, 10], 0.75, [0.15, 0.15], [{"tree", 50}, {"bush", 50},
{"vine", 5}, {"lily", 80}, {"wildgrass", 50}]]),
    "desert" : Biome([[10, 10], 2, [0.02, 0], [{"tree", 50}, {"bush", 50}, {"vine", 5},
{"lily", 20}, {"wildgrass", 10}]]),
    "dry tundra" : Biome([[10, 10], 0.2, [0.02, 0.5], [{"tree", 50}, {"bush", 50},
{"vine", 5}, {"wildgrass", 50}]]),
    "wet tundra" : Biome([[10, 10], 0.2, [0.2, 0.5], [{"tree", 50}, {"bush", 50}, {"vine",
5}, {"wildgrass", 50}]]),
    "volcano" : Biome([[10, 10], 2, [0.02, 0], [{"tree", 50}, {"bush", 50}, {"vine", 5},
{"lily", 20}, {"wildgrass", 10}, {"volcano", 1}]]),
    "ocean" : Biome([[10, 10], 0.5, [0.9, 0.05], []]),
}
```


CELL.PY

```

from math import sqrt
from angle import atan2
from world_facts import terrain_index

#anima
SEARCH_RANGE_LIMIT = 80

class Cell:
    def __init__(self, realm, spawnx, spawny):
        self.realm = realm
        self.x = spawnx
        self.y = spawny
        self.bucket = -1

    def distance_x(self, x):
        rw = self.realm.width

        if self.x - x > rw/2:
            sx = self.x - rw
        elif x - self.x > rw/2:
            sx = self.x + rw
        else:
            sx = self.x
        return abs(sx - x)

    def distance_y(self, y):
        rh = self.realm.height

        if self.y - y > rh/2:
            sy = self.y - rh
        elif y - self.y > rh/2:
            sy = self.y + rh
        else:
            sy = self.y

        return abs(sy-y)

    def displacement_x(self, x):
        rw = self.realm.width

        if self.x - x > rw/2:
            sx = self.x - rw
        elif x - self.x > rw/2:
            sx = self.x + rw
        else:
            sx = self.x
        return x - sx

    def displacement_y(self, y):

```

```

    rh = self.realm.height

    if self.y - y > rh/2:
        sy = self.y - rh
    elif y - self.y > rh/2:
        sy = self.y + rh
    else:
        sy = self.y

    return y - sy

def distance_to(self, object):
    if isinstance(object, Cell):
        return self.distance_to_point(object.x, object.y)
    if isinstance(object, tuple):
        return self.distance_to_point(object[0], object[1])
    return None

def distance_to_point(self, x, y):          #can be private
    return sqrt((self.displacement_x(x))**2 + (self.displacement_y(y))**2)

def distance_to_nearest_a(self, element):  #fail: return the limit of the search
(int)
    if isinstance(element, int):
        dist = 1
        terrain = terrain_index[element]
        while dist <= SEARCH_RANGE_LIMIT:
            for row in range((self.y - dist)%self.realm.height, (self.y + dist +
1)%self.realm.height):
                for col in range((self.x - dist)%self.realm.width, (self.x + dist +
1)%self.realm.width):
                    if self.distance_to((col, row)) <= dist and
self.realm.environment[row % self.realm.height][col % self.realm.width] == terrain:
                        return dist
                    dist += 1
        return SEARCH_RANGE_LIMIT

def distance_to_nearest_n(self, element):  #fail: return None
    if isinstance(element, int):
        dist = 1
        terrain = self.realm.terrain_index[element]
        while dist <= SEARCH_RANGE_LIMIT:
            for row in range((self.y - dist)%self.realm.height, (self.y + dist +
1)%self.realm.height):
                for col in range((self.x - dist)%self.realm.width, (self.x + dist +
1)%self.realm.width):
                    if self.distance_to((col, row)) <= dist and
self.realm.environment[row % self.realm.height][col % self.realm.width] == terrain:
                        return dist
                    dist += 1
        return None

```

```

        """instead:for pos in self.cells_to_check:
            if self.realm.environment[pos[0]][pos[1]] == "water":
                dist = self.distance_to_point(pos[1], pos[0])
                if min(dist, nearest_dist, self.vision) == dist:    #vision for emergency
clamping
                    nearest_dist = dist
                    nearest_water = (pos[0],pos[1])"""

def is_within(self, object, radius):
    if isinstance(object, Cell):
        if self.distance_to(object) <= radius:
            return True
        return False
    if isinstance(object, tuple):
        if self.is_within_point(object[0],object[1],radius):
            return True
        return False
    return None

def is_within_point(self, x, y, radius):
    if self.distance_to_point(x, y) <= radius:
        return True
    return False

def angle_to(self, object):
    if isinstance(object, Cell):
        return atan2(self.displacement_y(object.y), self.displacement_x(object.x))
    if isinstance(object, tuple):
        return atan2(self.displacement_y(object[1]), self.displacement_x(object[0]))

def move(self, dx, dy):
    self.x += dx
    self.y += dy

"""def provide(self, _=None):
    return 0 or "ERROR"""

def reproduce(self):
    pass

def end(self):
    del self

```

EXTRATRAITS.PY

```
import random

class ExtraTraits():
    def __init__(self, trait_set):
        self.traits = trait_set

    def __iter__(self):
        return iter(self.traits)

    def __contains__(self, trait):
        return trait in self.traits

    def __len__(self):
        return len(self.traits)

    def __str__(self):
        return str(self.traits)

    def __bool__(self):
        return bool(self.traits)

    def add(self, trait):
        self.traits.add(trait)

    def extend(self, traits):
        self.traits = set(list(self.traits) + list(traits)) #could also use union but this
ensures works with both list and set of traits

    def correct_traits(self):
        attacks = list(self.traits & attack_set)

        while len(attacks) > 1:
            cull = random.choice(attacks)
            self.traits.remove(cull)
            attacks.remove(cull)

    def attack(self):
        return [attack for attack in attack_set if attack in self.traits]

attack_set = {"claws", "pincers", "venomous", "sand-attack", "mimic-attack"}

trait_costs = {
    "egg-laying" : 0,
    "claws" : 0.2,
    "flippers" : 0.2,
    "abrupt-wanderer" : 0,
    "poisonous" : 0.1,
    "venomous" : 0.2,
```

```
"pincers" : 0.2,  
"snowcloak" : 0,  
"strong-gut" : 0.1,  
"sand-attack" : 0.2,  
"spiritful-child" : 0.1,  
"inky-shot" : 0.2,  
"berserker" : 0.05,  
"vampirism" : 0.1,  
"zombie" : 0,  
"gluttonous-child" : 0,  
"necro-mimicry" : 0,  
"absorb-victim" : 0.15,  
"mimic-attack" : 0.15,  
"leviathan" : 0,  
"play-dead" : 0,  
"plate-armour" : 0.4,  
"growth-syndrome" : 0,  
}
```

FLORA.PY

```

from cell import Cell
import fruit
import random
from math import sqrt

class Flora(Cell):
    def __init__(self, id, genotype, realm, spawnx, spawny, fruit=fruit.apple, fruitNo =
0):
        super().__init__(realm, spawnx, spawny)
        self.id = id
        self.realm = realm
        self.fruit = fruit
        self.capacity = genotype[0]
        self.batch_size = genotype[1]
        self.rate = genotype[2] * self.realm.temperature / max(1, min(10,
sqrt(self.distance_to_nearest_a(1))))
        self.fruitNo = fruitNo

        self.food_given = self.fruit.name
        self.food_effect = self.fruit.statusChange

    def bear(self, amount = 1):
        self.fruitNo += amount

    def provide(self, quantity = 1):
        quantity_provided = min(quantity, self.fruitNo)
        self.fruitNo -= quantity_provided
        return quantity_provided

    def tick(self):
        if random.uniform(0,1) < self.rate and self.fruitNo < self.capacity:
            self.bear(self.batch_size)

flora_species = {
    "tree" : [["grass"], 4, 3, 0.01, fruit.apple, 100],
    "bush" : [["grass"], 10, 4, 0.025, fruit.berry, 125],
    "vine" : [["grass", "water"], 1, 1, 0.0025, fruit.melon, 15],
    "reed" : [["water"], 2, 2, 0.02, fruit.lavendar, 40],
    "lily" : [["water"], 1, 1, 0.05, fruit.lily, 100],
    "wildgrass" : [["grass"], 1, 1, 0.01, fruit.wildgrass, 400],

    "beanstalk" : [["grass", "tree"], 1, 1, 1, fruit.bean, 3],
    #magical beanstalk rarely spawns magic bean giving mutations
}

#add plants with multiple different fruit each?

#add effects to plants, ie poison

```


FRUIT.PY

```
class Fruit():
    def __init__(self, name = "apple", statusChange = [-160, 0, 0, -80, 80]):
        self.name = name
        self.statusChange = statusChange

apple = Fruit()
lily = Fruit("lily", [-200, 0, 0, -100, 100])
lavendar = Fruit("lavendar", [-50, 0, 50, -250, 10])
melon = Fruit("melon", [-1000, -100, 0, 0, 100])
berry = Fruit("berry", [-50, 0, 0, 30])
wildgrass = Fruit("wildgrass", [-50, 0, 0, 0, 0])

bean = Fruit("bean", [-4000, -4000, -4000, -4000, 4000])

#capitalise fruit names??? gone :(
```

MAIN.PY

```

import pygame
import time, random

from simulation import Simulation
from realm import Realm
import cell
from fruit import *
from flora import Flora
from anima import Anima, food_list
from extratraits import ExtraTraits
import nickname
import realm_settings

from todisplay import *
import ui

DATABASE_SAMPLE_INTERVAL = 25 #10

#what happens during collisions? when two animas share same cell centre?

#ALWAYS REMEMBER TO ADD TRAILING COMMA FOR SINGLE ELEMENT TUPLE

def main():
    pygame.init()
    clock = pygame.time.Clock()
    screen = pygame.display.set_mode((1280,720))
    pygame.display.set_caption("EvoSim - setup")
    homeShell = ui.Shell("home")
    homeShell.track_index = 3
    ui.play_music(homeShell)

    while homeShell.running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                quit()
            ui.handle_home_ui(event, homeShell)

        clock.tick(5)

        screen.fill(ui.HOME_BACKGROUND_COLOUR)
        ui.tick_home_ui(screen, homeShell)
        pygame.display.flip()

    ui.myShell.music_volume = homeShell.music_volume

    mySim = Simulation()

    test_element_list = []
    #for i in range(20):

```

```

# test_element_list.append(Flora(i, random.uniform(-960, 960) , random.uniform(-
540, 540), 10, 0.1)) code expired

"""for i in range(10):
    test_element_list.append(Anima(i, "name/diet/strength/toughness", [("apple",),
[random.uniform(1, 2),0.5,0.3], random.uniform(15, 30), random.uniform(5,10),
random.uniform(5,15), random.uniform(5,15), random.uniform(20,30), random.uniform(0, 1),
random.uniform(30,100), random.uniform(40,120)], None, random.uniform(0, 99) ,
random.uniform(0, 99)))
    test_element_list.append(Anima(i, "name/diet/strength/toughness", [("berry",),
[random.uniform(1, 2),0.5,0.3], random.uniform(15, 30), random.uniform(5,10),
random.uniform(5,15), random.uniform(5,15), random.uniform(20,30), random.uniform(0, 1),
random.uniform(30,100), random.uniform(40,120)], None, random.uniform(0, 99) ,
random.uniform(0, 99)))
    for i in range(4):
        test_element_list.append(Anima(i, "name/diet/strength/toughness", [("meat",),
[random.uniform(1, 2),0.5,0.3], random.uniform(15, 30), random.uniform(5,10),
random.uniform(5,15), random.uniform(5,15), random.uniform(20,30), random.uniform(0, 1),
random.uniform(30,100), random.uniform(40,120)], None, random.uniform(0, 99) ,
random.uniform(0, 99)))
"""

#for i in range(400): #400
    #test_element_list.append(Anima(i, "name/diet/strength/toughness",
[(random.choice(food_list),), [random.uniform(1, 5),0.5,0.3], random.uniform(15, 30),
random.uniform(5,25), random.uniform(5,30), random.uniform(5,30), random.uniform(20,30),
random.uniform(0, 1), random.uniform(30,100), random.uniform(40,200),
random.uniform(5,50)], None, random.uniform(0, 600) , random.uniform(0, 400)))
    #test_element_list.append(Anima(i, "name/diet/strength/toughness", [("apple",),
[random.uniform(1, 5), [1,0.5,0.3]], random.uniform(15, 30), random.uniform(5,15),
random.uniform(4,10), random.uniform(4,10), random.uniform(20,30), random.uniform(0, 1),
random.uniform(30,100), random.uniform(40,200), random.uniform(5,50), []], None,
random.uniform(0, 300) , random.uniform(0, 300)))
    #test_element_list.append(Anima(i, "name/diet/strength/toughness", [("apple",),
[random.uniform(1, 5), [1,0.5,0.3]], random.uniform(15, 30), random.uniform(5,15),
random.uniform(4,10), random.uniform(4,10), random.uniform(30,80), random.uniform(0, 1),
random.uniform(30,100), random.uniform(40,200), random.uniform(5,50), ExtraTraits(set())],
None, random.uniform(0, 600) , random.uniform(0, 500)))
#for i in range(20): #20
    #test_element_list.append(Anima(i, "name/diet/strength/toughness", [("meat",),
[random.uniform(1, 8), [1,0.5,0.3]], random.uniform(15, 30), random.uniform(5,15),
random.uniform(4, 15), random.uniform(4,15), random.uniform(30,80), random.uniform(0, 1),
random.uniform(20,80), random.uniform(40,200), random.uniform(5,50), ExtraTraits(set())],
None, random.uniform(0, 600) , random.uniform(0, 500)))

test_element_list.extend(homeShell.realm_settings[7])

#homeShell.realm_settings[6] = [[flora[0], int(round(flora[1]*
homeShell.realm_scale_factor**2 * homeShell.realm_flora_factor))] for flora in
realm_settings.default_realm_settings[6]]
homeShell.realm_settings[6] =
realm_settings.generate_flora(homeShell.realm_flora_factor, homeShell.realm_scale_factor)

```

```

homeShell.realm_settings[7] =
realm_settings.generate_primordial_list(homeShell.realm_settings[1],
homeShell.realm_settings[2], int(round(realm_settings.herbivore_count *
homeShell.realm_scale_factor**2)), int(round(realm_settings.carnivore_count *
homeShell.realm_scale_factor**2)), int(round(realm_settings.any_count *
homeShell.realm_scale_factor**2)))
    print("Generating realm...")
    #testRealm = Realm(nickname.generate_nickname("realm"), 600, 400, 1, 0.2, 0.05,
[["tree", 3000], ["bush", 5000], ["vine", 1250], ["reed", 1000], ["lily", 2000],
["wildgrass", 25000]],test_element_list, "name/diet/strength/toughness") #birth new test
realm
    #name/genome
    #testRealm = Realm(nickname.generate_nickname("realm"), 300, 300, 1.25, 0.1, 0,
[["tree", 150]],test_element_list, "name/diet/extra_traits/strength/toughness") #birth new
test realm
    #testRealm = Realm(nickname.generate_nickname("realm"), 600, 500, 1.25, 0.1, 0,
[["tree", 600]],test_element_list, "name/diet/extra_traits/strength/toughness") #birth new
test realm
    testRealm = Realm(*homeShell.realm_settings)

    #lily 0/2000/3000
    mySim.add_realm(testRealm)

    """for element in testRealm.elements:
        print(dir(element))
    print(mySim.realms[0].elements)
    print(testRealm.elements)
    for element in testRealm.elements[0]:
        print(element.capacity, element.fruitNo, element.x, element.y, element.fruit,
element.id)"""

    #print(testRealm.environment)
    #print(len(testRealm.spatialhash.buckets))
    #print(testRealm.spatialhash.buckets)

    print(testRealm.check_water())
    print(testRealm.check_ice())

    print(testRealm.count_flora())
    print(testRealm.check_grass())

    #print(testRealm.environment)

    #simulation ui begins
    pygame.display.set_caption("EvoSim: " + testRealm.name)
    ui.play_music(ui.myShell)

    window = ((640-testRealm.width/2, 640+testRealm.width/2),(100, 100 +
testRealm.height)) #was 120, 120 + h for y
    """left = window[0][0]
    right = window[0][1]

```

```

top = window[1][0]
bottom = window[1][1]          COPY+PASTE""

#vis_realm = Visual_Realm(window, testRealm)

blit = ui.draw_world_blit(testRealm)

print("A new world arises!")

while ui.myShell.running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            ui.myShell.running = False
            ui.handle_ui(event, ui.myShell, window, testRealm)

    if not ui.myShell.paused:          #sim paused

        if testRealm.time % DATABASE_SAMPLE_INTERVAL == 0:
            testRealm.database.update()

        mySim.tick()

    clock.tick(5 * ui.myShell.sim_speed)

    screen.fill(ui.BACKGROUND_COLOUR)
    ui.tick_visual_realm(testRealm, screen, window, blit, ui.myShell)
    ui.tick_extra_ui(screen, window, ui.myShell)
    pygame.display.flip()

pygame.quit()

def dx(record):
    print(record + " : " + to_display(getattr(testRealm.database, record, "No such
record"))))

#rename to db?
print("Extra Traits " +
to_display(testRealm.database.calc_extra_traits(testRealm.elements[1])))
flood()
print(to_display(testRealm.database.anima_populations))
flood()
print(to_display(testRealm.database.death_causes))
flood()
print(to_display(testRealm.database.avg_genomes))
flood()
print(to_display(testRealm.database.diets))
flood()

```

```

print(to_display(testRealm.database.avg_lifespans))
flood()
display(testRealm.database.lifespan_quartiles)
flood()
display(testRealm.database.age_quartiles)
flood()
display(testRealm.database.calc_total_avg_lifespan())
flood()
display(testRealm.database.avg_statuses)
flood()
display(testRealm.database.offspring_count_distributions)
flood()
display([anima.diet for anima in testRealm.elements[1] if len(anima.diet) >= 2])
return 0

if __name__=="__main__":
    main()

#add carcasses to spatial hash and as class

"""for batch in range(400):
    for tick in range(25):
        #print(testRealm.time)
        testRealm.tick()
        vis_realm
        #print([[anima == anima.target.target]] for anima in
testRealm.elements[1] if anima.target != None])
        #print([[anima.distance_to(anima.target), anima.goals, anima.target,
(anima.vision+anima.speed+anima.target.speed>= anima.distance_to(anima.target))] for anima
in testRealm.elements[1] if isinstance(anima.target, Anima)])
        #print([[anima.round_status(),round(anima.x), round(anima.y),
round(anima.dx), round(anima.dy), anima.mode, anima.target, anima.goals] for anima in
testRealm.elements[1]]) #anima.status, anima.sig_genome(),anima.x, anima.y, anima.dx,
anima.dy, anima.mode, anima.target, anima.goals
        #print([flora.fruitNo for flora in testRealm.elements[0]])

        print(testRealm.count_anima())
        print(testRealm.time)
        testRealm.database.update()"""

#Use IntelliSense to learn about possible attributes.
# Hover to view descriptions of existing attributes.
# For more information, visit: https://go.microsoft.com/fwlink/?linkid=830387

```


NICKNAME.PY

```
import random

class Alias():
    def __init__(self, alias):
        self.alias = alias

    def get_alias(self):
        return self.alias

def generate_nickname(entity):

    titles = titles_index[entity]
    endings = endings_index[entity]

    nickname = ""
    if random.random() < 0.1:
        nickname += titles[random.randint(0,len(titles)-1)]
        nickname += " "

    first_type = random.choice([vowel_batch, consonant_batch])
    second_type = vowel_batch if first_type == consonant_batch else consonant_batch
    nickname_length = random.randint(2,3) #up to 5?
    for batch in range(nickname_length):
        if batch == 0 and random.random() < 0.01:
            nickname += "quen"
        else:
            nickname += first_type(batch)

            if batch == nickname_length-1 and batch != 0 and random.random() < 0.5:
                last_type = first_type
            else:
                nickname += second_type(batch)
                last_type = second_type

    if random.random() < 0.8:
        nickname += endings[0][random.randint(0,len(endings[0])-1)] if last_type == vowel_batch else endings[1][random.randint(0,len(endings[1])-1)]

    nickname_words = nickname.split()
    nickname = " ".join(word[0].upper() + word[1:] for word in nickname_words)
    return Alias(nickname)

def vowel_batch(batch_no):
    vowels = list("aeiou") * 2 + ["oo", "ou", "ui", "ao", "ee", "ei", "y"]
    return random.choice(vowels)

def consonant_batch(batch_no):
```

```

consonants = list("bcd fghjklmnp rstvwxyz") + ["th"] #qu
leaders = list("stgfbkcp")
followers = list("rlwh")
if batch_no >= 1:
    followers.extend("bsnt")

batch = random.choice(consonants)
if batch_no >= 1 and random.random() < 0.2 and len(batch) == 1:
    batch += batch
elif random.random() < 0.3 and batch[len(batch)-1] in leaders:
    batch += random.choice(followers)

return batch

titles_index = {
    "anima" : ["Mr", "Dr", "Ms", "Mrs", "Lil'", "Big", "Blue", "Tuku"],
    "realm" : ["The Great"]
}

endings_index = {
    "anima" : [[""], [""]],
    "realm" : ["topia", " Island", "'s World", " Land"], [" Island", "'s World", "
Land"]]
}

```

REALM_SETTINGS.PY

```

from realm import create_realm_name
from anima import Anima, green_food_list, food_list
from flora import flora_species
from extratraits import ExtraTraits
import random

width = 600
height = 500

herbivore_count = 380
carnivore_count = 30
any_count = 20

flora_factor = 25

#[["tree", 600], ["bush", 800], ["lily", 500]]

def generate_primordial_list(width, height, herbivore_count, carnivore_count, any_count):
    anima_list = []
    for i in range(herbivore_count):
        anima_list.append(Anima(i, "name/diet/strength/toughness",
[(random.choice(green_food_list),), [random.uniform(1, 5), [1,0.5,0.3]],
random.uniform(15, 30), random.uniform(5,15), random.uniform(4,10), random.uniform(4,10),
random.uniform(30,80), random.uniform(0, 1), random.uniform(30,100),
random.uniform(40,200), random.uniform(5,50), ExtraTraits(set())], None, random.uniform(0,
width) , random.uniform(0, height)))
    for i in range(carnivore_count):
        anima_list.append(Anima(i, "name/diet/strength/toughness", [("meat",),
[random.uniform(1, 8), [1,0.5,0.3]], random.uniform(15, 30), random.uniform(5,15),
random.uniform(4, 15), random.uniform(4,15), random.uniform(30,80), random.uniform(0, 1),
random.uniform(20,80), random.uniform(40,200), random.uniform(5,50), ExtraTraits(set())],
None, random.uniform(0, width) , random.uniform(0, height)))
    for i in range(any_count):
        anima_list.append(Anima(i, "name/diet/strength/toughness",
[(random.choice(food_list),), [random.uniform(1, 5), [1,0.5,0.3]], random.uniform(15, 30),
random.uniform(5,15), random.uniform(4,10), random.uniform(4,10), random.uniform(30,80),
random.uniform(0, 1), random.uniform(30,100), random.uniform(40,200),
random.uniform(5,50), ExtraTraits(set())], None, random.uniform(0, width) ,
random.uniform(0, height)))
    return anima_list

def generate_flora(batch_count, scale_factor):
    extra_batch = True if random.random() < batch_count // 1 else False
    batch_count = int(batch_count//1 + 2*int(extra_batch) - 1)
    flora_list = [["tree", 25]]
    for batch in range(batch_count):
        plant = random.choice(list(flora_species.keys()))
        n = flora_species[plant][5]

```

```
        flora_list.append([plant, int(round(random.gauss(n**scale_factor,
n**scale_factor/10))))])
    return flora_list

default_realm_settings = [
    create_realm_name(),
    width,
    height,
    1.0,
    0.15,
    0.02,
    generate_flora(flora_factor, 1),
    generate_primordial_list(width, height, herbivore_count, carnivore_count, any_count),
    "name/diet/extra_traits/strength/toughness",
]
```

REALM.PY

```

from world_facts import *
from biome import *
from flora import Flora, flora_species
from anima import Anima
from cell import Cell
from angle import *
from math import floor, ceil
from nickname import generate_nickname
from todisplay import to_display

from spatialhash import SpatialHash

from simulationdata import Realm_Database

import random

CONSOLE_ON = False

SPATIAL_HASH_CELL_SIZE = 20

MAX_WATER_GENERATION_ITERATION_COUNT = 999 #99
MAX_ICE_GENERATION_ITERATION_COUNT = 999
MAX_FLORA_GENERATION_ITERATION_COUNT = 999

WATER_MIN_RADIUS = 3
WATER_MAX_RADIUS = 15

ICE_MIN_RADIUS = 18
ICE_MAX_RADIUS = 55
MAX_ICE_CENTRE_Y = 60

POOL_ANGLE_GRADATION = 360

STARTING_FRUIT_RATIO = 0.25

class Realm:

    def __init__(self, name, width, height, temperature, wetness, iciness, floraness,
primordiallist, creature_naming_mode):
        self.time = 0
        self.name = name
        self.width = width
        self.height = height
        self.temperature = temperature
        self.wetness = wetness           #water coverage as a ratio to overall area
        self.iciness = iciness           #ice  ''
        self.floraness = floraness       #int list giving how many of each flora
species will be generated [[name1, population1], [name2, population2]]

```

```

        self.fruit_list = list(set([flora_species[species[0]][4] for species in
floranness]))

        self.elements = [[],[]]                #elements updated each realm
tick    [flora][anima]
        self.graveyard = [[],[]]
        self.ghostyard = [[],[]]
        #self.environment = [[],[]]            #elements not updated once simulation
starts    [water][ice]
        self.environment = []                  #stores terrain or flora at each x/y access by
environment[row][col] aka [y][x]
        self.spatialhash = SpatialHash(SPATIAL_HASH_CELL_SIZE, self.width, self.height)

        self.creature_naming_mode = creature_naming_mode

        self.database = Realm_Database(self)

        self.add_elements(primordialList)
        self.generate_world()

    def generate_world(self):
        self.environment = [["grass" for _ in range(self.width)] for _ in
range(self.height)]
        self.__generate_water()
        self.__generate_ice()    #should ice cannibalise water, or avoid in pool
generating??
        self.__generate_flora()

    def __generate_water(self):

        #self.environment[0] = [[False for _ in range(self.width)] for _0 in
range(self.height)]

        #water_limit = self.width * self.height * self.wetness
        print("Generating water...")
        iteration_count = 0
        while self.check_water() < self.wetness and iteration_count <
MAX_WATER_GENERATION_ITERATION_COUNT:
            cx = random.randint(0, self.width - 1) #floats?
            cy = random.randint(0, self.height - 1)

            pool_radius = random.randint(WATER_MIN_RADIUS, WATER_MAX_RADIUS)
            self.generate_pool(cx, cy, pool_radius, 1, "rounded")

            iteration_count += 1
            print(iteration_count)
            print("iteration^")

```



```

def __generate_ice(self):
    print("Generating ice...")
    iteration_count = 0
    while self.check_ice() < self.iceness and iteration_count <
MAX_ICE_GENERATION_ITERATION_COUNT:
        cx = random.randint(0, self.width - 1) #floats?
        cy = random.randint(0, MAX_ICE_CENTRE_Y)
        pool_radius = random.randint(ICE_MIN_RADIUS, ICE_MAX_RADIUS)
        self.generate_pool(cx, cy, pool_radius, 2, "flattened")

        iteration_count += 1
        print(iteration_count)
        print("iteration^")

def generate_pool(self, cx, cy, radius, type, shapetype):
    #using parametric polar curves to generate curved pools of varying shape

    angle_start = random.randint(0, POOL_ANGLE_GRADATION)
    r = radius
    pool_points = []
    maxr = 0
    velocity = 0

    #acceleration = random.uniform(-0.5, 0.5)
    for u_theta in range (POOL_ANGLE_GRADATION):
        normalising_factor = 2 * PI / POOL_ANGLE_GRADATION
        n_theta = u_theta * normalising_factor
        t = (angle_start + u_theta) * normalising_factor

        velocity += random.uniform(-0.15, 0.15)
        velocity *= 0.95

        if POOL_ANGLE_GRADATION - u_theta <= 0.1 * POOL_ANGLE_GRADATION:
            velocity += (radius - r)/5

        #if random.random() < 0.1:
            #acceleration = random.uniform(-0.5, 0.5)

    # velocity += acceleration

    if shapetype == "rounded":
        r = max(r + velocity, 1)
        #r = radius + 2.5 * sin(2 * t + angle_start * normalising_factor)
    elif shapetype == "flattened":
        r = max(r * cos(t) + random.uniform(-2, 2) * sin(t), 1)
    # -> collect points using rsintheta and rcostheta
    if r > maxr:
        maxr = r
    if shapetype == "rounded":
        pool_points.append((cx + r*cos(t), cy + r*sin(t)))
    elif shapetype == "flattened":

```

```

        pool_points.append((cx + r*cos(n_theta), cy + r*sin(n_theta)))
    #print(theta)
    for point in pool_points:
        #print(point)
        self.environment[int(point[1]) % self.height][int(point[0]) % self.width]
= terrain_index[type] #temporary code, need fill in possible solution: keep
choosing adjacent tile closest to next (polar?) function result until back to original
tile, then closed object can be easily filled in using edge of pool as boundaries for fill
in, for each row start filling in when reach one and stop when reach next, but need to
cover edge cases of pools that go off map, cant then use polarity, so how?
        #print((int(point[0]), int(point[1])))
    # -> draw inside polygon of points, checking for edge of world
    centre = Cell(self, cx, cy)
    maxr = int(maxr+1)
    if shapetype == "rounded":
        for row in range(cy-maxr, cy + maxr+1):
            for col in range(cx-maxr, cx + maxr+1):
                col = col % self.width
                row = row % self.height
                cell = Cell(self, col, row)
                arg = centre.angle_to(cell)
                u_theta = arg / normalising_factor
                p1 = pool_points[(floor(u_theta) - angle_start) %
POOL_ANGLE_GRADATION]
                p2 = pool_points[(floor(u_theta) - angle_start+ 1) %
POOL_ANGLE_GRADATION]
                d1 = centre.distance_to(p1)
                d2 = centre.distance_to(p2)
                lerpdist = d1 + (d2-d1) * (u_theta-floor(u_theta))
                if cell.is_within(centre, lerpdist + 1):
                    self.environment[row][col] = terrain_index[type]
                    #print((col,row))
    if shapetype == "flattened":
        for row in range(cy-maxr, cy + maxr+1):
            for col in range(cx-maxr, cx + maxr+1):
                if 0 <= col < self.width and 0 <= row < self.height:
                    cell = Cell(self, col, row)
                    arg = centre.angle_to(cell)
                    u_theta = arg / normalising_factor
                    p1 = pool_points[floor(u_theta)]
                    p2 = pool_points[ceil(u_theta % 2*PI)]
                    d1 = centre.distance_to(p1)
                    d2 = centre.distance_to(p2)
                    lerpdist = d1 + (d2-d1) * (u_theta-floor(u_theta))
                    if cell.is_within(centre, lerpdist + 0.5):
                        self.environment[row][col] = terrain_index[type]

    #fill in with terrain_index[type]

```

```

def __generate_flora(self):
    #species[0] is the name of the species
    #species[1] is the limit of the number of each species to be generated
    flora_id = self.count_flora() #id starts from 0
    for species in self.floraness:
        if species[0] in flora_species:
            for _ in range(int(species[1])):
                iteration_count = 0
                while iteration_count < MAX_FLORA_GENERATION_ITERATION_COUNT:
                    cx = random.randint(0, self.width - 1)
                    cy = random.randint(0, self.height - 1)
                    flo = flora_species[species[0]]

                    if self.environment[cy][cx] in flo[0]:

                        starting_fruit_amount = int(flo[1] * STARTING_FRUIT_RATIO)
                        if random.random() < (flo[1] * STARTING_FRUIT_RATIO) % 1:
                            starting_fruit_amount += 1
                        bud = Flora(flora_id, [flo[1], flo[2], flo[3]], self, cx, cy,
flo[4], starting_fruit_amount)    #could make eg bushes cluster together, bush dwellers,
vbush camouflage, sim. give animas foresight to see which terrain they will step into,
weightings toward preferred terrains, and camo in certain terrains, simulates
stealth/ambush

                        self.elements[0].append(bud)
                        self.environment[cy][cx] = species[0] +
str(flora_id)                    #does this mess too much with terrain
effects, ie amphibian factor in water, discrete object of whole tile taken up by plant...
                        self.spatialhash.add_element(bud)

                        flora_id += 1
                        iteration_count = MAX_FLORA_GENERATION_ITERATION_COUNT

                    iteration_count += 1

def realm_report(self):
    return 0

def status(self):
    return [self.check_water(), self.check_ice(), self.elements, self.graveyard]

def check_grass(self):
    return self.__check_nat_element(0)

def check_water(self):
    return self.__check_nat_element(1)

def check_ice(self):
    return self.__check_nat_element(2)

def __check_nat_element(self, nat_element):
    count = 0

```

```

    for row in self.environment:
        for tile in row:
            count += 1 if tile == terrain_index[nat_element] else 0
    ratio = count/(self.width * self.height)
    return ratio

def __count_nat_element(self, nat_element):
    count = 0
    for row in self.environment:
        for tile in row:
            count += 1 if tile == terrain_index[nat_element] else 0
    return count

def count_flora(self):
    """count = 0
    for element in [entity for group in self.elements for entity in group]:
        if isinstance(element, Flora):
            count += 1"""
    return len(self.elements[0])

def flora_id(self):
    return self.count_flora() + len(self.graveyard[0])

def count_anima(self):
    """count = 0
    for element in [entity for group in self.elements for entity in group]:
        if isinstance(element, Anima):
            count += 1"""
    return len(self.elements[1])

def anima_id(self):
    return self.count_anima() + len(self.graveyard[1]) + len(self.ghostyard[1])

def add_element(self, element):
    if isinstance(element, Flora):
        self.elements[0].append(element)
    elif isinstance(element, Anima):
        self.elements[1].append(element)

    self.database.append_groups_by_diet(element)
    self.spatialhash.add_element(element)
    element.realm = self

def add_elements(self, elementListToAdd):
    for element in elementListToAdd:
        self.add_element(element)

def bury_element(self, element):
    #self.spatialhash.bury_element(element)      maybe spatial hash no longer has to
perform checks?

```

```

    if isinstance(element, Flora):
        self.elements[0].remove(element)
        self.graveyard[0].append(element)
    elif isinstance(element, Anima):
        self.elements[1].remove(element)
        self.graveyard[1].append(element)

    self.database.remove_groups_by_diet(element)

def ascend_element(self, element):
    self.spatialhash.remove_element(element)
    if isinstance(element, Flora):
        self.graveyard[0].remove(element)
        self.ghostyard[0].append(element)
    elif isinstance(element, Anima):
        self.graveyard[1].remove(element)
        self.ghostyard[1].append(element)

def bury_elements(self, element_list):
    for element in element_list:
        self.bury_element(element)

def ascend_elements(self, element_list):
    for element in element_list:
        self.ascend_element(element)

def tick(self):
    #increment time for all elements
    for flora in self.elements[0]:
        flora.tick()
    for anima in self.elements[1]:
        anima.tickThink()
    for anima in self.elements[1]:
        anima.tickAct()
        if CONSOLE_ON == True:
            pass
    for anima in self.elements[1]:
        anima.tickRespond()

    for anima in self.graveyard[1]:
        anima.tickDecay()

    self.spatialhash.tock(self.elements[1])

    for anima in self.elements[1]:
        if anima.is_dead():

            #trigger death lines
            self.bury_element(anima)
            #anima.end()
            poison_obituary = ""

```

```

        if anima.is_poisoned():
            poison_obituary = " while poisoned"
            print(f"{anima.nickname} died of {anima.autopsy()} aged
{to_display(anima.age)}{poison_obituary}. Its final status was {to_display(anima.status)}
and it was the parent of {[offspring.nickname for offspring in anima.offspring]}")

        self.database.increment_death_causes(anima.autopsy_index())

    for anima in self.graveyard[1]:
        if anima.food_chunks == 0:
            print(f"{anima.nickname}'s body has fully decayed.")
            self.ascend_element(anima)
            #move to heaven list and remove from spatial hash

    self.time += 1

def create_realm_name():
    return generate_nickname("realm").get_alias()

def construct_from_biome(biome, name, width, height, primordialList=None):
    if primordialList == ["default"]:
        primordialList = [creatures[creature] for creature in biome.native_fauna] ***

    return Realm(name, width, height, temperature = biome.temperature, wetness =
biome.wetness, iceness = biome.iceness, floranness = biome.SCALEBYAREA)

#(self, name, width, height, temperature, wetness, iceness, floranness, primordialList,
creature_naming_mode):

"""console_action_index = {          #leave for now due to eat complications, also anima
needs a reference- class vs instance methods
    Anima.do_nothing : None,
    Anima.eat : None,
    Anima.drink : f"{anima.nickname} drank...",
    Anima.sleep : f"Shhh! {self.nickname} is sleeping...",
}"""

```


SIMULATION.PY

```
class Simulation:

    def __init__(self):
        self.realms = []
        self.vis_realms = []
        self.time = 0

    def add_realm(self, newRealm):
        self.realms.append(newRealm)

    def add_vis_realm(self, newRealm):
        self.vis_realms.append(newRealm)

    def tick(self):
        #increment time for all realms

        for realm in self.realms:
            realm.tick()
        for vis_realm in self.vis_realms:
            vis_realm.tick()

        self.time += 1

    def tock(self, realmName):
        #increment time for a specific world
        for realm in self.realms:
            if realm.name == realmName:
                realm.tick()
        for v in self.vis_realms:
            if v.realm.name == realmName:
                v.tick()
```

SIMULATIONDATA.PY

```

import statistics
from todisplay import to_display
from anima import food_list, extra_traits

class Realm_Database():
    def __init__(self, realm):
        self.realm = realm
        self.groups_by_diet = [[],[],[[]]
        self.anima_id = -1
        self.ghost_id = -1

        self.death_causes = [0,0,0,0,0]

        self.anima_populations = []
        self.avg_genomes = []
        self.diets = []
        self.avg_lifespans = []
        self.lifespan_quartiles = []
        self.avg_ages = []
        self.age_quartiles = []

        self.avg_statuses = []
        self.needs = []
        self.fruits = []
        self.fruits_per_creature = []
        self.offspring_count_distributions = []
        self.ancestries = []          #use parent lists to find ancestors/common ancestors
and track descendants
        self.geomaps = []

    def append_groups_by_diet(self, anima):
        diet_types = ["herbivore", "carnivore", "omnivore"]
        self.groups_by_diet[diet_types.index(anima.diet_type())].append(anima)

    def remove_groups_by_diet(self, corpse):
        diet_types = ["herbivore", "carnivore", "omnivore"]
        self.groups_by_diet[diet_types.index(corpse.diet_type())].remove(corpse)

    def update_groups_by_diet(self):

        diet_types = ["herbivore", "carnivore", "omnivore"]

        """
        if len(self.realm.elements[1]) > self.anima_id - len(self.realm.graveyard[1]) -
len(self.realm.ghostyard[1]) + 1:
            for anima in self.realm.elements[1][self.anima_id -
len(self.realm.graveyard[1]) - len(self.realm.ghostyard[1]) + 1 :]: #append new animas
only
                self.groups_by_diet[diet_types.index(anima.diet_type())].append(anima)

```

```

        self.anima_id = self.realm.anima_id() - 1    #-1 then +1 later, so new anima is
first evaluated
        """

        #only remove when in ghostyard, so corpses still count for data collection (it is
otherwise hard to coordinate both graveyard and ghostyard)

        """
        if len(self.realm.ghostyard[1]) > self.ghost_id + 1:
            for dead_anima in self.realm.ghostyard[1][self.ghost_id + 1:]:
                group = self.groups_by_diet[diet_types.index(dead_anima.diet_type())]

                if dead_anima not in group:
                    print("ERROR: dead_anima missing:", dead_anima)
                    print("Diet:", dead_anima.diet_type())
                    print("Group contains:", group)
                    print("ghost_id:", self.ghost_id)
                    print("anima_id:", self.anima_id)

                self.groups_by_diet[diet_types.index(dead_anima.diet_type())].remove(dead_
anima)
            self.ghost_id = len(self.realm.ghostyard[1]) - 1
        """

def increment_death_causes(self, cause_index):
    self.death_causes[cause_index] += 1

def calc_avg_status(self, anima_list):
    avg_status = []
    status_list = [anima.status for anima in anima_list]
    if status_list:
        for i in range(len(status_list[0])):
            avg_status.append(get_avg([max(status[i], 0) for status in
status_list]))    #min value is always 0 here... for future use, let this be known
        return avg_status

def calc_avg_genome(self, anima_list):
    avg_genome = []
    genome_list = [anima.genome for anima in anima_list]
    if genome_list:
        for i in range(len(genome_list[0])):
            avg_genome.append(get_avg([genome[i] for genome in genome_list]))
        return avg_genome

def calc_genome_quartiles(self, anima_list):
    avg_genome = []
    genome_list = [anima.genome for anima in anima_list]

    #should put all this into my own quartile function with type case
    if len(genome_list) >= 2:

```

```

        for i in range(len(genome_list[0])):
            if isinstance(genome_list[0][i], (int, float)):
                avg_genome.append(statistics.quantiles([genome[i] for genome in
genome_list]))
            else:
                avg_genome.append("D/V")    #meaning discrete value
        return avg_genome
    else:
        return None

def calc_diets(self, anima_list):
    diet_counter = [[food, 0] for food in food_list]
    diet_list = [anima.diet for anima in anima_list]

    if diet_list:
        for diet in diet_list:
            for food in diet:
                for item in diet_counter:
                    if food == item[0]:
                        item[1] += 1
    return diet_counter

def calc_extra_traits(self, anima_list):
    trait_counter = [[trait, 0] for trait in extra_traits]
    traits_list = [anima.extra_traits for anima in anima_list]

    if traits_list:
        for traits in traits_list:
            for trait in traits:
                for item in trait_counter:
                    if trait == item[0]:
                        item[1] += 1
    return trait_counter

def calc_total_avg_lifespan(self):
    dead_ones = self.realm.graveyard[1] + self.realm.ghostyard[1]
    if dead_ones:
        return mean([anima.age for anima in dead_ones])
    else:
        return None

def calc_avg_lifespan(self):
    dead_ones = self.realm.graveyard[1] +
self.realm.ghostyard[1][len(self.realm.ghostyard[1])-10 : ]
    if dead_ones:
        return mean([anima.age for anima in dead_ones])
    else:
        return None

def calc_lifespan_quantiles(self):

```

```

        dead_ones = self.realm.graveyard[1] +
self.realm.ghostyard[1][len(self.realm.ghostyard[1])-10 : ]
        if len(dead_ones) >= 2:
            return statistics.quantiles([anima.age for anima in dead_ones])
        else:
            return None

    def calc_avg_age(self, anima_list):
        if anima_list:
            return mean([anima.age for anima in anima_list])

    def calc_age_quartiles(self, anima_list):
        if len(anima_list) >= 2:
            return statistics.quantiles([anima.age for anima in anima_list])
        else:
            return None

    def calc_offspring_count_distribution(self, anima_list):

        offspring_count_list = [len(anima.offspring) for anima in anima_list]

        if offspring_count_list:
            offspring_count_counter = [[i, 0] for i in range(max(offspring_count_list) +
1)]

            for offspring_count in offspring_count_list:
                offspring_count_counter[offspring_count][1] += 1
            return offspring_count_counter
        return None

    def update(self):
        time = self.realm.time
        #self.update_groups_by_diet()
        groups_by_diet = self.groups_by_diet

        self.anima_populations.append((time, len(groups_by_diet[0]),
len(groups_by_diet[1])))
        self.avg_genomes.append((time, self.calc_avg_genome(groups_by_diet[0]),
self.calc_avg_genome(groups_by_diet[1])))
        self.diets.append((time, self.calc_diets(self.realm.elements[1])))
        self.avg_lifespans.append((time, self.calc_avg_lifespan())) #,
self.calc_avg_lifespan(groups_by_diet[1])
        self.lifespan_quartiles.append((time, self.calc_lifespan_quartiles())) #,
self.calc_lifespan_quartiles(groups_by_diet[1])
        self.avg_ages.append((time, self.calc_avg_age(self.realm.elements[1])))
        self.age_quartiles.append((time, self.calc_age_quartiles(self.realm.elements[1])))

        self.avg_statuses.append((time, self.calc_avg_status(groups_by_diet[0]),
self.calc_avg_status(groups_by_diet[1])))
        self.offspring_count_distributions.append((time,
self.calc_offspring_count_distribution(groups_by_diet[0]),
self.calc_offspring_count_distribution(groups_by_diet[1])))

```

```

def get_avg_genome(self):
    return to_display(self.calc_avg_genome(self.realm.elements[1]))

def get_diets(self):
    return to_display(self.calc_diets(self.realm.elements[1]))

def get_avg(genotype_list):          #should be xs maybe

    if isinstance(genotype_list[0], (int, float)):
        avg = mean(genotype_list)

    elif isinstance(genotype_list[0], (list)):
        return [get_avg([genotype[g] for genotype in genotype_list]) for g in
range(len(genotype_list[0]))]

    elif isinstance(genotype_list[0], (tuple)):
        return mode(genotype_list)

    else:
        avg = None

    return avg

def mean(xs):
    return sum(xs)/len(xs)

def mode(xs):
    return statistics.mode(xs)

class Sim_Database():
    def __init__(self, simulation):
        self.sim = simulation
        self.table = []

    def add_realms(self):
        for realm in self.sim.realms:
            if realm not in self.table:
                self.add_realms(realm)

    def add_realms(self, realm):
        self.table.append(realm)

```


SPATIALHASH.PY

```
class SpatialHash:
    def __init__(self, cell_size, world_width, world_height):          #called world. not
realm
        self.width = int((world_width-1) / cell_size) + 1
        self.height = int((world_height-1) / cell_size) + 1
        self.unit = cell_size
        self.buckets = [[] for _0 in range(self.width * self.height)]

    def __get_hash(self, x, y):
        return int(x//self.unit) + int((y//self.unit)*self.width)

    def get_neighbours(self, x, y, radius):          #WRAP AROUND REALM??
        r = int((radius-0.000001)/self.unit) + 1

        my_bucket = self.__get_hash(x, y)
        neighbour_list = []
        for i in range(-r, r+1):
            for j in range(-r, r+1):
                bucket_to_check = ((my_bucket%self.width + i)%self.width) +
((int(my_bucket/self.width)+j) % self.height) * self.width
                #if bucket_to_check >= 0 and bucket_to_check < len(self.buckets) and not
(i== -1 and my_bucket % self.width == 0) and not (i==1 and (my_bucket+1) % self.width ==
0):
                    neighbour_list.extend(self.buckets[bucket_to_check])
        return neighbour_list

    def add_element(self, element):
        bucket = self.__get_hash(element.x, element.y)
        self.buckets[bucket].append(element)
        element.bucket = bucket

    def remove_element(self, element):
        bucket = element.bucket
        #print(element.bucket)
        #print([b for b in range(self.width * self.height) if element in self.buckets[b]])
        self.buckets[bucket].remove(element)

    def update_element(self, element):
        if element.bucket == -1:
            self.add_element(element)
        #skip exception for flora?
        else:
            current_b = self.__get_hash(element.x, element.y)
            prev_b = element.bucket
            if current_b != prev_b:
                if prev_b >= 0:
                    self.buckets[prev_b].remove(element)
                self.buckets[current_b].append(element)
                element.bucket = current_b
```

```
#tick

def tock(self, element_list):
    for element in element_list:
        self.update_element(element)
```

TODISPLAY.PY

```
#the medium
from extratraits import ExtraTraits

def round_list(xs):
    return [round(item) for item in xs]

def sig(n, form=None):

    if isinstance(n, str):
        return n

    if isinstance(n, int):
        if form:
            fn = str(n)
            while n < 10 ** (form-1):
                n *= 10
            fn = str(0) + fn
            return fn
        return str(n)

    if isinstance(n, (float)):
        if n < 1:
            n = round(n*100)/100
        elif n < 1000:
            n = round(n*10)/10
        elif n < 10000:
            n = str(round(n/10)/100) + 'k'
        else:
            n = str((round(n/100))/10) + 'k'
        return str(n)

    if isinstance(n, ExtraTraits):
        if not n:
            return "{}"
        else:
            return str(n)

    if isinstance(n, type(None)):
        return "N/A"

    return "type_error : " + str(type(n))

def sig_list(xs):    #not used?
    return [sig(item) for item in xs]

def to_display(data, form=None):
    if data == "flood":
        return "/" * (88 + 128)
    elif isinstance(data, list) or isinstance(data, tuple):
```

```
        return "[" + "|".join([to_display(datum) for datum in data]) + "], "  
    else:  
        return sig(data, form)  
  
def display(data):  
    print(to_display(data))  
  
def flood():  
    display("flood")
```

UI_DESIGN.PY

```

import pygame
import sys
from ui_element import *
from ui_sim_plane import * #for colours in tutorial
from ui_terminal import Terminal
from ui_factfile import genome_view

from realm_settings import default_realm_settings, create_realm_name
import realm_settings

pygame.init()

font = pygame.font.SysFont(None, 24)
home_font = pygame.font.SysFont(None, 28)
big_font = pygame.font.SysFont(None, 40)
terminal_font = pygame.font.SysFont("consolas", 10)

terminal = Terminal()
sys.stdout = terminal

HOME_BACKGROUND_COLOUR = "blue"
BACKGROUND_COLOUR = "white"
GENERAL_TEXT_COLOUR = "royalblue1"
TRACK_TEXT_COLOUR = "pink"

class Shell():
    def __init__(self, category = "sim"):
        if category == "home":
            self.sfx_volume = 0.5
            self.music_volume = 0.5
            self.sfx_muted = False
            self.music_muted = False

            self.track_index = 0

            self.running = True

            self.brightness = 1

            self.realm_settings = default_realm_settings[:]
            self.realm_scale_factor = 1
            self.realm_flora_factor = realm_settings.flora_factor

        else:
            self.sfx_volume = 0.5
            self.music_volume = 0.5
            self.sfx_muted = False
            self.music_muted = False

```

```

        self.track_index = 0

        self.ui_visible = True
        self.sim_speed = 1
        self.paused = False
        self.running = True
        self.brightness = 1

        self.tutorial_index = 0
        self.ctutorial_index = 0

        self.anima_to_view = None

myShell = Shell()

#HOME

hmusic_up = Button((250,70,30,30), "+", category="home")
hmusic_down = Button((290,70,30,30), "-", category="home")
hmusic_mute = Button((1000,70,65,30), "Mute", category="home")
hmusic_unmute = Button((1000,120,80,30), "Unmute", category="home")

hchange_realm_name = Button((850,136,60,25), "New", category="home")

hsize_up = Button((813,193,25,25), "+", category="home")
hsize_down = Button((447,193,25,25), "-", category="home")
hsize_slider = Slider((485,200,310,10), (0.1,1), 1, category="home")

htemp_up = Button((813,293,25,25), "+", category="home")
htemp_down = Button((447,293,25,25), "-", category="home")
htemp_slider = Slider((485,300,310,10), (0,3), default_realm_settings[3], category="home")

hwet_up = Button((813,393,25,25), "+", category="home")
hwet_down = Button((447,393,25,25), "-", category="home")
hwet_slider = Slider((485,400,310,10), (0,1), default_realm_settings[4], category="home")

hflora_up = Button((813,493,25,25), "+", category="home")
hflora_down = Button((447,493,25,25), "-", category="home")
hflora_slider = Slider((485,500,310,10), (0,100), realm_settings.flora_factor,
category="home")

start = Button((540,600,200,45), "GENERATE WORLD", category="home")

#SFX
sfx_up = Button((20,20,30,30), "+")
sfx_down = Button((60,20,30,30), "-")
sfx_slider = Slider((100,25,150,10), (0, 1), myShell.sfx_volume)
sfx_mute = Button((260,20,60,30), "Mute")
sfx_unmute = Button((330,20,70,30), "Unmute")

```



```

#MUSIC
music_play = Button((20,70,80,30), "Play/Pause")
music_next = Button((110,70,60,30), "Next")
music_prev = Button((180,70,60,30), "Prev")

music_up = Button((250,70,30,30), "+")
music_down = Button((290,70,30,30), "-")
music_slider = Slider((330,75,150,10), (0, 1), myShell.music_volume)

music_mute = Button((490,70,60,30), "Mute")
music_unmute = Button((560,70,70,30), "Unmute")

#BRIGHTNESS / CONTRAST
brightness_slider = Slider((20,130,200,10), (0.2, 1.5), 1)
contrast_toggle = Button((230,120,120,30), "Hi-Contrast", toggleable=True)

#LANGUAGE
languages = ["EN", "CN", "TM"]
lang_buttons = [Button((20+i*60,170,50,30), lang) for i, lang in enumerate(languages)]

#UI
hide_ui = Button((20,210,100,30), "Hide UI")

#SIM CONTROL
speed_buttons = [Button((20+i*60,260,50,30), f"x{s}") for i,s in enumerate([1,2,4,8])]
pause_btn = Button((520,650,100,30), "Pause")
end_btn = Button((635,650,100,30), "End")

#TUTORIAL
tutorial_texts = [
    "Welcome to your very own realm! (Press the > icon)",
    "Each of those little circles moving around in your realm is an organism, also known as an 'anima'",
    "You can click on an anima to pull up a fact file giving information about it. Try it!",
    "Anima need to eat, drink, sleep and stay healthy. If they can do all of that, they might mate",
    "Otherwise, they'll die!",
    "The offspring of two anima will have traits which are a genetic combination of its parents'",
    "Let me give you an idea of what some of these traits might be:",
    "Larger organisms require more food but are better in fights and age more slowly*",
    "Vision is another trait which determines how far an organism can sense the other things in the simulation",
    "An organism's vision range is represented by how large its eye is",
    "Animas can develop unique extra traits such as a venomous bite, plate armour, or neverending growth",
    "All in all, animas have " + "14" + " genetic traits, PLUS any unique extra traits",
    "The fact file of an anima gives further stats including its complete genome and immediate family",

```

```

    "The first two organisms displayed in the 'Family' section are its parents, and up its
4 eldest children will be displayed underneath",
    "(Note: first generation organisms won't have parents, only children)",
    "You can control the music, sound effects, simulation speed, and more, using all the
buttons you can see around here...",
    "That's the basic tutorial! Keep reading for info that's a little more niche ->",
    "An anima will increase in size until they reach their genetically given maturity age
(given in the fact file in simulation ticks)",
    "An anima's age is given in the fact file in decaticks (unit of 10 ticks)",
    "An anima's size is the geometric mean of their strength (how hard they can hit) and
their toughness (how much they can take)",
    "An anima's needs have the following numbers corresponding to them:",
    "0: hunger, 1: thirst, 2: sleepiness; 3: damage, 4: mating drive",
    "These apply to both the goal numbers in an anima's goals, and the index of each
status in an anima's status list",
    "That's about it for now...",
    "Have fun, and happy evolving!",
]

colour_tutorial_texts = [
    "An anima's state can be observed from the colours of its body and eye. For instance,
an anima's eye turns gold when it sleeps*",
    "*This applies to the default colour set",
    "Default body colours: purple = poisoned, orange-red = hurt, red = wounded...",
    "...senescent = white, child = green, grey-blue = adult meat-eater, aquamarine = adult
non-meat-eater ",
    "Default eye colours: blue = wandering, purple = found food, light blue = found water,
lilac = searching for mate, black = hunting, white = fleeing",
    "You can't tell everything from just the colours though - click on an anima to pull up
a fact file giving you all the details!",
]

evo_fact_texts = [
    "Evolution is driven by natural selection!"
]

next_tut = Button((20,320,60,30), ">")
prev_tut = Button((90,320,60,30), "<")

next_ctut = Button((20,450,60,30), ">")
prev_ctut = Button((90,450,60,30), "<")

pygame.mixer.init()

tracks = ["peritune-folk-chinese(chosic.com).mp3", "00664805.mp3", "Alan Walker -
Faded.mp3", "Alan Walker - The Spectre.mp3"]

def play_music(shell):
    pygame.mixer.music.load(tracks[shell.track_index])

```

```

pygame.mixer.music.play(-1)

anima_selection_range = 8

def handle_ui(event, shell, sim_window, realm):
    if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        left = sim_window[0][0]
        right = sim_window[0][1]
        top = sim_window[1][0]
        bottom = sim_window[1][1]

        mx, my = pygame.mouse.get_pos()
        if left < mx < right and top < my < bottom:
            shell.anima_to_view = None
            genome_view.hide()

            nearest_anima = None
            nearest_dist = anima_selection_range

            for anima in realm.elements[1] + realm.graveyard[1]:
                dist = anima.distance_to_point(mx - left, my - top)
                if dist <= nearest_dist:
                    nearest_anima = anima
                    nearest_dist = dist
                    genome_view.show()
            shell.anima_to_view = nearest_anima

    for b in Button.button_list:
        b.handle_event(event)

    for b in lang_buttons + speed_buttons:
        b.handle_event(event)

    for s in Slider.slider_list:
        s.handle_event(event)

    #SFX
    if sfx_up.clicked: shell.sfx_volume = min(1, shell.sfx_volume+0.05);
    sfx_slider.set_status(shell.sfx_volume)
    if sfx_down.clicked: shell.sfx_volume = max(0, shell.sfx_volume-0.05);
    sfx_slider.set_status(shell.sfx_volume)
    if sfx_mute.clicked: shell.sfx_muted = True
    if sfx_unmute.clicked: shell.sfx_muted = False

    if sfx_slider.dragging: shell.sfx_volume = sfx_slider.status
    pygame.mixer.set_num_channels(8)
    for i in range(8):
        pygame.mixer.Channel(i).set_volume(0 if shell.sfx_muted else shell.sfx_volume)

```

```

#MUSIC
if music_play.clicked:
    if pygame.mixer.music.get_busy():
        pygame.mixer.music.pause()
    else:
        pygame.mixer.music.unpause()

if music_next.clicked:
    shell.track_index = (shell.track_index+1)%len(tracks)
    play_music(shell)

if music_prev.clicked:
    shell.track_index = (shell.track_index-1)%len(tracks)
    play_music(shell)

    if music_up.clicked: shell.music_volume = min(1, shell.music_volume+0.05);
music_slider.set_status(shell.music_volume)
    if music_down.clicked: shell.music_volume = max(0, shell.music_volume-0.05);
music_slider.set_status(shell.music_volume)
    if music_mute.clicked: shell.music_muted = True
    if music_unmute.clicked: shell.music_muted = False

if music_slider.dragging: shell.music_volume = music_slider.status
pygame.mixer.music.set_volume(0 if shell.music_muted else shell.music_volume)

#BRIGHTNESS
shell.brightness = brightness_slider.status

#LANGUAGE
for i,b in enumerate(lang_buttons):
    if b.clicked:
        print("Language:", languages[i])

#UI
if hide_ui.clicked:
    shell.ui_visible = not shell.ui_visible

#SIM
for i,b in enumerate(speed_buttons):
    if b.clicked:
        shell.sim_speed = [1,2,4,8][i]

if pause_btn.clicked:
    shell.paused = not shell.paused

if end_btn.clicked:
    shell.running = False

#TUTORIAL
if next_tut.clicked:
    shell.tutorial_index = min(len(tutorial_texts)-1, shell.tutorial_index+1)

```

```

if prev_tut.clicked:
    shell.tutorial_index = max(0, shell.tutorial_index-1)

if next_ctut.clicked:
    shell.ctutorial_index = min(len(colour_tutorial_texts)-1, shell.ctutorial_index+1)
if prev_ctut.clicked:
    shell.ctutorial_index = max(0, shell.ctutorial_index-1)

if genome_view.clicked:
    if not genome_view.status:
        genome_view.set_text("VIEW GENOME")
    else:
        genome_view.set_text("VIEW ANIMA")

def handle_home_ui(event, shell):
    for b in Button.home_button_list:
        b.handle_event(event)

    for s in Slider.home_slider_list:
        s.handle_event(event)

    if hmusic_up.clicked: shell.music_volume = min(1, shell.music_volume+0.05);
music_slider.set_status(shell.music_volume)
    if hmusic_down.clicked: shell.music_volume = max(0, shell.music_volume-0.05);
music_slider.set_status(shell.music_volume)
    if hmusic_mute.clicked: shell.music_muted = True
    if hmusic_unmute.clicked: shell.music_muted = False

    if hchange_realm_name.clicked : shell.realm_settings[0] = create_realm_name()

    if hsize_up.clicked : shell.realm_scale_factor = min(1, shell.realm_scale_factor +
0.05); hsize_slider.set_status(shell.realm_scale_factor)
    if hsize_down.clicked : shell.realm_scale_factor = max(0.1, shell.realm_scale_factor -
0.05); hsize_slider.set_status(shell.realm_scale_factor)
    if hsize_slider.dragging : shell.realm_scale_factor = hsize_slider.status

    shell.realm_settings[1] = int(default_realm_settings[1] * shell.realm_scale_factor)
    shell.realm_settings[2] = int(default_realm_settings[2] * shell.realm_scale_factor)

    if htemp_up.clicked: shell.realm_settings[3] = min(3, shell.realm_settings[3] + 0.05);
htemp_slider.set_status(shell.realm_settings[3])
    if htemp_down.clicked : shell.realm_settings[3] = max(0, shell.realm_settings[3] -
0.05); htemp_slider.set_status(shell.realm_settings[3])
    if htemp_slider.dragging : shell.realm_settings[3] = htemp_slider.status

    if hwet_up.clicked: shell.realm_settings[4] = min(1, shell.realm_settings[4] + 0.01);
hwet_slider.set_status(shell.realm_settings[4])
    if hwet_down.clicked : shell.realm_settings[4] = max(0, shell.realm_settings[4] -
0.01); hwet_slider.set_status(shell.realm_settings[4])
    if hwet_slider.dragging : shell.realm_settings[4] = hwet_slider.status

```

```
    if hflora_up.clicked: shell.realm_flora_factor = min(100, shell.realm_flora_factor +
1); hflora_slider.set_status(shell.realm_flora_factor)
    if hflora_down.clicked : shell.realm_flora_factor = max(0, shell.realm_flora_factor -
1); hflora_slider.set_status(shell.realm_flora_factor)
    if hflora_slider.dragging : shell.realm_flora_factor = hflora_slider.status

pygame.mixer.music.set_volume(0 if shell.music_muted else shell.music_volume)

if start.clicked: shell.running = False

return
```


UI_ELEMENT.PY

```
import pygame

class Button():
    button_list = []          #this is for sim buttons
    home_button_list = []    #for buttons on home screen / setup

    def __init__(self, rect, text="", toggleable=False, off_colour=(200,200,200),
hover_colour = (200,150,200), on_colour=(200,100,200), border_colour = "grey",
text_colour="white", category = "sim"):
        self.rect = pygame.Rect(rect)
        self.text = text
        self.toggleable = toggleable
        self.status = False
        self.hovered = False
        self.clicked = False
        self.off_colour = off_colour
        self.hover_colour = hover_colour
        self.on_colour = on_colour
        self.border_colour = border_colour
        self.text_colour = text_colour

        self.visible = True

        if category == "sim":
            Button.button_list.append(self)
        elif category == "home":
            Button.home_button_list.append(self)

    @classmethod
    def get_button_list(cls):
        return cls.button_list

    @classmethod
    def get_home_button_list(cls):
        return cls.home_button_list

    def set_rect(self, rect):
        self.rect = pygame.Rect(rect)

    def set_text(self, text):
        self.text = text

    def hide(self):
        self.visible = False

    def show(self):
        self.visible = True
```

```

def toggle_visibility(self):
    self.visible = not self.visible

def handle_event(self, event):
    self.clicked = False
    if event.type == pygame.MOUSEBUTTONDOWN:
        if self.rect.collidepoint(event.pos):
            self.clicked = True
            if self.toggleable:
                self.status = not self.status

def draw(self, screen, font):
    if self.status and (not self.rect.collidepoint(pygame.mouse.get_pos()) or
pygame.MOUSEBUTTONDOWN):
        colour = self.on_colour
    elif self.rect.collidepoint(pygame.mouse.get_pos()):
        colour = self.hover_colour
    else:
        colour = self.off_colour

    pygame.draw.rect(screen, colour, self.rect)
    pygame.draw.rect(screen, self.border_colour, self.rect, 2)

    if self.text:
        txt = font.render(self.text, True, self.text_colour)
        screen.blit(txt, txt.get_rect(center = self.rect.center))

class Slider():
    slider_list = []
    home_slider_list = []

    def __init__(self, rect, value_range=(0,100), default=0, slider_colour = (0,50,150),
knob_colour = (100,100,150), category = "sim"):
        self.rect = pygame.Rect(rect)
        self.min = value_range[0]
        self.max = value_range[1]
        self.status = max(self.min, min(self.max, default)) #status is int slider value
        self.dragging = False # is currently being dragged?
        self.slider_colour = slider_colour
        self.knob_colour = knob_colour

        self.visible = True

        if category == "sim":
            Slider.slider_list.append(self)
        elif category == "home":
            Slider.home_slider_list.append(self)

    @classmethod
    def get_slider_list(cls):
        return cls.slider_list

```

```

@classmethod
def get_home_slider_list(cls):
    return cls.home_slider_list

def hide(self):
    self.visible = False

def show(self):
    self.visible = True

def toggle_visibility(self):
    self.visible = not self.visible

def set_status(self, value):
    self.status = max(self.min, min(self.max, value))

def handle_event(self, event):
    if event.type == pygame.MOUSEBUTTONDOWN:
        if self.rect.collidepoint(event.pos):
            self.dragging = True

    elif event.type == pygame.MOUSEBUTTONUP:
        self.dragging = False

    elif event.type == pygame.MOUSEMOTION and self.dragging:
        self.status = self.min + max(0, min(1, (event.pos[0]-
self.rect.x)/self.rect.width)) * (self.max-self.min)

def draw(self, screen):
    pygame.draw.rect(screen, self.slider_colour, self.rect)
    pygame.draw.circle(screen, self.knob_colour, (self.rect.x + int(self.rect.width *
(self.status-self.min)/(self.max-self.min)), self.rect.centery), self.rect.height * 1.15)

```

UI_FACTFILE.PY

```

import pygame
from todisplay import *
from ui_sim_plane import draw_anima
from ui_element import Button, Slider
from flora import Flora

pygame.init()
rfont = pygame.font.SysFont(None, 18)
rcolour = "steelblue3"
row_size = 25

genome_view = Button((0,0,0,0), "GENOME VIEW", True, text_colour="green3")
genome_view.hide()

def draw_anima_record(anima, realm, screen):
    record = pygame.Rect((640 + realm.width/2 + 50, 90, 220, 500), width = 0,
border_radius = 0)
    pygame.draw.rect(screen, "wheat", record)

    txt = rfont.render(anima.get_alias(), True, rcolour)
    screen.blit(txt, txt.get_rect(center = (record.centerx, record.y+25)))

    txt = rfont.render("ID " + str(anima.id), True, rcolour)
    screen.blit(txt, txt.get_rect(center = (record.centerx, record.y+50)))

    txt = rfont.render("LOC : " + to_display((anima.x, anima.y)), True, rcolour)
    screen.blit(txt, txt.get_rect(center = (record.centerx, record.y+75)))

    txt = rfont.render("AGE : " + str(int(anima.age/10)), True, rcolour)
    screen.blit(txt, txt.get_rect(center = (record.centerx, record.y+100)))

    txt = rfont.render("STATUS : " + to_display(anima.status), True, rcolour)
    screen.blit(txt, txt.get_rect(center = (record.centerx, record.y+100 + row_size)))

    txt = rfont.render("GOALS : " + to_display(anima.goals), True, rcolour)
    screen.blit(txt, txt.get_rect(center = (record.centerx, record.y+100 + 2*row_size)))

    txt = rfont.render("FLEEING FROM " + anima.target.nickname, True, rcolour) if
anima.mode == 3 and hasattr(anima.target, "nickname") else rfont.render("APPROACHING " +
(anima.target.food_given + " PLANT AT " + str((anima.target.x, anima.target.y)) if
isinstance(anima.target, Flora) else anima.target.nickname if hasattr(anima.target,
"nickname") else "WATER AT " + str(anima.target) if isinstance(anima.target, tuple) else
"A GHOST"), True, rcolour) if not anima.target == None else rfont.render("SLEEPING", True,
rcolour) if anima.asleep else rfont.render("WANDERING", True, rcolour)
    screen.blit(txt, txt.get_rect(center = (record.centerx, record.y+100 + 3*row_size)))

    txt = rfont.render("FAMILY :", True, rcolour)
    screen.blit(txt, txt.get_rect(center = (record.centerx, record.y+100 + 4*row_size)))

```

```

#txt = rfont.render("GENOME : " + to_display(anima.genome), True, rcolour)
#screen.blit(txt, txt.get_rect(center = (record.centerx, record.y+100 + 4*row_size)))
#genome button triggers switch to genome view page ***
draw_genome_view_button(record)

if len(anima.parents) >= 1:
    draw_anima(anima.parents[0], screen, (record.centerx + record.x)/2, record.y+100 +
5*row_size)

    if len(anima.parents) >= 2:
        draw_anima(anima.parents[1], screen, (record.centerx + record.x+record.width)/2,
record.y+100 + 5*row_size)

    if len(anima.offspring) >= 1:
        draw_anima(anima.offspring[0], screen, (record.centerx + record.x)/2, record.y+100
+ 6*row_size)

    if len(anima.offspring) >= 2:
        draw_anima(anima.offspring[1], screen, (record.centerx + record.x+record.width)/2,
record.y+100 + 6*row_size)

    if len(anima.offspring) >= 3:
        draw_anima(anima.offspring[2], screen, (record.centerx + record.x)/2, record.y+100
+ 7*row_size)

    if len(anima.offspring) >= 4:
        draw_anima(anima.offspring[3], screen, (record.centerx + record.x+record.width)/2,
record.y+100 + 7*row_size)

#energy consumption
txt = rfont.render("DECEASED FROM " + to_display(anima.autopsy()).upper(), True,
rcolour) if anima.is_dead() else rfont.render("", True, rcolour)
screen.blit(txt, txt.get_rect(center = (record.centerx, record.y+100 + 9*row_size)))

txt = rfont.render("CNK : " + anima.nickname, True, rcolour)
screen.blit(txt, txt.get_rect(center = (record.centerx, record.y+100 + 11*row_size)))

#txt = rfont.render("GOALS : " + to_display(anima.goals), True, rcolour)
#screen.blit(txt, txt.get_rect(center = (record.centerx, record.y+100 + 12*row_size)))

#target and parents have sprites printed to display which can be clicked on to select
them ***

genotype_list = ["DIET", "SPEED", "TURNING SPEED", "VISION RANGE", "STRENGTH",
"TOUGHNESS", "MATING DRIVE", "BOOST FACTOR", "INJURY THRESHOLD", "MATURITY AGE",
"REFRACTORY", "EXTRA TRAITS"]

def draw_genome_record(anima, realm, screen):

```

```

genome = anima.genome
record = pygame.Rect((640 + realm.width/2 + 50, 90, 220, 500), width = 0,
border_radius = 0)
pygame.draw.rect(screen, "wheat", record)

txt = rfont.render(anima.get_alias(), True, rcolour)
screen.blit(txt, txt.get_rect(center = (record.centerx, record.y+25)))

for g in range(len(anima.genome)):
    display = to_display(genome[g])
    if g == 4:
        display += f"({to_display(anima.strength)})"
    elif g == 5:
        display += f"({to_display(anima.toughness)})"
    txt = rfont.render(genotype_list[g] + " : " + display, True, rcolour)
    screen.blit(txt, txt.get_rect(center = (record.centerx, record.y + 55 +
g*row_size)))

#genome button triggers switch back to anima view page ***
draw_genome_view_button(record)

def draw_genome_view_button(record):
    render = rfont.render(genome_view.text, True, (0,0,0))
    gvw, gvh = render.get_size()
    gvw += 40
    gvh += 20
    genome_view.set_rect((record.centerx-gvw/2, record.y+100 + 14*row_size - gvh/2, gvw,
gvh))

```


UI_SIM_PLANE.PY

```
import pygame
from math import sqrt

fruit_set = {
    "apple" : "red",
    "berry" : "blue",
    "lily" : "yellow",
    "lavendar" : "purple",
    "melon" : "darkolivegreen1",
    "wildgrass" : "green",
    "bean" : "forestgreen",
}

og_set = {
    "grass" : "green",
    "flora" : "purple",
    "fruit" : None,
    "child" : "yellow",
    "old" : "yellow",
    "corpse" : "orangered",
    "adult" : "yellow",
}

classic_set = {
    "grass" : "green",
    "flora" : "forestgreen",
    "fruit" : fruit_set,
    "water" : "blue",
    "child" : "turquoise1",
    "old" : "white",
    "corpse" : "red4",
    "adult" : "yellow",
    "meateater" : "slateblue3",
    "found_food" : "purple",
    "found_water" : "steelblue1",
    "roam" : "dodgerblue",
    "vision" : "yellow",
    "highlight" : "yellow",
}

mature_set = {
    "grass" : "darkolivegreen3",
    "flora" : "forestgreen",
    "water" : "dodgerblue",
    "fruit" : fruit_set,
    "child" : "green",
    "old" : "white",
    "corpse" : "red",
```

```

    "adult" : "aquamarine",
    "meateater" : "slateblue3",
    "found_food" : "purple",
    "found_water" : "steelblue1",
    "roam" : "blue",
    "vision" : "yellow",
    "highlight" : "yellow", #magenta
}

declutter_classic_set = {
    "grass" : "green",
    "flora" : "forestgreen",
    "fruit" : None,
    "child" : "turquoise1",
    "old" : "white",
    "corpse" : "red4",
    "adult" : "yellow",
}

declutter_mature_set = mature_set
declutter_mature_set["fruit_set"] = None

#olivegreen?

colour_set = classic_set

def set_sim_colours(colour_set):
    global GRASS_COLOUR, FLORA_COLOUR, WATER_COLOUR, CHILD_COLOUR, SENESCENT_COLOUR,
    CORPSE_COLOUR, MEAT_EATER_COLOUR, ADULT_COLOUR, FOUND_FOOD_COLOUR, FOUND_WATER_COLOUR,
    ROAM_COLOUR
    global VISION_COLOUR, HIGHLIGHT_COLOUR
    global FRUIT_COLOUR_SET

    GRASS_COLOUR = colour_set["grass"]
    FLORA_COLOUR = colour_set["flora"]
    WATER_COLOUR = colour_set["water"]
    CHILD_COLOUR = colour_set["child"]
    SENESCENT_COLOUR = colour_set["old"]
    CORPSE_COLOUR = colour_set["corpse"]
    MEAT_EATER_COLOUR = colour_set["meateater"]
    ADULT_COLOUR = colour_set["adult"]

    FOUND_FOOD_COLOUR = colour_set["found_food"]
    FOUND_WATER_COLOUR = colour_set["found_water"]
    ROAM_COLOUR = colour_set["roam"]

    VISION_COLOUR = colour_set["vision"]
    HIGHLIGHT_COLOUR = colour_set["highlight"]

    FRUIT_COLOUR_SET = colour_set["fruit"]

```

```

set_sim_colours(colour_set)

#if (a;b) and (c;d):
#    1/(ab) >= 1/(cd)
#where x;y is such that x is the time required to check condition and y is the pass rate
of condition. Equilibrium where ab = cd; there order has no effect on efficiency

def draw_world_blit(realm):

    #only use these variables here
    width = realm.width
    height = realm.height

    blit = pygame.Surface((width, height))

    blit.fill(GRASS_COLOUR)

    for row in range(height):
        for col in range(width):
            if realm.environment[row][col] == "water":
                pygame.draw.rect(blit, WATER_COLOUR, (col, row, 1, 1))
    for row in range(height):
        for col in range(width):
            if realm.environment[row][col] == "ice":
                pygame.draw.rect(blit, "white", (col, row, 1, 1))
    for flora in realm.elements[0]:
        if not flora.fruit.name in ("lavendar", "lily", "wildgrass"):
            pygame.draw.rect(blit, FLORA_COLOUR, (round(flora.x-1), round(flora.y-1), 3,
3))
            if flora.fruit.name == "bean":
                pygame.draw.rect(blit, FRUIT_COLOUR_SET["bean"], (round(flora.x-1),
round(flora.y-1), 3, 3))
    return blit

def draw_anima(anima, screen, cx, cy):
    bodycolour = "purple" if anima.is_poisoned() else "orangered" if anima.is_injured()
else "orange" if anima.status[3] >= 25 else SENESCENT_COLOUR if anima.is_senescent() else
CHILD_COLOUR if not anima.is_mature() else MEAT_EATER_COLOUR if "meat" in anima.diet else
ADULT_COLOUR
    pygame.draw.circle(screen, bodycolour, (cx, cy), int(round(min(max(sqrt(anima.size()),
1), 7))))

    #spaghetti code :
    eyecolour = "gold" if anima.asleep else "black" if anima.hunting else
FOUND_WATER_COLOUR if anima.mode == 2 and isinstance(anima.target, tuple) else
FOUND_FOOD_COLOUR if anima.mode == 2 else "white" if anima.mode == 3 else "violet" if 4 in
anima.goals else ROAM_COLOUR
    pygame.draw.circle(screen, eyecolour, (cx, cy),
int(round(min(max(sqrt(anima.vision/2), 1), min(max(sqrt(anima.size())-1, 1), 6))))

```

UI_TERMINAL.PY

```
import sys

max_width = 70

class Terminal():
    def __init__(self, capacity = 11):
        self.capacity = capacity
        self.content = []
        self.stdout = sys.__stdout__

    def write(self, text):
        self.stdout.write(text)

        for line in text.split("\n"):
            if line.strip() != "":
                self.content.append(line)
        while len(self.content) > self.capacity:
            self.content.pop(0)

    def flush(self):          #required for use of stdout
        self.stdout.flush()

def draw_terminal(screen, bottom, terminal, font):
    height = 0

    for line in terminal.content:
        nextline = line
        while nextline != None:
            if len(nextline) > max_width:
                nextline = line[:max_width]
                line = line[max_width:]
            else:
                line = None
            txt = font.render(nextline, True, "blue")
            height += txt.get_height()
            screen.blit(txt, (30,bottom-height))

            nextline = line
```

UI.PY

```
import pygame
from math import sqrt

from ui_sim_plane import *
from ui_element import *
from ui_design import *
from ui_factfile import draw_anima_record, draw_genome_record, genome_view
from todisplay import to_display
from ui_terminal import *

"""
class Visual_Realm():
    def __init__(self, window, realm):
        self.window = window
        self.realm = realm
        self.scale = 1

        diff = window[0][1] - window[0][0] - realm.width
        if diff > 0:
            self.window[0][0] += diff/2
            self.window[0][1] -= diff/2
        elif diff < 0:
            self.window[0][0] -= diff/2
            self.window[0][1] += diff/2

    def tick(self):
        """

def tick_visual_realm(realm, screen, window, blit, shell):
    left = window[0][0]
    right = window[0][1]
    top = window[1][0]
    bottom = window[1][1]

    screen.blit(blit, (left,top))

    if shell.anima_to_view:
        anima = shell.anima_to_view
        if not anima.asleep and not anima.is_dead():
            pygame.draw.circle(screen, VISION_COLOUR, (round(left + anima.x), round(top +
            anima.y)), round(anima.vision+0.5))
            pygame.draw.circle(screen, HIGHLIGHT_COLOUR, (round(left + anima.x), round(top +
            anima.y)), int(round(min(max(sqrt(anima.size()), 1), 7)) + 1)) # was sqrt size

        genome_view.draw(screen, font)
        if genome_view.status == False:
            draw_anima_record(anima, realm, screen)
```

```

        elif genome_view.status == True:
            draw_genome_record(anima, realm, screen)

    if FRUIT_COLOUR_SET:
        for flora in realm.elements[0]:
            if flora.fruitNo != 0:
                pygame.draw.rect(screen, FRUIT_COLOUR_SET[flora.fruit.name], (round(left +
flora.x), round(top + flora.y), 1, 1))

        for corpse in realm.graveyard[1]:
            pygame.draw.circle(screen, CORPSE_COLOUR, (round(left + corpse.x), round(top +
corpse.y)), min(max(corpse.food_chunks/2, 1), 5))

        for anima in realm.elements[1]:
            draw_anima(anima, screen, round(left + anima.x), round(top + anima.y))

def tick_extra_ui(screen, window, shell):
    hide_ui.draw(screen, font)

    if not shell.ui_visible:
        return

    for button in Button.get_button_list():
        if button.visible:
            button.draw(screen, font)

    for slider in Slider.get_slider_list():
        if slider.visible:
            slider.draw(screen)

    screen.blit(font.render(f"Track {to_display(shell.track_index+1,2)}", True,
TRACK_TEXT_COLOUR), (650,75))

    txt = tutorial_texts[shell.tutorial_index]
    width, height = font.size(txt)
    pygame.draw.rect(screen, "white", (17, 357, width+6, height+6)) #grey93
    screen.blit(font.render(txt, True, GENERAL_TEXT_COLOUR), (20,360))

    txt = colour_tutorial_texts[shell.ctutorial_index]
    width, height = font.size(txt)
    pygame.draw.rect(screen, "white", (17, 487, width+6, height+6))
    screen.blit(font.render(txt, True, GENERAL_TEXT_COLOUR), (20,490))

    draw_terminal(screen, 600, terminal, terminal_font) #600 -> window[1][1]

def tick_home_ui(screen, shell):
    for button in Button.get_home_button_list():
        button.draw(screen, home_font)

    for slider in Slider.get_home_slider_list():

```



```

        slider.draw(screen)

    the_text = big_font.render("Twinkledelux's EvoSim!", False, "yellow")
    screen.blit(the_text, the_text.get_rect(center = (screen.get_width()/2, 100)))

    txt = home_font.render("This realm is called... " + shell.realm_settings[0], False,
"green")
    screen.blit(txt, txt.get_rect(center = (screen.get_width()/2 - 5, 150)))

    txt = home_font.render("Size : " + to_display(shell.realm_settings[1]) + "x" +
to_display(shell.realm_settings[2]), False, "aquamarine")
    screen.blit(txt, txt.get_rect(center = (screen.get_width()/2 - 5, 240)))

    txt = home_font.render("Temperature : " + to_display(shell.realm_settings[3]), False,
"aquamarine")
    screen.blit(txt, txt.get_rect(center = (screen.get_width()/2 - 5, 340)))

    txt = home_font.render("Water coverage : " + to_display(shell.realm_settings[4]),
False, "aquamarine")
    screen.blit(txt, txt.get_rect(center = (screen.get_width()/2 - 5, 440)))

    txt = home_font.render("Vegetation density : " + to_display(shell.realm_flora_factor),
False, "aquamarine")
    screen.blit(txt, txt.get_rect(center = (screen.get_width()/2 - 5, 540)))

#grass is green, flora is purple : legacy
#aquamarine

```

WORLD_FACTS.PY

```
terrain_index = {  
    0 : "grass",  
    1 : "water",  
    2 : "ice"  
}  
  
#default given a 100x100 unit (10000 cells) : [wetness, iciness, flora list]  
#make more things into realm attributes, eg senescence age and pool max/min, ice y range
```

APPENDIX - NOTES

The following files have been excluded due to having no content expressed in the main program (though 'biome.py' is currently also guilty of this). This can be due to either having no content at all, being used for test purposes only, or simply having content within the unused be directly written into other files instead of importing it. These files are:

'creature.py', 'element.py', 'helper.py', 'radius.py', 'testgauss_testonly.py', 'wip.py'

Thank you for reading this documentation!

This was a project by twinkledelux (Ethan Cheng 5045)